

PDAs Accept Context-Free Languages

Theorem:

$$\left\{ \begin{array}{l} \text{Context-Free} \\ \text{Languages} \\ \text{(Grammars)} \end{array} \right\} = \left\{ \begin{array}{l} \text{Languages} \\ \text{Accepted by} \\ \text{PDAs} \end{array} \right\}$$

Proof - Step 1:

$$\left\{ \begin{array}{c} \text{Context-Free} \\ \text{Languages} \\ \text{(Grammars)} \end{array} \right\} \subseteq \left\{ \begin{array}{c} \text{Languages} \\ \text{Accepted by} \\ \text{PDAs} \end{array} \right\}$$

Convert any context-free grammar G
to a PDA M with: $L(G) = L(M)$

Proof - Step 2:

$$\left\{ \begin{array}{c} \text{Context-Free} \\ \text{Languages} \\ \text{(Grammars)} \end{array} \right\} \supseteq \left\{ \begin{array}{c} \text{Languages} \\ \text{Accepted by} \\ \text{PDAs} \end{array} \right\}$$

Convert any PDA M to a context-free grammar G with: $L(G) = L(M)$

Proof - step 1

Convert

Context-Free Grammars
to
PDAs

Take an arbitrary context-free grammar G

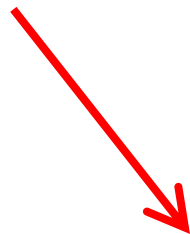
We will convert G to a PDA M such that:

$$L(G) = L(M)$$

Conversion Procedure:

For each
production in G

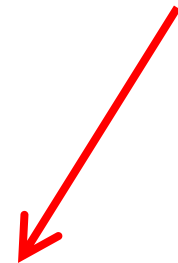
$$A \rightarrow w$$



$$\varepsilon, A \rightarrow w$$

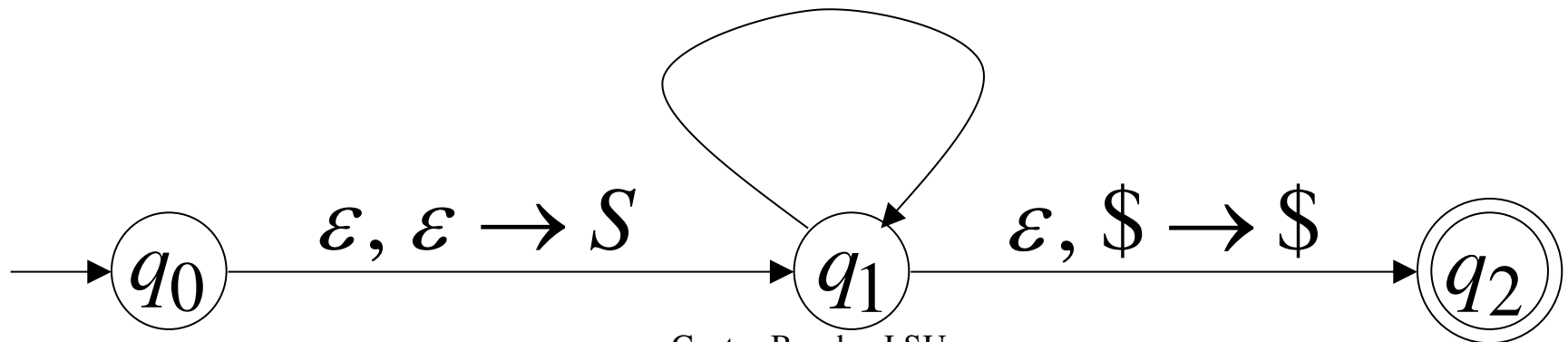
For each
terminal in G

a



$$a, a \rightarrow \varepsilon$$

Add transitions



Example

Grammar

$$S \rightarrow aSTb$$

$$S \rightarrow b$$

$$T \rightarrow Ta$$

$$T \rightarrow \varepsilon$$

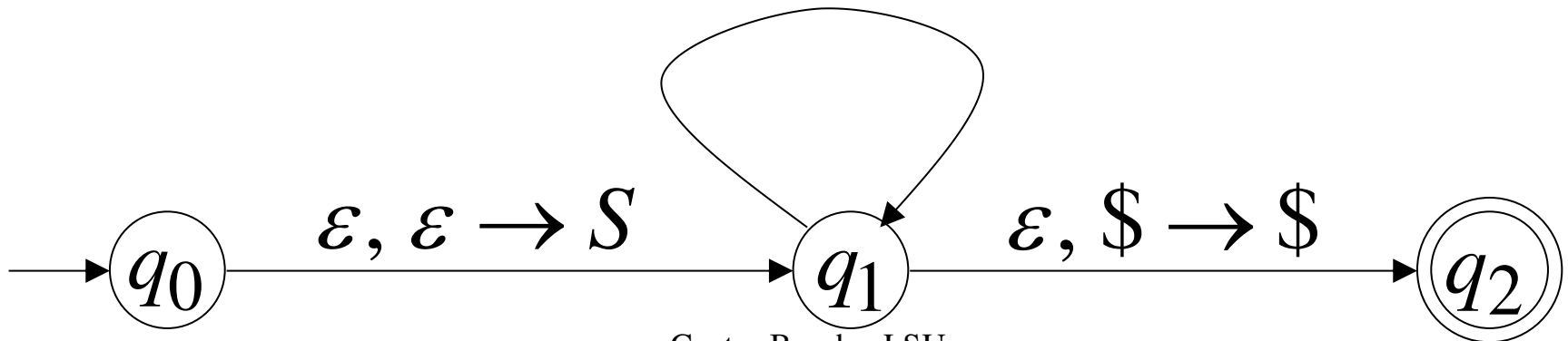
PDA

$$\varepsilon, S \rightarrow aSTb$$

$$\varepsilon, S \rightarrow b$$

$$\varepsilon, T \rightarrow Ta \quad a, a \rightarrow \varepsilon$$

$$\varepsilon, T \rightarrow \varepsilon \quad b, b \rightarrow \varepsilon$$



PDA simulates leftmost derivations

Grammar

Leftmost Derivation

S
 $\Rightarrow \dots$
 $\Rightarrow \sigma_1 \cdots \sigma_k X_1 \cdots X_m$
 $\Rightarrow \dots$
 $\Rightarrow \sigma_1 \cdots \sigma_k \sigma_{k+1} \cdots \sigma_n$

PDA Computation

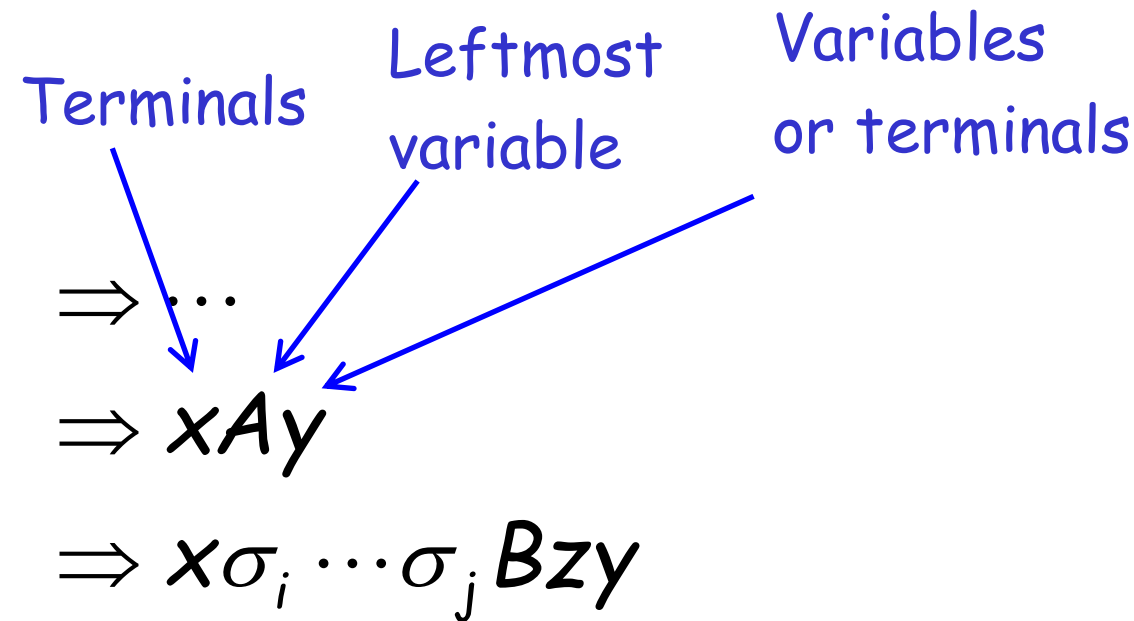
$(q_0, \sigma_1 \cdots \sigma_k \sigma_{k+1} \cdots \sigma_n, \$)$
 $\succ (q_1, \sigma_1 \cdots \sigma_k \sigma_{k+1} \cdots \sigma_n, S\$)$
 $\succ \dots$
 $\succ (q_1, \sigma_{k+1} \cdots \sigma_n, X_1 \cdots X_m \$)$
 $\succ \dots$
 $\succ (q_2, \varepsilon, \$)$

Scanned
symbols

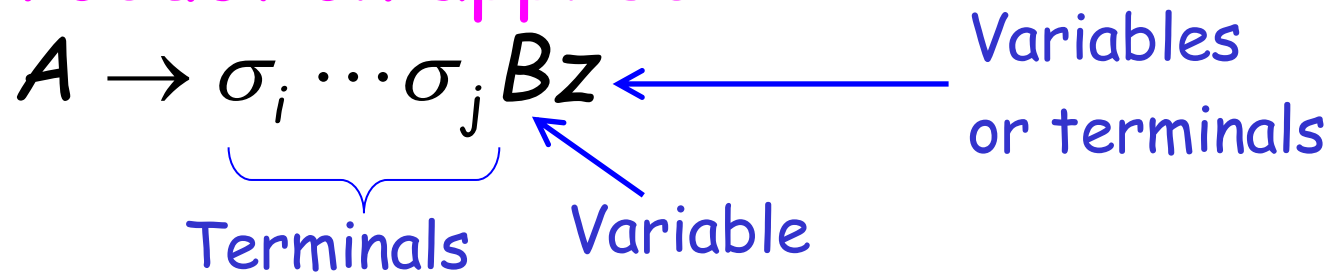
Stack
contents

Grammar

Leftmost Derivation



Production applied



Grammar

Leftmost Derivation

$\Rightarrow \dots$

$\Rightarrow xAy$

$\Rightarrow x\sigma_i \cdots \sigma_j Bzy$

Production applied

$A \rightarrow \sigma_i \cdots \sigma_j Bz$

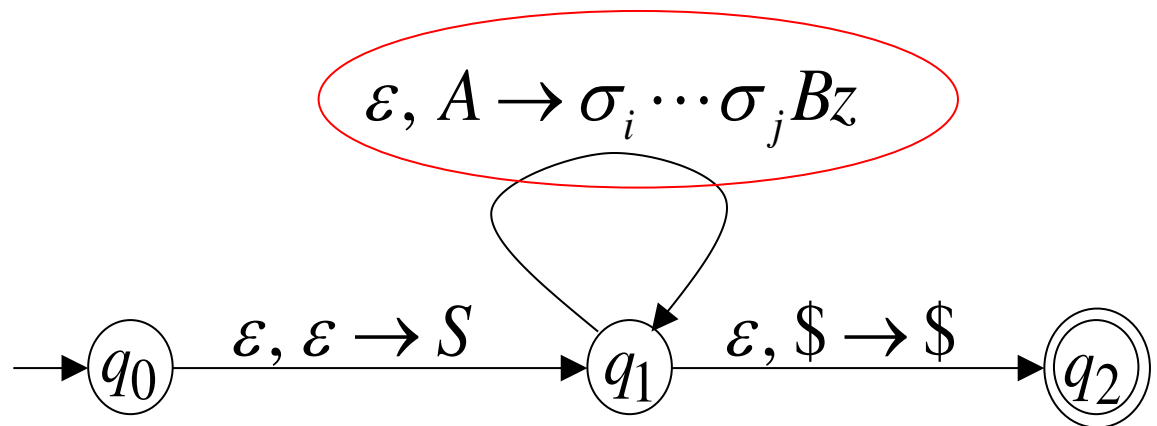
PDA Computation

$\succ \dots$

$\succ (q_1, \sigma_i \cdots \sigma_n, Ay\$)$

$\succ (q_1, \sigma_i \cdots \sigma_n, \sigma_i \cdots \sigma_j Bzy\$)$

Transition applied



Grammar

Leftmost Derivation

$\Rightarrow \dots$

$\Rightarrow xAy$

$\Rightarrow x\sigma_i \cdots \sigma_j Bzy$

PDA Computation

$\succ \dots$

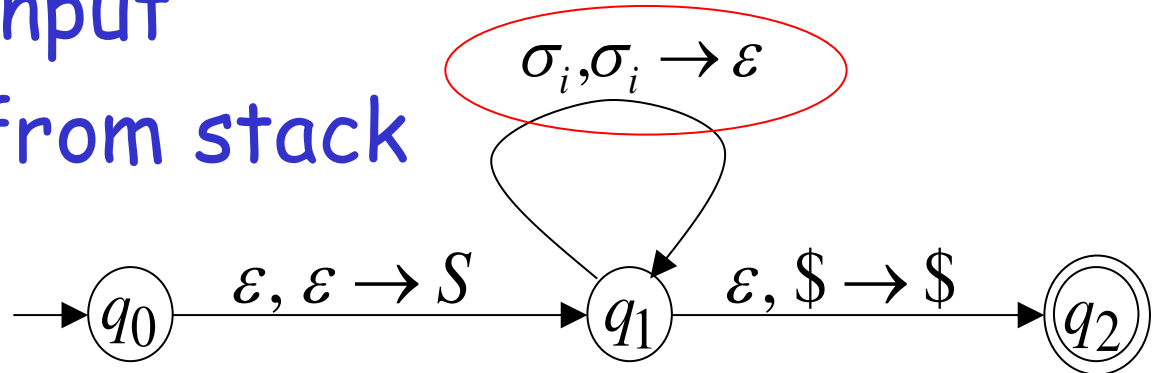
$\succ (q_1, \sigma_i \cdots \sigma_n, Ay\$)$

$\succ (q_1, \sigma_i \cdots \sigma_n, \sigma_i \cdots \sigma_j Bzy\$)$

$\succ (q_1, \sigma_{i+1} \cdots \sigma_n, \sigma_{i+1} \cdots \sigma_j Bzy\$)$

Read σ_i from input
and remove it from stack

Transition applied



Grammar

Leftmost Derivation

$\Rightarrow \dots$

$\Rightarrow xAy$

$\Rightarrow x\sigma_i \cdots \sigma_j Bzy$

All symbols $\sigma_i \cdots \sigma_j$
have been removed
from top of stack

PDA Computation

$\succ \dots$

$\succ (q_1, \sigma_i \cdots \sigma_n, Ay\$)$

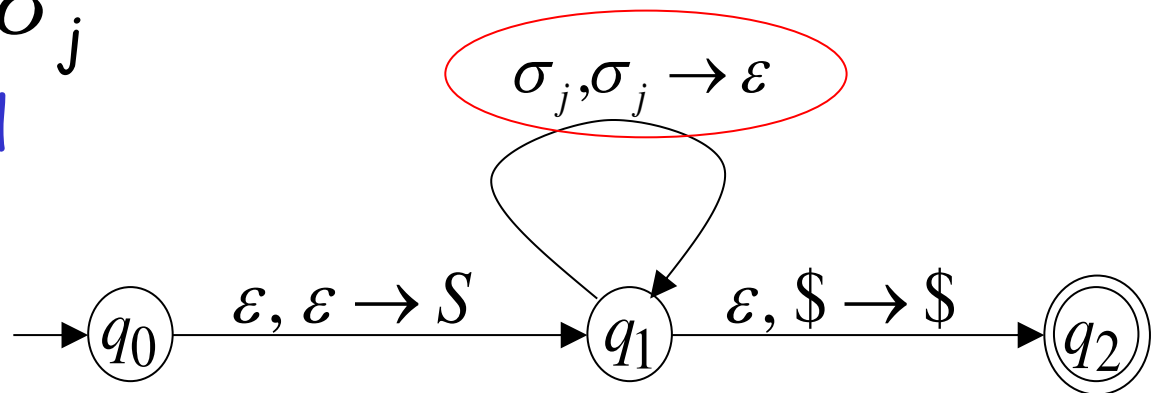
$\succ (q_1, \sigma_i \cdots \sigma_n, \sigma_i \cdots \sigma_j Bzy\$)$

$\succ (q_1, \sigma_{i+1} \cdots \sigma_n, \sigma_{i+1} \cdots \sigma_j Bzy\$)$

$\succ \dots$

$\succ (q_1, \sigma_{j+1} \cdots \sigma_n, Bzy\$)$

Last Transition applied



The process repeats with the next
leftmost variable

$\Rightarrow \dots$

$\Rightarrow xAy$

$\Rightarrow x\sigma_i \cdots \sigma_j Bzy$

$\Rightarrow x\sigma_i \cdots \sigma_j \sigma_{j+1} \cdots \sigma_k Cpzy$

$\succ \dots$

$\succ (q_1, \sigma_{j+1} \cdots \sigma_n, Bzy\$)$

$\succ (q_1, \sigma_{j+1} \cdots \sigma_n, \sigma_{j+1} \cdots \sigma_k Cpzy\$)$

$\succ \dots$

$\succ (q_1, \sigma_{k+1} \cdots \sigma_n, Cpzy\$)$

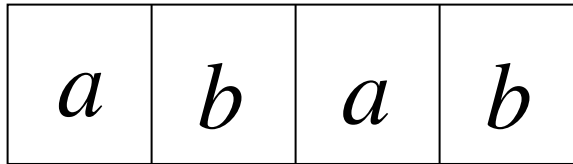
Production applied

$B \rightarrow \sigma_{j+1} \cdots \sigma_k Cp$

And so on.....

Example:

Input



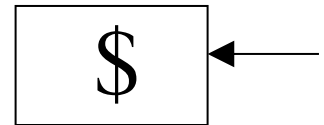
Time 0

$$\varepsilon, S \rightarrow aSTb$$

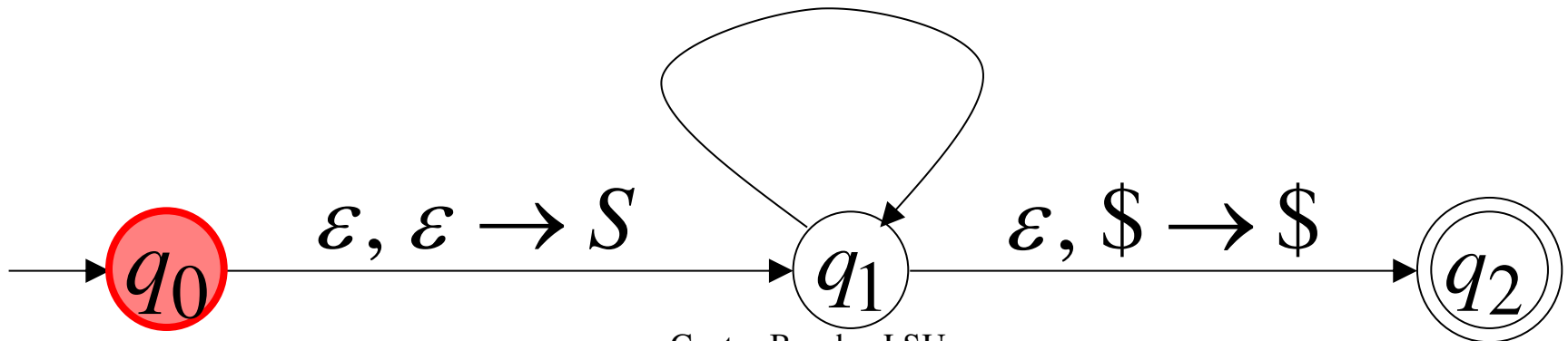
$$\varepsilon, S \rightarrow b$$

$$\varepsilon, T \rightarrow Ta \quad a, a \rightarrow \varepsilon$$

$$\varepsilon, T \rightarrow \varepsilon \quad b, b \rightarrow \varepsilon$$

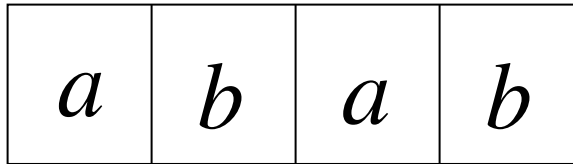


Stack



Derivation: S

Input



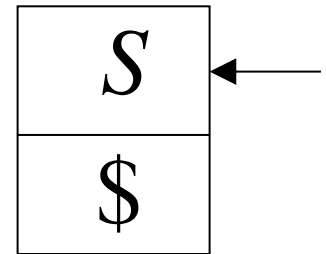
Time 1

$$\varepsilon, S \rightarrow aSTb$$

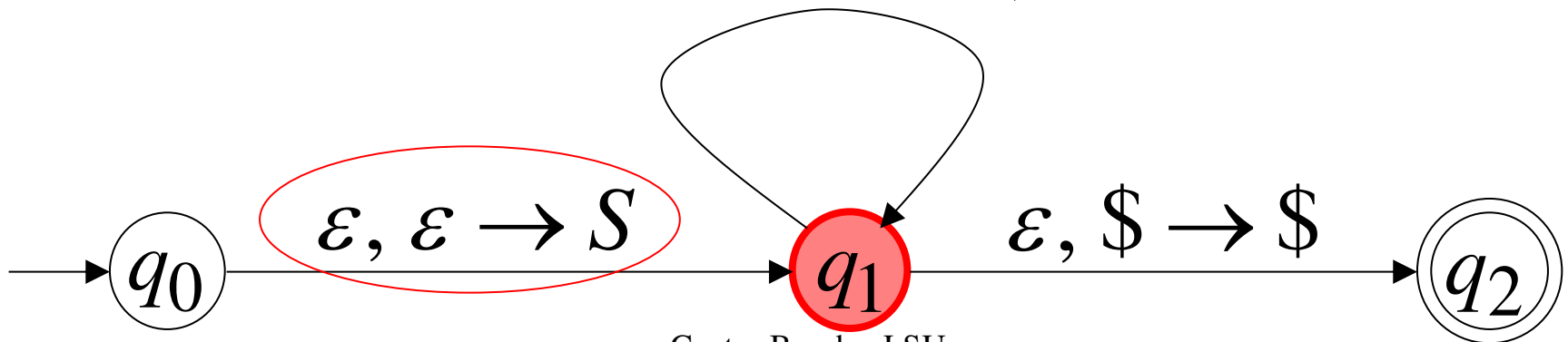
$$\varepsilon, S \rightarrow b$$

$$\varepsilon, T \rightarrow Ta \quad a, a \rightarrow \varepsilon$$

$$\varepsilon, T \rightarrow \varepsilon \quad b, b \rightarrow \varepsilon$$

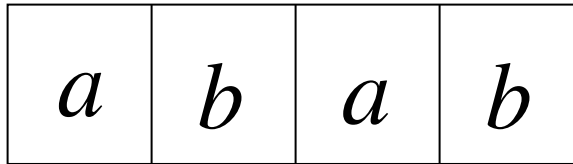


Stack



Derivation: $S \Rightarrow aSTb$

Input



$a b a b$

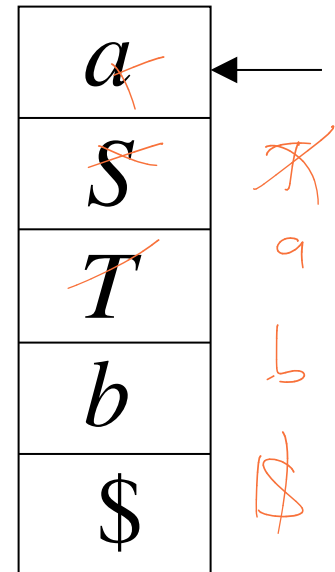
Time 2

$\varepsilon, S \rightarrow aSTb$

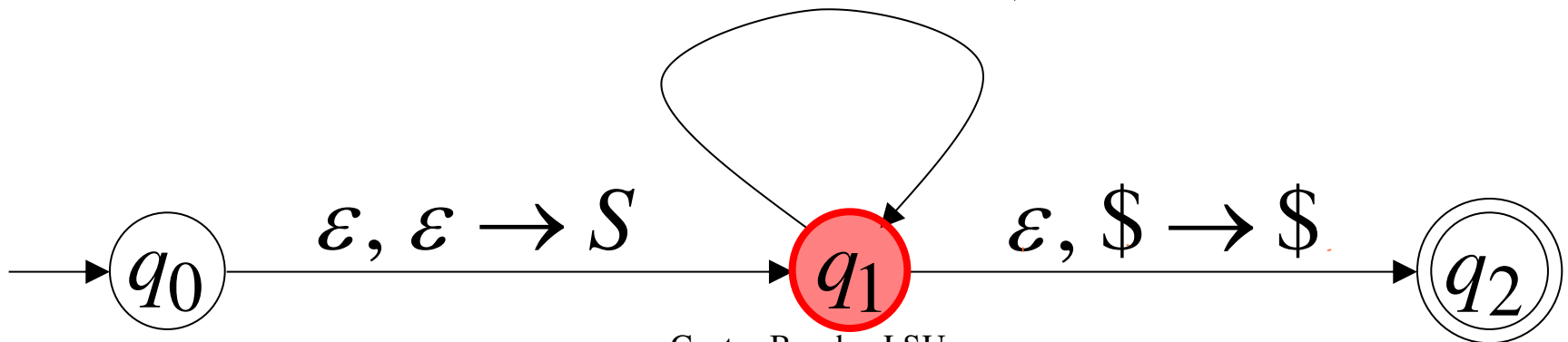
$\varepsilon, S \rightarrow b$

$\varepsilon, T \rightarrow Ta \quad a, a \rightarrow \varepsilon$

$\varepsilon, T \rightarrow \varepsilon \quad b, b \rightarrow \varepsilon$

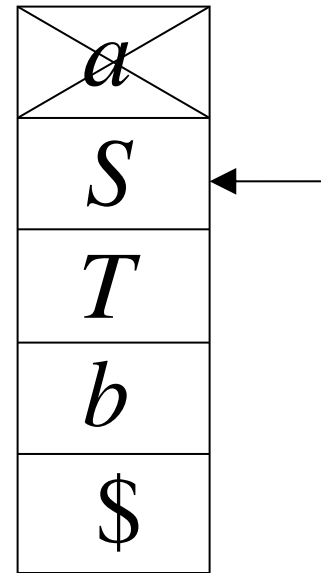
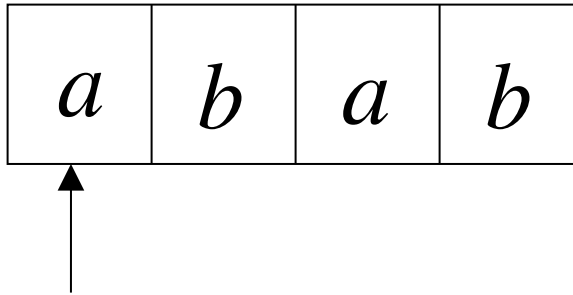


Stack



Derivation: $S \Rightarrow aSTb$

Input



Time 3

$\varepsilon, S \rightarrow aSTb$

$\varepsilon, S \rightarrow b$

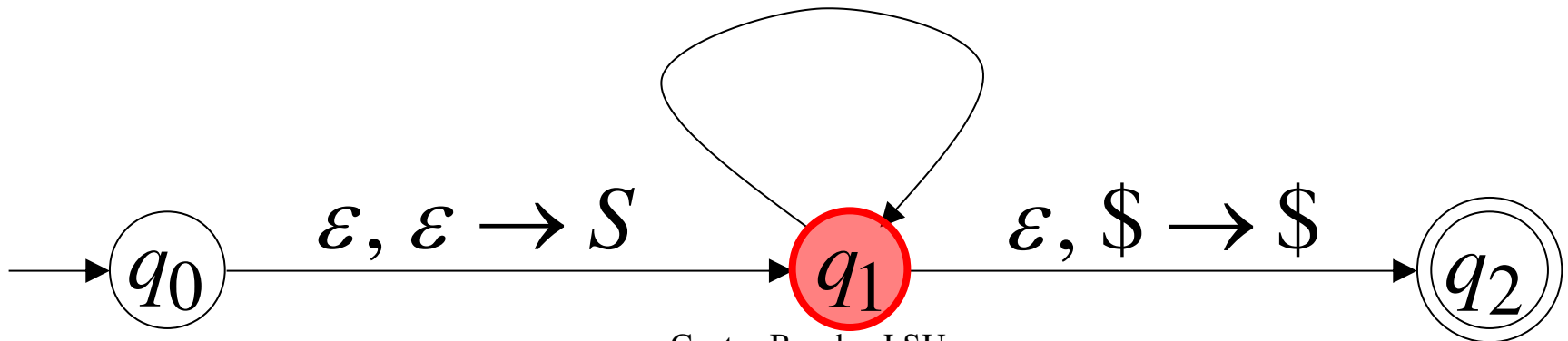
$\varepsilon, T \rightarrow Ta$

$a, a \rightarrow \varepsilon$

$\varepsilon, T \rightarrow \varepsilon$

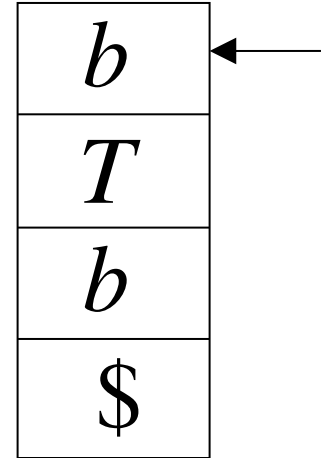
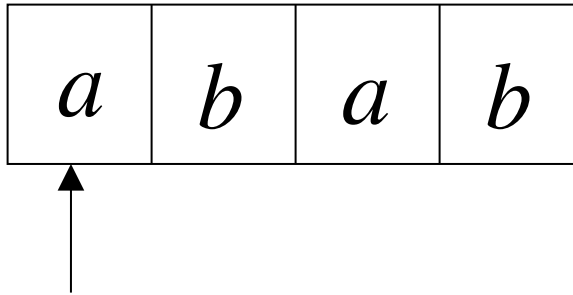
$b, b \rightarrow \varepsilon$

Stack



Derivation: $S \Rightarrow aSTb \Rightarrow abTb$

Input



Time 4

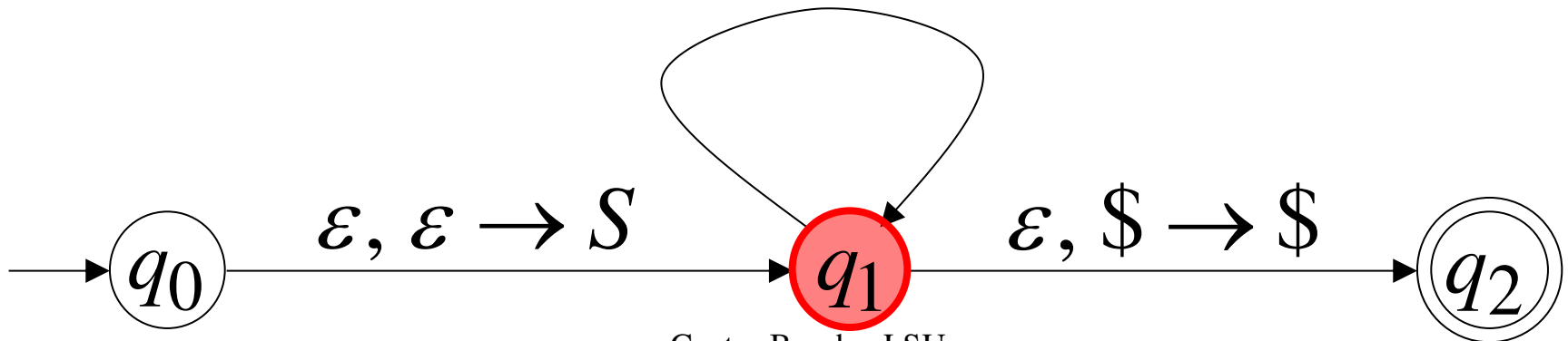
$\varepsilon, S \rightarrow aSTb$

$\varepsilon, S \rightarrow b$

$\varepsilon, T \rightarrow Ta \quad a, a \rightarrow \varepsilon$

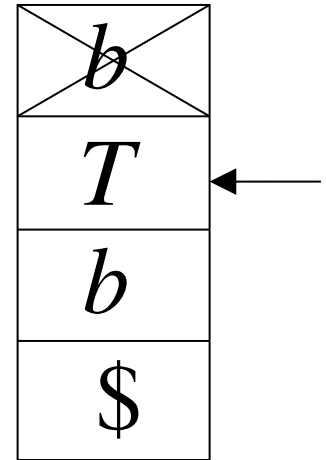
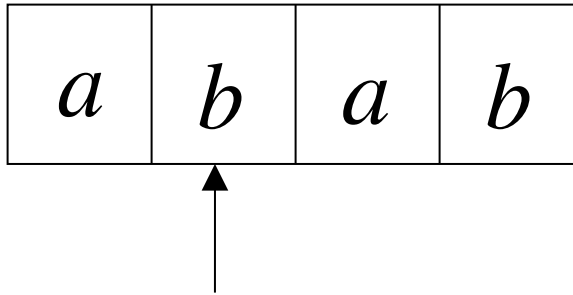
$\varepsilon, T \rightarrow \varepsilon \quad b, b \rightarrow \varepsilon$

Stack



Derivation: $S \Rightarrow aSTb \Rightarrow abTb$

Input



Time 5

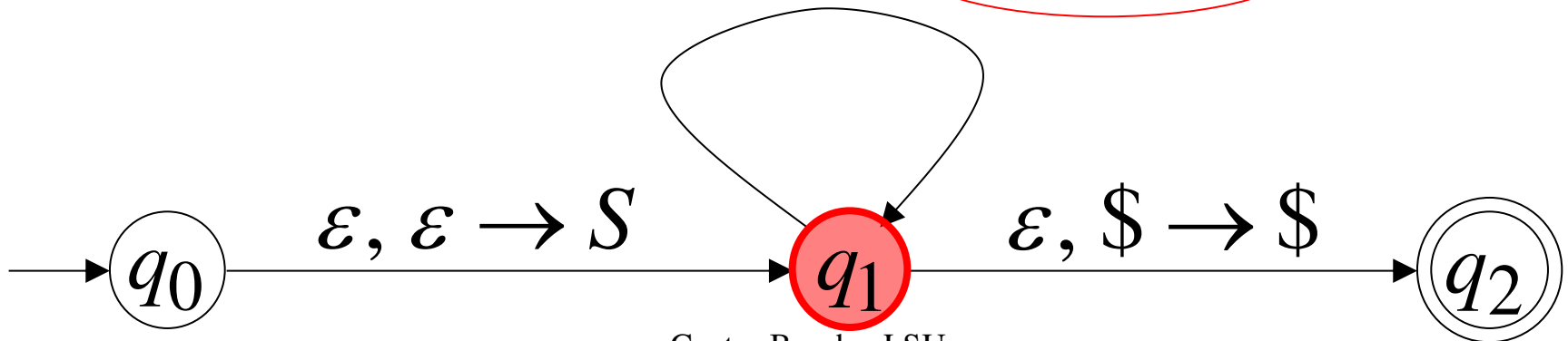
$\varepsilon, S \rightarrow aSTb$

$\varepsilon, S \rightarrow b$

$\varepsilon, T \rightarrow Ta$ $a, a \rightarrow \varepsilon$

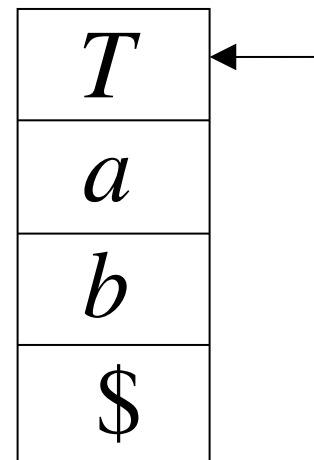
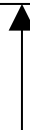
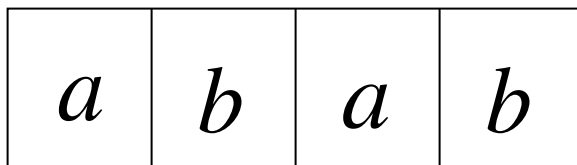
$\varepsilon, T \rightarrow \varepsilon$ $b, b \rightarrow \varepsilon$

Stack



Derivation: $S \Rightarrow aSTb \Rightarrow abTb \Rightarrow abTab$

Input



Stack

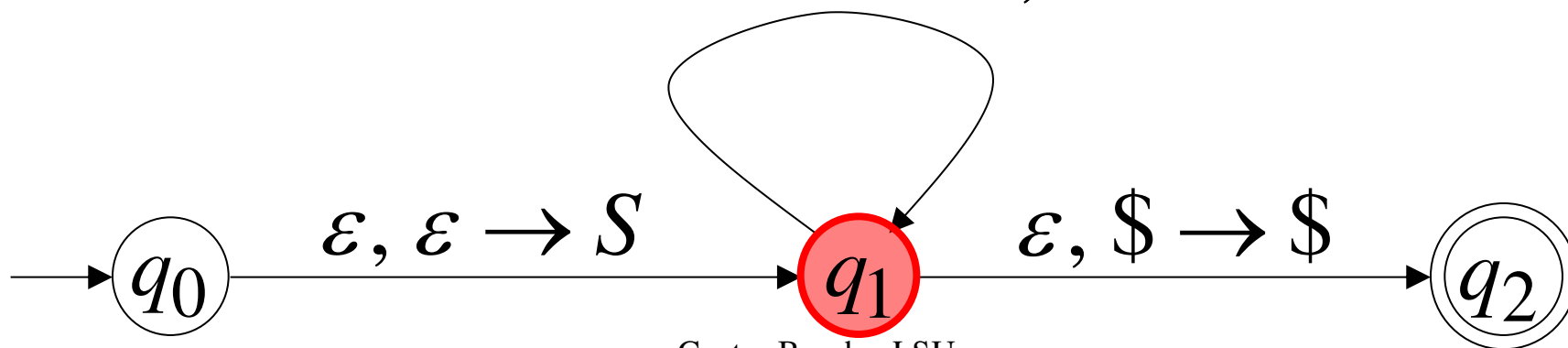
Time 6

$\varepsilon, S \rightarrow aSTb$

$\varepsilon, S \rightarrow b$

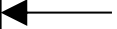
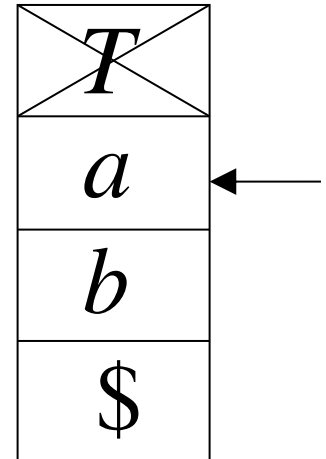
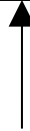
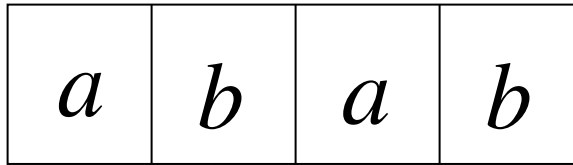
$\varepsilon, T \rightarrow Ta$ $a, a \rightarrow \varepsilon$

$\varepsilon, T \rightarrow \varepsilon$ $b, b \rightarrow \varepsilon$



Derivation: $S \Rightarrow aSTb \Rightarrow abTb \Rightarrow abTab \Rightarrow abab$

Input



Time 7

$\varepsilon, S \rightarrow aSTb$

$\varepsilon, S \rightarrow b$

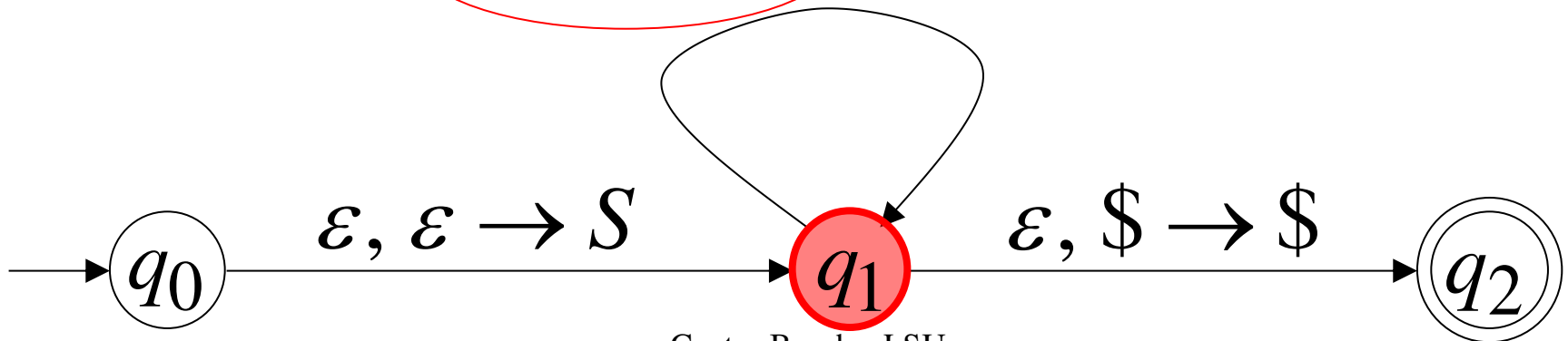
$\varepsilon, T \rightarrow Ta$

$a, a \rightarrow \varepsilon$

$\varepsilon, T \rightarrow \varepsilon$

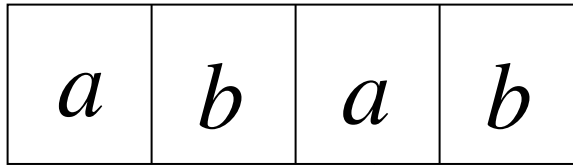
$b, b \rightarrow \varepsilon$

Stack



Derivation: $S \Rightarrow aSTb \Rightarrow abTb \Rightarrow abTab \Rightarrow abab$

Input



Time 8

$\varepsilon, S \rightarrow aSTb$

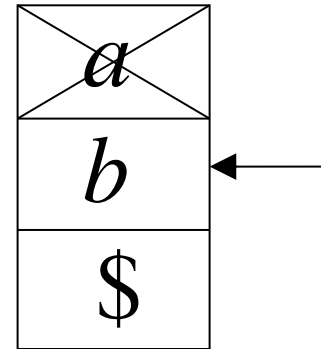
$\varepsilon, S \rightarrow b$

$\varepsilon, T \rightarrow Ta$

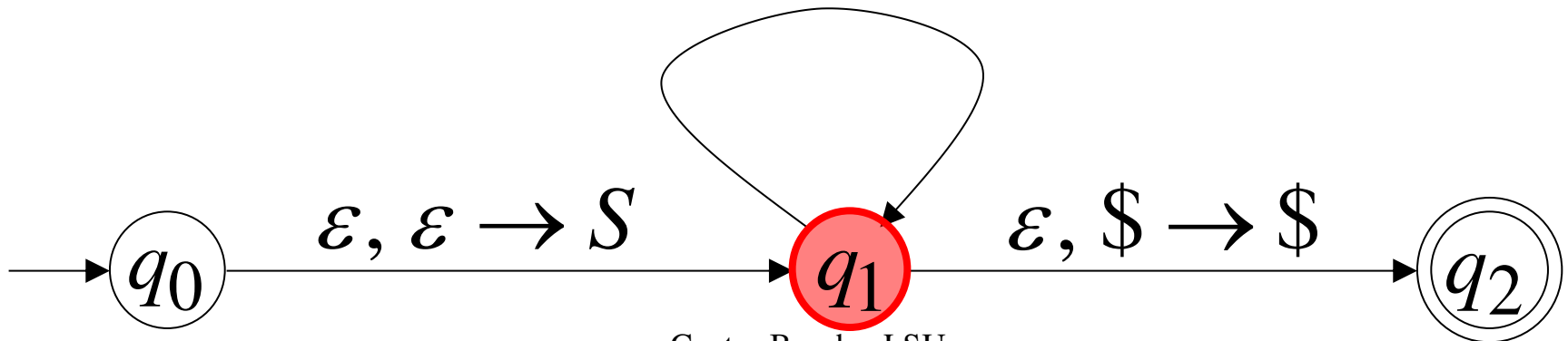
$a, a \rightarrow \varepsilon$

$\varepsilon, T \rightarrow \varepsilon$

$b, b \rightarrow \varepsilon$

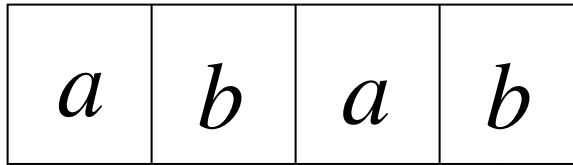


Stack



Derivation: $S \Rightarrow aSTb \Rightarrow abTb \Rightarrow abTab \Rightarrow abab$

Input



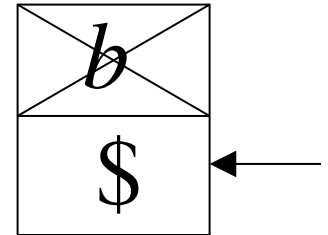
Time 9

$\varepsilon, S \rightarrow aSTb$

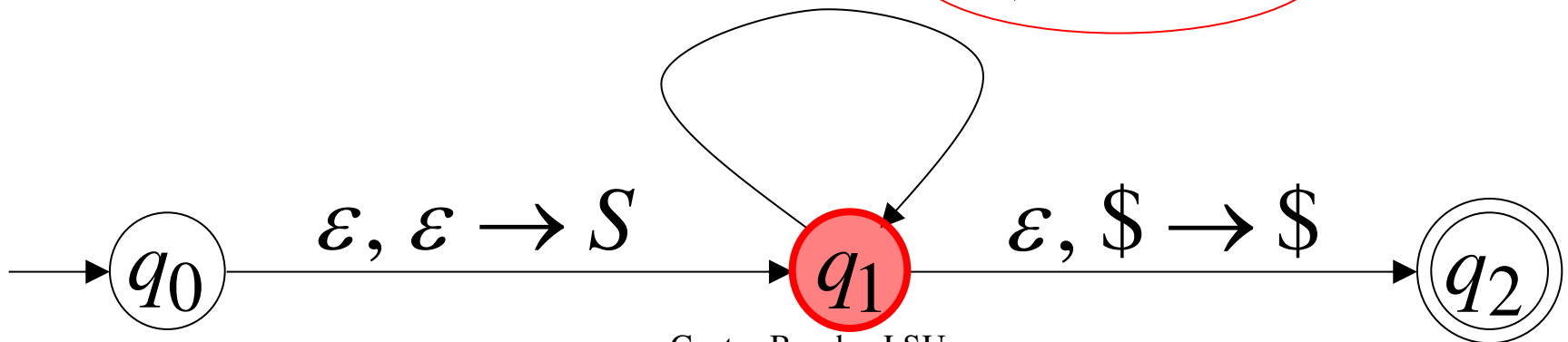
$\varepsilon, S \rightarrow b$

$\varepsilon, T \rightarrow Ta \quad a, a \rightarrow \varepsilon$

$\varepsilon, T \rightarrow \varepsilon \quad b, b \rightarrow \varepsilon$

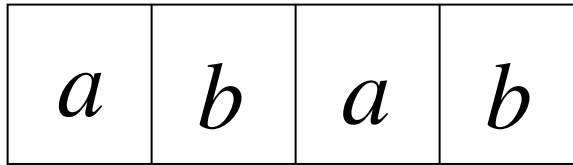


Stack



Derivation: $S \Rightarrow aSTb \Rightarrow abTb \Rightarrow abTab \Rightarrow abab$

Input



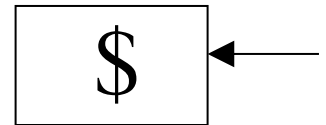
Time 10

$\varepsilon, S \rightarrow aSTb$

$\varepsilon, S \rightarrow b$

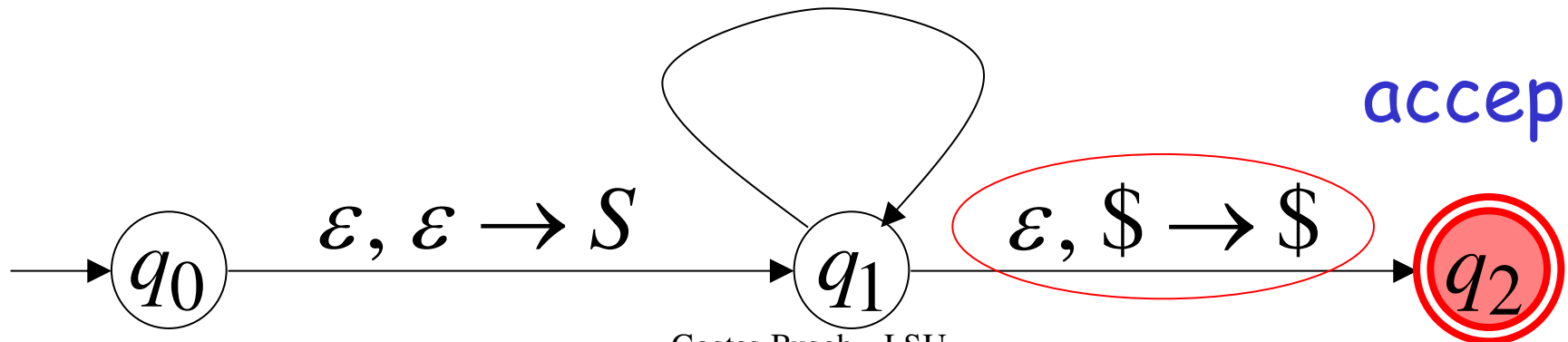
$\varepsilon, T \rightarrow Ta \quad a, a \rightarrow \varepsilon$

$\varepsilon, T \rightarrow \varepsilon \quad b, b \rightarrow \varepsilon$



Stack

accept



Grammar

Leftmost Derivation

S

$\Rightarrow aSTb$

$\Rightarrow abTb$

$\Rightarrow abTab$

$\Rightarrow abab$

PDA Computation

$(q_0, abab, \$)$

$\succ (q_1, abab, S\$)$

$\succ (q_1, bab, STb\$)$

$\succ (q_1, bab, bTb\$)$

$\succ (q_1, ab, Tb\$)$

$\succ (q_1, ab, Tab\$)$

$\succ (q_1, ab, ab\$)$

$\succ (q_1, b, b\$)$

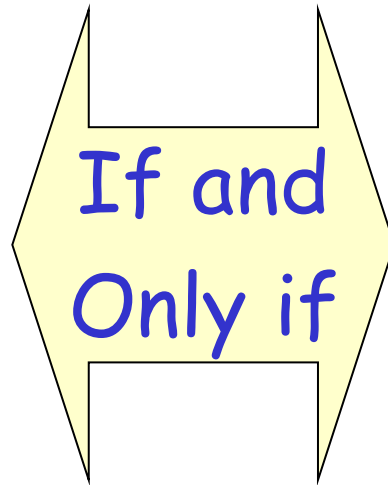
$\succ (q_1, \varepsilon, \$)$

$\succ (q_2, \varepsilon, \$)$

In general, it can be shown that:

Grammar G
generates
string w

$$S \overset{*}{\Rightarrow} w$$



PDA M
accepts w

$$(q_0, w, \$) \overset{*}{\succ} (q_2, \varepsilon, \$)$$

Therefore $L(G) = L(M)$

Proof - step 2

Convert

PDAs

to

Context-Free Grammars

Take an arbitrary PDA M

We will convert M
to a context-free grammar G such that:

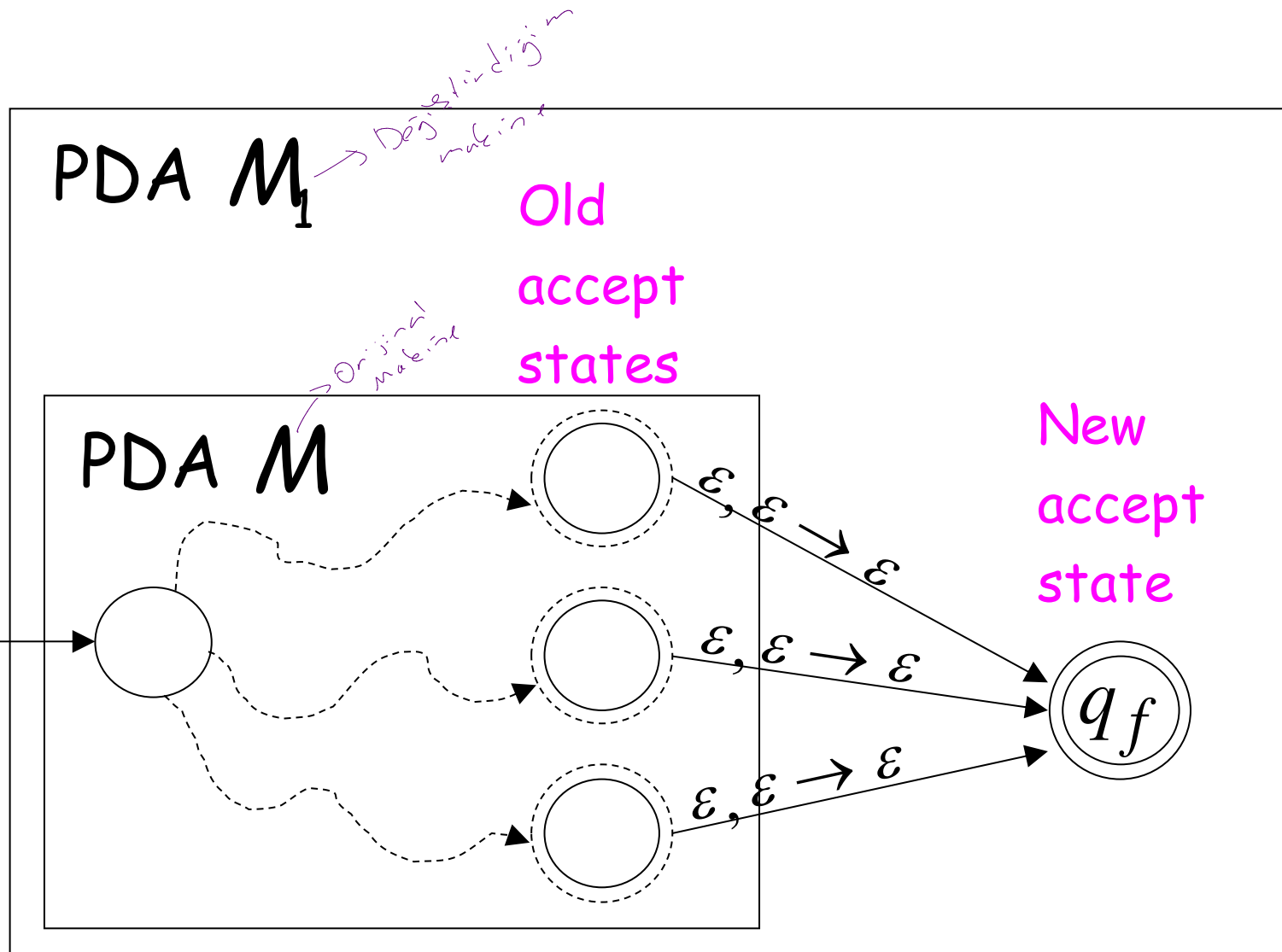
$$L(M) = L(G)$$

First modify PDA M so that:

Code
on the
left

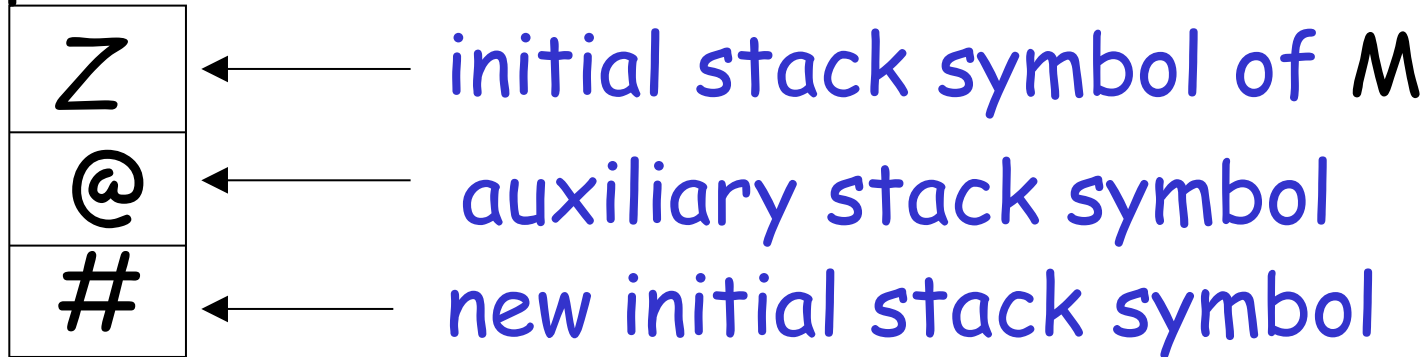
1. The PDA has a single accept state
2. Use new initial stack symbol $\#$
3. On acceptance the stack contains only stack symbol $\#$ (this symbol is not used in any transition)
4. Each transition either pushes a symbol or pops a symbol but not both together

1. The PDA has a single accept state



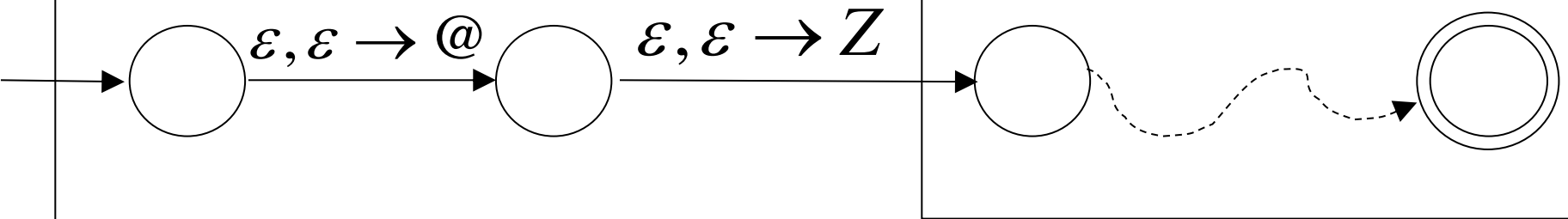
2. Use new initial stack symbol

Top of stack



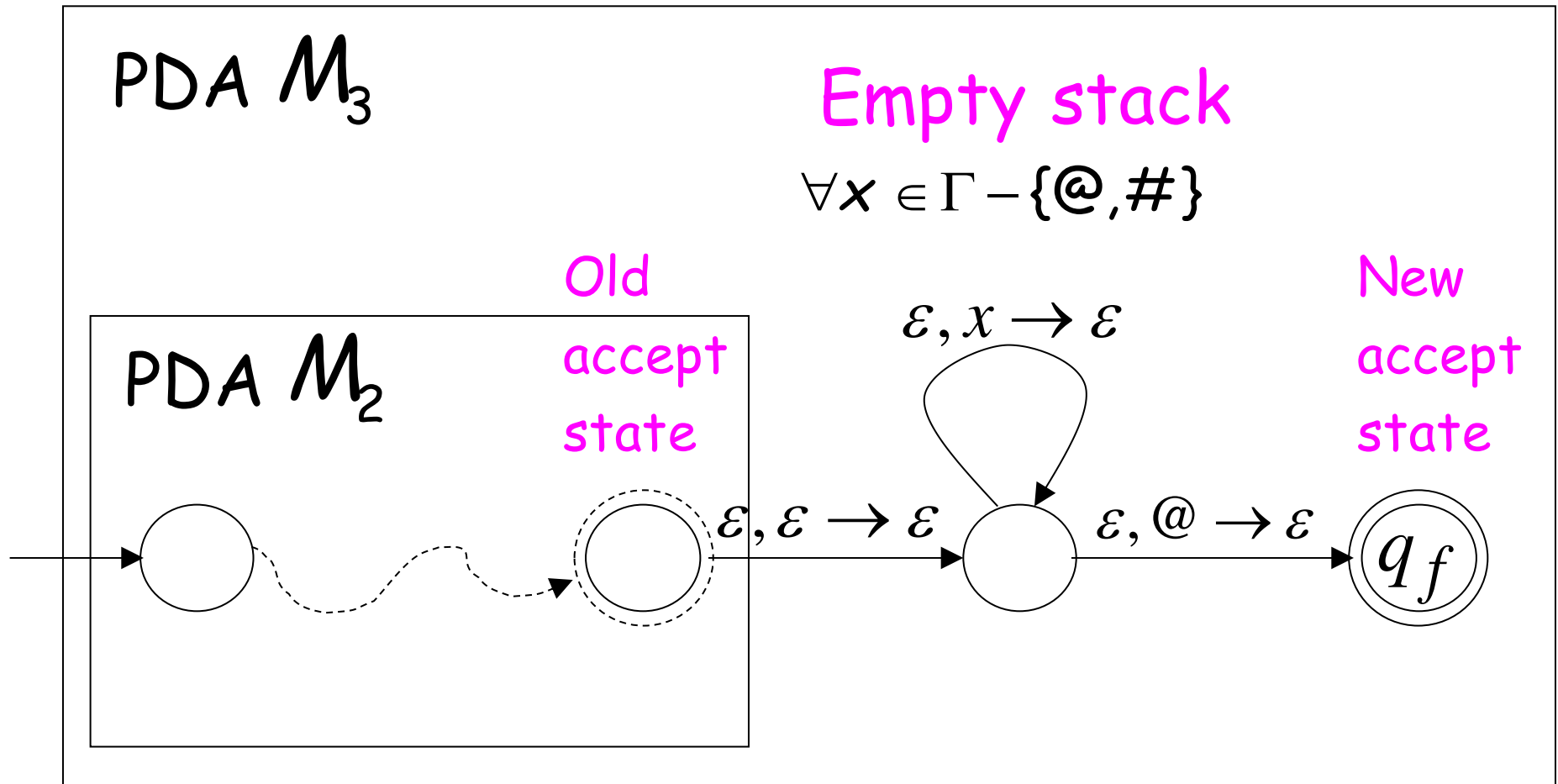
PDA M_2

PDA M_1

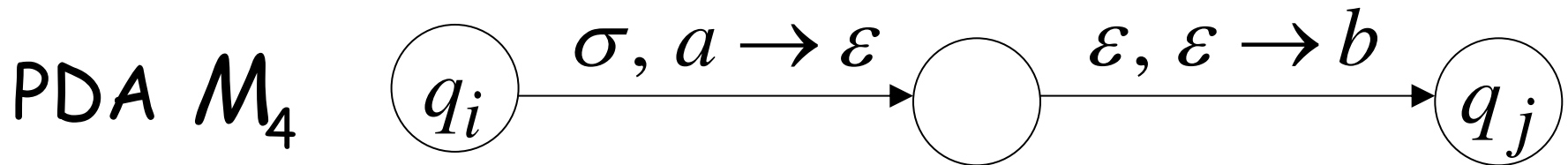
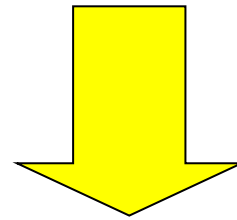
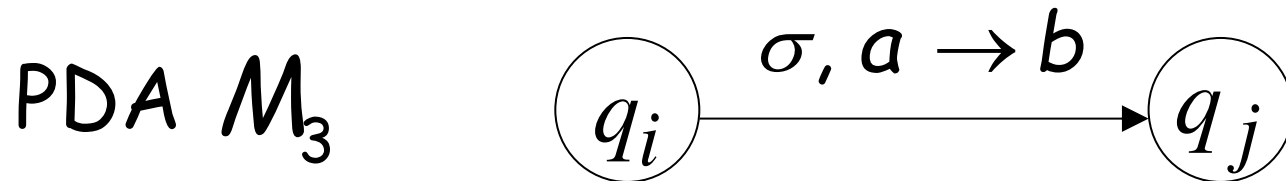


M_1 still thinks that Z is the initial stack

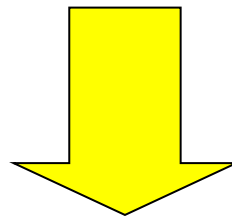
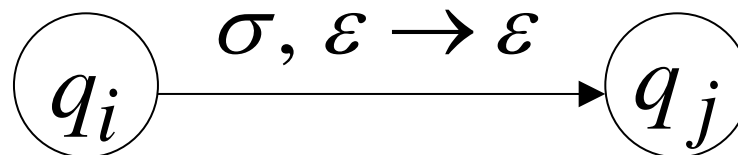
3. On acceptance the stack contains only stack symbol #
(this symbol is not used in any transition)



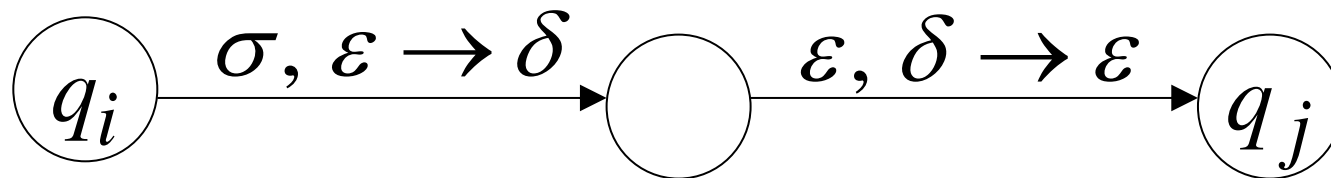
4. Each transition either pushes a symbol or pops a symbol but not both together



PDA M_3



PDA M_4

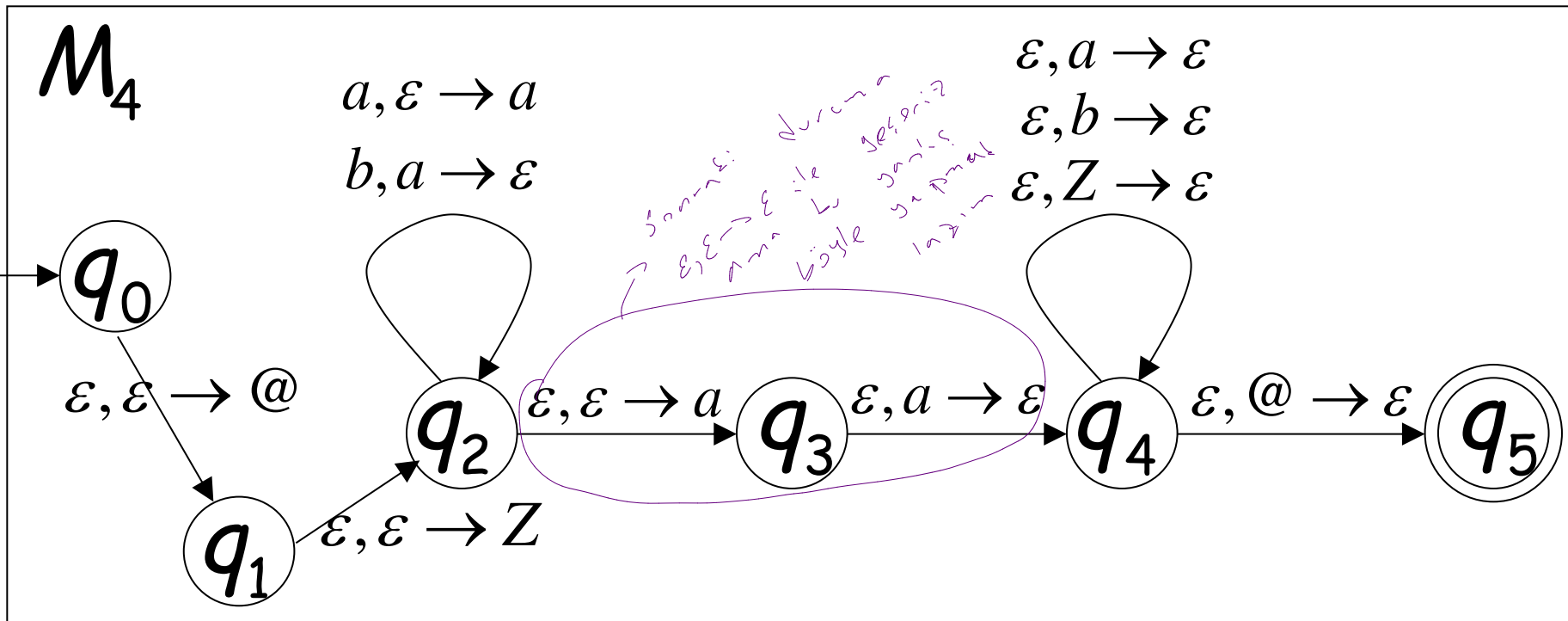
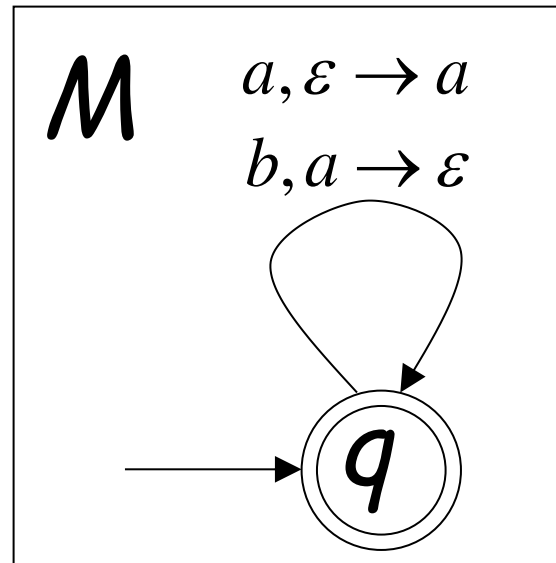


Where δ is a symbol of the stack alphabet

PDA M_4 is the final modified PDA

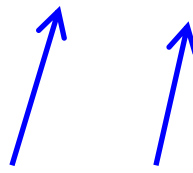
Note that the new initial stack symbol $\#$ is never used in any transition

Example:



Grammar Construction

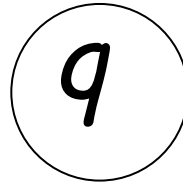
Variables: A_{q_i, q_j}



States of PDA

PDA

Kind 1: for each state



Grammar

$$A_{qq} \rightarrow \varepsilon$$

PDA

Kind 2: for every three states

p

q

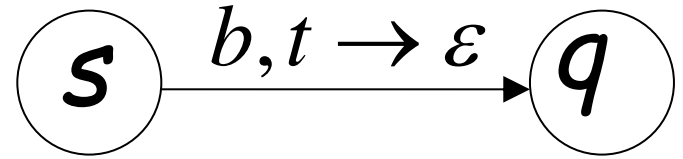
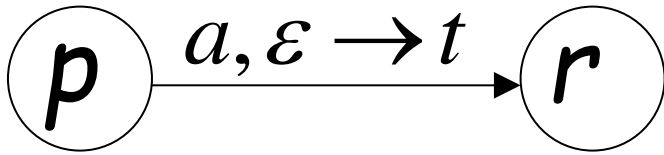
r

Grammar

$$A_{pq} \rightarrow A_{pr} A_{rq}$$

PDA

Kind 3: for every pair of such transitions

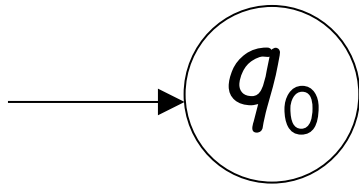


Grammar

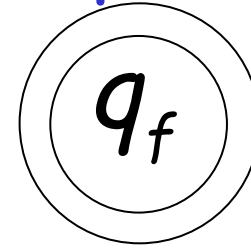
$$A_{pq} \rightarrow aA_{rs}b$$

PDA

Initial state



Accept state



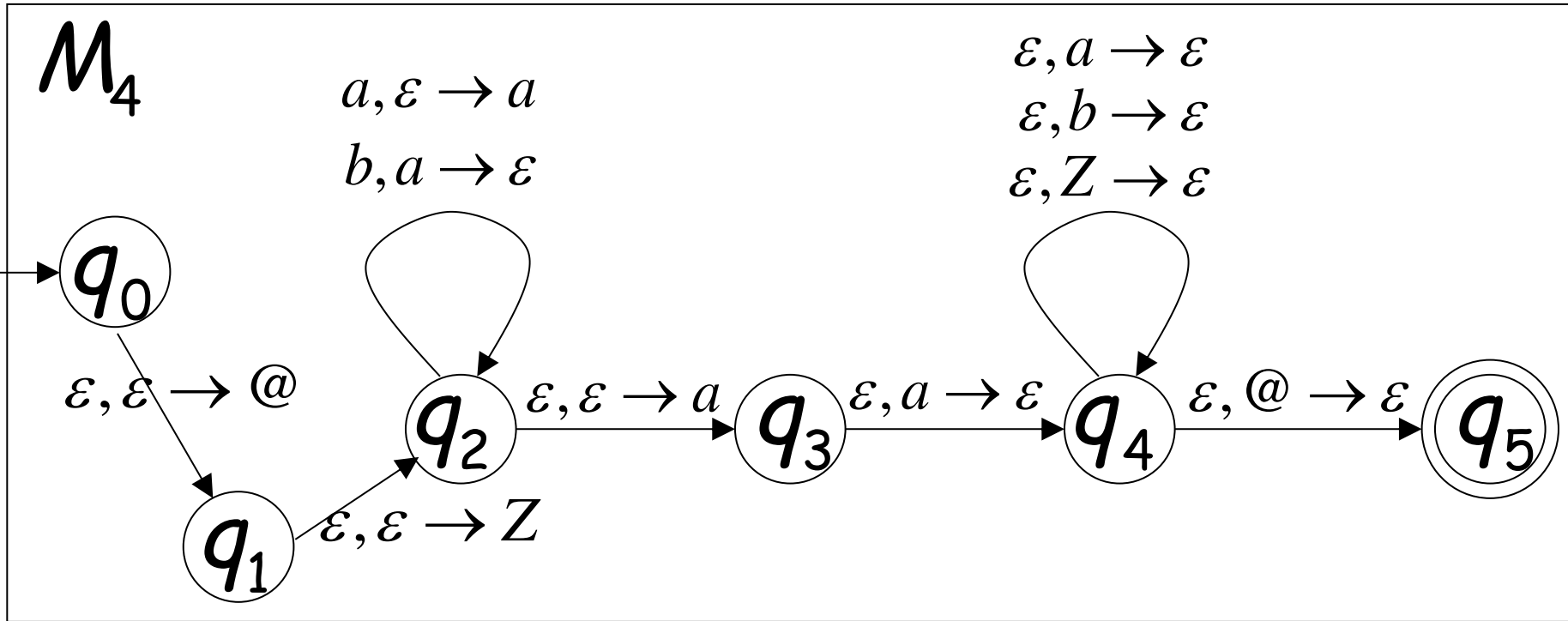
Grammar

Start variable

$A_{q_0 q_f}$

Example:

PDA



Grammar

Kind 1: from single states

$$A_{q_0q_0} \rightarrow \varepsilon$$

$$A_{q_1q_1} \rightarrow \varepsilon$$

$$A_{q_2q_2} \rightarrow \varepsilon$$

$$A_{q_3q_3} \rightarrow \varepsilon$$

$$A_{q_4q_4} \rightarrow \varepsilon$$

$$A_{q_5q_5} \rightarrow \varepsilon$$

Kind 2: from triplets of states

$$A_{q_0q_0} \rightarrow A_{q_0q_0} A_{q_0q_0} \mid A_{q_0q_1} A_{q_1q_0} \mid A_{q_0q_2} A_{q_2q_0} \mid A_{q_0q_3} A_{q_3q_0} \mid A_{q_0q_4} A_{q_4q_0} \mid A_{q_0q_5} A_{q_5q_0}$$

$$A_{q_0q_1} \rightarrow A_{q_0q_0} A_{q_0q_1} \mid A_{q_0q_1} A_{q_1q_1} \mid A_{q_0q_2} A_{q_2q_1} \mid A_{q_0q_3} A_{q_3q_1} \mid A_{q_0q_4} A_{q_4q_1} \mid A_{q_0q_5} A_{q_5q_1}$$

⋮

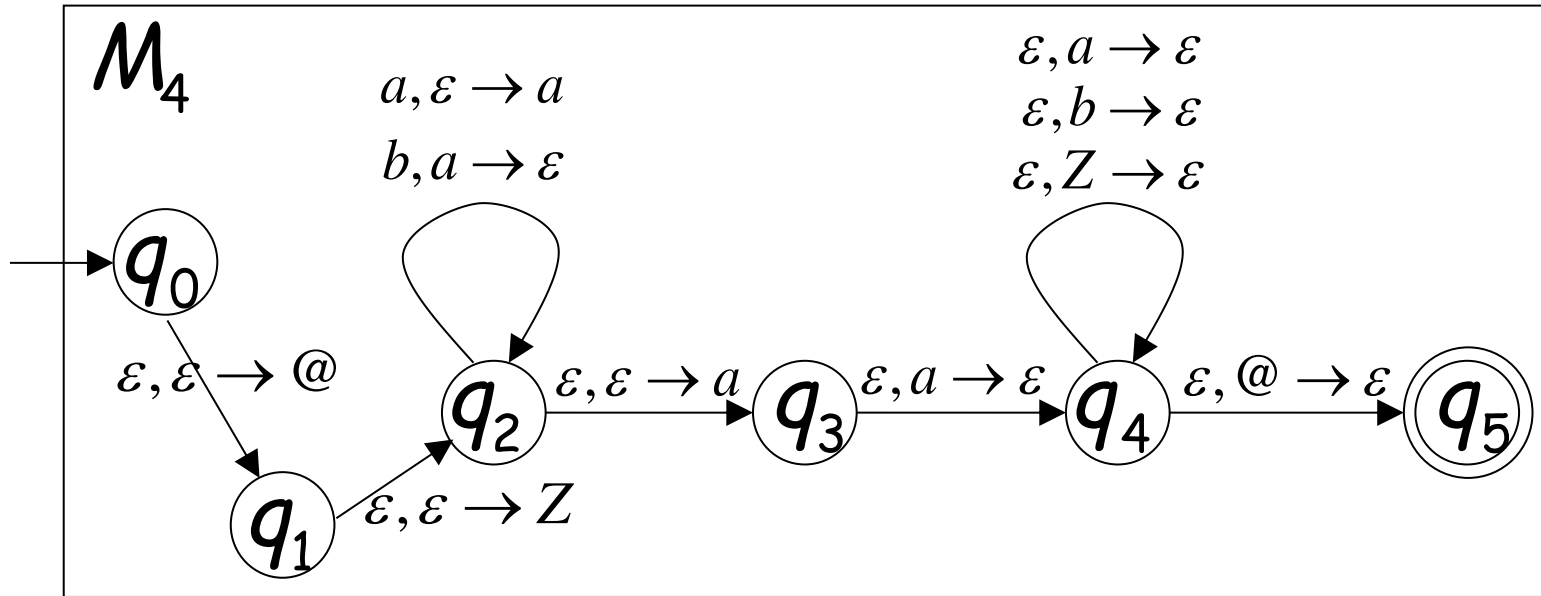
$$A_{q_0q_5} \rightarrow A_{q_0q_0} A_{q_0q_5} \mid A_{q_0q_1} A_{q_1q_5} \mid A_{q_0q_2} A_{q_2q_5} \mid A_{q_0q_3} A_{q_3q_5} \mid A_{q_0q_4} A_{q_4q_5} \mid A_{q_0q_5} A_{q_5q_5}$$

⋮

$$A_{q_5q_5} \rightarrow A_{q_5q_0} A_{q_0q_5} \mid A_{q_5q_1} A_{q_1q_5} \mid A_{q_5q_2} A_{q_2q_5} \mid A_{q_5q_3} A_{q_3q_5} \mid A_{q_5q_4} A_{q_4q_5} \mid A_{q_5q_5} A_{q_5q_5}$$

Start variable $A_{q_0q_5}$

Kind 3: from pairs of transitions



$$A_{q_0 q_5} \rightarrow A_{q_1 q_4}$$

$$A_{q_2 q_4} \rightarrow a A_{q_2 q_4}$$

$$A_{q_2 q_2} \rightarrow A_{q_2 q_2} b$$

$$A_{q_1 q_4} \rightarrow A_{q_2 q_4}$$

$$A_{q_2 q_2} \rightarrow a A_{q_2 q_2} b$$

$$A_{q_2 q_4} \rightarrow A_{q_3 q_3}$$

$$A_{q_2 q_4} \rightarrow a A_{q_2 q_3}$$

$$A_{q_2 q_4} \rightarrow A_{q_3 q_4}$$

Suppose that a PDA M is converted
to a context-free grammar G

G

Brace matching

We need to prove that $L(G) = L(M)$

or equivalently

$$L(G) \subseteq L(M)$$

$$L(G) \supseteq L(M)$$

$$L(G) \subseteq L(M)$$

We need to show that if G has derivation:

$$A_{q_0 q_f} \xRightarrow{*} w \quad (\text{string of terminals})$$

Then there is an accepting computation in M :

$$(q_0, w, \#) \xrightarrow{*} (q_f, \varepsilon, \#)$$

with input string w

We will actually show that if G has derivation:

$$A_{pq} \stackrel{*}{\Rightarrow} w$$

Then there is a computation in M :

$$(p, w, \varepsilon) \stackrel{*}{\succ} (q, \varepsilon, \varepsilon)$$

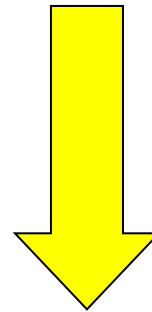
Therefore:

$$A_{q_0 q_f} \stackrel{*}{\Rightarrow} w$$



$$(q_0, w, \varepsilon) \stackrel{*}{\succ} (q_f, \varepsilon, \varepsilon)$$

Since there is no transition
with the # symbol



$$(q_0, w, \#) \stackrel{*}{\succ} (q_f, \varepsilon, \#)$$

Lemma:

If $A_{pq} \xRightarrow{*} w$ (string of terminals)

then there is a computation
from state p to state q on string w
which leaves the stack empty:

$$(p, w, \varepsilon) \xRightarrow{*} (q, \varepsilon, \varepsilon)$$

Proof Intuition:

$$A_{pq} \Rightarrow \dots \Rightarrow W$$

Type 2

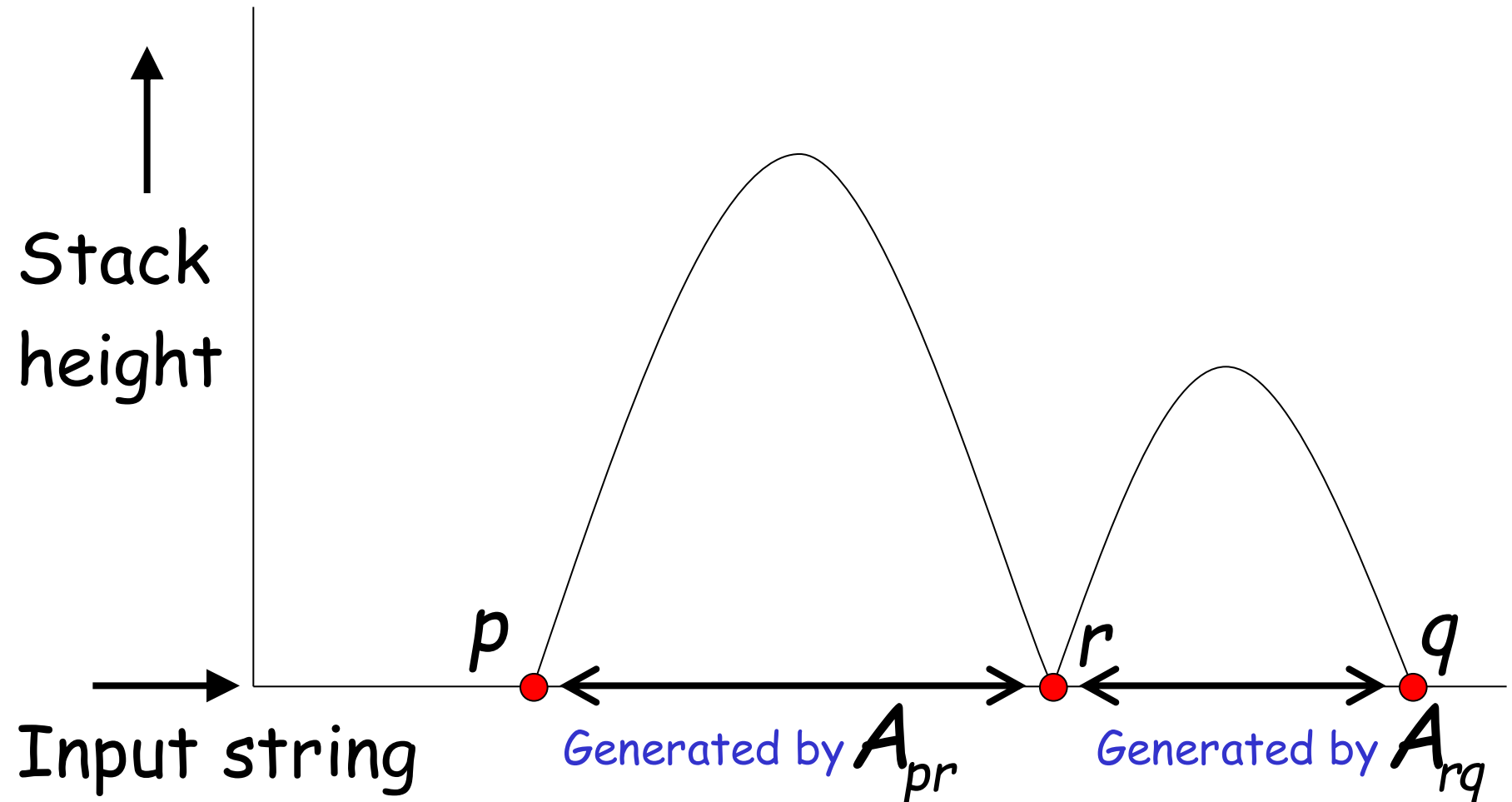
Case 1: $A_{pq} \Rightarrow A_{pr} A_{rq} \Rightarrow \dots \Rightarrow W$

Type 3

Case 2: $A_{pq} \Rightarrow a A_{rs} b \Rightarrow \dots \Rightarrow W$

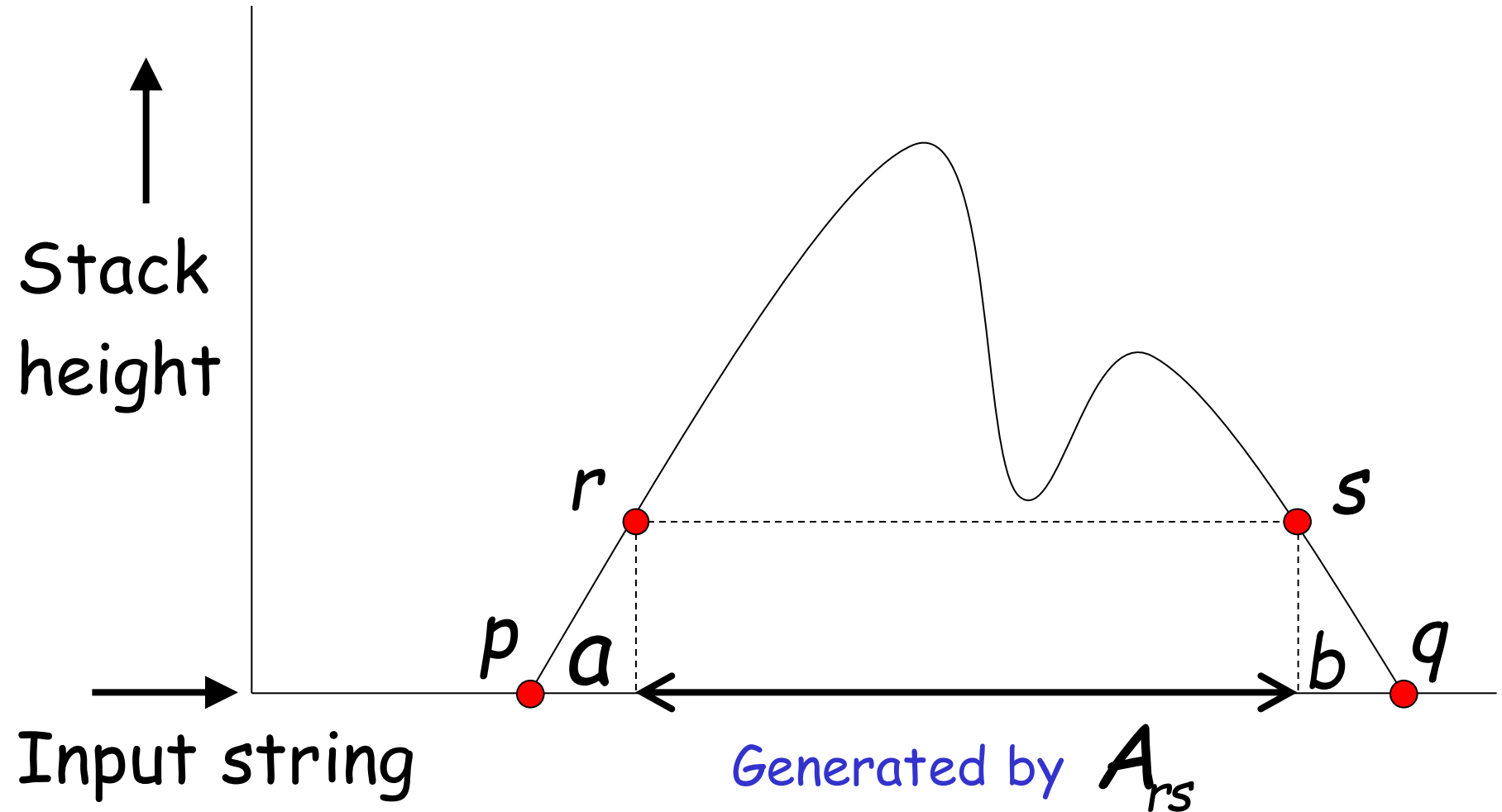
Type 2

Case 1: $A_{pq} \Rightarrow A_{pr} A_{rq} \Rightarrow \dots \Rightarrow w$



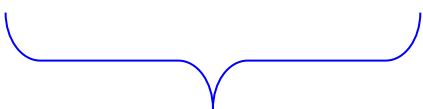
Type 3

Case 2: $A_{pq} \Rightarrow aA_{rs}b \Rightarrow \dots \Rightarrow w$



Formal Proof:

We formally prove this claim
by induction on the number
of steps in derivation:

$$A_{pq} \Rightarrow \cdots \Rightarrow W$$


number of steps

Induction Basis: $A_{pq} \Rightarrow w$

(one derivation step)

A Kind 1 production must have been used:

$$A_{pp} \rightarrow \varepsilon$$

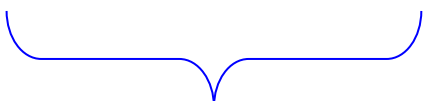
Therefore, $p = q$ and $w = \varepsilon$

This computation of PDA trivially exists:

$$(p, \varepsilon, \varepsilon) \stackrel{*}{\succ} (p, \varepsilon, \varepsilon)$$

Induction Hypothesis:

$$A_{pq} \Rightarrow \cdots \Rightarrow w$$

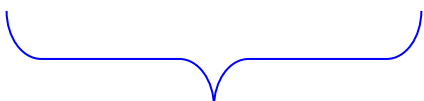

 k derivation steps

suppose it holds:

$$(p, w, \varepsilon) \overset{*}{\succ} (q, \varepsilon, \varepsilon)$$

Induction Step:

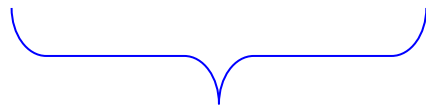
$$A_{pq} \Rightarrow \cdots \Rightarrow w$$


 $k + 1$ derivation steps

We have to show:

$$(p, w, \varepsilon) \stackrel{*}{\succ} (q, \varepsilon, \varepsilon)$$

$$A_{pq} \Rightarrow \dots \Rightarrow w$$



$k + 1$ derivation steps

Type 2

Case 1: $A_{pq} \Rightarrow A_{pr} A_{rq} \Rightarrow \dots \Rightarrow w$

Type 3

Case 2: $A_{pq} \Rightarrow a A_{rs} b \Rightarrow \dots \Rightarrow w$

Type 2

Case 1: $A_{pq} \Rightarrow A_{pr} A_{rq} \Rightarrow \dots \Rightarrow w$

$k + 1$ steps

We can write $w = yz$

$$A_{pr} \Rightarrow \dots \Rightarrow y$$

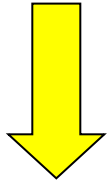
At most k steps

$$A_{rq} \Rightarrow \dots \Rightarrow z$$

At most k steps

$$A_{pr} \Rightarrow \cdots \Rightarrow y$$

At most k steps



From induction
hypothesis, in PDA:

$$(p, y, \varepsilon) \stackrel{*}{\succ} (r, \varepsilon, \varepsilon)$$

$$A_{rq} \Rightarrow \cdots \Rightarrow z$$

At most k steps

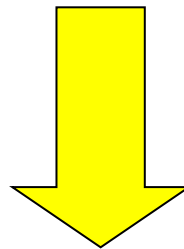


From induction
hypothesis, in PDA:

$$(r, z, \varepsilon) \stackrel{*}{\succ} (q, \varepsilon, \varepsilon)$$

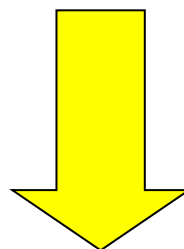
$$(p, y, \varepsilon) \succ^* (r, \varepsilon, \varepsilon)$$

$$(r, z, \varepsilon) \succ^* (q, \varepsilon, \varepsilon)$$



$$(p, yz, \varepsilon) \succ^* (r, z, \varepsilon) \succ^* (q, \varepsilon, \varepsilon)$$

since $w = yz$



$$(p, w, \varepsilon) \succ^* (q, \varepsilon, \varepsilon)$$

Type 3

Case 2: $A_{pq} \Rightarrow aA_{rs}b \Rightarrow \dots \Rightarrow w$

$k + 1$ steps

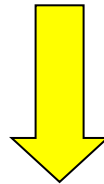
We can write $w = ayb$

$$A_{rs} \Rightarrow \dots \Rightarrow y$$

At most k steps

$$A_{rs} \Rightarrow \cdots \Rightarrow y$$

At most k steps

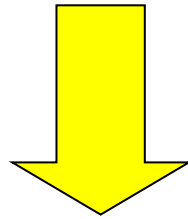


From induction hypothesis,
the PDA has computation:

$$(r, y, \varepsilon) \stackrel{*}{\succ} (s, \varepsilon, \varepsilon)$$

Type 3

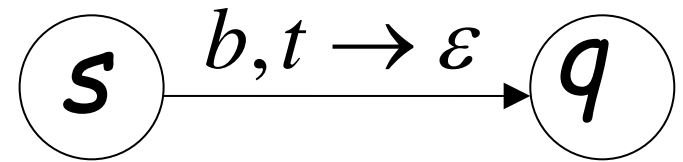
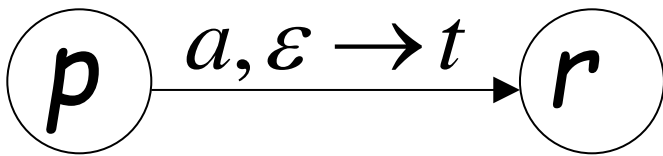
$$A_{pq} \Rightarrow aA_{rs}b \Rightarrow \dots \Rightarrow w$$

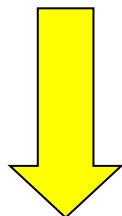
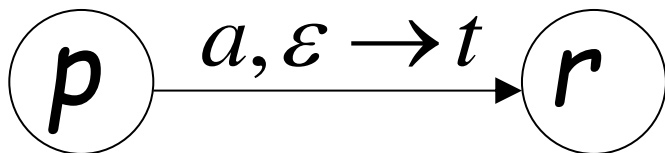


Grammar contains production

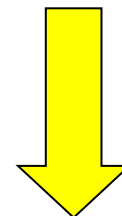
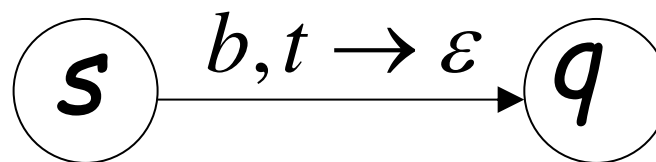
$$A_{pq} \rightarrow aA_{rs}b$$

And PDA Contains transitions





$$(p, ayb, \varepsilon) \succ (r, yb, t)$$



$$(s, b, t) \succ (q, \varepsilon, \varepsilon)$$

We know

$$(r, y, \varepsilon) \succ^* (s, \varepsilon, \varepsilon) \quad \Longrightarrow \quad (r, yb, t) \succ^* (s, b, t)$$

We also know

$$(p, ayb, \varepsilon) \succ (r, yb, t)$$

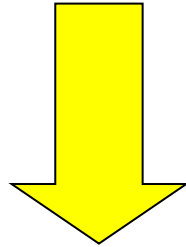
$$(s, b, t) \succ (q, \varepsilon, \varepsilon)$$

Therefore:

$$(p, ayb, \varepsilon) \succ (r, yb, t) \succ^* (s, b, t) \succ (q, \varepsilon, \varepsilon)$$

$$(p, ayb, \varepsilon) \succ (r, yb, t) \succ^* (s, b, t) \succ (q, \varepsilon, \varepsilon)$$

since $w = ayb$



$$(p, w, \varepsilon) \succ^* (q, \varepsilon, \varepsilon)$$

END OF PROOF

So far we have shown:

$$L(G) \subseteq L(M)$$

With a similar proof we can show

$$L(G) \supseteq L(M)$$

Therefore: $L(G) = L(M)$



BLM2502

Theory of

Computation

Spring 2016

BLM2502 Theory of Computation

» Course Outline

» Week Content

- » 1 Introduction to Course
- » 2 Computability Theory, Complexity Theory, Automata Theory, Set Theory, Relations, Proofs, Pigeonhole Principle
- » 3 Regular Expressions
- » 4 Finite Automata
- » 5 Deterministic and Nondeterministic Finite Automata
- » 6 Epsilon Transition, Equivalence of Automata
- » 7 Pumping Theorem
- » 8 April 10 - 14 week is the first midterm week
- » 9 Context Free Grammars, Parse Tree, Ambiguity
- » 10 Pumping Theorem, Normal Forms
- » 11 Pushdown Automata
- » 12 **Turing Machines, Recognition and Computation, Church-Turing Hypothesis**
- » 13 Turing Machines, Recognition and Computation, Church-Turing Hypothesis
- » 14 May 22 – 27 week is the second midterm week
- » 15 Review
- » 16 Final Exam date will be announced



Turing Machines

The Language Hierarchy

$a^n b^n c^n$?

ww ?

Context-Free Languages

$a^n b^n$

ww^R

Regular Languages

a^*

$a^* b^*$

The diagram consists of three concentric ellipses. The outermost ellipse is labeled 'Languages accepted by Turing Machines'. Inside it is an ellipse labeled 'Context-Free Languages'. Inside that is the innermost ellipse labeled 'Regular Languages'. Each level contains specific language examples.

Languages accepted by
Turing Machines

$a^n b^n c^n$

ww

Context-Free Languages

$a^n b^n$

ww^R

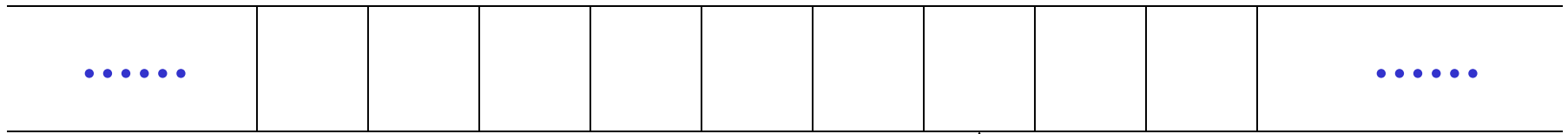
Regular Languages

a^*

$a^* b^*$

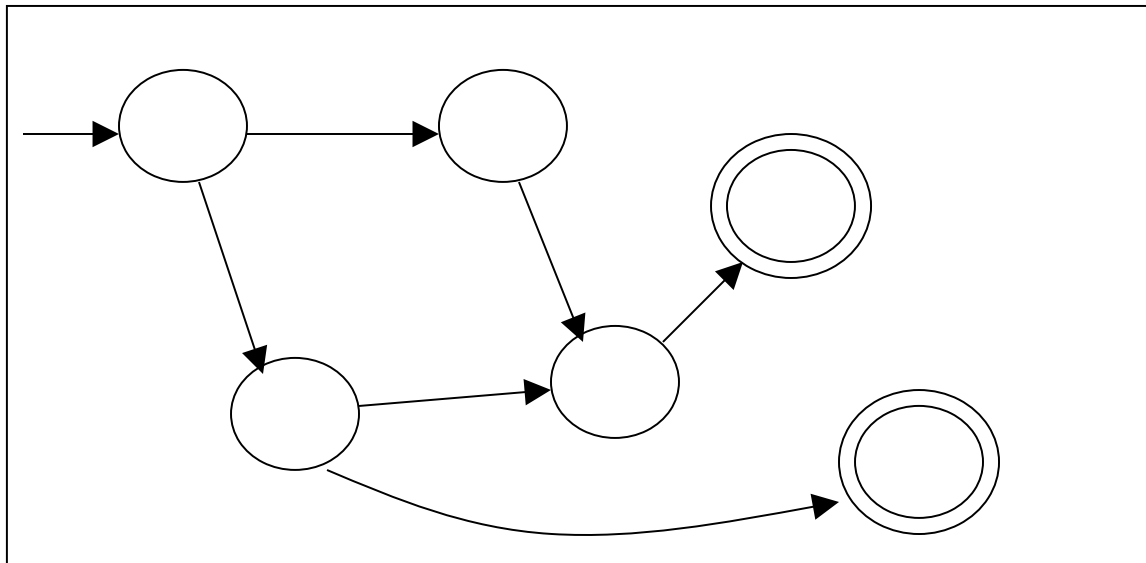
A Turing Machine

Tape



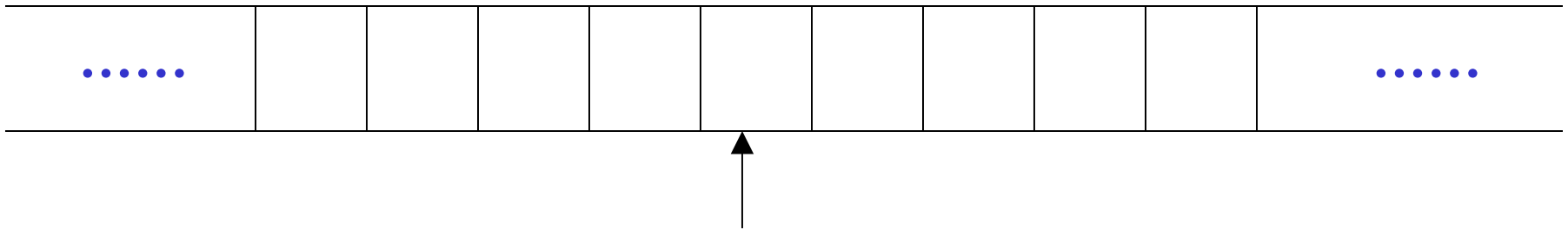
Read-Write head

Control Unit



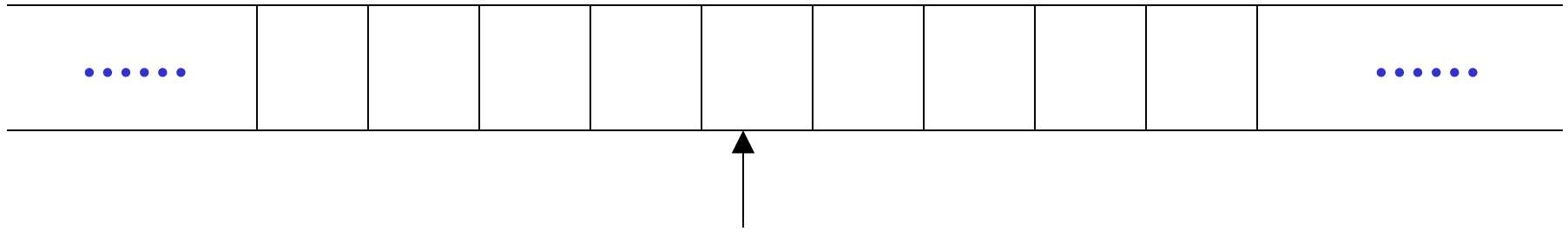
The Tape

No boundaries -- infinite length



Read-Write head

The head moves Left or Right



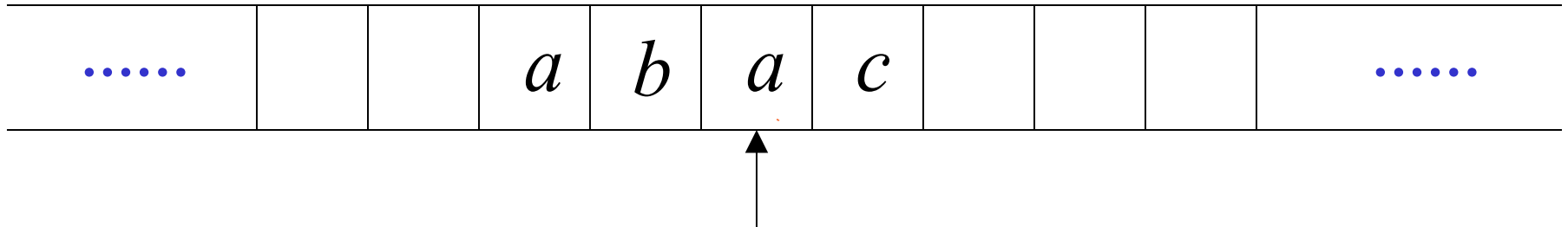
Read-Write head

The head at each transition (time step):

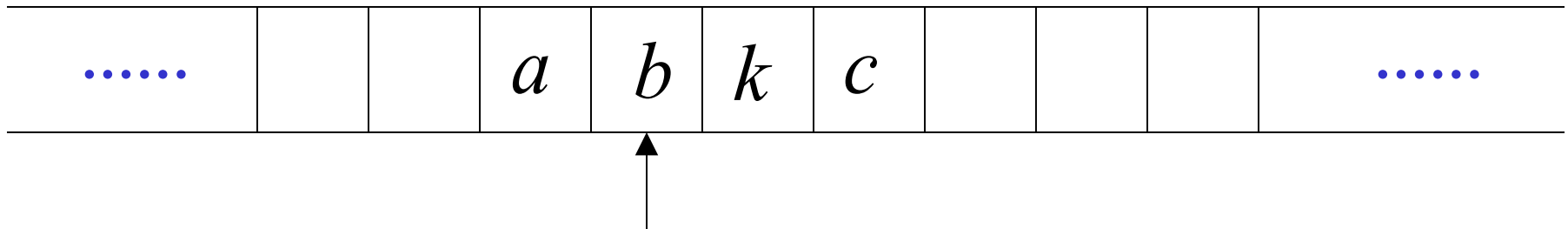
1. Reads a symbol
2. Writes a symbol
3. Moves Left or Right

Example:

Time 0

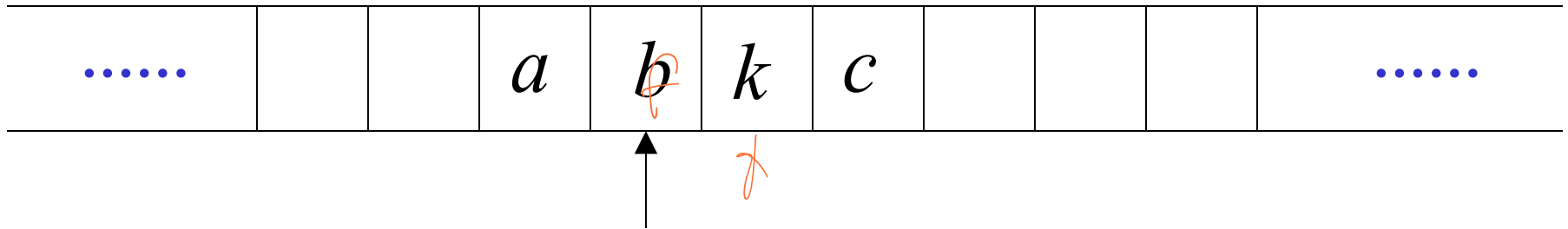


Time 1

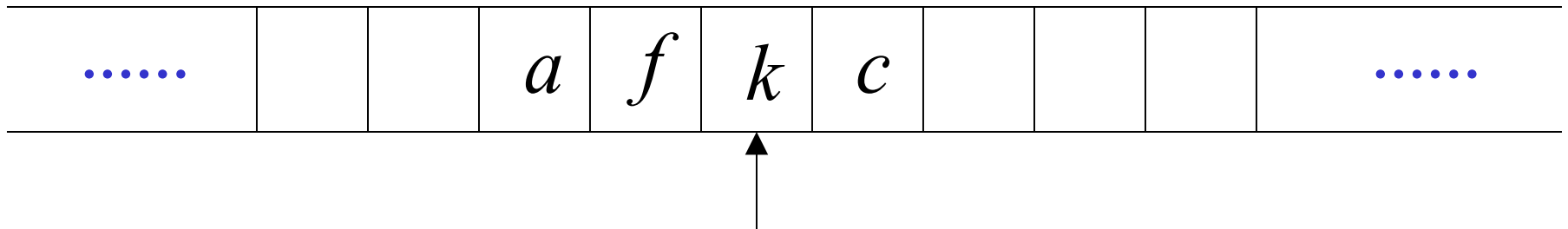


1. Reads a
2. Writes k
3. Moves Left

Time 1

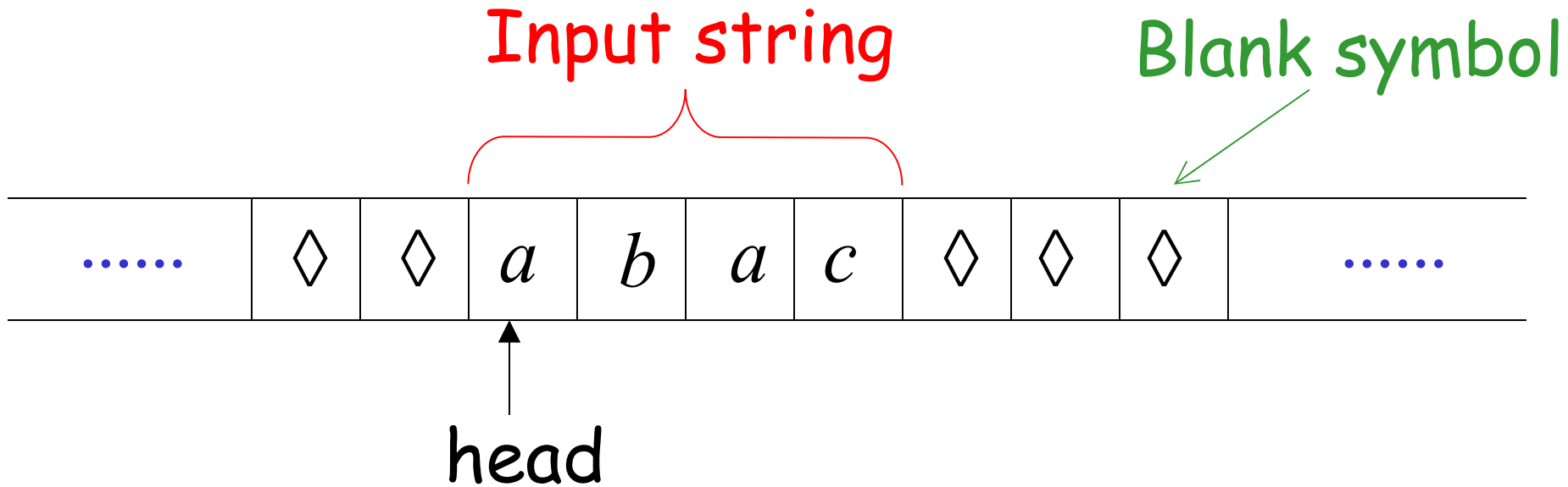


Time 2



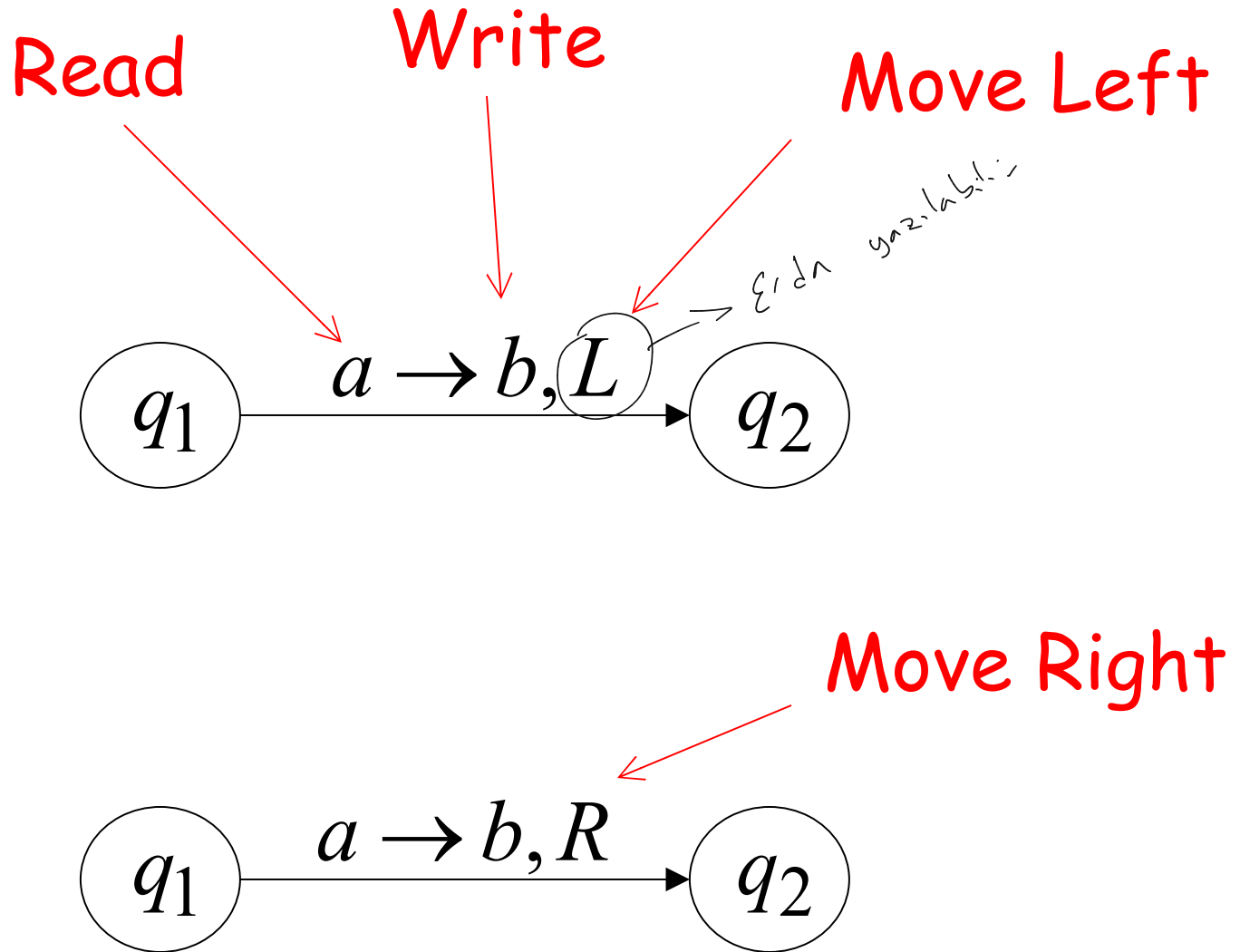
1. Reads b
2. Writes f
3. Moves Right

The Input String



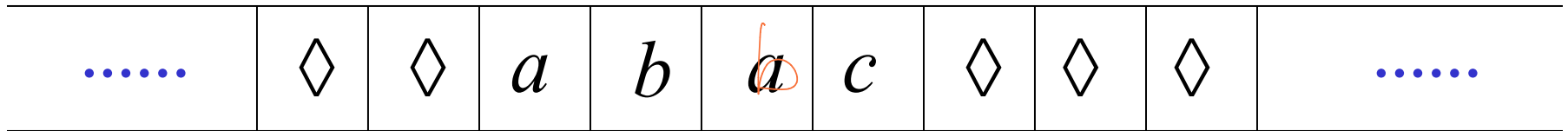
Head starts at the leftmost position
of the input string

States & Transitions



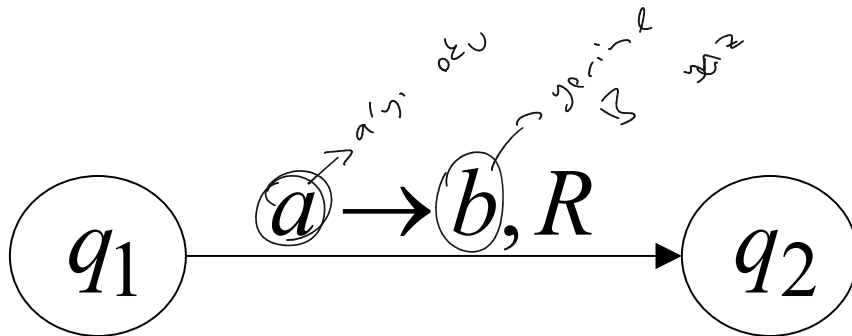
Example:

Time 1

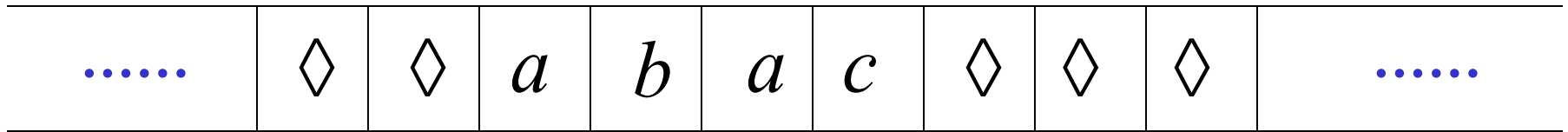


q_1

current state

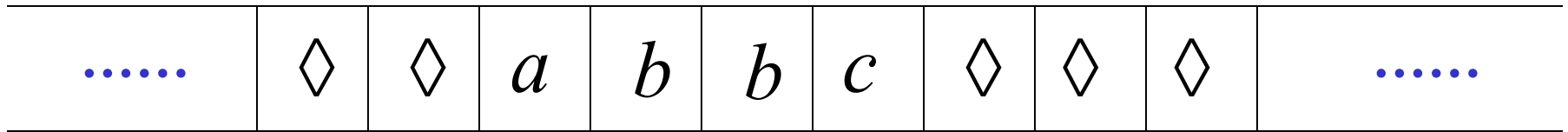


Time 1

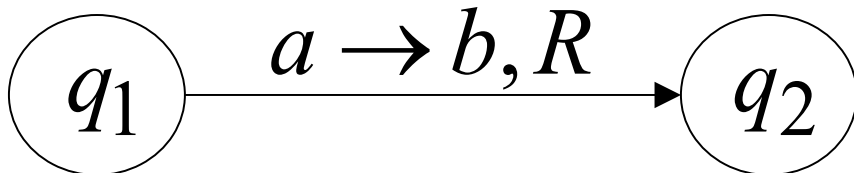


q_1

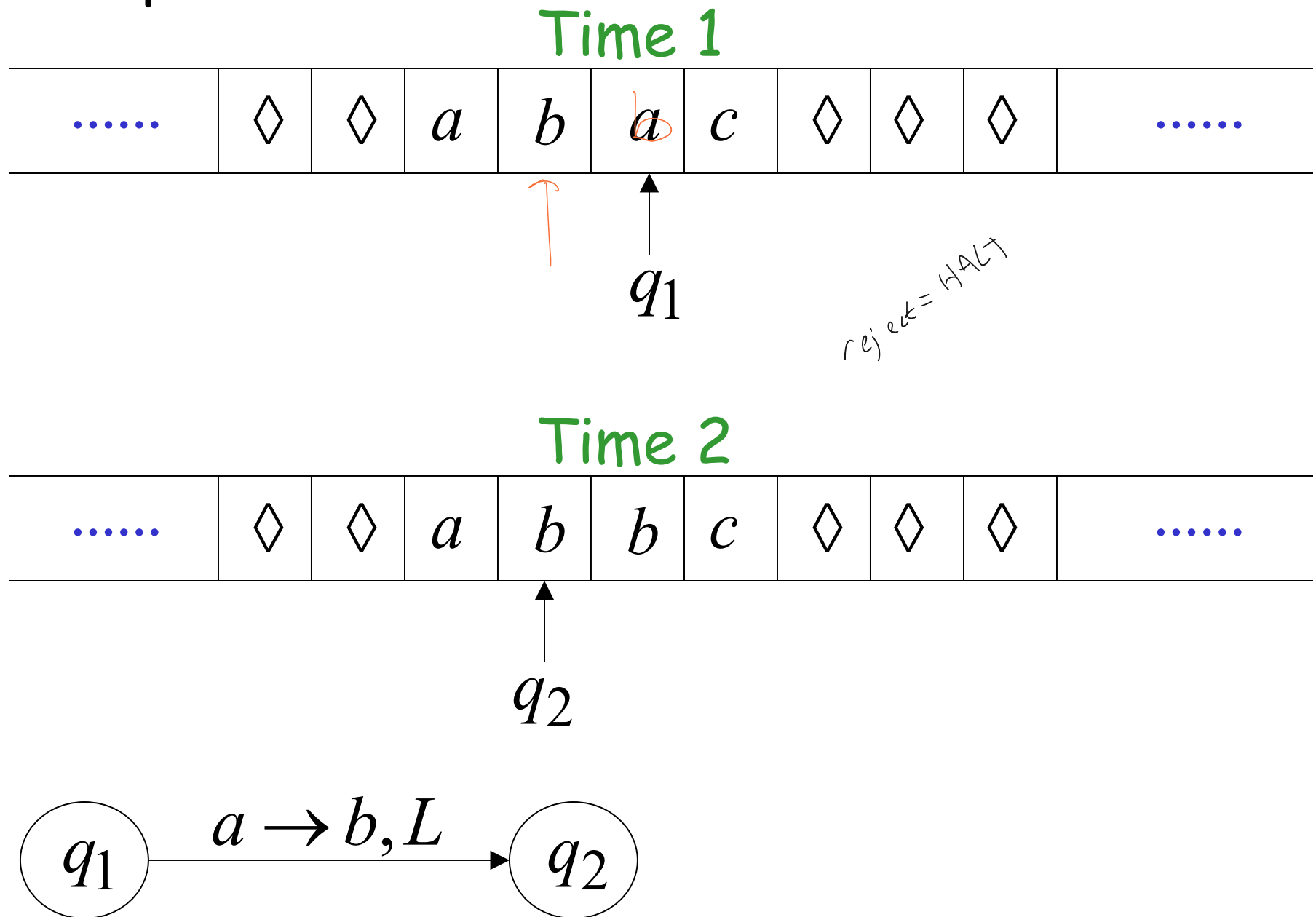
Time 2



q_2

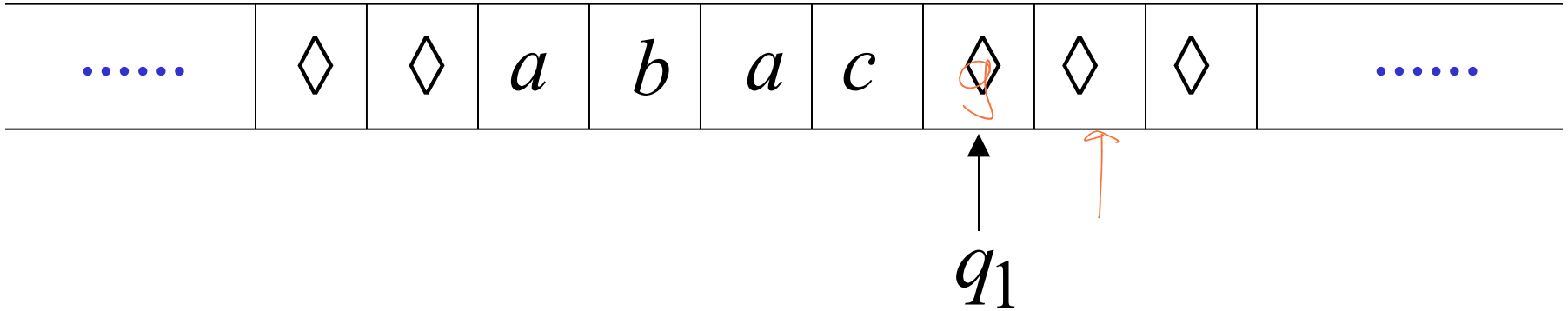


Example:

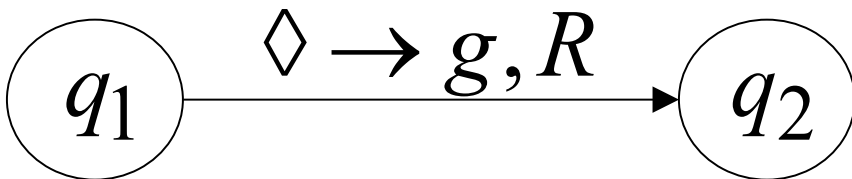
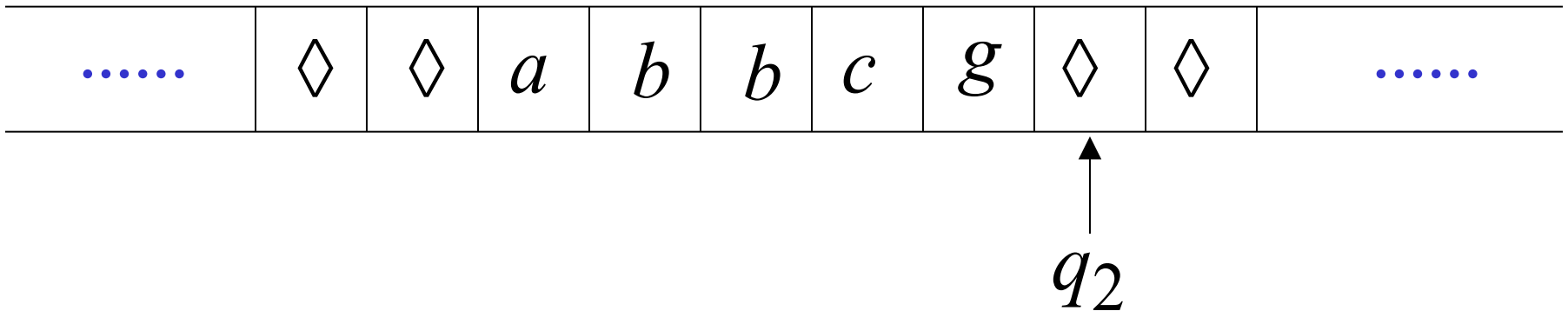


Example:

Time 1



Time 2

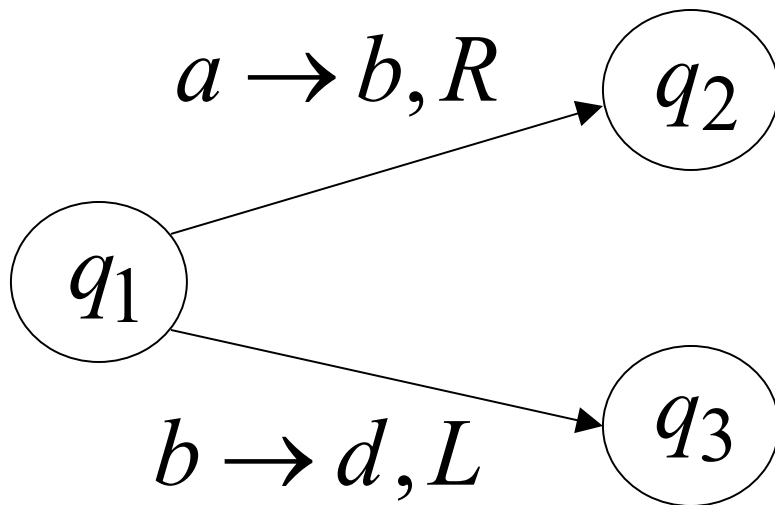


Determinism

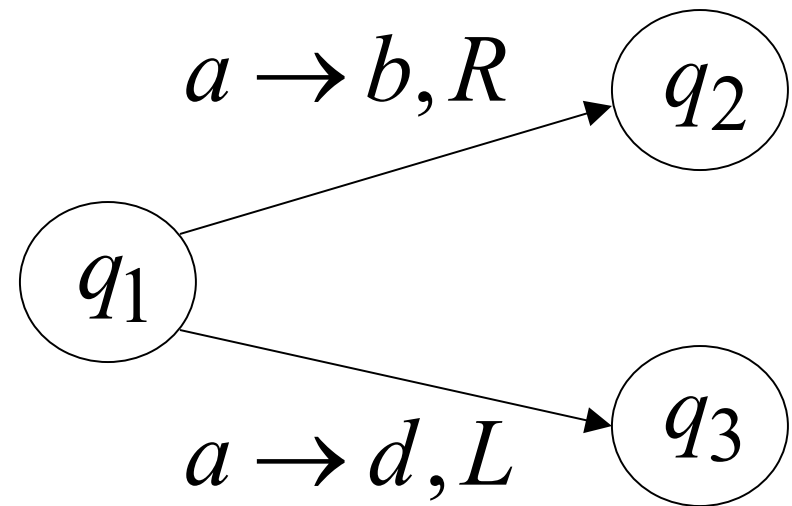
Turing Machines are deterministic

DFA ☆
g.b.

Allowed



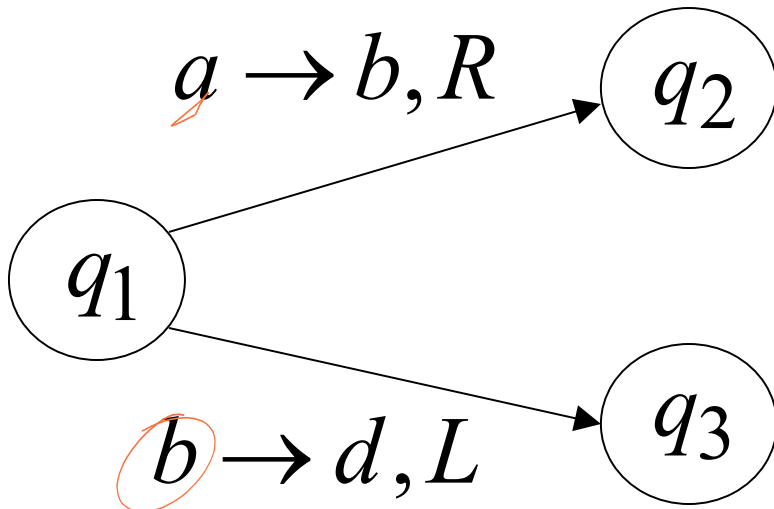
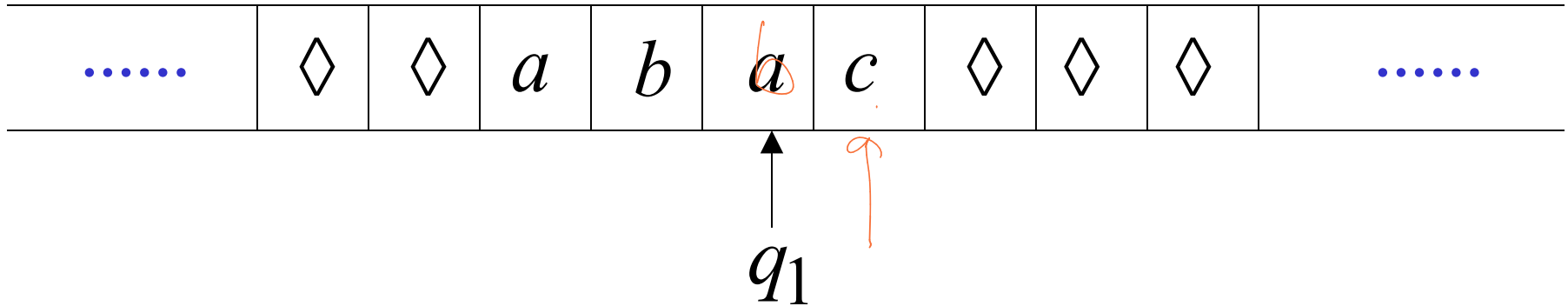
Not Allowed



No epsilon transitions allowed

Partial Transition Function

Example:



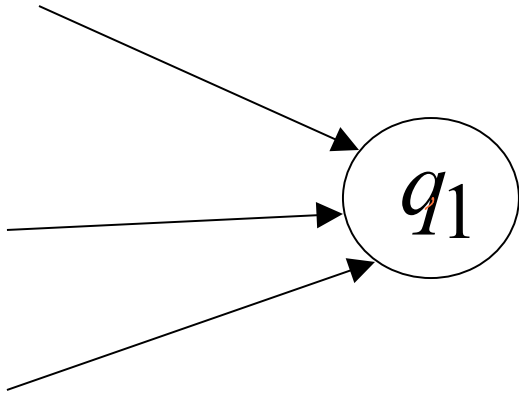
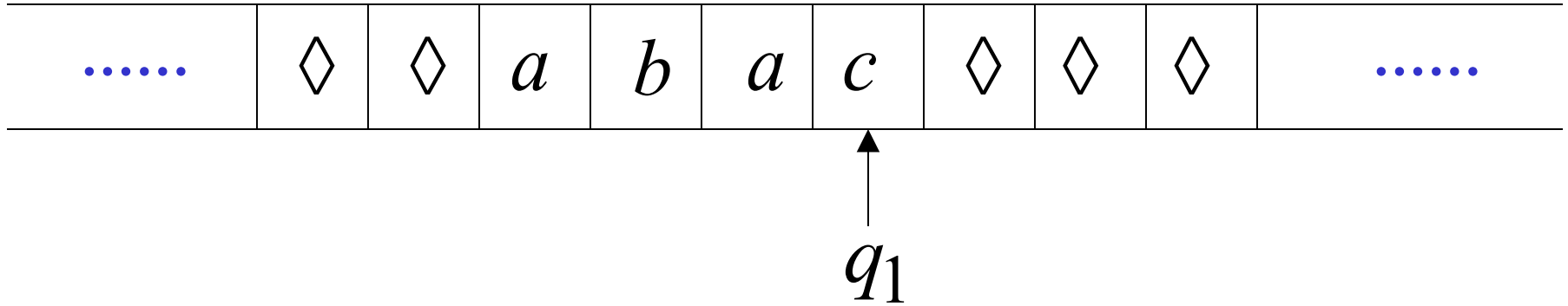
Allowed:

No transition
for input symbol c

Halting

The machine **halts** in a state if there is no transition to follow

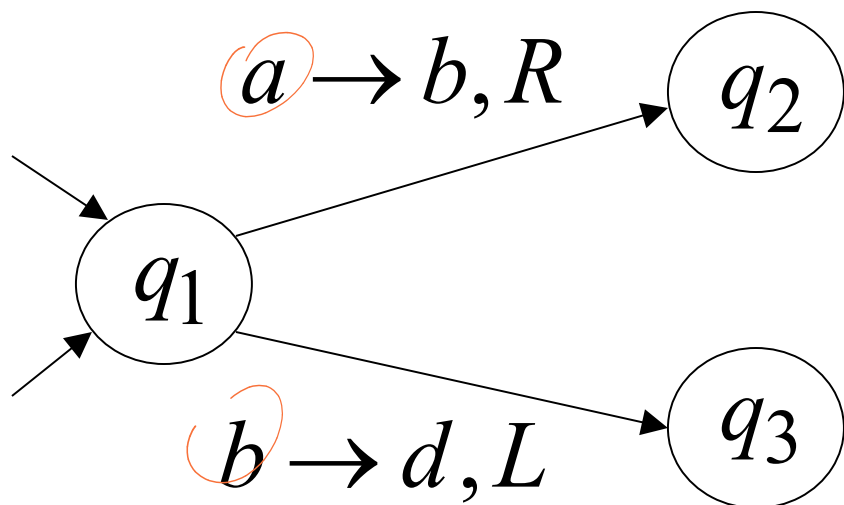
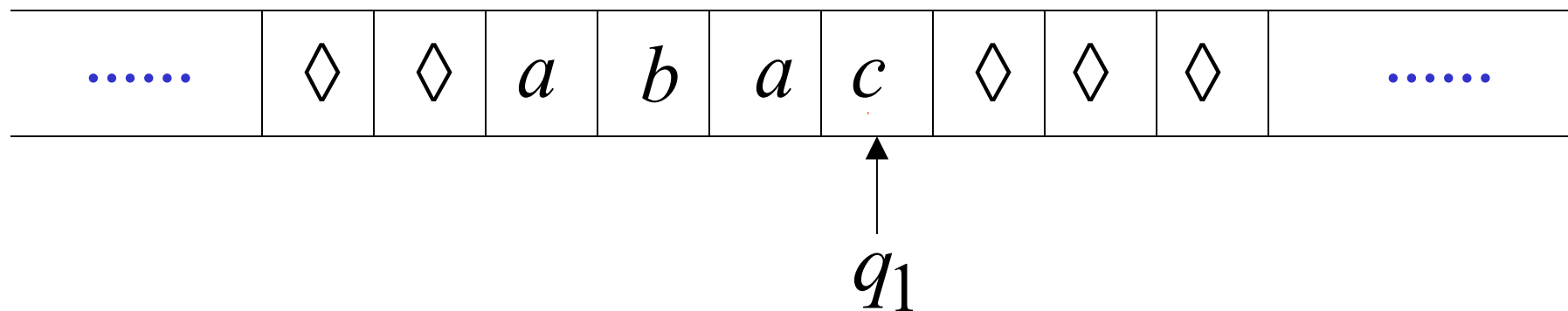
Halting Example 1:



No transition from q_1

HALT!!!

Halting Example 2:



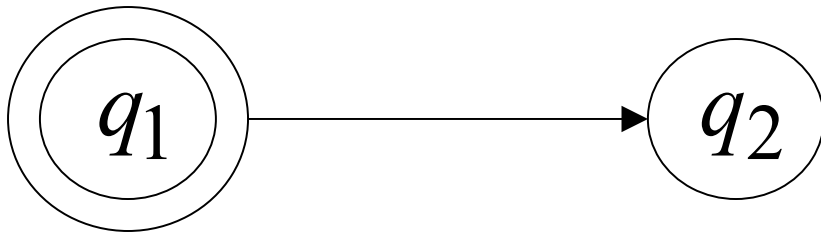
No possible transition
from q_1 and symbol c

HALT!!!

Accepting States



Allowed



Not Allowed

- Accepting states have no outgoing transitions
- The machine halts and accepts

Acceptance

Accept Input
string



If machine halts
in an accept state

Reject Input
string



If machine halts
in a non-accept state

*→ compiler or
glibc is an un-
supported*
or
If machine enters
an *infinite loop*

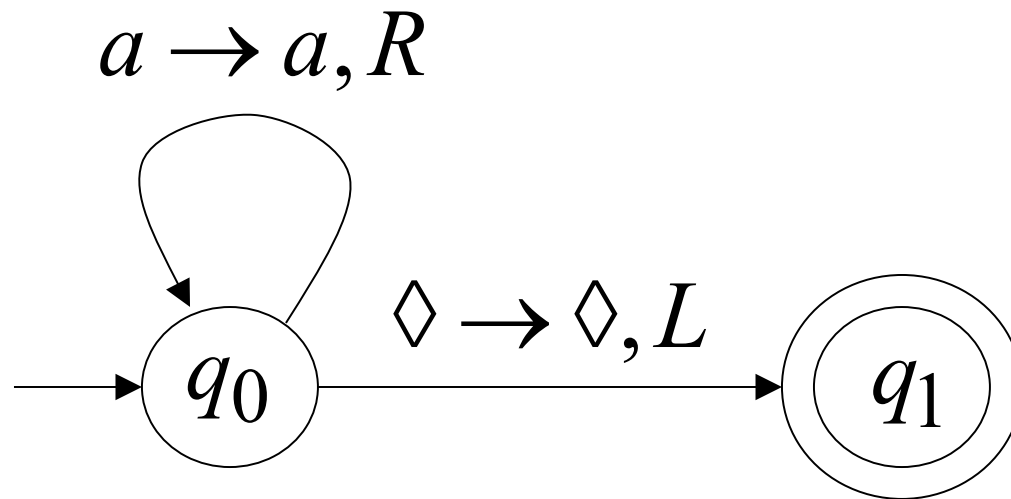
Observation:

In order to accept an input string,
it is not necessary to scan all the
symbols in the string

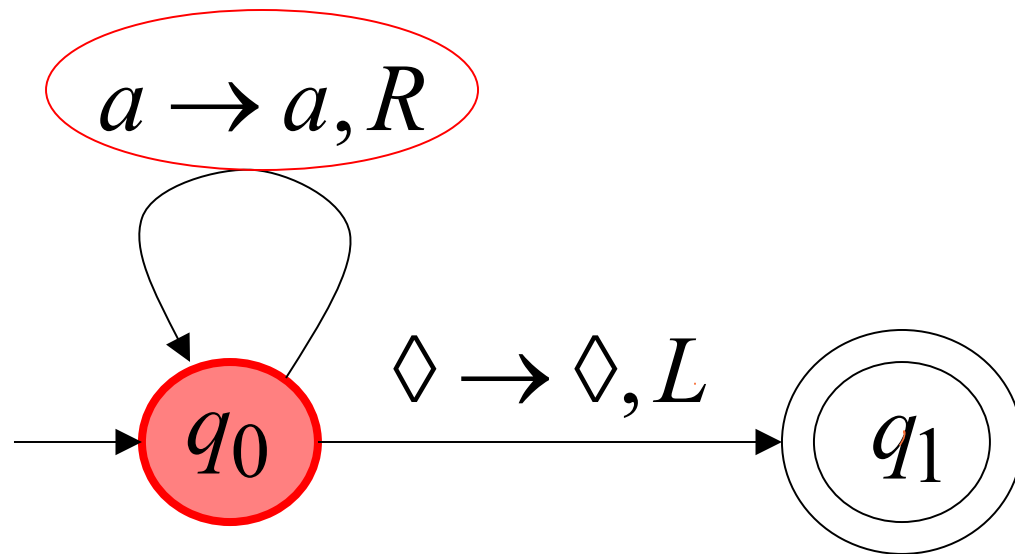
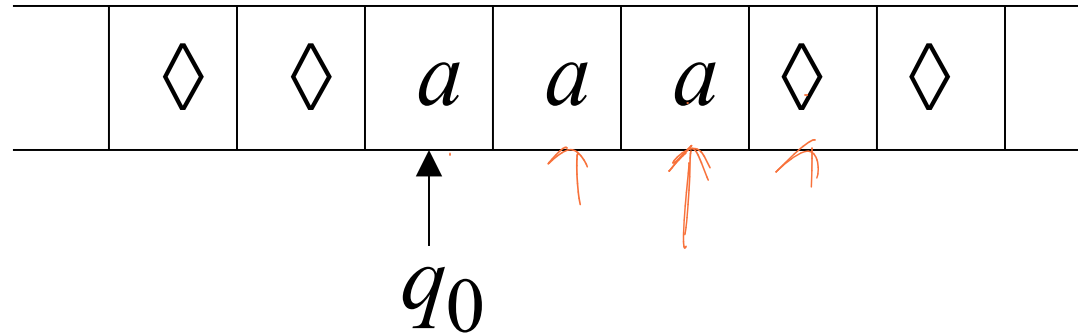
Turing Machine Example

Input alphabet $\Sigma = \{a, b\}$

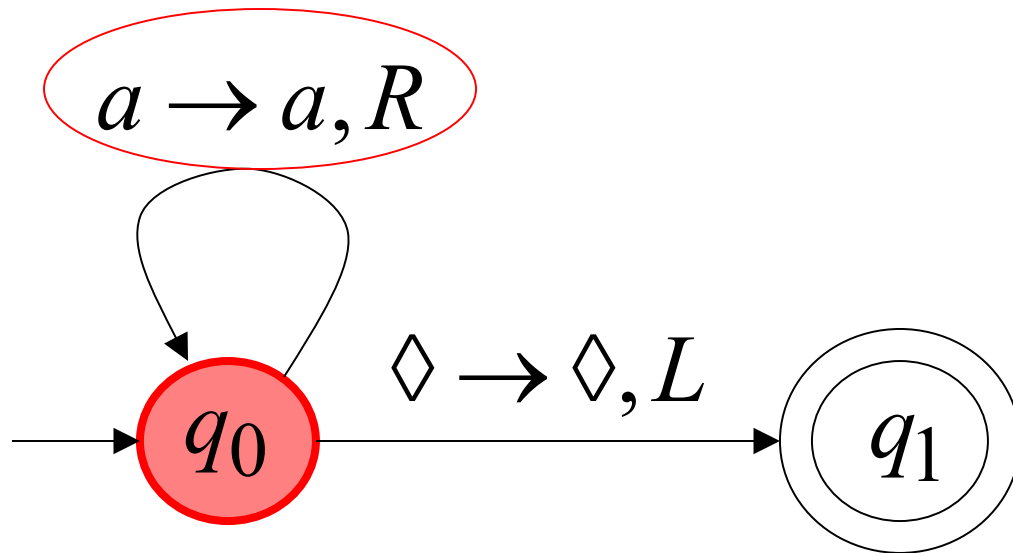
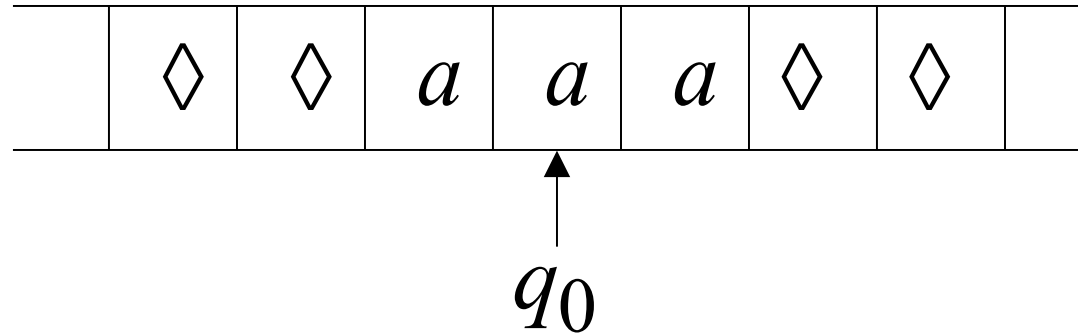
Accepts the language: a^*



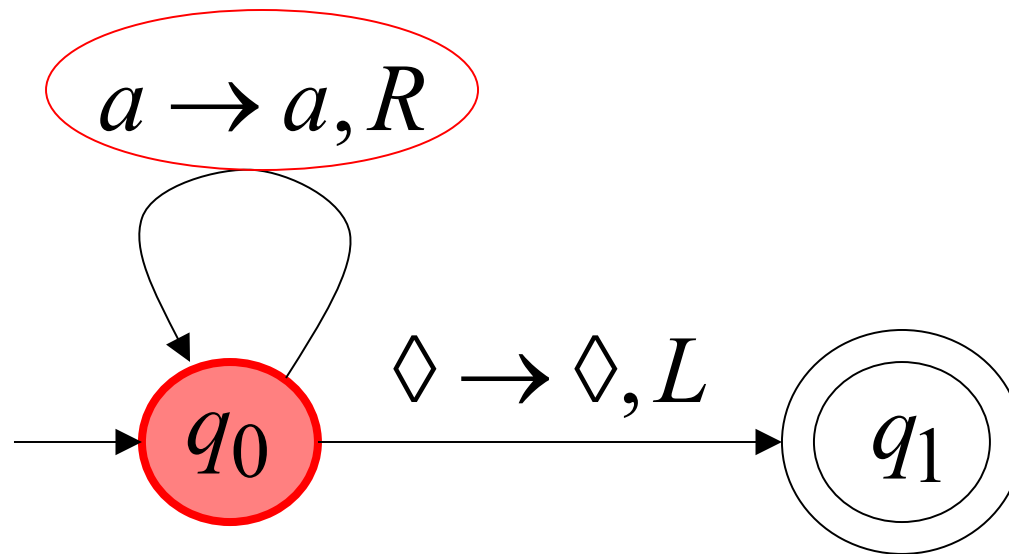
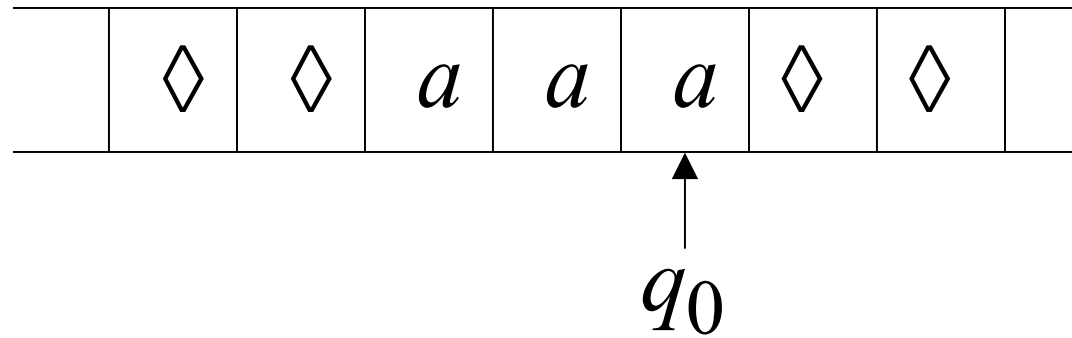
Time 0



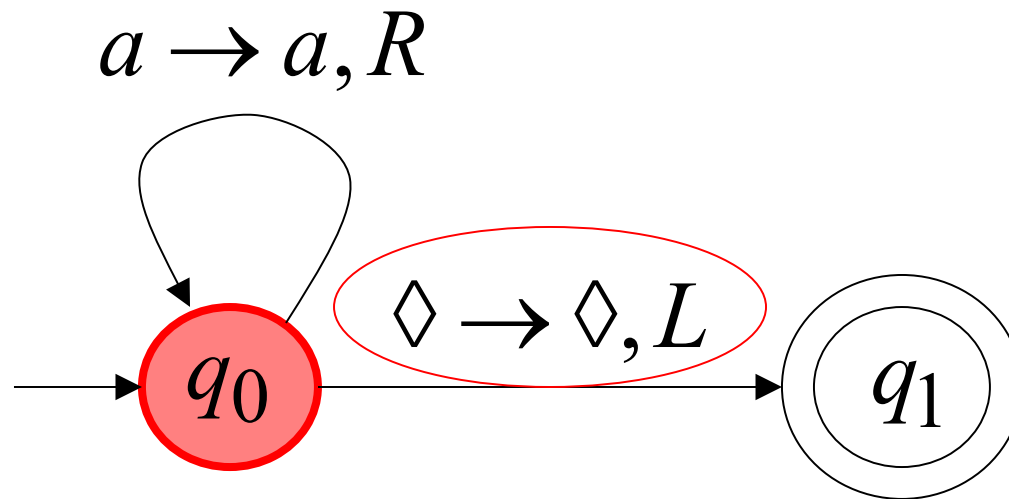
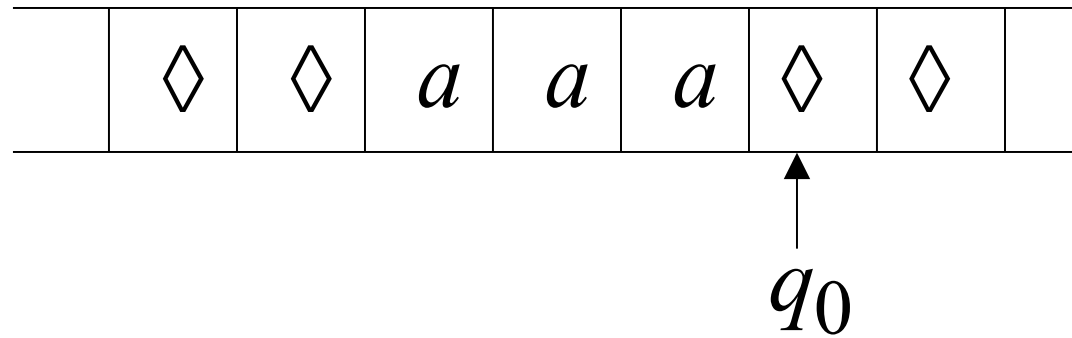
Time 1



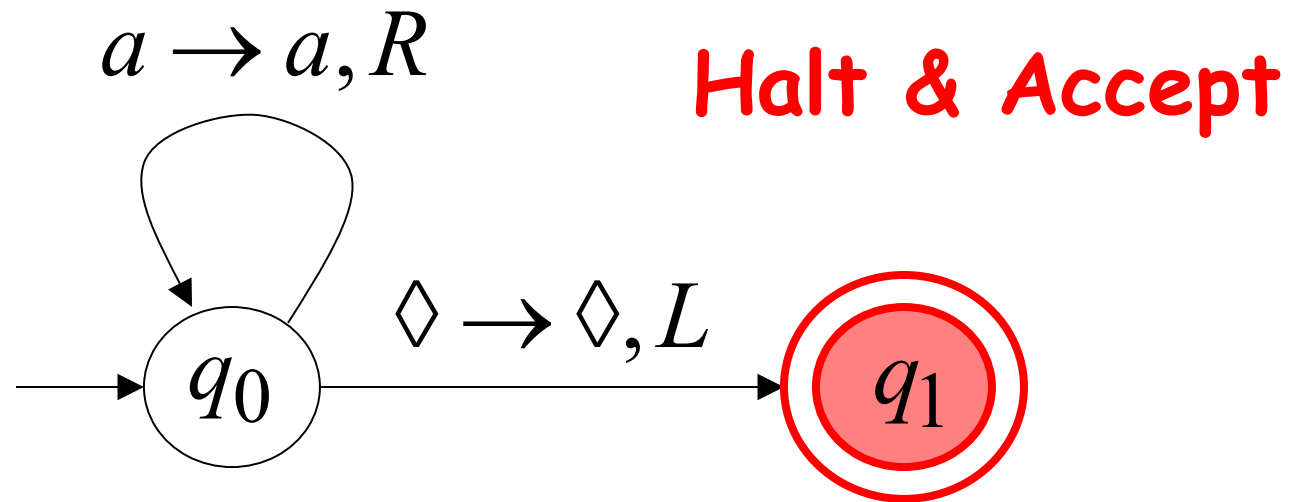
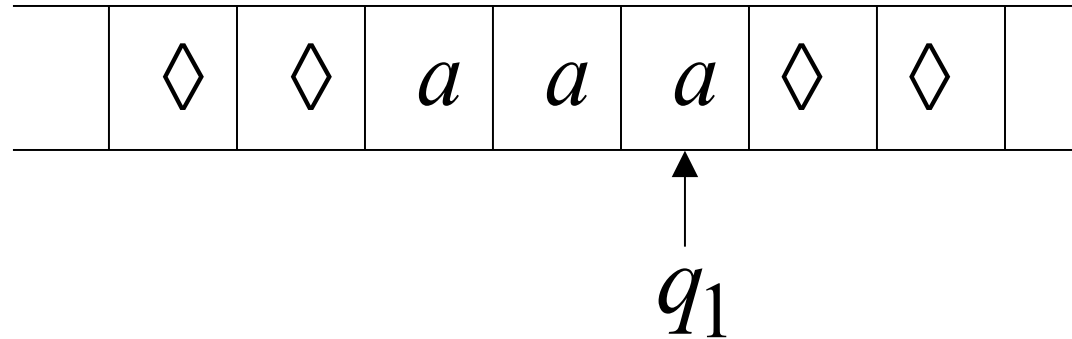
Time 2



Time 3

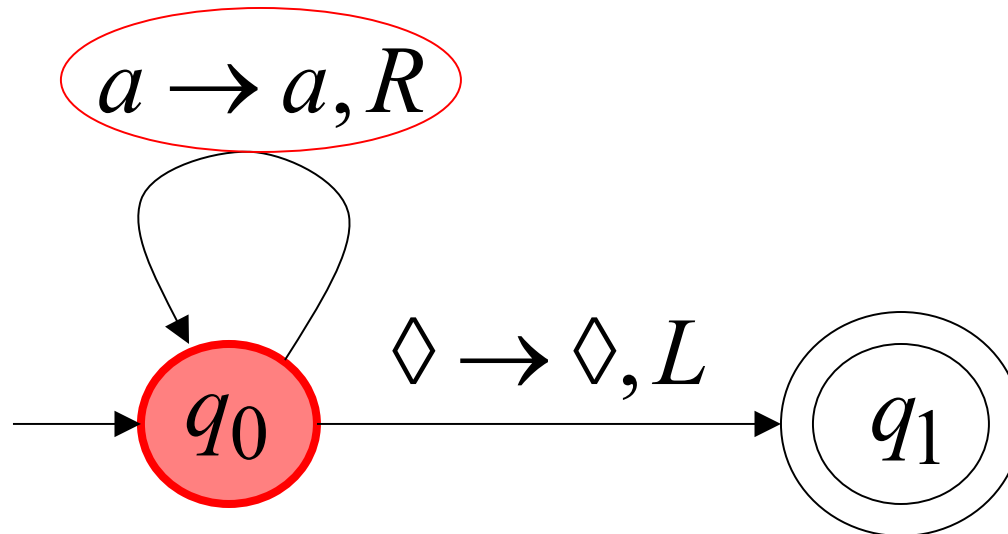
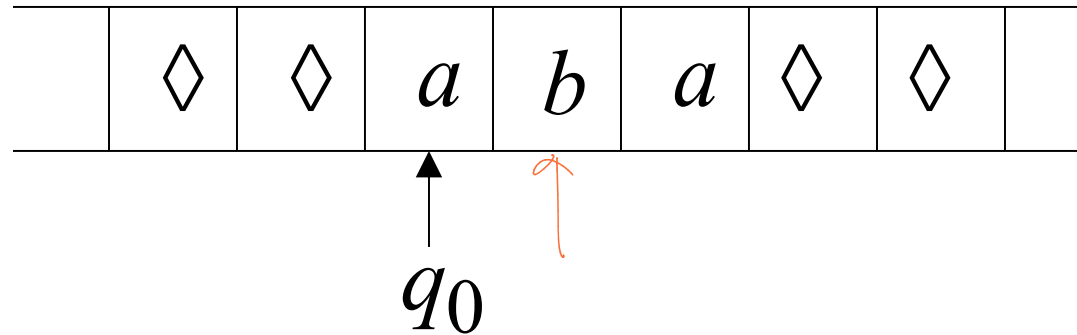


Time 4

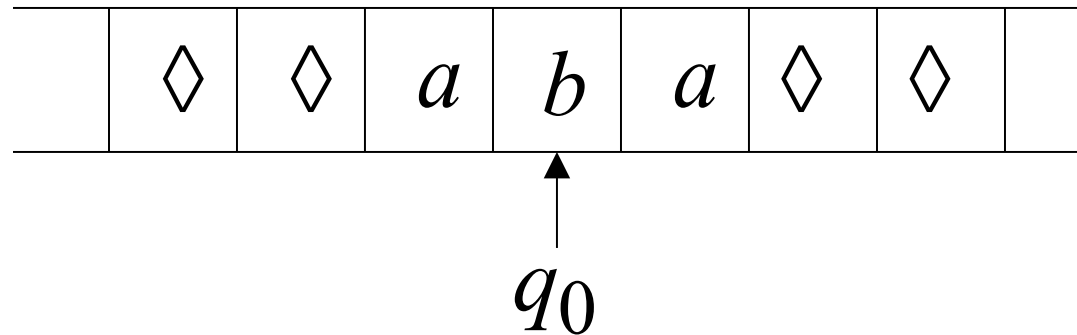


Rejection Example

Time 0

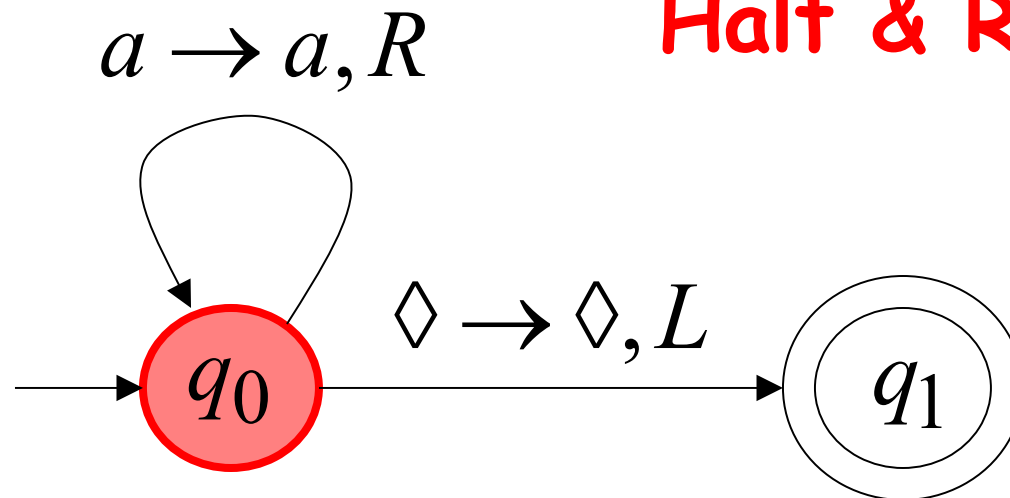


Time 1



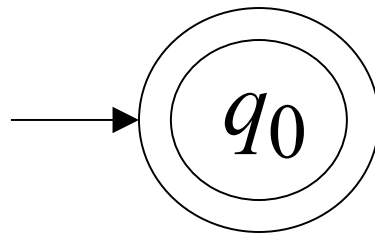
No possible Transition

Halt & Reject

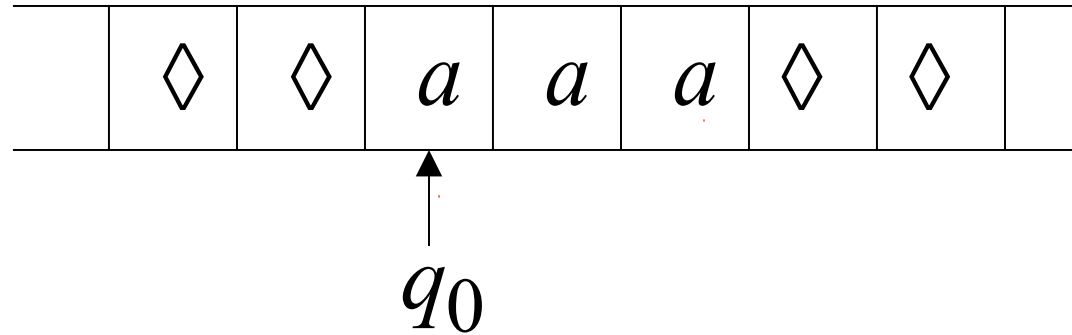


A simpler machine for same language
but for input alphabet $\Sigma = \{a\}$

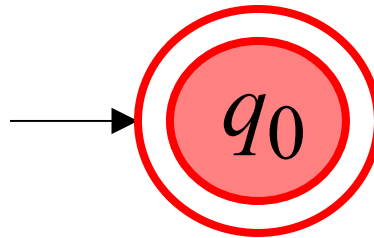
Accepts the language: a^*



Time 0



Halt & Accept



Not necessary to scan input

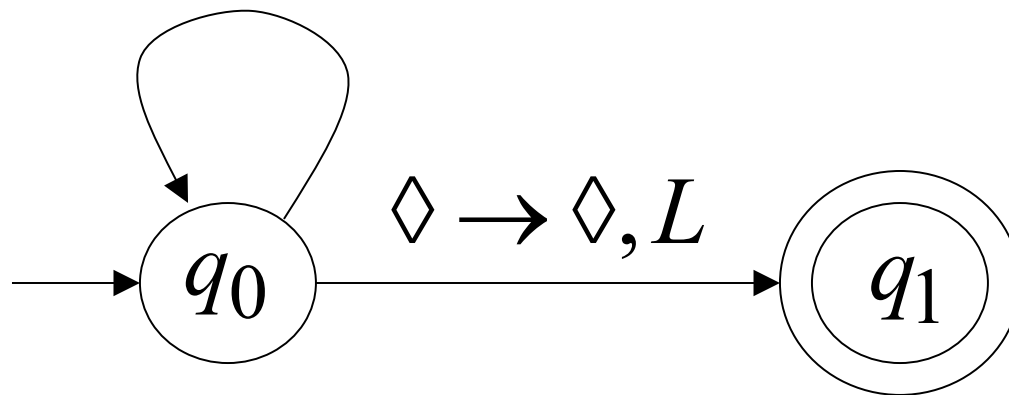
Infinite Loop Example

A Turing machine

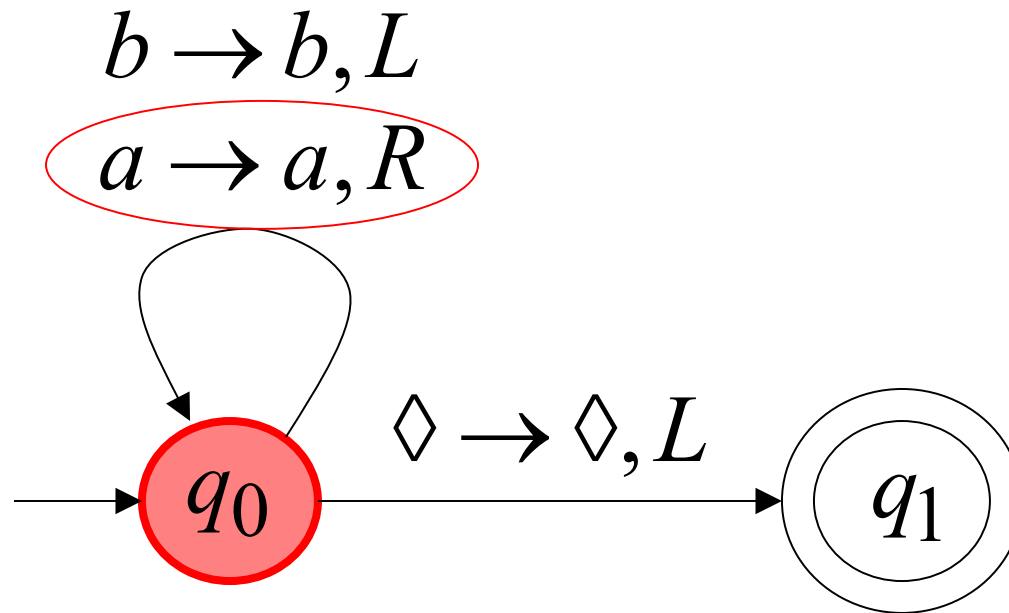
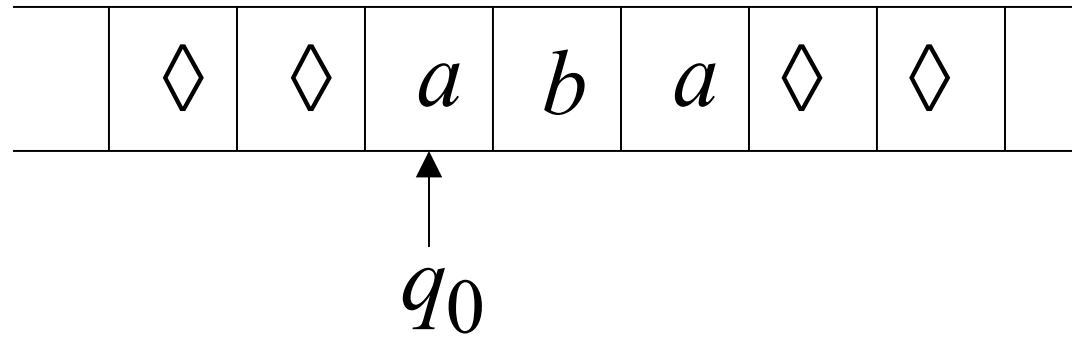
for language $a^* + b(a + b)^*$

$b \rightarrow b, L$

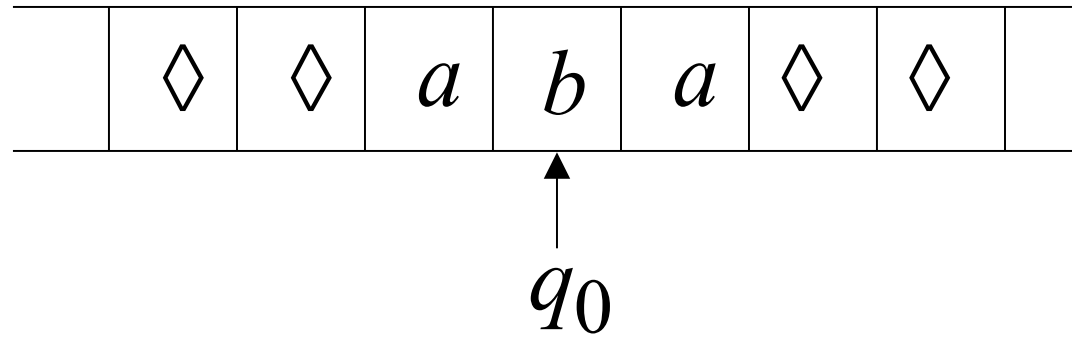
$a \rightarrow a, R$



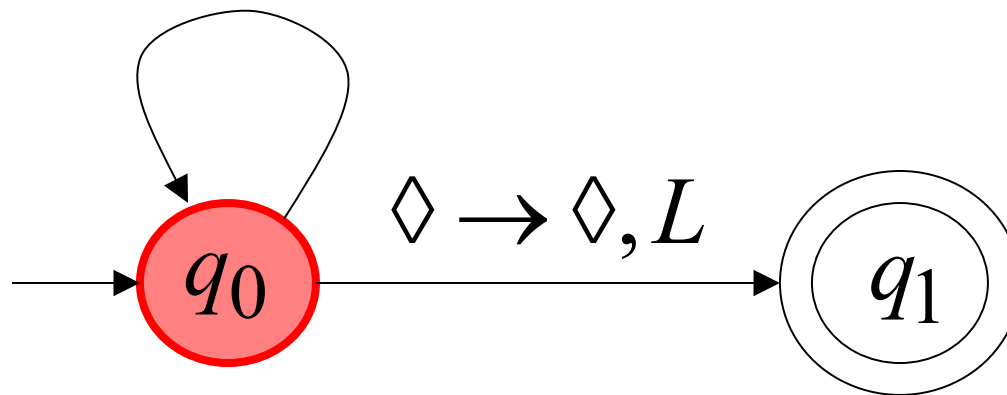
Time 0



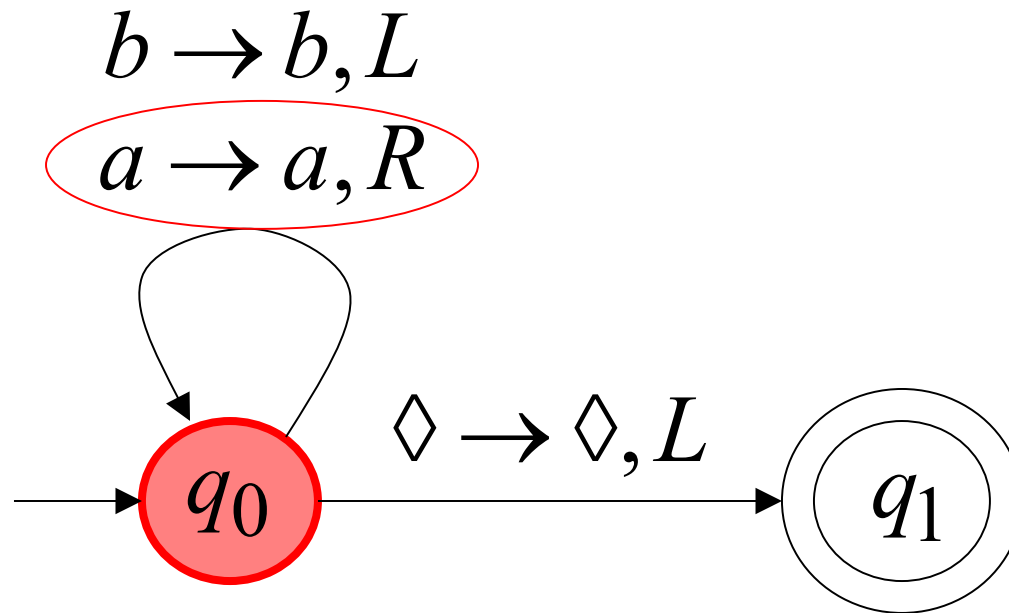
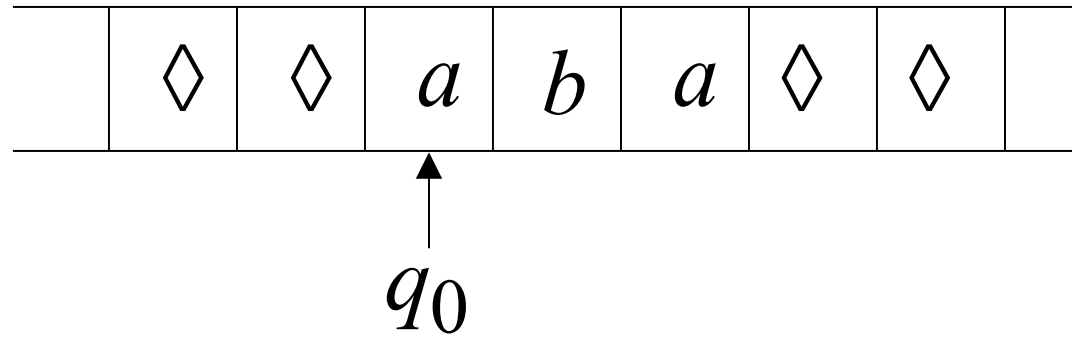
Time 1



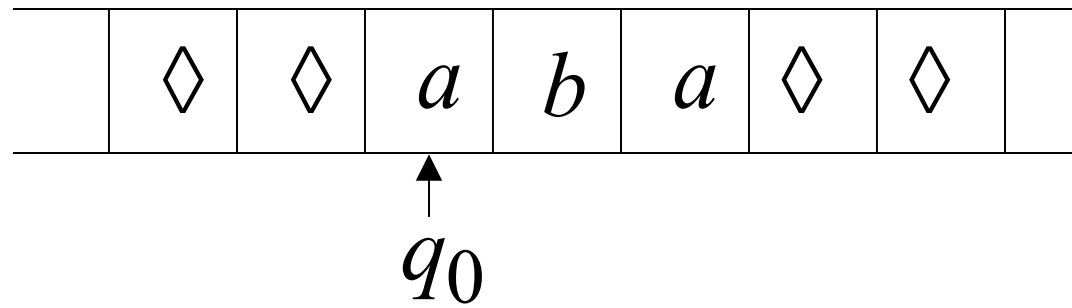
$b \rightarrow b, L$
 $a \rightarrow a, R$



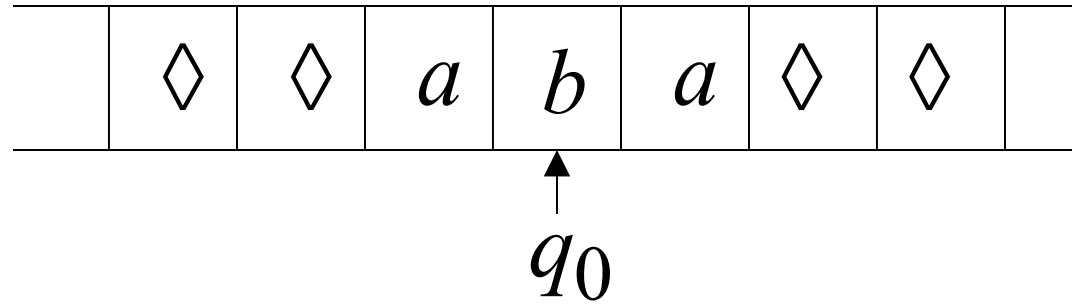
Time 2



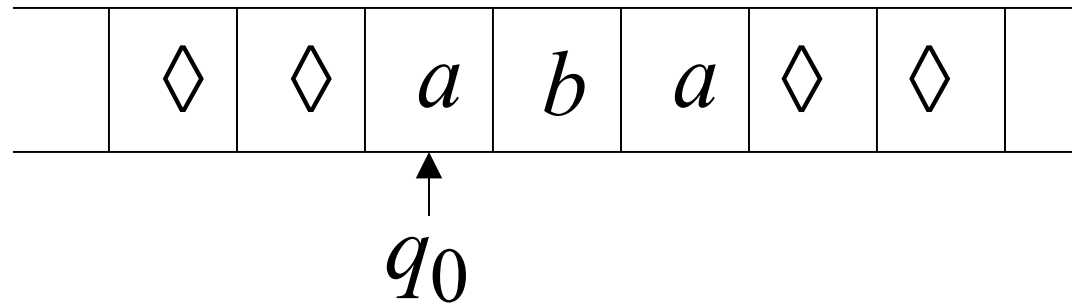
Time 2



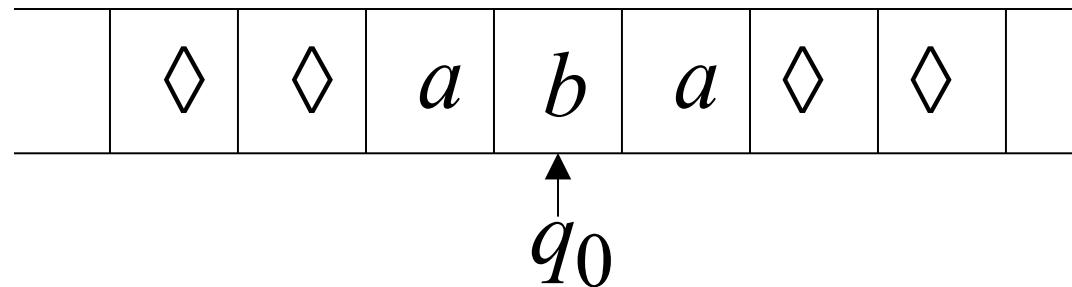
Time 3



Time 4



Time 5



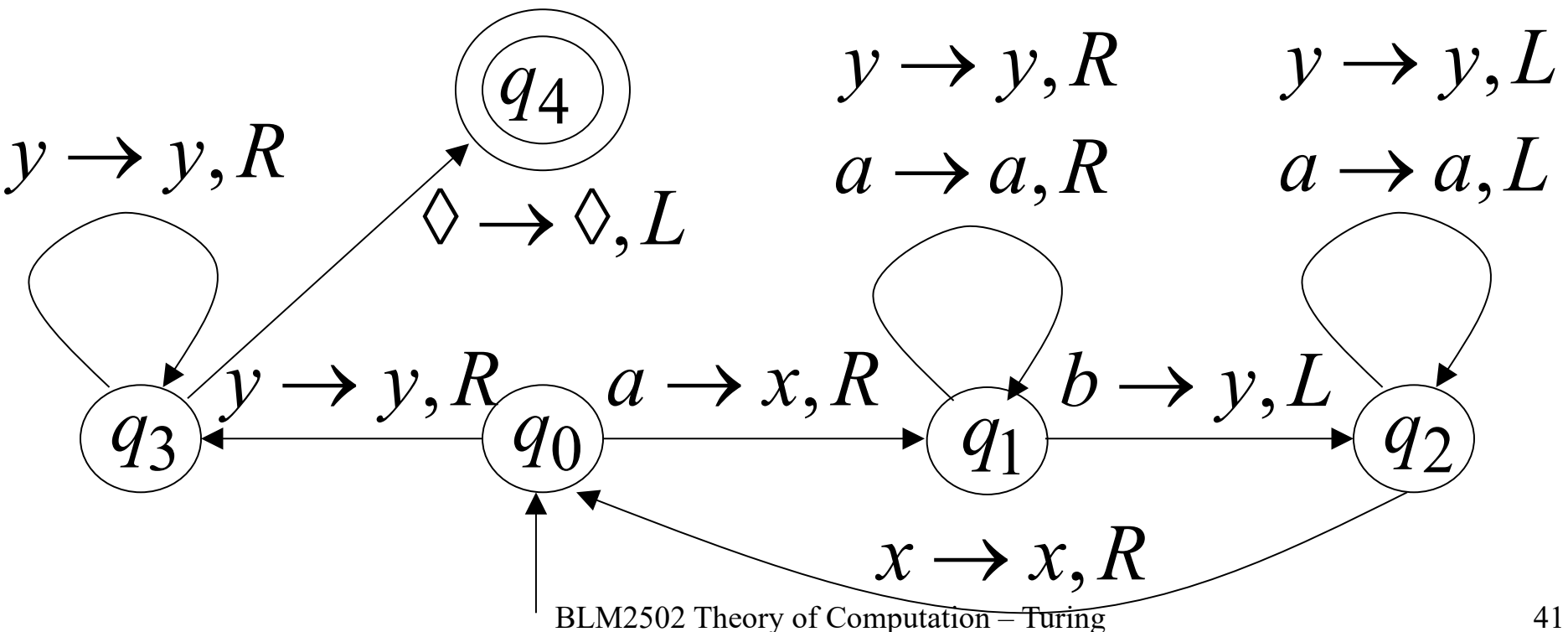
Infinite loop

Because of the **infinite loop**:

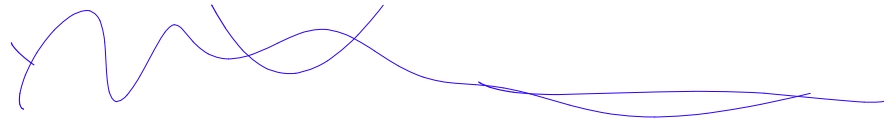
- The accepting state cannot be reached
- The machine never halts
- The input string is **rejected**

Another Turing Machine Example

Turing machine for the language $\{a^n b^n\}$
 $n \geq 1$



Basic Idea:



Match **a**'s with **b**'s:

Repeat:

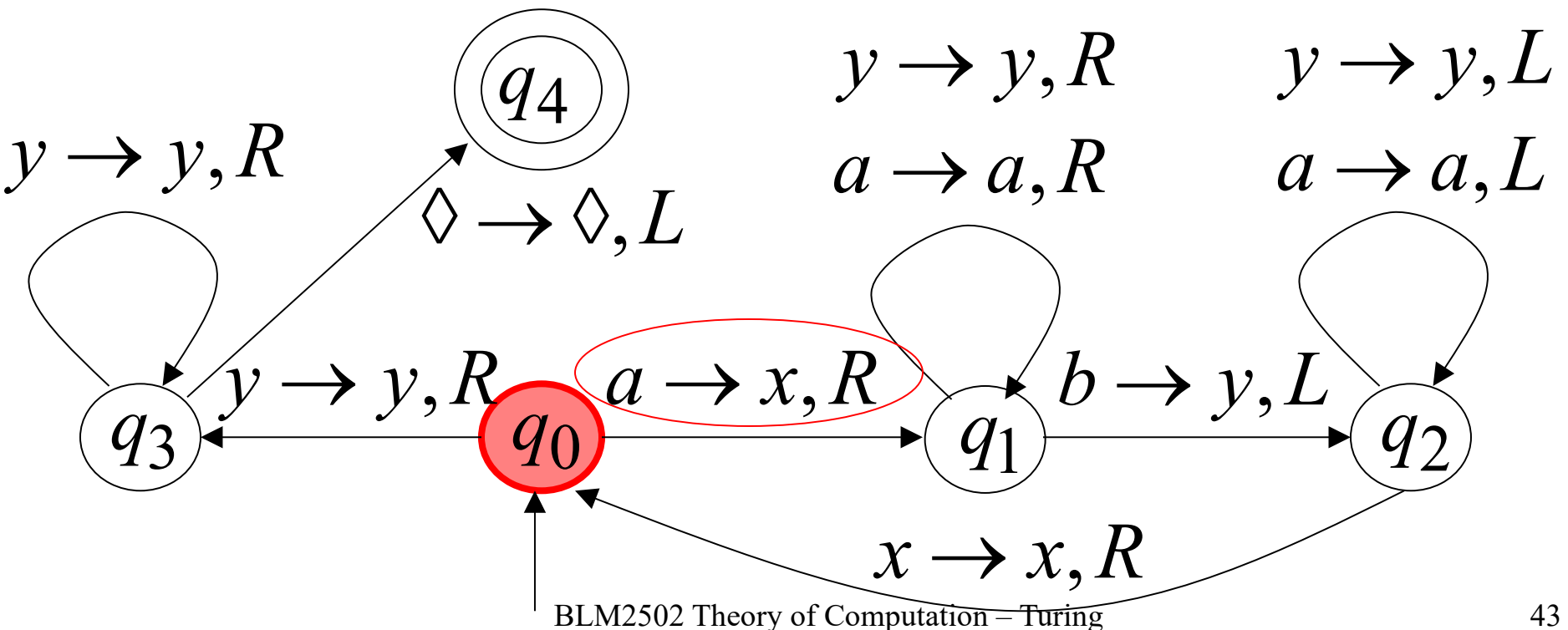
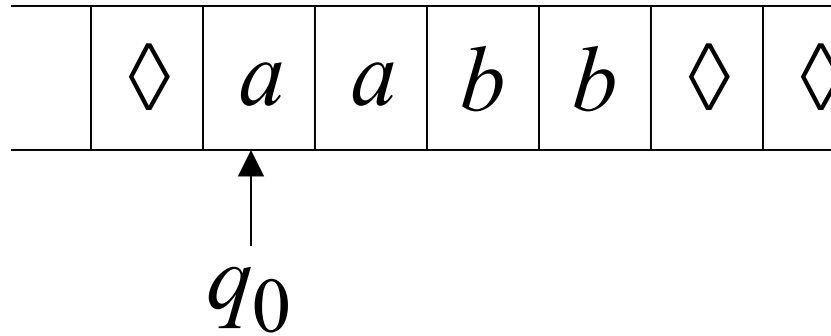
replace leftmost **a** with **x**

find leftmost **b** and replace it with **y**

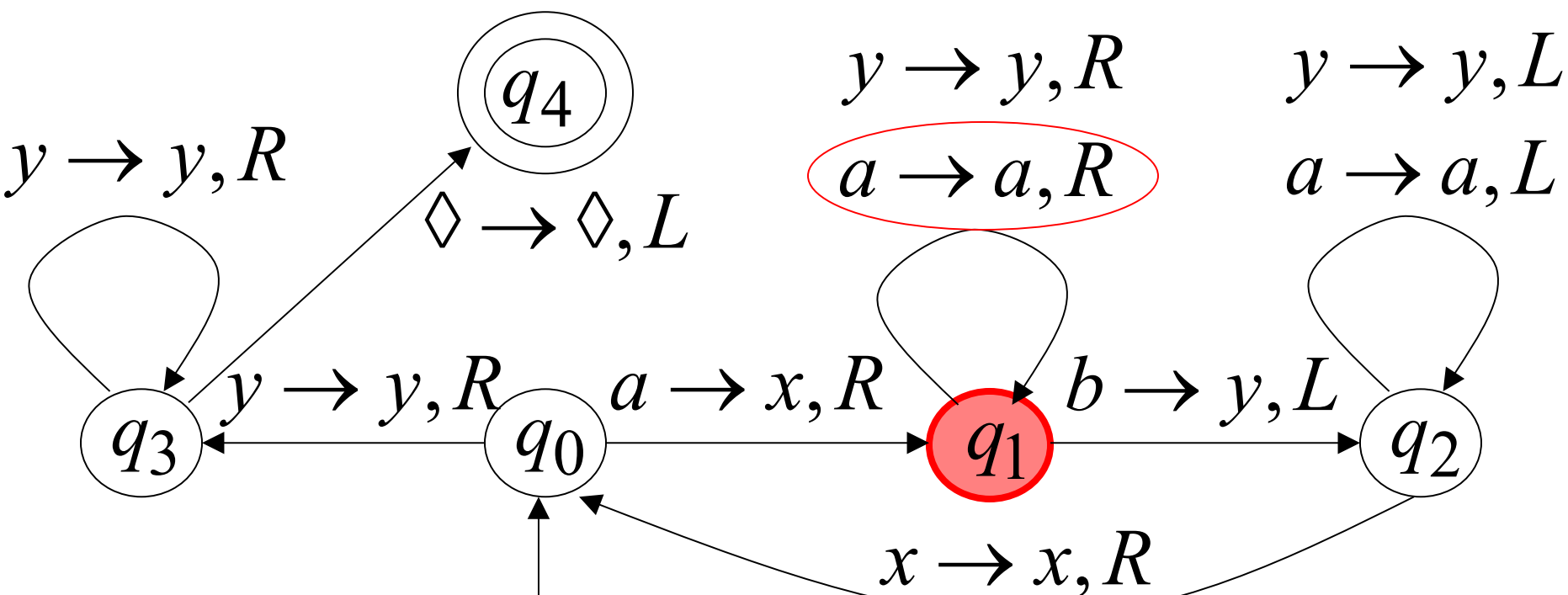
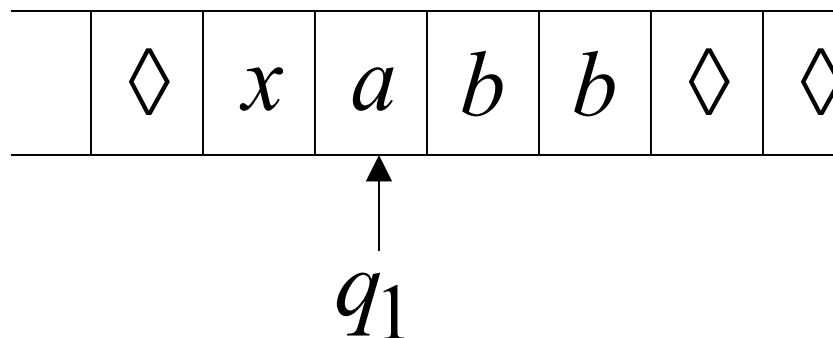
Until there are no more **a**'s or **b**'s

If there is a remaining **a** or **b** reject

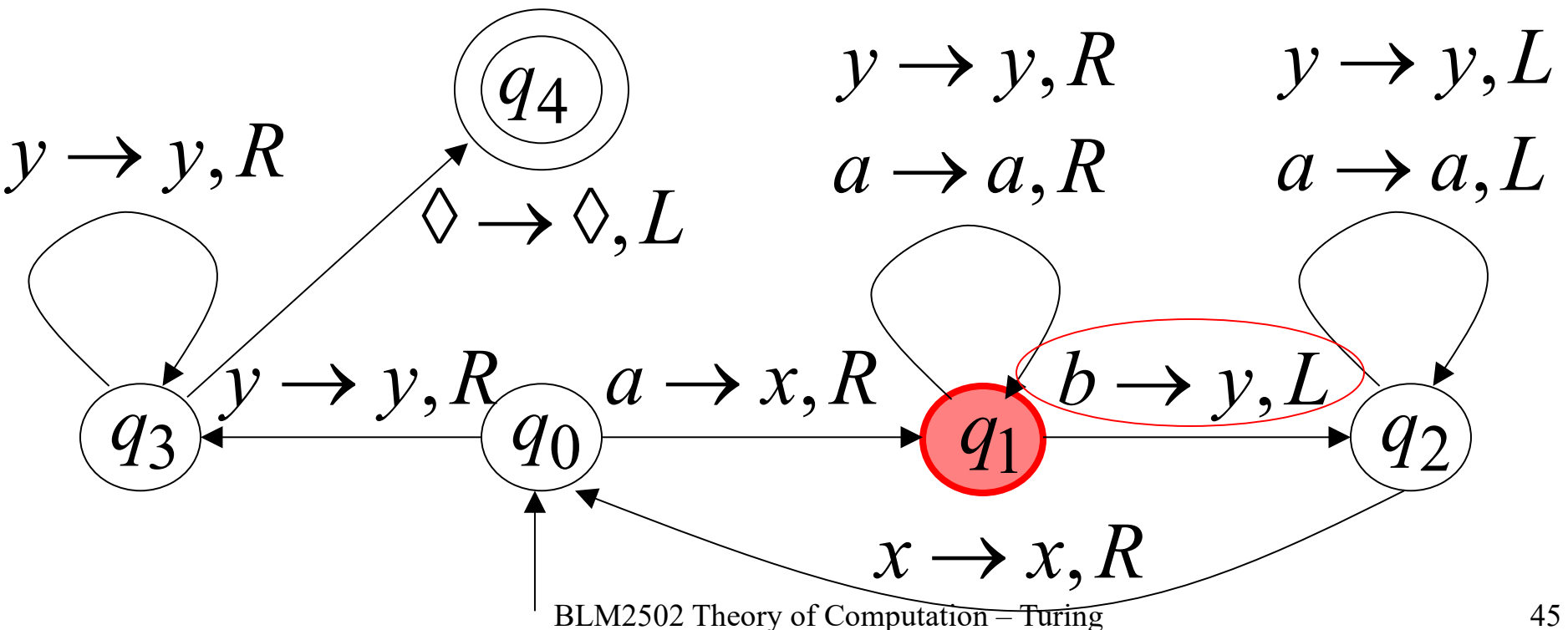
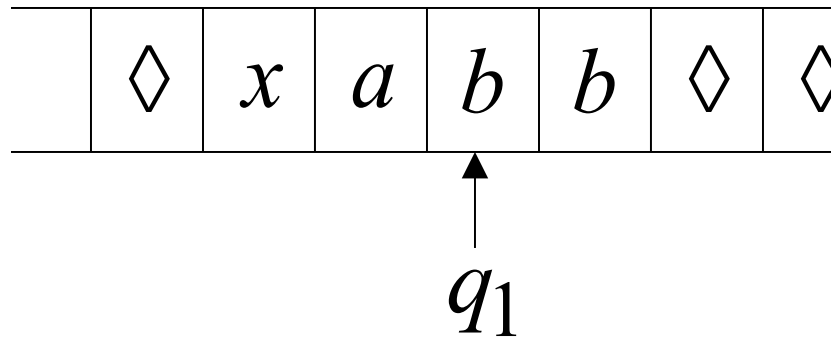
Time 0



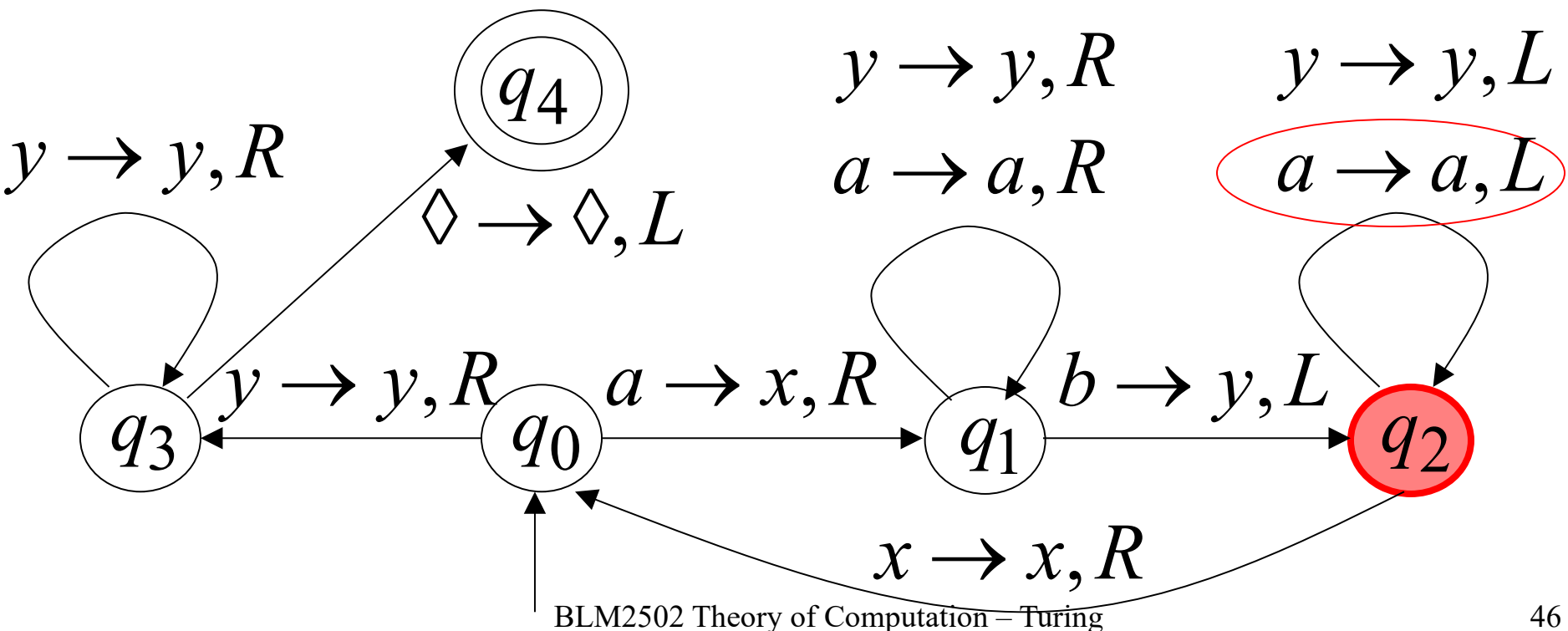
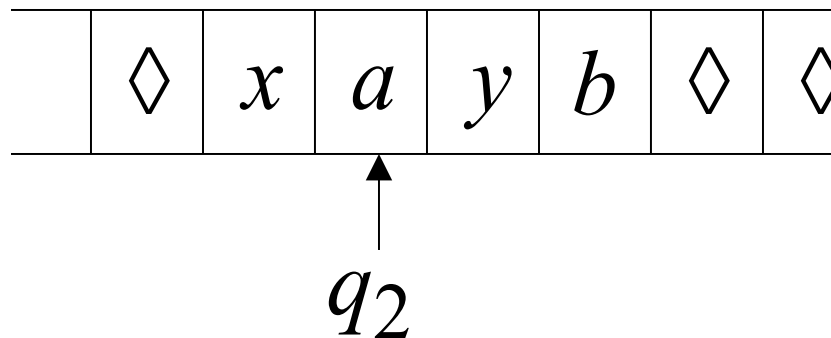
Time 1



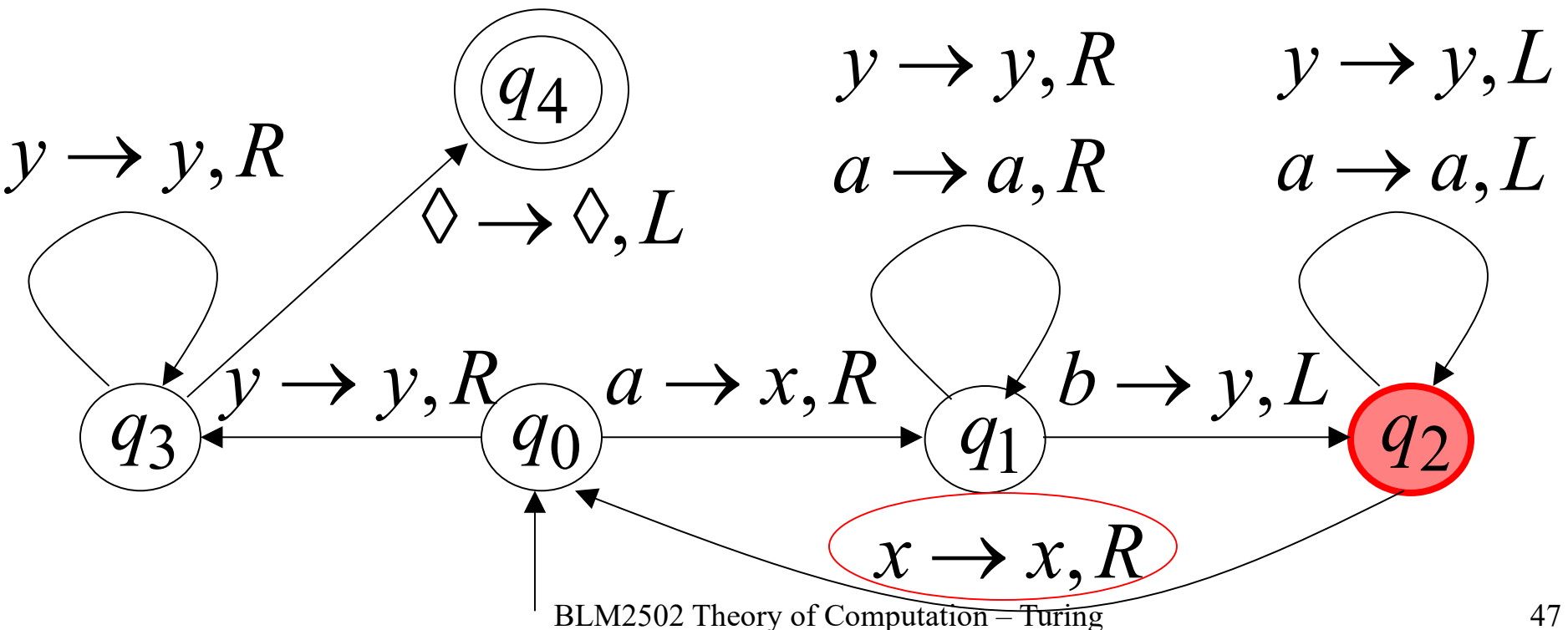
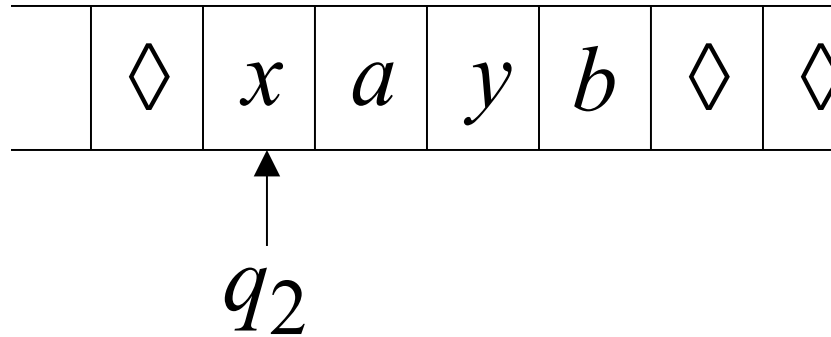
Time 2



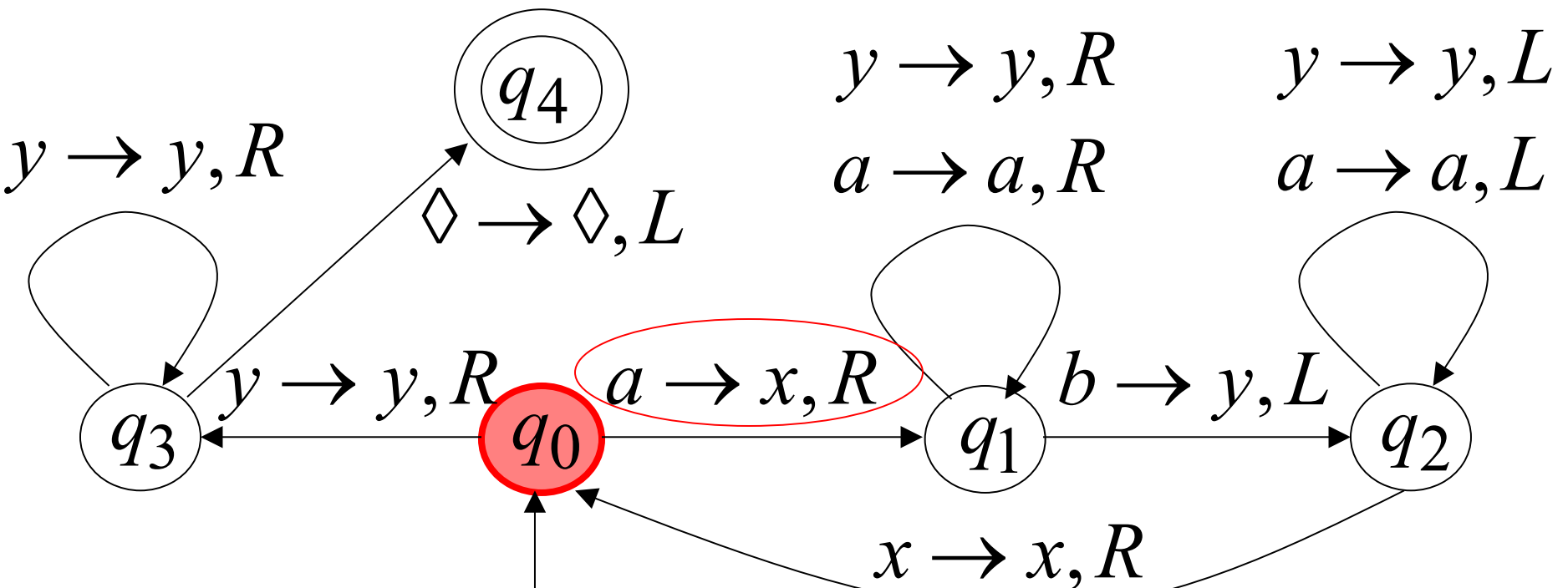
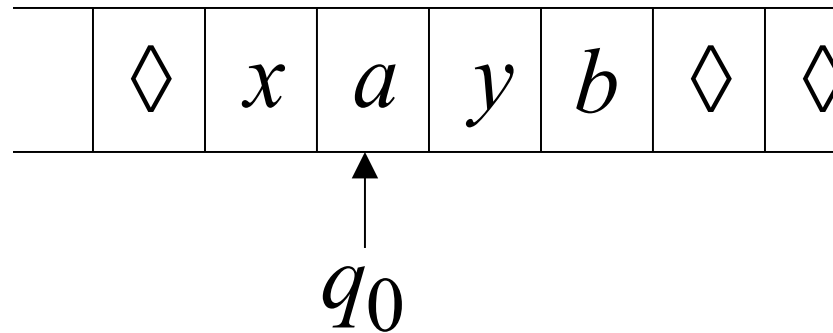
Time 3



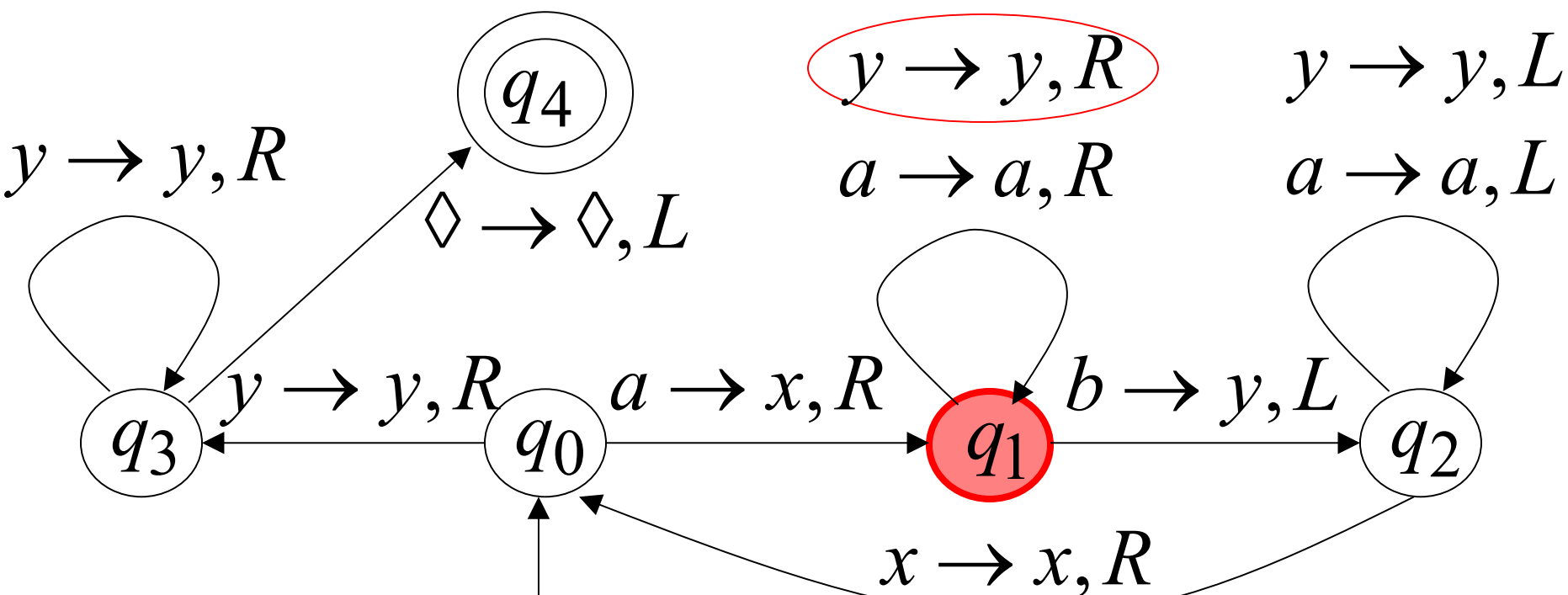
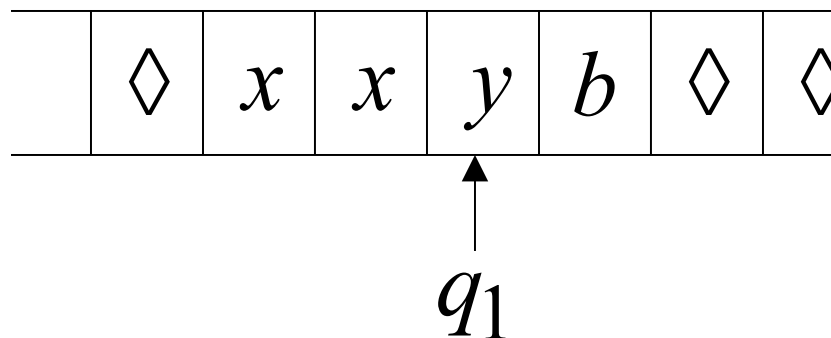
Time 4



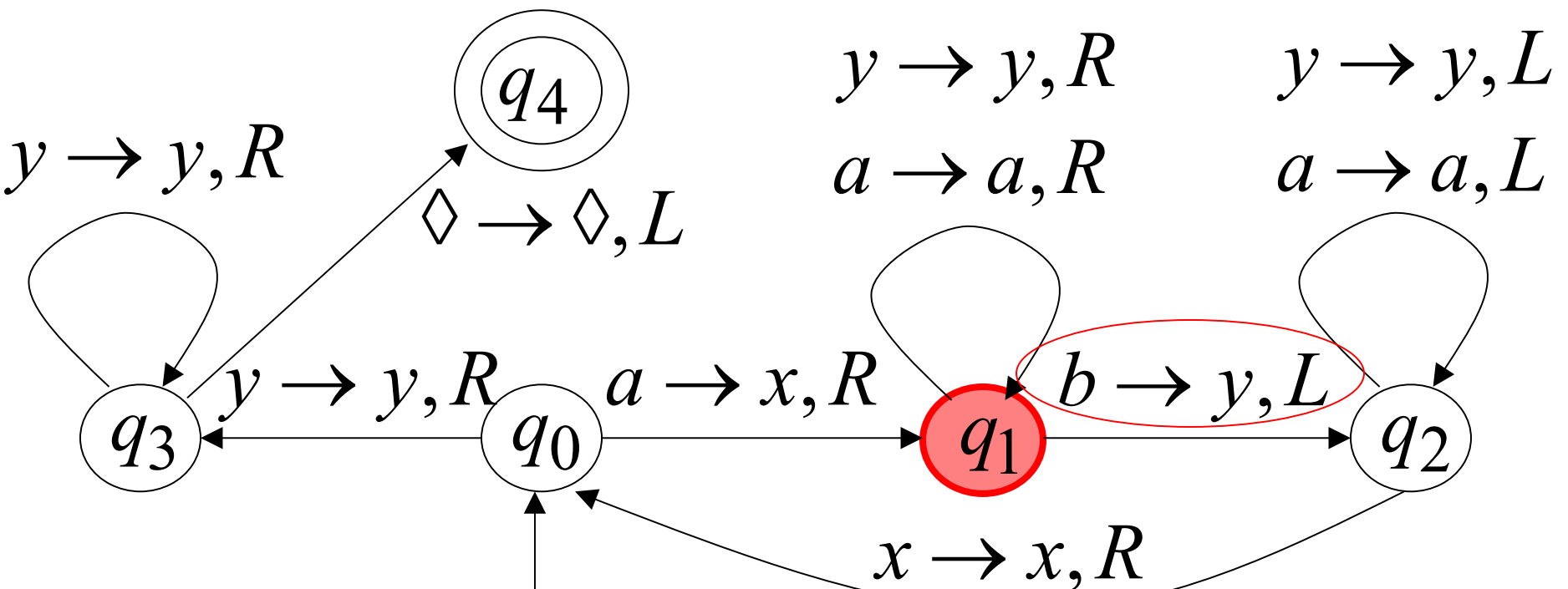
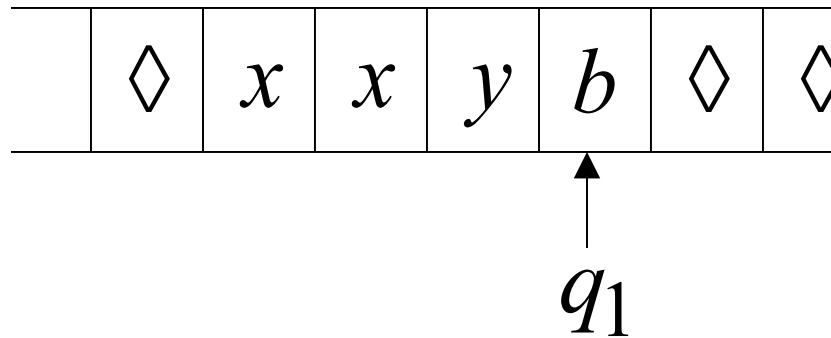
Time 5



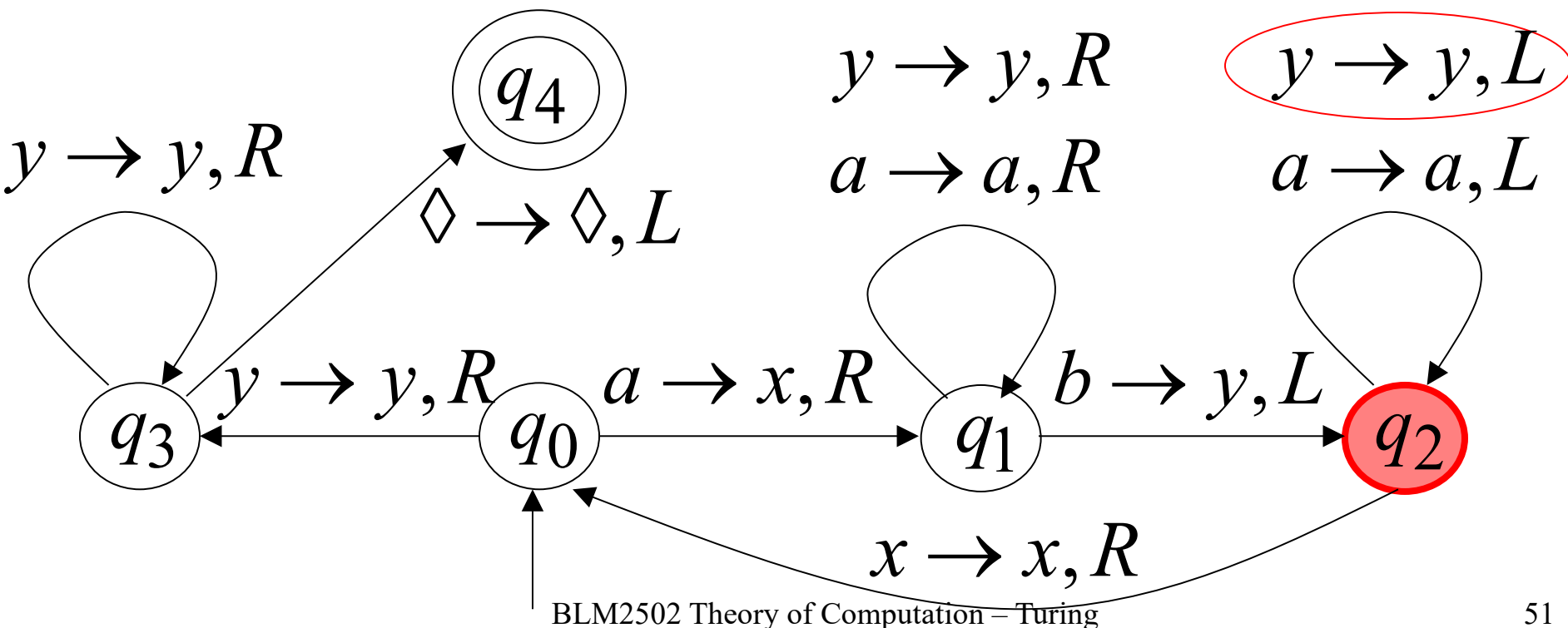
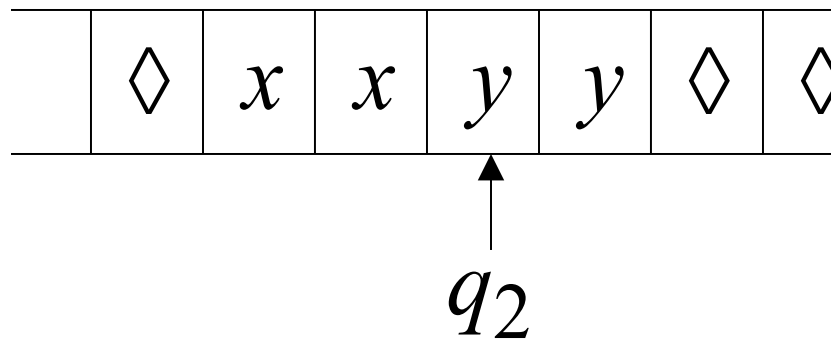
Time 6



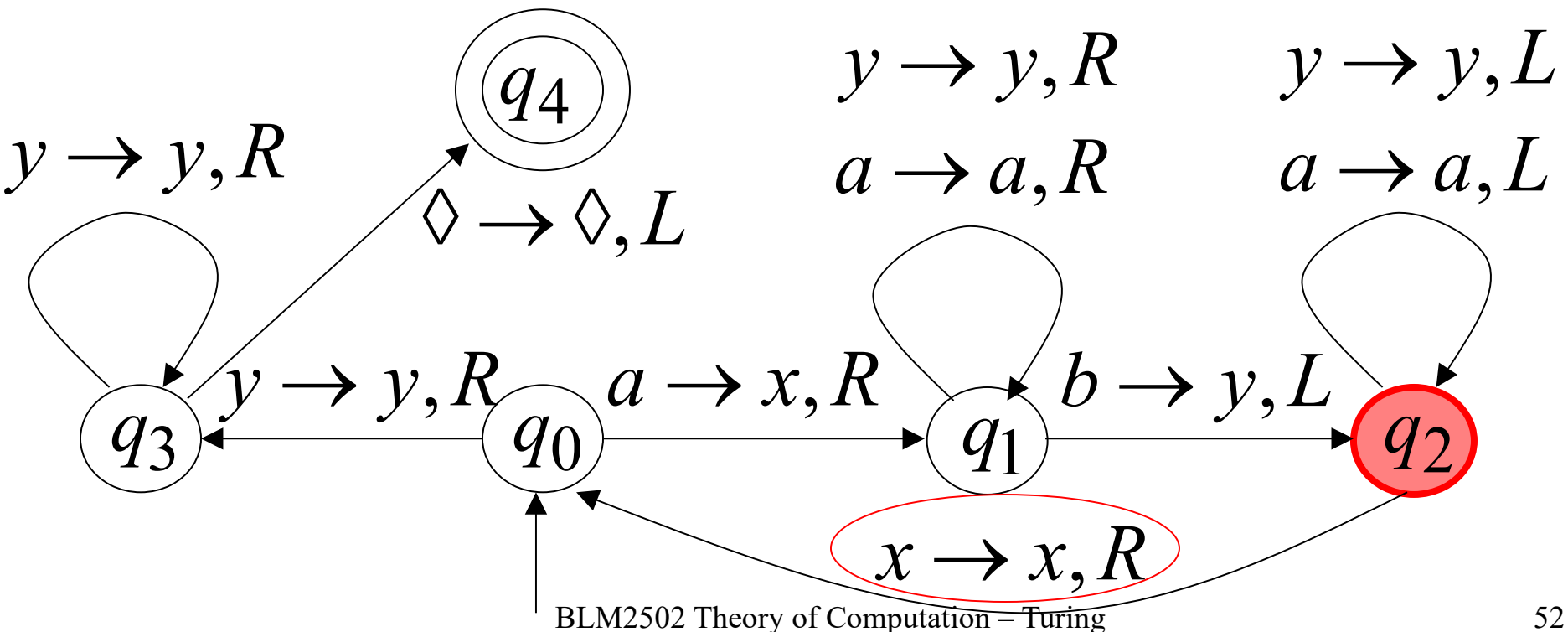
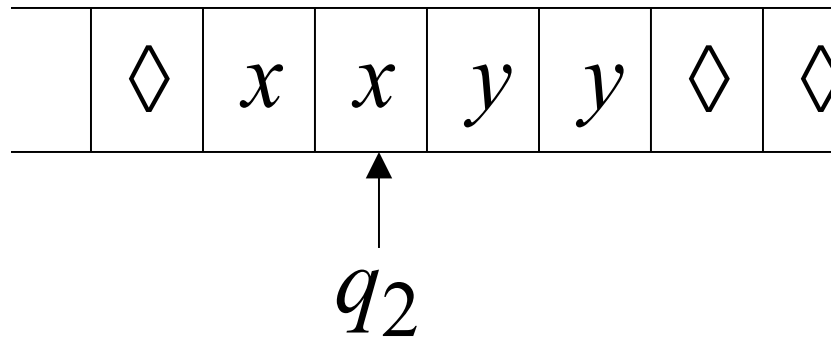
Time 7



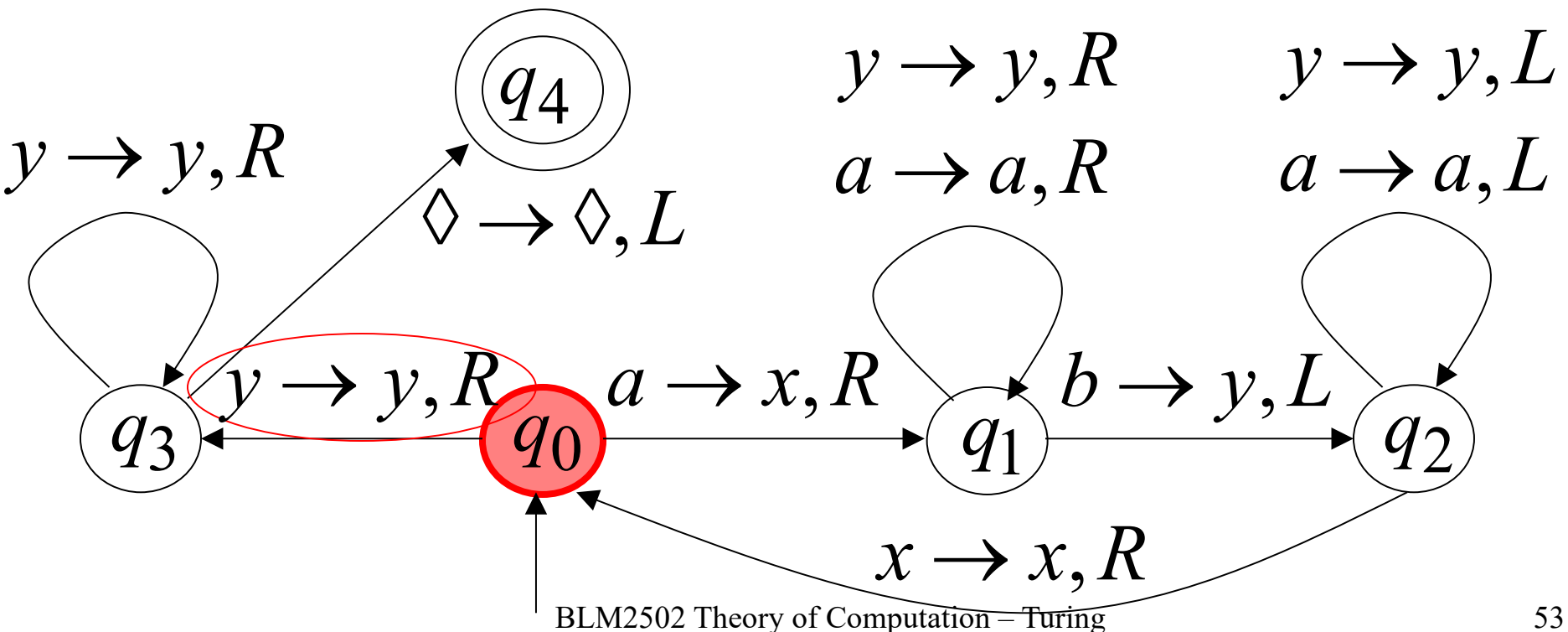
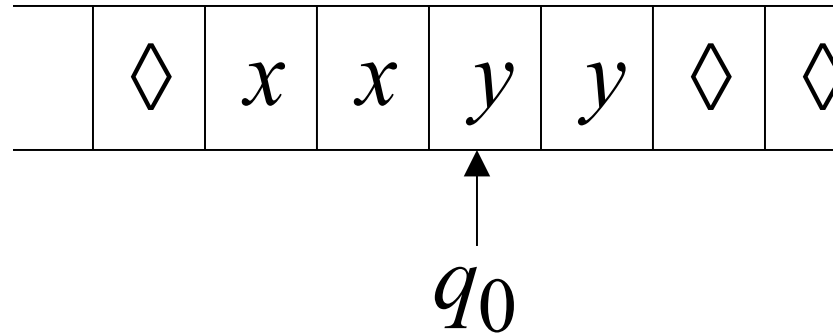
Time 8



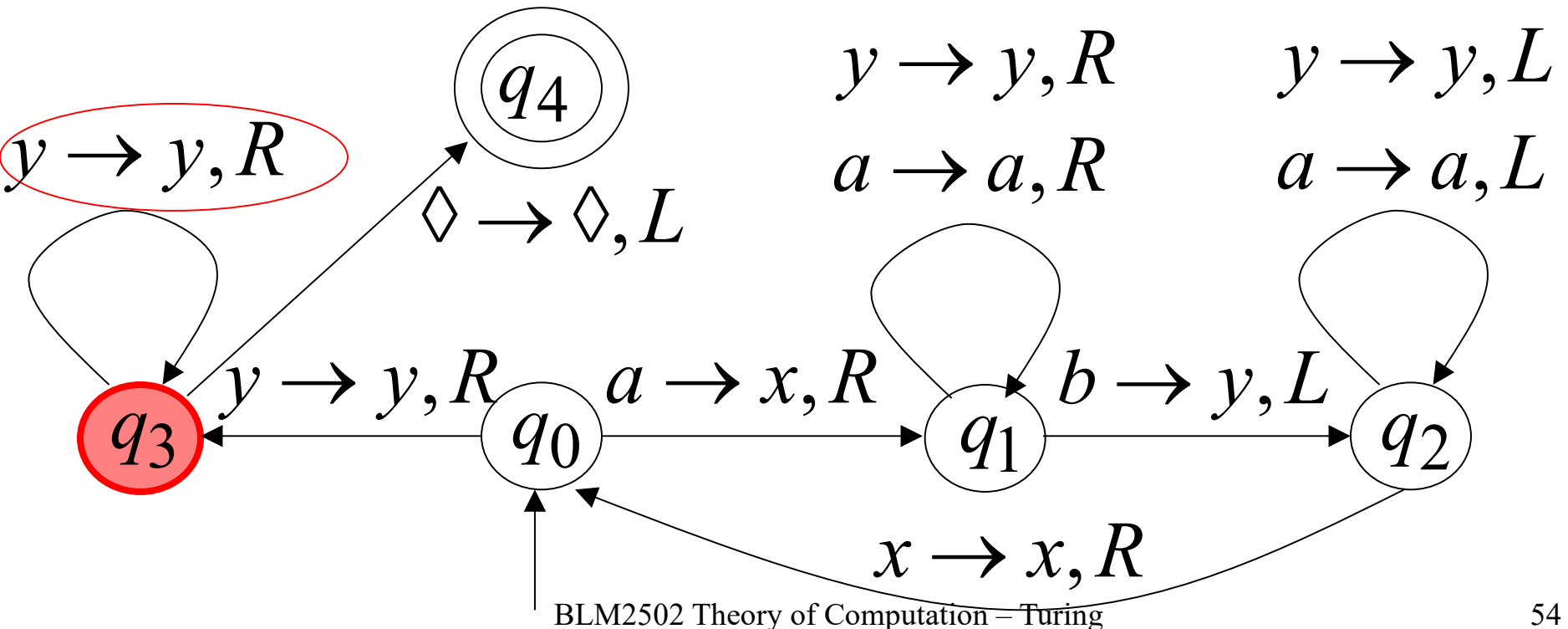
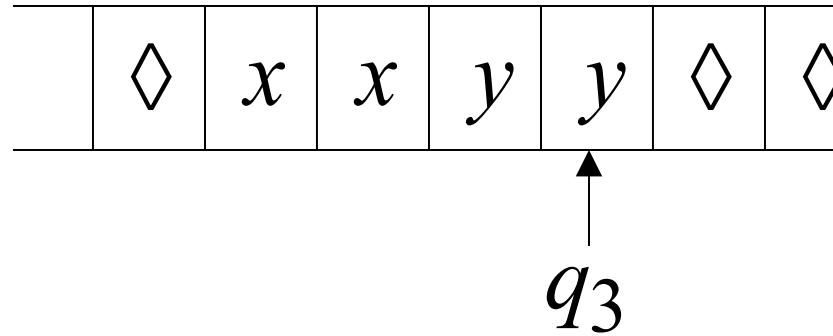
Time 9



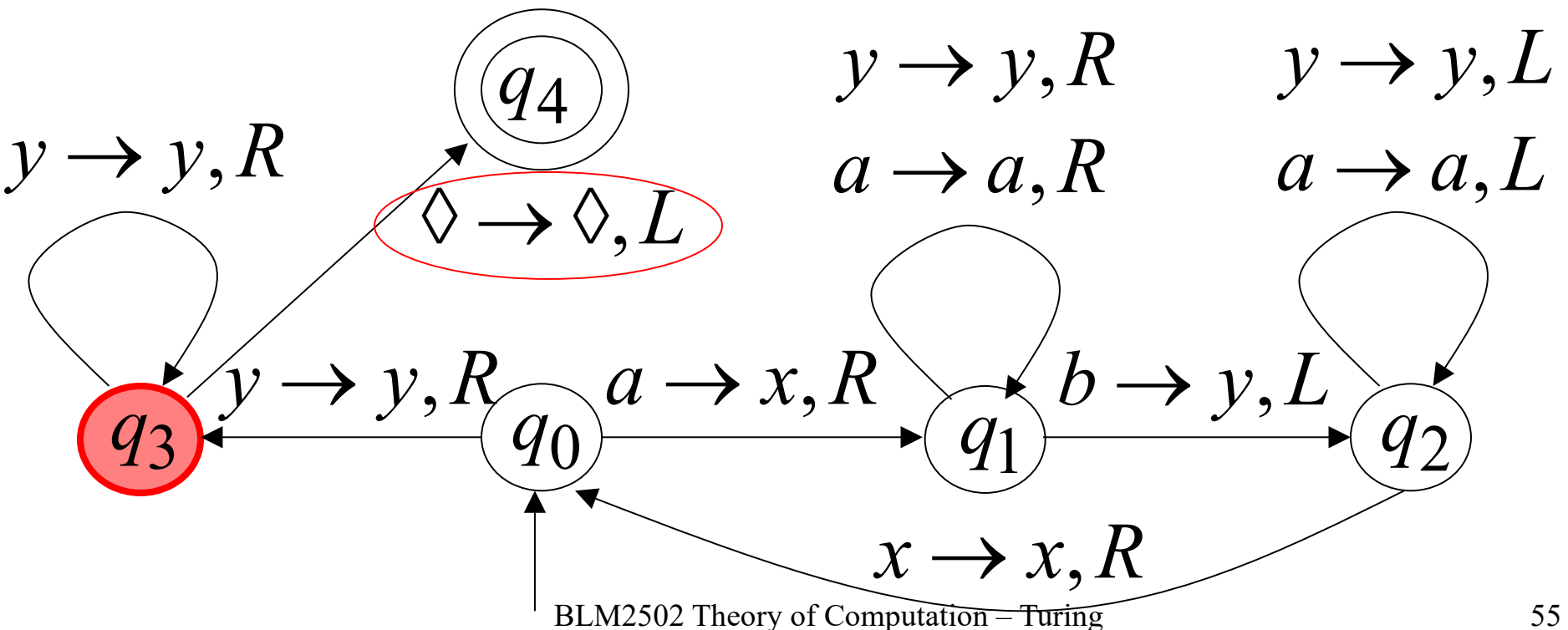
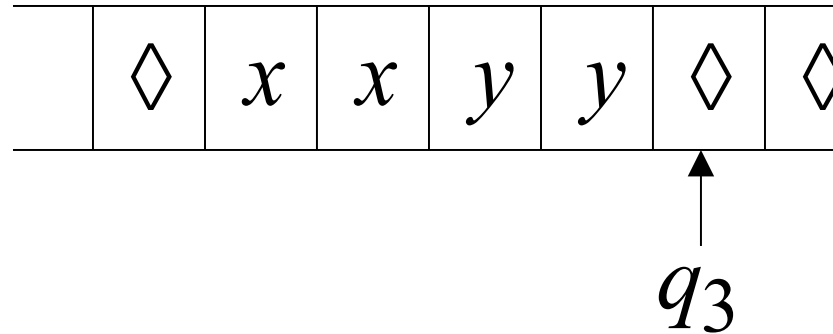
Time 10



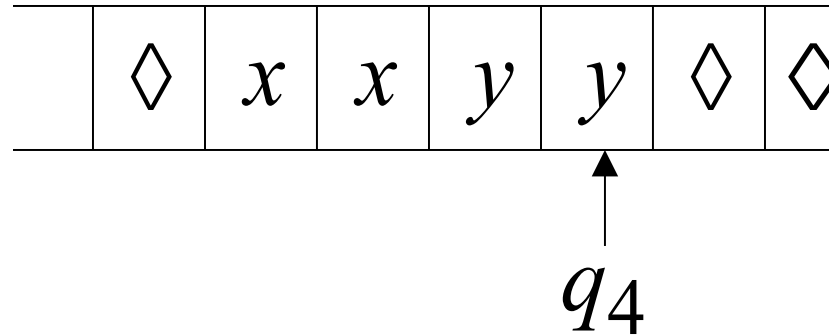
Time 11



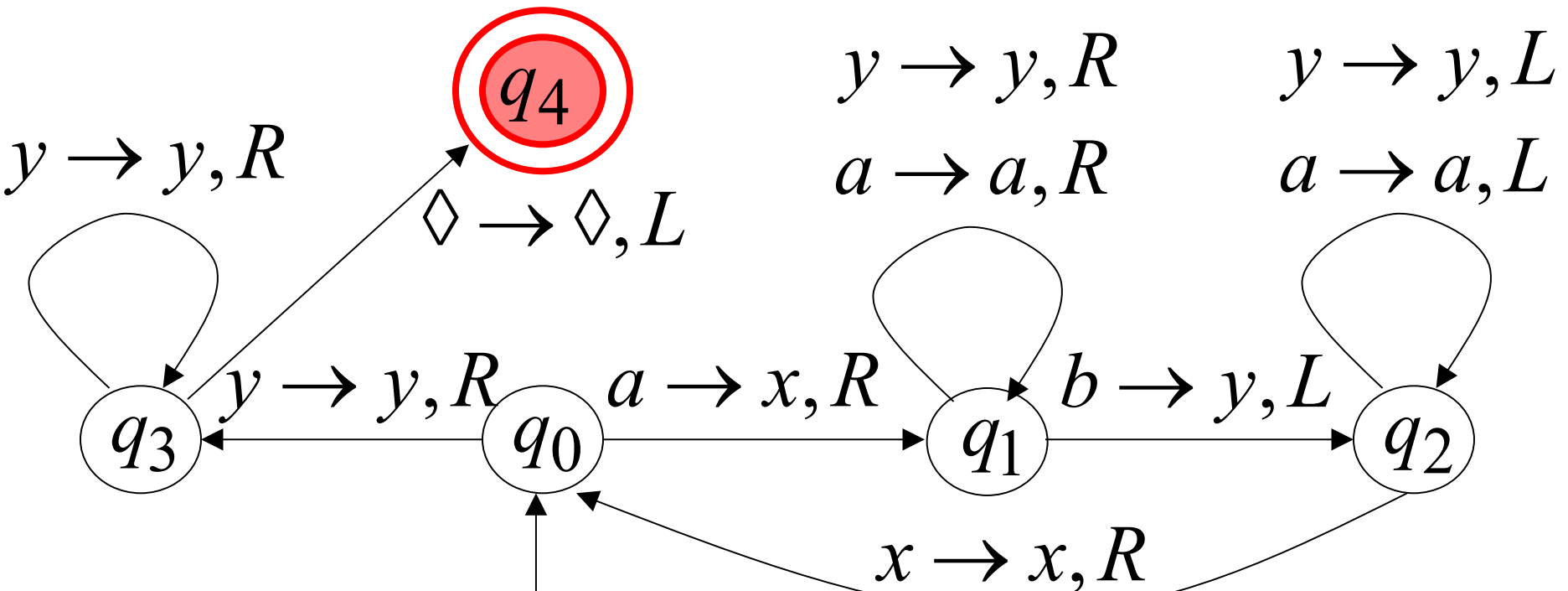
Time 12



Time 13



Halt & Accept



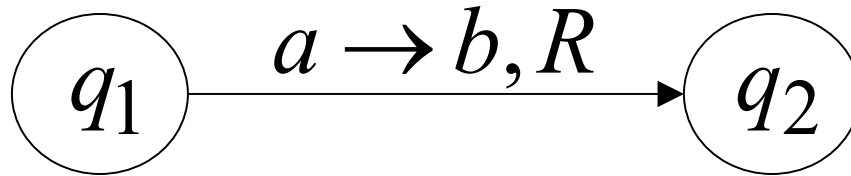
Observation:

If we modify the
machine for the language $\{a^n b^n\}$

we can easily construct
a machine for the language $\{a^n b^n c^n\}$

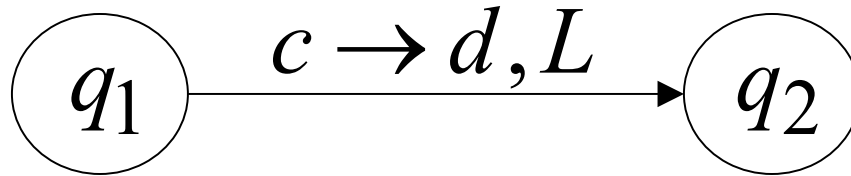
Formal Definitions for Turing Machines

Transition Function



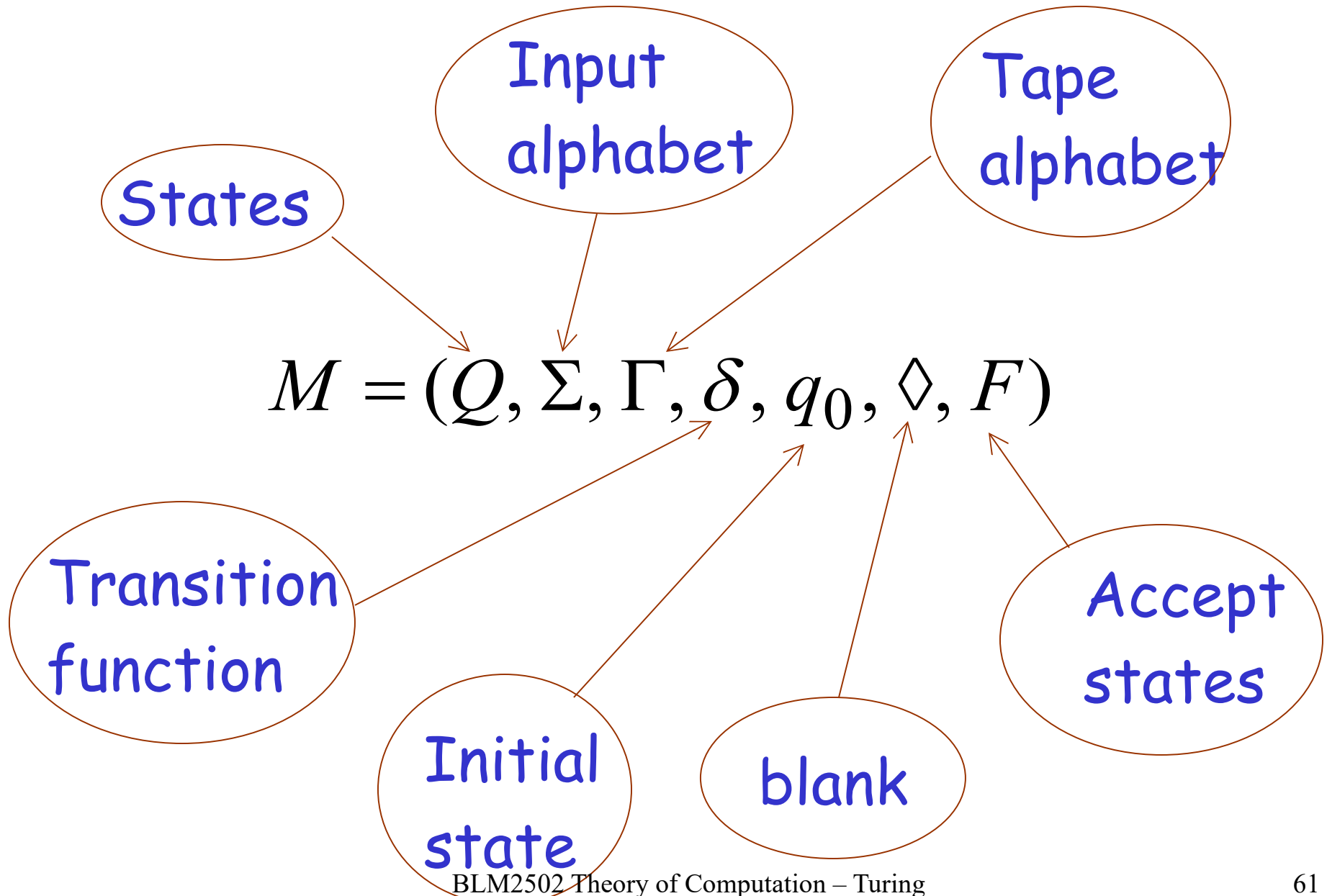
$$\delta(q_1, a) = (q_2, b, R)$$

Transition Function

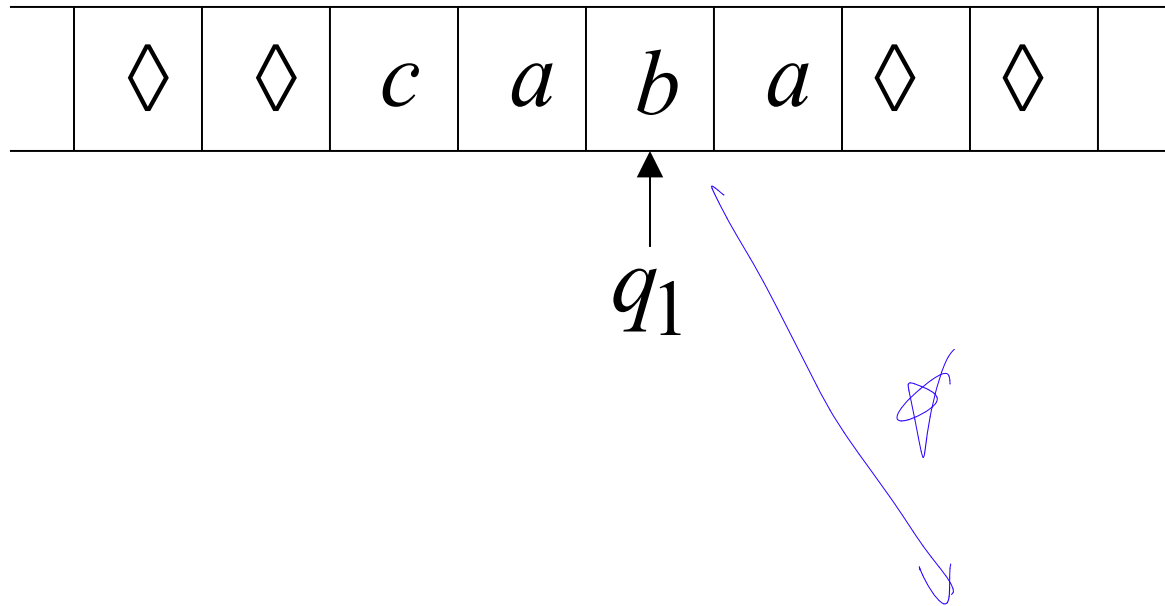


$$\delta(q_1, c) = (q_2, d, L)$$

Turing Machine:

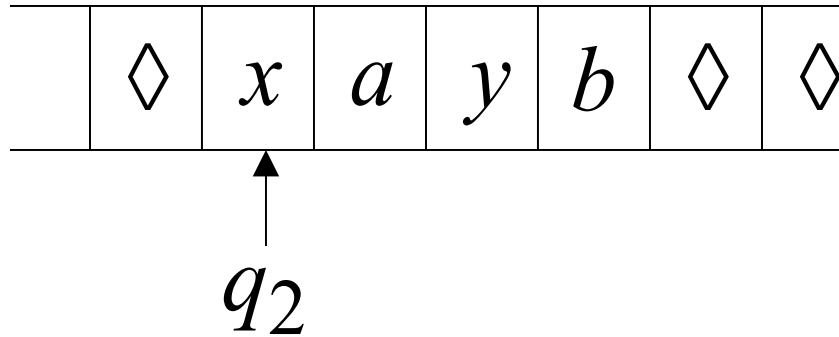


Configuration

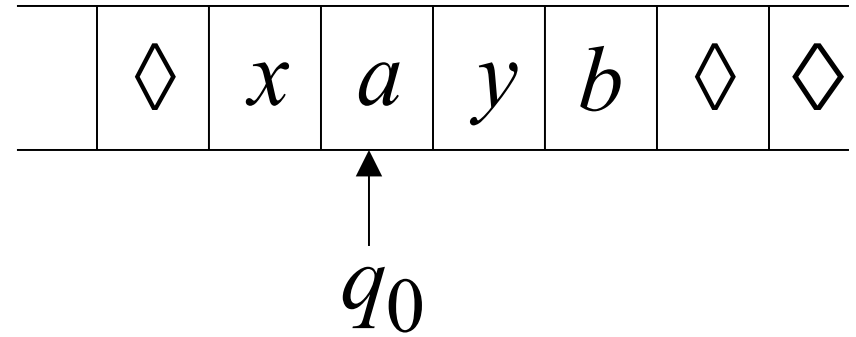


Instantaneous description: $ca q_1 ba$

Time 4



Time 5

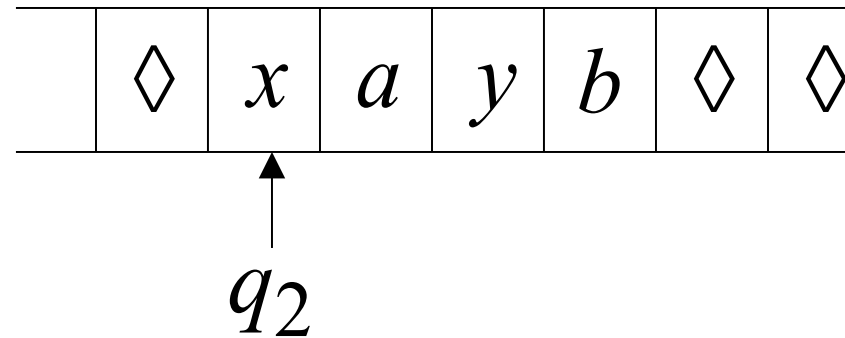


A Move:

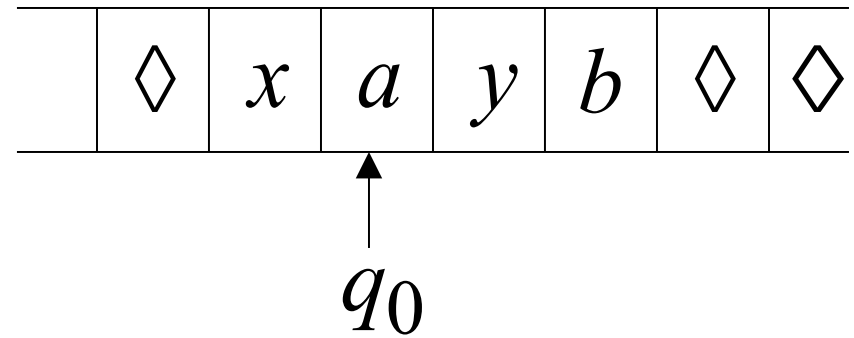
$$q_2 xayb \succ x q_0 ayb$$

(yields in one move)

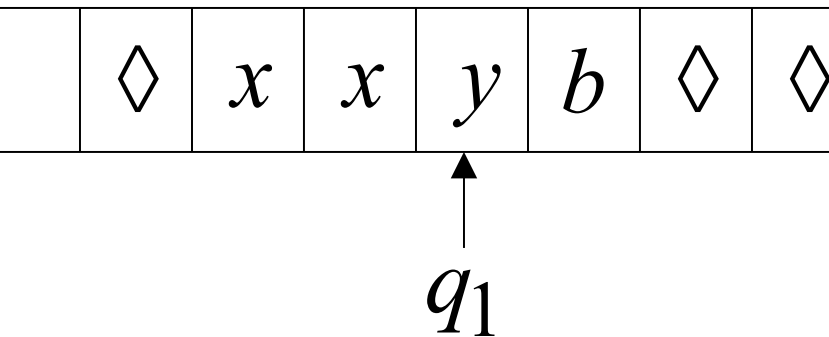
Time 4



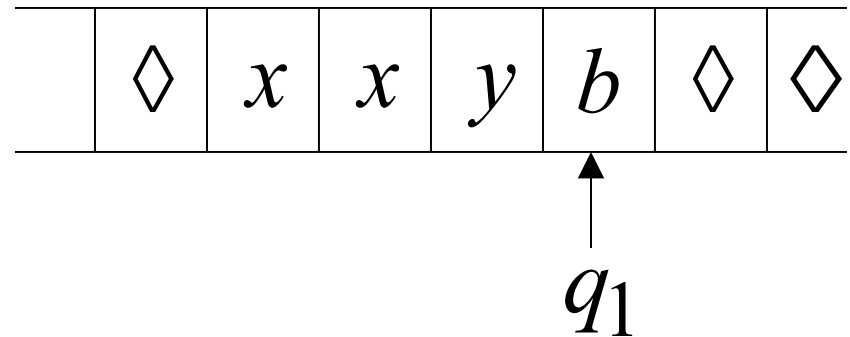
Time 5



Time 6



Time 7



A computation

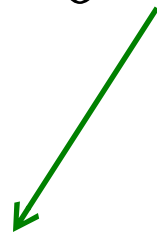
$q_2 \ x a y b \succ x \ q_0 \ a y b \succ x x \ q_1 \ y b \succ x x y \ q_1 \ b$

$$q_2 xayb \succ x q_0 ayb \succ xx q_1 yb \succ xxy q_1 b$$

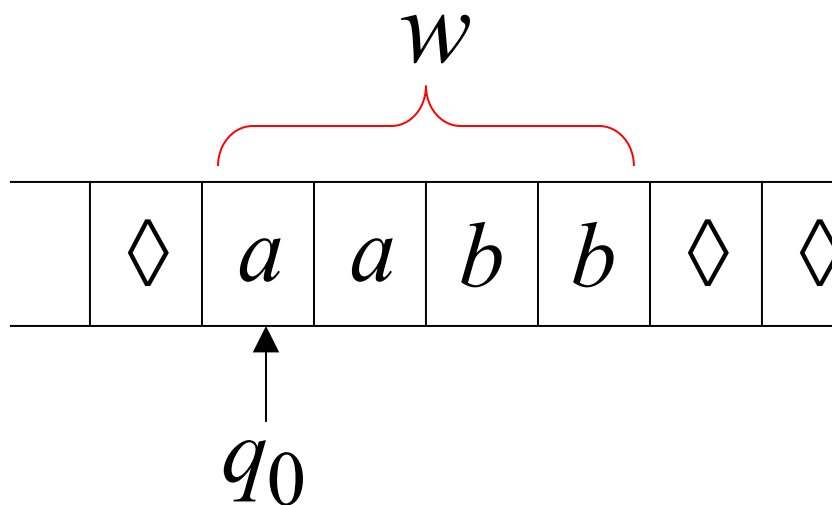
Equivalent notation:

$$q_2 xayb \overset{*}{\succ} xxy q_1 b$$

Initial configuration: $q_0 w$



Input string



The Accepted Language

For any Turing Machine M

$$L(M) = \{w : q_0 \xrightarrow{*} x_1 q_f x_2\}$$

Initial state

Accept state

If a language L is accepted
by a Turing machine M
then we say that L is:

- Turing Recognizable

Other names used:

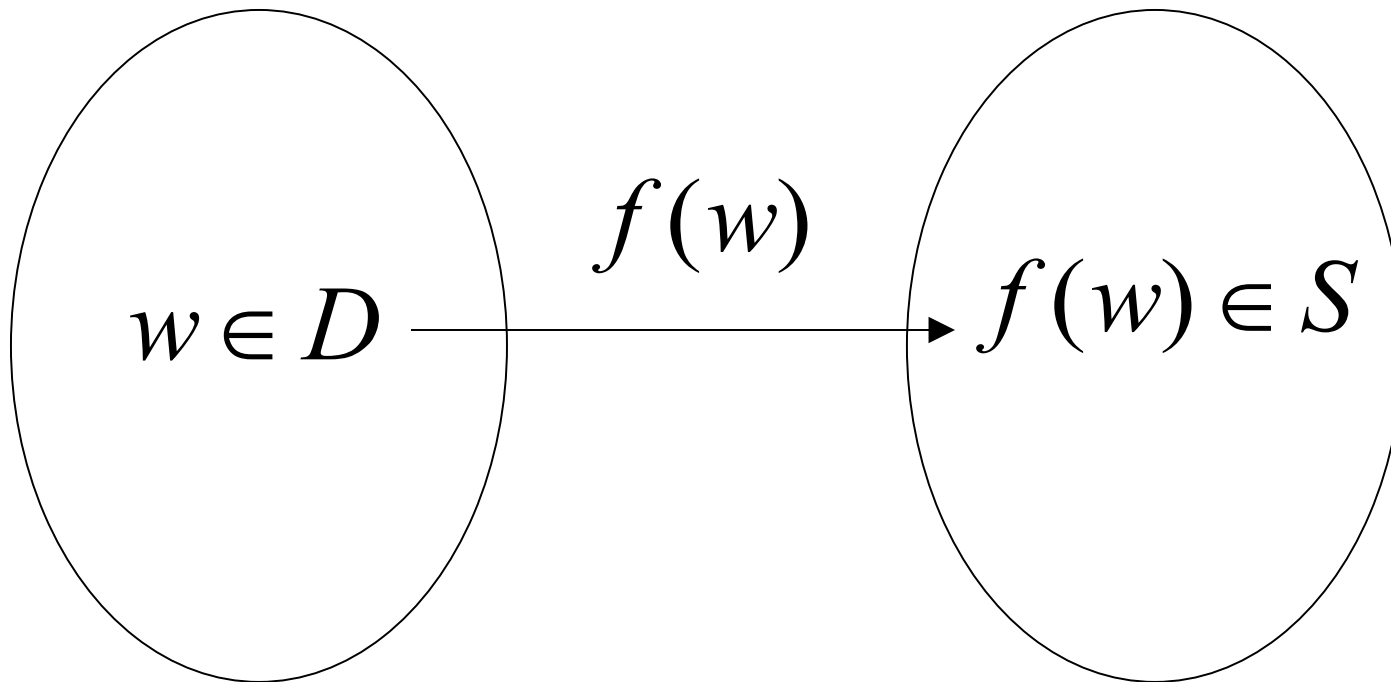
- Turing Acceptable
- Recursively Enumerable

Computing Functions with Turing Machines

A function $f(w)$ has:

Domain: D

Result Region: S



A function may have many parameters:

Example: Addition function

$$f(x, y) = x + y$$

Integer Domain

Decimal: 5

Binary: 101

Unary: 11111

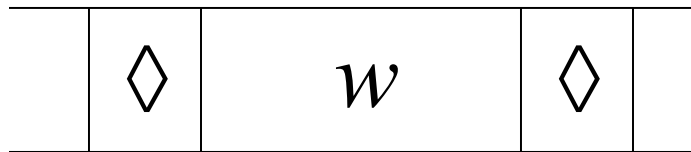
We prefer **unary** representation:

easier to manipulate with Turing machines

Definition:

A function f is computable if
there is a Turing Machine M such that:

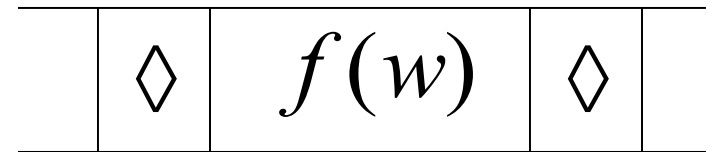
Initial configuration



q_0

initial state

Final configuration



q_f

accept state

For all $w \in D$ Domain

In other words:

A function f is computable if
there is a Turing Machine M such that:

$$q_0 w \xrightarrow{*} q_f f(w)$$

Initial
Configuration

Final
Configuration

For all $w \in D$ Domain

Example

The function $f(x, y) = x + y$ is computable

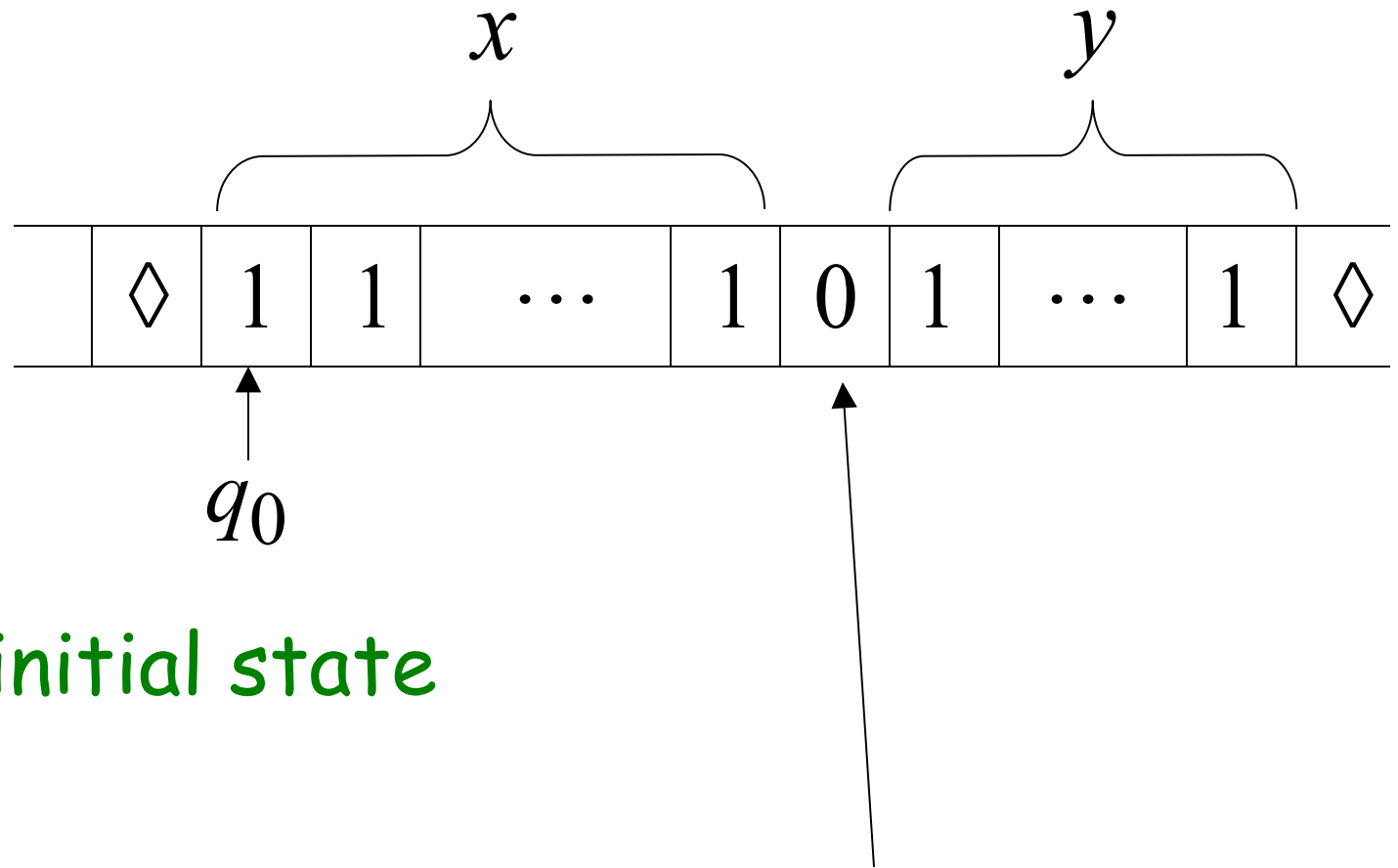
x, y are integers

Turing Machine:

Input string: $x0y$ unary

Output string: $xy0$ unary

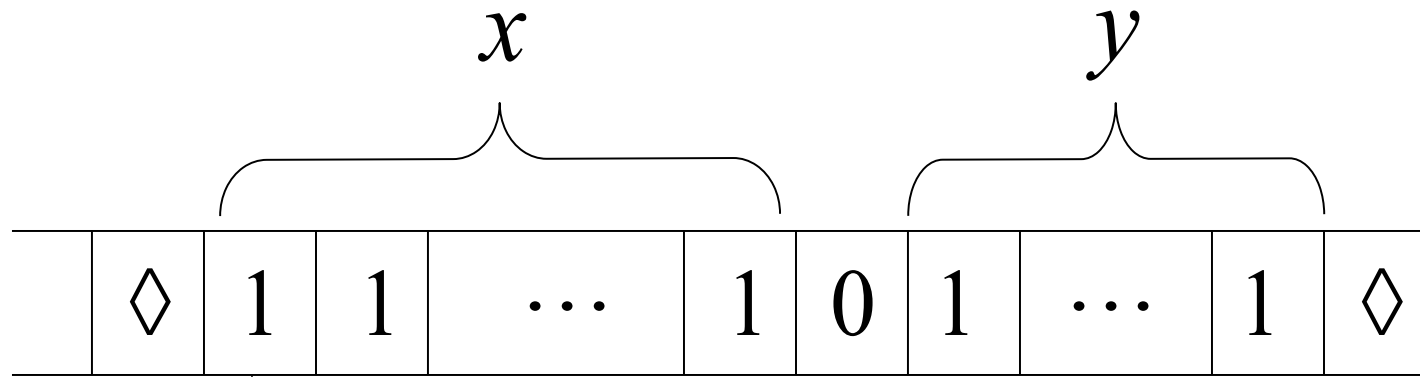
Start



initial state

The 0 is the delimiter that separates the two numbers

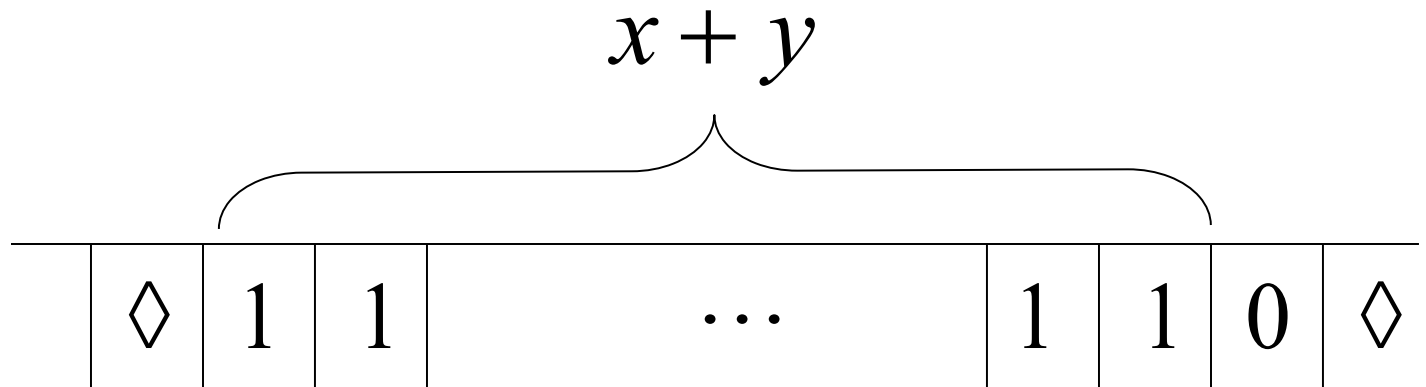
Start



q_0 initial state

*Aliz özensiz
yapılabilir mi?*

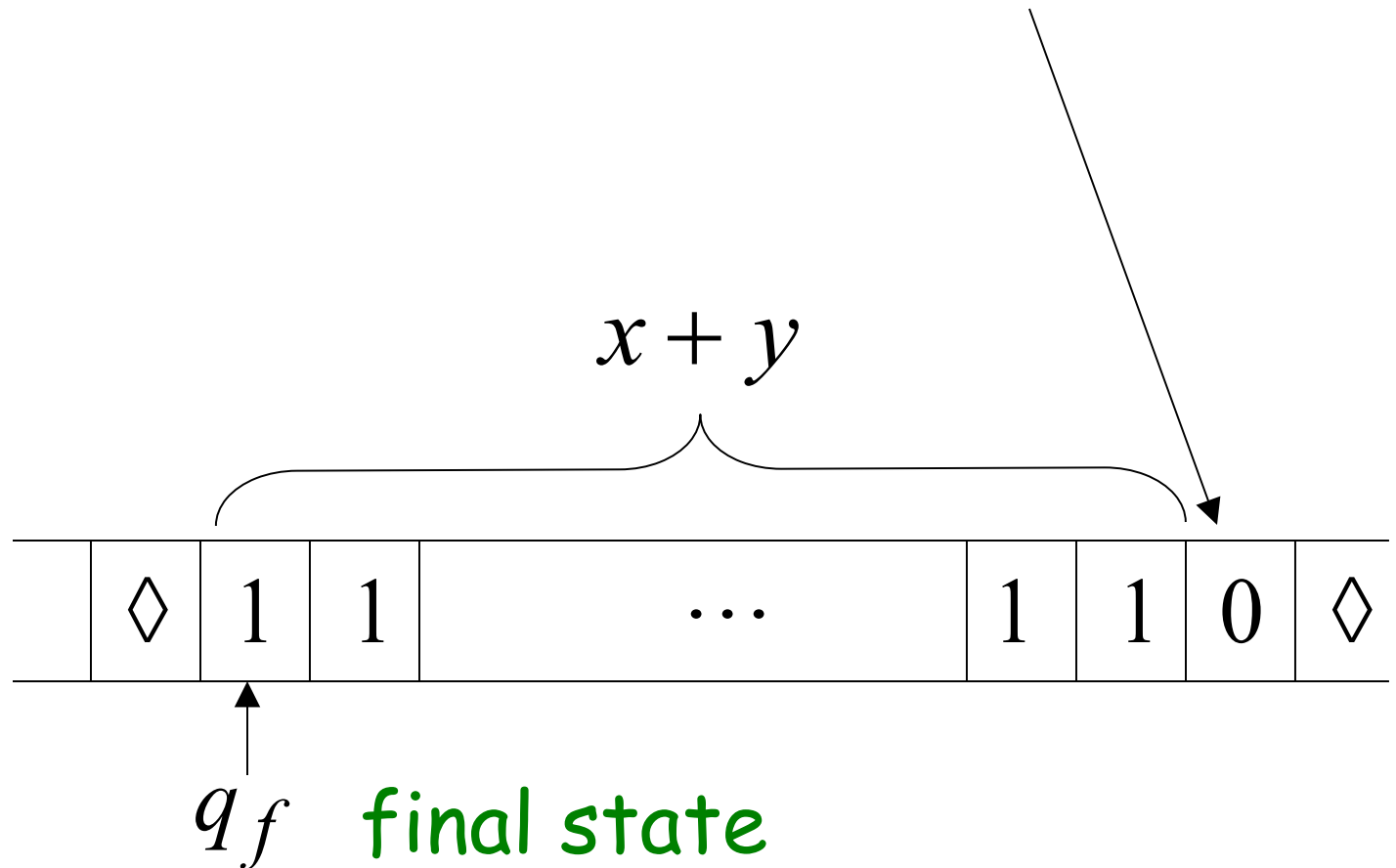
Finish



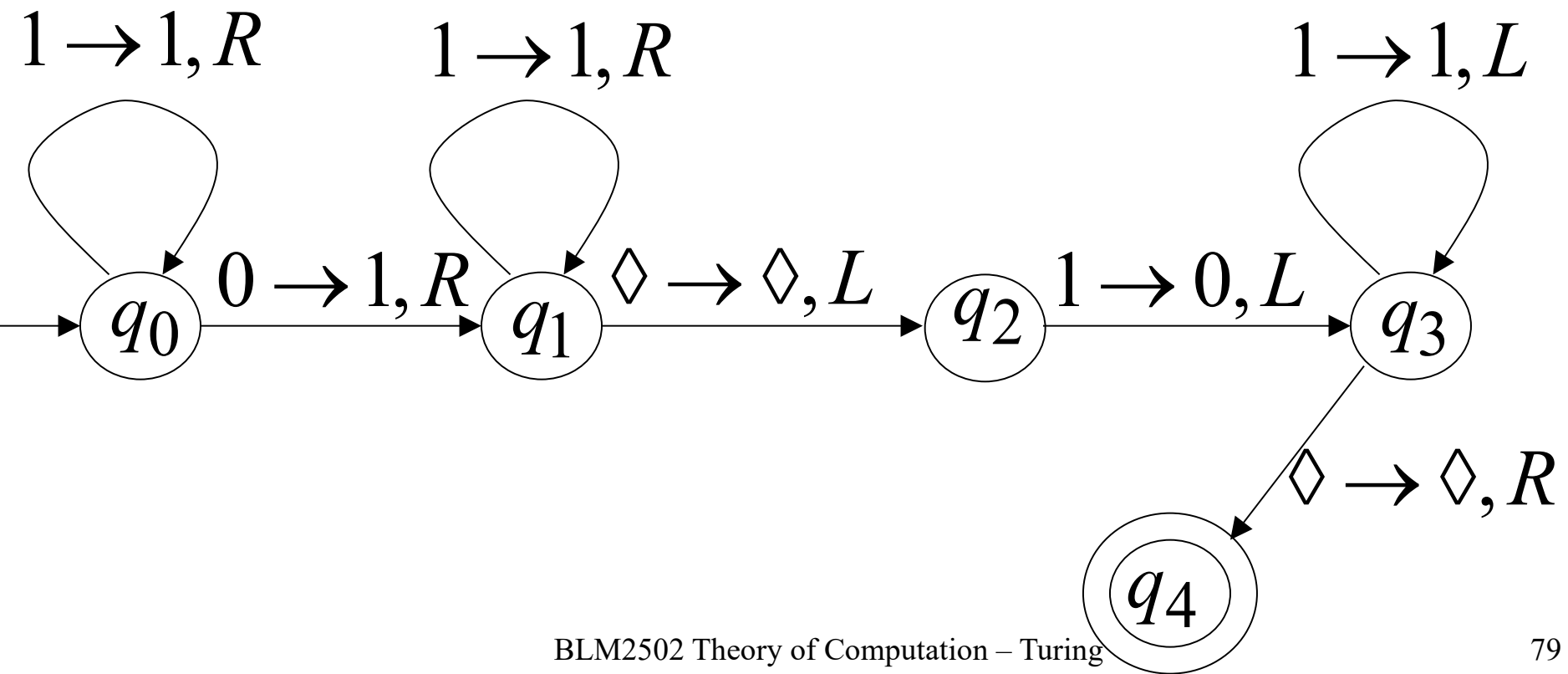
q_f final state

The 0 here helps when we use the result for other operations

Finish



Turing machine for function $f(x, y) = x + y$

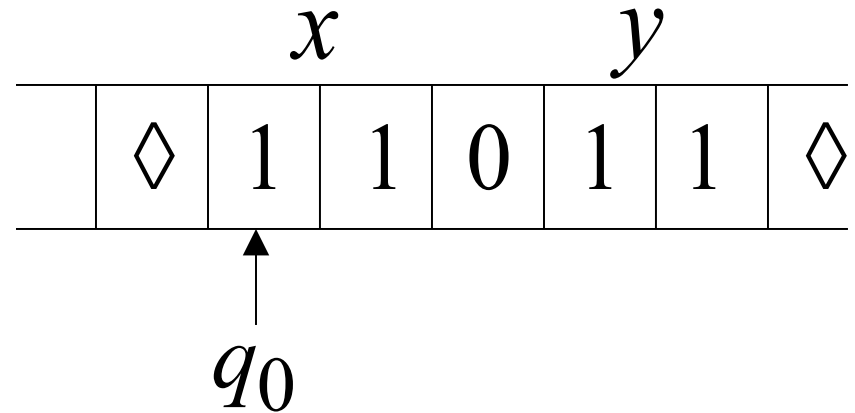


Execution Example:

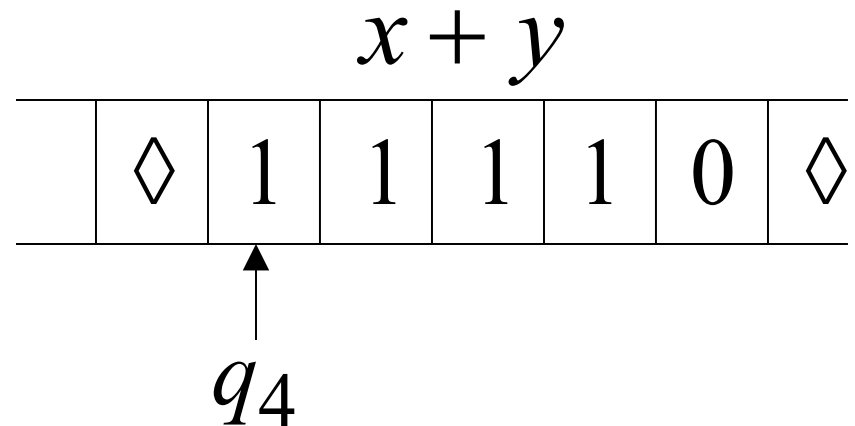
$$x = 11 \quad (=2)$$

$$y = 11 \quad (=2)$$

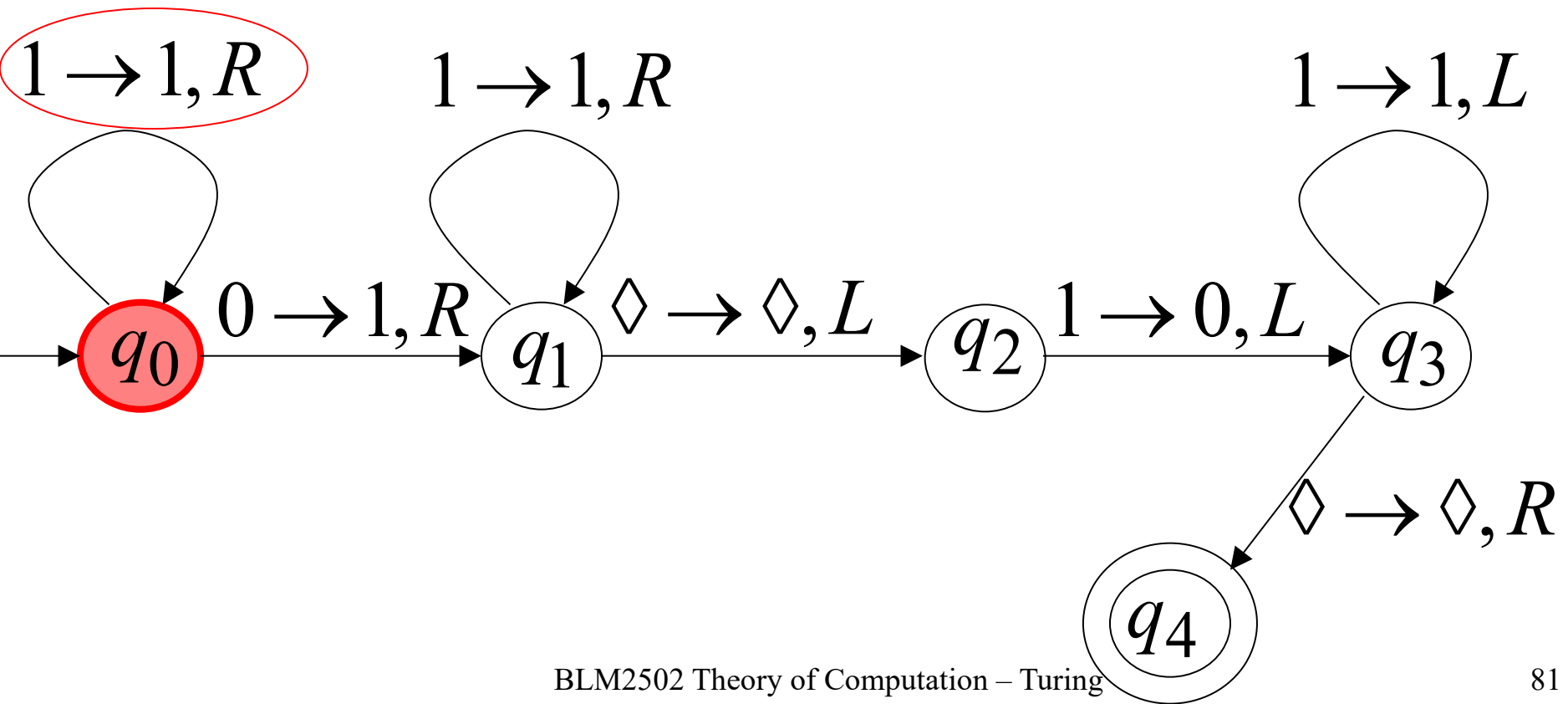
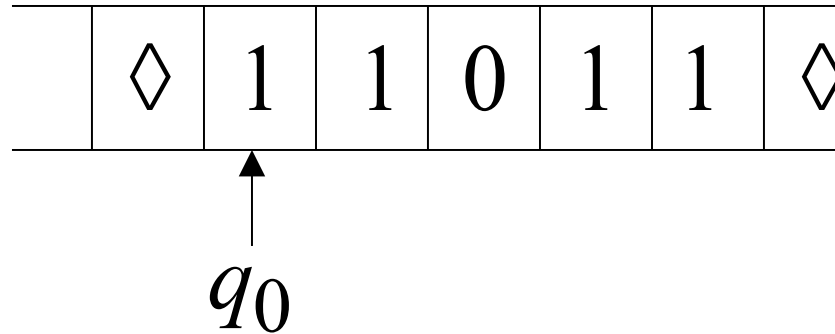
Time 0



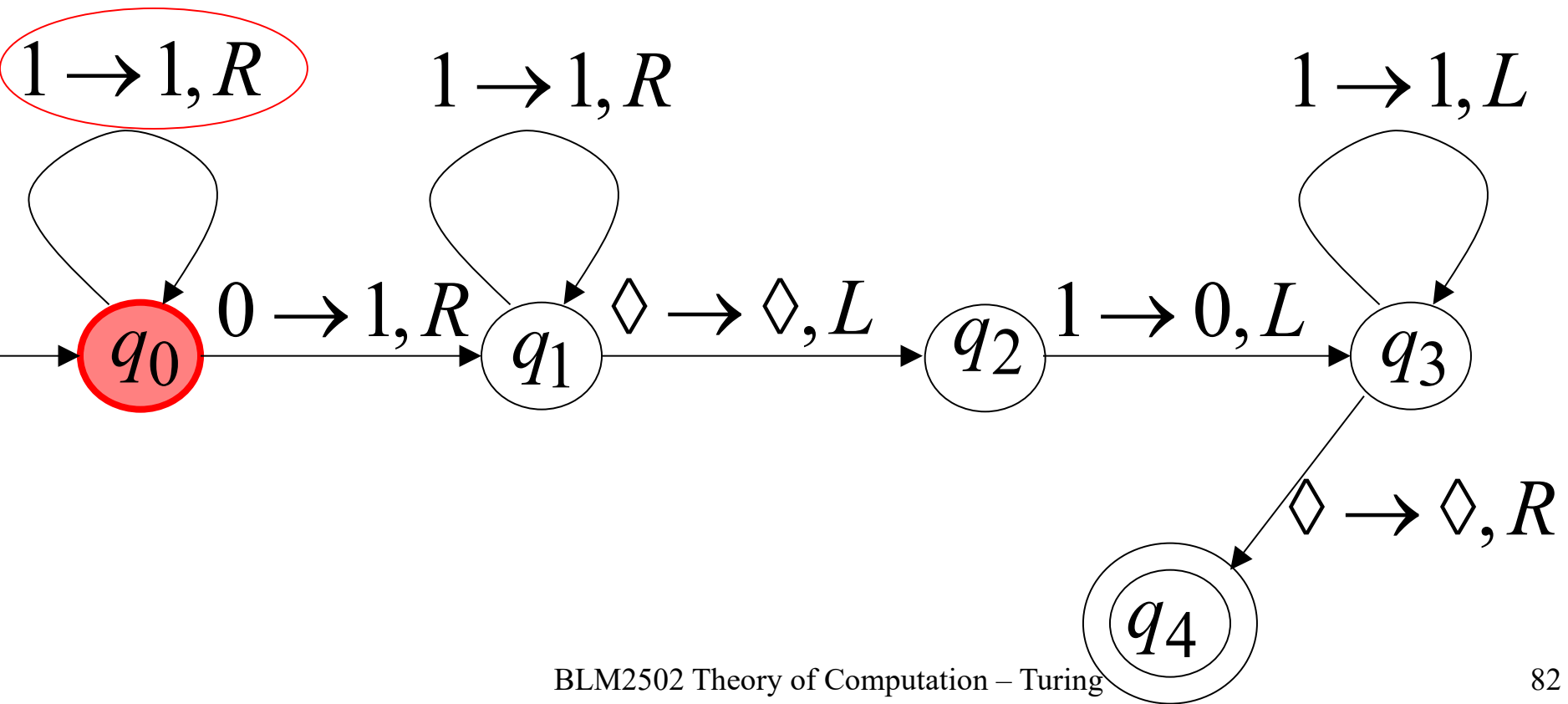
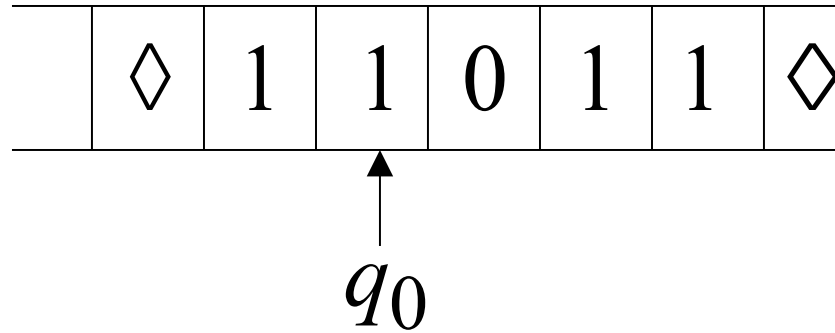
Final Result



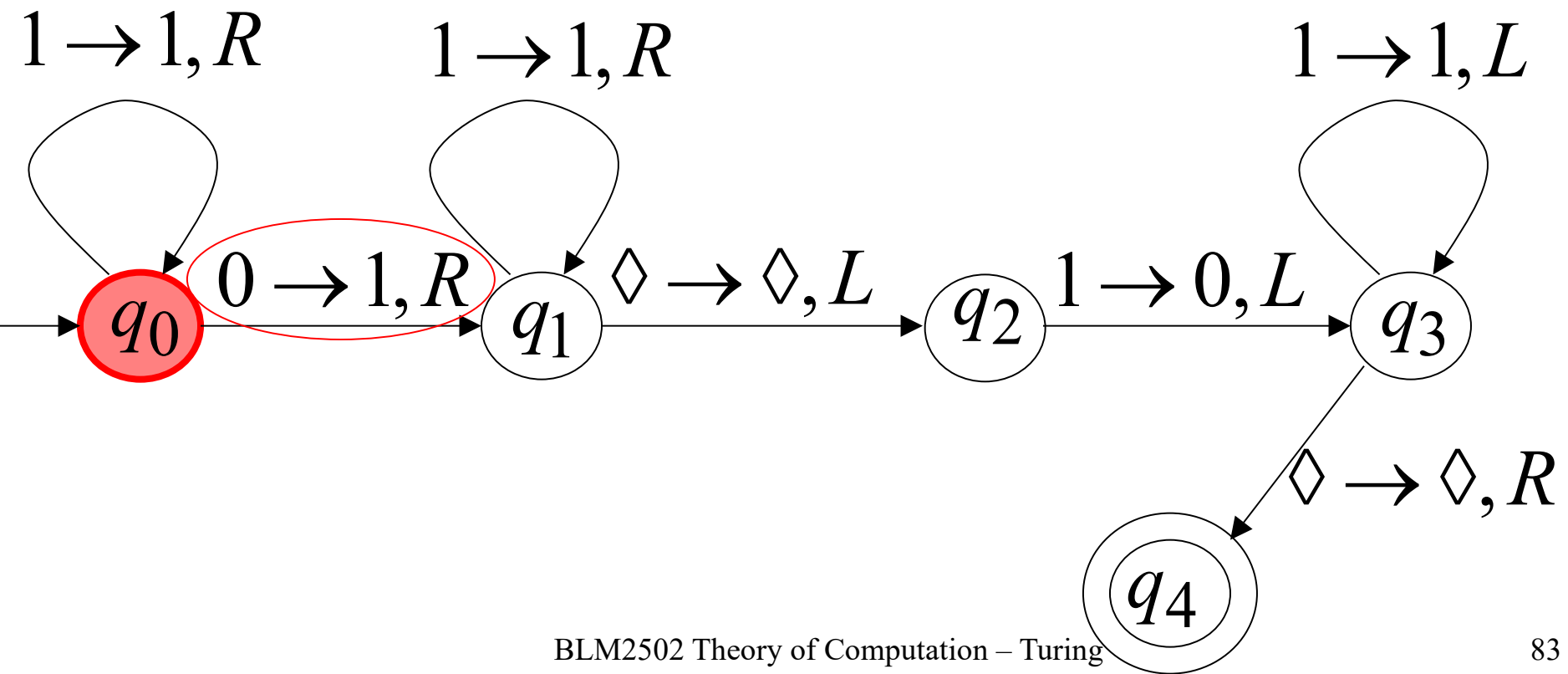
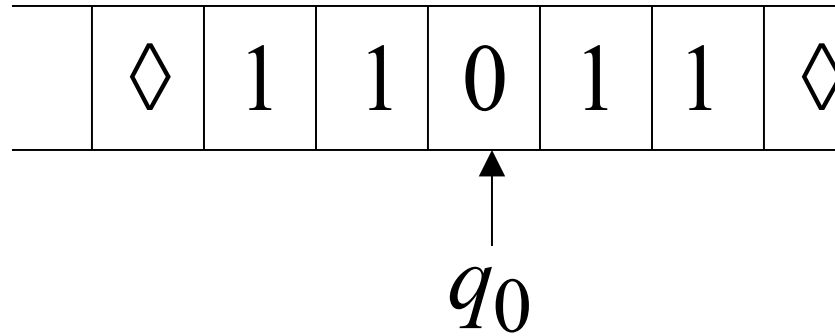
Time 0



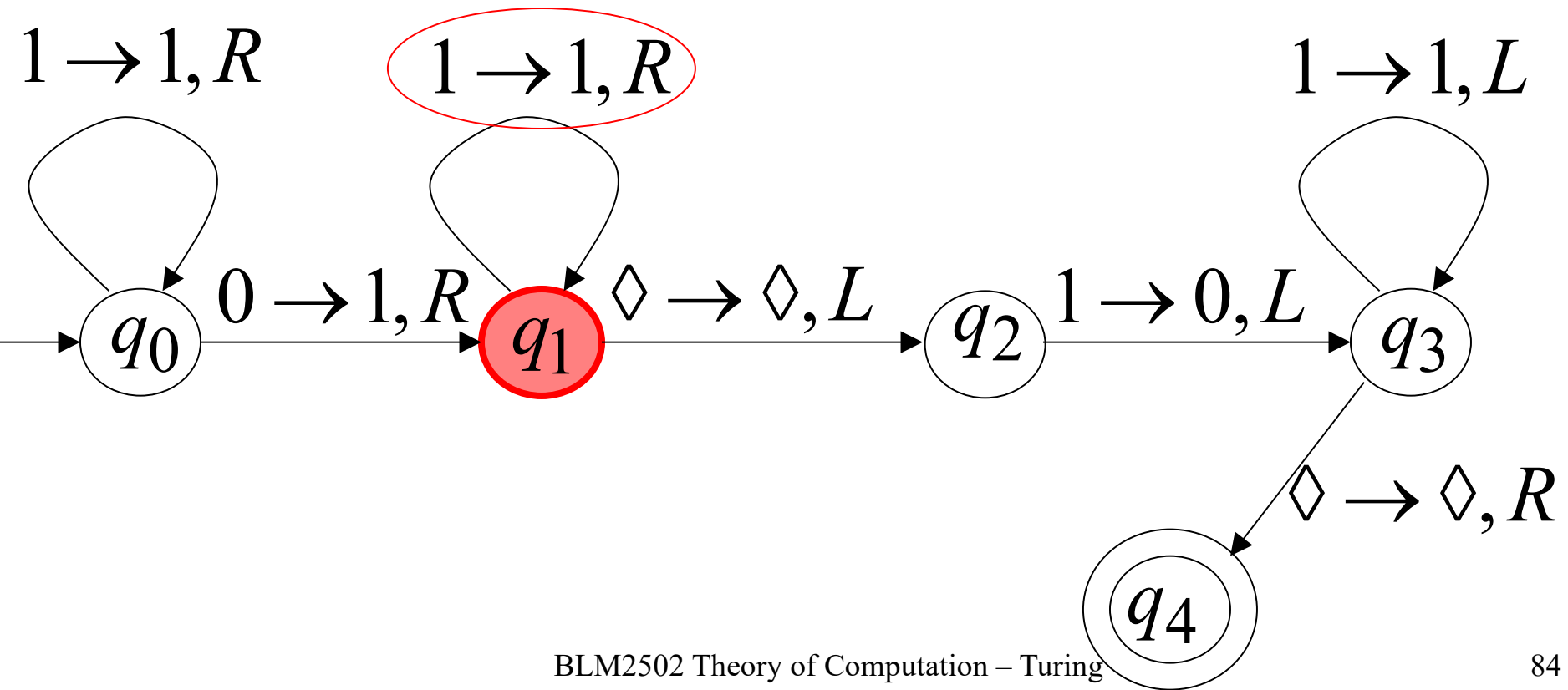
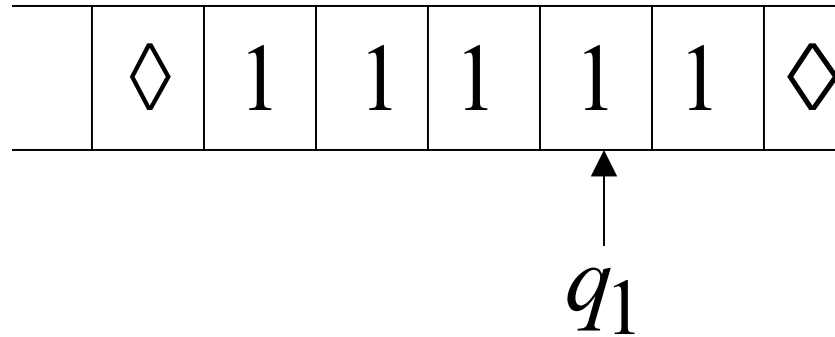
Time 1



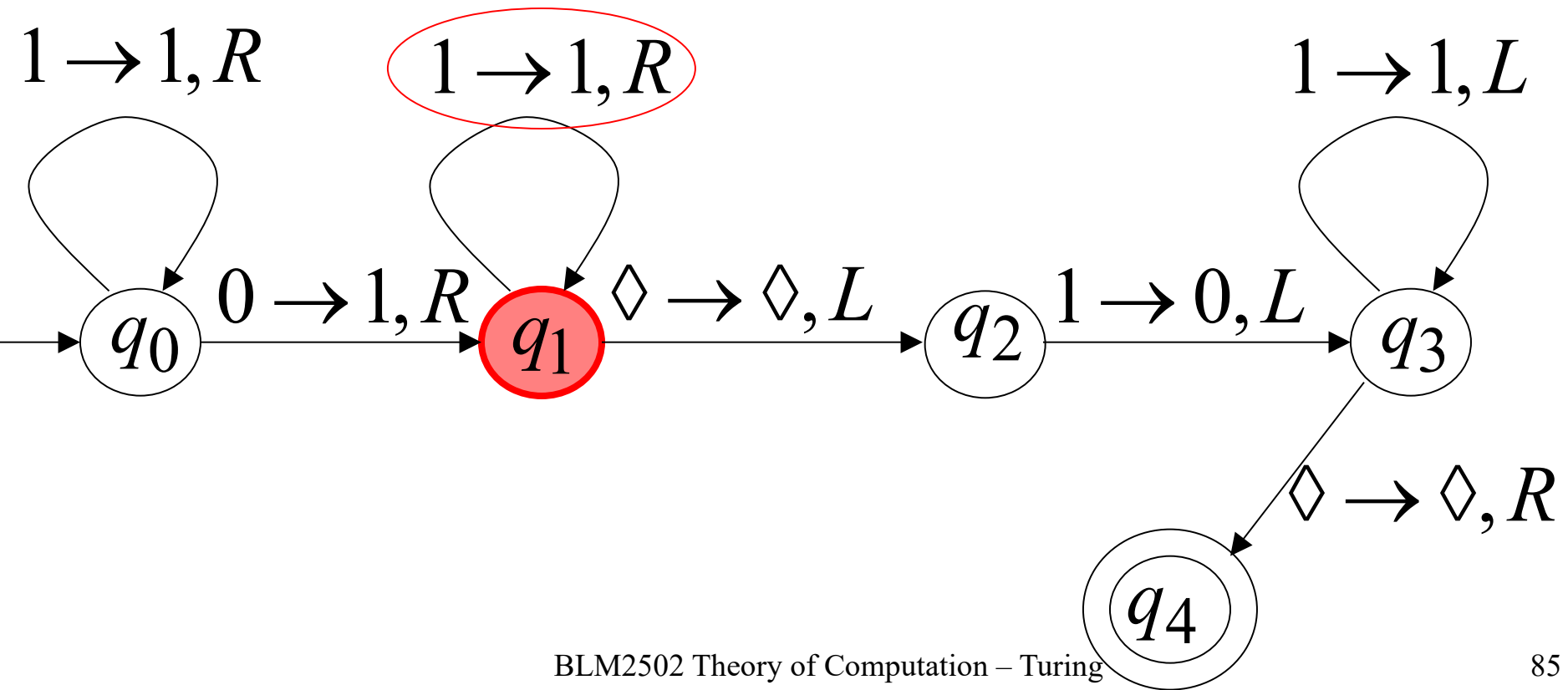
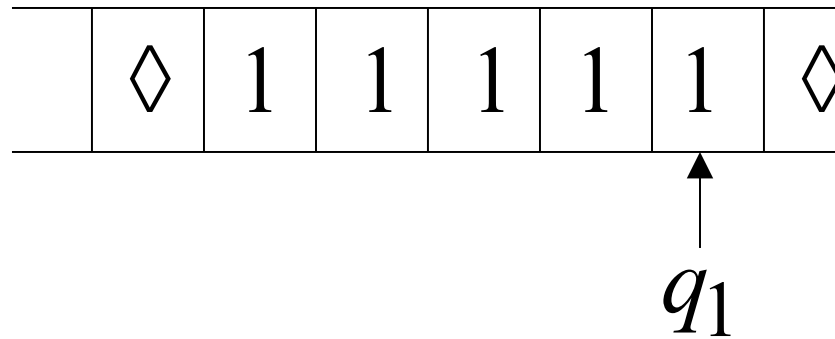
Time 2



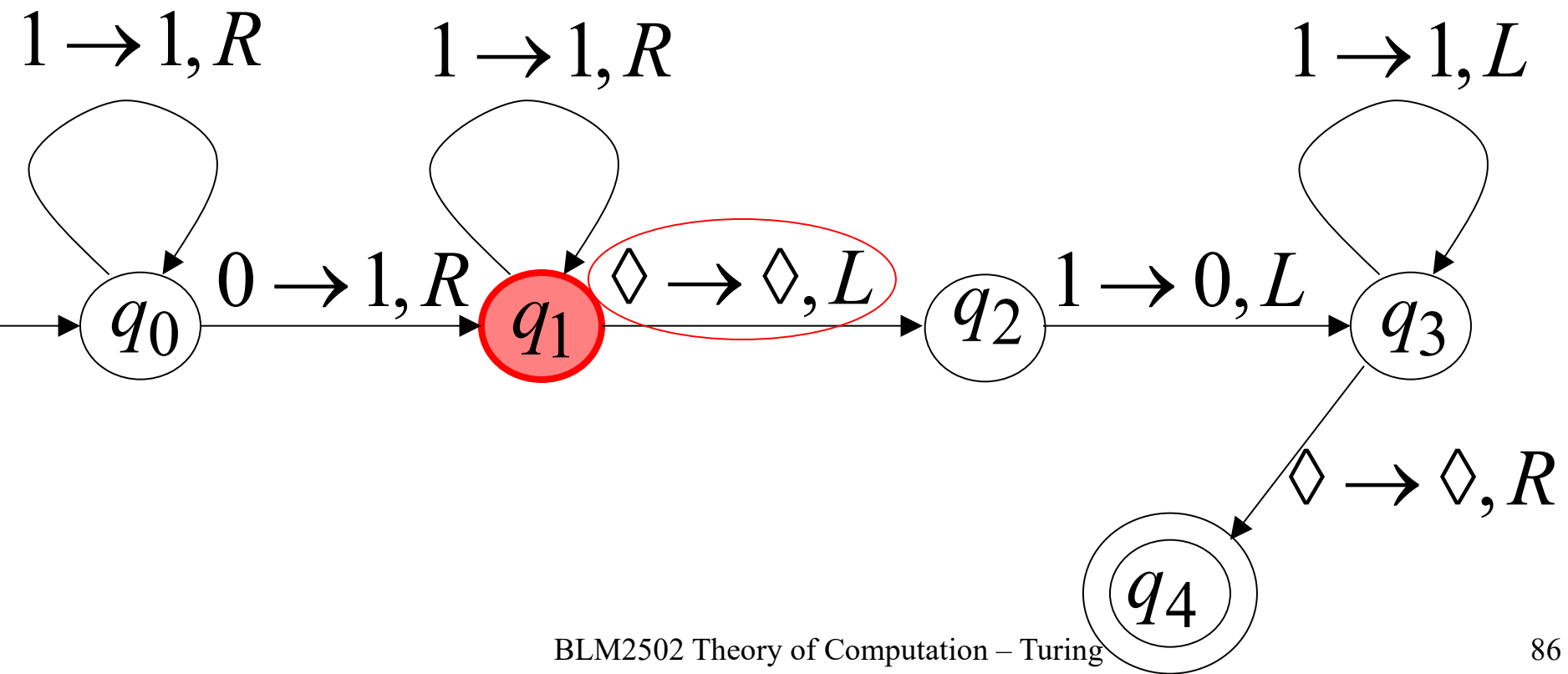
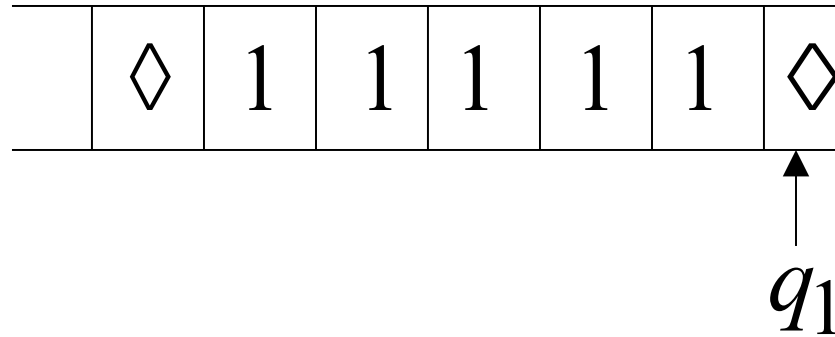
Time 3



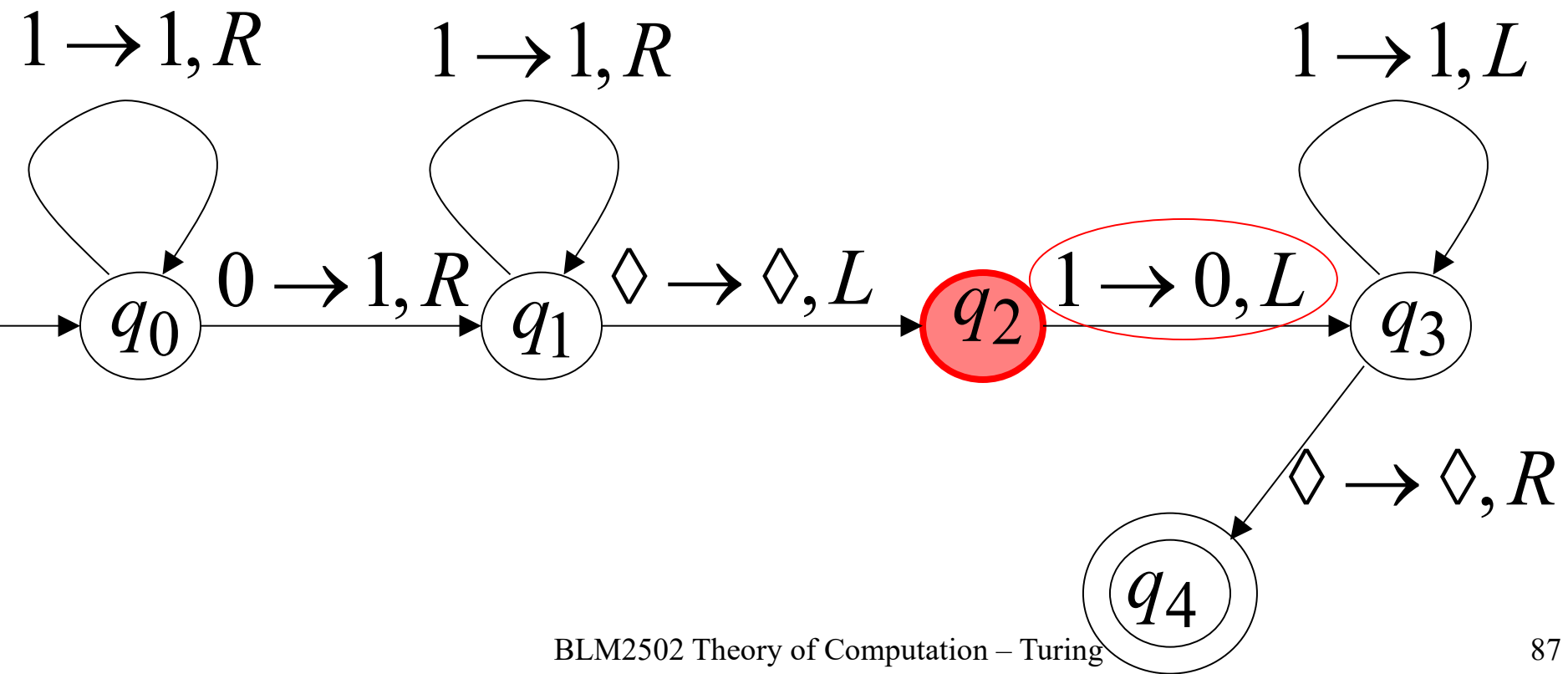
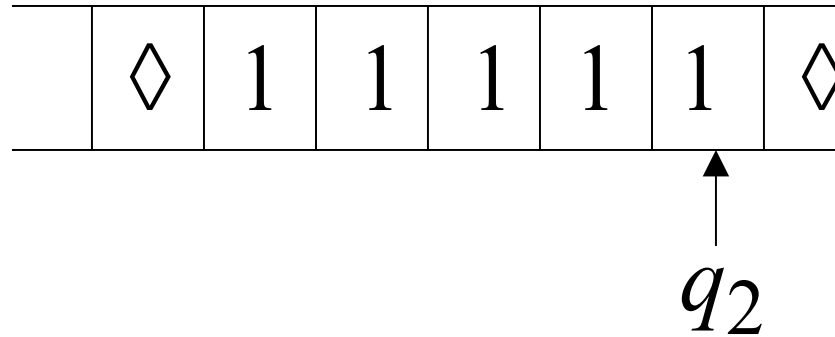
Time 4



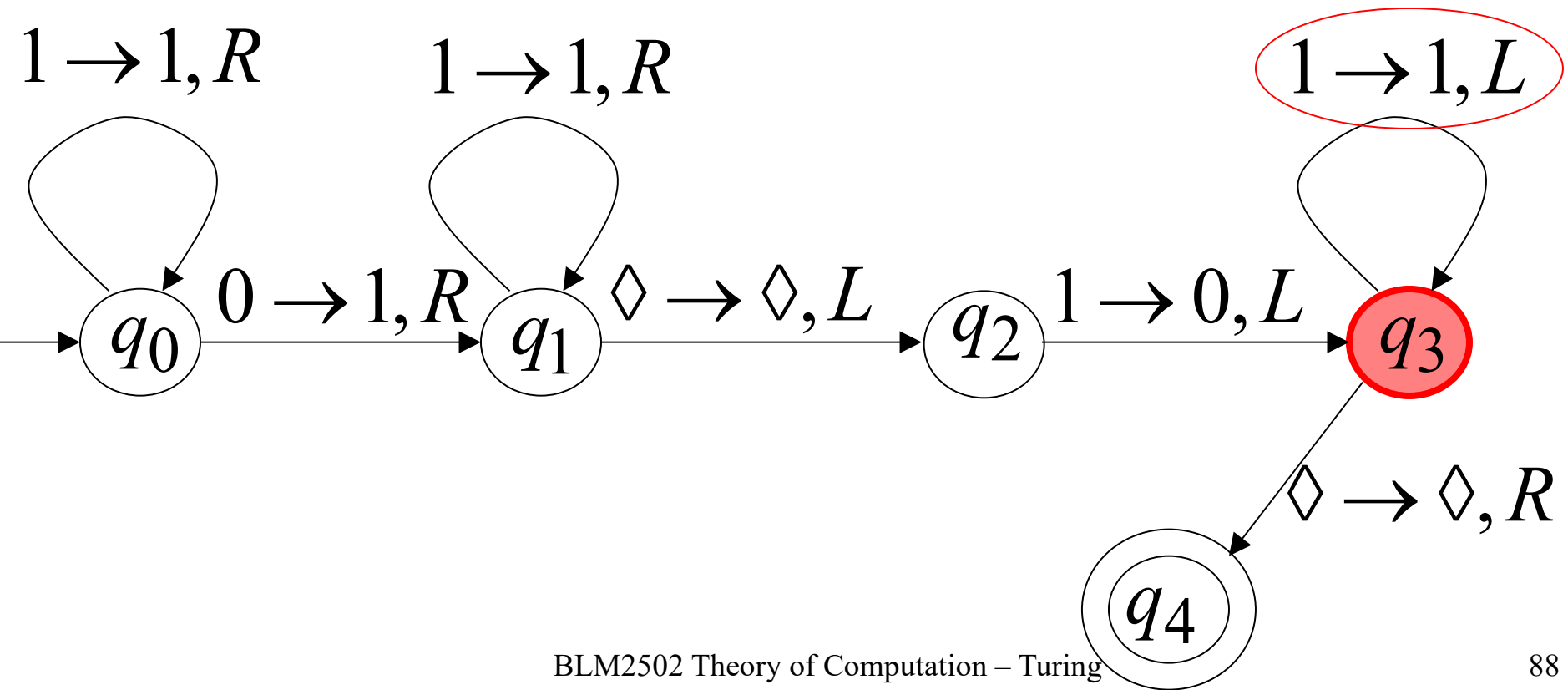
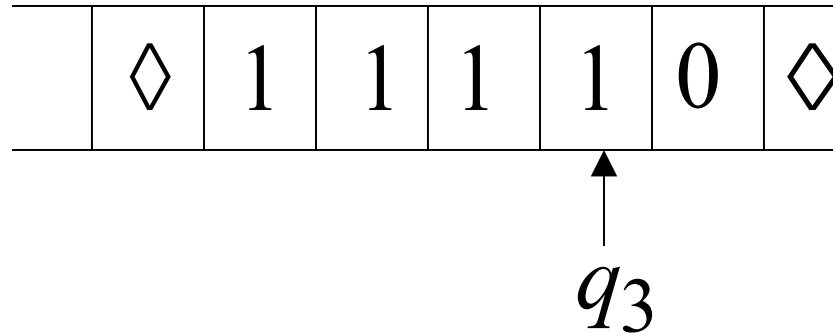
Time 5



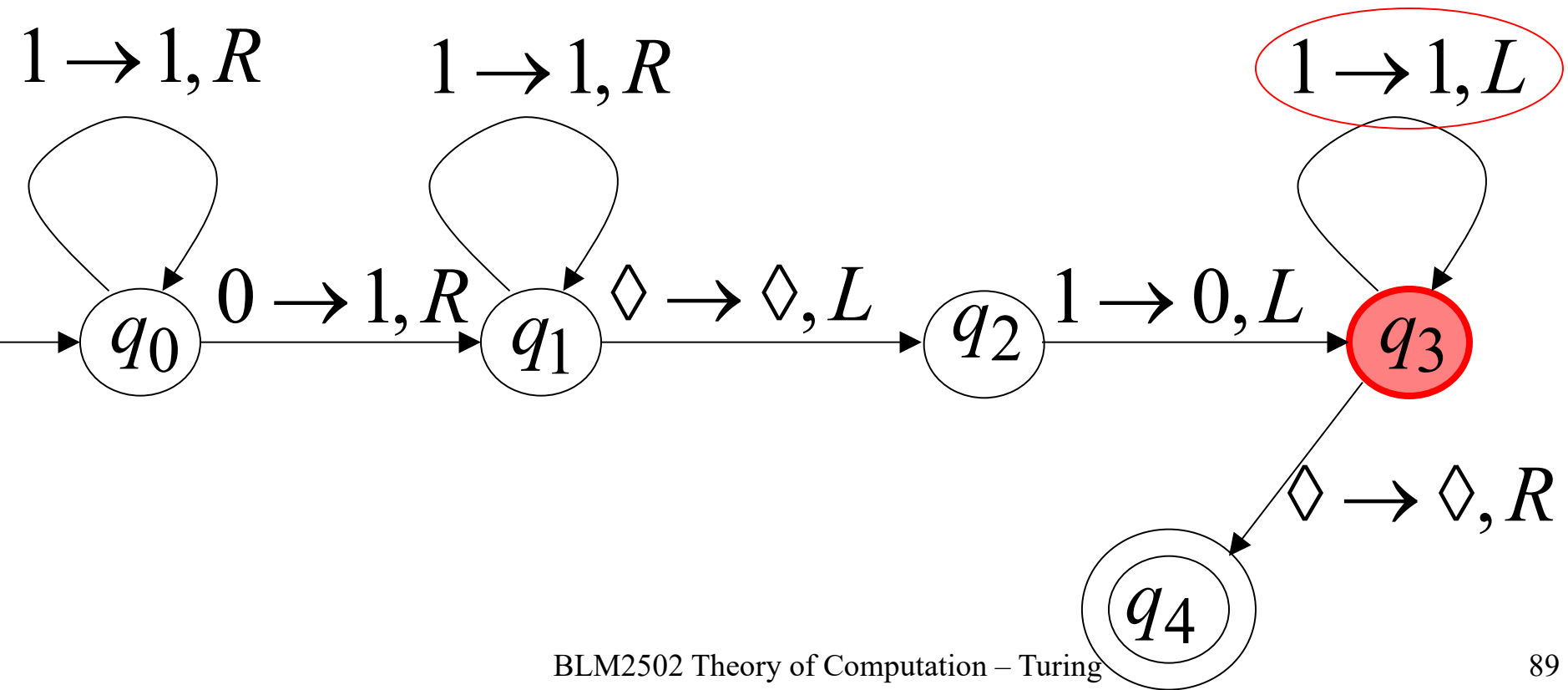
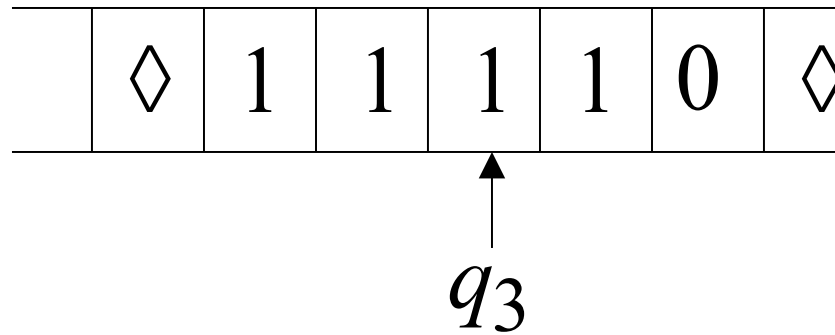
Time 6



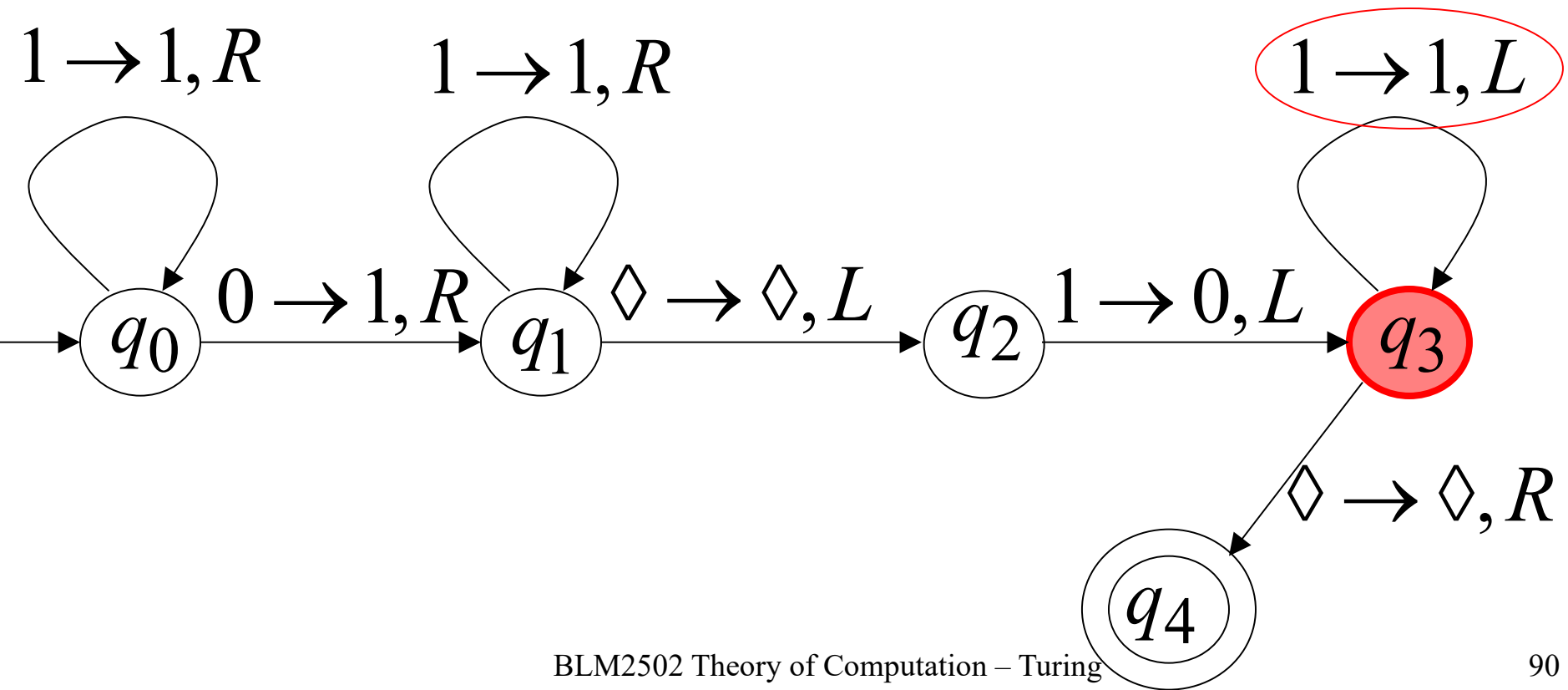
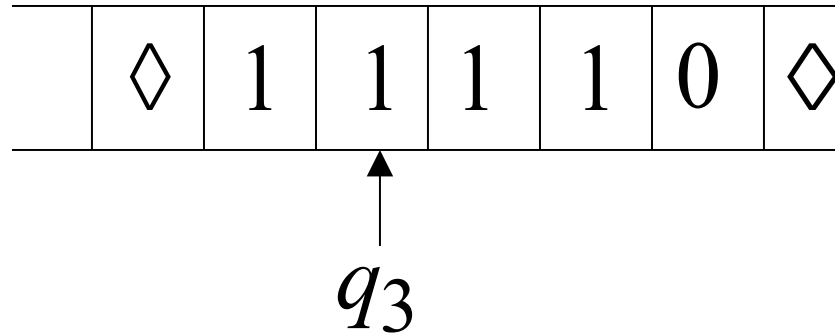
Time 7



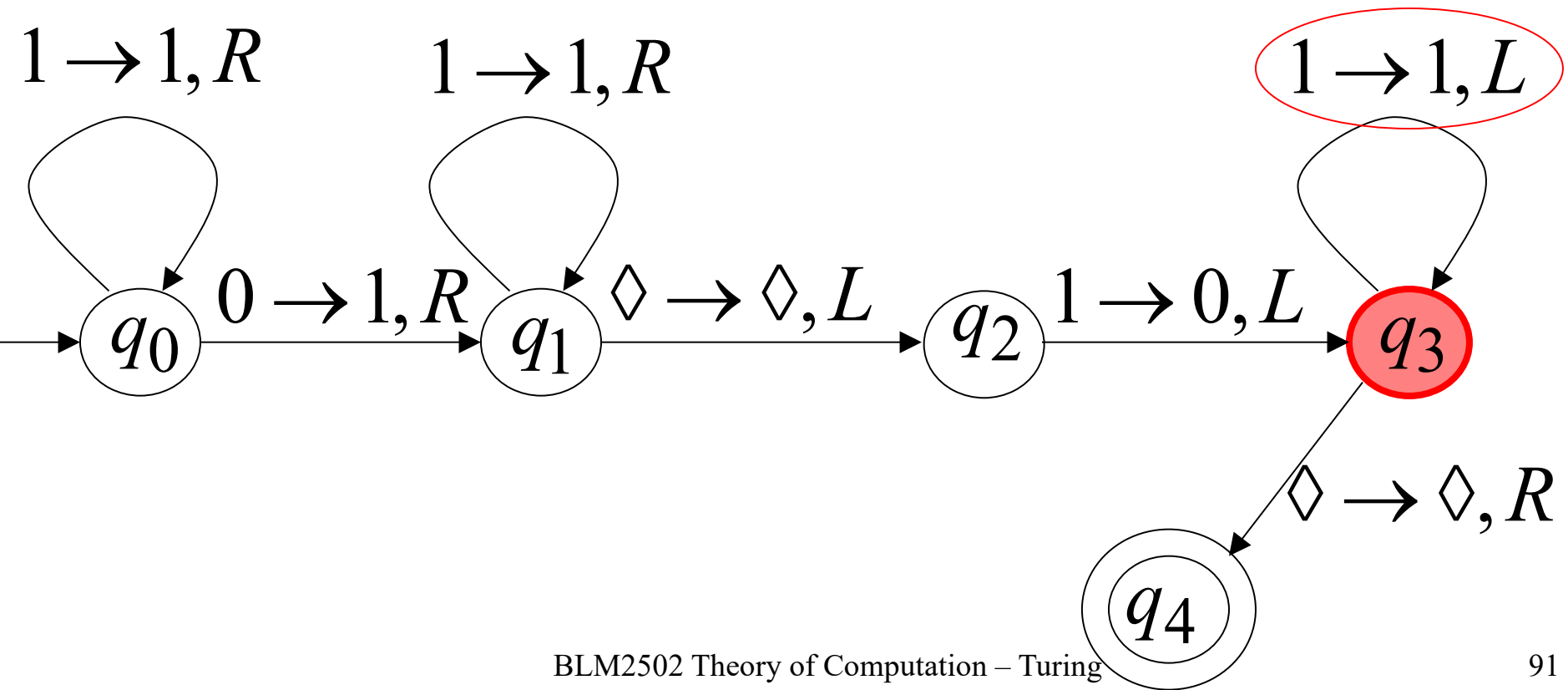
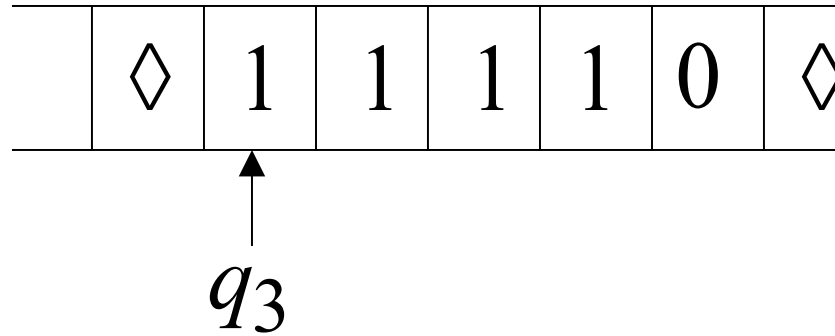
Time 8



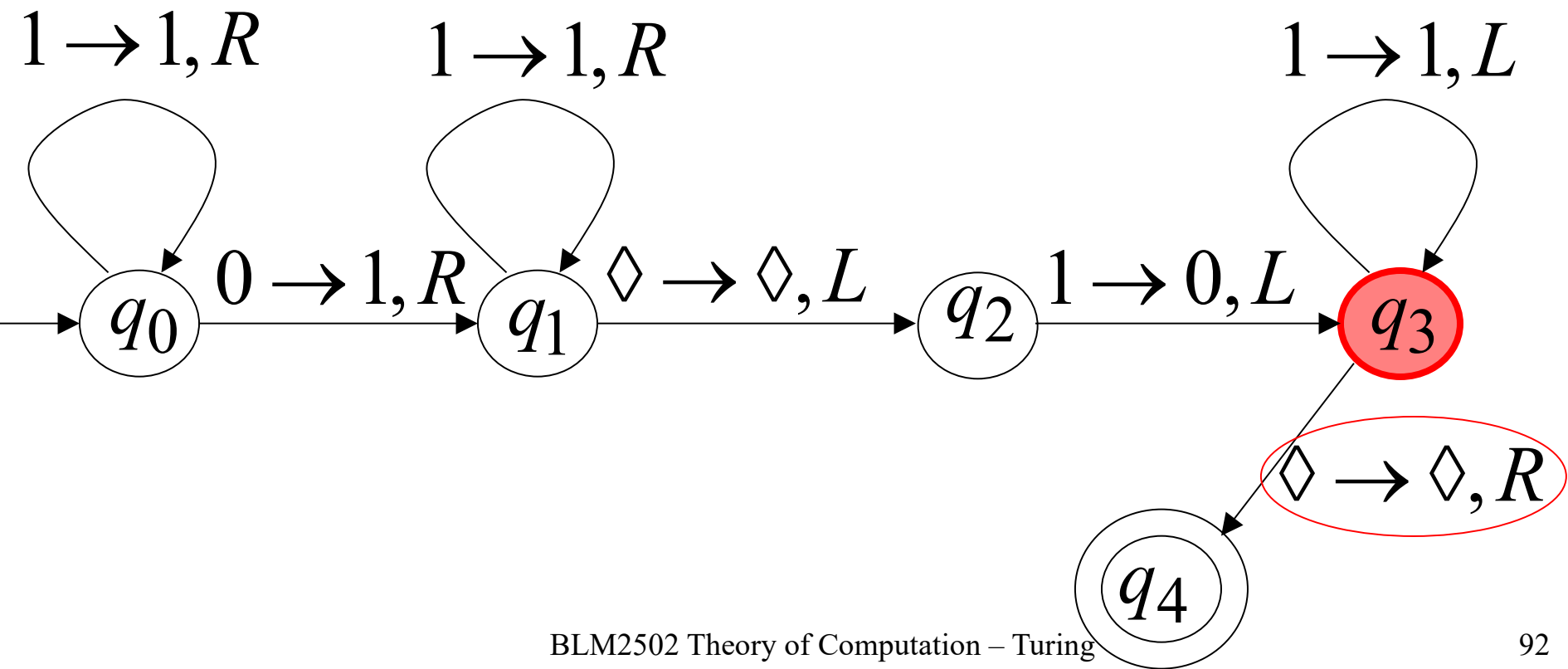
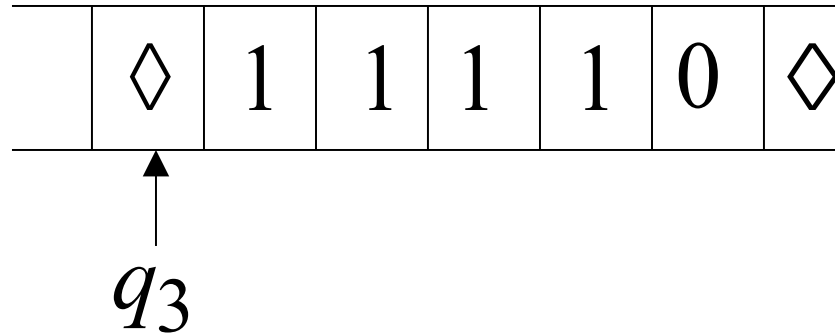
Time 9



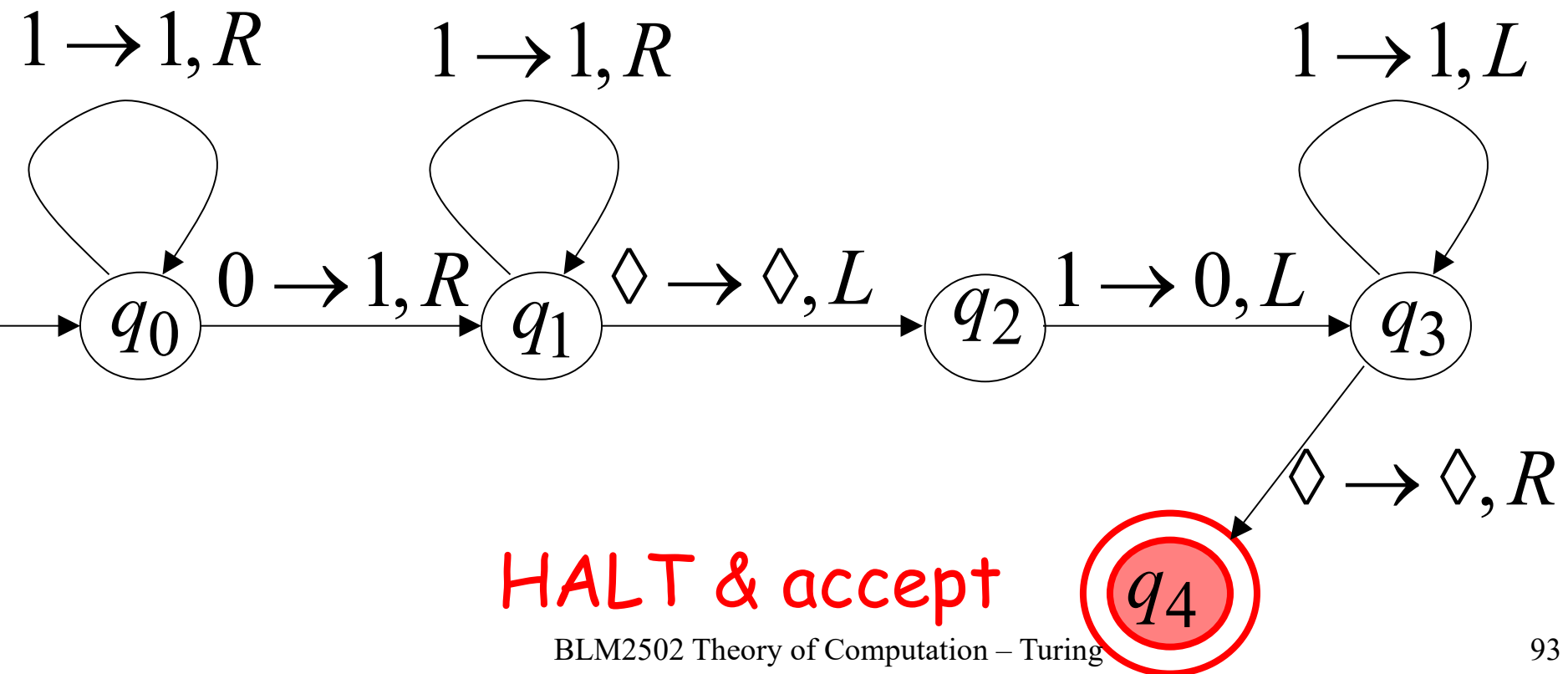
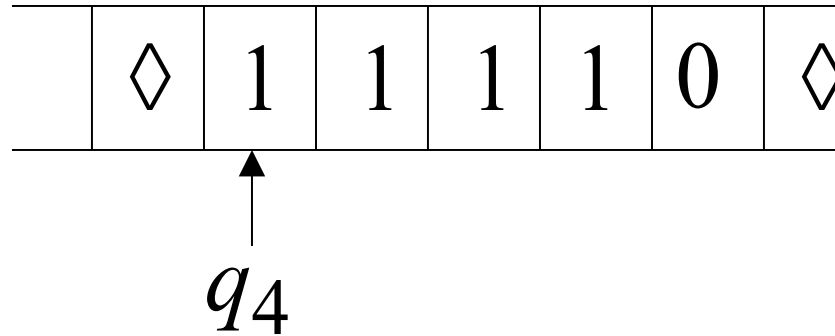
Time 10



Time 11



Time 12



Another Example

The function $f(x) = 2x$ is computable

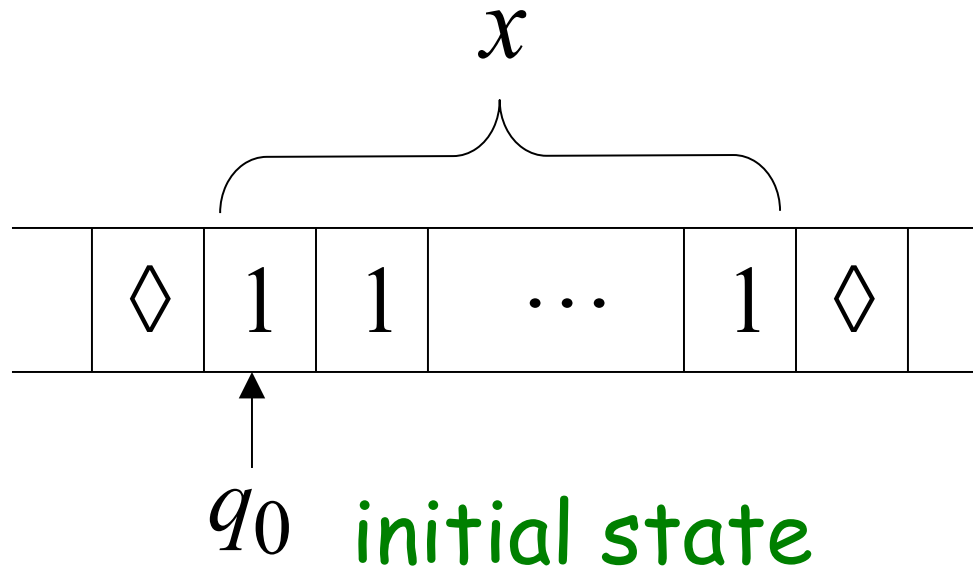
x is integer

Turing Machine:

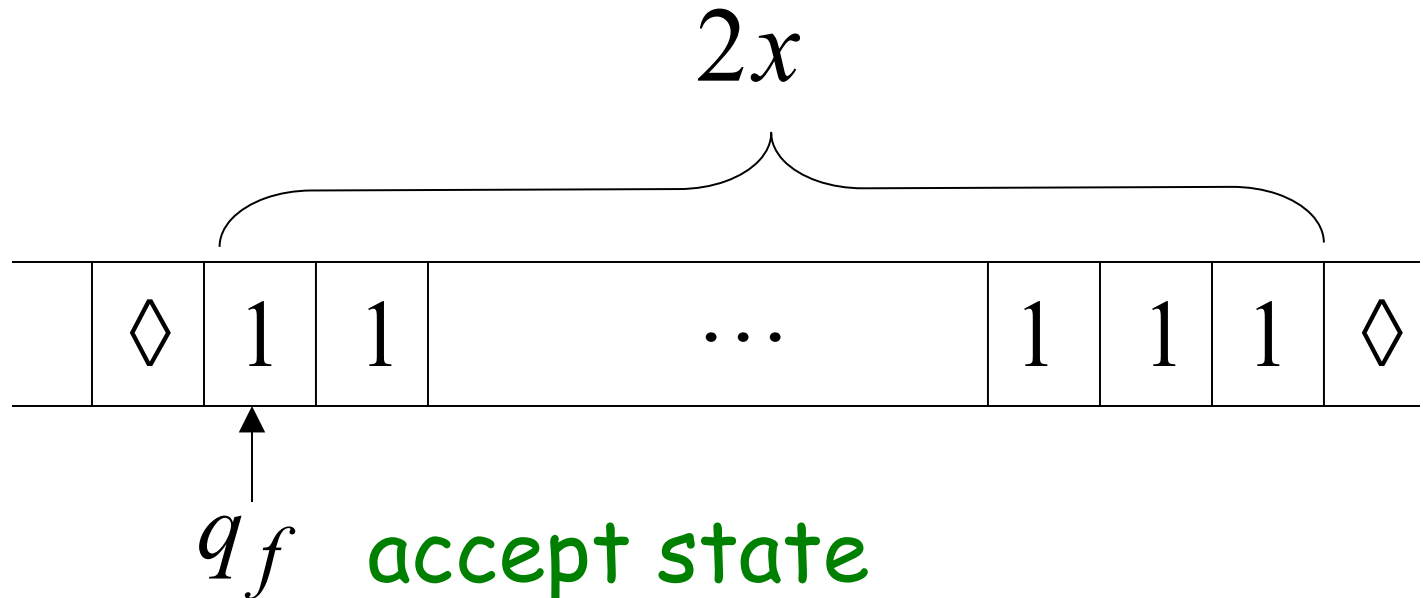
Input string: x unary

Output string: xx unary

Start



Finish

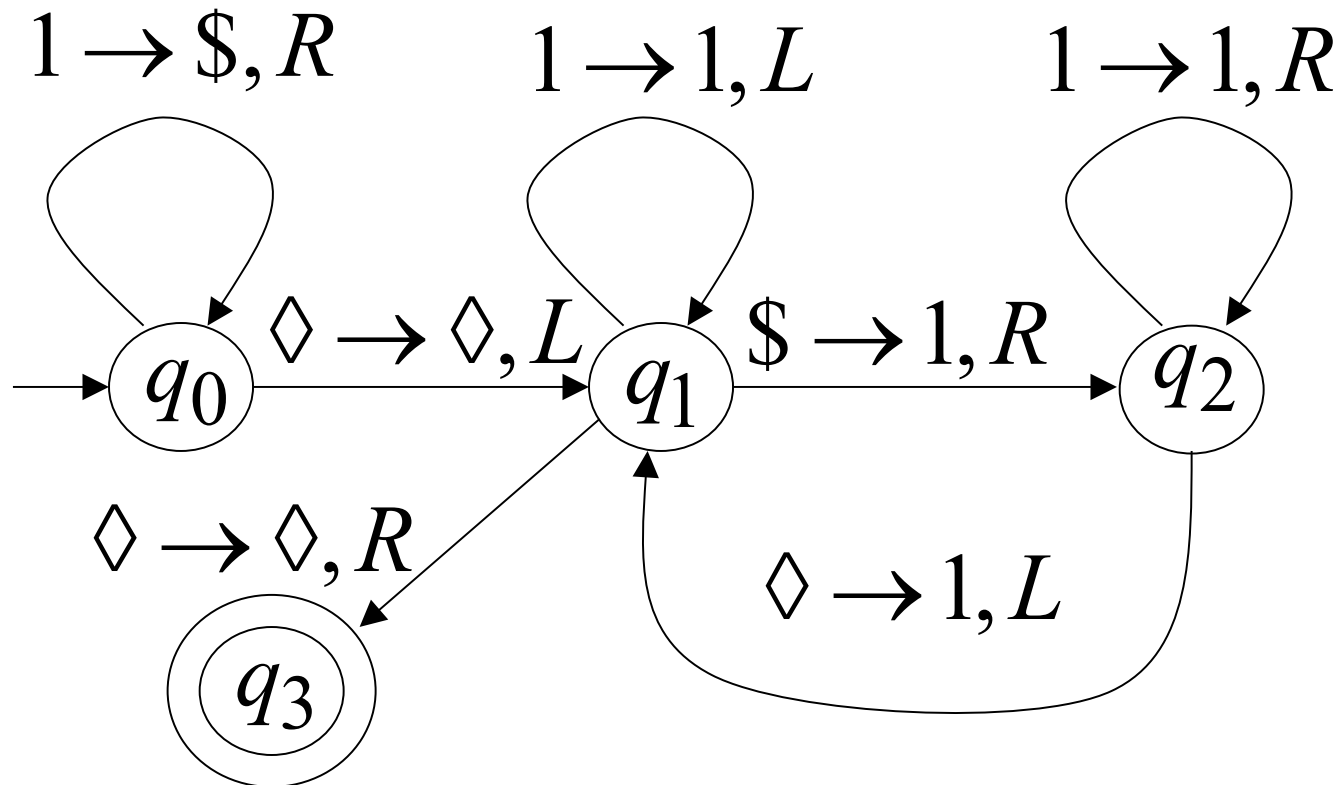


Turing Machine Pseudocode for $f(x) = 2x$

- Replace every 1 with \$
- Repeat:
 - Find rightmost \$, replace it with 1
 - Go to right end, insert 1

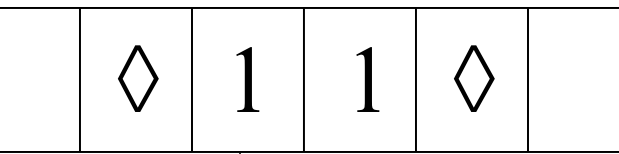
Until no more \$ remain

Turing Machine for $f(x) = 2x$



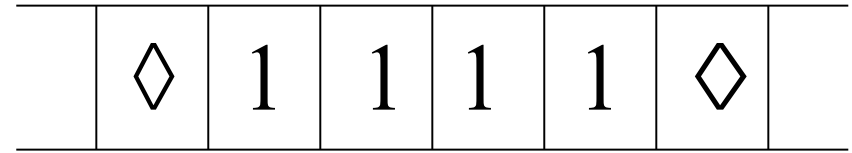
Example

Start

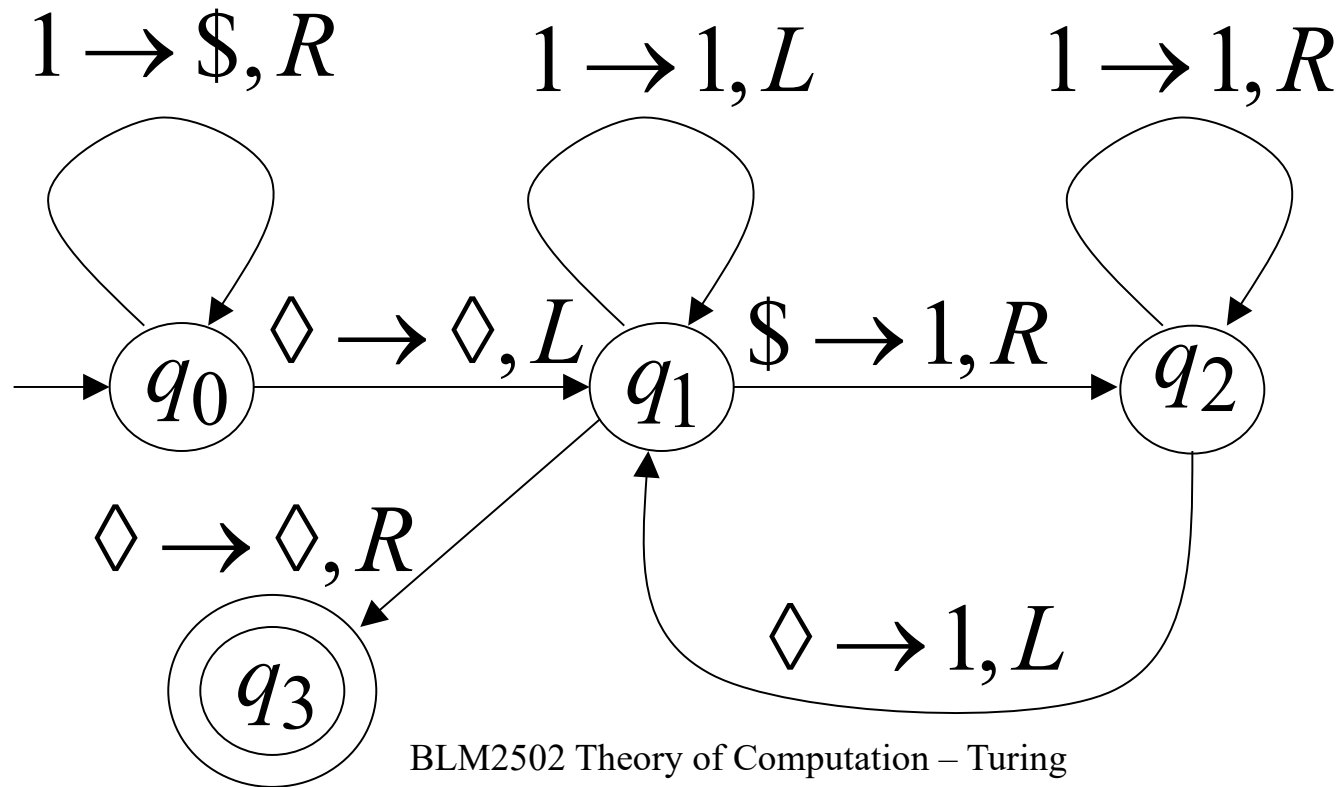


q_0

Finish



q_3



Another Example

The function is computable

$$f(x, y) = \begin{cases} 1 & \text{if } x > y \\ 0 & \text{if } x \leq y \end{cases}$$

Input: $x0y$

Output: 1 or 0

Turing Machine Pseudocode:

- Repeat

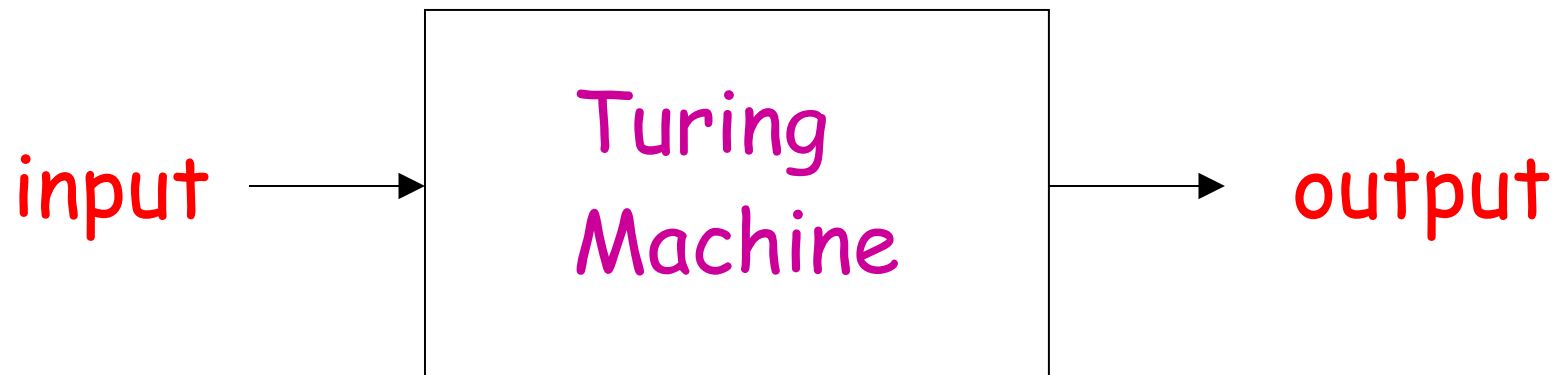
Match a 1 from x with a 1 from y

Until all of x or y is matched

- If a 1 from x is not matched
erase tape, write 1 $(x > y)$
else
erase tape, write 0 $(x \leq y)$

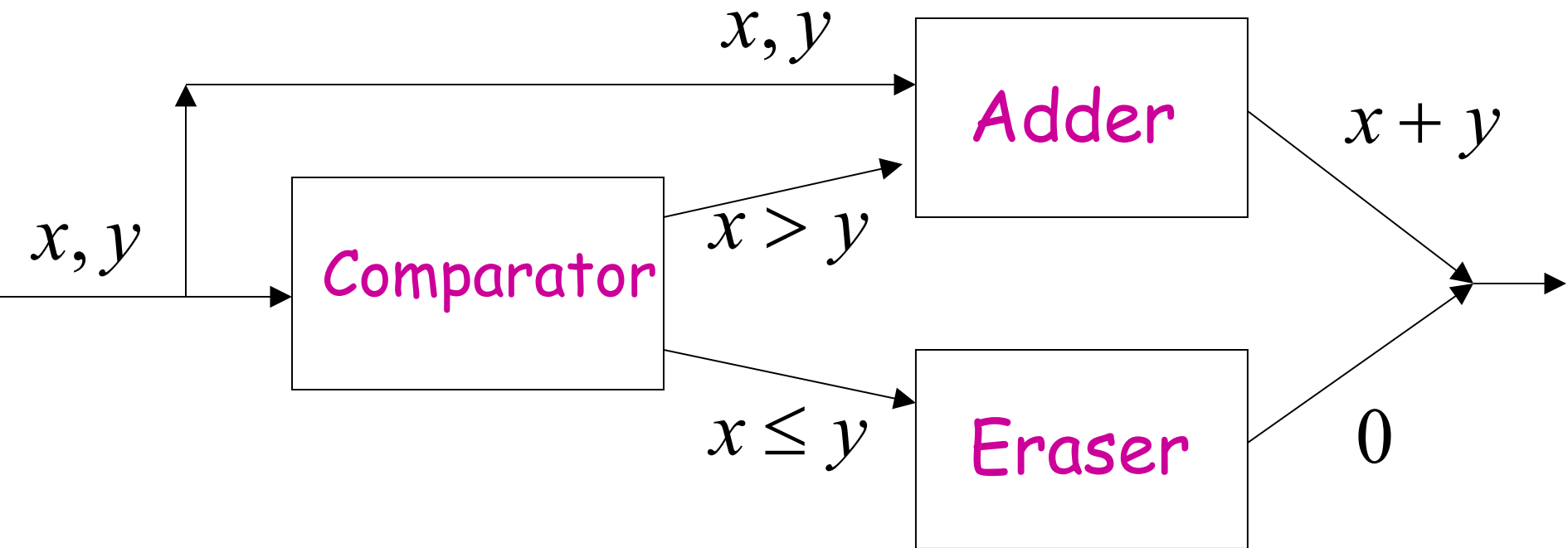
Combining Turing Machines

Block Diagram



Example:

$$f(x, y) = \begin{cases} x + y & \text{if } x > y \\ 0 & \text{if } x \leq y \end{cases}$$



Turing's Thesis

Turing's thesis (1930):

Any computation carried out
by mechanical means
can be performed by a Turing Machine

Algorithm:

An algorithm for a problem is a Turing Machine which solves the problem

The algorithm describes the steps of the mechanical means

This is easily translated to computation steps of a Turing machine

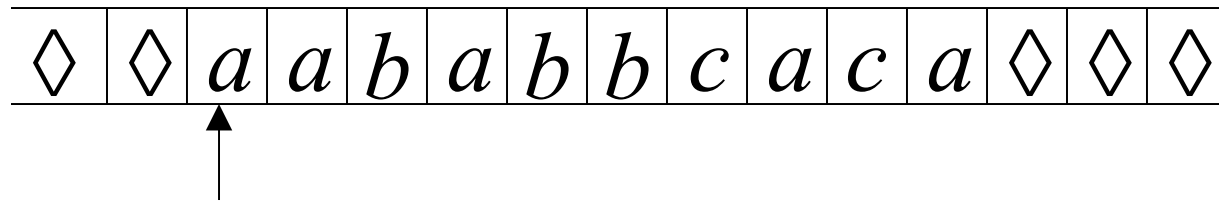
When we say: There exists an algorithm

We mean: There exists a Turing Machine
that executes the algorithm

Variations of the Turing Machine

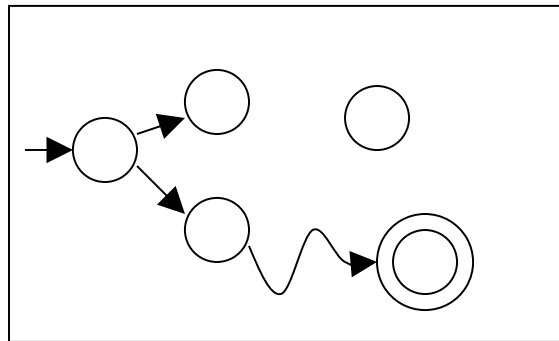
The Standard Model

Infinite Tape



Read-Write Head (Left or Right)

Control Unit



Deterministic

Variations of the Standard Model

- Turing machines with:
- Stay-Option
 - Semi-Infinite Tape
 - Multitape
 - Multidimensional
 - Nondeterministic

Different Turing Machine **Classes**

Same Power of two machine classes:

both classes accept the
same set of languages

We will prove:

each new class has the same power
with Standard Turing Machine

(accept Turing-Recognizable Languages)

Same Power of two classes means:

for any machine M_1 of first class

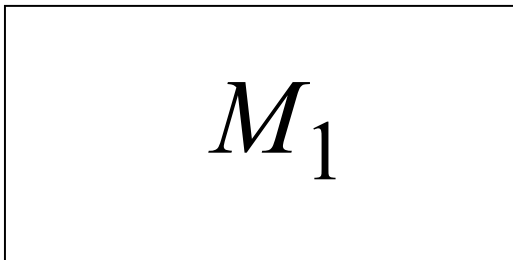
there is a machine M_2 of second class

such that: $L(M_1) = L(M_2)$

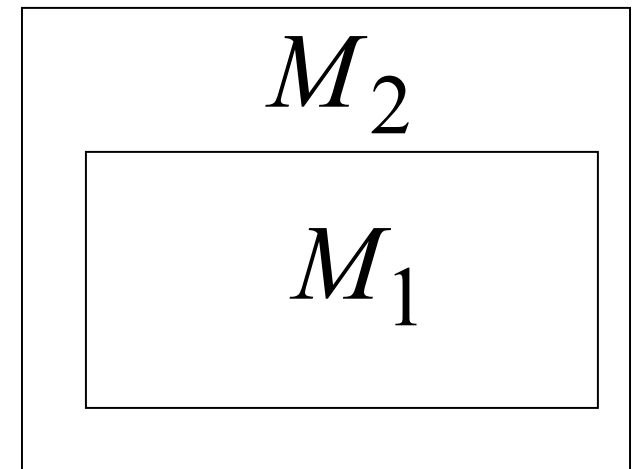
and vice-versa

Simulation: A technique to prove same power.
Simulate the machine of one class
with a machine of the other class

First Class
Original Machine

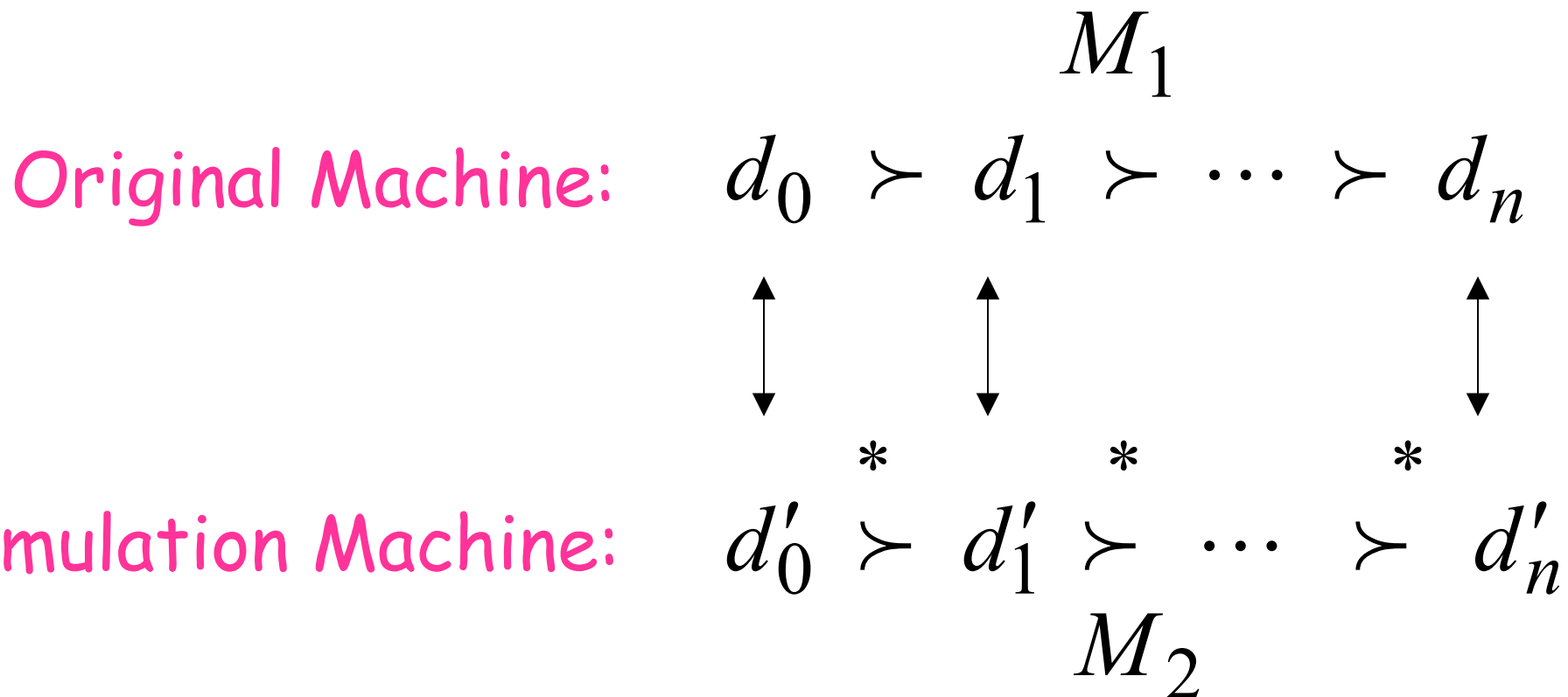


Second Class
Simulation Machine



simulates M_1

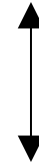
Configurations in the Original Machine M_1
 have corresponding configurations
 in the Simulation Machine M_2



Accepting Configuration

Original Machine:

d_f



Simulation Machine:

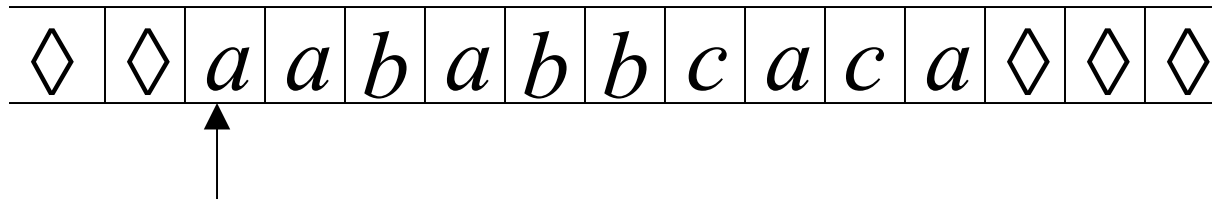
d'_f

the Simulation Machine
and the Original Machine
accept the same strings

$$L(M_1) = L(M_2)$$

Turing Machines with Stay-Option

The head can stay in the same position

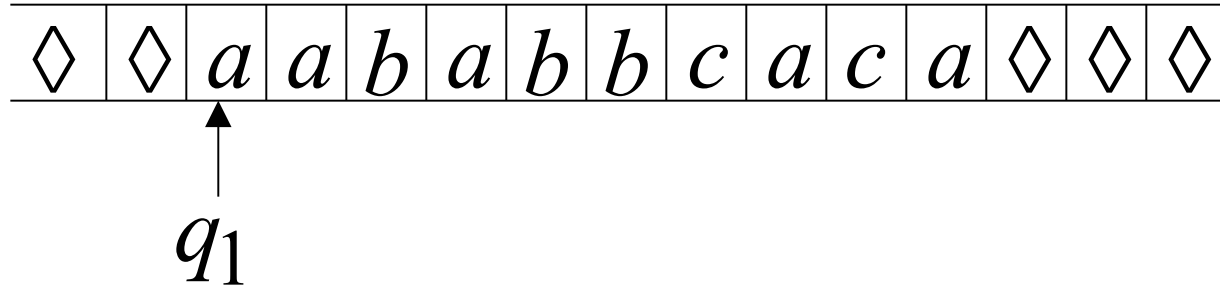


Left, Right, Stay

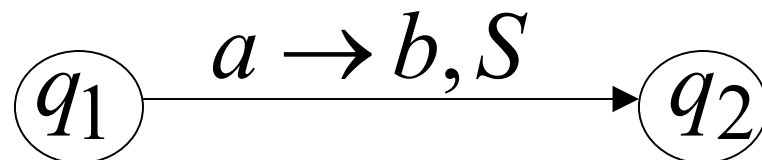
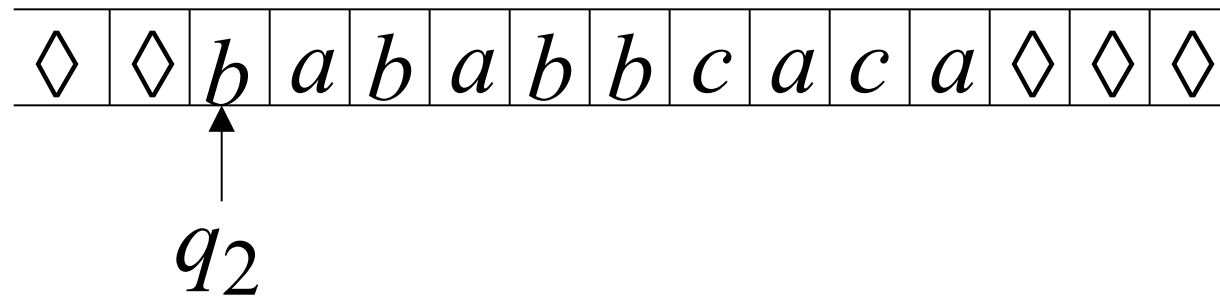
L,R,S: possible head moves

Example:

Time 1



Time 2



Theorem: Stay-Option machines
have the same power with
Standard Turing machines

Proof: 1. Stay-Option Machines
simulate Standard Turing machines

2. Standard Turing machines
simulate Stay-Option machines

1. Stay-Option Machines

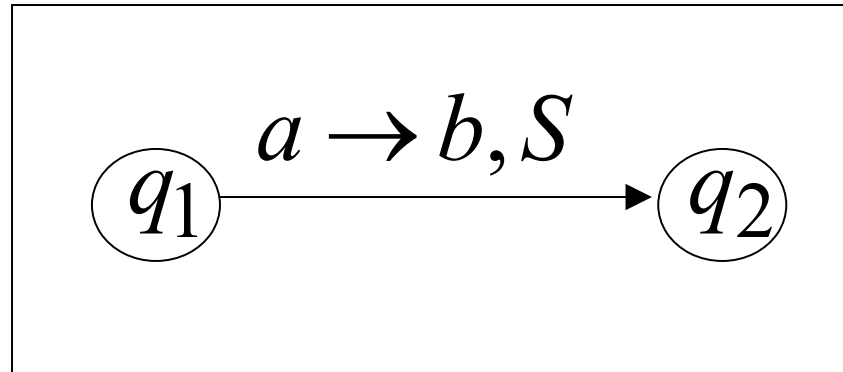
simulate Standard Turing machines

Trivial: any standard Turing machine
is also a Stay-Option machine

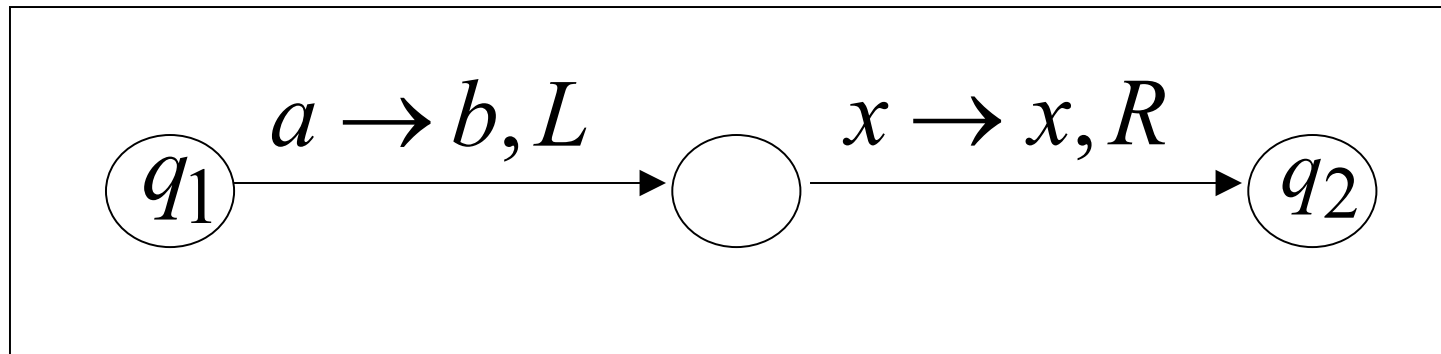
2. Standard Turing machines simulate Stay-Option machines

We need to simulate the **stay** head option
with two head moves, one **left** and one **right**

Stay-Option Machine



Simulation in Standard Machine

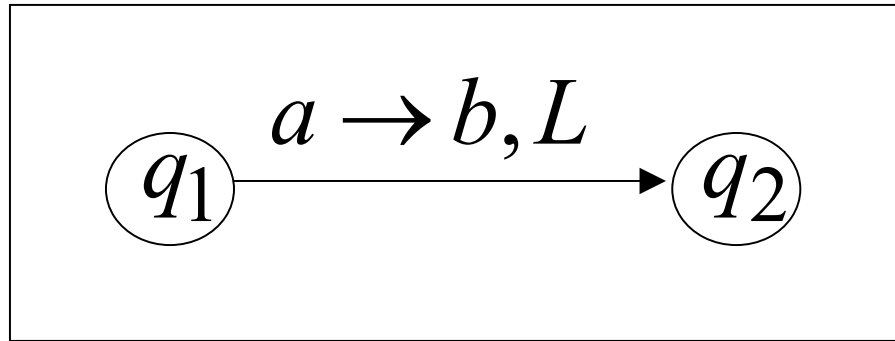


Busch's
simulation

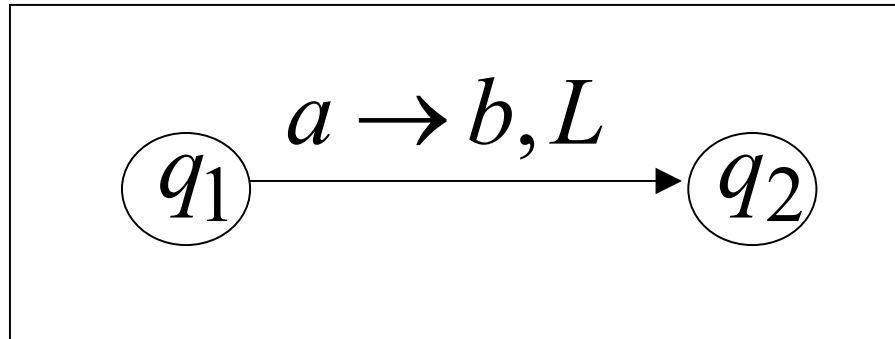
For every possible tape symbol x

For other transitions nothing changes

Stay-Option Machine



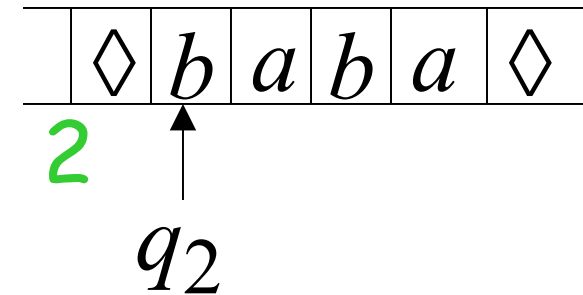
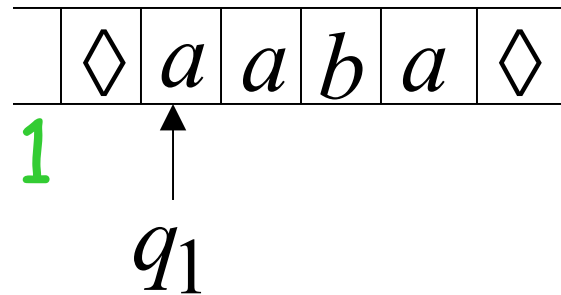
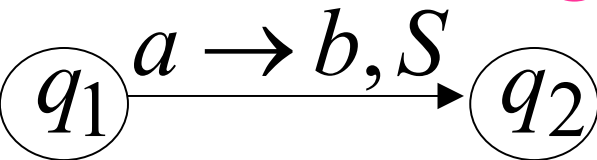
Simulation in Standard Machine



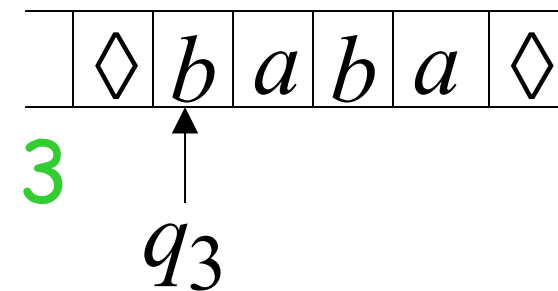
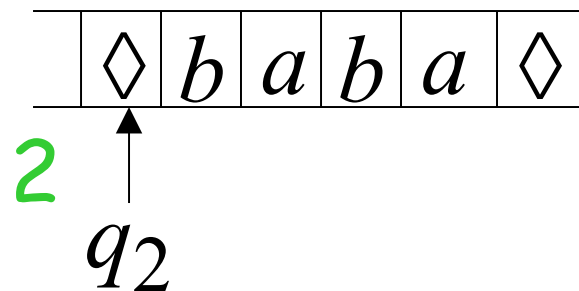
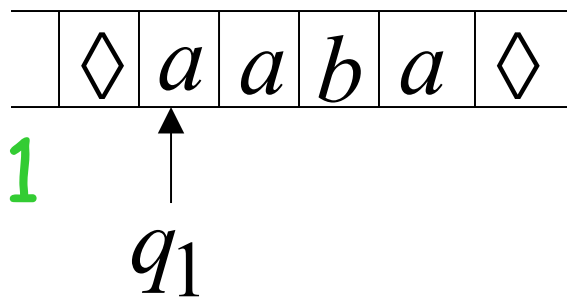
Similar for Right moves

example of simulation

Stay-Option Machine:



Simulation in Standard Machine:



END OF PROOF

A useful trick: Multiple Track Tape

helps for more complicated simulations

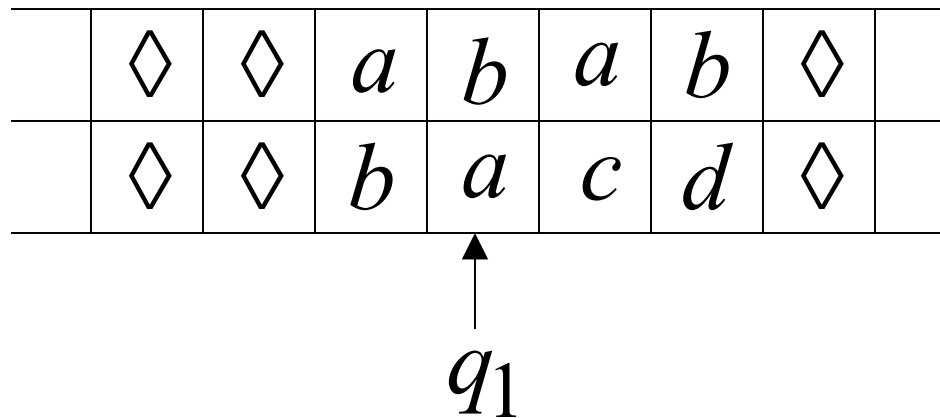
One Tape

	◇	◇	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>	◇		track 1
	◇	◇	<i>b</i>	<i>a</i>	<i>c</i>	<i>d</i>	◇		track 2

One head

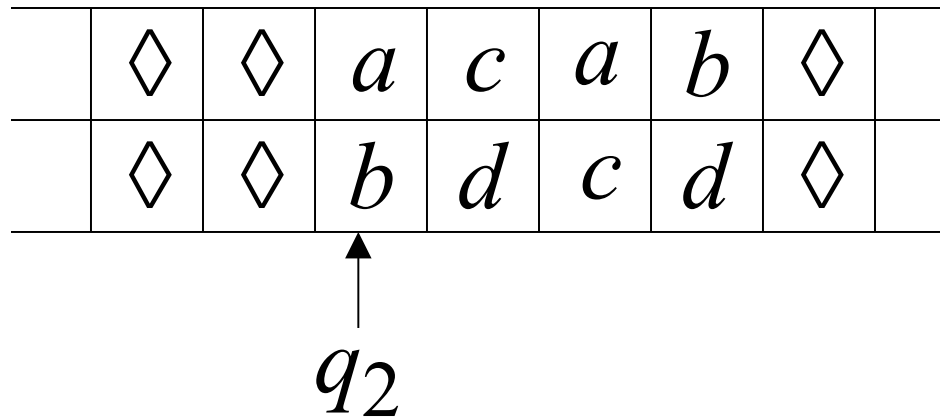
One symbol (*a*, *b*)

It is a standard Turing machine, but each tape alphabet symbol describes a pair of actual useful symbols



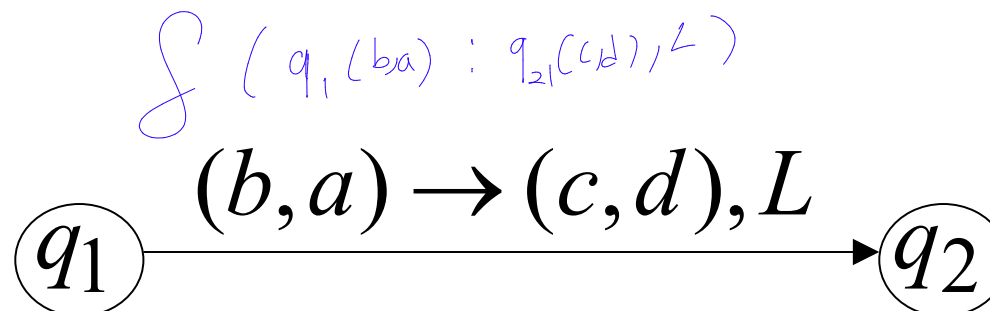
track 1

track 2



track 1

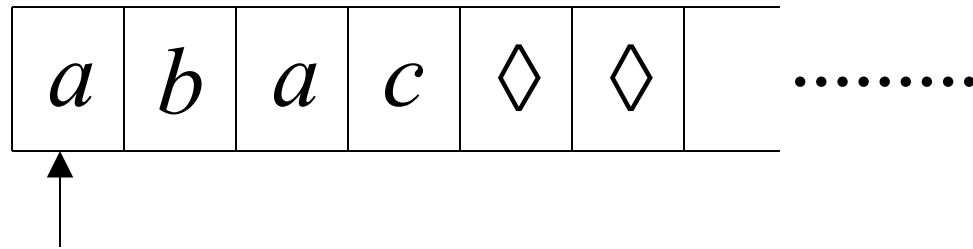
track 2



$\Sigma = \{ (a,b), (c,d), (c,a), (b,d), \dots \}$
 Alphabet: $\{a, b, c, d\}$

Semi-Infinite Tape

The head extends infinitely only to the right



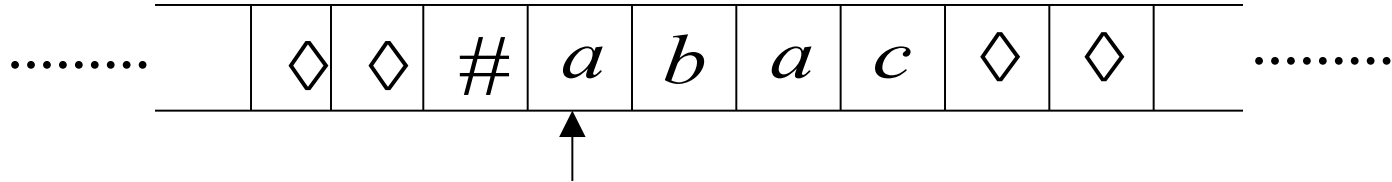
- Initial position is the leftmost cell
- When the head moves left from the border, it returns back to leftmost position

Theorem: Semi-Infinite machines
have the same power with
Standard Turing machines

Proof: 1. Standard Turing machines
simulate Semi-Infinite machines

2. Semi-Infinite Machines
simulate Standard Turing machines

1. Standard Turing machines simulate Semi-Infinite machines:

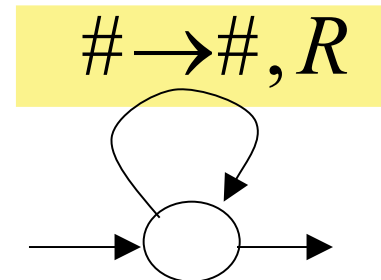


Standard Turing Machine

Semi-Infinite machine modifications

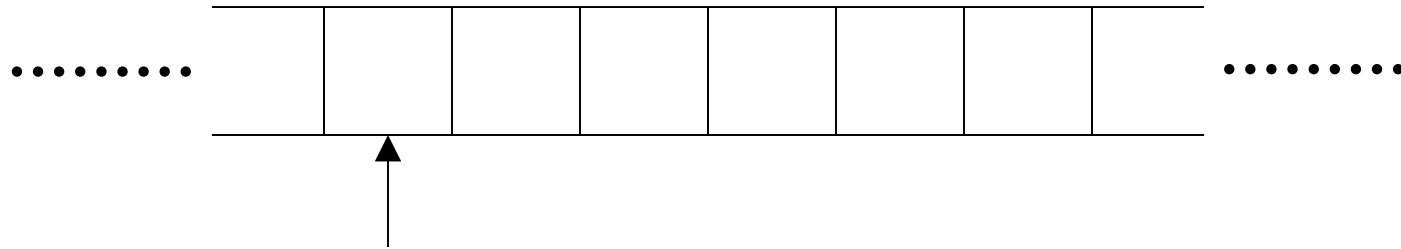
a. insert special symbol $\#$
at left of input string

b. Add a self-loop
to every state
(except states with no
outgoing transitions)

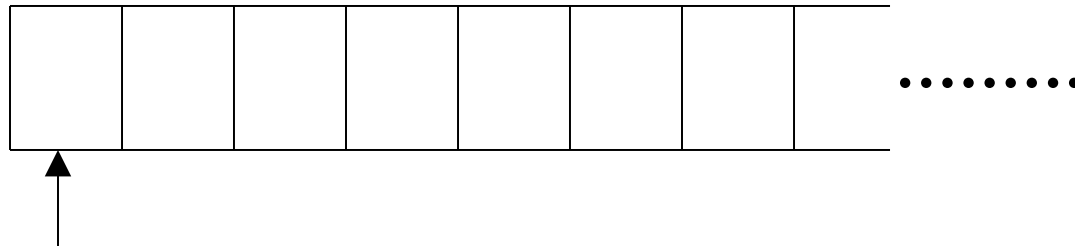


2. Semi-Infinite tape machines simulate Standard Turing machines:

Standard machine



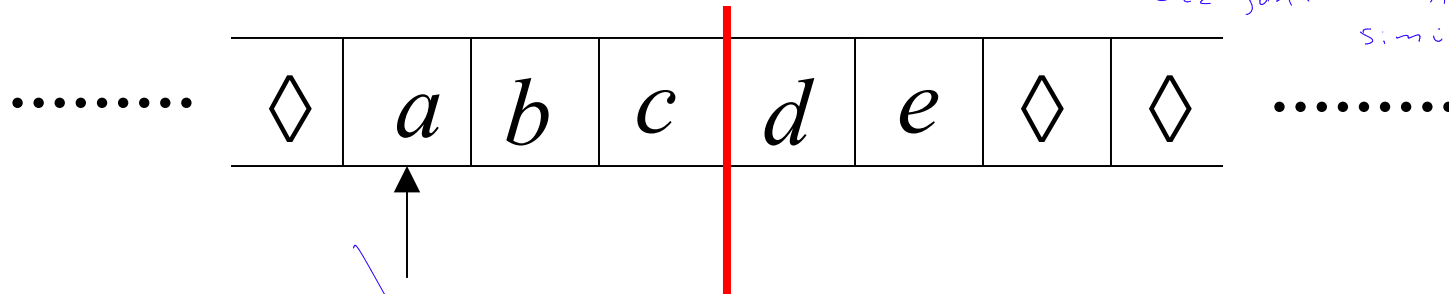
Semi-Infinite tape machine



Squeeze infinity of both directions
to one direction

Standard machine

→ Sensez makineyle
Ekle yun sonlu makine
simüle edilebilir



reference point

Bu yaklaşımla simüle edilebilir: 2

Bu yaklaşım

Semi-Infinite tape machine with two tracks

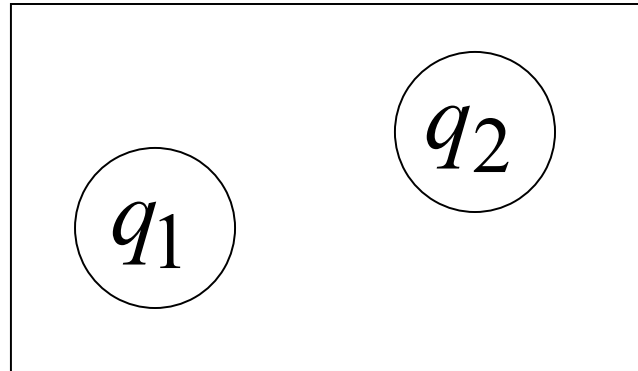
Right part

Left part

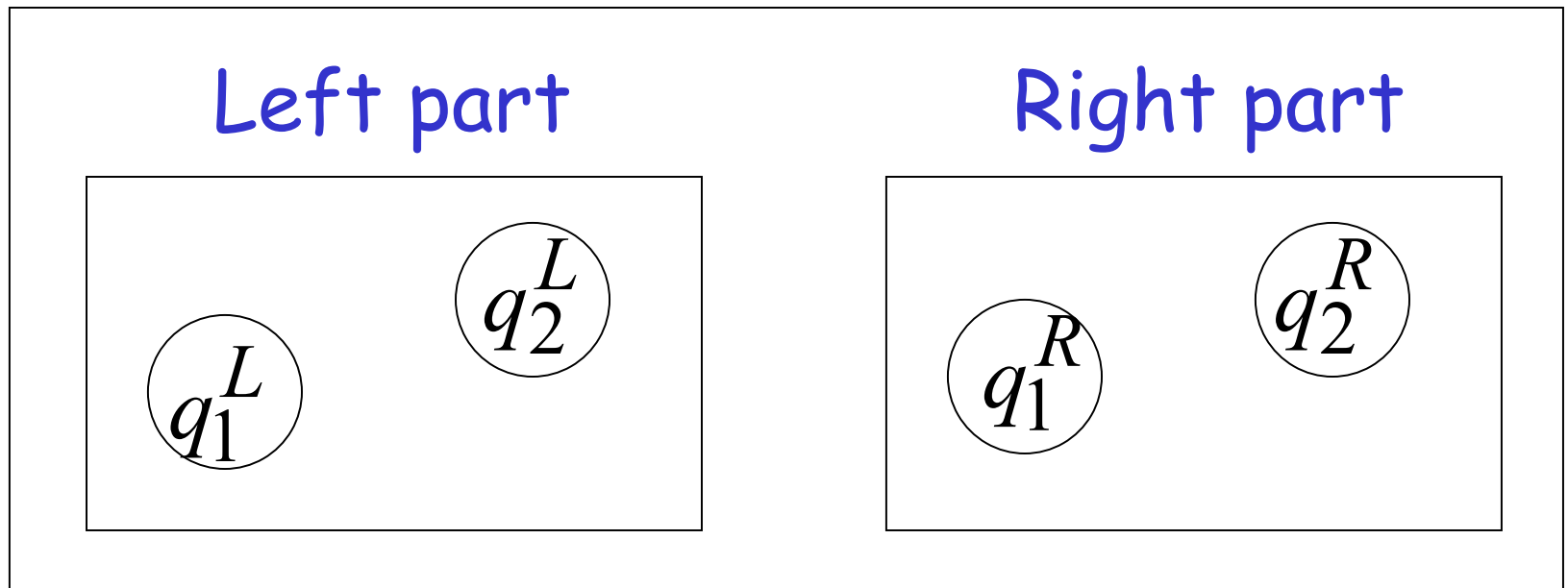
#	d	e	◇	◇	◇	
#	c	b	a	◇	◇	

.....

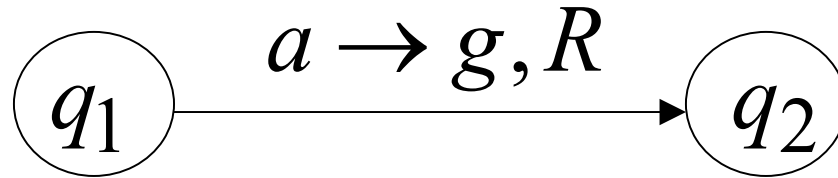
Standard machine



Semi-Infinite tape machine

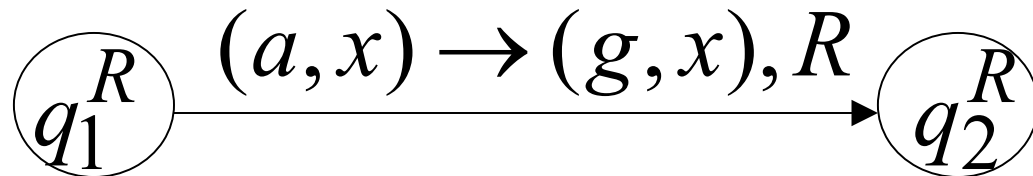


Standard machine

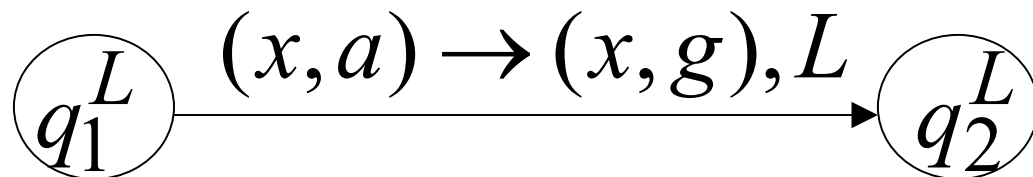


Semi-Infinite tape machine

Right part



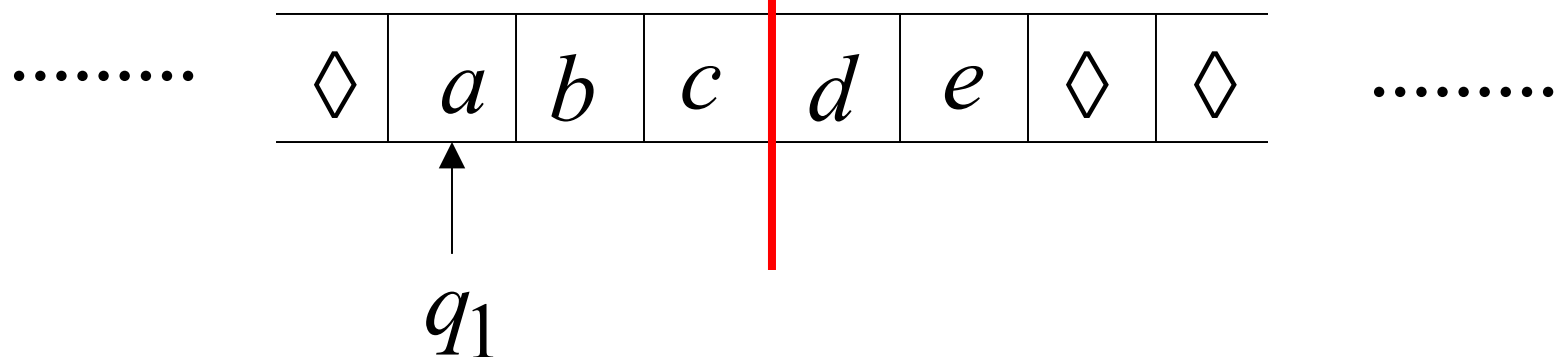
Left part



For all tape symbols x

Time 1

Standard machine



Semi-Infinite tape machine

Right part

#	<i>d</i>	<i>e</i>	◇	◇	◇	
---	----------	----------	---	---	---	--

.....

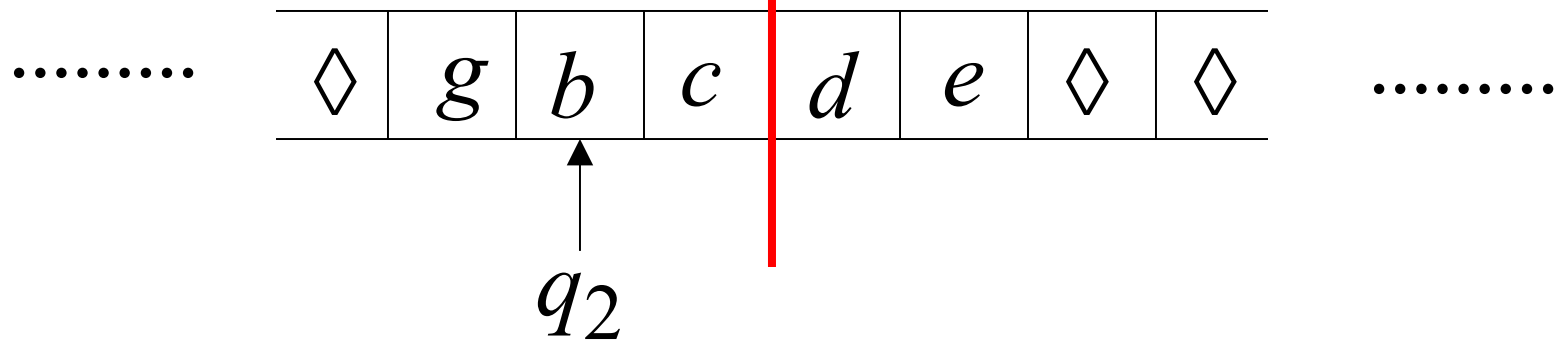
Left part

#	<i>c</i>	<i>b</i>	<i>a</i>	◇	◇	
---	----------	----------	----------	---	---	--

q_1^L

Time 2

Standard machine



Semi-Infinite tape machine

Right part

#	<i>d</i>	<i>e</i>	◇	◇	◇	
---	----------	----------	---	---	---	--

.....

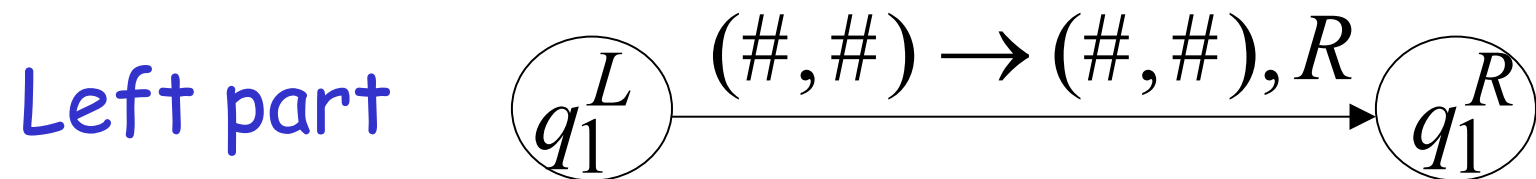
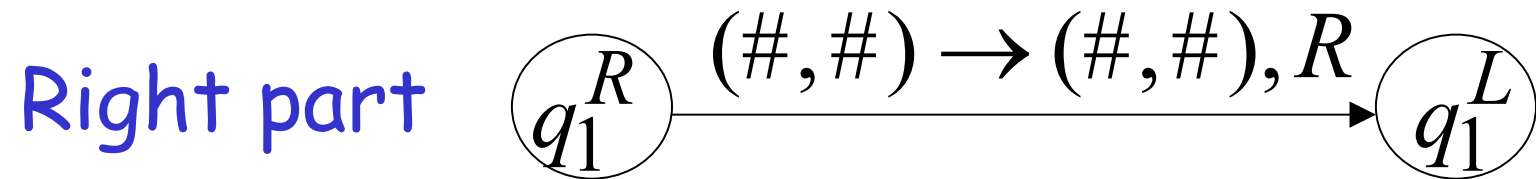
Left part

#	<i>c</i>	<i>b</i>	<i>g</i>	◇	◇	
---	----------	----------	----------	---	---	--

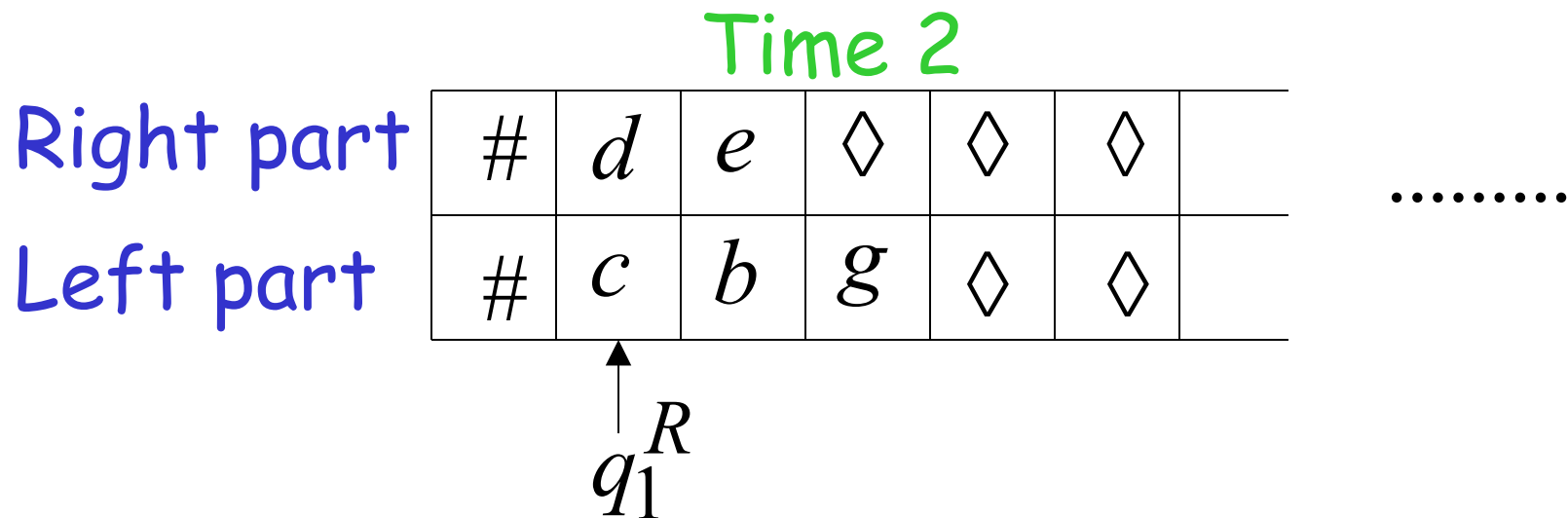
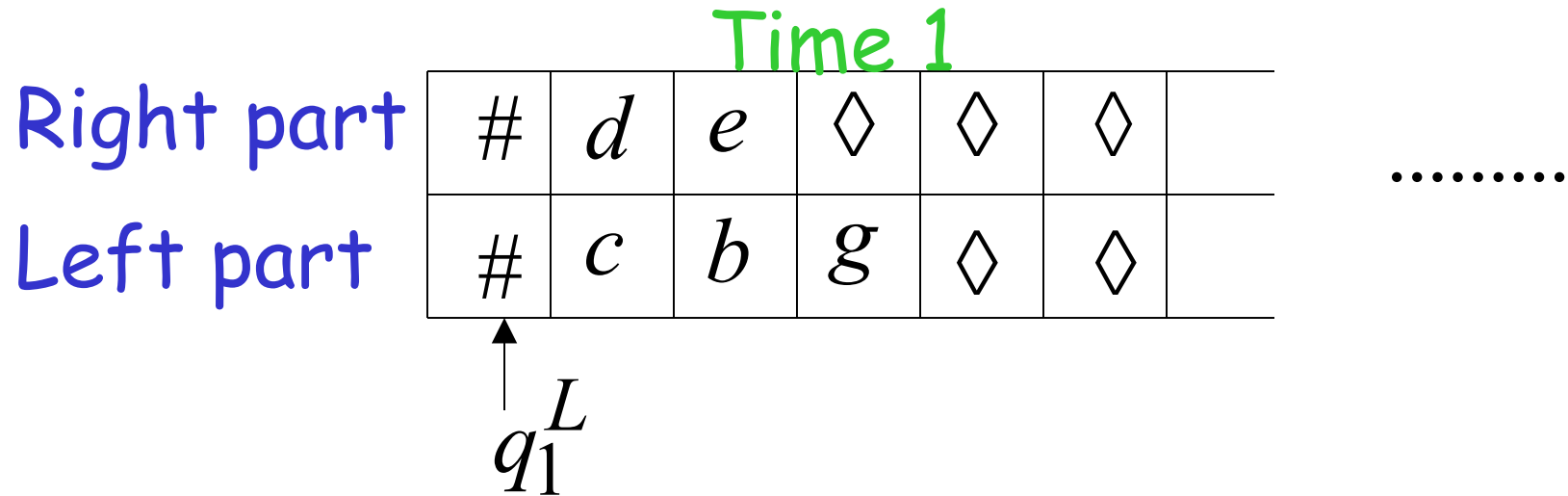
q_2^L

At the border:

Semi-Infinite tape machine

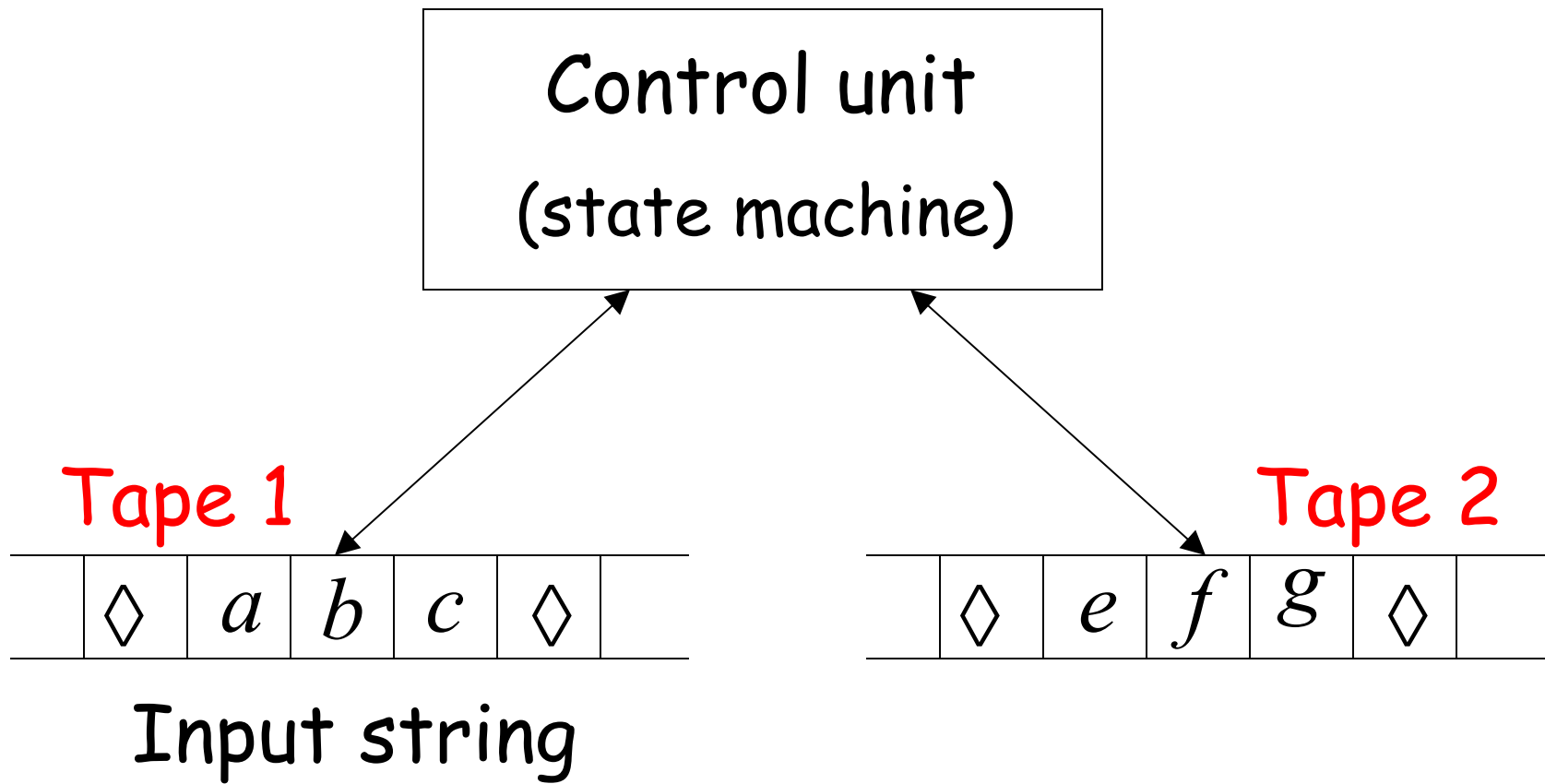


Semi-Infinite tape machine

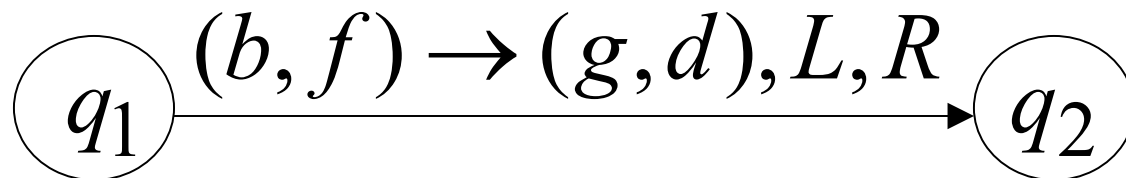
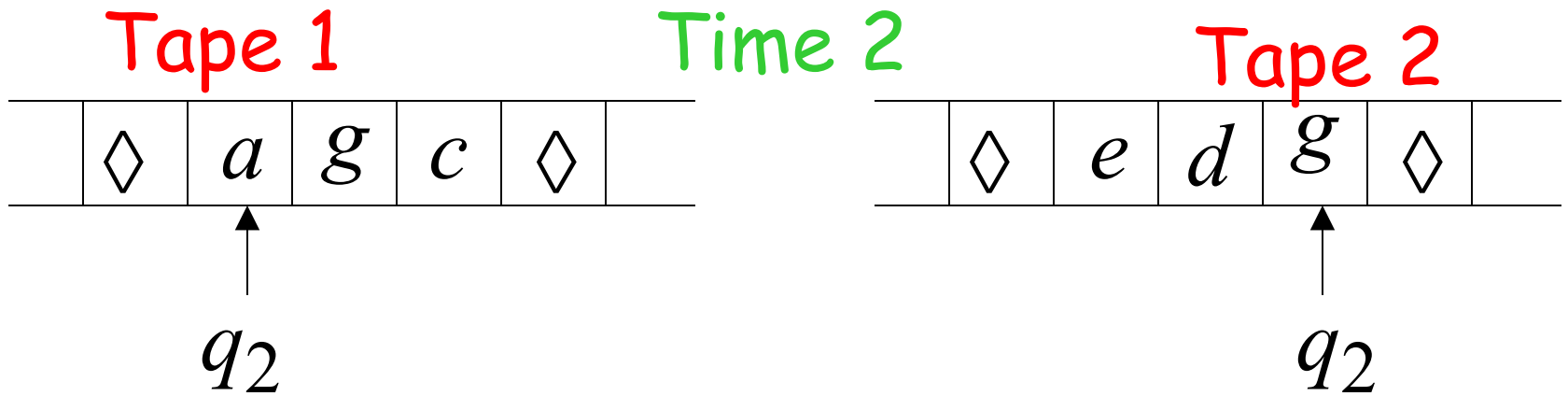
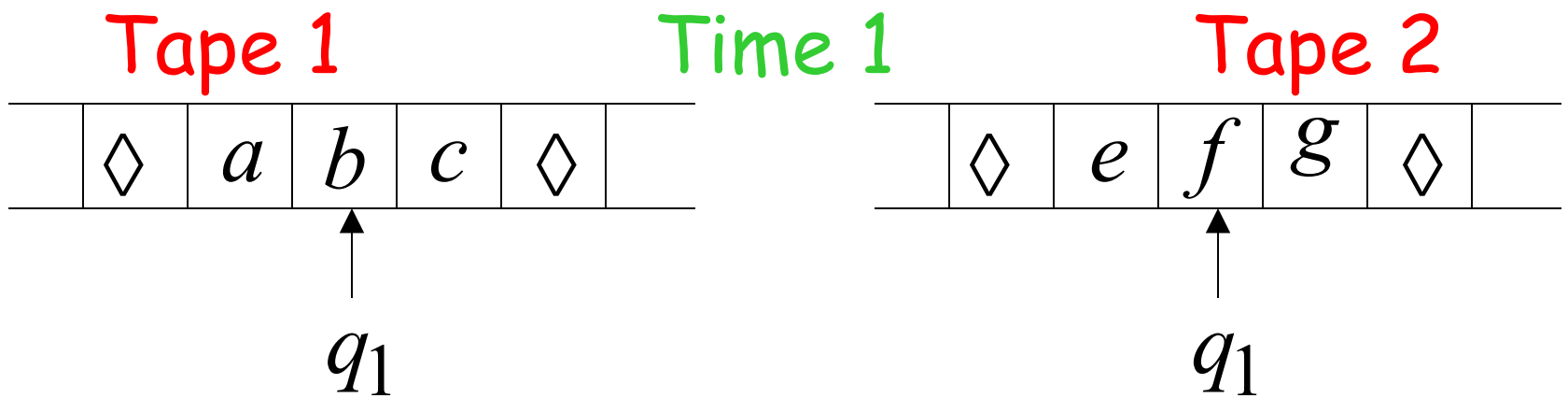


END OF PROOF

Multi-tape Turing Machines



Input string appears on Tape 1



Theorem: Multi-tape machines
have the same power with
Standard Turing machines

Proof: 1. Multi-tape machines
simulate Standard Turing machines

2. Standard Turing machines
simulate Multi-tape machines

1. Multi-tape machines simulate Standard Turing Machines:

Trivial: Use one tape

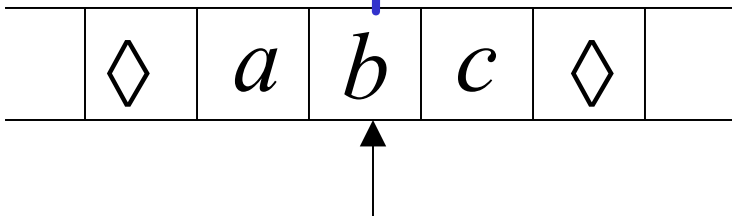
2. Standard Turing machines simulate Multi-tape machines:

Standard machine:

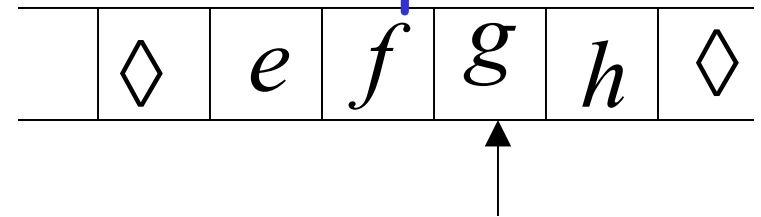
- Uses a multi-track tape to simulate the multiple tapes
- A tape of the Multi-tape machine corresponds to a pair of tracks

Multi-tape Machine

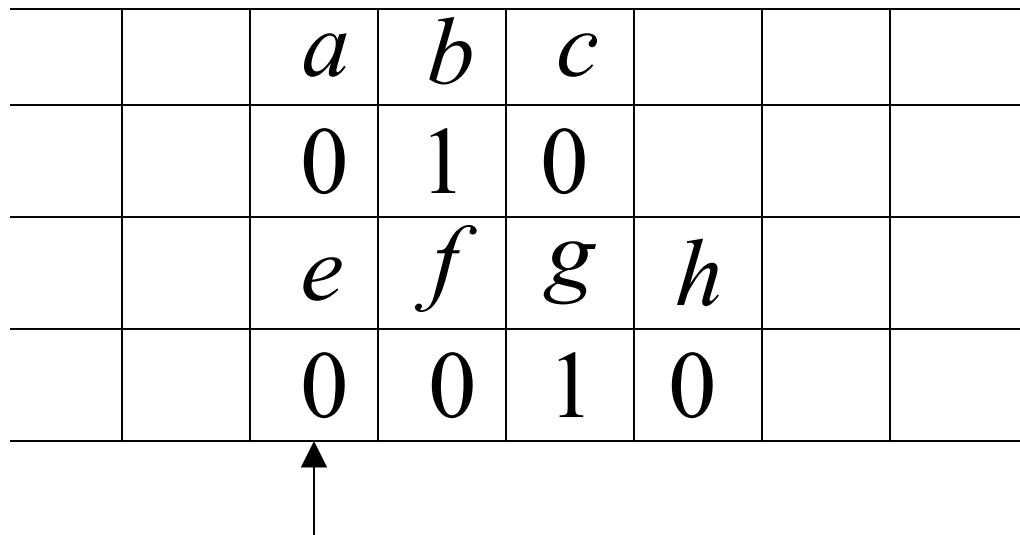
Tape 1



Tape 2



Standard machine with four track tape



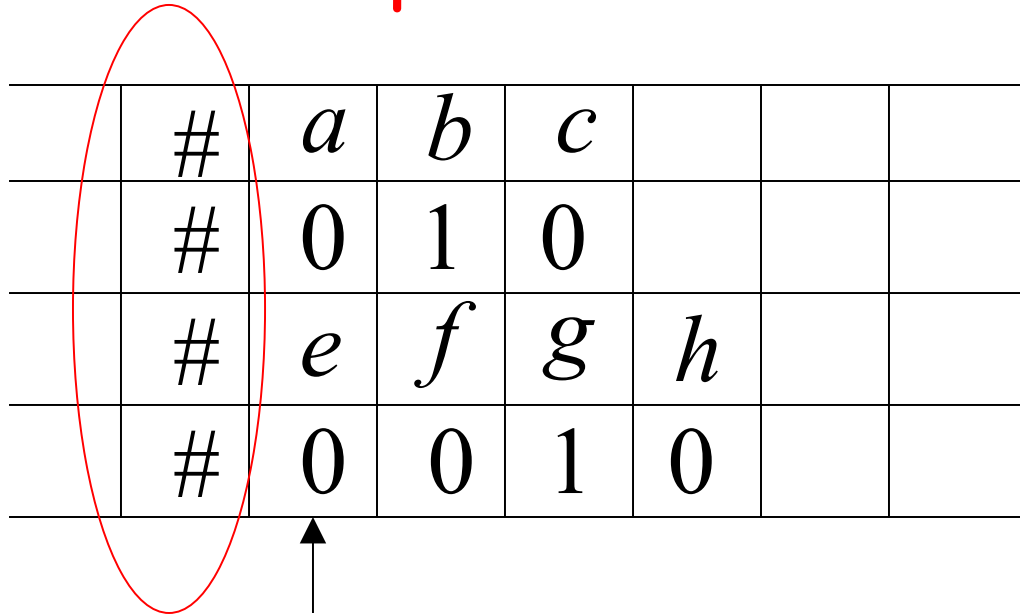
Tape 1

head position

Tape 2

head position

Reference point



The diagram shows a 4x8 grid representing a multi-tape Turing machine tape. The first column contains the '#' symbol in all four rows. The second column contains 'a', '0', 'e', and '0'. The third column contains 'b', '1', 'f', and '0'. The fourth column contains 'c', '0', 'g', and '1'. The fifth column contains 'h' in the third row. A red oval encircles the first column, and a black arrow points to the bottom of this oval, indicating the reference point.

#	a	b	c				
#	0	1	0				
#	e	f	g	h			
#	0	0	1	0			

Tape 1

head position

Tape 2

head position

Repeat for each Multi-tape state transition:

1. Return to reference point 1. dan geyzer, update sonra digeri update edip agni adimlere ugguluyor.
2. Find current symbol in Track 1 and update
3. Return to reference point
4. Find current symbol in Tape 2 and update

END OF PROOF

Same power doesn't imply same speed:

$$L = \{a^n b^n\}$$

↓ unrecognizable

Standard Turing machine: $O(n^2)$ time

Go back and forth $O(n^2)$ times
to match the a's with the b's

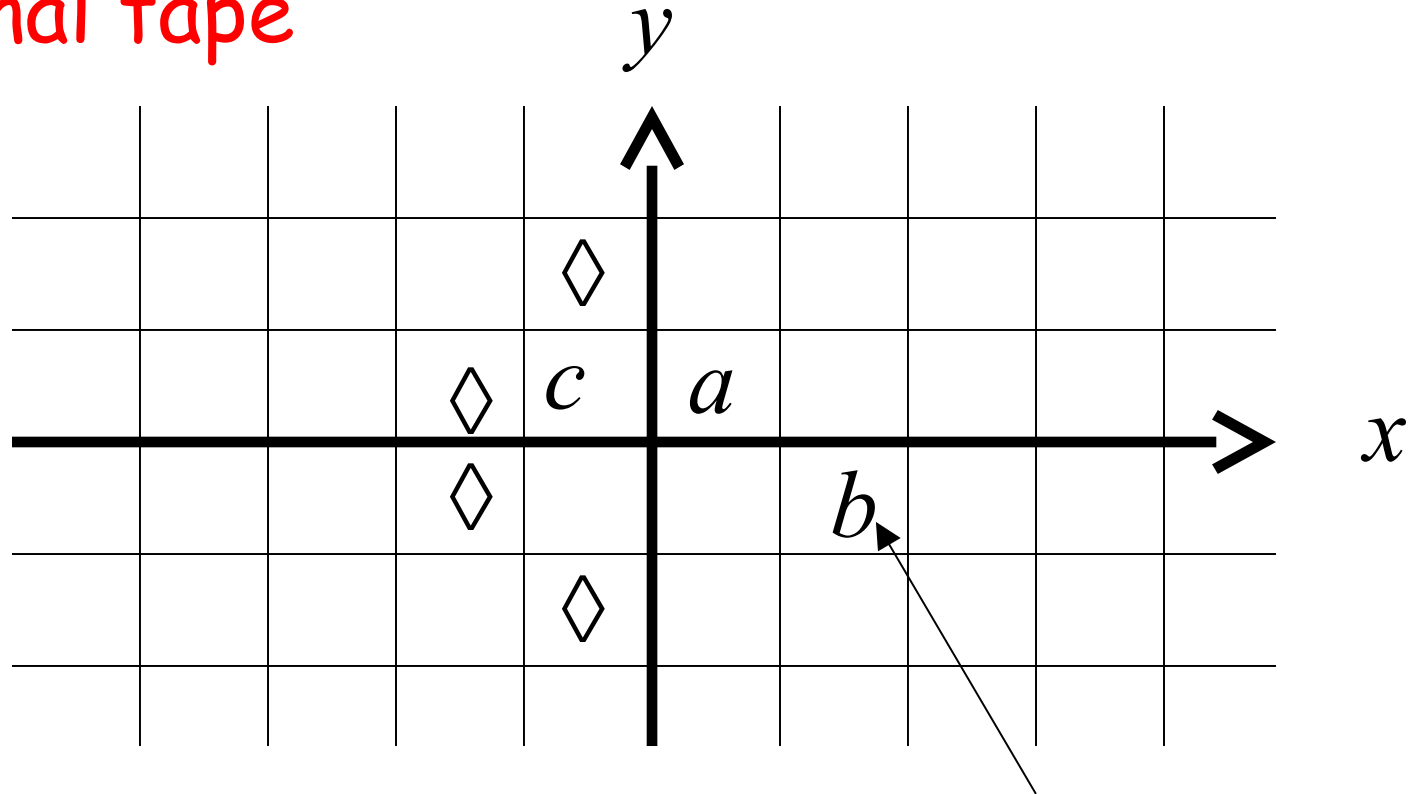
2-tape machine: $O(n)$ time

1. Copy b^n to tape 2 ($O(n)$ steps)

2. Compare a^n on tape 1
and b^n on tape 2 ($O(n)$ steps)

Multidimensional Turing Machines

2-dimensional tape



MOVES: L,R,U,D

U: up D: down

HEAD

Position: +2, -1

Theorem: Multidimensional machines
have the same power with
Standard Turing machines

Proof: 1. Multidimensional machines
simulate Standard Turing machines

2. Standard Turing machines
simulate Multi-Dimensional machines

1. Multidimensional machines simulate Standard Turing machines

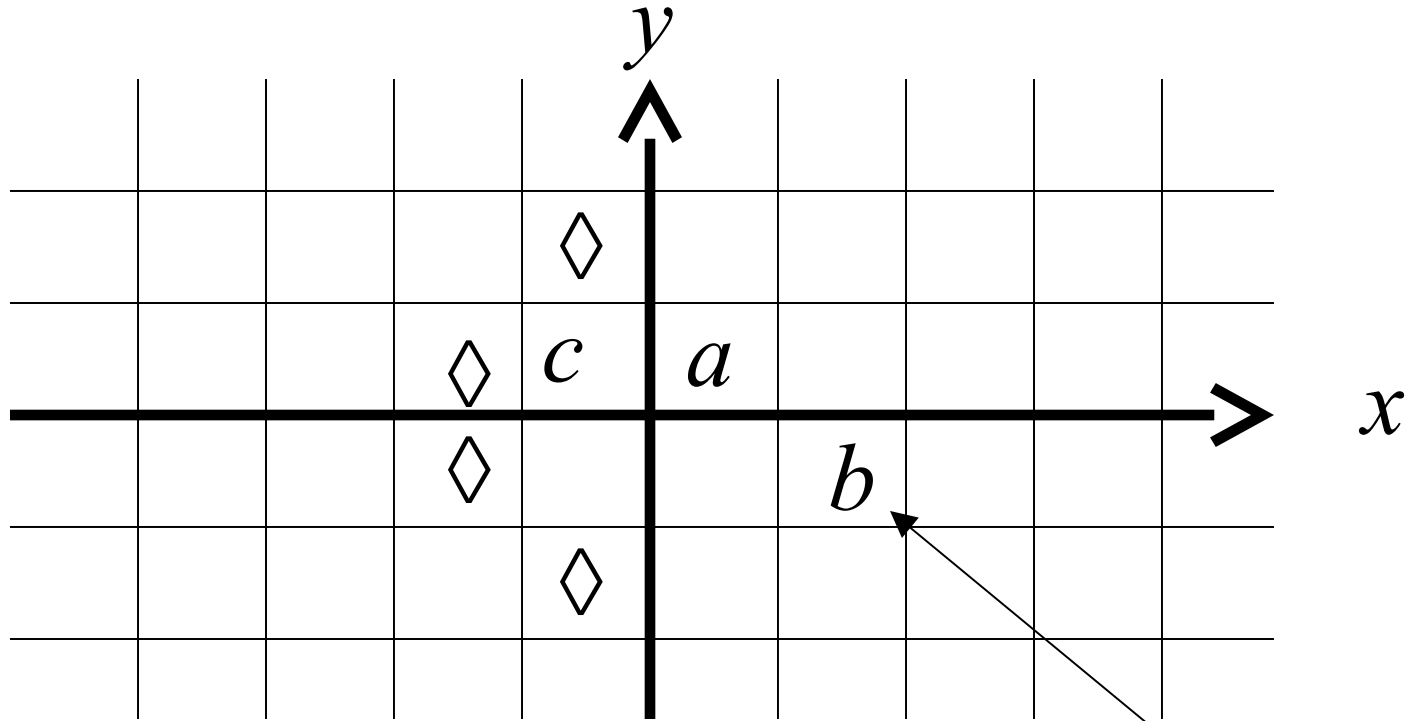
Trivial: Use one dimension

2. Standard Turing machines simulate Multidimensional machines

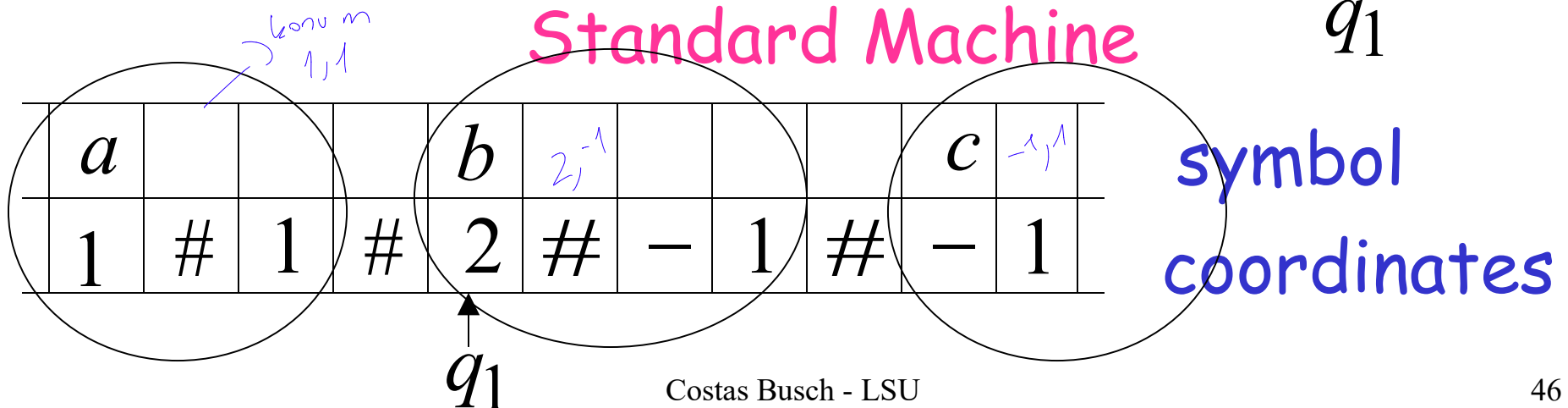
Standard machine:

- Use a two track tape
- Store symbols in track 1
- Store coordinates in track 2

2-dimensional machine



Standard Machine



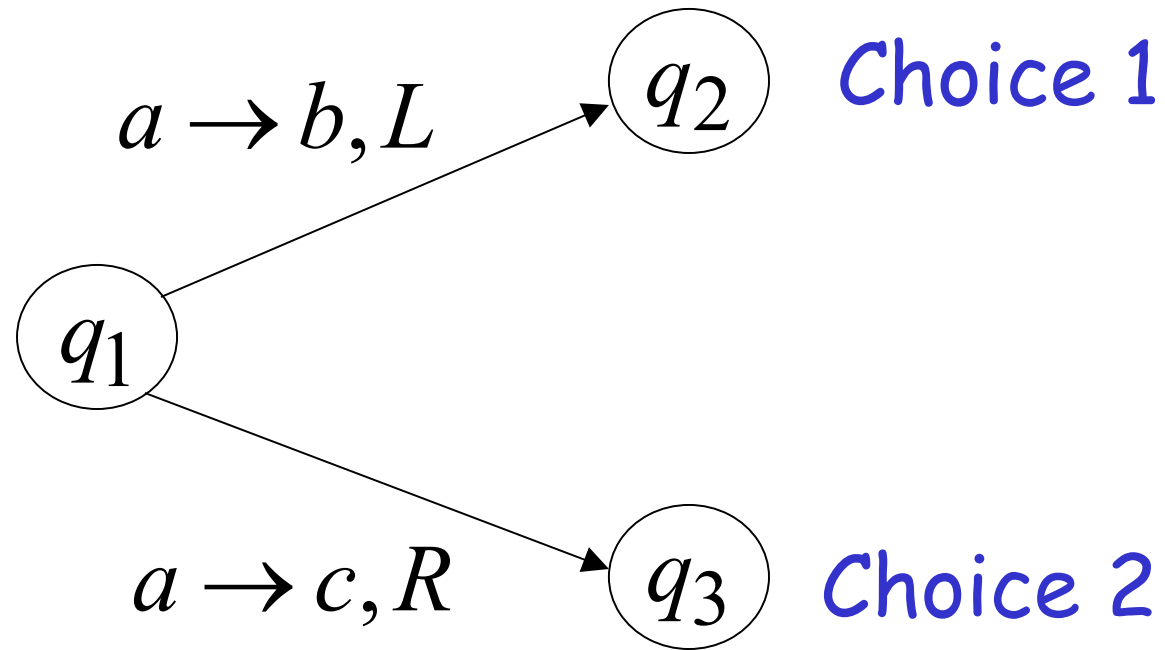
Standard machine:

Repeat for each transition followed in the 2-dimensional machine:

1. Update current symbol
2. Compute coordinates of next position
3. Find next position on tape

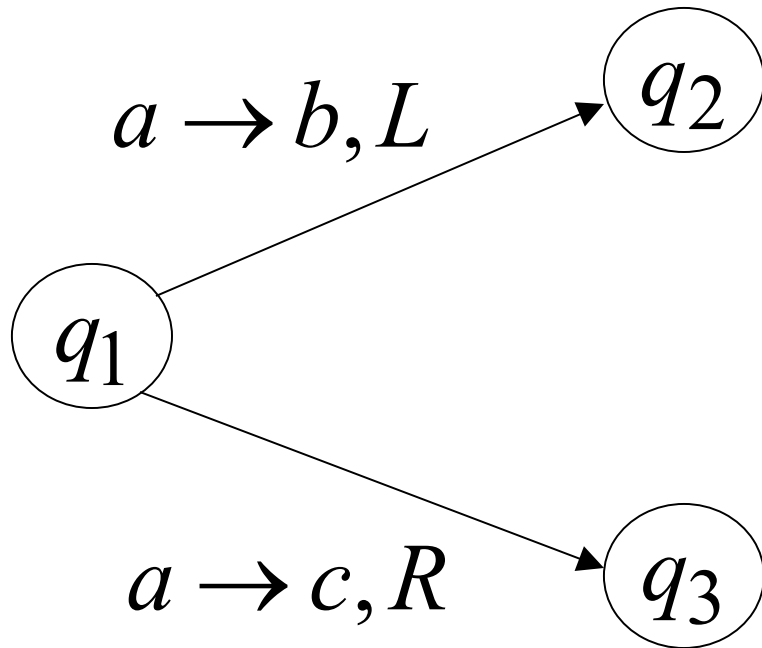
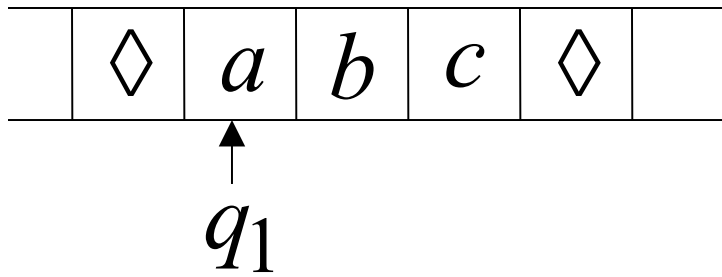
END OF PROOF

Nondeterministic Turing Machines



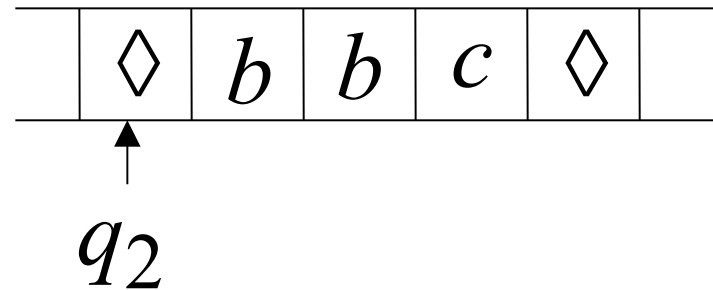
Allows Non Deterministic Choices

Time 0

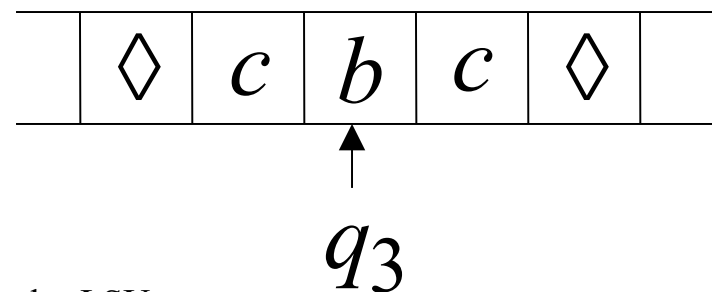


Time 1

Choice 1



Choice 2



Input string w is accepted if
there is a computation:

*

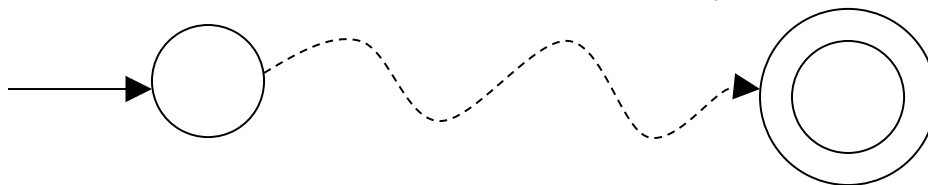
$$q_0 w \succ x q_f y$$

Initial configuration

Final Configuration

Any accept state

There is a computation:



Theorem: Nondeterministic machines
have the same power with
Standard Turing machines

- Proof:**
1. Nondeterministic machines
simulate Standard Turing machines
 2. Standard Turing machines
simulate Nondeterministic machines

1. Nondeterministic Machines simulate Standard (deterministic) Turing Machines

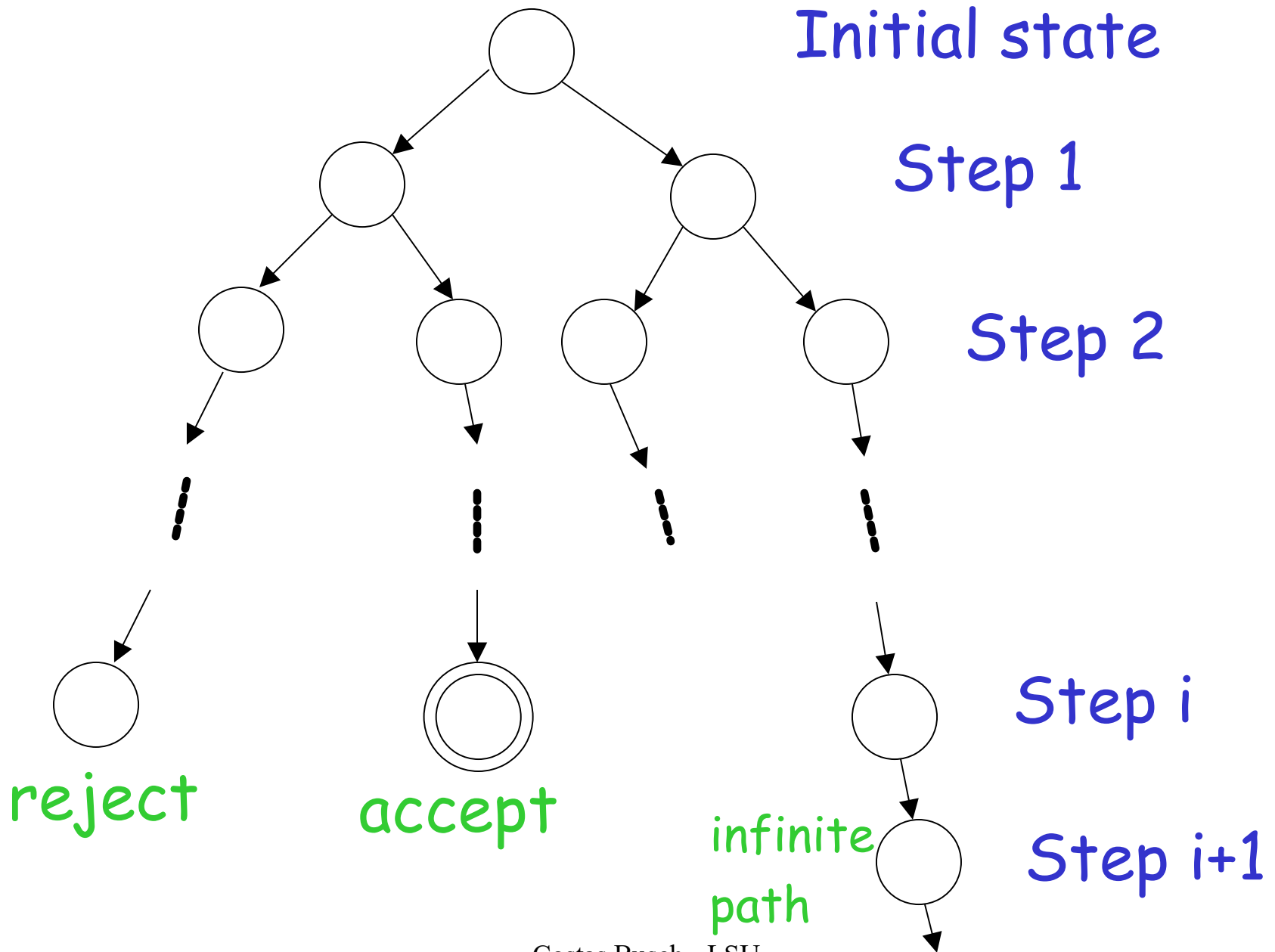
Trivial: every deterministic machine
is also nondeterministic

2. Standard (deterministic) Turing machines simulate Nondeterministic machines:

Deterministic machine:

- Uses a 2-dimensional tape
(equivalent to standard Turing machine with one tape)
- Stores all possible computations of the non-deterministic machine on the 2-dimensional tape

All possible computation paths

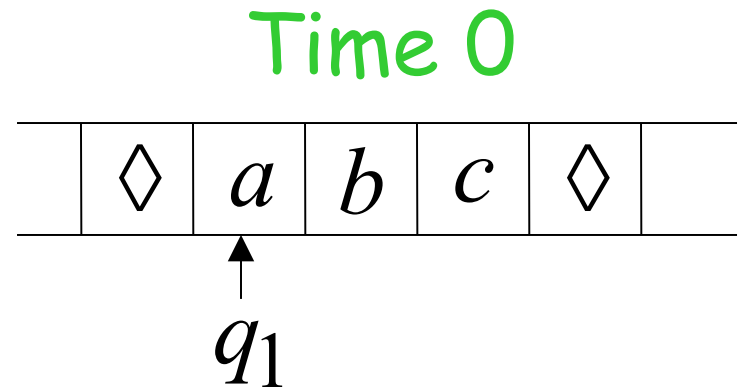
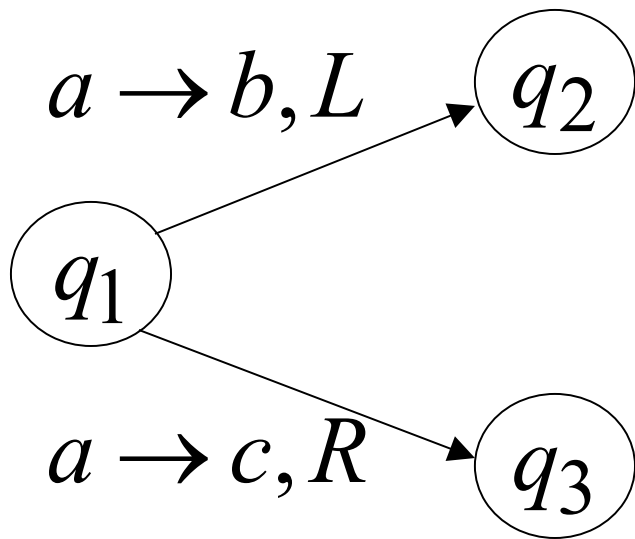


The Deterministic Turing machine simulates all possible computation paths:

- simultaneously
- step-by-step
- with **breadth-first** search

depth-first may result getting stuck at exploring
an infinite path before discovering the accepting path

NonDeterministic machine



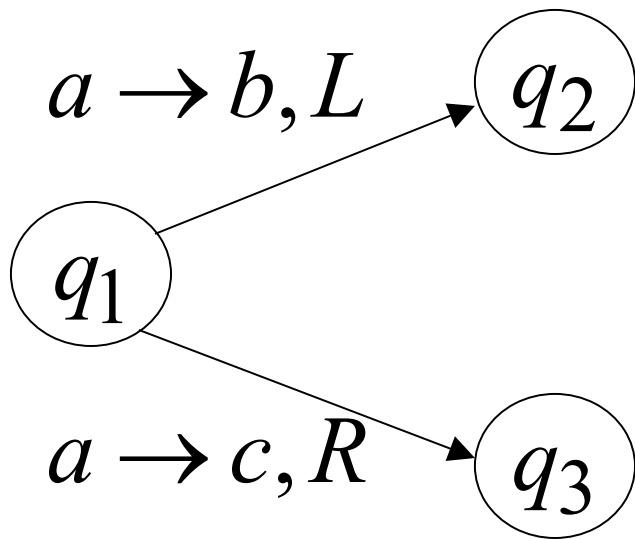
Deterministic machine

	#	#	#	#	#	#	
	#	<i>a</i>	<i>b</i>	<i>c</i>	#		
	#	<i>q</i> ₁			#		
	#	#	#	#	#		

current
configuration

NonDeterministic machine

Time 1



	◇	<i>b</i>	<i>b</i>	<i>c</i>	◇	
--	---	----------	----------	----------	---	--

Choice 1

q_2

	◇	<i>c</i>	<i>b</i>	<i>c</i>	◇	
--	---	----------	----------	----------	---	--

Choice 2

q_3

Deterministic machine

	#	#	#	#	#	#	
#		<i>b</i>	<i>b</i>	<i>c</i>	#		
#	q_2				#		
#		<i>c</i>	<i>b</i>	<i>c</i>	#		
#			q_3		#		

Computation 1

Computation 2

Deterministic Turing machine

Repeat

For each configuration in current step of non-deterministic machine,
if there are two or more choices:

1. Replicate configuration
2. Change the state in the replicas

Until either the input string is accepted
or rejected in all configurations

If the non-deterministic machine accepts the input string:

The deterministic machine accepts and halts too

The simulation takes in the worst case exponential time compared to the shortest length of an accepting path

If the non-deterministic machine does not accept the input string:

1. The simulation halts if all paths reach a halting state

OR

2. The simulation never terminates if there is a never-ending path (infinite loop)

In either case the deterministic machine rejects too (1. by halting or 2. by simulating the infinite loop)

END OF PROOF

A Universal Turing Machine

Turing Machines
agilité ou

A limitation of Turing Machines:

Turing Machines are "hardwired"

they execute

only one program

Real Computers are re-programmable

Solution: Universal Turing Machine

Attributes:

- Reprogrammable machine
- Simulates any other Turing Machine

Universal Turing Machine
simulates any Turing Machine M

Input of Universal Turing Machine:

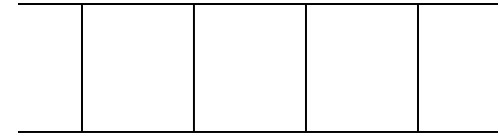
Description of transitions of M

Input string of M

Three tapes



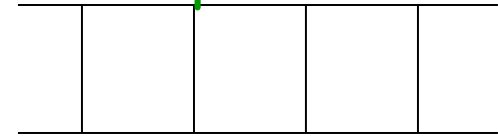
Tape 1



Description of M

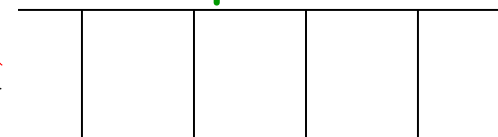
*diger makinenin formal tiple
vermek lazim*

Tape 2



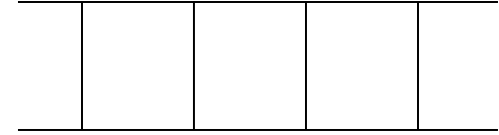
Tape Contents of M

Tape 3



State of M

Tape 1



Description of M

We describe Turing machine M
as a string of symbols:

We encode M as a string of symbols

Alphabet Encoding

Symbols:

a

b

c

d

...



Encoding:

1

11

111

1111

State Encoding

States: q_1 q_2 q_3 q_4 $\dots$



Encoding: 1 11 111 1111

Head Move Encoding

Move: L R



Encoding: 1 11

Transition Encoding

Transition: $\delta(q_1, a) = (q_2, b, L)$

Encoding:

1 0 1 0 1 1 0 1 1 0 1

separator

Turing Machine Encoding

Transitions:

$$\delta(q_1, a) = (q_2, b, L)$$

$$\delta(q_2, b) = (q_3, c, R)$$

Encoding:

1 0 1 0 1 1 0 1 1 0 1 0 0 1 1 1 0 1 1 1 0 1 1

separator

Tape 1 contents of Universal Turing Machine:

binary encoding
of the simulated machine M

Tape 1

1 0 1 0 11 0 11 0 10011 0 1 10 111 0 111 0 1100...



A Turing Machine is described
with a binary string of 0's and 1's

Therefore:

The set of Turing machines
forms a language:

each string of this language is
the binary encoding of a Turing Machine

Language of Turing Machines

$L = \{$ 1010110101, (Turing Machine 1)
101011101011, (Turing Machine 2)
1110101111010111,
..... }

Countable Sets

Infinite sets are either:

Countable

or

Uncountable

Countable set:

There is a one to one correspondence (injection)
of
elements of the set
to
Positive integers (1,2,3,...)

Every element of the set is mapped to a positive number
such that no two elements are mapped to same number

Example: The set of even integers
is countable

Even integers:
(positive) 0, 2, 4, 6, ...

Correspondence:

Positive integers: 1, 2, 3, 4, ...

$2n$ corresponds to $n + 1$

Example: The set of rational numbers
is countable

Rational numbers: $\frac{1}{2}, \frac{3}{4}, \frac{7}{8}, \dots$

Naive Approach

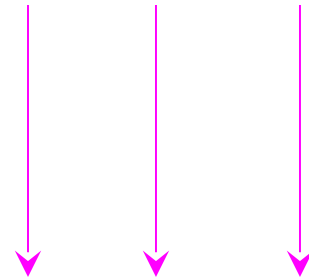
Rational numbers:

Correspondence:

Positive integers:

Nominator 1

$$\frac{1}{1}, \frac{1}{2}, \frac{1}{3}, \dots$$



$$1, 2, 3, \dots$$

Doesn't work:

we will never count

numbers with nominator 2:

$$\frac{2}{1}, \frac{2}{2}, \frac{2}{3}, \dots$$

Better Approach

$$\frac{1}{1} \qquad \frac{1}{2} \qquad \frac{1}{3} \qquad \frac{1}{4} \qquad \dots$$

$$\frac{2}{1} \qquad \frac{2}{2} \qquad \frac{2}{3} \qquad \dots$$

$$\frac{3}{1} \qquad \frac{3}{2} \qquad \dots$$

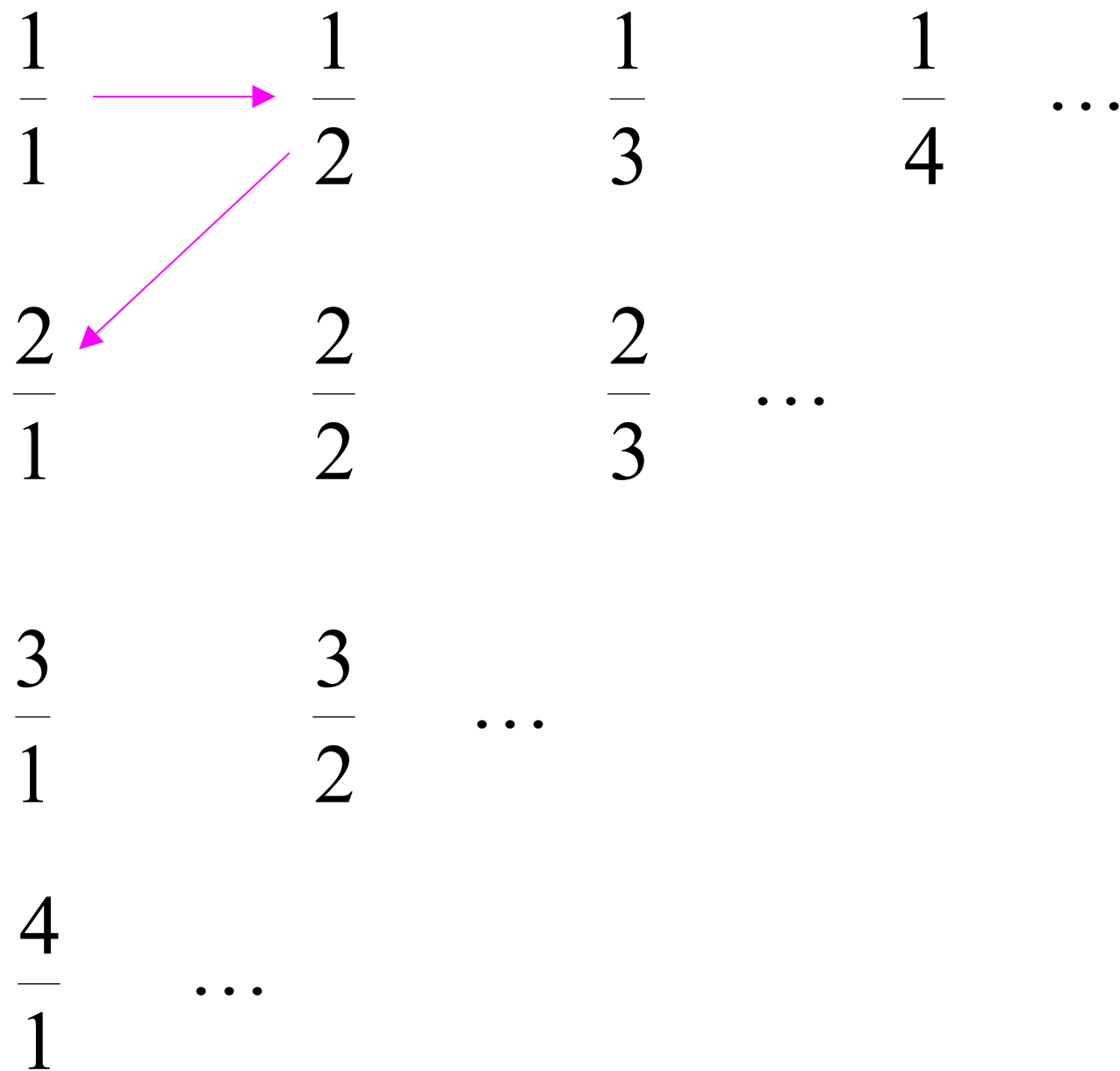
$$\frac{4}{1} \qquad \dots$$

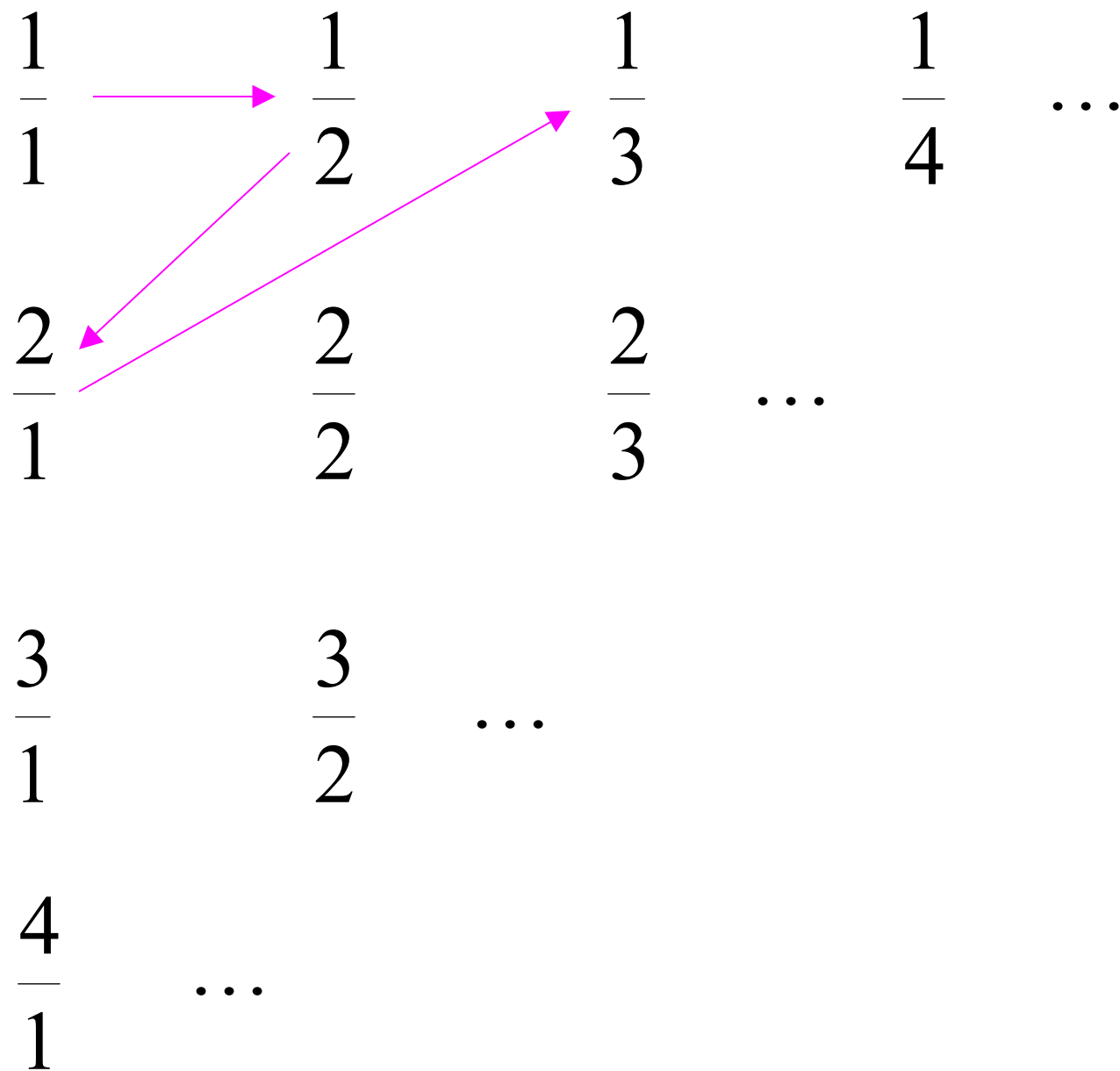
$$\frac{1}{1} \xrightarrow{\text{pink arrow}} \frac{1}{2} \quad \frac{1}{3} \quad \frac{1}{4} \quad \dots$$

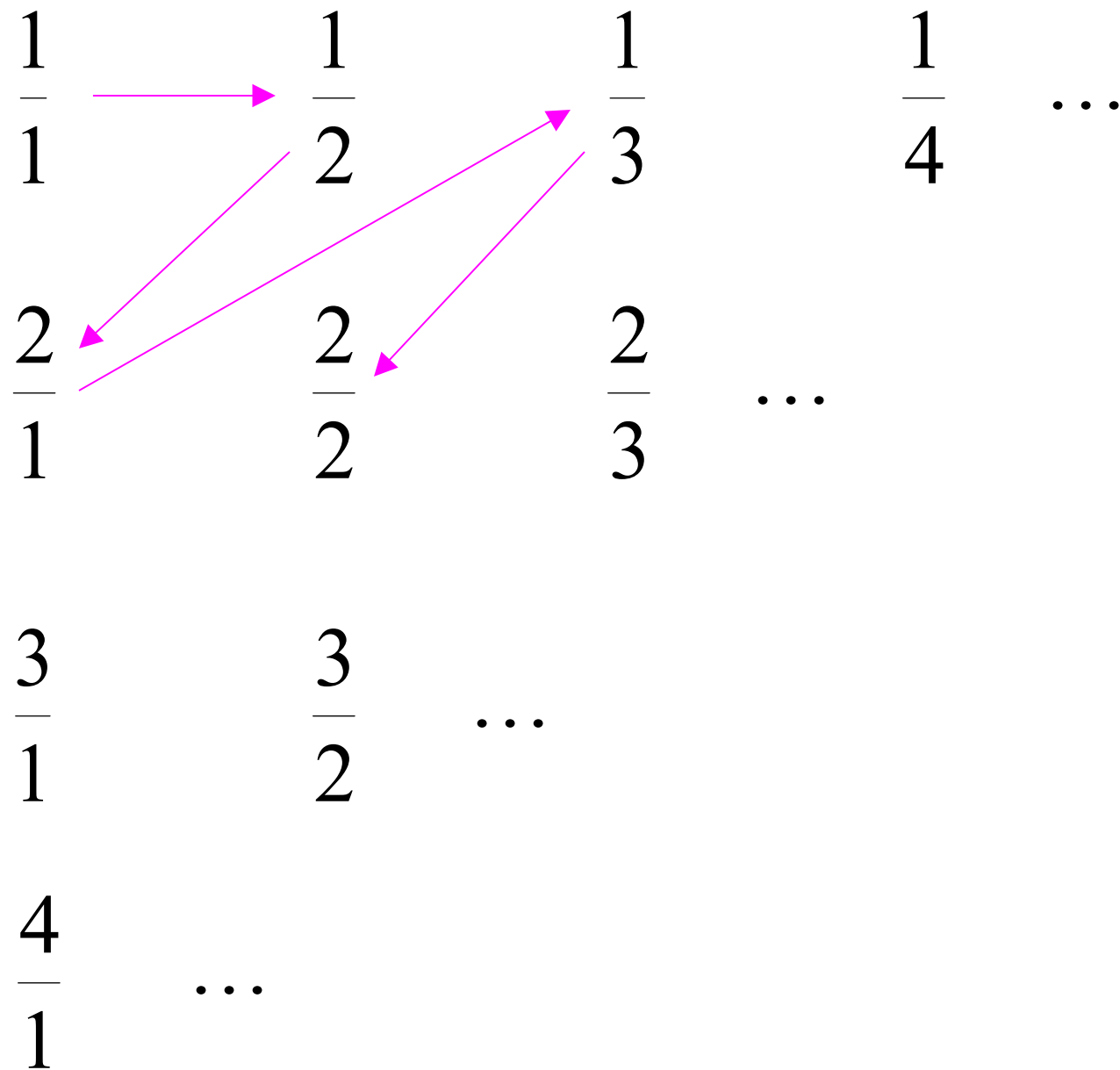
$$\frac{2}{1} \quad \frac{2}{2} \quad \frac{2}{3} \quad \dots$$

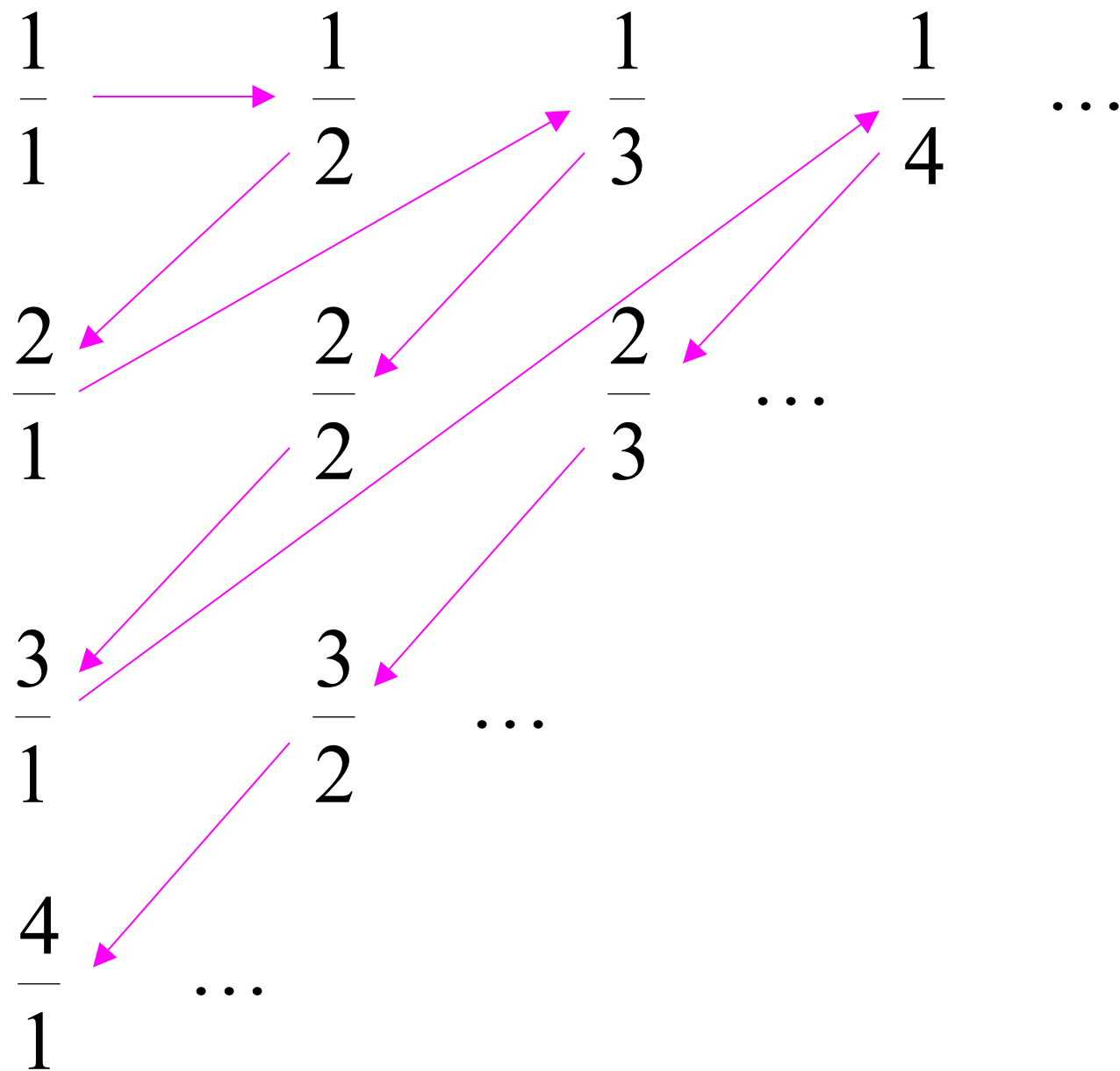
$$\frac{3}{1} \quad \frac{3}{2} \quad \dots$$

$$\frac{4}{1} \quad \dots$$









Rational Numbers:

$$\frac{1}{1}, \frac{1}{2}, \frac{2}{1}, \frac{1}{3}, \frac{2}{2}, \dots$$

Correspondence:

Positive Integers:

$$1, 2, 3, 4, 5, \dots$$

We proved:

the set of rational numbers is countable
by describing an enumeration procedure
(enumerator)
for the correspondence to natural numbers

Definition

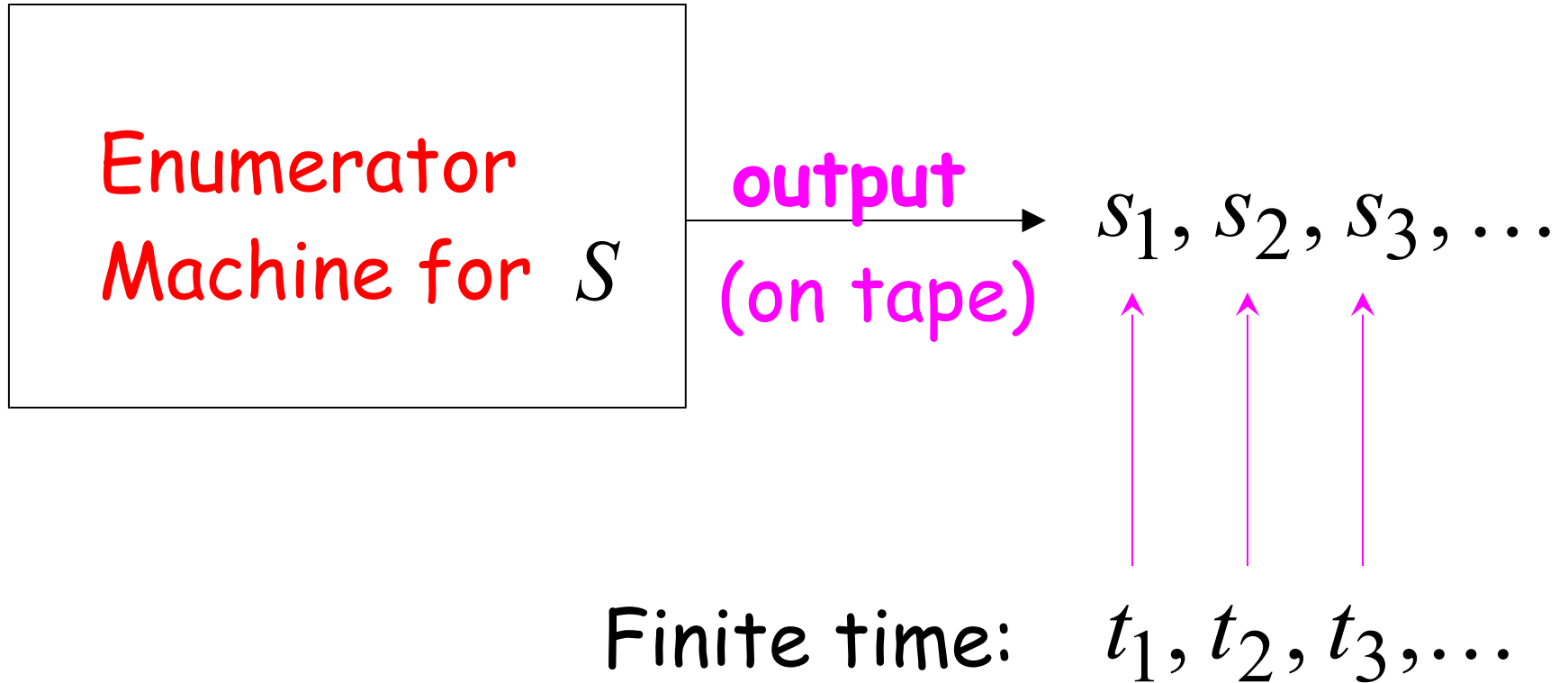
Let S be a set of strings (Language)

An **enumerator** for S is a Turing Machine
that generates (prints on tape)
all the strings of S one by one

and

each string is generated in finite time

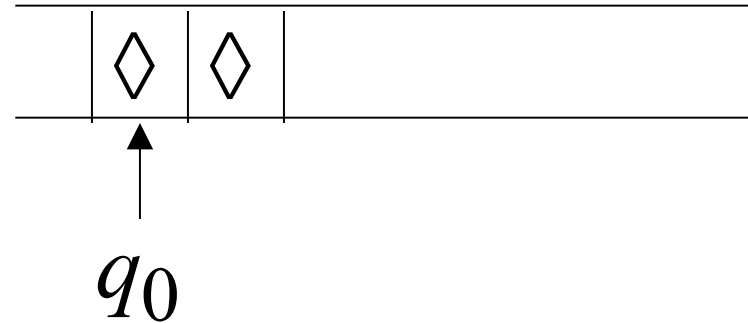
strings $s_1, s_2, s_3, \dots \in S$



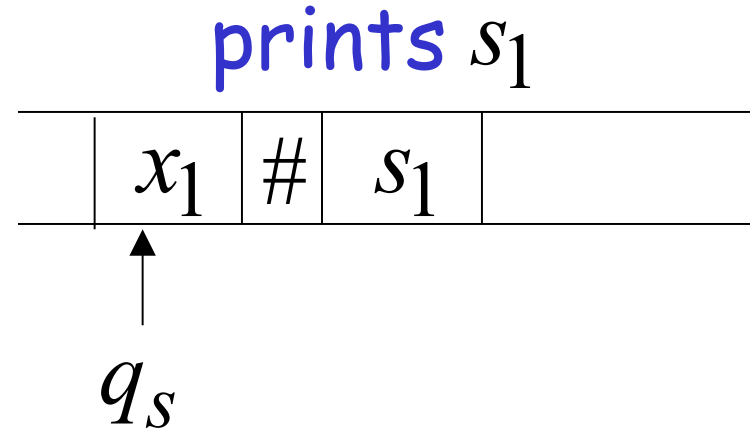
Enumerator Machine

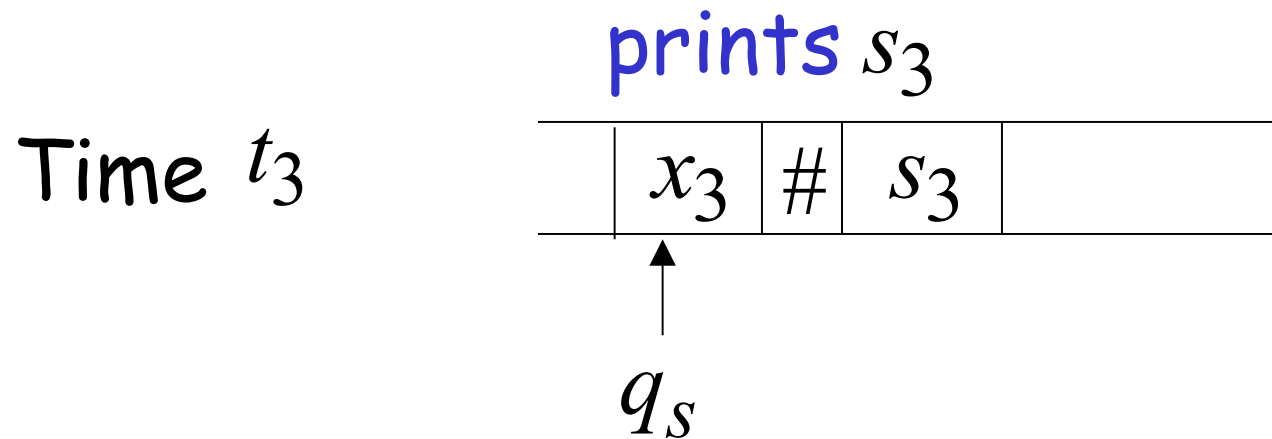
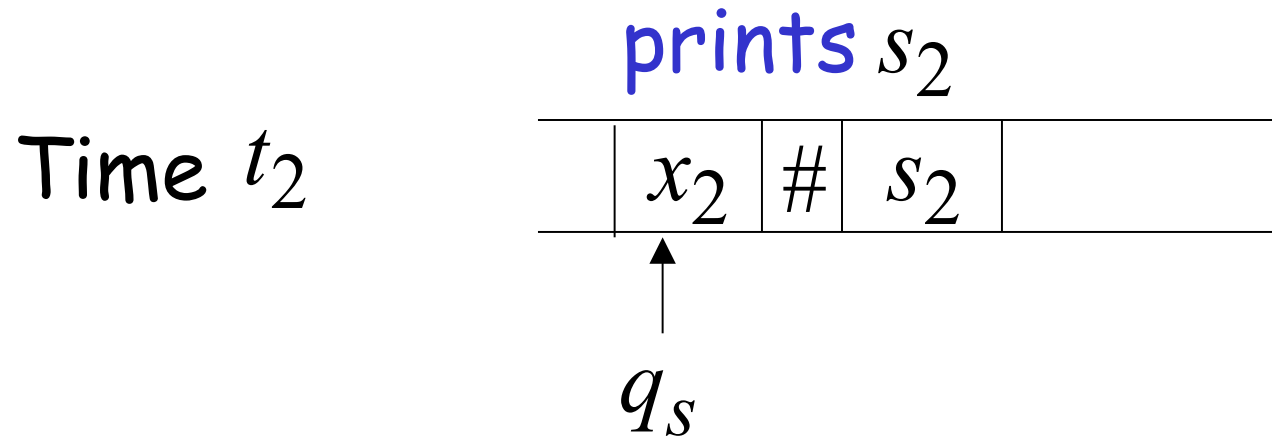
Configuration

Time 0



Time t_1





Observation:

If for a set S there is an enumerator,
then the set is countable

↳ B.2: diil iuin enumerator varsa
countable...

The enumerator describes the
correspondence of S to natural numbers

Example: The set of strings $S = \{a,b,c\}^+$
is countable

Approach:

We will describe an enumerator for S

Naive enumerator:

Produce the strings in lexicographic order:

$$s_1 = a$$

$$s_2 = aa$$

$$\vdots \quad aaa$$

$$aaaa$$

.....

Doesn't work:

strings starting with b
will never be produced

Better procedure: **Proper Order** (Canonical Order)

1. Produce all strings of length 1
2. Produce all strings of length 2
3. Produce all strings of length 3
4. Produce all strings of length 4
- ⋮

Produce strings in
Proper Order:

$s_1 =$	<i>a</i>	}	length 1
$s_2 =$	<i>b</i>		
$\vdots$	<i>c</i>		
	<i>aa</i>	}	length 2
	<i>ab</i>		
	<i>ac</i>		
	<i>ba</i>		
	<i>bb</i>		
	<i>bc</i>		
	<i>ca</i>		
	<i>cb</i>		
	<i>cc</i>		
	<i>aaa</i>	}	length 3
	<i>aab</i>		
	<i>aac</i>		
	<i>.....</i>		

Theorem: The set of all Turing Machines
is countable

Proof: Any Turing Machine can be encoded
with a binary string of 0's and 1's

Find an enumeration procedure
for the set of Turing Machine strings

Enumerator:

Repeat

1. Generate the next binary string of 0's and 1's in proper order
2. Check if the string describes a Turing Machine
 - if **YES**: print string on output tape
 - if **NO**: ignore string

Binary strings

Turing Machines

0 ignore

1 ignore

00 ignore

01

⋮

1 0 1 0 1 1 0 1 1 0 0

1 0 1 0 1 1 0 1 1 0 1

⋮

1 0 1 1 0 1 0 1 0 0 1 0 1 0 1 1 0 1

⋮

s_1 → 1 0 1 0 1 1 0 1 1 0 1

Handwritten notes:
- Turing machine
- users is article
- enumerate file

s_2 → 1 0 1 1 0 1 0 1 0 0 1 0 1 0 1 1 0 1

End of Proof

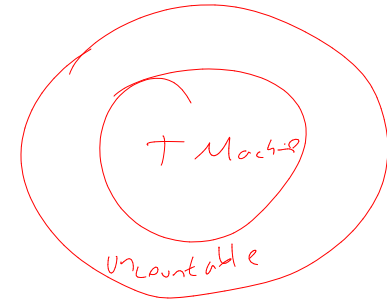
Simpler Proof:

Each Turing machine binary string is mapped to the number representing its value

Uncountable Sets

We will prove that there is a language L
which is not accepted by any Turing machine

Technique:



Turing machines are countable

Languages are uncountable

(there are more languages than Turing Machines)

Theorem:

If S is an infinite countable set, then

the powerset 2^S of S is uncountable.

↓
infinite
countable
set in
powerset is
uncountable dir.

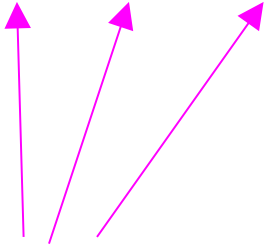
The powerset 2^S contains all possible subsets of S

Example: $S = \{a, b\}$ $2^S = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}$

Proof:

Since S is countable, we can list its elements in some order

$$S = \{s_1, s_2, s_3, \dots\}$$



Elements of S

Elements of the powerset 2^S have the form:

$$\emptyset$$

$$\{s_1, s_3\}$$

$$\{s_5, s_7, s_9, s_{10}\}$$

$$\vdots$$

They are subsets of S

We encode each subset of S
with a binary string of 0's and 1's

Subset of S	Binary encoding				
	s_1	s_2	s_3	s_4	$\dots$
$\{s_1\}$	1	0	0	0	$\dots$
$\{s_2, s_3\}$	0	1	1	0	$\dots$
$\{s_1, s_3, s_4\}$	1	0	1	1	$\dots$

Every infinite binary string corresponds to a subset of S :

Example:

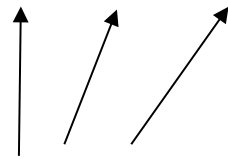
1001110...

Corresponds to: $\{s_1, s_4, s_5, s_6, \dots\} \in 2^S$

Let's assume (for contradiction)
that the powerset 2^S is countable

Then: we can list the elements of the
powerset in some order

$$2^S = \{t_1, t_2, t_3, \dots\}$$



Subsets of S

Powerset element

Binary encoding example

t_1	1	0	0	0	0	...
-------	---	---	---	---	---	-----

t_2	1	1	0	0	0	...
-------	---	---	---	---	---	-----

t_3	1	1	0	1	0	...
-------	---	---	---	---	---	-----

t_4	1	1	0	0	1	...
-------	---	---	---	---	---	-----

...

...

t = the binary string whose bits
are the complement of the diagonal

t_1	1	0	0	0	0	...
t_2	1	1	0	0	0	...
t_3	1	1	0	1	0	...
t_4	1	1	0	0	1	...

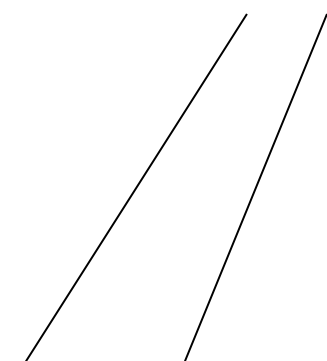
Binary string: $t = 0011\dots$

(binary complement of diagonal)

The binary string

$$t = 0011\dots$$

corresponds
to a subset of S :


$$t = \{s_3, s_4, \dots\} \in 2^S$$

t = the binary string whose bits
are the complement of the diagonal

t_1	1	0	0	0	0	...
t_2	1	1	0	0	0	...
t_3	1	1	0	1	0	...
t_4	1	1	0	0	1	...

$t = 0011 \dots$

Question: $t = t_1$? **NO:** differ in 1st bit

t = the binary string whose bits
are the complement of the diagonal

t_1	1	0	0	0	0	...
t_2	1	1	0	0	0	...
t_3	1	1	0	1	0	...
t_4	1	1	0	0	1	...

$t = 0011 \dots$

Question: $t = t_2$? **NO:** differ in 2nd bit

t = the binary string whose bits
are the complement of the diagonal

t_1	1	0	0	0	0	...
t_2	1	1	0	0	0	...
t_3	1	1	0	1	0	...
t_4	1	1	0	0	1	...

$t = 0011 \dots$

Question: $t = t_3$? **NO:** differ in 3rd bit

Thus: $t \neq t_i$ for every i

since they differ in the i th bit

However, $t \in 2^S \Rightarrow t = t_i$ for some i

Contradiction!!!

Therefore the powerset 2^S is uncountable

End of proof

An Application: Languages

Consider Alphabet : $A = \{a, b\}$

The set of all strings:

$$S = \{a, b\}^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$$

infinite and countable

because we can enumerate
the strings in proper order

Consider Alphabet : $A = \{a, b\}$

The set of all strings:

$$S = \{a, b\}^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$$

infinite and countable

Any language is a subset of S :

$$L = \{aa, ab, aab\}$$

Consider Alphabet : $A = \{a, b\}$

The set of all Strings:

$$S = A^* = \{a, b\}^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$$

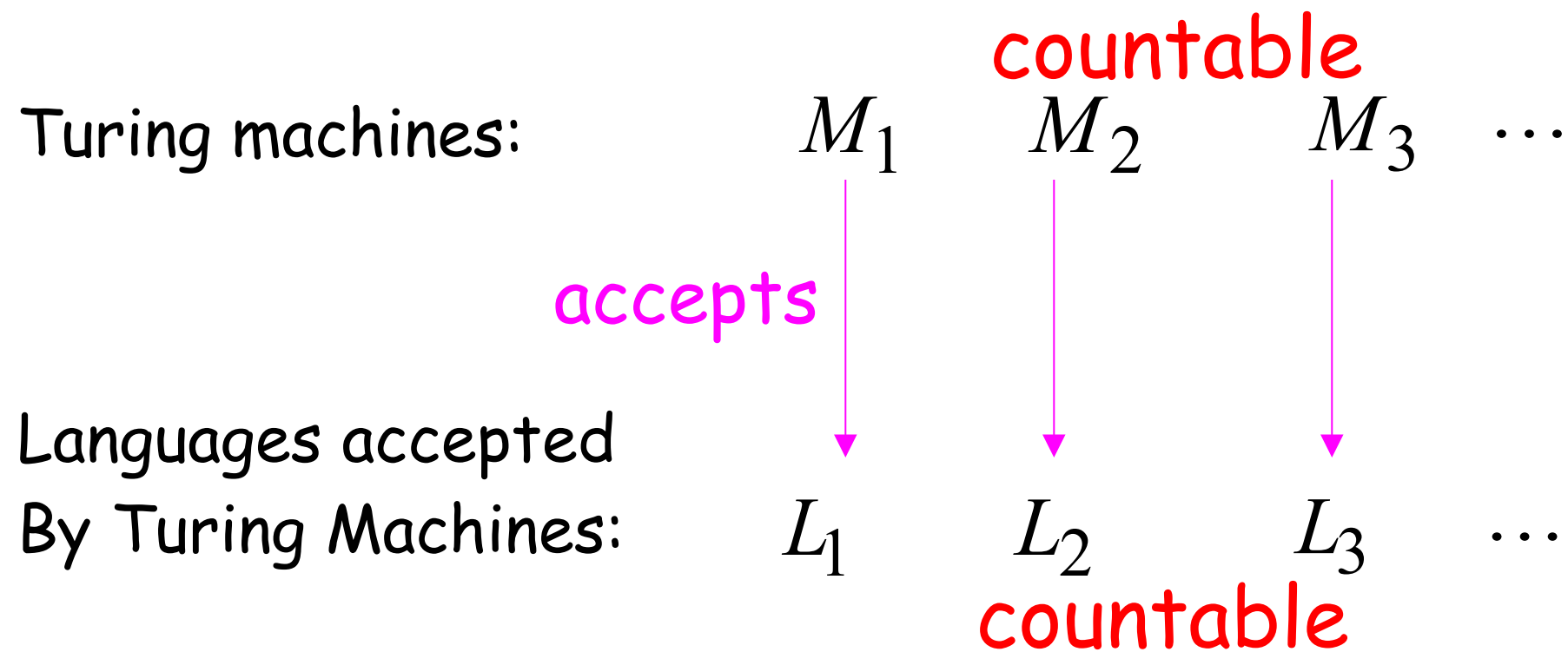
infinite and countable

The powerset of S contains all languages:

$$2^S = \{\emptyset, \{\varepsilon\}, \{a\}, \{a, b\}, \{aa, b\}, \dots, \{aa, ab, aab\}, \dots\}$$

uncountable

Consider Alphabet : $A = \{a, b\}$



Denote: $X = \{L_1, L_2, L_3, \dots\}$ Note: $X \subseteq 2^S$

countable

$(S = \{a, b\}^*)$

Languages accepted
by Turing machines:

X countable

All possible languages: 2^S uncountable

Therefore: $X \neq 2^S$

(since $X \subseteq 2^S$, we get $X \subset 2^S$)

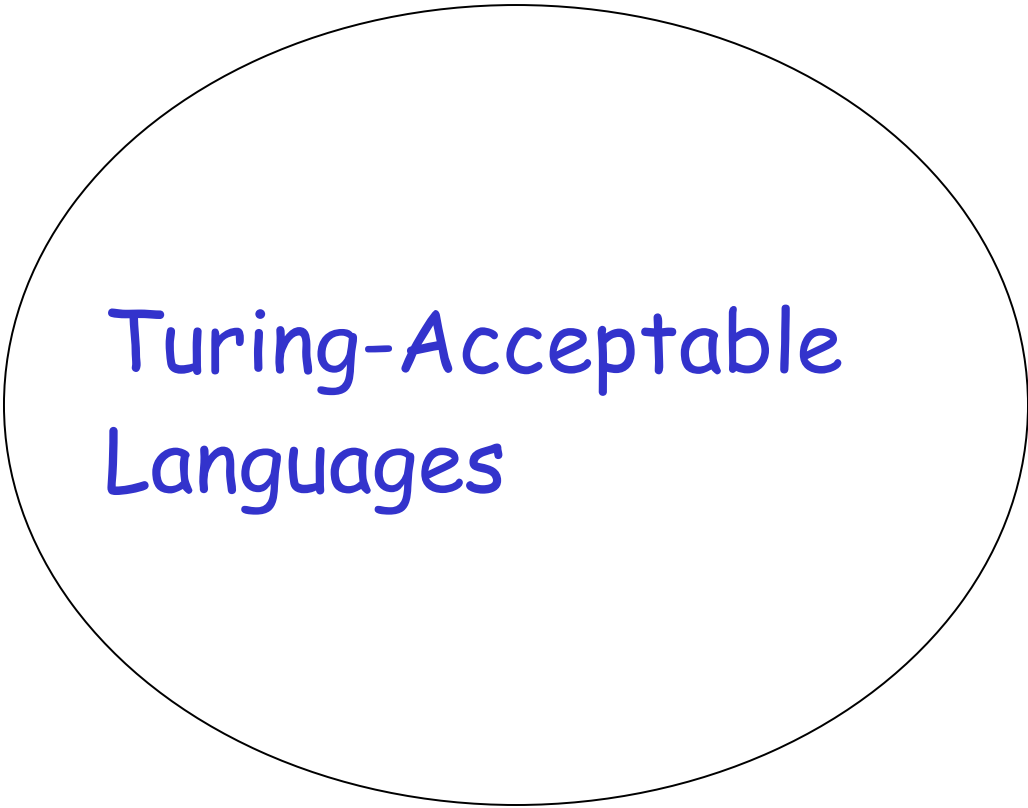
Conclusion:

There is a language L not accepted
by any Turing Machine:

$$X \subset 2^S \implies \exists L \in 2^S \text{ and } L \notin X$$

Non Turing-Acceptable Languages

L

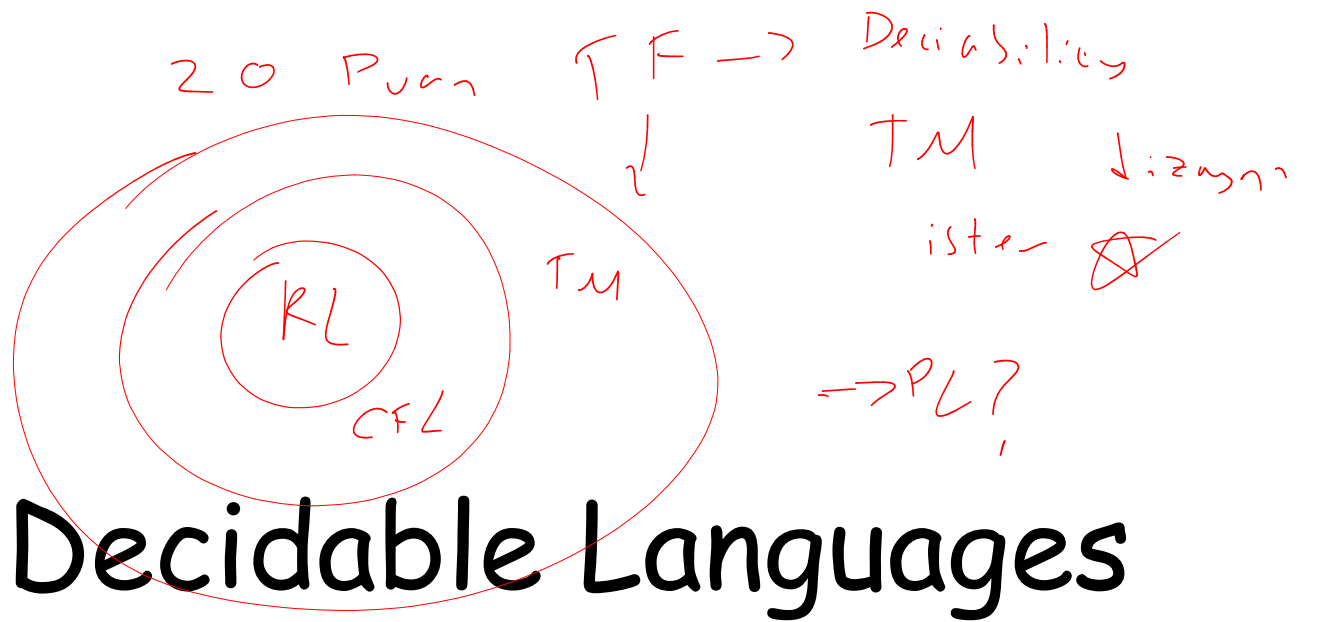


Turing-Acceptable
Languages

Note that: $X = \{L_1, L_2, L_3, \dots\}$

is a *multi-set* (elements may repeat)
since a language may be accepted
by more than one Turing machine

However, if we remove the repeated elements,
the resulting set is again countable since every element
still corresponds to a positive integer



Push down automata

Recall that:

TF sorusu
fulan gelir

A language L is **Turing-Acceptable**
if there is a Turing machine M
that accepts L

Also known as: **Turing-Recognizable**

or

Recursively-enumerable
languages

Turing-Acceptable $\rightarrow$ Birk Language: Turing machine valid

For any input string w :

$w \in L \implies M$ halts in an accept state

$w \notin L \implies M$ halts in a non-accept state
or loops forever

Definition:

A language L is **decidable** if there is a Turing machine (decider) M which accepts L and halts on every input string



Also known as *recursive languages*

Turing-Decidable

For any input string w :

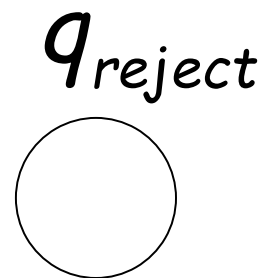
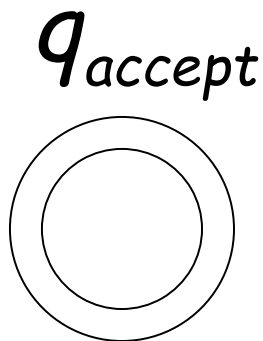
$w \in L \implies M$ halts in an accept state

$w \notin L \implies M$ halts in a non-accept state

Observation:

Every decidable language is Turing-Acceptable

Sometimes, it is convenient to have Turing machines with single accept and reject states

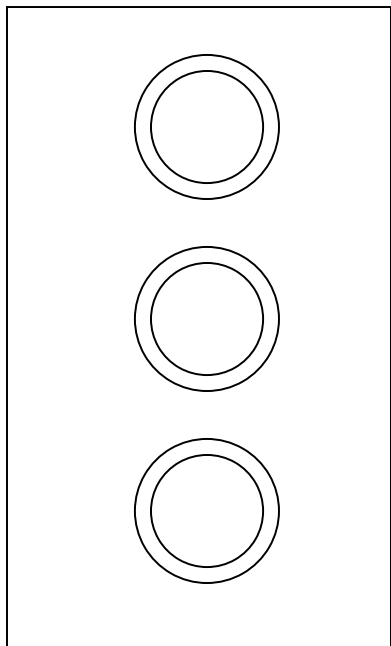


These are the only halting states

That result to possible
halting configurations

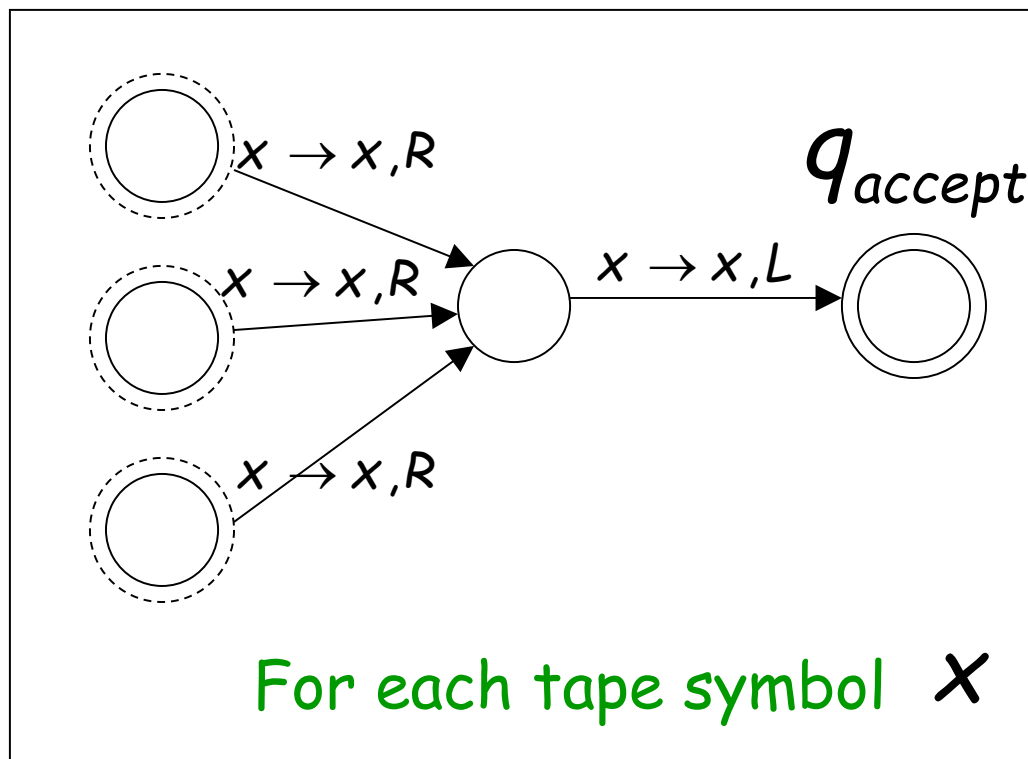
We can convert any Turing machine to have single accept and reject states

Old machine



Multiple
accept states

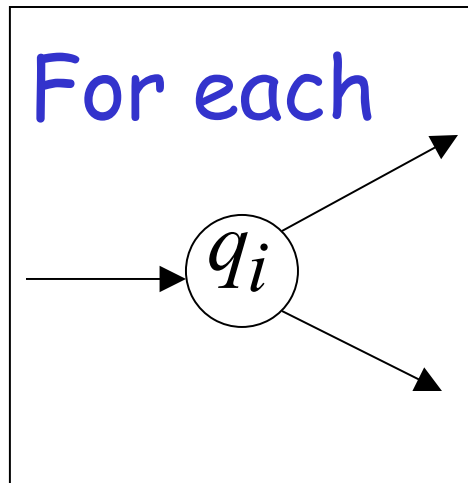
New machine



One accept state

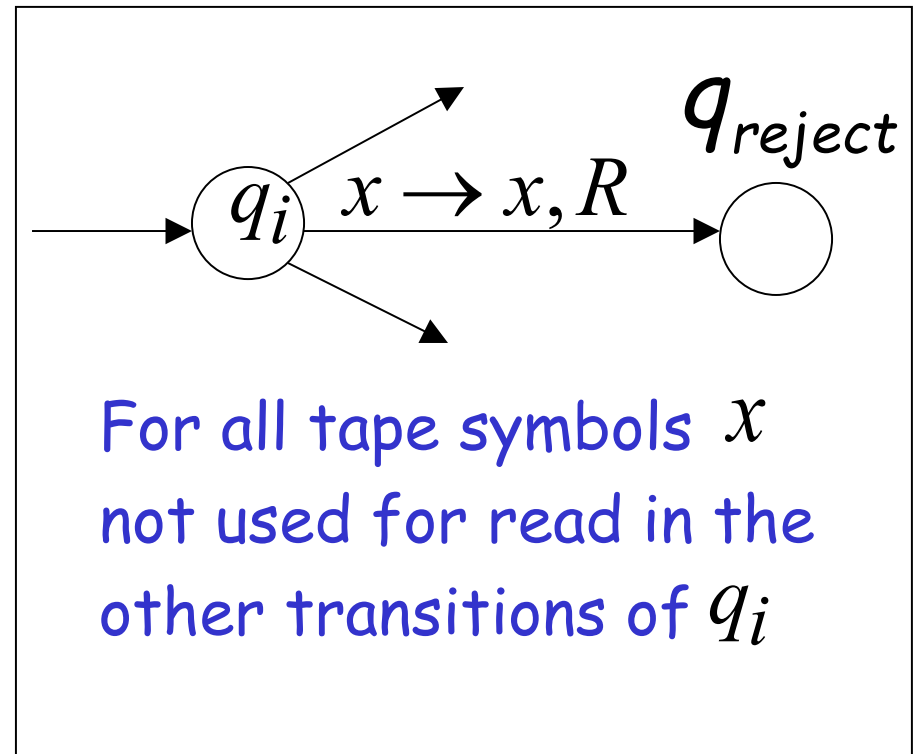
Do the following for each possible halting state:

Old machine



Multiple
reject states

New machine



One reject state

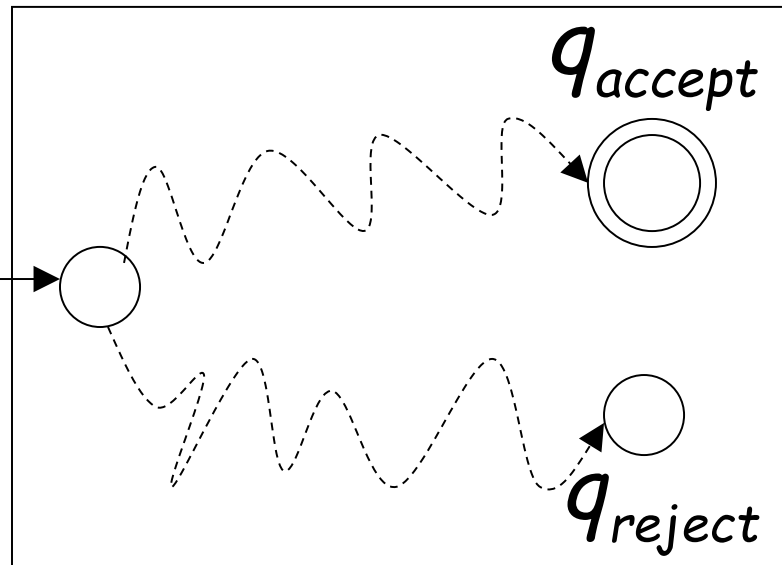
For a decidable language L :

loop a given string T.M.

Decider for L

Input string

w



Decision
On Halt:

Accept

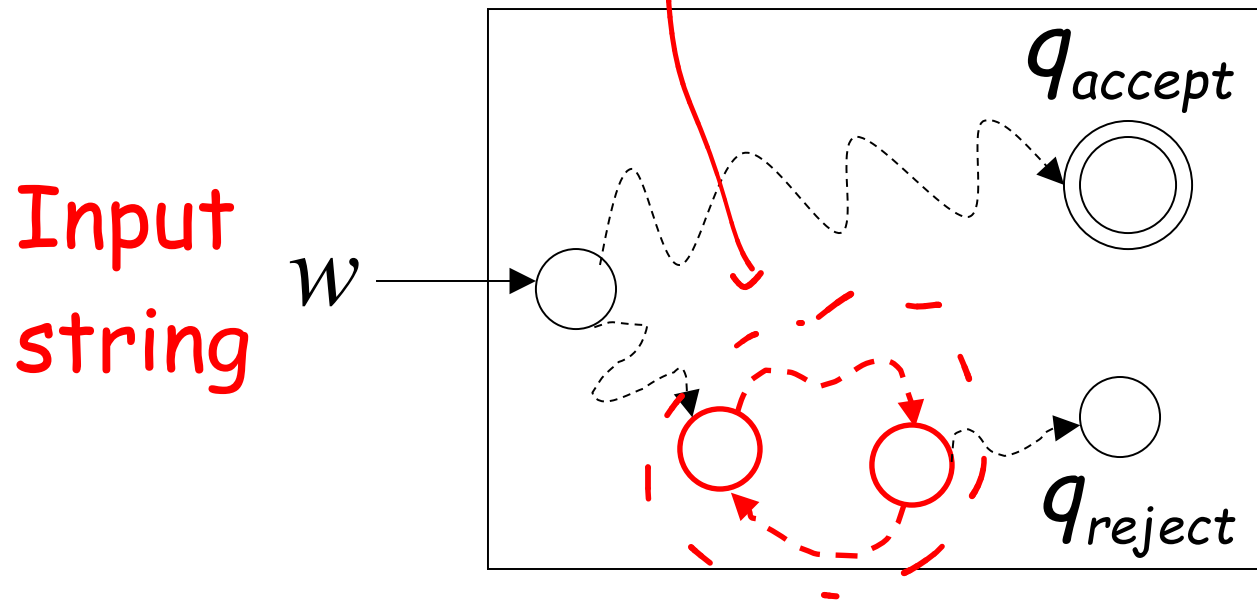
Reject

For each input string, the computation halts in the accept or reject state

For a Turing-Acceptable language L :

Loop Oblivi.

Turing Machine for L



It is possible that for some input string the machine enters an infinite loop

A computational problem is decidable
if the corresponding language is decidable

We also say that the problem is solvable

Problem: Is number x prime?

Corresponding language:

$$PRIMES = \{2, 3, 5, 7, \dots\}$$

We will show it is decidable

Decider for *PRIMES* :

On input number x :

Divide x with all possible numbers
between 2 and $\sqrt{x}$

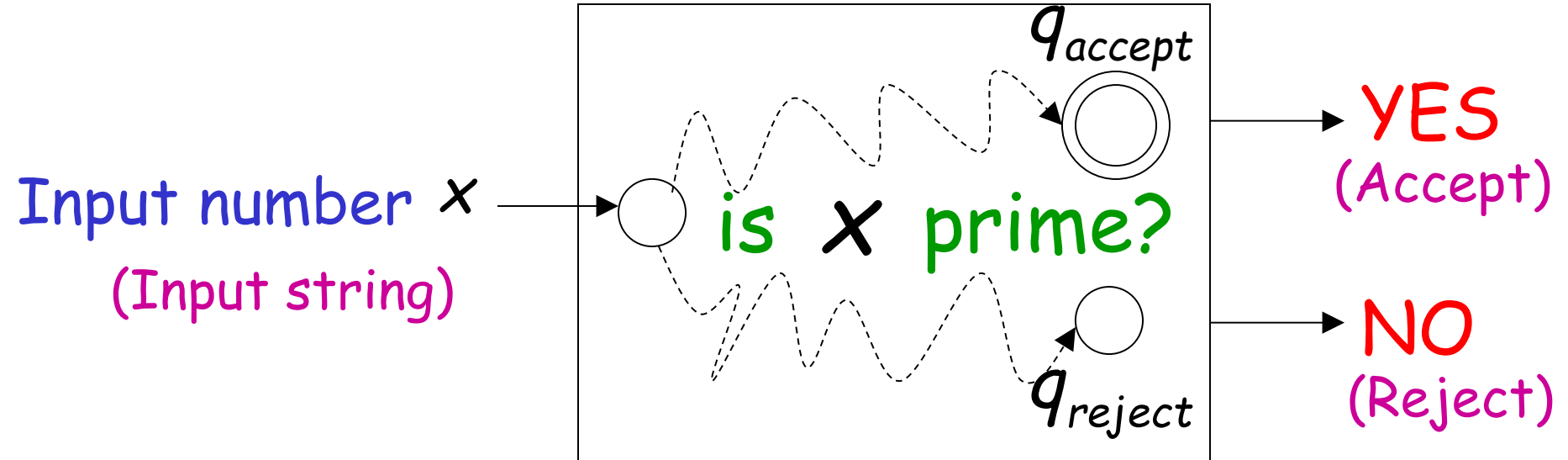
If any of them divides x

Then reject

Else accept

the decider Turing machine can be designed based on the algorithm

Decider for *PRIMES*



Problem: Does DFA M accept
the empty language $L(M) = \emptyset$?

*EMPTY
Problem*

Corresponding Language: (Decidable)

$EMPTY_{DFA} =$

$\{\langle M \rangle : M \text{ is a DFA that accepts empty language } \emptyset\}$

*→ DFA in
format*

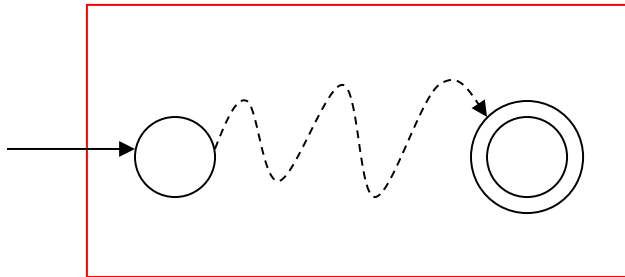
Description of DFA M as a string
(For example, we can represent M as a
binary string, as we did for Turing machines)

Decider for $EMPTY_{DFA}$:

On input $\langle M \rangle$:

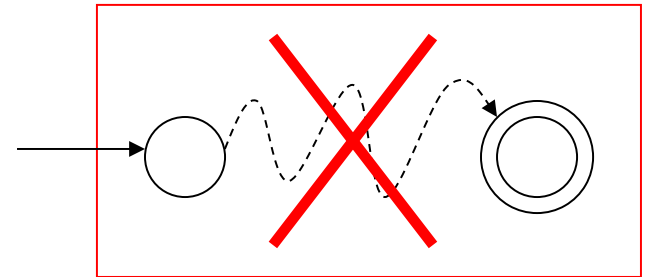
Determine whether there is a path from the initial state to any accepting state

DFA M



$$L(M) \neq \emptyset$$

DFA M



$$L(M) = \emptyset$$

Decision: **Reject** $\langle M \rangle$

Accept $\langle M \rangle$

Problem: Does DFA M accept a finite language?

Corresponding Language: (Decidable)

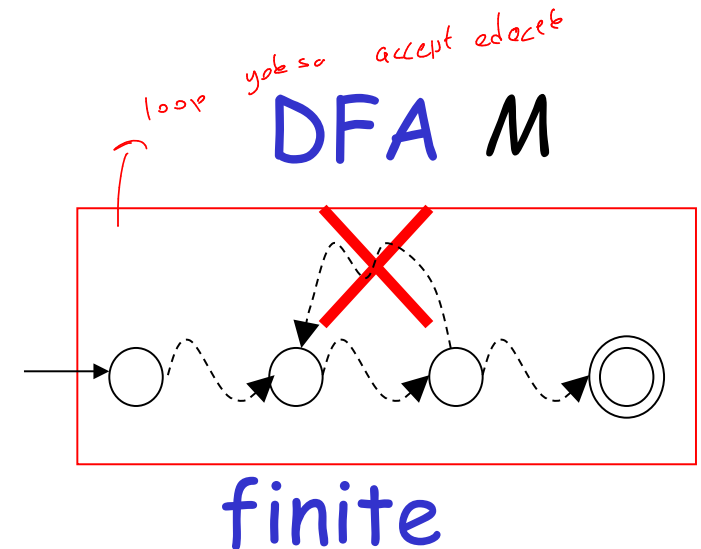
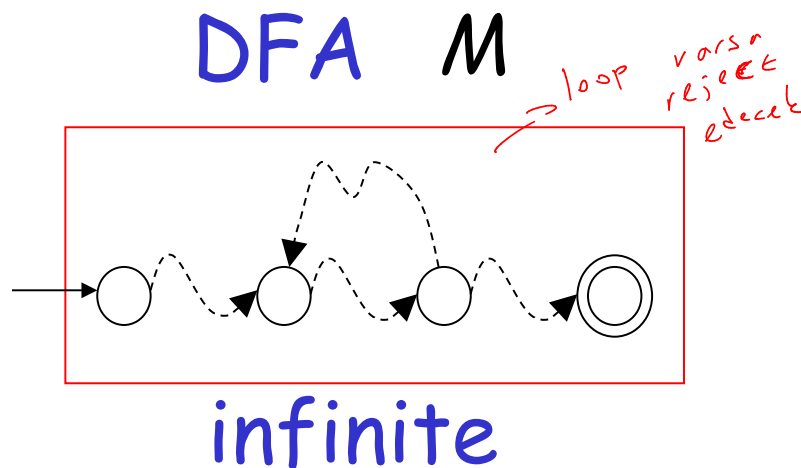
$FINITE_{DFA} =$

$\{\langle M \rangle : M \text{ is a DFA that accepts a finite language}\}$

Decider for $FINITE_{DFA}$:

On input $\langle M \rangle$:

Check if there is a walk with a cycle
from the initial state to an accepting state



Decision: **Reject** $\langle M \rangle$
(NO)

Accept $\langle M \rangle$
(YES)

Problem: Does DFA M accept string w ?

Corresponding Language: (Decidable)

A_{DFA} *→ Acceptance*

$= \{ \langle M, w \rangle : M \text{ is a DFA that accepts string } w \}$
→ M: dfa, w: string, M: label

Decider for A_{DFA} :

On input string $\langle M, w \rangle$:

Run DFA M on input string w

If M accepts w

Then accept $\langle M, w \rangle$ (and halt)

Else reject $\langle M, w \rangle$ (and halt)

Problem: Do DFAs M_1 and M_2
accept the same language?

Corresponding Language: (Decidable)

$EQUAL_{DFA} =$ DFA M_1 ve M_2 aynı Language'ı mı kabul eder

$\{\langle M_1, M_2 \rangle : M_1 \text{ and } M_2 \text{ are DFAs that accept the same languages}\}$

Decider for $EQUAL_{DFA}$:

On input $\langle M_1, M_2 \rangle$:

Let L_1 be the language of DFA M_1

Let L_2 be the language of DFA M_2

Construct DFA M such that:

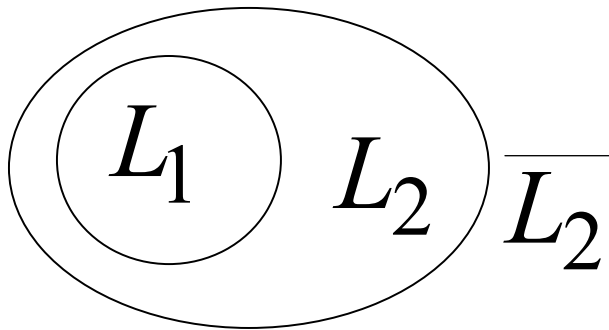
$$L(M) = (L_1 \cap \overline{L_2}) \cup (\overline{L_1} \cap L_2)$$


(combination of DFAs)

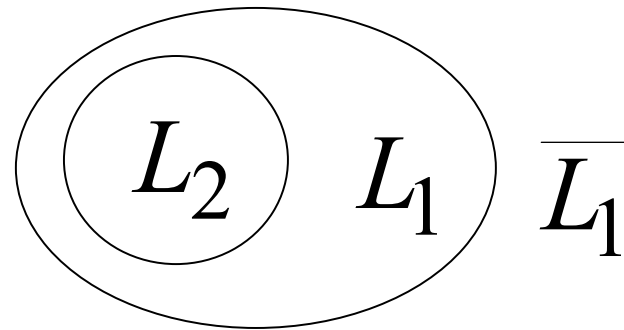
$$(L_1 \cap \overline{L_2}) \cup (\overline{L_1} \cap L_2) = \emptyset$$



$$L_1 \cap \overline{L_2} = \emptyset \quad \text{and} \quad \overline{L_1} \cap L_2 = \emptyset$$



$$L_1 \subseteq L_2$$



$$L_2 \subseteq L_1$$



$$L_1 = L_2$$

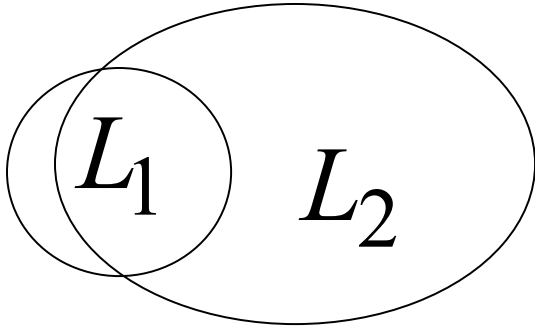
$$(L_1 \cap \overline{L_2}) \cup (\overline{L_1} \cap L_2) \neq \emptyset$$



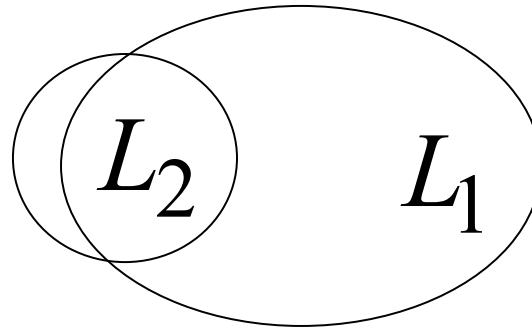
$$L_1 \cap \overline{L_2} \neq \emptyset$$

or

$$\overline{L_1} \cap L_2 \neq \emptyset$$



$$L_1 \not\subseteq L_2$$



$$L_2 \not\subseteq L_1$$



$$L_1 \neq L_2$$

Therefore, we only need
to determine whether

$$L(M) = (L_1 \cap \overline{L_2}) \cup (\overline{L_1} \cap L_2) = \emptyset$$

which is a solvable problem for DFAs:

*EMPTY*_{DFA}

Theorem:

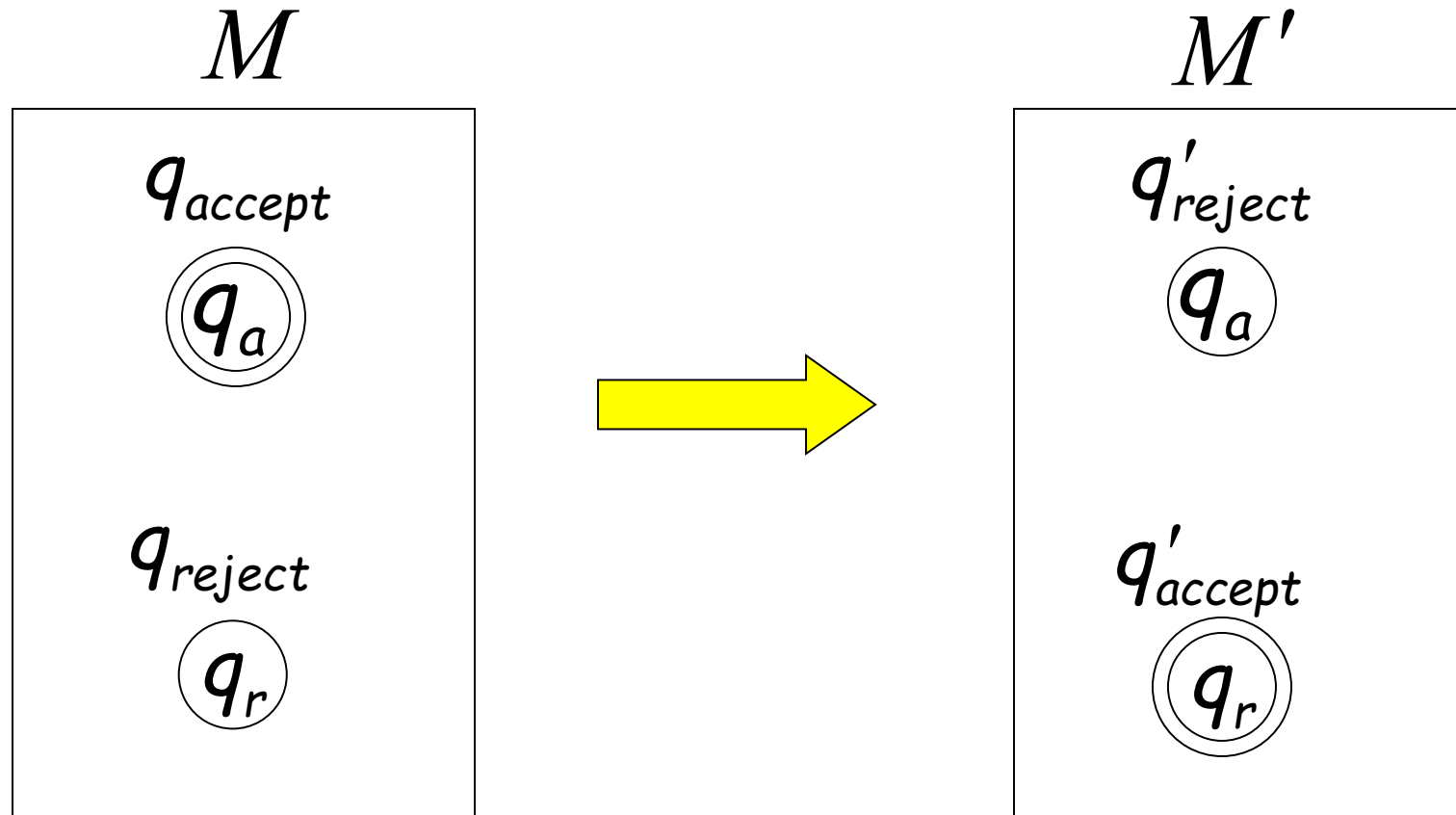
If a language L is decidable,
then its complement $\bar{L}$ is decidable too

↓
Accept
Stateleri: Accept olmahtan sikarizoruz

Proof:

Build a Turing machine M' that
accepts $\bar{L}$ and halts on every input string
(M' is decider for $\bar{L}$)

Transform accept state to reject and vice-versa



Turing Machine M'

On each input string w do:

1. Let M be the decider for L
2. Run M with input string w
If M accepts then reject
If M rejects then accept

Accepts $\bar{L}$ and halts on every input string

Undecidable Languages

Undecidable Languages

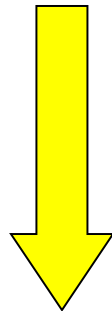
An undecidable language has no decider:
Any Turing machine that accepts L
does not halt on some input string

We will show that:

There is a language which is
Turing-Acceptable and undecidable

We will prove that there is a language L :

- L is Turing-acceptable
- $\overline{L}$ is **not** Turing-acceptable
(not accepted by any Turing Machine)



the complement of a
decidable language is decidable

Therefore, L is undecidable

Non Turing-Acceptable $\overline{L}$

Turing-Acceptable L

Decidable

Consider alphabet $\{a\}$

Strings of $\{a\}^+$:

$a, aa, aaa, aaaa, \dots$

$a^1 \quad a^2 \quad a^3 \quad a^4 \quad \dots$

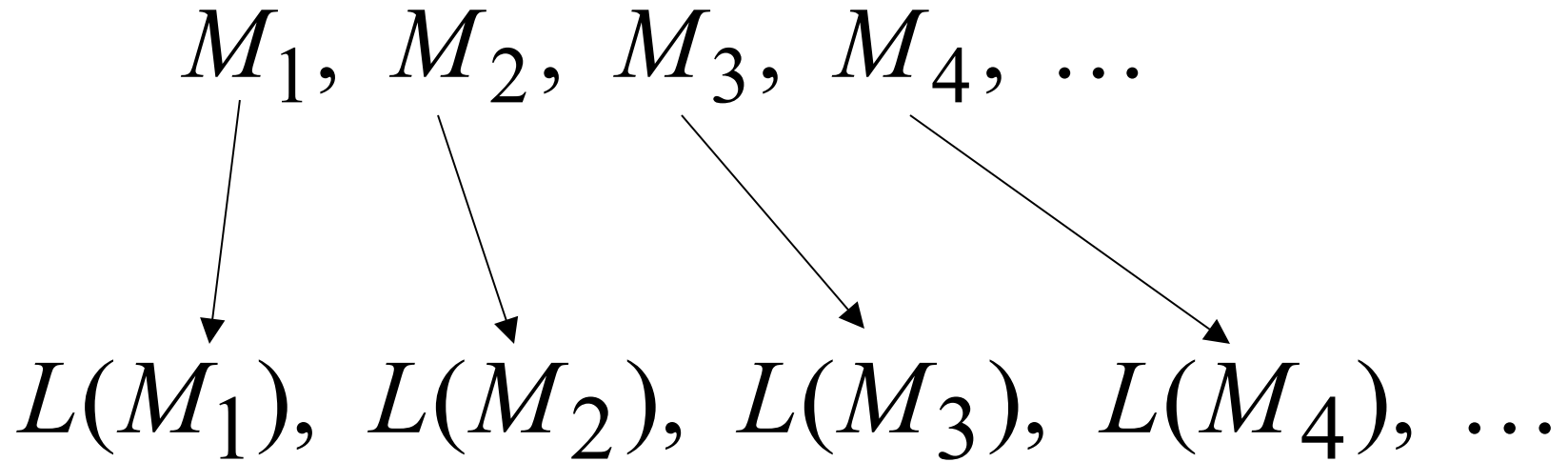
Consider Turing Machines
that accept languages over alphabet $\{a\}$

They are countable:

$$M_1, M_2, M_3, M_4, \dots$$

(There is an enumerator that generates them)

Each machine accepts some language over $\{a\}$



Note that it is possible to have

$$L(M_i) = L(M_j) \quad \text{for } i \neq j$$

Since, a language could be accepted by more than one Turing machine

Example language accepted by M_i

$$L(M_i) = \{aa, aaaa, aaaaaa\}$$

$$L(M_i) = \{a^2, a^4, a^6\}$$

Binary representation

	a^1	a^2	a^3	a^4	a^5	a^6	a^7	$\dots$
$L(M_i)$	0	1	0	1	0	1	0	$\dots$

Example of binary representations

	a^1	a^2	a^3	a^4	$\dots$
$L(M_1)$	0	1	0	1	$\dots$
$L(M_2)$	1	0	0	1	$\dots$
$L(M_3)$	0	1	1	1	$\dots$
$L(M_4)$	0	0	0	1	$\dots$

Consider the language

$$L = \{a^i : a^i \in L(M_i)\}$$

L consists of the 1's in the diagonal

	a^1	a^2	a^3	a^4	$\dots$
$L(M_1)$	0	1	0	1	$\dots$
$L(M_2)$	1	0	0	1	$\dots$
$L(M_3)$	0	1	1	1	$\dots$
$L(M_4)$	0	0	0	1	$\dots$

$$L = \{a^3, a^4, \dots\}$$

Consider the language $\overline{L}$

$$\overline{L} = \{a^i : a^i \notin L(M_i)\}$$

$$L = \{a^i : a^i \in L(M_i)\}$$

$\overline{L}$ consists of the 0's in the diagonal

	a^1	a^2	a^3	a^4	$\dots$
$L(M_1)$	0	1	0	1	$\dots$
$L(M_2)$	1	0	0	1	$\dots$
$L(M_3)$	0	1	1	1	$\dots$
$L(M_4)$	0	0	0	1	$\dots$

$\bar{L} = \{a^1, a^2, \dots\}$

Theorem:

Language $\overline{L}$ is not Turing-Acceptable

Proof:

Assume for contradiction that
 $\overline{L}$ is Turing-Acceptable

Let M_k be the Turing machine
that accepts $\overline{L}$: $L(M_k) = \overline{L}$

	a^1	a^2	a^3	a^4	$\dots$
$L(M_1)$	0	1	0	1	$\dots$
$L(M_2)$	1	0	0	1	$\dots$
$L(M_3)$	0	1	1	1	$\dots$
$L(M_4)$	0	0	0	1	$\dots$

$$L(M_k) = \bar{L} = 1100\dots$$

Question: $M_k = M_1$?

	a^1	a^2	a^3	a^4	$\dots$
$L(M_1)$	0	1	0	1	$\dots$
$L(M_2)$	1	0	0	1	$\dots$
$L(M_3)$	0	1	1	1	$\dots$
$L(M_4)$	0	0	0	1	$\dots$

$$L(M_k) = \bar{L} = 1100\dots$$

$$a^1 \in L(M_k)$$

$$a^1 \notin L(M_1)$$

$$M_k \neq M_1$$

Answer:

	a^1	a^2	a^3	a^4	$\dots$
$L(M_1)$	0	1	0	1	$\dots$
$L(M_2)$	1	0	0	1	$\dots$
$L(M_3)$	0	1	1	1	$\dots$
$L(M_4)$	0	0	0	1	$\dots$

$$L(M_k) = \bar{L} = 1100\dots$$

Question: $M_k = M_2$?

	a^1	a^2	a^3	a^4	$\dots$
$L(M_1)$	0	1	0	1	$\dots$
$L(M_2)$	1	0	0	1	$\dots$
$L(M_3)$	0	1	1	1	$\dots$
$L(M_4)$	0	0	0	1	$\dots$

$$L(M_k) = \bar{L} = 1100\dots$$

$$a^2 \in L(M_k)$$

$$a^2 \notin L(M_2)$$

Answer:

$$M_k \neq M_2$$

	a^1	a^2	a^3	a^4	$\dots$
$L(M_1)$	0	1	0	1	$\dots$
$L(M_2)$	1	0	0	1	$\dots$
$L(M_3)$	0	1	1	1	$\dots$
$L(M_4)$	0	0	0	1	$\dots$

$$L(M_k) = \bar{L} = 1100\dots$$

Question: $M_k = M_3$?

	a^1	a^2	a^3	a^4	$\dots$
$L(M_1)$	0	1	0	1	$\dots$
$L(M_2)$	1	0	0	1	$\dots$
$L(M_3)$	0	1	1	1	$\dots$
$L(M_4)$	0	0	0	1	$\dots$

$$L(M_k) = \bar{L} = 1100\dots$$

$$a^3 \notin L(M_k)$$

$$a^3 \in L(M_3)$$

$$M_k \neq M_3$$

Answer:

Similarly: $M_k \neq M_i$ for any i

Because either:

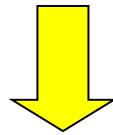
$$a^i \in L(M_k)$$

or

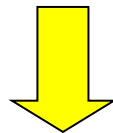
$$a^i \notin L(M_k)$$

$$a^i \notin L(M_i)$$

$$a^i \in L(M_i)$$



the machine M_k cannot exist



$\overline{L}$ is not Turing-Acceptable

End of Proof

Non Turing-Acceptable

$\overline{L}$

Turing-Acceptable

Decidable

We will prove that the language

$$L = \{a^i : a^i \in L(M_i)\}$$

Is Turing-
Acceptable

There is a
Turing machine
that accepts L

Braden Samson
yes

Undecidable

Each machine
that accepts L
doesn't halt
on some input string

Theorem: The language

$$L = \{a^i : a^i \in L(M_i)\}$$

Is Turing-Acceptable

Proof: We will give a Turing Machine that
accepts L

Turing Machine that accepts L

For any input string w

- Suppose $w = a^i$
- Find Turing machine M_i
(using the enumerator for Turing Machines)
- Simulate M_i on input string a^i
- If M_i accepts, then accept w

End of Proof

Therefore:

Turing-Acceptable

$$L = \{a^i : a^i \in L(M_i)\}$$

Not Turing-acceptable

$$\overline{L} = \{a^i : a^i \notin L(M_i)\}$$

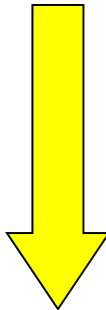
Non Turing-Acceptable $\overline{L}$

Turing-Acceptable L

Decidable

$L?$

Theorem: $L = \{a^i : a^i \in L(M_i)\}$
is undecidable

Proof: If L is decidable
 the complement of a
decidable language is decidable

Then $\overline{L}$ is decidable

However, $\overline{L}$ is not Turing-Acceptable!

Contradiction!!!!

Not Turing-Acceptable $\overline{L}$

Turing-Acceptable L

Decidable



BLM2502

The Theory of Computation

Spring 2022

Lecturer: Dr. Oğuz Altun

Email: oaltun@yildiz.edu.tr

Office: D036

Office Hours: Tuesday 12:00-13:00

» **Course: BLM 2502 The theory of Computation**

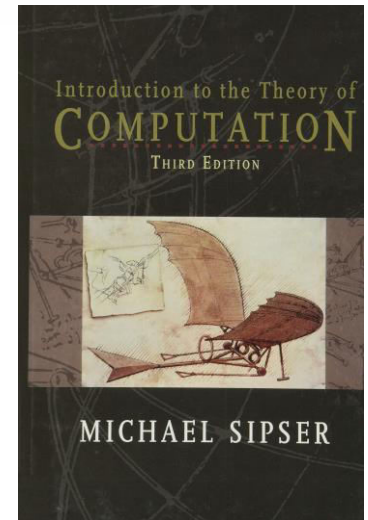
> **Tuesday 09:00-12:00**

> **ECTS 3 credits**

BLM2502 Theory of Computation

» Book

- > Michael Sipser, Introduction to the Theory of Computation (3E), Thomson
- > No handouts, its your responsibily to take notes.



- > Lectures from the author:

<https://ocw.mit.edu/courses/mathematics/18-404j-theory-of-computation-fall-2020/>

» **Course Goals:**

- » To introduce fundamental theoretical concepts for the computational complexity of algorithmic problems, and also to introduce basic techniques used for lexical analysis and syntax parsing in programming languages.

» **Main Topics Covered:**

- » Finite Automata
 - Deterministic and nondeterministic finite automata
 - Regular expressions
- » Pushdown Automata
 - Context-free languages and grammars
 - Parsing
- » Turing Machines
 - Turing recognizable languages
 - Decidability
 - Time and Space complexity
 - NP-completeness

» **Grading:**

- » Homework assignments: 15%
- » Pop-up quizzes (~8 quizzes): 15%
- » Midterm: 30%
- » Final: 40%



BLM2502

Theory of

Computation

Spring 2017

BLM2502 Theory of Computation

» Book

- > Michael Sipser, Introduction to the Theory of Computation (3E), Thomson
- > No handouts, its your responsibly to take notes.

BLM2502 Theory of Computation

- » In this Theory of Computation course we will try to answer the following questions:
- » What are the mathematical properties of computer hardware and software?
- » What is a computation and what is an algorithm? Can we give rigorous mathematical definitions of these notions?
- » What are the limitations of computers? Can “everything” be computed? (As we will see, the answer to this question is “no”.)
- » **Purpose of the Theory of Computation:**
 - Develop formal mathematical models of computation that reflect real-world computers.**

BLM2502 Theory of Computation

» Complexity Theory

- » The main question asked in this area is “What makes some problems computationally hard and other problems easy?”
- » Informally, a problem is called “easy”, if it is efficiently solvable. Examples of “easy” problems are
 - > Sorting a sequence of, e.g, 1,000,000 numbers,
 - > Searching for a name in a telephone directory
 - > Computing the fastest way to drive from Esenler to Beşiktaş.

BLM2502 Theory of Computation

» Complexity Theory

» On the other hand, a problem is called “hard”, if it cannot be solved efficiently, or if we don’t know whether it can be solved efficiently. Examples of “hard” problems are

- > Time table scheduling for all courses
- > Factoring a 300-digit integer into its prime factors
- > Computing a layout for chips in VLSI.

» Central Question in Complexity Theory:

Classify problems according to their degree of “difficulty”. Give a rigorous proof that problems that seem to be “hard” are really “hard”.

BLM2502 Theory of Computation

» Computability Theory

- » In the 1930's, scientists discovered that some of the fundamental mathematical problems cannot be solved by a "computer". (computers were invented in 1940s). For example: "Is an arbitrary mathematical statement true or false?"
- » To attack such a problem, we need formal definitions of the notions of
 - > computer
 - > algorithm
 - > computation

BLM2502 Theory of Computation

» Computability Theory

- » The theoretical models that were proposed in order to understand solvable and unsolvable problems led to the development of real computers.
- » Central Question in Computability Theory:

Classify problems as being solvable or unsolvable.

BLM2502 Theory of Computation

» Automata Theory

» Automata Theory deals with definitions and properties of different types of “computation models”. Examples :

- > Finite Automata. These are used in text processing, compilers, and hardware design.
- > Context-Free Grammars. These are used to define programming languages and in Artificial Intelligence.
- > Turing Machines. These form a simple abstract model of a “real” computer, such as your PC at home.

» Central Question in Automata Theory:

Do these models have the same power, or can one model solve more problems than the other?

BLM2502 Theory of Computation

» Set Theory

» A set is a collection of well-defined objects.

- > The set of **natural numbers** is $\mathbf{N} = \{1, 2, 3, \dots\}$.
- > The set of **integers** is $\mathbf{Z} = \{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$.
- > The set of **rational numbers** is $\mathbf{Q} = \{m/n : m \in \mathbf{Z}, n \in \mathbf{Z}, n \neq 0\}$.
 - + What is irrational numbers?
- > The set of **real numbers** is denoted by $\mathbf{R}$.
- > If A and B are sets, then A is a **subset** of B, written as $A \subseteq B$, if every element of A is also an element of B.
- > If B is a set, then the **power set** $P(B)$ of B is defined to be the set of all subsets of B:
 - ❖ $P(B) = \{A : A \subseteq B\}$.
 - ❖ Observe $\phi \in P(B)$ and $B \in P(B)$.

BLM2502 Theory of Computation

» Set Theory

- > If A and B are two sets, then
 - + their union is defined as
 - $A \cup B = \{x : x \in A \text{ or } x \in B\}$,
 - + their intersection is defined as
 - $A \cap B = \{x : x \in A \text{ and } x \in B\}$,
 - + their difference is defined as
 - $A \setminus B = \{x : x \in A \text{ and } x \notin B\}$,
 - + the Cartesian product of A and B is defined as
 - $A \times B = \{(x, y) : x \in A \text{ and } y \in B\}$,
 - + the complement of A is defined as
 - $\bar{A} = \{x : x \notin A\}$.

BLM2502 Theory of Computation

[, 0, 00, 11, 1, 10, 01

» Set Theory

- > A **binary relation** on two sets A and B is a subset of $A \times B$.
- > A **function** f from A to B , denoted by $f : A \rightarrow B$, is a binary relation R , having the property that for each element $a \in A$, there is exactly one ordered pair in R , whose first component is a . We will also say that $f(a) = b$, or f maps a to b , or the image of a under f is b . The set A is called the **domain** of f , and the set $\{b \in B : \text{there is an } a \in A \text{ with } f(a) = b\}$ is called the **range** of f .
- > A binary relation $R \subseteq A \times A$ is an **equivalence relation**, if it satisfies the following three conditions:
 - + R is **reflexive**: For every element in $a \in A$, we have $(a, a) \in R$.
 - + R is **symmetric**: For all a and b in A , if $(a, b) \in R$, then $(b, a) \in R$.
 - + R is **transitive**: For all a, b , and c in A , if $(a, b) \in R$ and $(b, c) \in R$, then also $(a, c) \in R$.

BLM2502 Theory of Computation

» Boolean Logic

- » The Boolean values are 1 and 0, that represent true and false, respectively. The basic Boolean operations:
- » **negation** (or NOT), represented by $\neg$,
- » **conjunction** (or AND), represented by $\wedge$,
- » **disjunction** (or OR), represented by $\vee$,
- » **exclusive-or** (or XOR), represented by $\oplus$,
- » **equivalence**, represented by $\leftrightarrow$ or $\Leftrightarrow$,
- » **implication**, represented by $\rightarrow$ or $\Rightarrow$.

BLM2502 Theory of Computation

» Truth Table (0=false, 1=true)

NOT	AND	OR	XOR	equivalence	implication
$\neg 0 = 1$	$0 \wedge 0 = 0$	$0 \vee 0 = 0$	$0 \oplus 0 = 0$	$0 \Leftrightarrow 0 = 1$	$0 \Rightarrow 0 = 1$
$\neg 1 = 0$	$0 \wedge 1 = 0$	$0 \vee 1 = 1$	$0 \oplus 1 = 1$	$0 \Leftrightarrow 1 = 0$	$0 \Rightarrow 1 = 1$
	$1 \wedge 0 = 0$	$1 \vee 0 = 1$	$1 \oplus 0 = 1$	$1 \Leftrightarrow 0 = 0$	$1 \Rightarrow 0 = 0$
	$1 \wedge 1 = 1$	$1 \vee 1 = 1$	$1 \oplus 1 = 0$	$1 \Leftrightarrow 1 = 1$	$1 \Rightarrow 1 = 1$

» Implication (if ... then ...)

> antecedent (condition)->consequence (promise)

» E.g.

> p: "you take out the trash".

> q:"you get a dollar"

> $p \Rightarrow q$ is false only if you take out the trash but don't get a dolar.

BLM2502 Theory of Computation

» Proof Techniques

» In mathematics, a **theorem** is a statement that is true. A **proof** is a sequence of mathematical statements that form an argument to show that a theorem is true.

- > **Axioms:** assumptions about the underlying mathematical structures
- > **Hypotheses:** a supposition or proposed explanation made based on limited evidence as a starting point for further investigation.
- > **Theorem:** described above
- > **Lemmas:** previously proved theorems
- > **Corollaries:** Special cases of theorem

» There is no specified way of producing a proof, but there are some generic strategies that could be of help.

BLM2502 Theory of Computation

» Proof Techniques

» Tips:

- > **Read** and completely understand the statement of the theorem to be proved. Most often this is the hardest part. **Rewrite** the statement in your own words.
- > Sometimes, theorems contain theorems inside them. For example, “Property A if and only if property B”, requires showing **two statements**:
 - + (a) If property A is true, then property B is true ($A \rightarrow B$).
 - + (b) If property B is true, then property A is true ($B \rightarrow A$).
- > Another example is the theorem “Set A equals set B.” To prove this, we need to prove that $A \subseteq B$ and $B \subseteq A$. That is, we need to show that each element of set A is in set B, and that each element of set B is in set A.

BLM2502 Theory of Computation

» Proof Techniques

» Tips:

- > Try to **work out a few simple cases** of the theorem just to get a grip on it (i.e., crack a few simple cases first).
- > Try to **write down the proof** once you have it. This is to ensure the correctness of your proof. Often, mistakes are found at the time of writing.
- > Finding proofs takes time, we do not come prewired to produce proofs. **Be patient**, think, express and write clearly and try to be precise as much as possible.

BLM2502 Theory of Computation

» Proof Techniques

- > Direct Proofs or Constructive Proofs or Proof by Construction
- > Nonconstructive Proofs
- > Proofs by Contradiction
- > Proofs by Induction
- > Pigeon Principle

BLM2502 Theory of Computation

» Proof Techniques - **Direct Proofs**

- » As the name suggests, in a direct proof of a theorem, we just approach the theorem directly.
- » **Theorem:** If n is an odd positive integer, then n^2 is odd as well.
- » **Proof:** An odd positive integer n can be written as:
 $n = 2k + 1$, for some integer $k \geq 0$. Then:
 - ❖ $n^2 = (2k + 1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$.
 - ❖ Since $2(2k^2 + 2k)$ is even, and “even plus one is odd”, we can conclude that
 - ❖ n^2 is odd.

BLM2502 Theory of Computation

» Proof Techniques - Constructive Proofs

(Proof By Construction)

- » Many theorems state that a particular type of object exists
- » One way to prove is to find a way to construct one such object
- » This technique is called proof by construction
- » **Theorem:** There exists a rational number p which can be expressed as a^b , with a and b both irrational.

A constructive proof of the above theorem on irrational powers of irrationals would give an actual example, such as:

$$a = \sqrt{2}, \quad b = \log_2 9, \quad a^b = 3.$$

The square root of 2 is irrational, and 3 is rational. $\log_2 9$ is also irrational: if it were equal to $\frac{m}{n}$, then, by the properties of

logarithms, 9^n would be equal to 2^m , but the former is odd, and the latter is even.

BLM2502 Theory of Computation

- » Proof Techniques - **Proof by Contradiction**
- » The proof by contradiction is grounded in the fact that **any proposition must be either true or false, but not both true and false at the same time.**
- » One common way to prove a theorem is to assume that the theorem is false, and then show that this assumption leads to an obviously false consequence (also called a contradiction)
- » This type of reasoning is used frequently in everyday life.

BLM2502 Theory of Computation

» Proof Techniques - **Proof by Contradiction**

- » Let us define a number is rational if it can be expressed as p/q where p and q are integers; if it cannot, then the number is called irrational.
- » **Theorem:** $\sqrt{2}$ (the square root of 2) is irrational.
- » **Proof:** Assume that $\sqrt{2}$ is rational. Then, it can be written as p/q for some positive integers p and q such that **p and q does not have a common factor.**
- » Then, we have $p^2/q^2 = 2$, or $2q^2 = p^2$
- » (continued in next page)

BLM2502 Theory of Computation

» Proof Techniques - **Proof by Contradiction**

- » Since $2q^2$ is an even number, p^2 is also an even number
- » This implies that **p is an even number** (why?)
- » So, $p = 2r$ for some integer r , and so, $2q^2 = p^2 = (2r)^2 = 4r^2$
- » This implies $2r^2 = q^2$
- » So, **q is an even number**
- » Something wrong happens!.. We now have:
- » “p and q does not have common factor”
- » AND
- » “p and q have common factor”
- » **This is a contradiction**
- » Thus, the assumption is wrong, so that $\sqrt{2}$ is irrational

BLM2502 Theory of Computation

» Proof Techniques - **Proof by Induction**

- » For each positive integer n , let $P(n)$ be a mathematical statement that depends on n . Assume we wish to prove that $P(n)$ is true for all positive integers n . A proof by induction of such a statement is carried out as follows:
 - » **Basis:** Prove that $P(1)$ is true.
 - » **Induction step:** Prove that for all $n \geq 1$, the following holds: If $P(n)$ is true, then $P(n + 1)$ is also true.
 - » In the induction step, we choose an arbitrary integer n and assume that $P(n)$ is true; this is called the induction hypothesis. Then we prove that $P(n + 1)$ is also true.

BLM2502 Theory of Computation

» Proof Techniques - **Proof by Induction**

» **Theorem:** For all positive integers n , we have

$$1 + 2 + 3 + \dots + n = n(n + 1)/2$$

» **Proof:** Start with the basis of the induction. If $n = 1$, then the left-hand side is equal to 1, and so is the right-hand side. So the theorem is true for $n = 1$.

» For the induction step, let $n \geq 1$ and assume that the theorem is true for n , i.e., assume that

$$1 + 2 + 3 + \dots + n = n(n + 1)/2$$

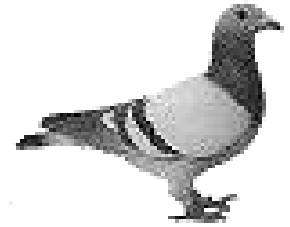
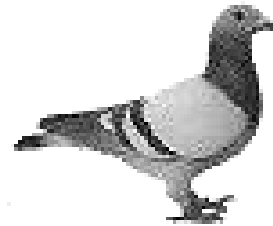
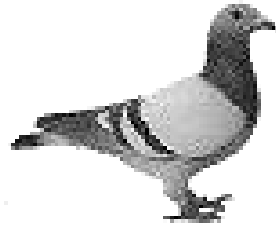
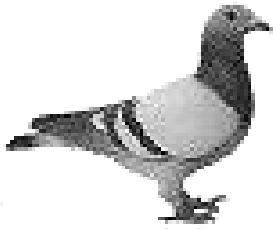
» We have to prove that the theorem is true for $n + 1$

BLM2502 Theory of Computation

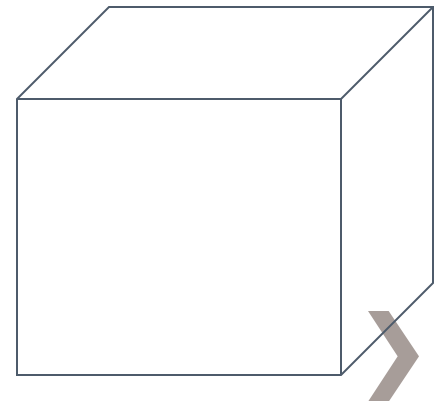
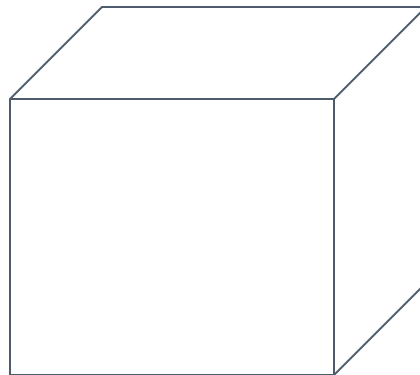
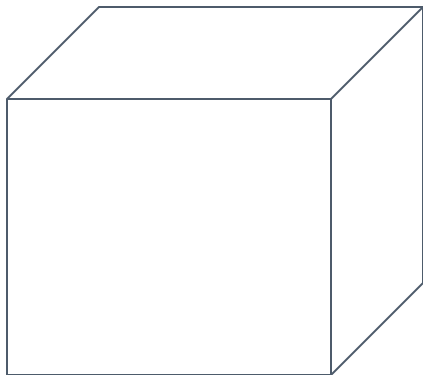
» Proof Techniques – **Pigeonhole Principle**

- » If $n + 1$ or more objects are placed into n boxes, then there is at least one box containing two or more objects.
- » In other words, if A and B are two sets such that $|A| > |B|$, then there is no one-to-one function from A to B .

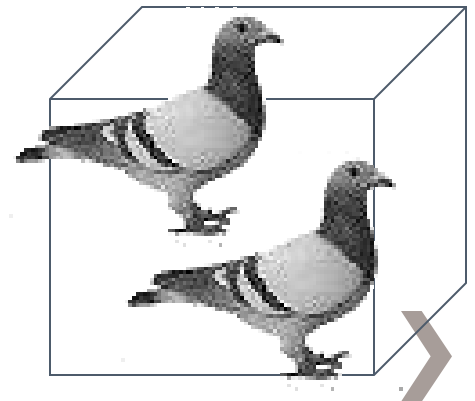
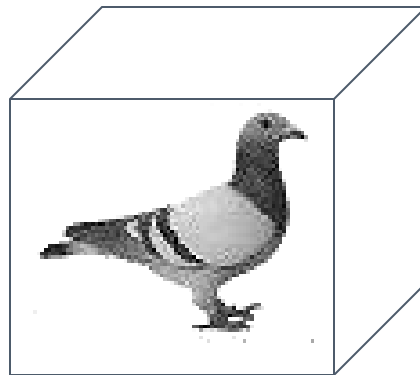
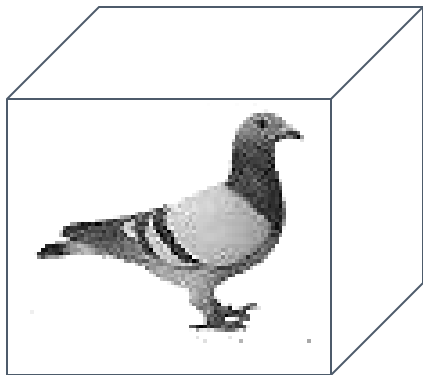
4 pigeons



3 pigeonholes



A pigeonhole must
contain at least two pigeons



- » Proof techniques: Pigeonhole Principle.
- » Example: Prove that if seven distinct numbers are selected from $\{1, 2, \dots, 11\}$, then some two of these numbers sum to 12
- » All numbers from 1 to 11 can be put into following **6 pigeonholes**: $\{1, 11\}, \{2, 10\}, \{3, 9\}, \{4, 8\}, \{5, 7\}, \{6\}$.
- » We select **7** distinct numbers (**pigeons**). First 6 pigeon can be put to different pigeonholes. But after that we have to put to an existing pigeonhole. The pigeonhole of 6 can hold only one pigeon 😊.

BLM2502 Theory of Computation

» Proof Techniques – Pigeonhole Principle

» **Theorem:** Let n be a positive integer. Every sequence of $n^2 + 1$ distinct natural numbers contains a subsequence of length $n + 1$ that is either increasing or decreasing.

» **Proof:** For example consider the sequence

(8,11,9,1,4,6,12,10,5,7)

of $10 = 3^2 + 1$ numbers. This sequence contains a decreasing subsequence of length $4 = 3 + 1$, shown. There are other subsequences of length 4, too.

Proof: Let $a_1, a_2, \dots, a_{n^2+1}$ be a sequence of $n^2 + 1$ distinct real numbers. Associate an ordered pair with each term of the sequence, namely, associate (i_k, d_k) to the term a_k , where i_k is the length of the longest increasing subsequence starting at a_k , and d_k is the length of the longest decreasing subsequence starting at a_k .

Suppose that there are no increasing or decreasing subsequences of length $n + 1$. Then i_k and d_k are both positive integers less than or equal to n , for $k = 1, 2, \dots, n^2 + 1$. Hence, by the product rule there are n^2 possible ordered pairs for (i_k, d_k) . By the pigeonhole principle, two of these $n^2 + 1$ ordered pairs are equal. In other words, there exist terms a_s and a_t , with $s < t$ such that $i_s = i_t$ and $d_s = d_t$. We will show that this is impossible. Because the terms of the sequence are distinct, either $a_s < a_t$ or $a_s > a_t$. If $a_s < a_t$, then, because $i_s = i_t$, an increasing subsequence of length $i_t + 1$ can be built starting at a_s , by taking a_s followed by an increasing subsequence of length i_t beginning at a_t . This is a contradiction. Similarly, if $a_s > a_t$, the same reasoning shows that d_s must be greater than d_t , which is a contradiction. 

Sequence=(8,11,9,1,4,6,12,10,5,7)

$$a_1 = 8, (i_1 = 3, d_1 = 3)$$

$$a_2 = 11, (i_2 = 2, d_2 = 4)$$

$$a_3 = 9, (i_3 = 2, d_3 = 3)$$

...

If there are no sequences of length $n + 1$, there are only n^2 pairs. Then since we have $n^2 + 1$ distinct numbers. At least 2 pairs are equal by pigeonhole principle. But this is not possible as numbers are **distinct**.



BLM2502

Theory of

Computation

Spring 2017

BLM2502 Theory of Computation

» Course Outline

- | » Week | Content |
|--------|---|
| » 1 | Introduction to Course |
| » 2 | Computability Theory, Complexity Theory, Automata Theory, Set Theory, Relations, Proofs, Pigeonhole Principle |
| » 3 | Regular Languages |
| » 4 | Finite Automata |
| » 5 | Deterministic and Nondeterministic Finite Automata |
| » 6 | Epsilon Transition, Equivalence of Automata |
| » 7 | Pumping Theorem |
| » | |
| » 9 | Context Free Grammars |
| » 10 | Parse Tree, Ambiguity, |
| » 11 | Pumping Theorem |
| » 13 | Turing Machines, Recognition and Computation, Church-Turing Hypothesis |
| » 14 | Turing Machines, Recognition and Computation, Church-Turing Hypothesis |
| » 15 | Review |



Regular Expressions

BLM2502 Theory of Computation

Regular Languages

» Keywords / Definitions:

- > **Alphabet:** A finite nonempty set of symbols. The members of the alphabet are the **symbols** of the alphabet. We generally use capital Greek letters Σ and Γ to designate alphabets. The following are a few examples of alphabets.

$$\Sigma_1 = \{0,1\}$$

$$\Sigma_2 = \{a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u,v,w,x,y,z\}$$

$$\Gamma = \{0, 1, x, y, z\}$$

- > A **string** over an alphabet is a finite sequence of symbols from that alphabet, usually written next to one another and not separated by commas. If $\Sigma_1 = \{0,1\}$, then 01101 is a string over Σ_1 . If $\Sigma_2 = \{a, b, c, \dots, z\}$, then abracadabra is a string over Σ_2 .

BLM2502 Theory of Computation

Regular Expressions

» Keywords / Definitions:

> If w is a string over Σ , the **length** of w , written $|w|$, is the number of symbols that it contains. The string of length zero is called the **empty string** and is written as ϵ (or λ). The empty string plays the role similar to 0 in a number system.

> If w has length n , we can write

$w = w_1 w_2 \dots w_n$ where each $w_i \in \Sigma$.
 Handwritten notes: "symbols" with an arrow pointing to w_i , "reverse" with an arrow pointing to w^R , and "string" with an arrow pointing to w .

The reverse of w , written as w^R is the string obtained by writing w in the opposite order (i.e., $w_n w_{n-1} \dots w_1$).
 Handwritten notes: "reverse" with an arrow pointing to w^R , and "string" with an arrow pointing to w . Examples: $abc = w$ and $cba = w^R$.

String z is a **substring** of w if z appears consecutively within w .
 For example, cad is a substring of abracadabra.
 Handwritten note: "string" with an arrow pointing to w .

BLM2502 Theory of Computation

Regular Expressions

» Keywords / Definitions:

- > If we have string x of length m and string y of length n , the **concatenation** of x and y , written xy , is the string obtained by appending y to the end of x , as in $x_1 \dots x_m y_1 \dots y_n$. To concatenate a string with itself many times we use the superscript notation:

$$x^3 = xxx, \text{ 3 times}$$

$$(x^n) = \overset{n}{\text{times}} \text{ } xx \dots x \text{ (n times)}$$

$$x = abc$$

$$y = hello$$

$$xy = abc \text{ hello}$$

To indicate all possible recurrences, a special superscript (in fact a special operator) is used:

$$* \text{ (kleene star)} \rightarrow x^* = \{ x^0, x^1, \dots, x^n \}$$

- > **Language**: A set of strings (finite or infinite?) over an alphabet. Languages are used to describe computation problems.

$$L_1 = \{ 01, 00, 11 \} \quad \Sigma_1 = \{ 0, 1 \}$$

BLM2502 Theory of Computation

Regular Expressions

» Keywords / Definitions:

> **Regular Languages**: A subset of all languages. There is no way to define regular set. However, it is possible to say whether a set is regular or not.

$$A^* = \{1, 2, 22, 11, \dots\}$$

> Best way is using **Finite Automata**. If some finite automata recognizes the language, then the language is regular.

> Regular (set) operations are used to build up regular sets:

> Union, concatenation, and kleene star operations are as follows.

$$A = \{1, 2, 22\}$$

> **Union**: $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$.

$$B = \{ab, aa, ba\}$$

> **Concatenation**: $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$.

$$AB = A \circ B = \{1ab, 1aa, 1ba, 2ab, 2aa, 2ba, 22ab, 22aa, 22ba\}$$

> **Star**: $A^* = \{x_1 x_2 \dots x_k \mid k > 0 \text{ and each } x_i \in A\}$.

BLM2502 Theory of Computation

Regular Expressions

- » Keywords / Definitions:
- > Class of regular languages is closed under **union**
 $\rightarrow$ If A and B is regular $A \cup B$ is regular
 - > Class of regular languages is closed under **intersection**
 $\rightarrow A \cap B$ regular
 - > Class of regular languages is closed under **complement**
 $\rightarrow A^c$ is regular
 - > Class of regular languages is closed under **concatenation**
 $\rightarrow A \cdot B$ is regular
 - > Class of regular languages is closed under (kleene) **star**
 $\rightarrow A^*$ is regular

BLM2502 Theory of Computation

Alphabet: $= \{a, b\}$

Strings:

a

ab

abba

aaabbbbaabab

u = ab

v = bbbbaaa

w = abba

BLM2502 Theory of Computation

Decimal numbers

Alphabet: $\Sigma = \{0,1,2,\dots,9\}$

Strings:

102345 567463386 ...

Binary numbers

Alphabet: $\Sigma = \{0,1\}$

Strings:

100010001 101101111 ...

BLM2502 Theory of Computation

Unary numbers

Alphabet: $\Sigma = \{1\}$

Strings:

1 11 111111 ...

Decimal equivalent:

1,2,6...

BLM2502 Theory of Computation

» Examples on String Operations

$$w = a_1a_2...a_n, v = b_1b_2...b_m \quad a, b \in \{x, y\}$$

$$\text{eg, } w = xyxy, v = xxxyyy$$

Concenation:

$$wv = a_1a_2...a_nb_1b_2...b_n$$

$$\text{eg, } xyxyxxxxyy$$

Reverse:

$$w^R = a_na_{n-1}...a_1$$

$$\text{eg, } yxyyx$$

BLM2502 Theory of Computation

» Examples on String Operations

$$w = a_1a_2...a_n, v = b_1b_2...b_m \quad a, b \in \{x, y\}$$

Length:

$$|w| = n, |v| = m$$

$$\text{eg, } |yxyyx| = 5; |xxxyyy| = 6$$

$$|wv| = |w| + |v|$$

$$\text{eg, } |yxyyxxxxyyy| = 11 \text{ (recall example above)}$$

Empty String:

$$|\varepsilon| = 0$$

BLM2502 Theory of Computation

» Examples on String Operations

$$w = a_1a_2\dots a_n, v = b_1b_2\dots b_m \quad a, b \in \{x, y\}$$

Substring:

$a_1, a_2, \dots, a_n$ (each symbol of the string are its substrings)

$a_1, a_1a_2, a_1a_2a_3\dots$ (these are prefix substrings)

$a_2, a_2a_3, a_2a_3a_4, \dots$

$a_n, a_{n-1}a_n, a_{n-2}a_{n-1}a_n, \dots$ (these are suffix substrings)

special cases:

ε is prefix and suffix of each string

any string is prefix and suffix of itself

BLM2502 Theory of Computation

» Examples on String Operations

Special cases:

ϵ is prefix and suffix of each string

any string is prefix and suffix of itself

String:  ϵ

Prefix	Suffix
ϵ	abba
a	bba
ab	ba
abb	a
abba	ϵ

Observe that:

$$\epsilon w = w\epsilon = w$$

BLM2502 Theory of Computation

» Examples on String Operations

$$w = a_1a_2...a_n, v = b_1b_2...b_m \quad a, b \in \{x, y\}$$

Self Concenation:

$$ww = w^2, www = w^3, \text{ etc}$$

$$w^0 = \varepsilon$$

eg, $w = abba$;

$$(abba)^0 = \varepsilon$$

$$(abba)^1 = abba$$

$$(abba)^2 = abbaabba$$

$$(abba)^3 = abbaabbaabba$$

Kleene Star:

$$w^* = \{w^0, w^1, w^2, w^3, \dots\}$$

BLM2502 Theory of Computation

» The * operation:

The set of all possible strings from the alphabet

$$\Sigma^+ = \{a, b\}$$

$$\Sigma^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, \dots\} \rightarrow \text{it is infinite}$$

» The + operation:

The set of all possible strings from the alphabet excluding ϵ

$$\Sigma = \{a, b\}$$

$$\Sigma^+ = \boxed{\Sigma^* - \{\epsilon\}} = \{a, b, aa, ab, ba, bb, aaa, aab, \dots\}$$

BLM2502 Theory of Computation

» Language:

A Language over an alphabet $\Sigma = \{a, b\}$

is a subset of $\Sigma^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$

Examples:

$$L_1 = \{\epsilon\}$$

$$L_2 = \{a, b, aa, ab, ba, bb\}$$

$$L_3 = \{\epsilon, a, aa, aaa, aaaa, \dots\}$$

$$L_4 = \{a^n b^n, n \geq 0\} = \{\epsilon, ab, aabb, aaabbbb, \dots\}$$

$$\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

$$\begin{aligned} \text{PRIME_NUMBERS} &= \{x \mid x \in \Sigma^* \text{ and } x \text{ is prime}\} \\ &= \{2, 3, 5, 7, 11, \dots\} \end{aligned}$$

$$\text{EVEN_NUMBERS} = \{x \mid x \in \Sigma^* \text{ and } x \text{ is even}\} = \{0, 2, 4, \dots\}$$

$$\text{ODD_NUMBERS} = \{x \mid x \in \Sigma^* \text{ and } x \text{ is odd}\} = \{1, 3, 5, \dots\}$$

BLM2502 Theory of Computation

» Unary Addition

Alphabet $\Sigma = \{1, +, =\}$

ADDITION = $\{x + y = z : x = 1^m, y = 1^n, z = 1^k; k = m + n\}$

$111 + 11 = 11111 \in \text{ADDITION}$

$111 + 111 = 1111 \notin \text{ADDITION}$

» Squaring

Alphabet $\Sigma = \{1, \#\}$

SQUARES = $\{x \# y : x = 1^m, y = 1^n, n = m^2\}$

$111 \# 111111111 \in \text{SQUARES}$

$1111 \# 11111111 \notin \text{SQUARES}$

BLM2502 Theory of Computation

» Operations On Languages

Set Operations: Regular sets are closed under union, intersection, negation and complement.

$$A = \{a, ab, aaaa\}$$

$$B = \{ab, bb\}$$

$$A \cup B = \{a, ab, bb, aaaa\}$$

$$A \cap B = \{ab\}$$

$$A - B = \{a, \cancel{bb}, aaaa\}$$

$$\bar{A} = \Sigma^* - A = \{\overline{a}, \overline{ab}, \overline{aaaa}\} = \{\epsilon, aa, ba, bb, aba, \dots\}$$

Note that :

$$\emptyset = \{\} \neq \{\epsilon\}$$

$$|\emptyset| = |\{\}| = 0 \neq |\{\epsilon\}|$$

$$|\{\epsilon\}| = 1 \quad * \text{ This is set size}$$

$$|\epsilon| = 0 \quad * \text{ This is string length}$$

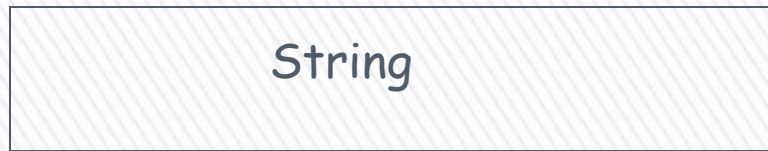


Finite Automata

BLM2502 Theory of Computation

»

Input Tape



Output



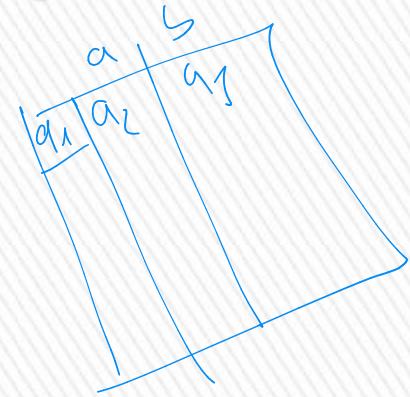
BLM2502 Theory of Computation

Finite Automata

» A finite automaton is a 5-tuple:

$(Q, \Sigma, \delta, q_0, F)$ where;

1. Q is a finite set called the **states**,
2. Σ is a finite set called the **alphabet**,
3. $\delta: Q \times \Sigma \rightarrow Q$ is the **transition function**,
4. $q_0 \in Q$ is the **start state**, and q_0
5. $F \subseteq Q$ is the **set of accept states**.



$$F = \{q_0, q_1\}$$

BLM2502 Theory of Computation

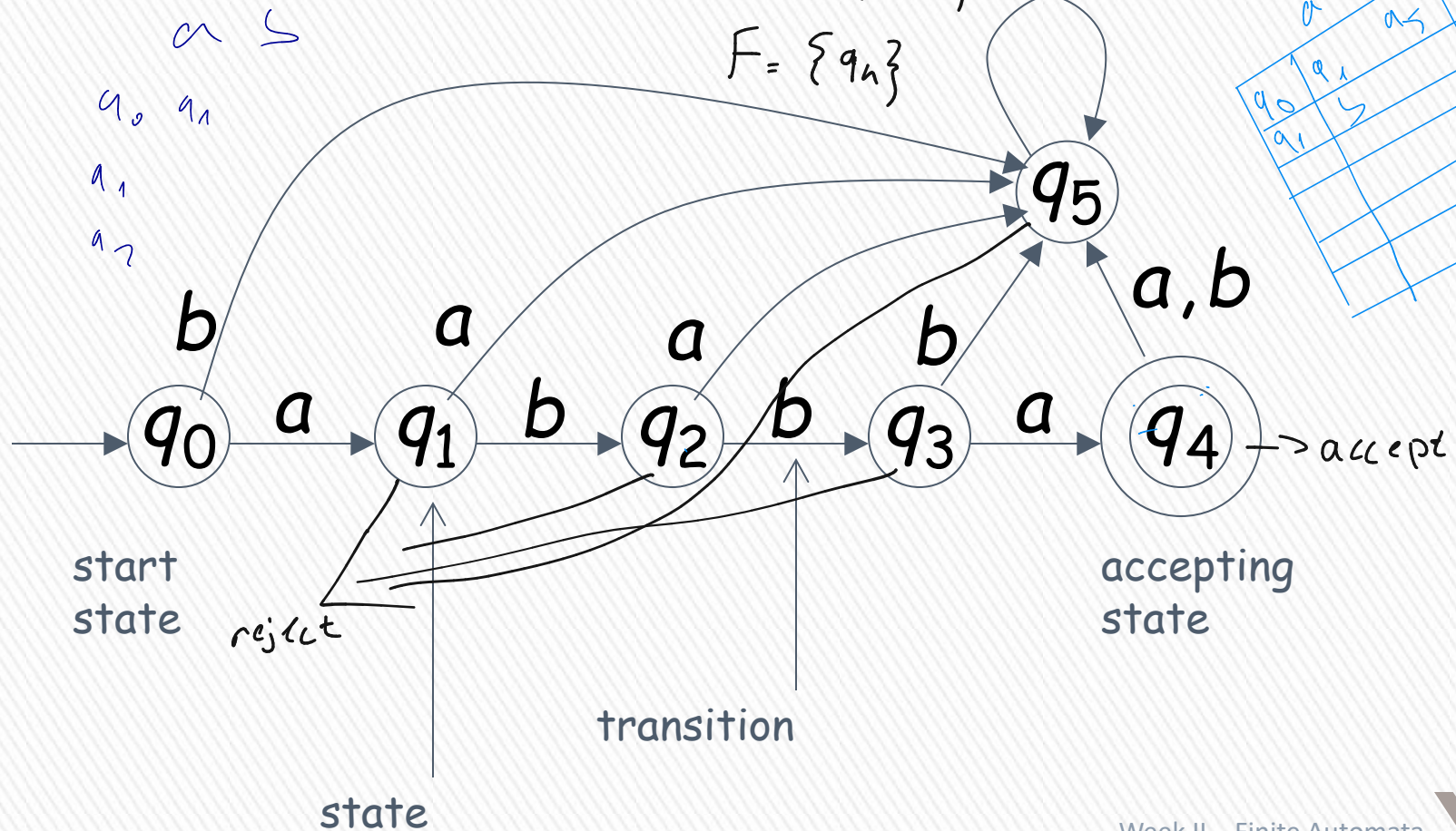
$$Q = \{q_0, q_1, q_2, \dots\}$$

$$\Sigma = \{a, b\} \quad a, b$$

$$F = \{q_n\}$$

start state q_0

Transition Graph



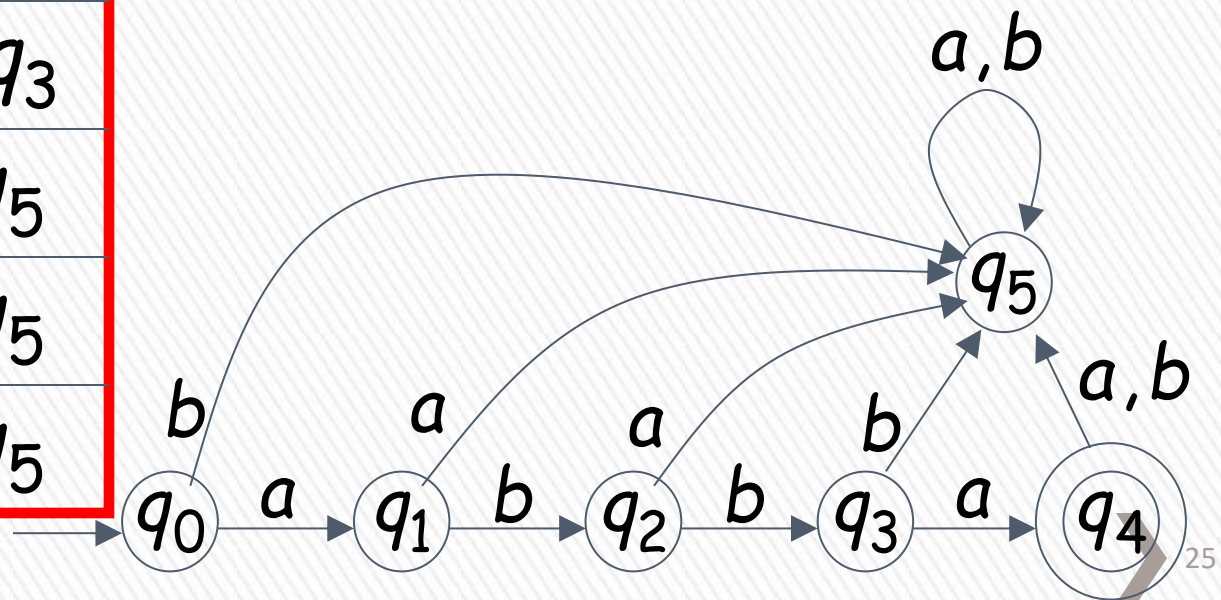
BLM2502 Theory of Computation

symbols

states

δ	a	b
q_0	q_1	q_5
q_1	q_5	q_2
q_2	q_5	q_3
q_3	q_4	q_5
q_4	q_5	q_5
q_5	q_5	q_5

Transition Table



BLM2502 Theory of Computation

- » You can think of the transition function as being the “program” of the finite automaton M . This function tells us what M can do in “one step”:
 - » Let r be a state of Q and let a be a symbol of the alphabet Σ . If the finite automaton M is in state r and reads the symbol a , then it switches from state r to state $\delta(r, a)$. (In fact, $\delta(r, a)$ may be equal to r .)
- » Example:
 - » $A = \{w : w \text{ is a binary string containing an odd number of } 1\text{s}\}.$

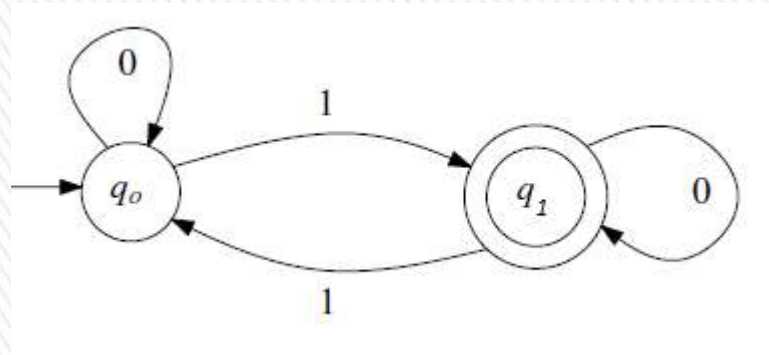
BLM2502 Theory of Computation

Design:

- » The finite automaton reads the input string w from left to right and keeps track of the number of 1s it has seen. After having read the entire string w , it checks whether this number is odd (in which case w is accepted) or even (in which case w is rejected).
- » Using this approach, the finite automaton needs a state for every integer $i \geq 0$, indicating that the number of 1s read so far is equal to i .

BLM2502 Theory of Computation

- » Design – Continued
- » However, this design is not feasible since FA have finite number of states.
- » ???
- » A better, and correct approach, is to keep track of whether the number of 1s read so far is even or odd.



BLM2502 Theory of Computation

$$M = (Q, \Sigma, \delta, q_0, F)$$

$$Q = \{q_0, q_1\}$$

$$\Sigma = \{0, 1\} \text{ (trivial from the problem)}$$

$$q_0 \in Q$$

$$F = \{q_1\}$$

δ :

$$(q_0, 0) \rightarrow q_0$$

$$(q_0, 1) \rightarrow q_1$$

$$(q_1, 0) \rightarrow q_1$$

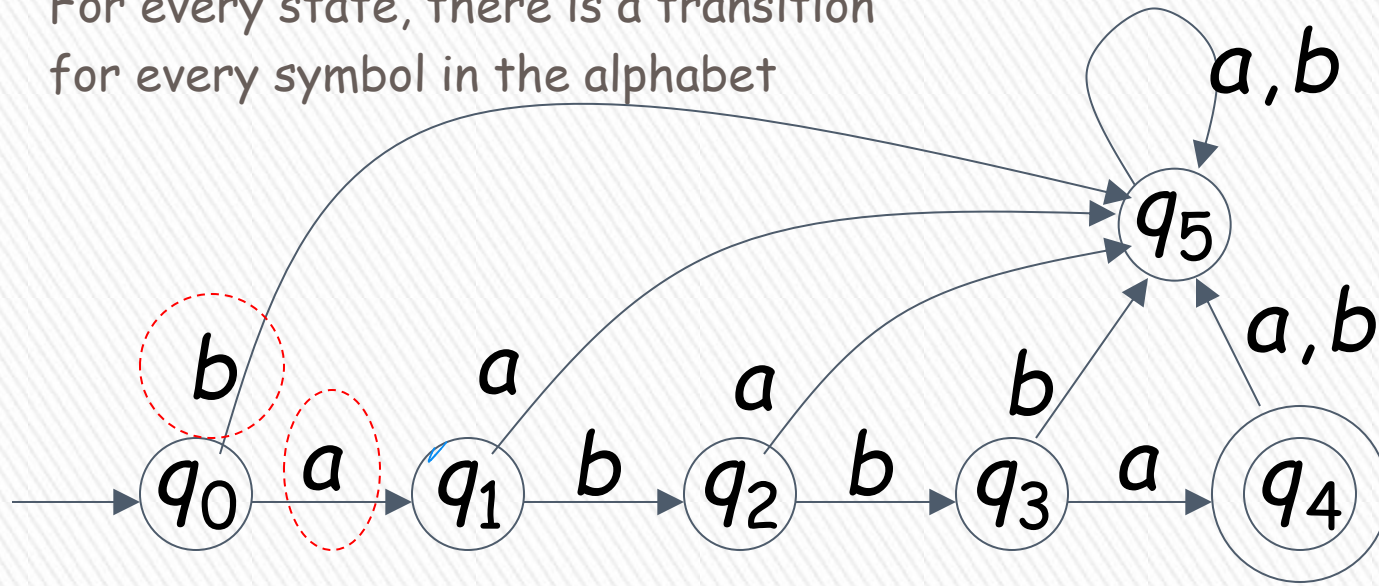
$$(q_1, 1) \rightarrow q_0$$

δ :	0	1
q_0	q_0	q_1
q_1	q_1	q_0

BLM2502 Theory of Computation

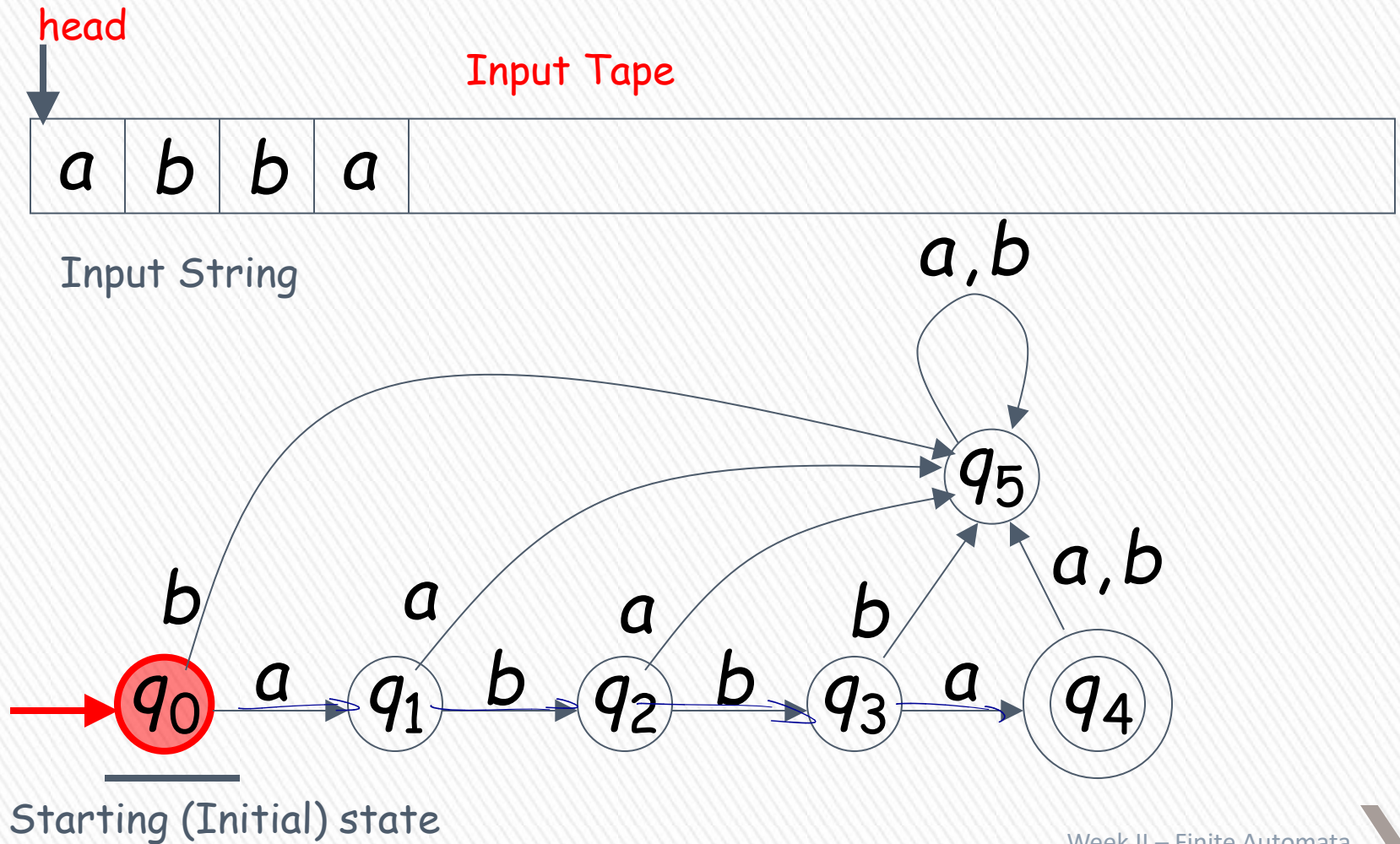
» DFA - Deterministic Finite Automata

For every state, there is a transition for every symbol in the alphabet



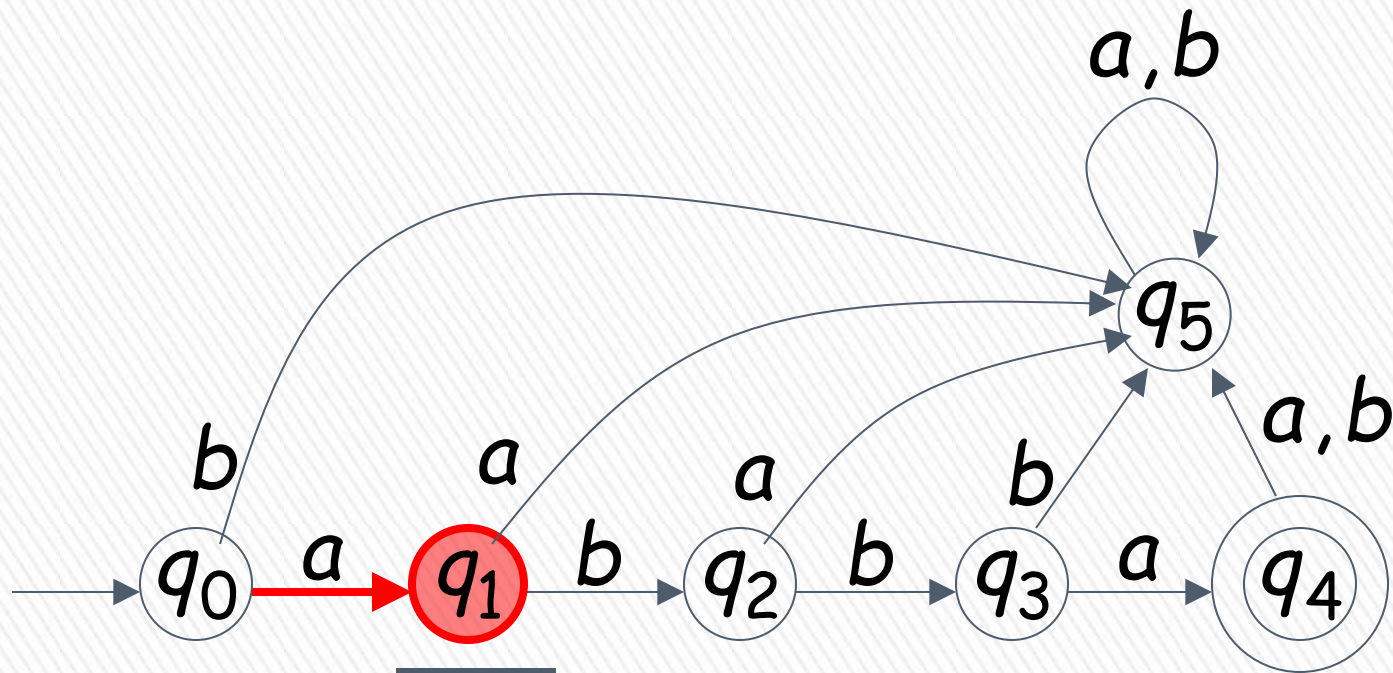
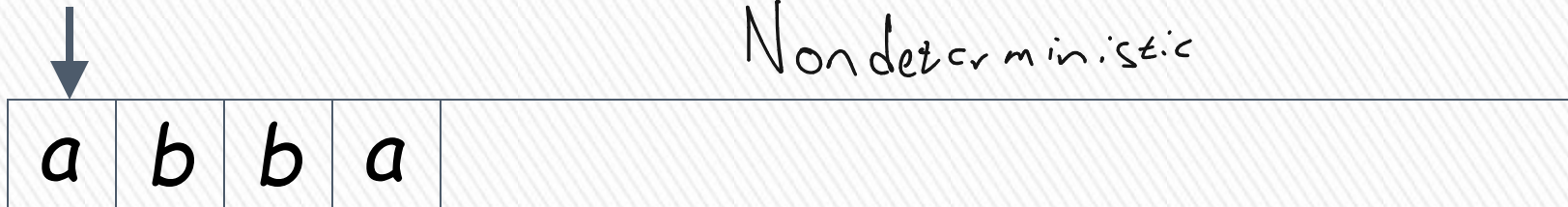
Alphabet $\Sigma = \{a, b\}$

BLM2502 Theory of Computation

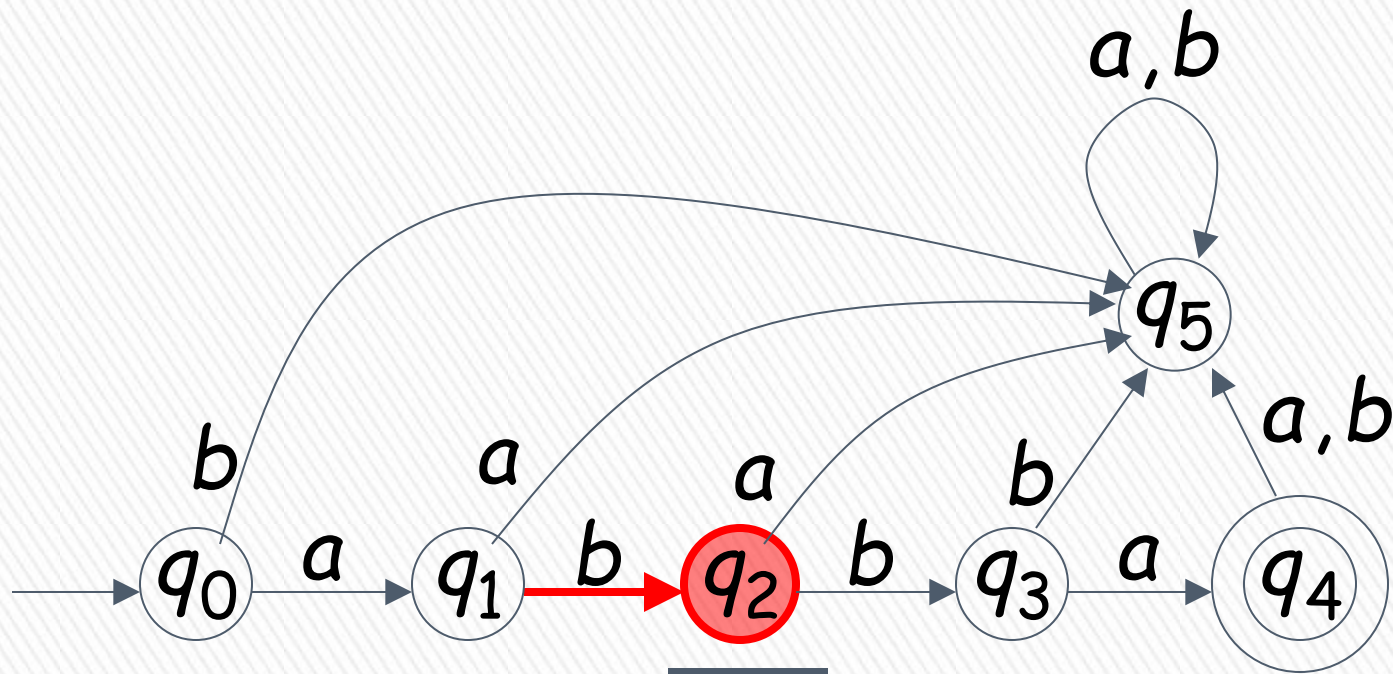


BLM2502 Theory of Computation

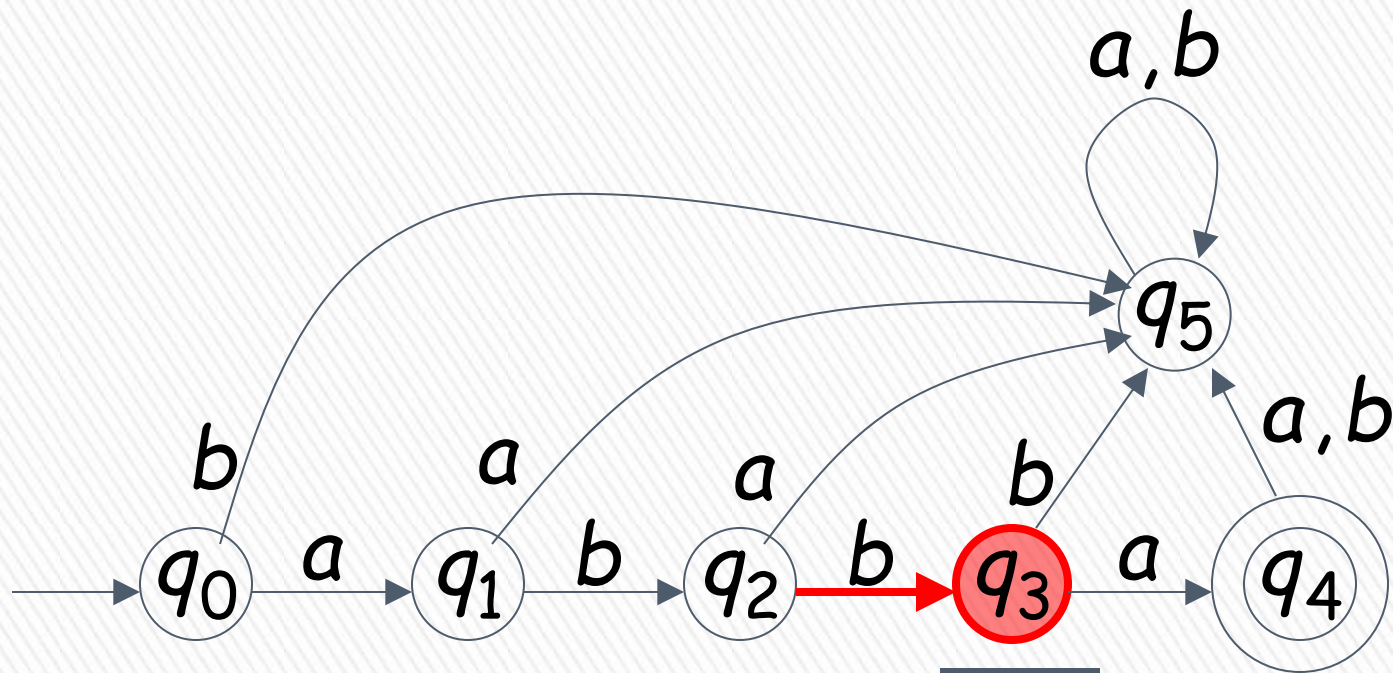
Non deterministic



BLM2502 Theory of Computation

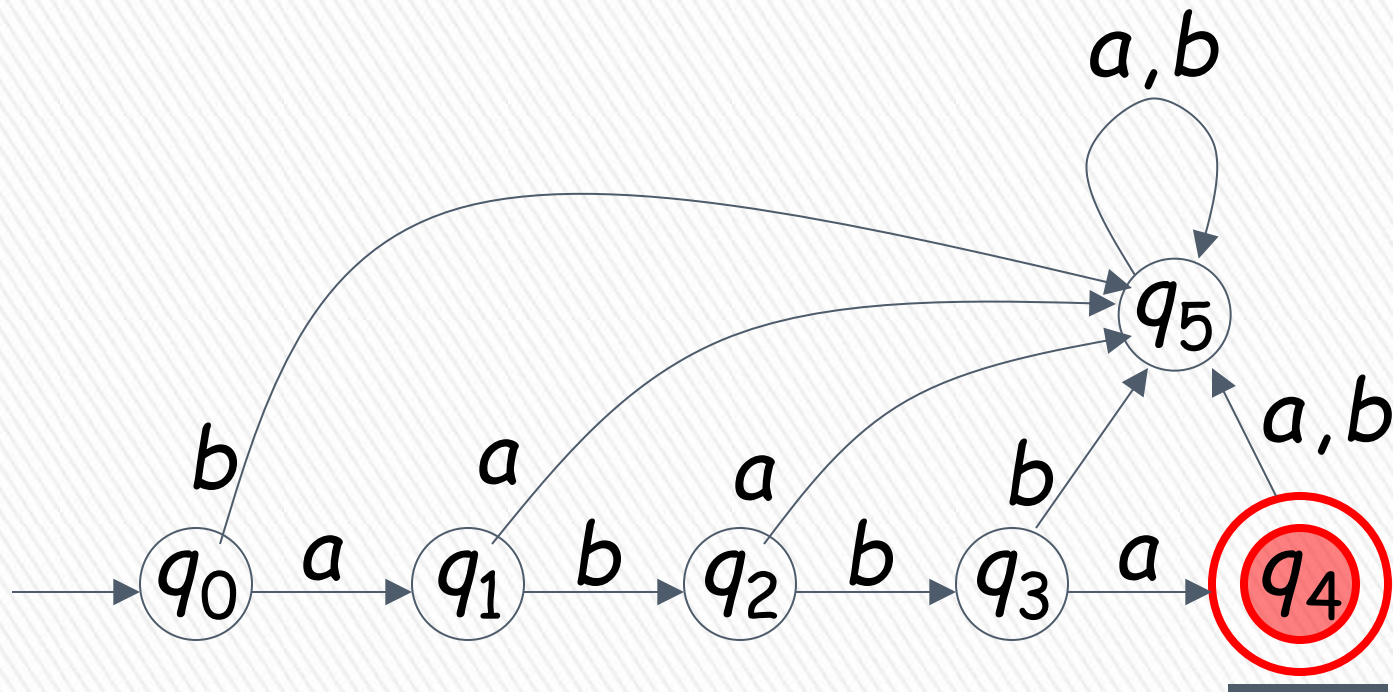


BLM2502 Theory of Computation



BLM2502 Theory of Computation

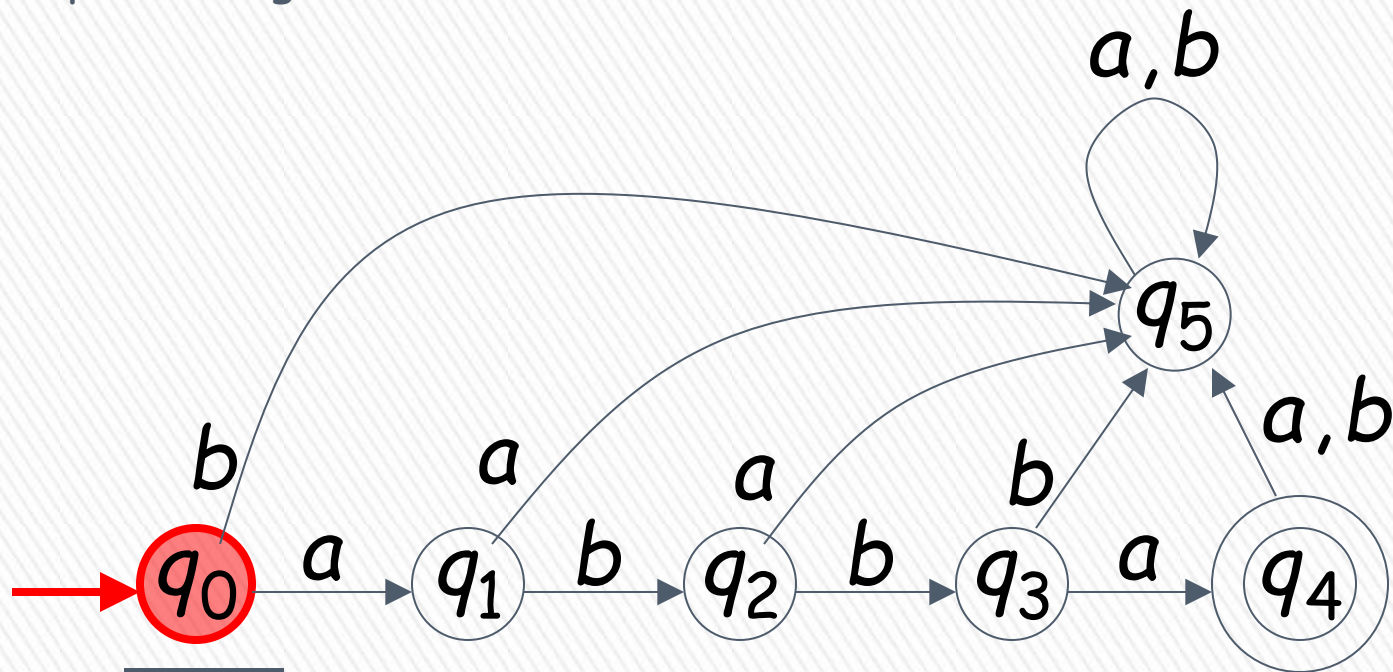
Input finished
↓



BLM2502 Theory of Computation

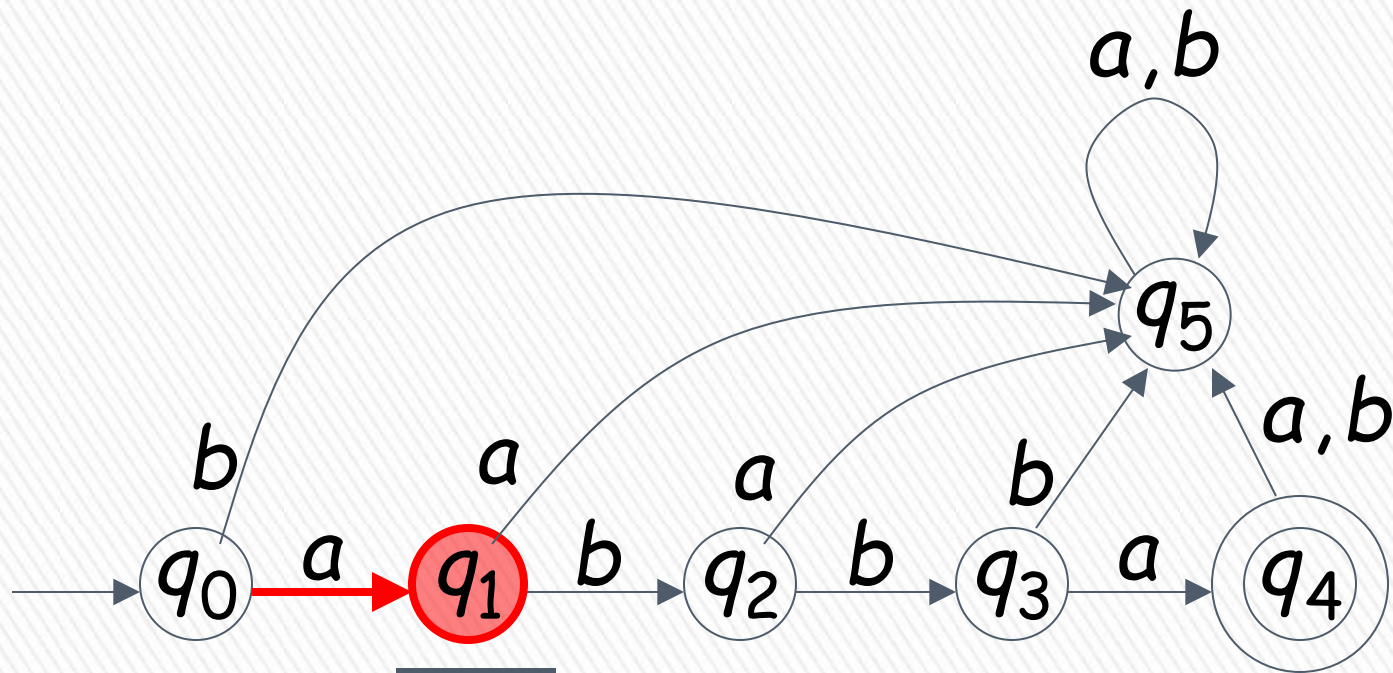


Input String

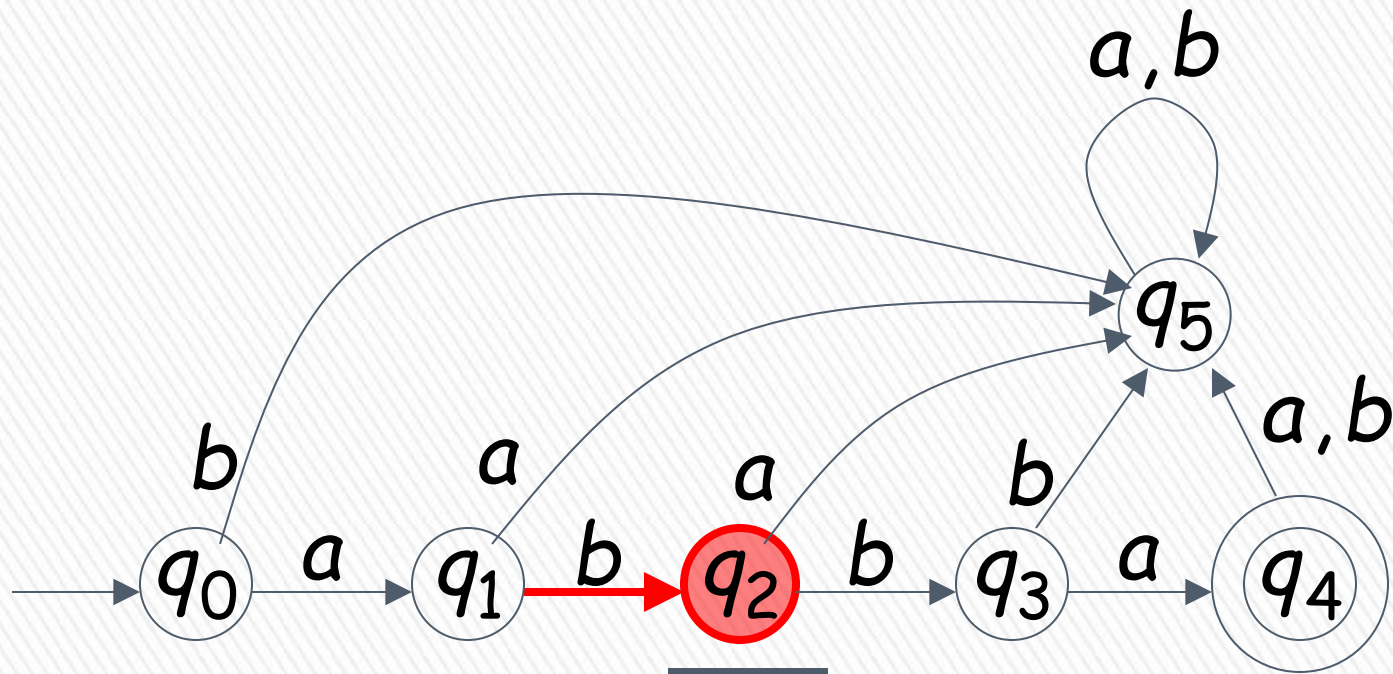


A Rejection Case

BLM2502 Theory of Computation

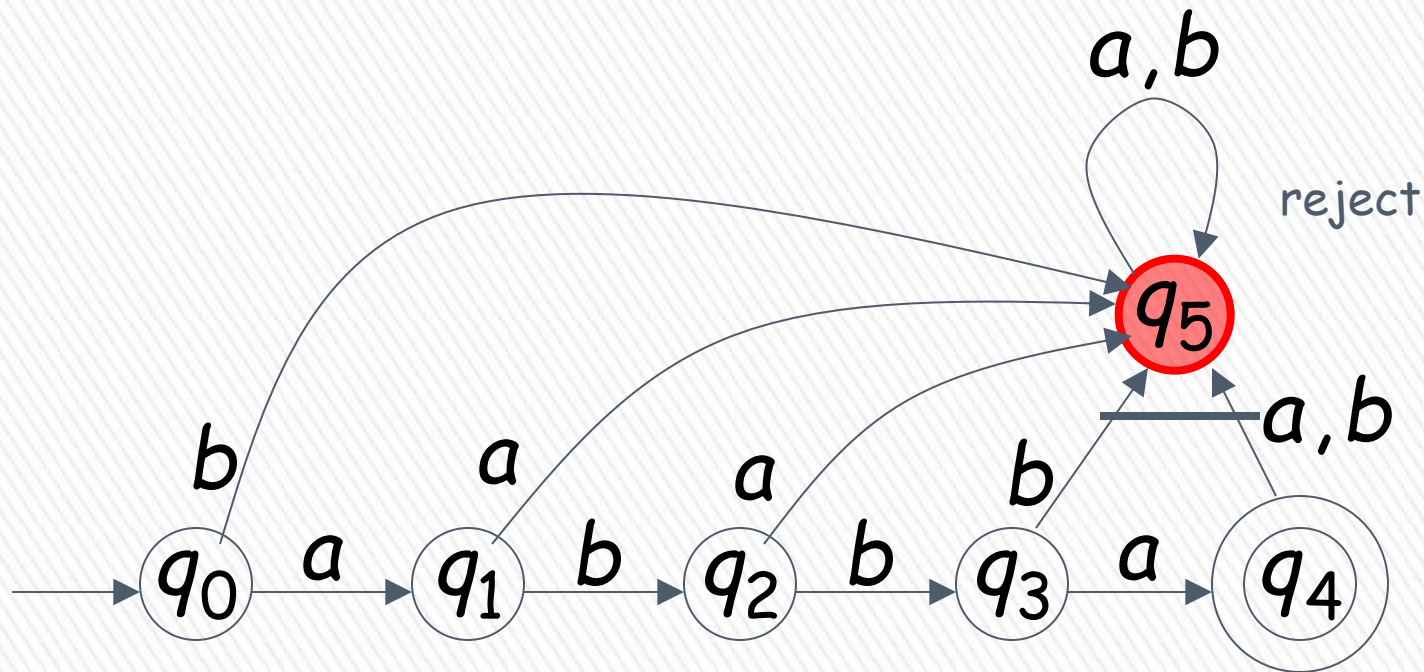


BLM2502 Theory of Computation



BLM2502 Theory of Computation

↓ Input finished

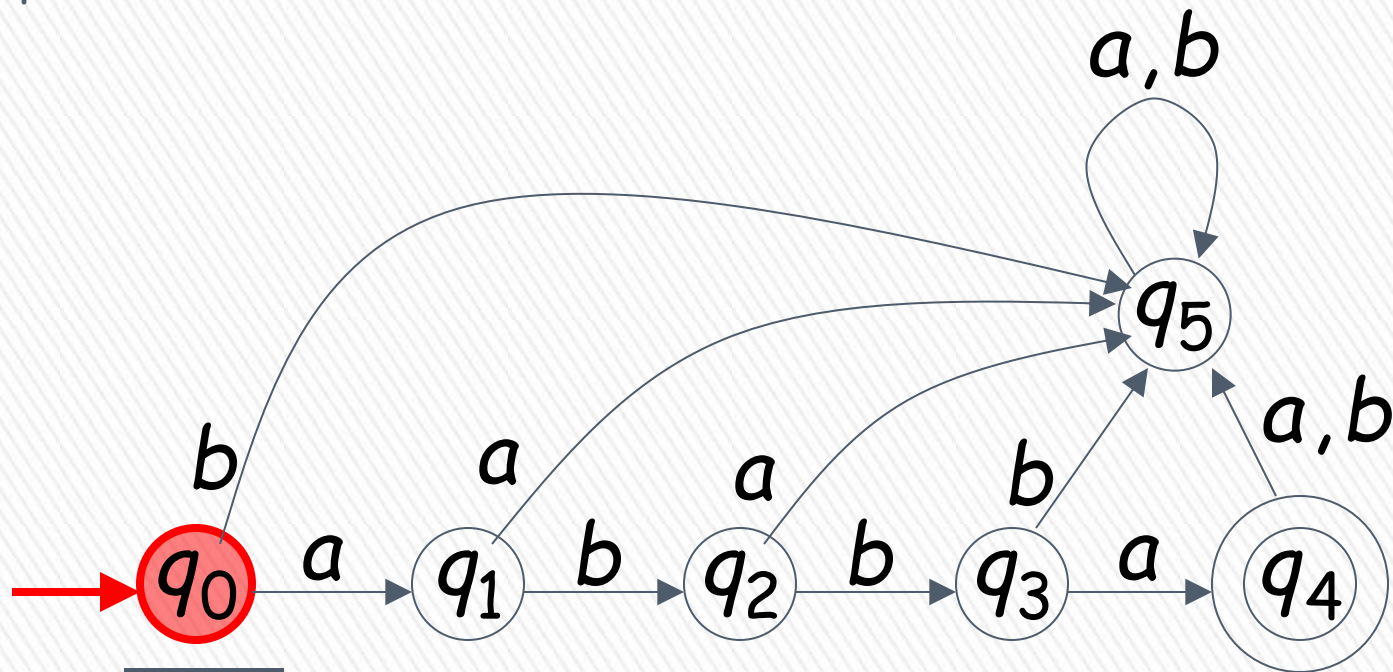


BLM2502 Theory of Computation

↓ Tape is empty



Input Finished



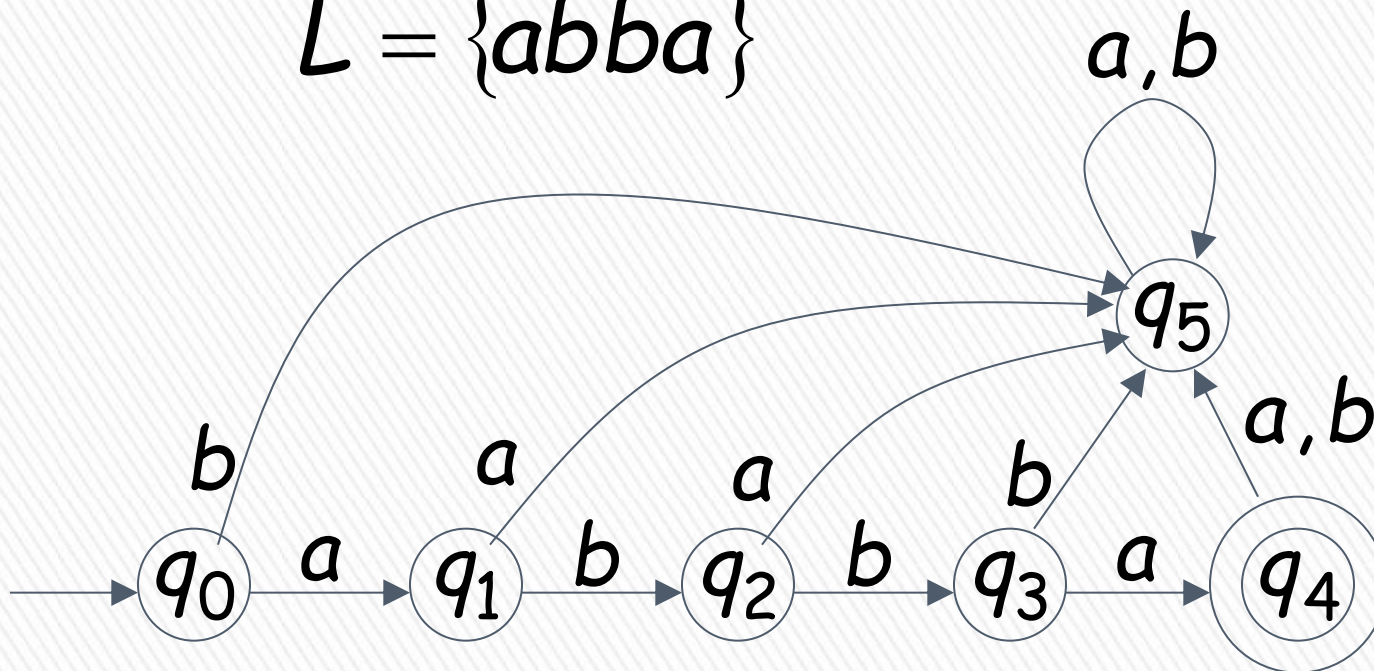
reject

Another Rejection Case

BLM2502 Theory of Computation

Language Accepted:

$$L = \{abba\}$$



BLM2502 Theory of Computation

To accept a string:

- all the input string is scanned and
- the last state is accepting

To reject a string:

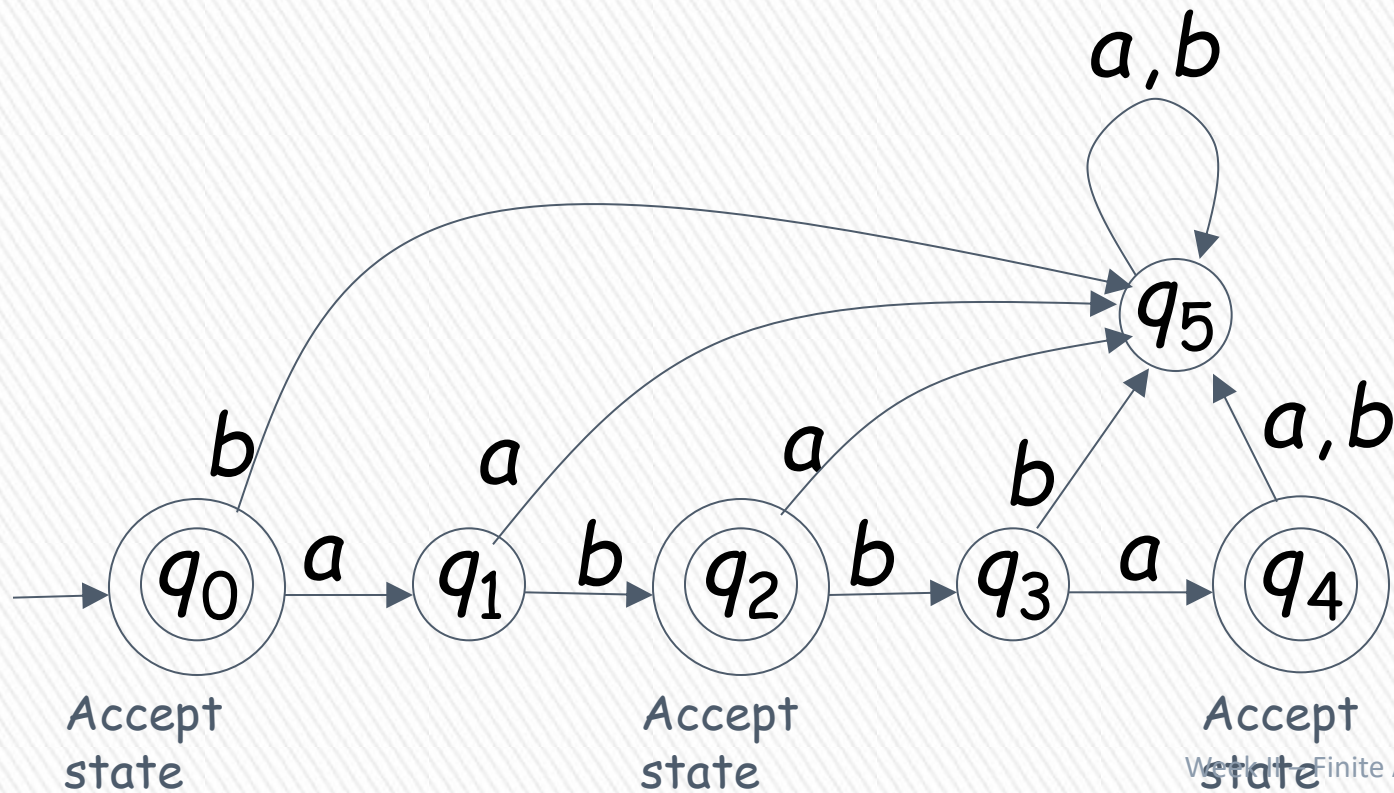
- all the input string is scanned and
- the last state is non-accepting



BLM2502 Theory of Computation

$L = \{\epsilon, ab, abba\}$

abba



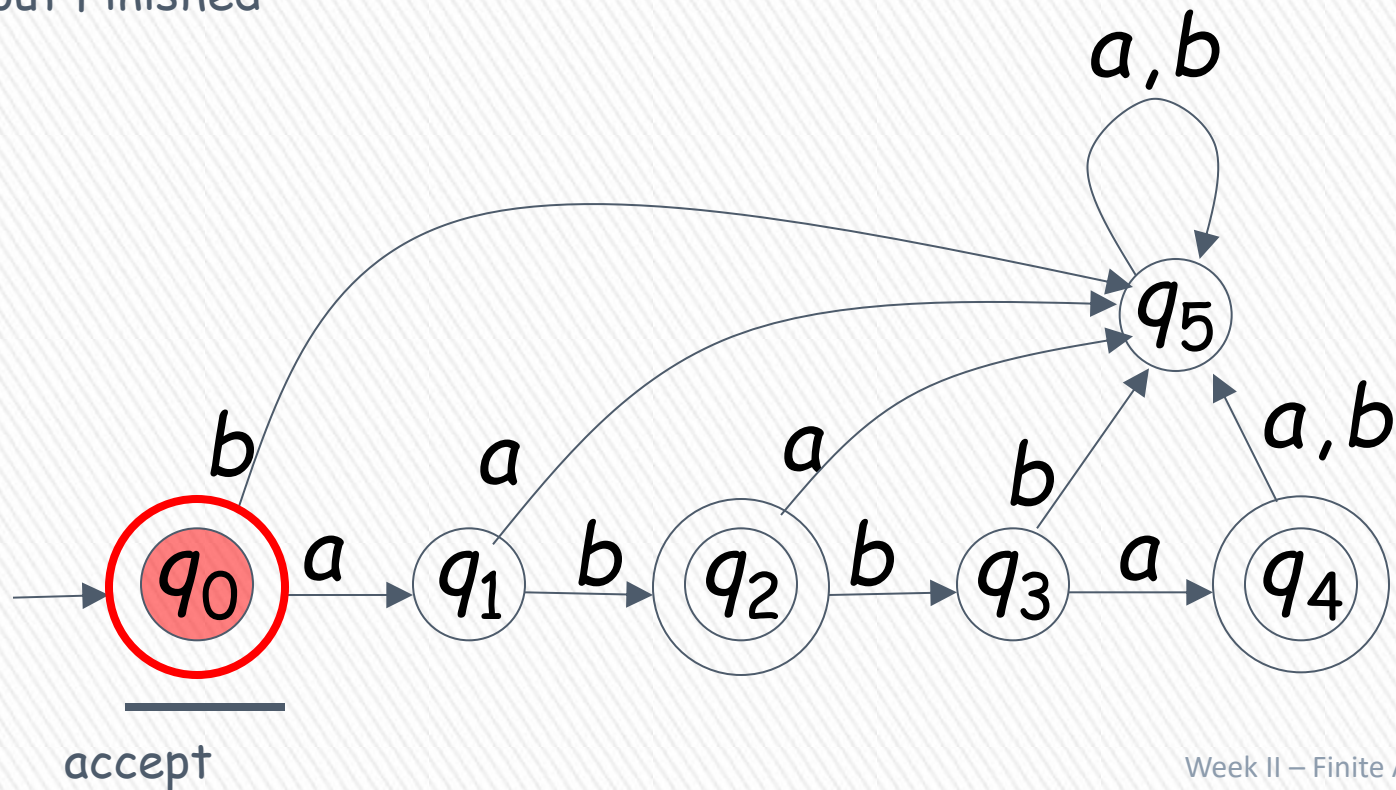
BLM2502 Theory of Computation



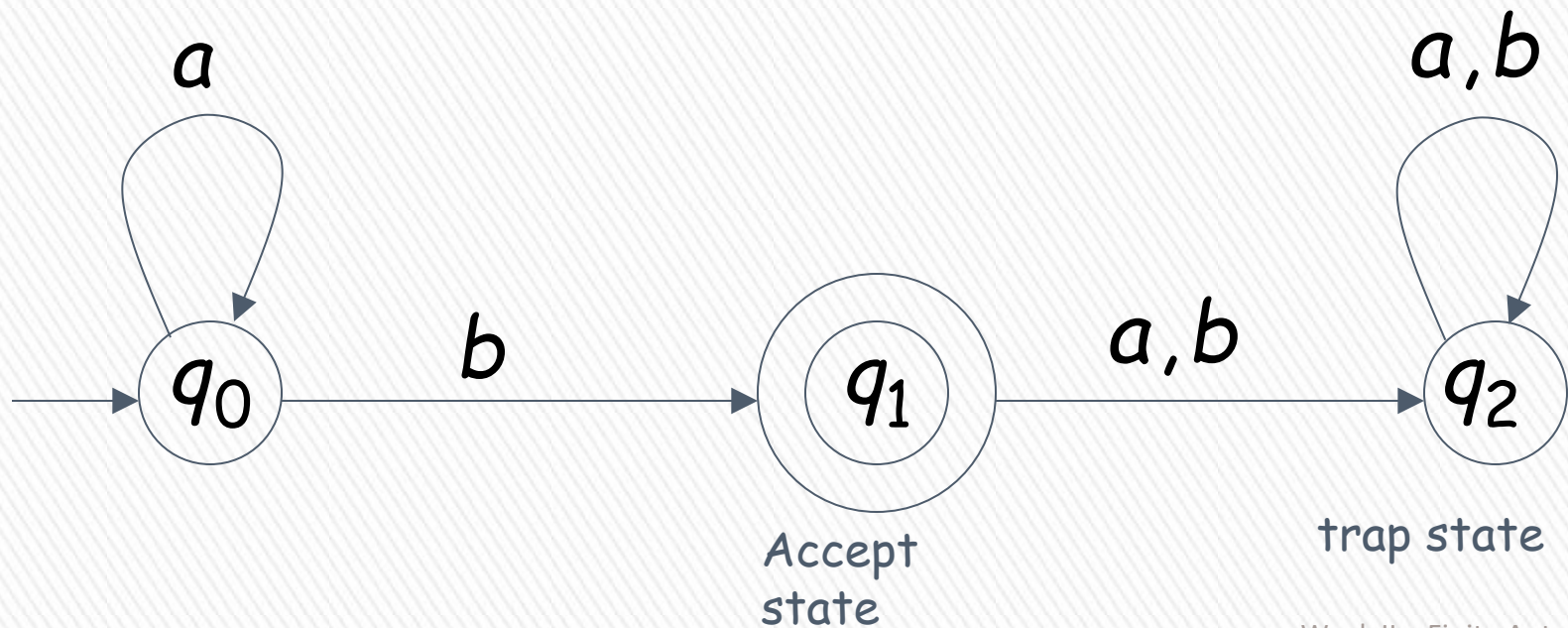
Empty Tape



Input Finished



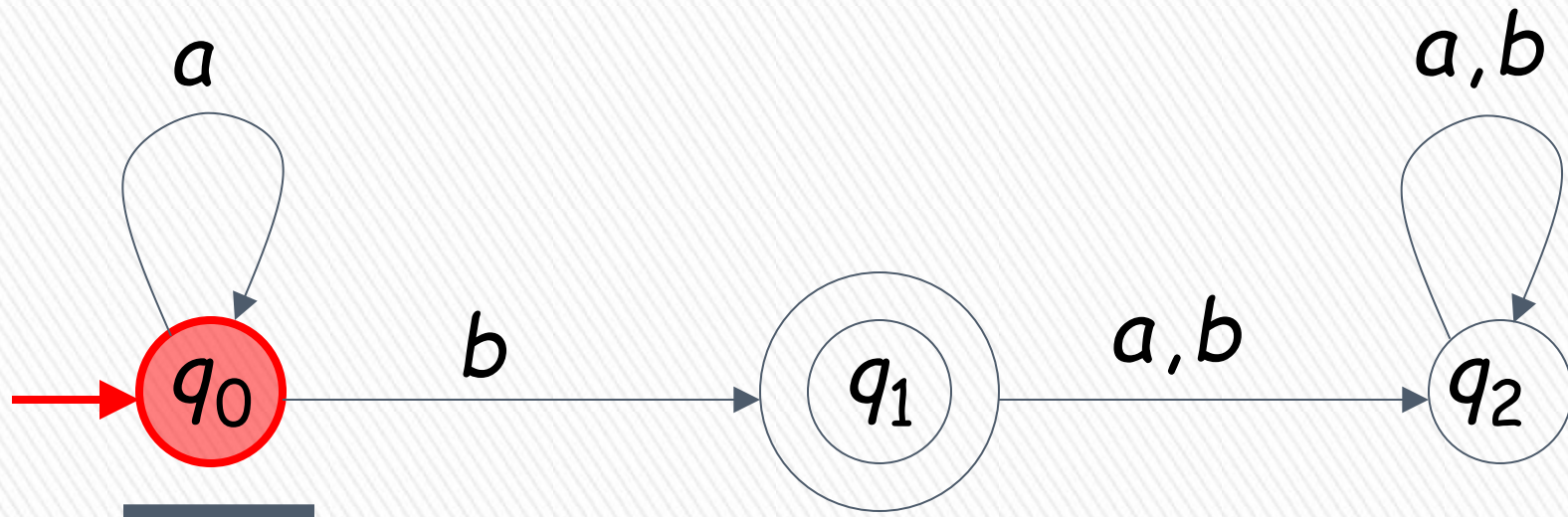
BLM2502 Theory of Computation



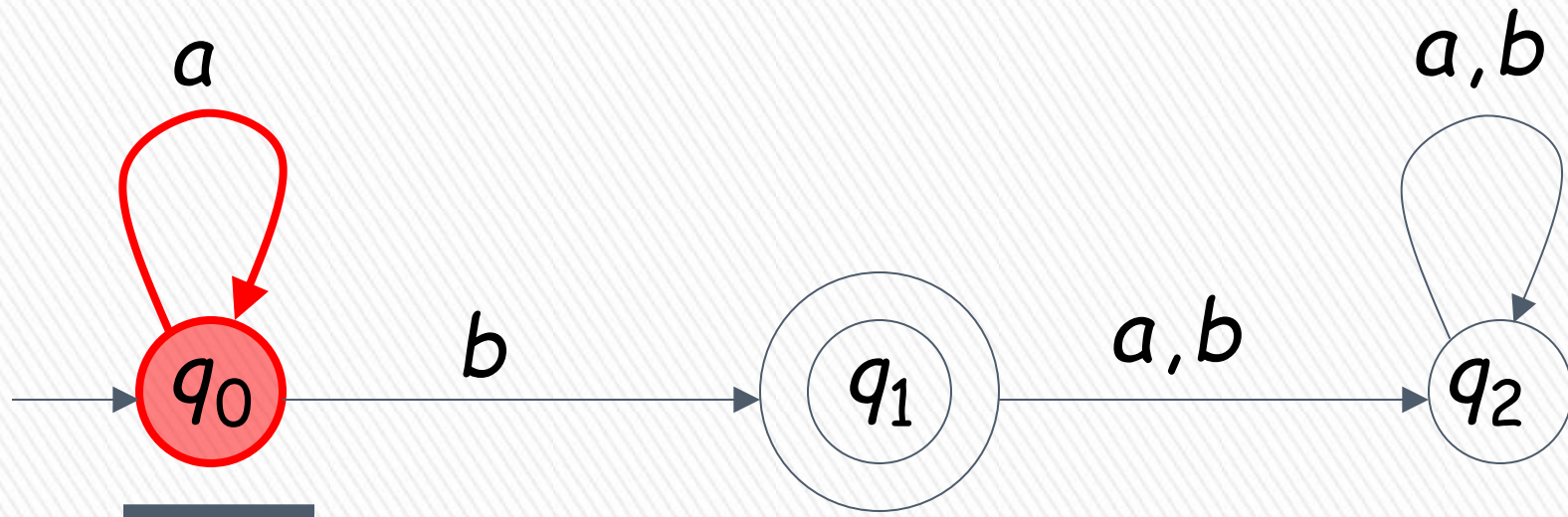
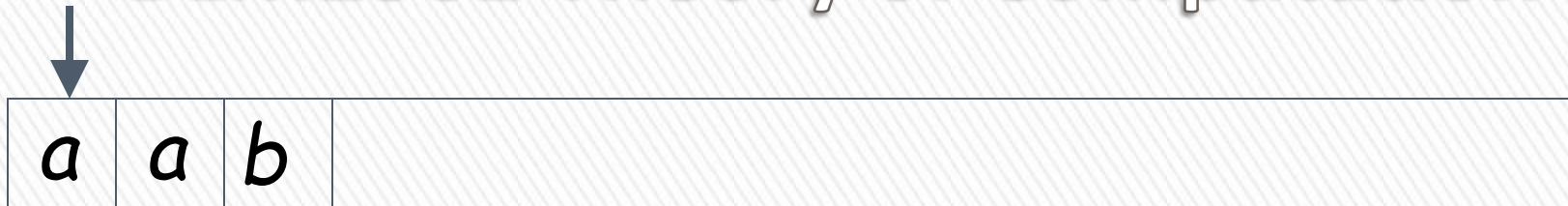
BLM2502 Theory of Computation



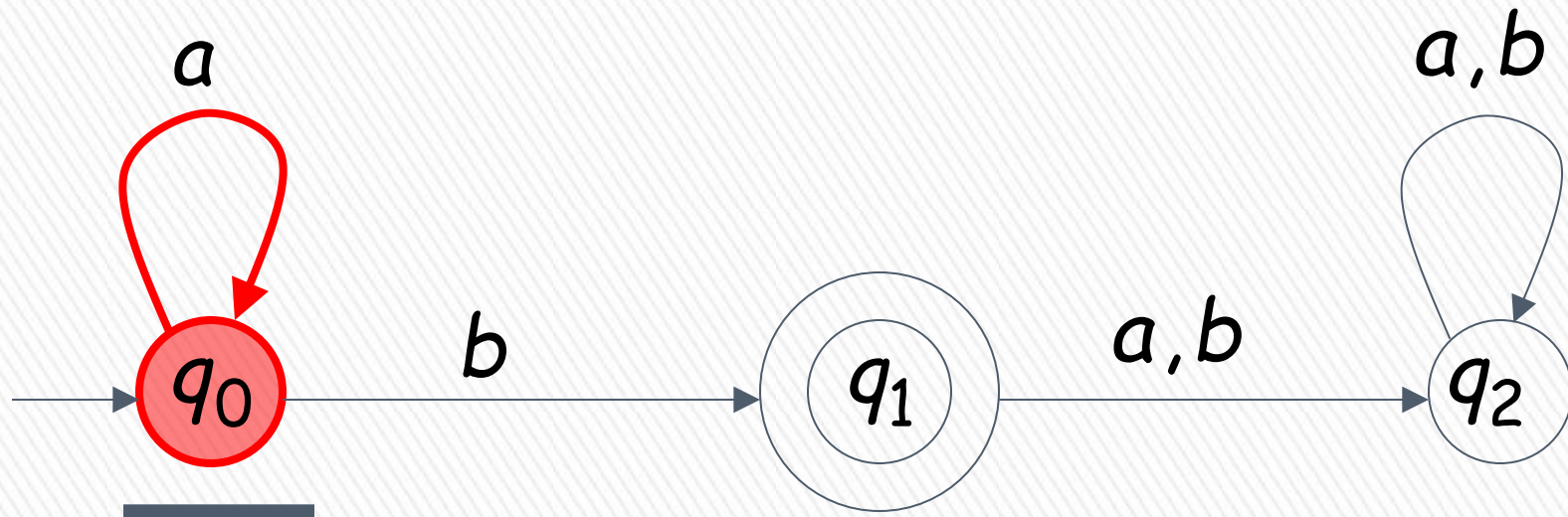
Input String



BLM2502 Theory of Computation

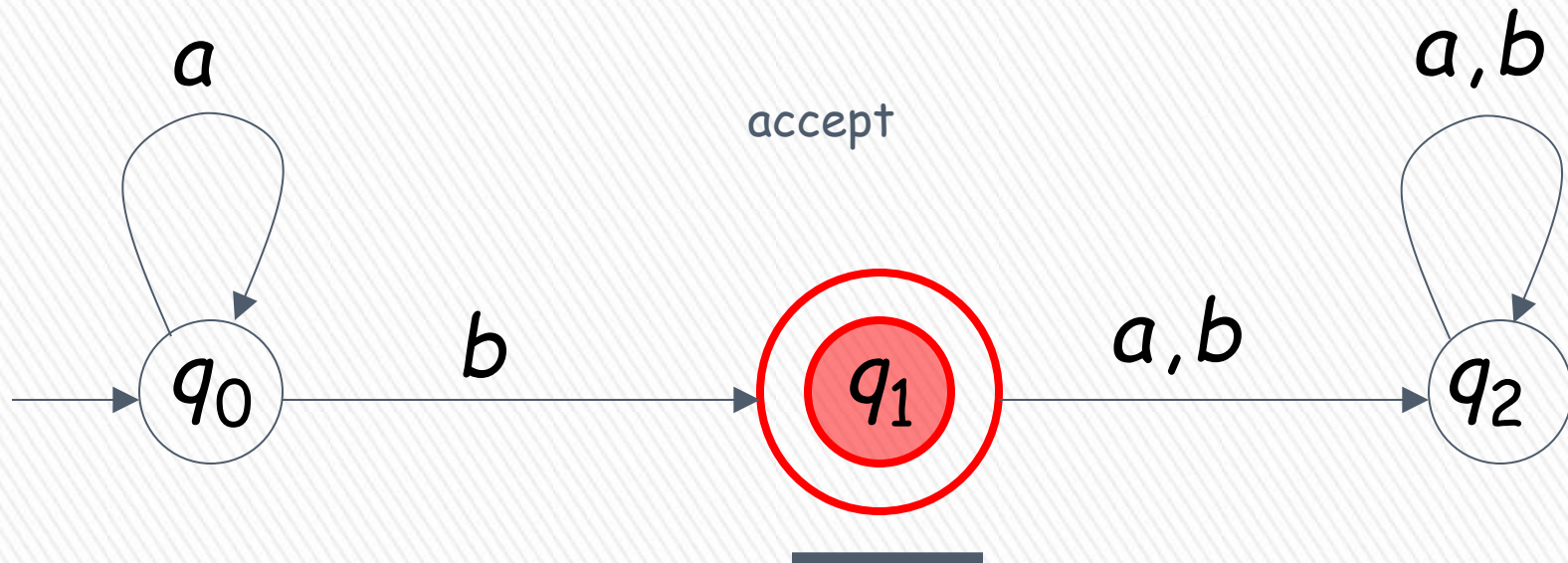


BLM2502 Theory of Computation



BLM2502 Theory of Computation

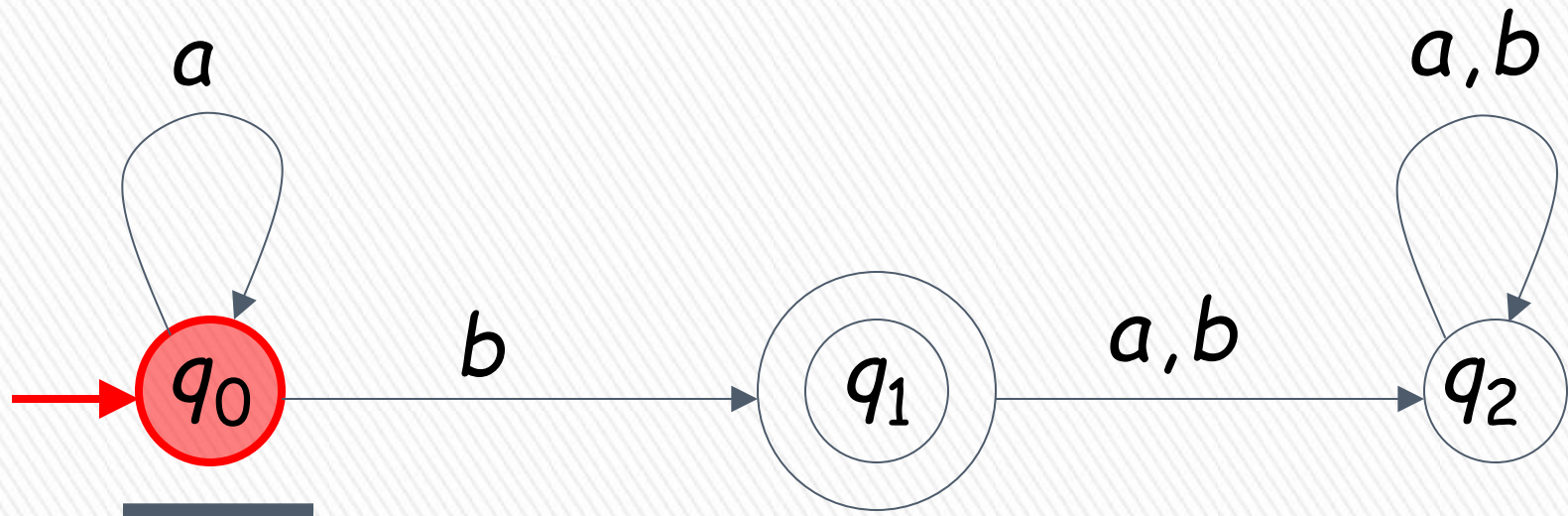
↓ Input finished



BLM2502 Theory of Computation

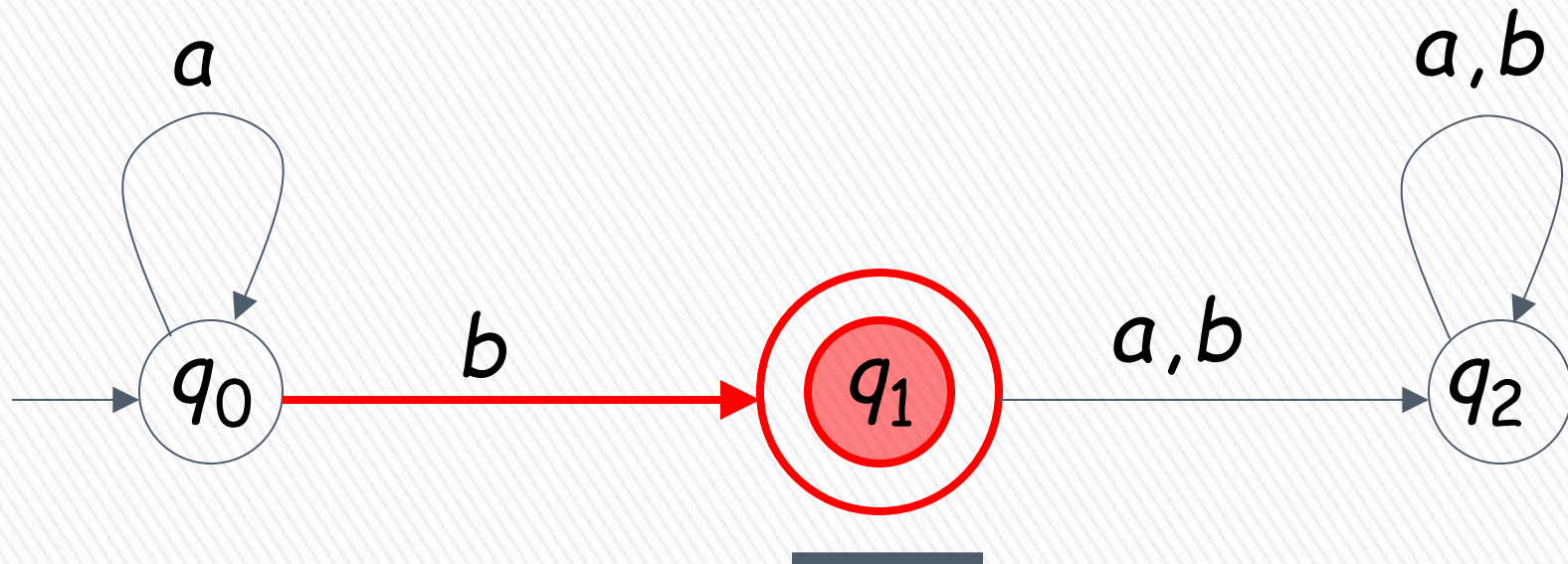
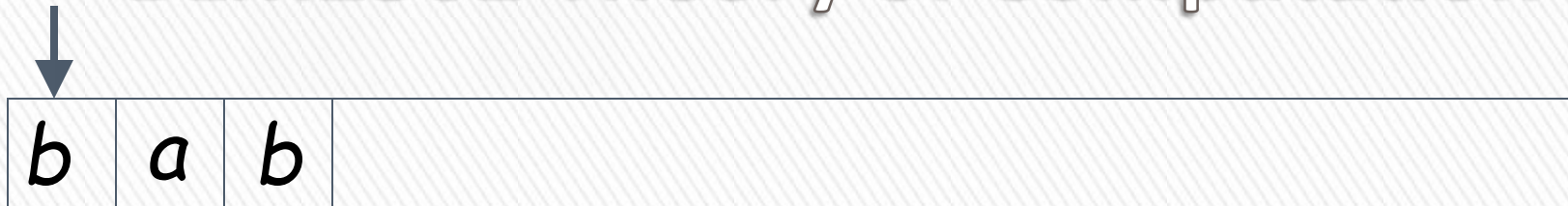


Input String

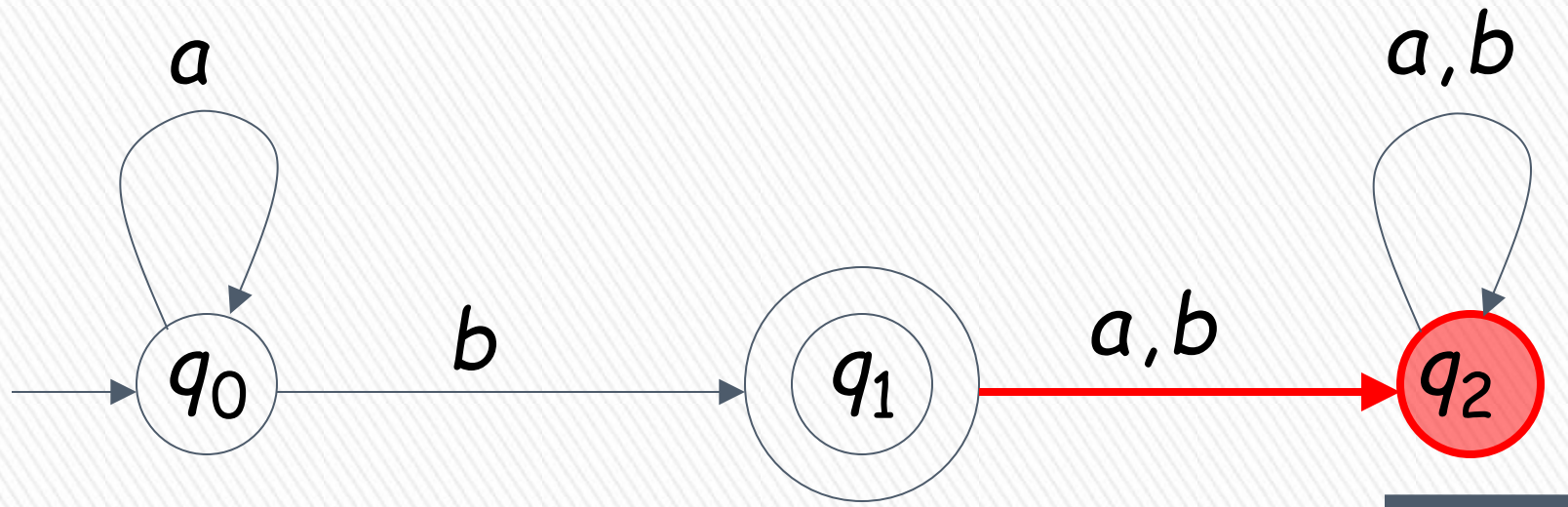


A rejection case

BLM2502 Theory of Computation

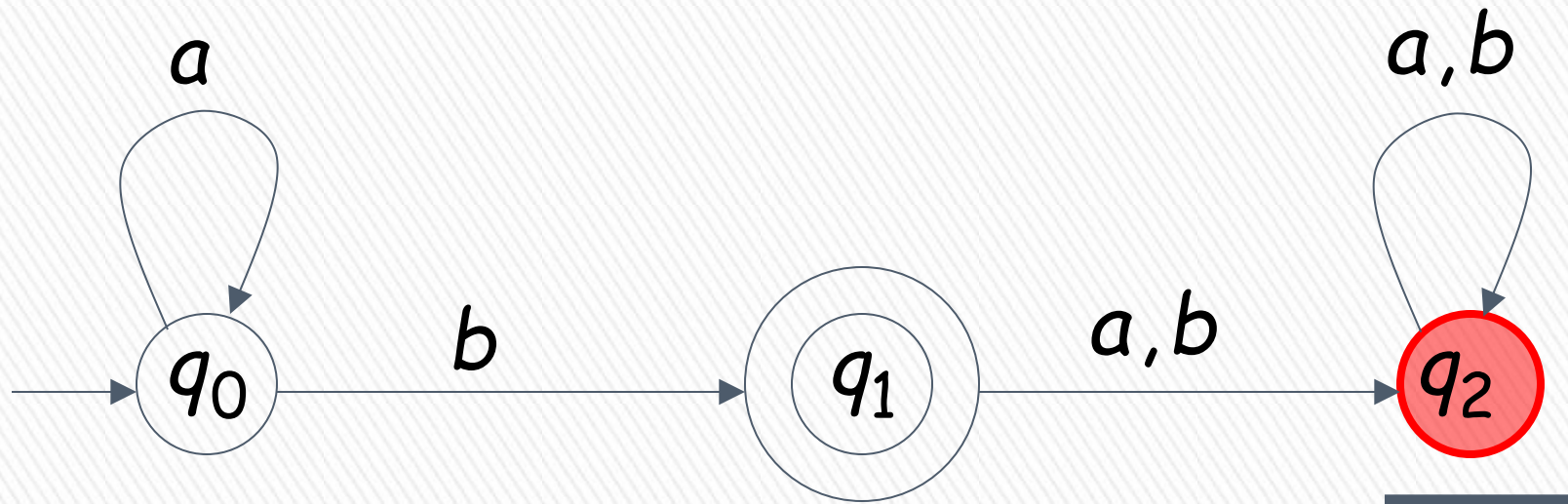


BLM2502 Theory of Computation



BLM2502 Theory of Computation

↓ Input finished

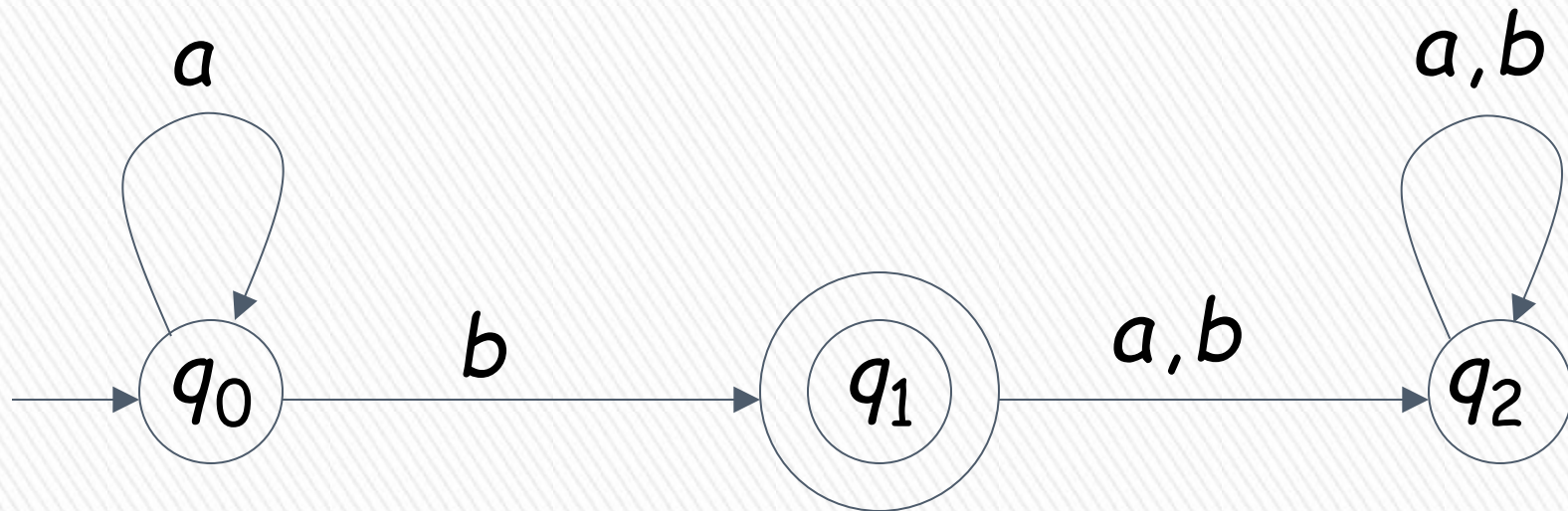


reject

BLM2502 Theory of Computation

Language Accepted:

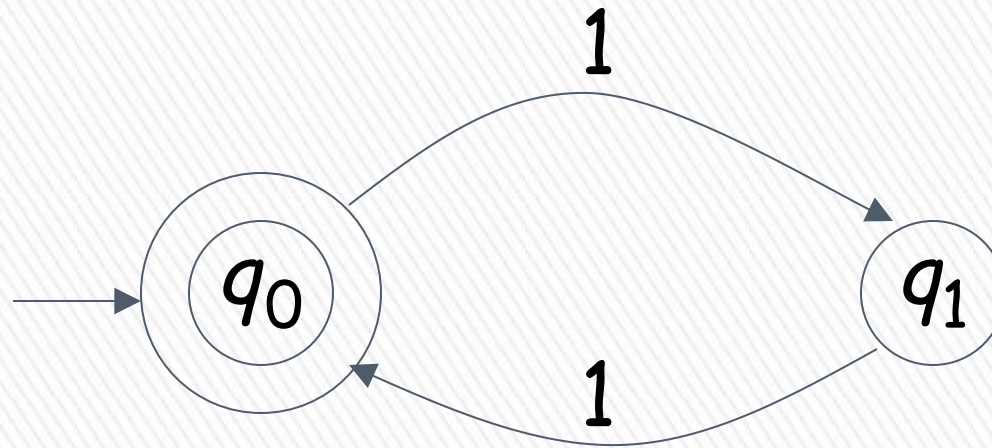
$$L = \{a^n b : n \geq 0\}$$



BLM2502 Theory of Computation

$$L = \{1^n : n = 2k, k \text{ is integer}\}$$

Alphabet: $\Sigma = \{1\}$



Language Accepted:

$$\begin{aligned} \text{EVEN} &= \{x: x \text{ is in } \Sigma^* \text{ and } x \text{ is even}\} \\ &= \{\epsilon, 11, 1111, 111111, \dots\} \end{aligned}$$

BLM2502 Theory of Computation

» Extended Transition Function

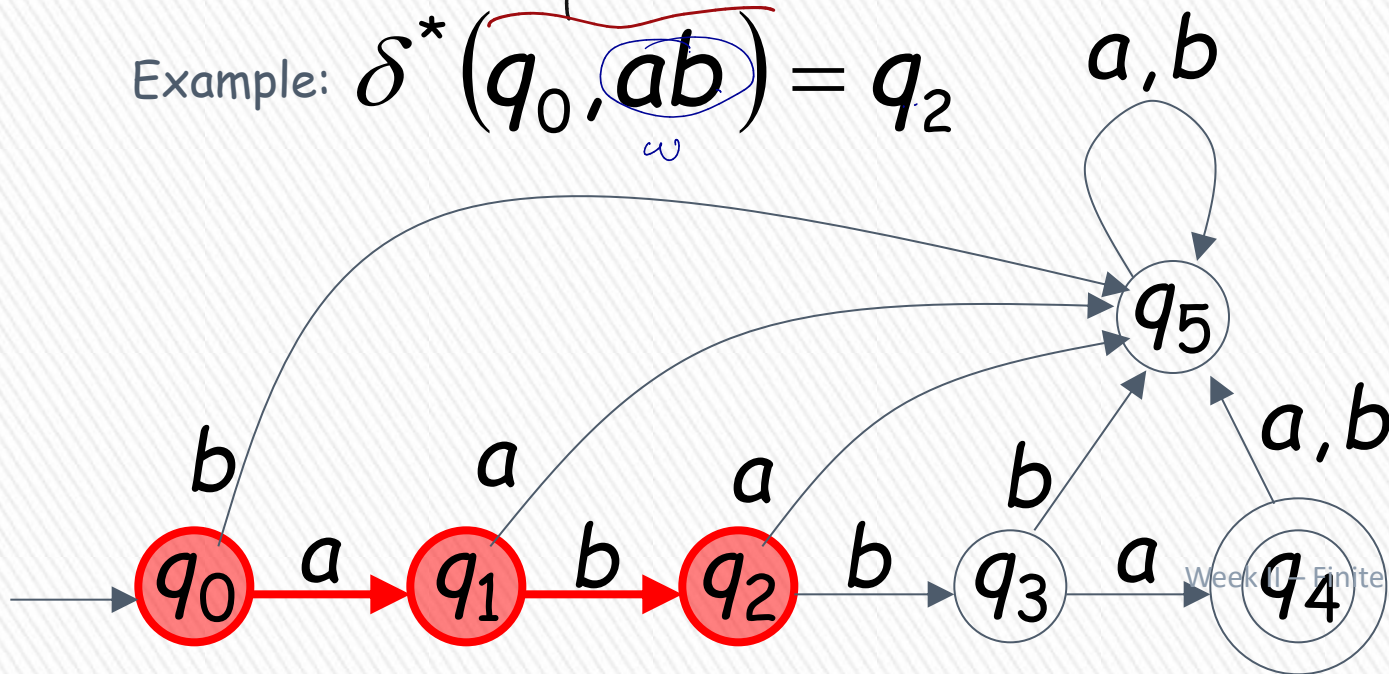
$$\delta^* : Q \times \Sigma^* \rightarrow Q$$

$$\delta(q, w) \rightarrow q'$$

> Describes the resulting state after scanning string w from state q

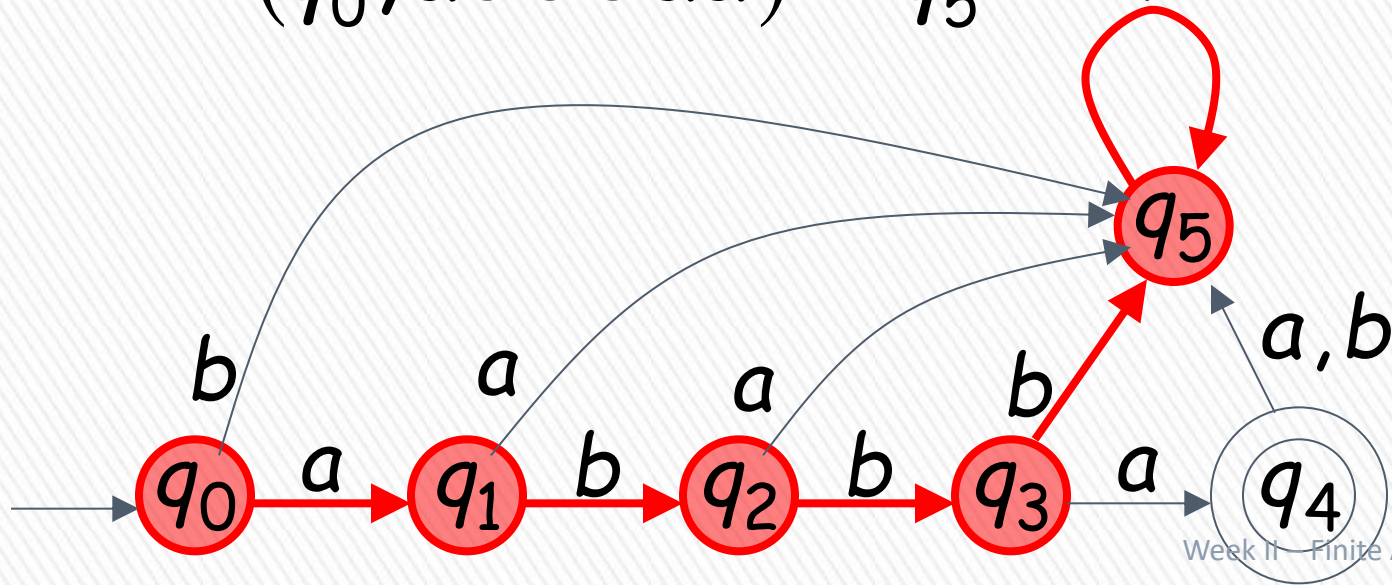
$$\begin{aligned} \delta(q_0, a) &= q_1 \\ \delta(q_0, ab) &= q_2 \end{aligned}$$

Example: $\delta^*(q_0, ab) = q_2$



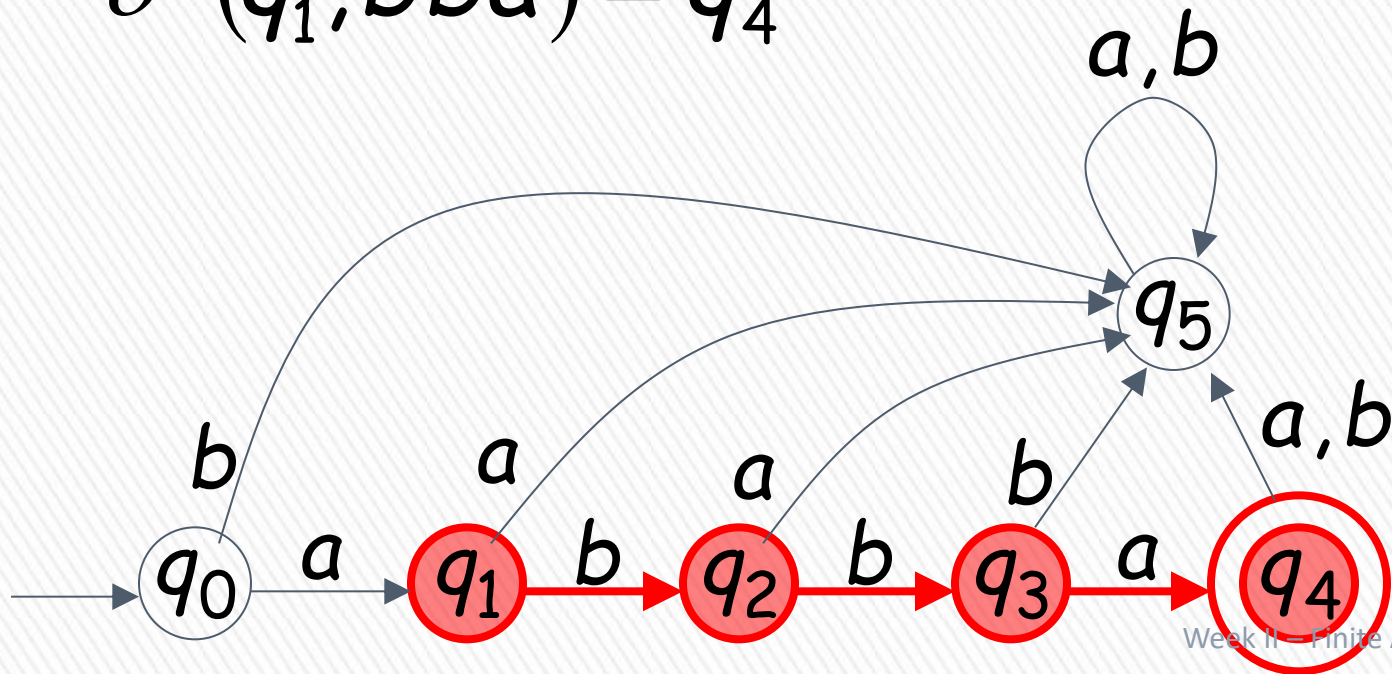
BLM2502 Theory of Computation

$$\delta^*(q_0, abbbbaa) = q_5$$



BLM2502 Theory of Computation

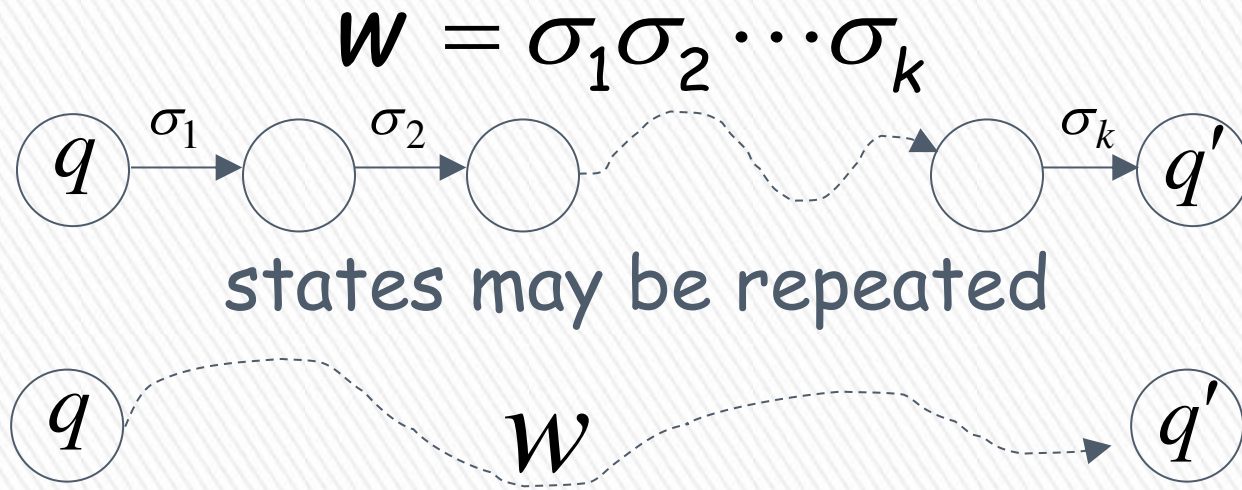
$$\delta^*(q_1, bba) = q_4$$



BLM2502 Theory of Computation

In general:

$\delta^*(q, w) \rightarrow q'$ implies that there is a walk of transitions



BLM2502 Theory of Computation

Language Accepted By DFA

The **language** accepted by an automaton M , is denoted as $L(M)$ and contains all the strings accepted by M

We say that a language L' is accepted (or recognized) by the DFA M if

$$L(M) = L'$$

An automaton accepts one and only one language.
A language can be accepted by a number of automata

BLM2502 Theory of Computation

» For a DFA $\mathbf{M} = (Q, \Sigma, \delta, q_0, F)$

» Language accepted by $\mathbf{M}$:

$$L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \in F\}$$



BLM2502 Theory of Computation

» Language rejected by **M** :

$$\overline{L(M)} = \{w \in \Sigma^* : \delta^*(q_0, w) \notin F\}$$



BLM2502 Theory of Computation



BLM2502 Theory of Computation

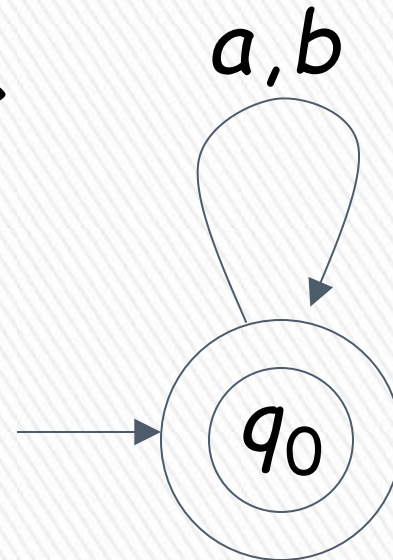
More DFA Examples



$$L(M) = \{ \}$$

Empty language

$$\Sigma = \{a, b\}$$

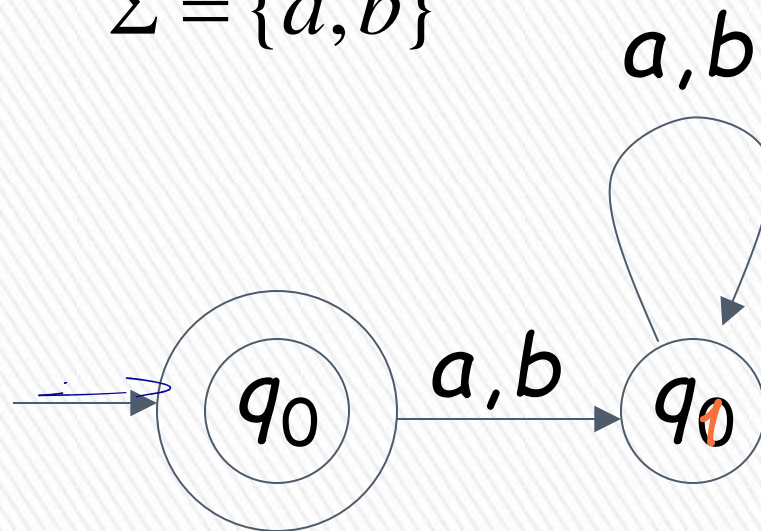


$$L(M) = \Sigma^*$$

All strings

BLM2502 Theory of Computation

$$\Sigma = \{a, b\}$$



$$L(M) = \{\lambda\}$$

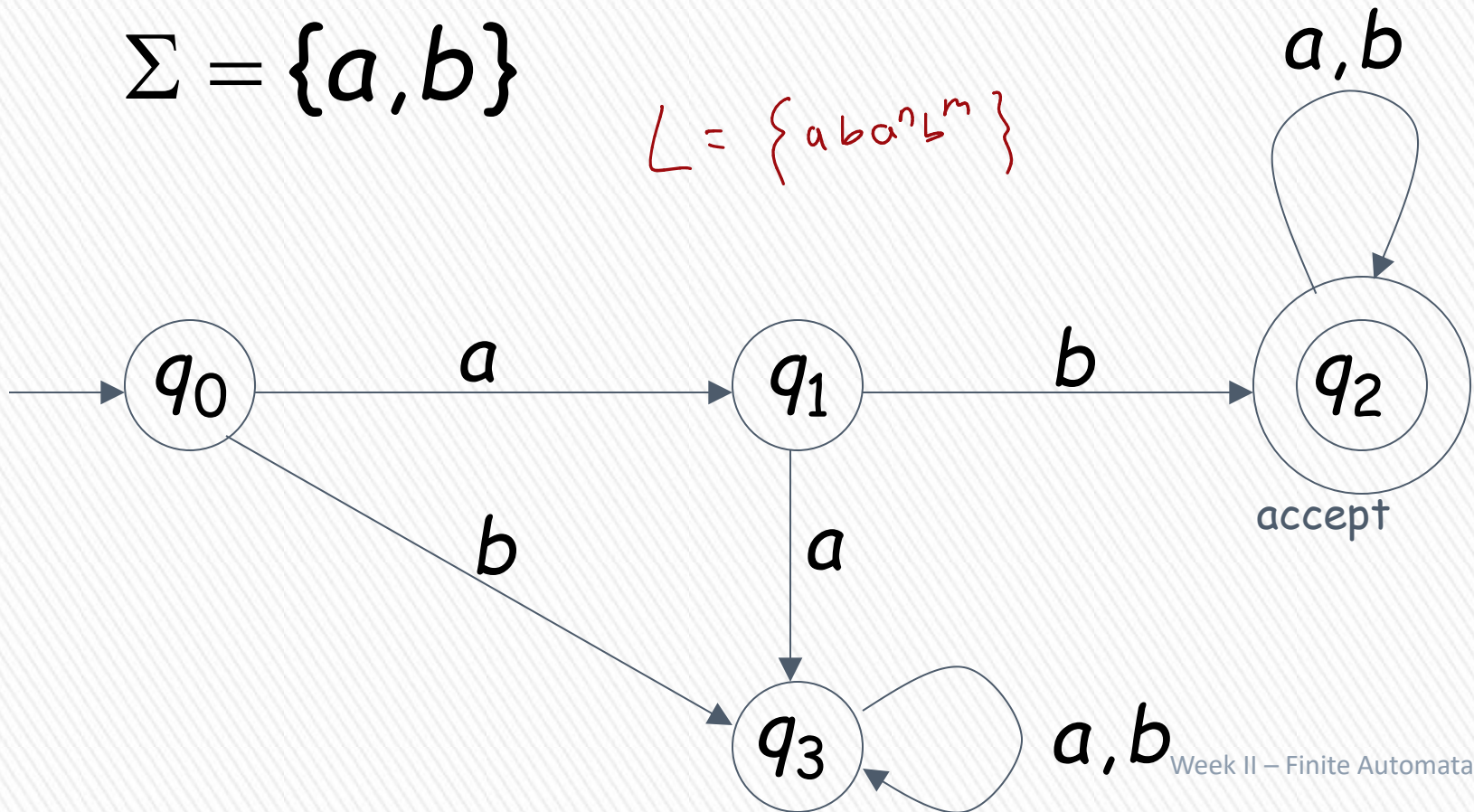
Language of the empty string

BLM2502 Theory of Computation

$L(M) = \{ \text{all strings with prefix } ab \}$

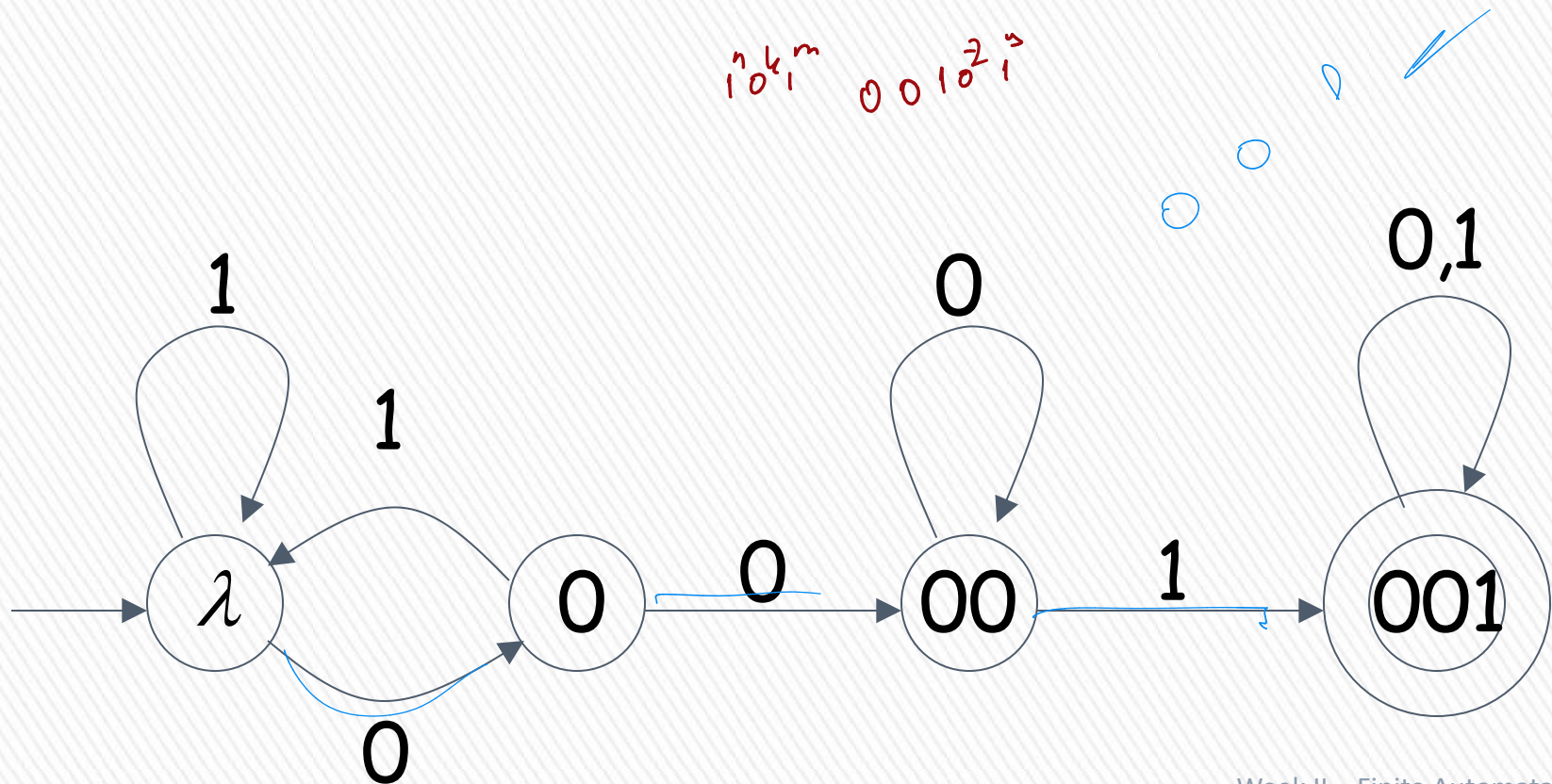
$\Sigma = \{a, b\}$

$L = \{aba^n b^m\}$



BLM2502 Theory of Computation

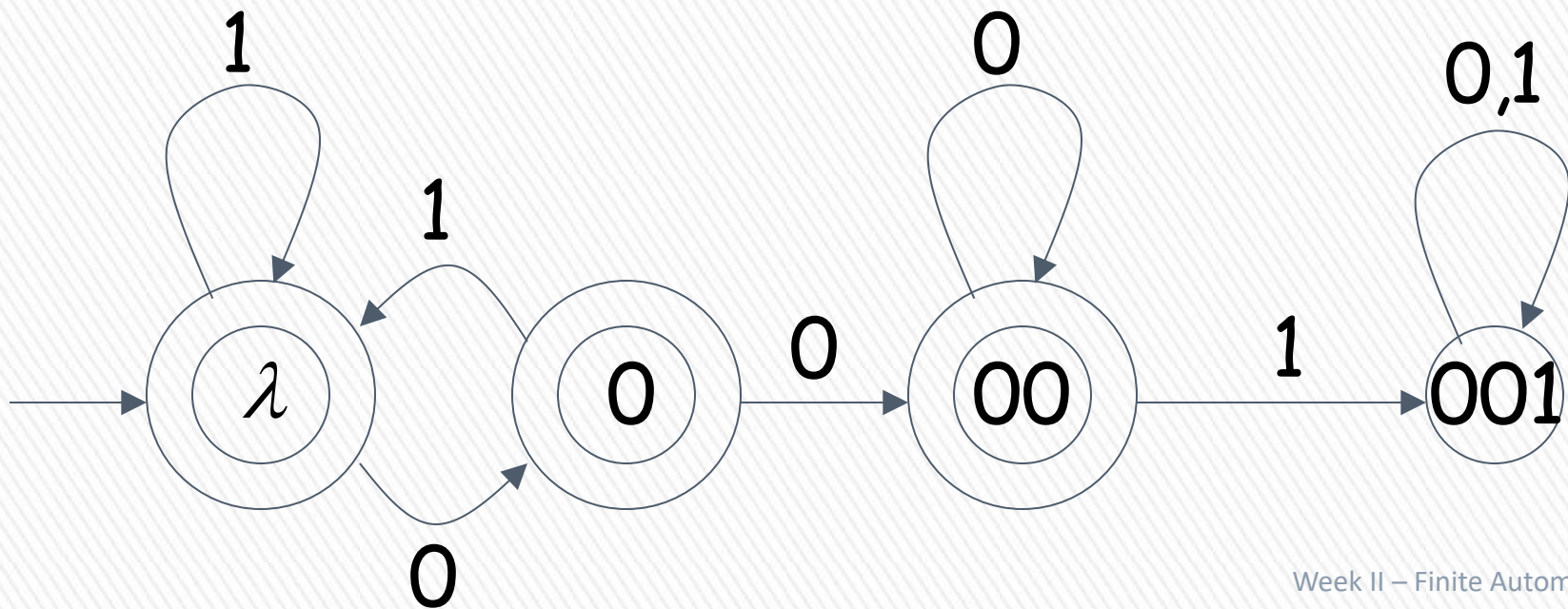
$L(M) = \{ \text{all binary strings containing substring } 001 \}$



BLM2502 Theory of Computation

$L(M) = \{ \text{all binary strings without substring } 001 \}$

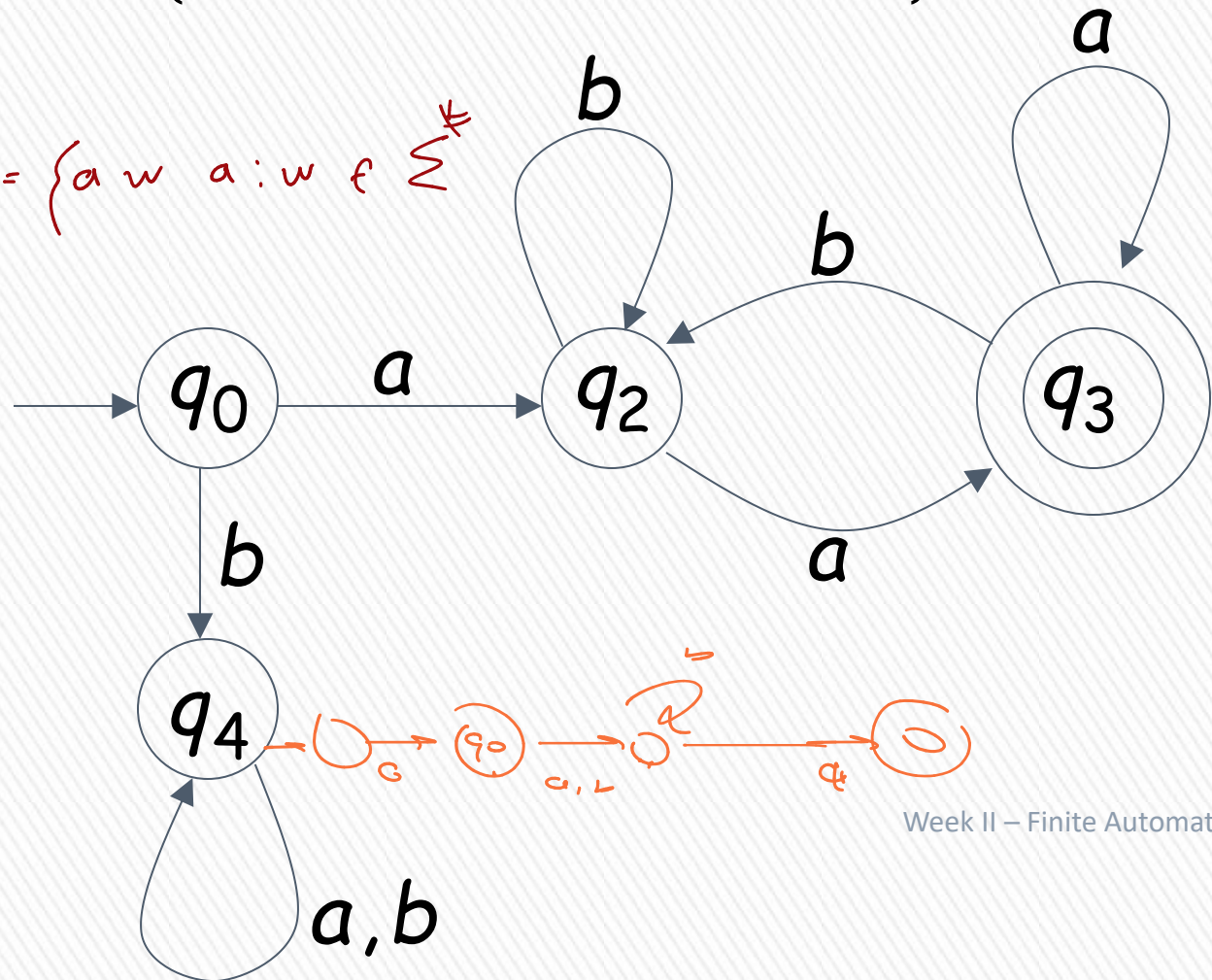
all binary strings without substring
001



BLM2502 Theory of Computation

$$L(M) = \{awa : w \in \{a,b\}^*\}$$

$$L(M) = \{aw a : w \in \Sigma^*\}$$



BLM2502 Theory of Computation

$\{abba\}$ $\{\lambda, ab, abba\}$

$\{a^n b : n \geq 0\}$ $\{awa : w \in \{a, b\}^*\}$

$\{x : x \in \{1\}^* \text{ and } x \text{ is even}\}$

$\{\}$ $\{\lambda\}$ $\{a, b\}^*$

$\{\text{all binary strings without substring } 001 \}$

$\{\text{all strings in } \{a, b\}^* \text{ with prefix } ab \}$

There exist automata that accept these languages (see previous slides).

BLM2502 Theory of Computation

There exist languages which are not Regular:

$$L = \{a^n b^n : n \geq 0\}$$

$$ADDITION = \{x + y = z : x = 1^n, y = 1^m, \\ z = 1^k, n + m = k\}$$

There is no DFA that accepts these languages



BLM2502

Theory of Computation

BLM2502 Theory of Computation

» Course Outline

Week	Content
1.	Introduction to Course
2.	Computability Theory, Complexity Theory, Automata Theory, Set Theory, Relations, Proofs, Pigeonhole Principle
3.	Regular Expressions
4.	Finite Automata
5.	Deterministic and Nondeterministic Finite Automata
6.	Epsilon Transition, Equivalence of Automata
7.	Pumping Theorem
8.	Context Free Grammars
9.	Parse Tree, Ambiguity,
10.	Pumping Theorem
11.	Turing Machines, Recognition and Computation, Church-Turing Hypothesis
12.	Turing Machines, Recognition and Computation, Church-Turing Hypothesis
13.	Review

NFA

Non-Deterministic

Finite Automata

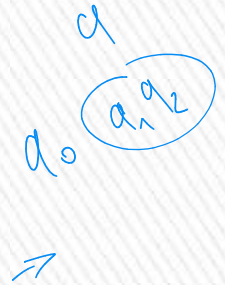


Deterministic Same inputs always,
Same outputs

Formal Definition of NFA

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q : Set of states, i.e. $\{q_0, q_1, q_2\}$



Σ : Input alphabet, i.e. $\{a, b\}$ $\epsilon \notin \Sigma$

δ : Transition function $Q \times \Sigma \rightarrow 2^Q$

q_0 : Initial state

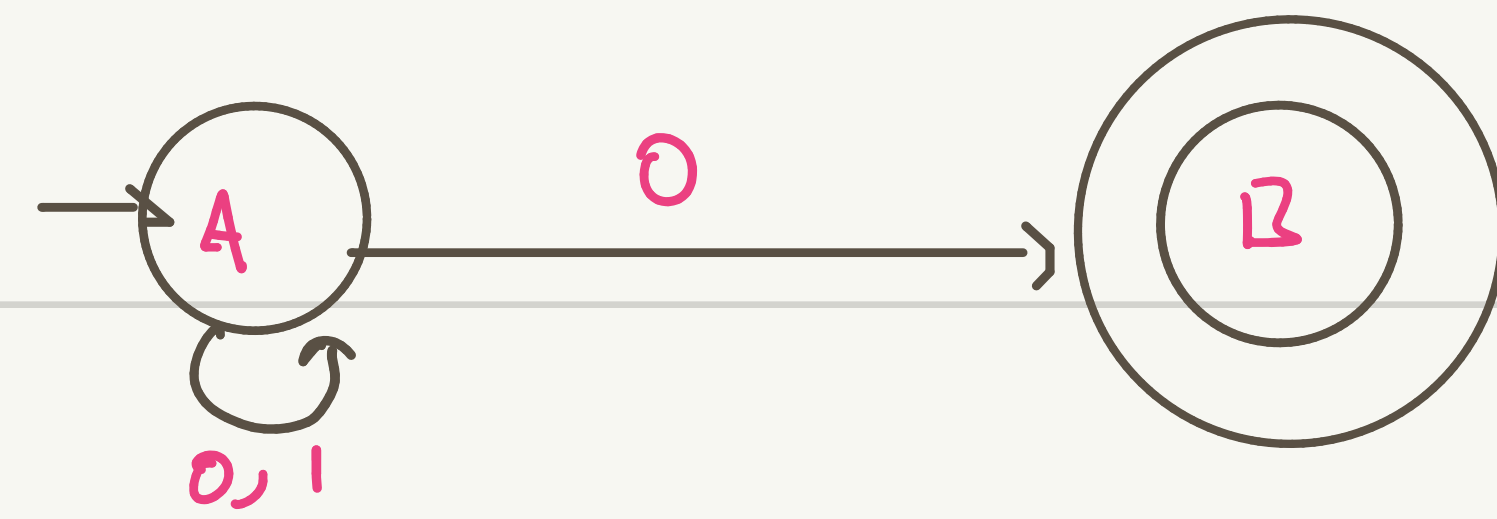
F : Accepting states

A

A	B	A	B
AB	ϵ	AB	ϵ
A	B	A	B
AB	ϵ	AB	ϵ

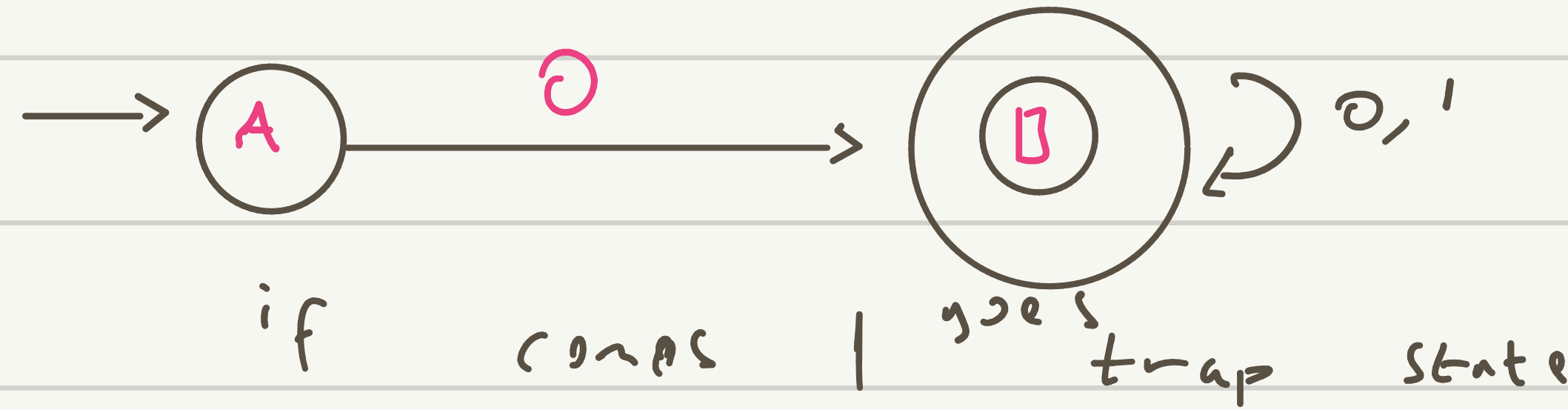
B





→ Non deterministic $L = \{w \mid w \text{ ends with } 0\}$

↳ Bu anas, olarak düşünürüz ve anasından herhang
biri L yi islerigorsa yazılabilir.

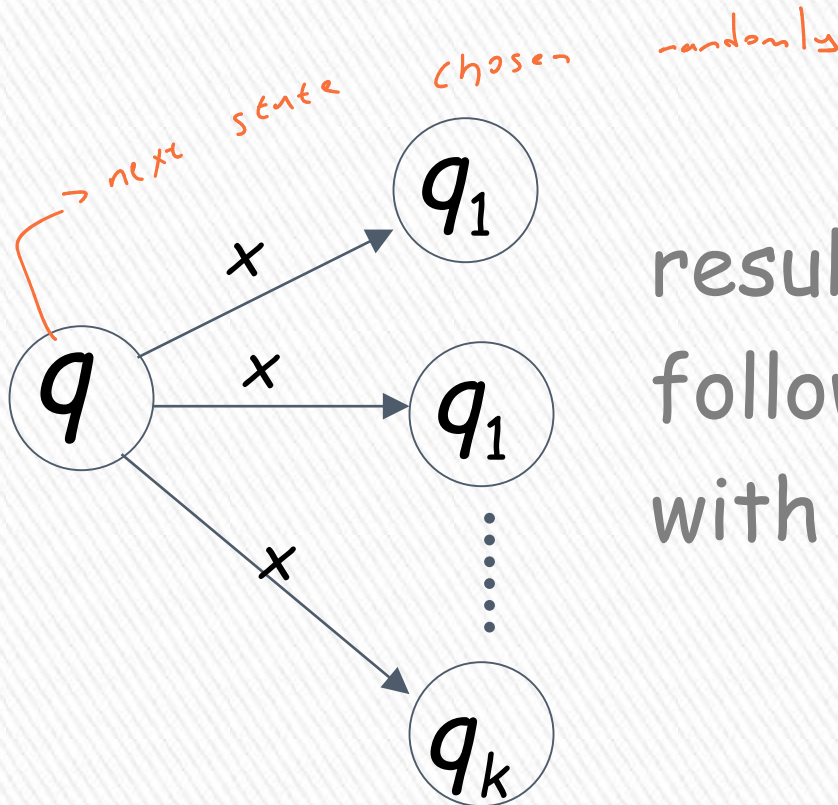


$L = \{w \mid w \text{ starts with } 0\}$

Transition Function δ

non-deterministic

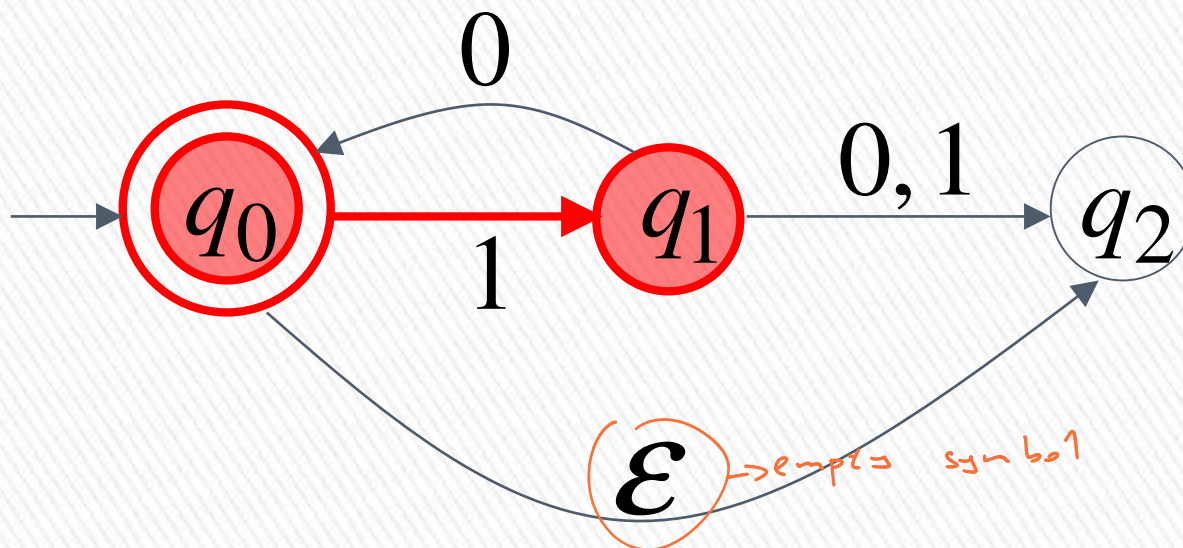
$$\delta(q, x) = \{q_1, q_2, \dots, q_k\}$$



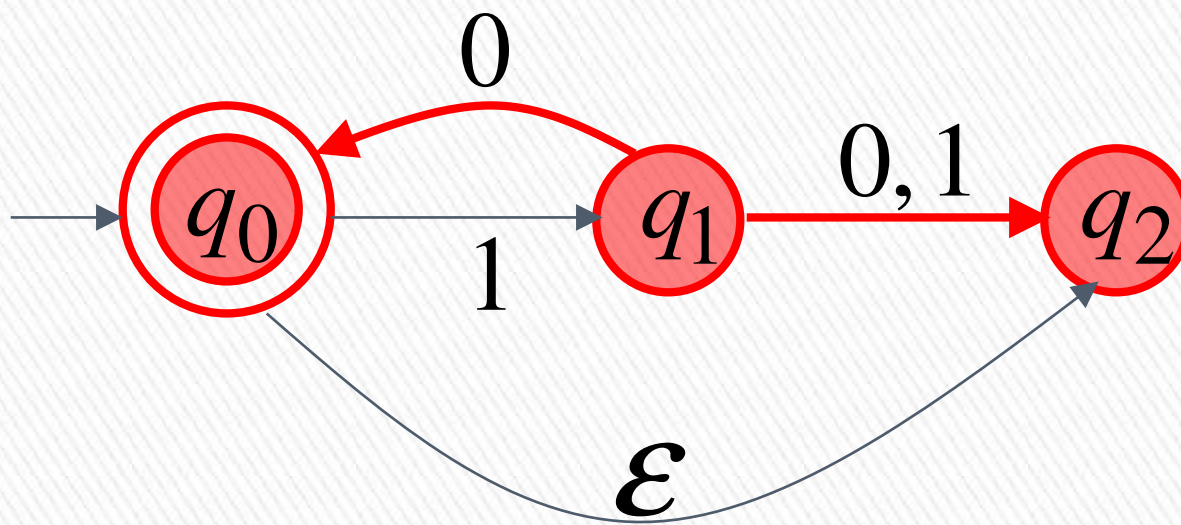
resulting states with
following **one** transition
with symbol x



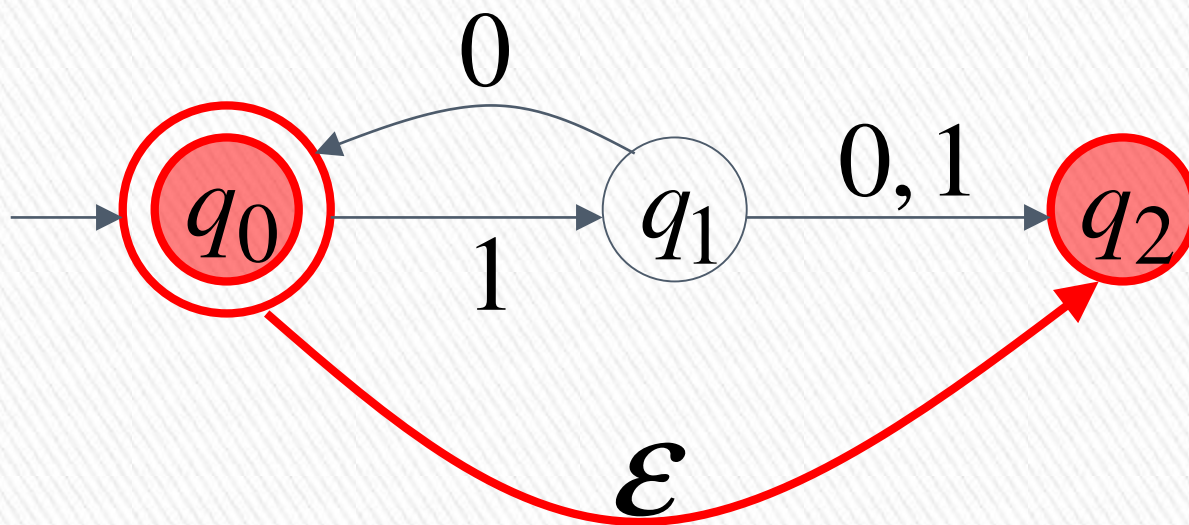
$$\delta(q_0, 1) = \{q_1\}$$



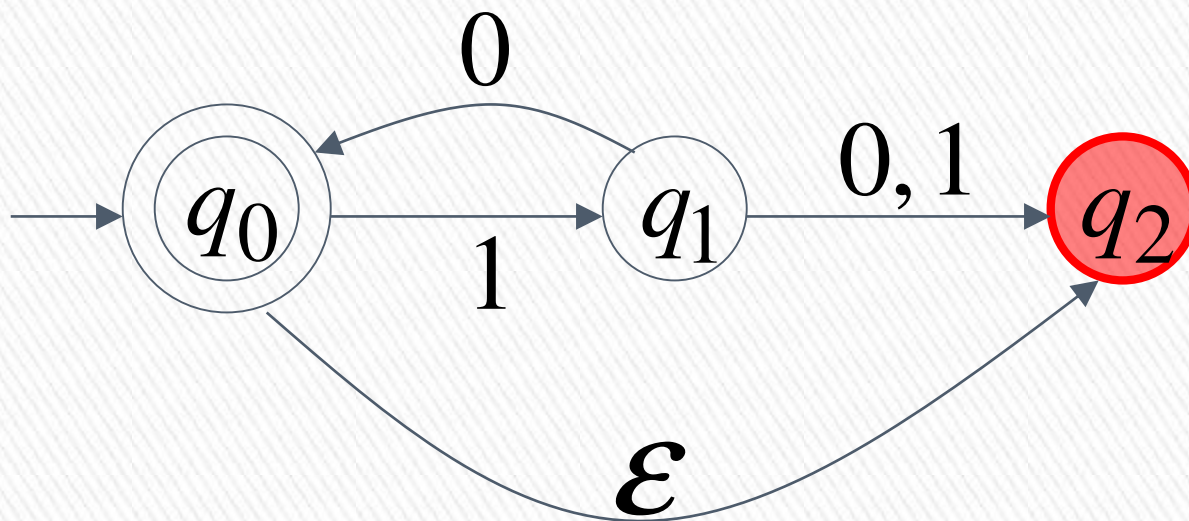
$$\delta(q_1, 0) = \{q_0, q_2\}$$



$$\delta(q_0, \varepsilon) = \{q_2\}$$

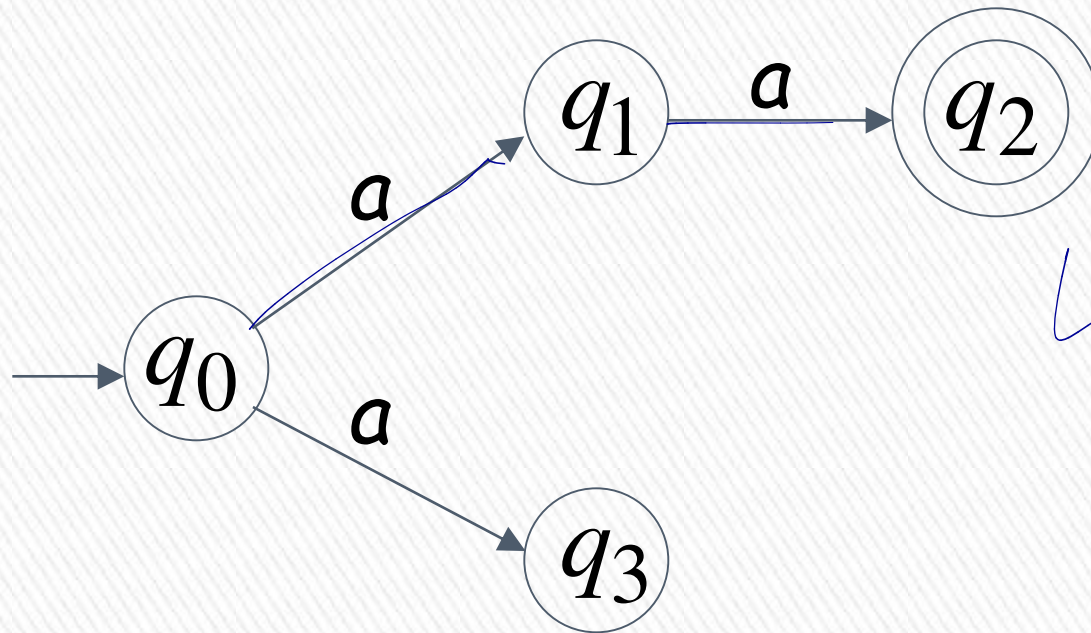


$$\delta(q_2, 1) = \emptyset$$



Nondeterministic Finite Automaton (NFA)

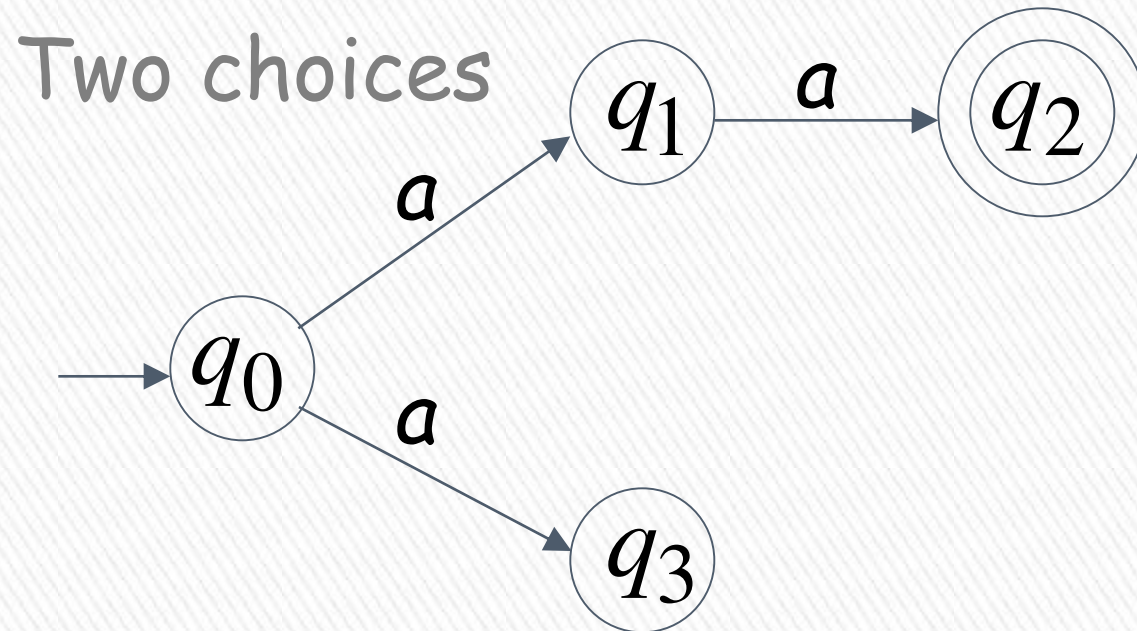
Alphabet = $\{a\}$



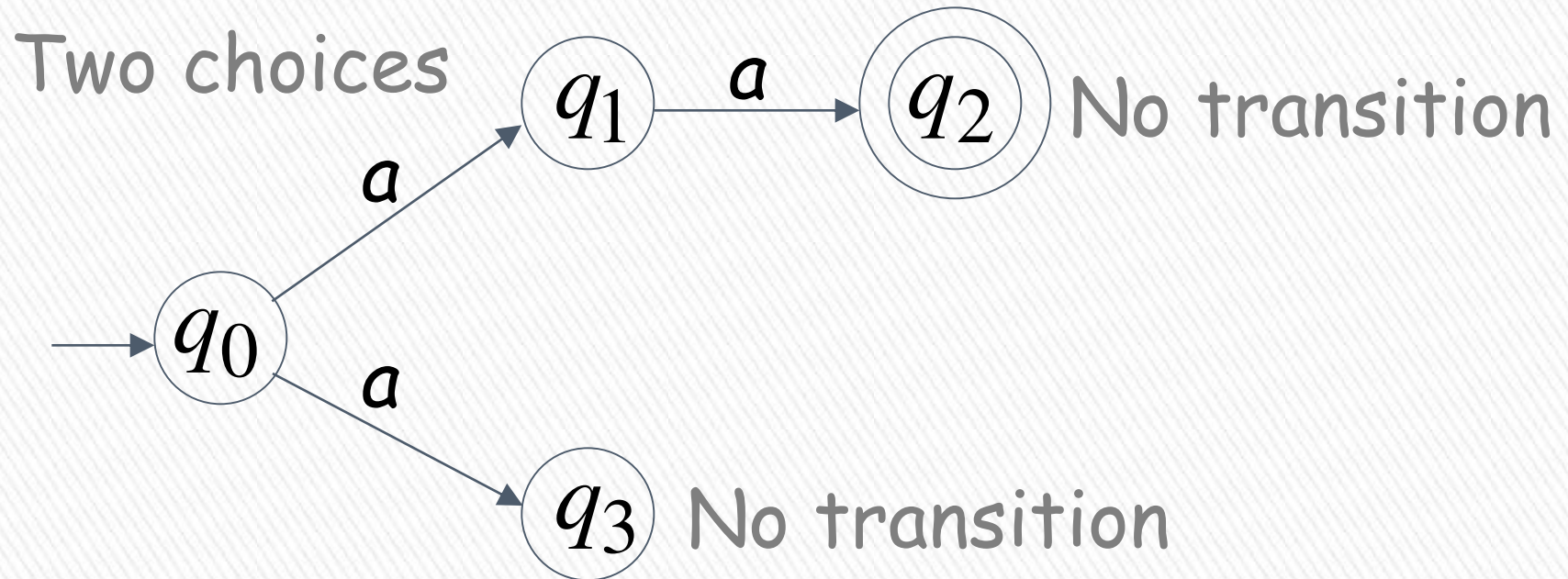
$L = \{a^n\}$



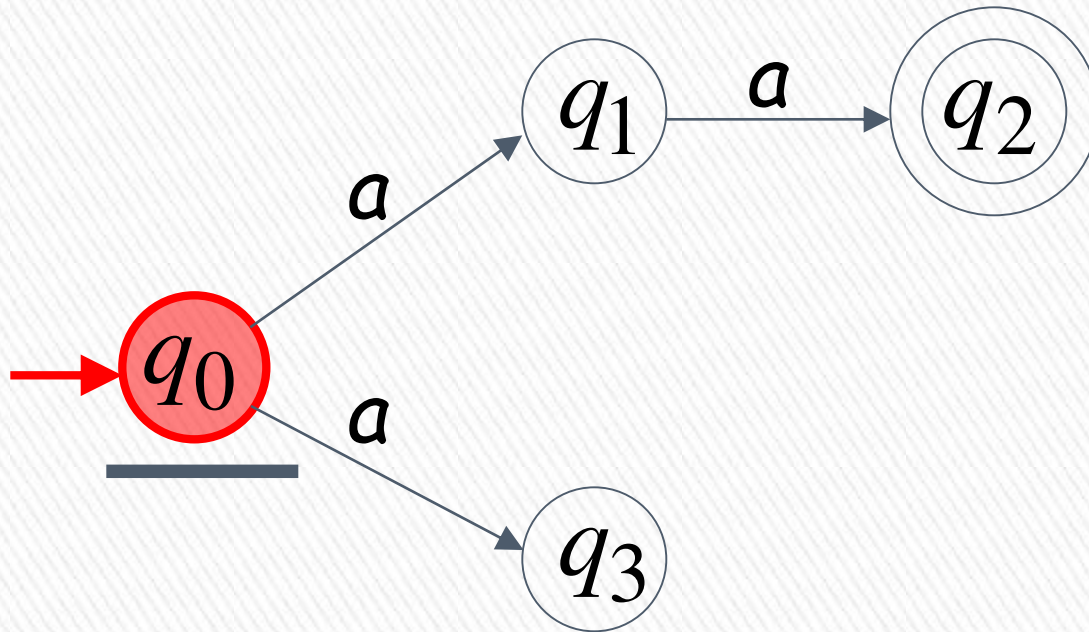
Alphabet = $\{a\}$



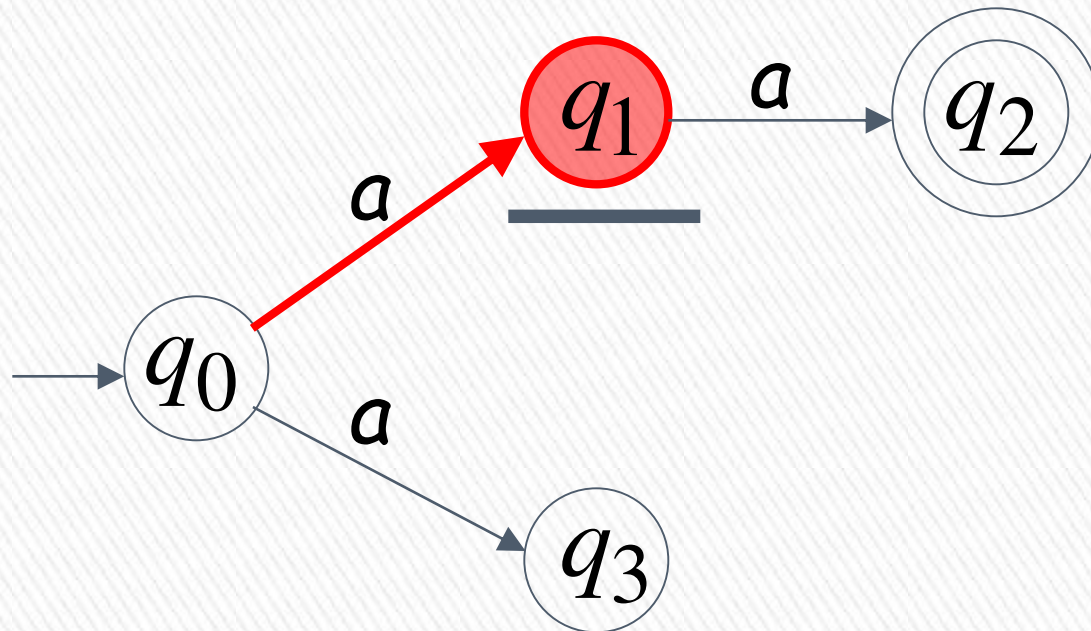
Alphabet = $\{a\}$



First Choice



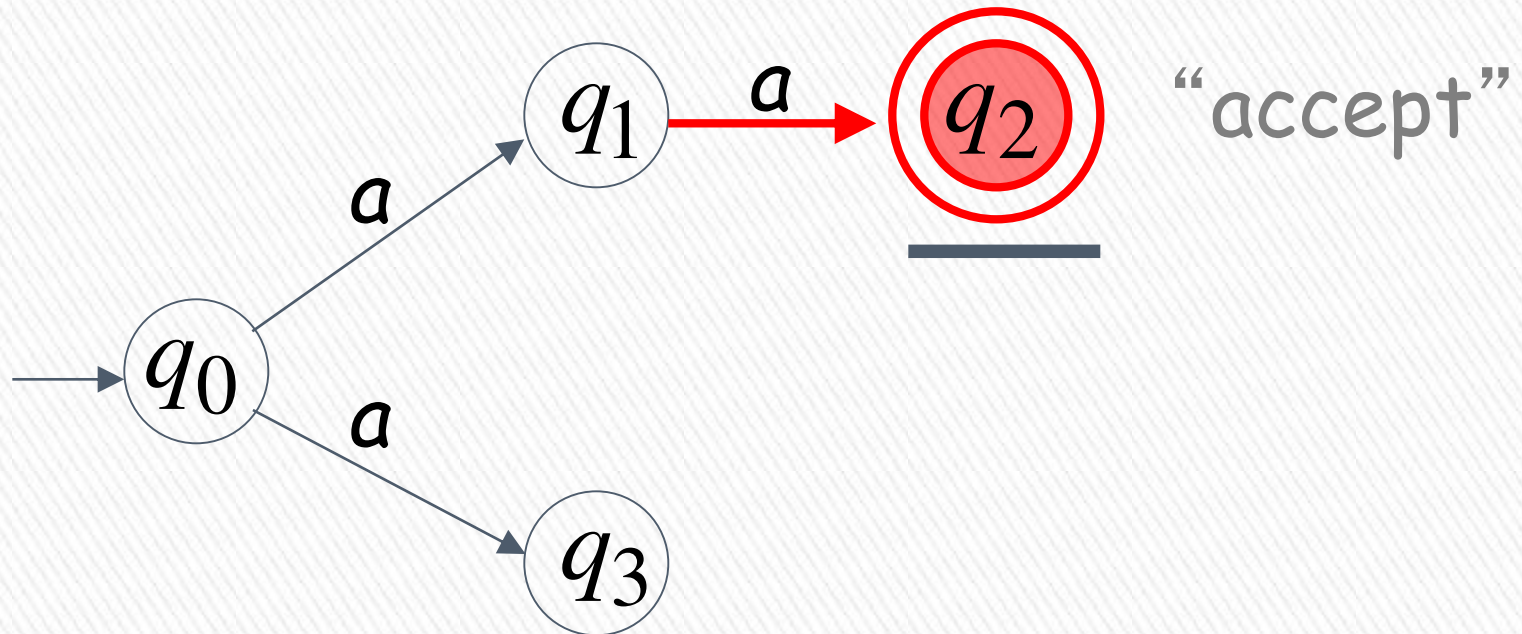
First Choice



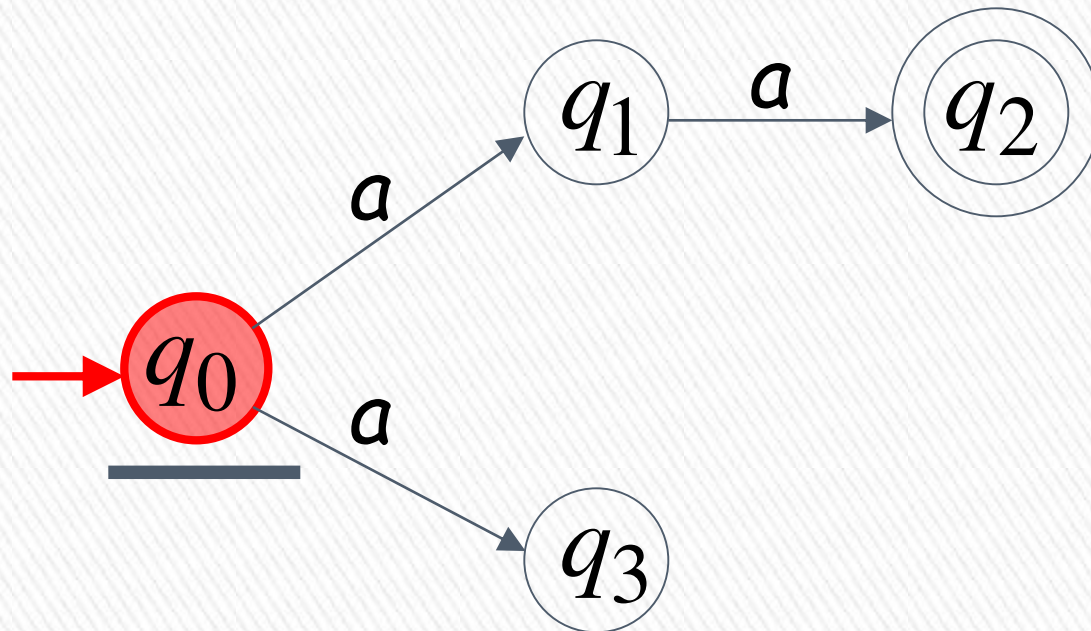
First Choice



All input is consumed



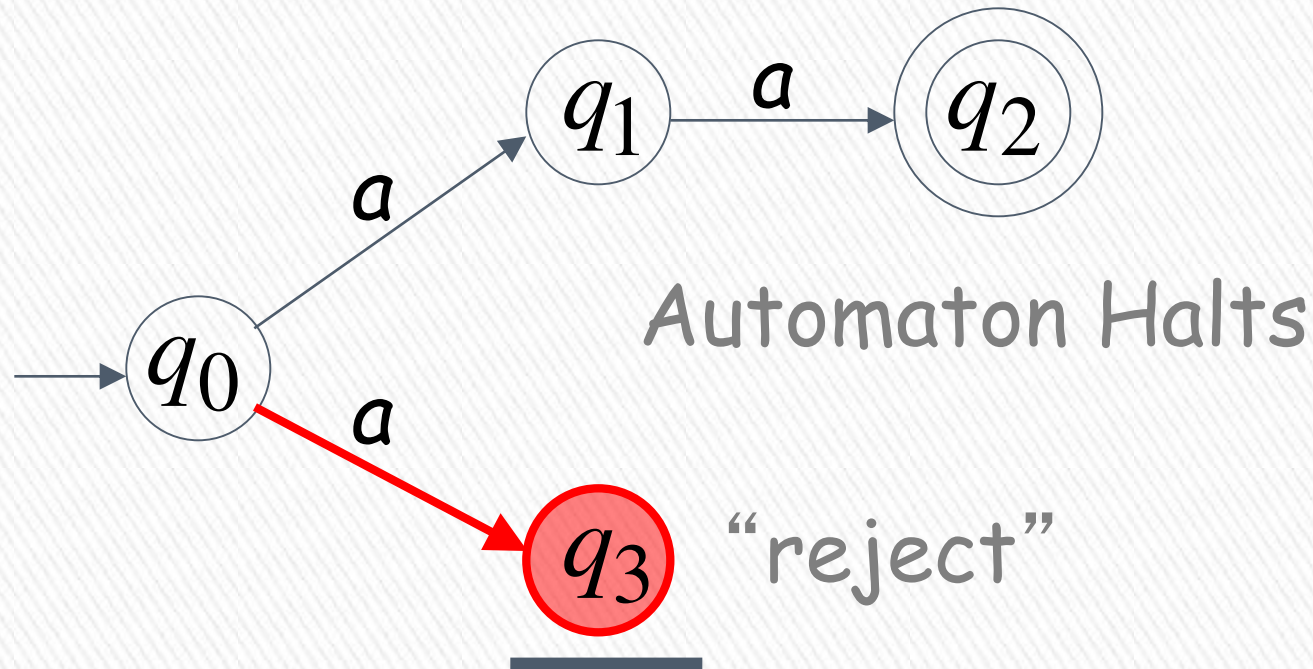
Second Choice



Second Choice



Input cannot be consumed



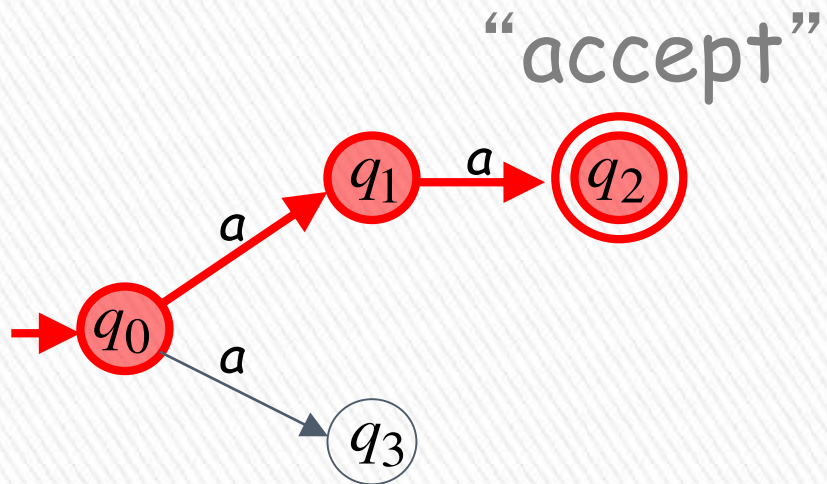
An NFA accepts a string:

if there exists a computation of the NFA that accepts the string

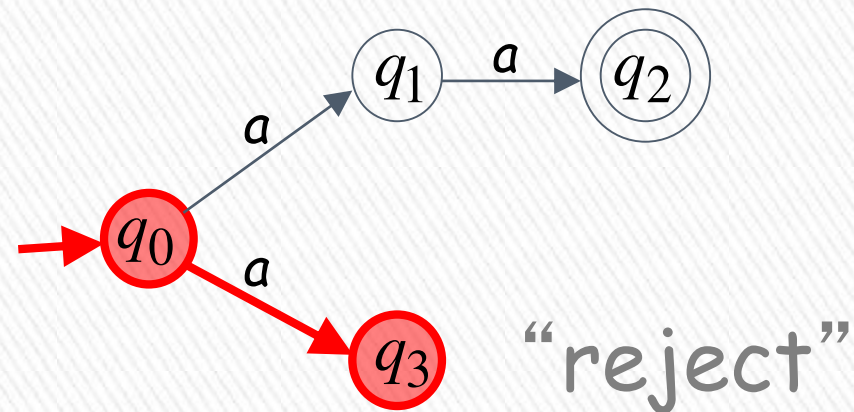
i.e., all the input string is processed and the automaton is in an accepting state



aa is accepted by the NFA:

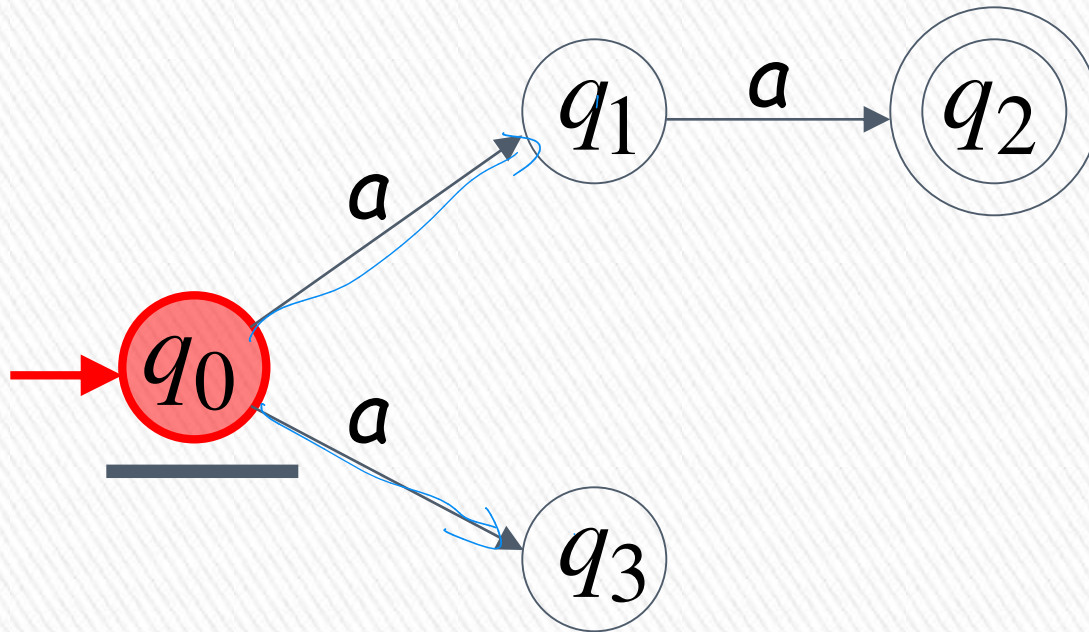
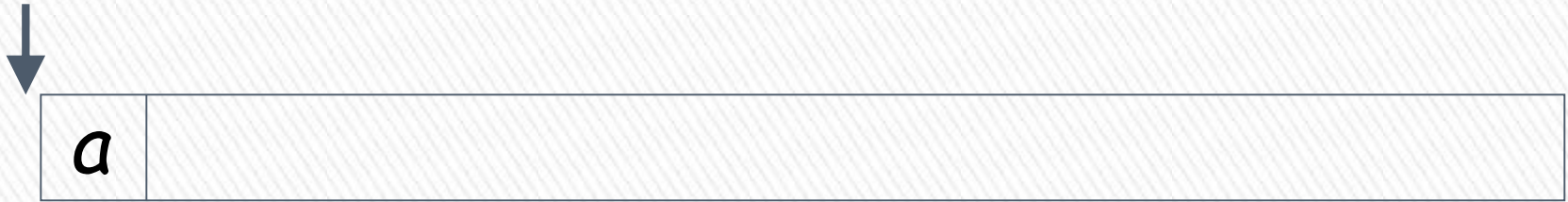


because this
computation
accepts *aa*



this computation
is ignored

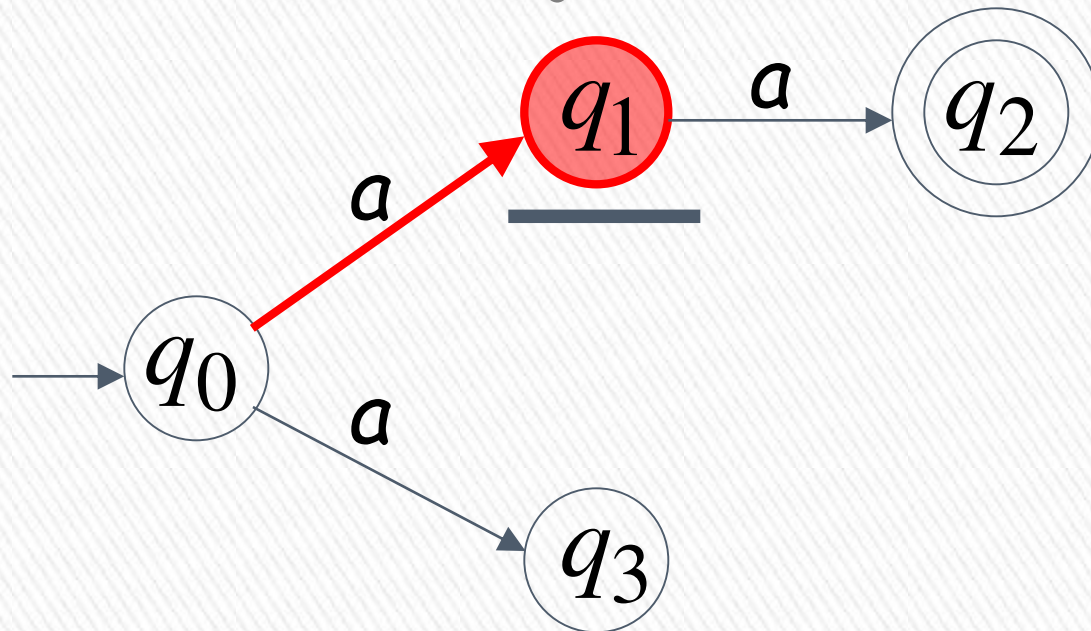
Rejection example



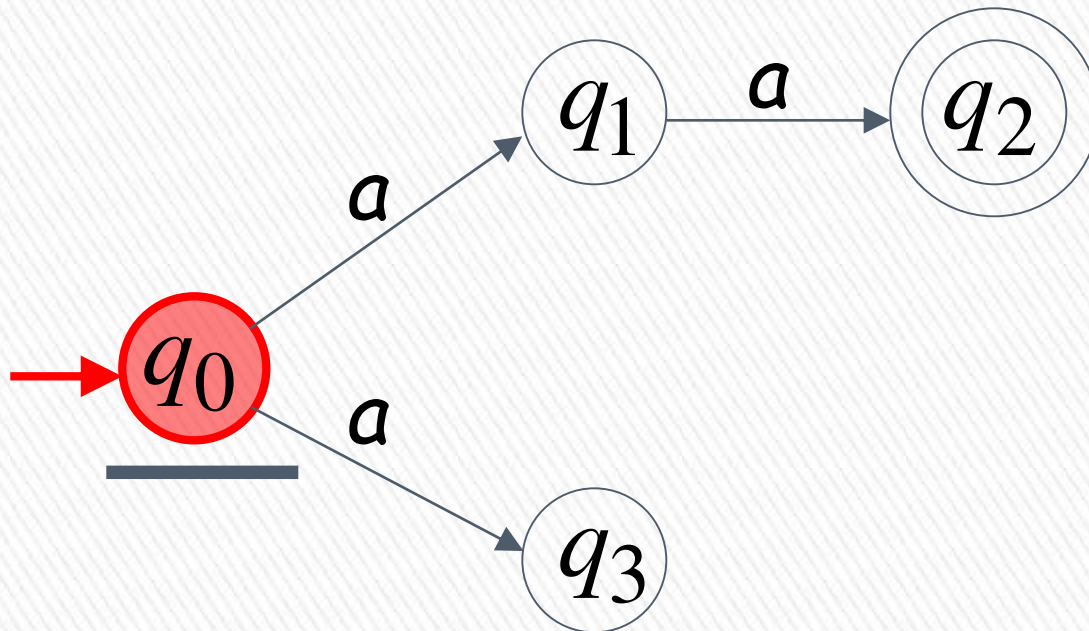
First Choice



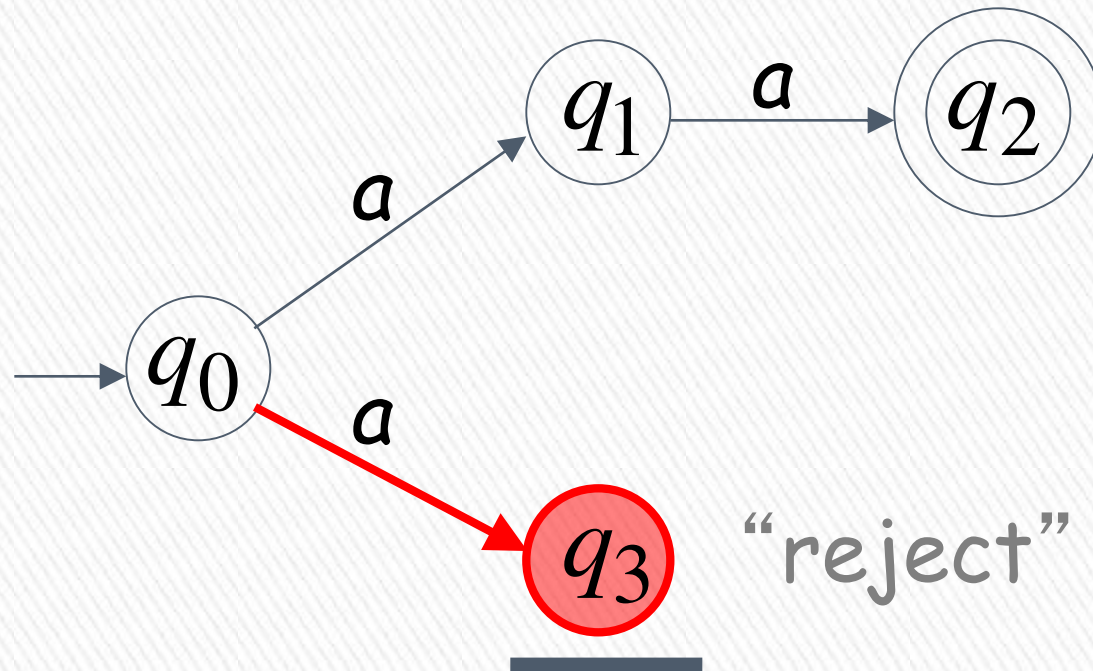
“reject”



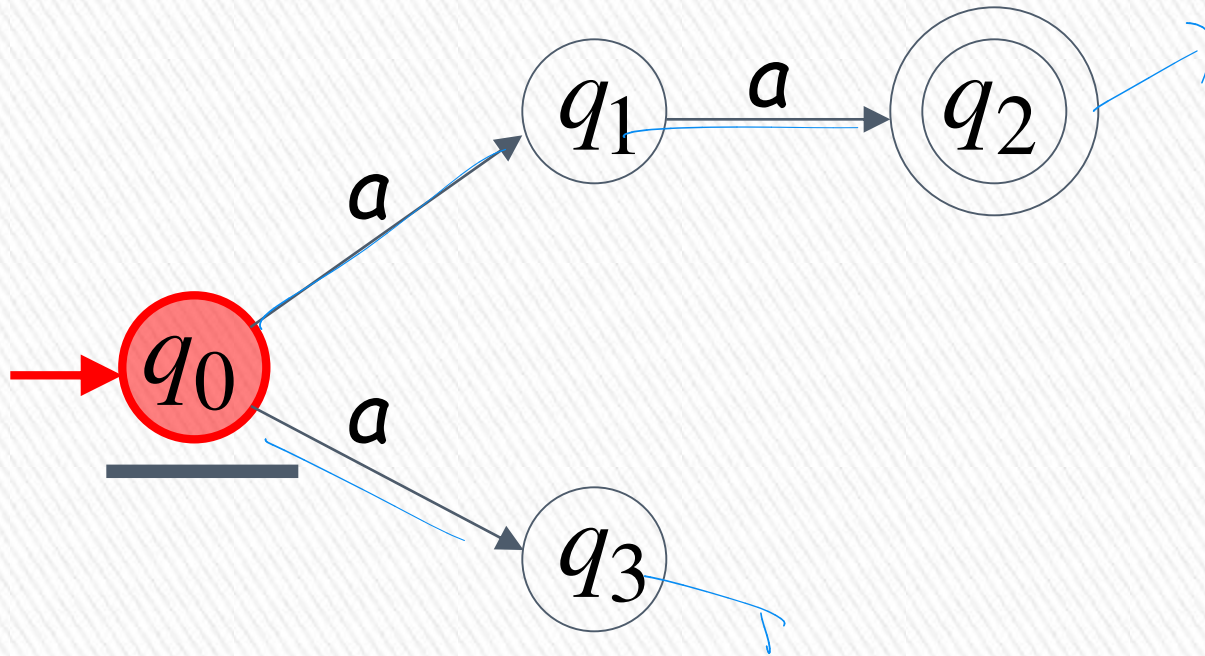
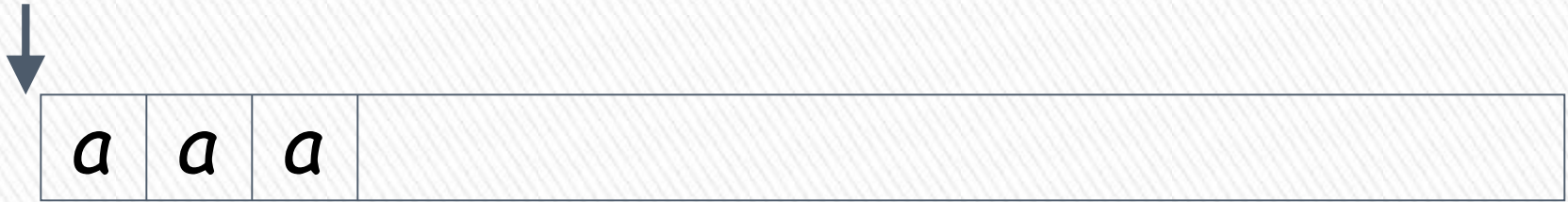
Second Choice



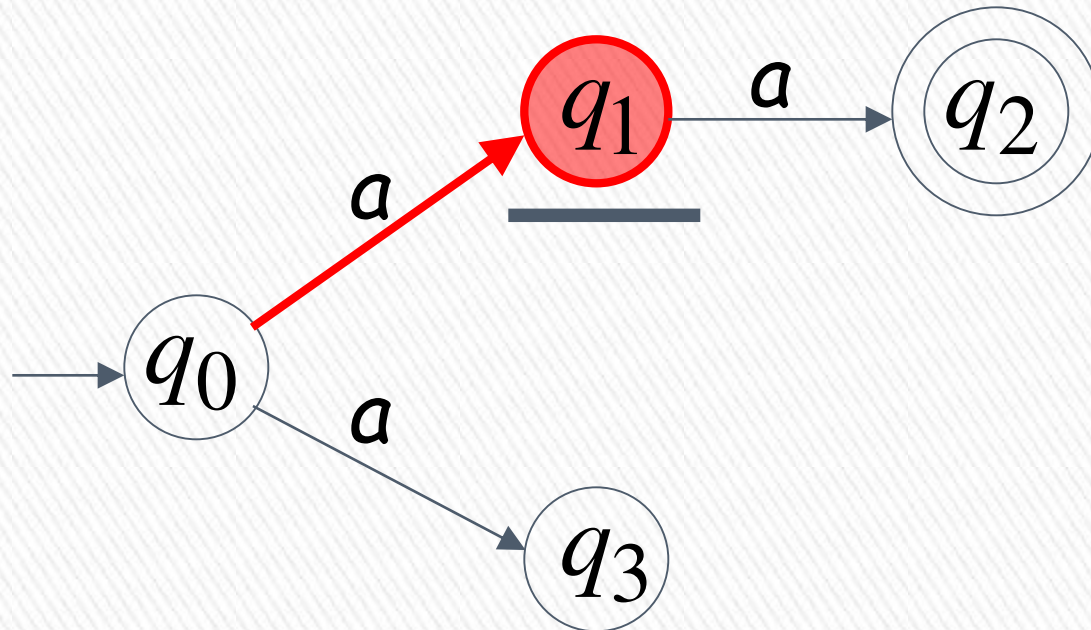
Second Choice



Another Rejection example



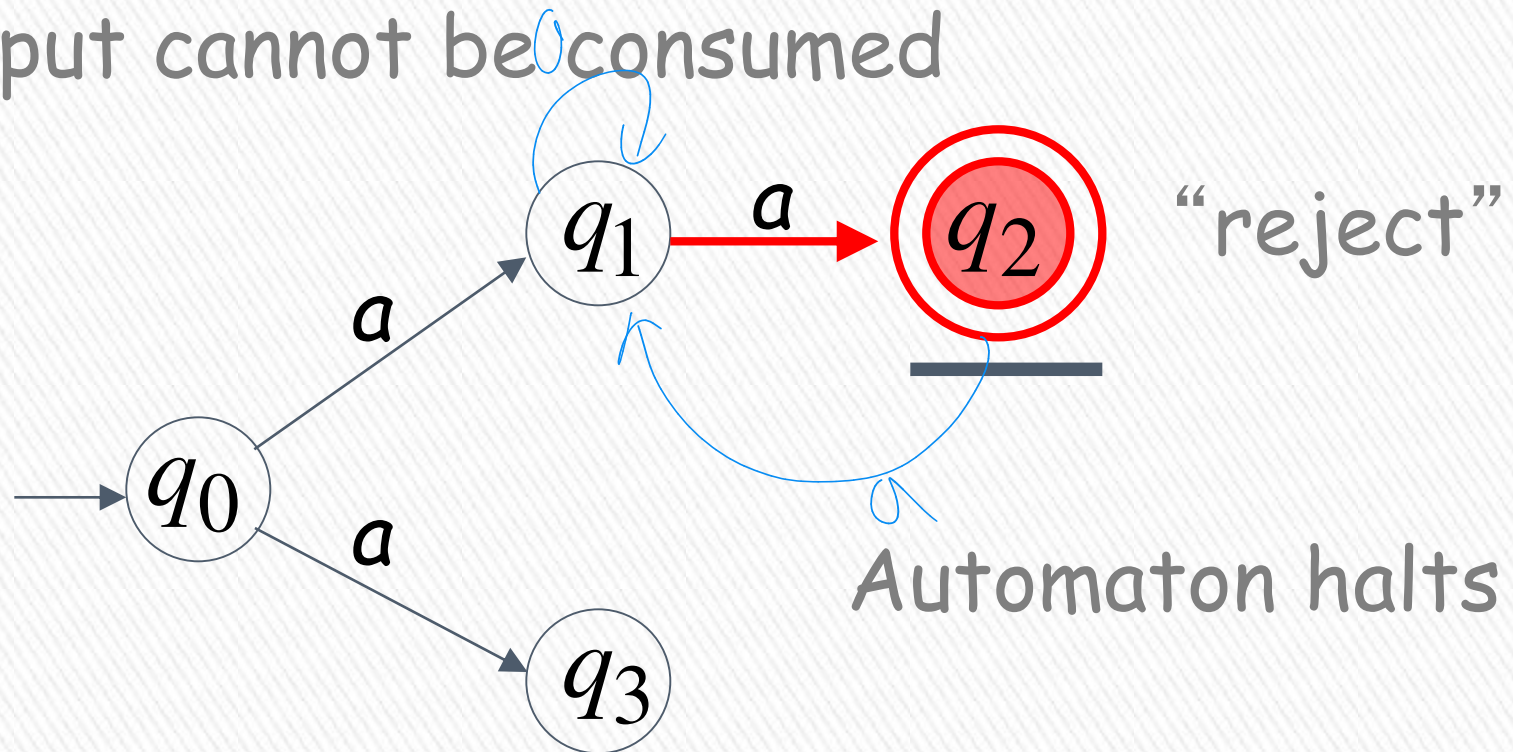
First Choice



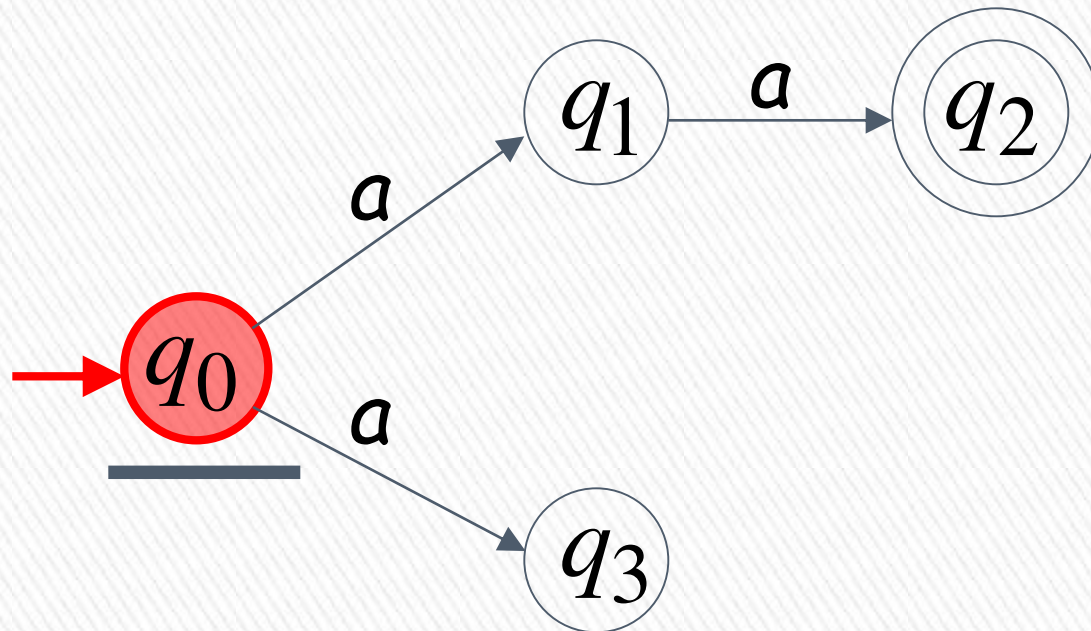
First Choice



Input cannot be consumed



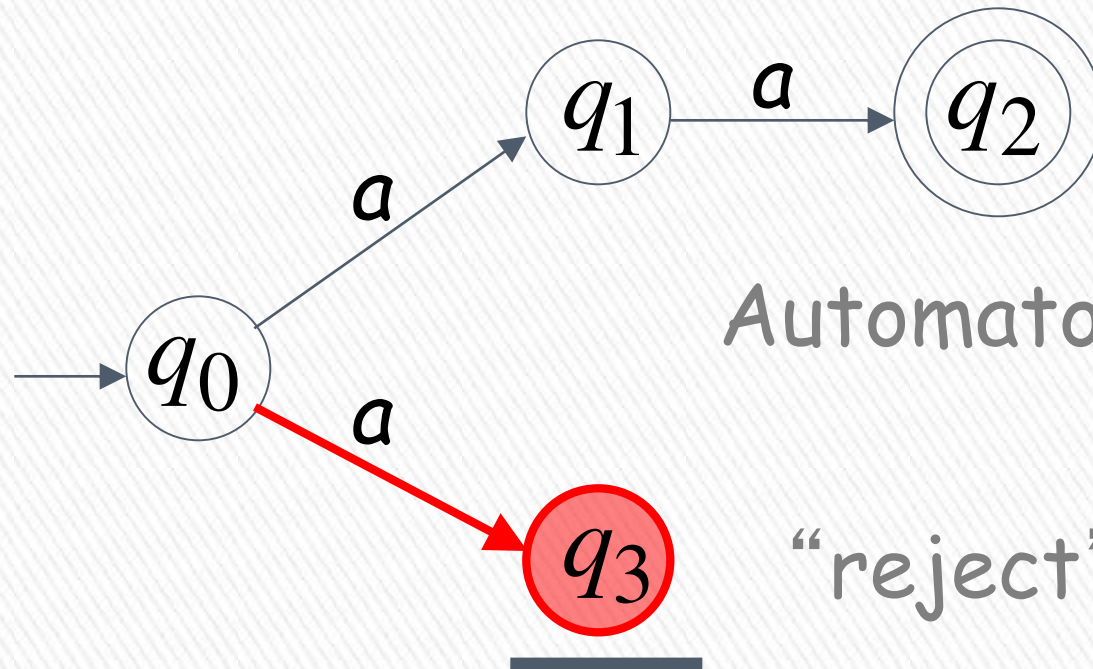
Second Choice



Second Choice



Input cannot be consumed



Automaton halts

“reject”



An NFA rejects a string:

if there is no computation of the NFA that accepts the string.

For each computation:

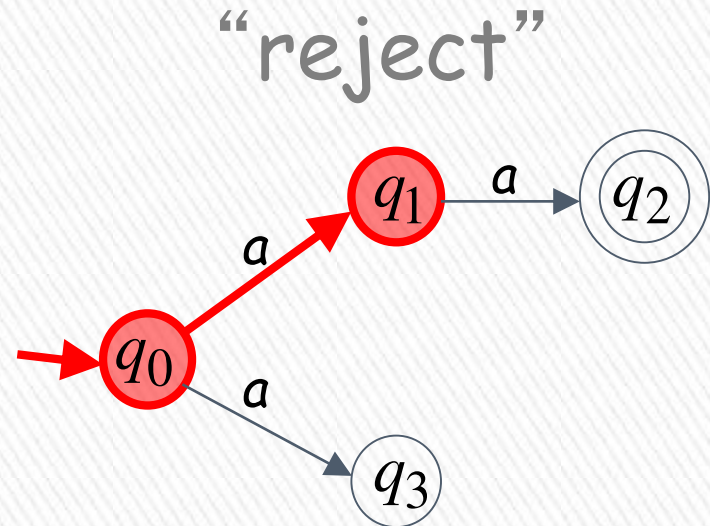
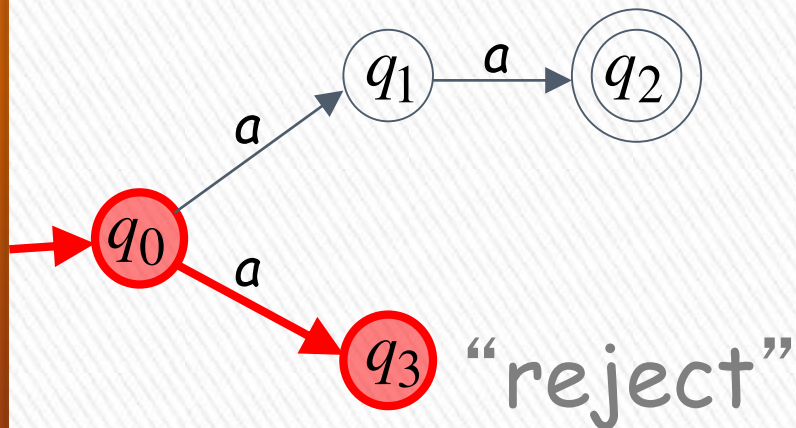
- All the input is consumed and the automaton is in a non final state

OR

- The input cannot be consumed

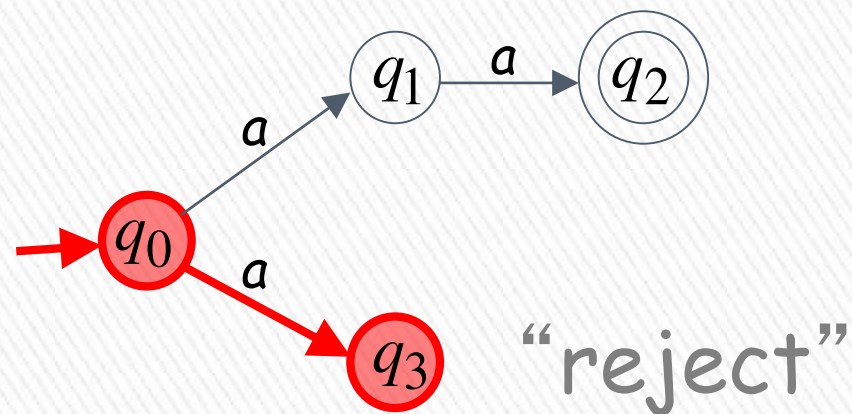
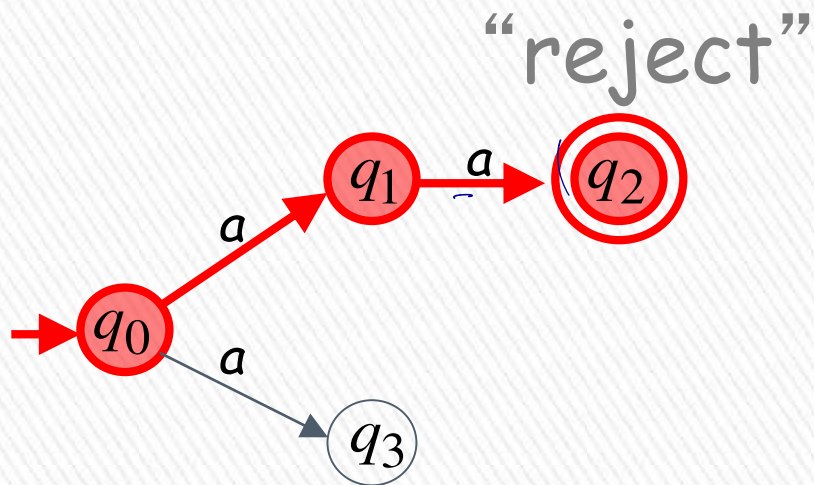


a is rejected by the NFA:



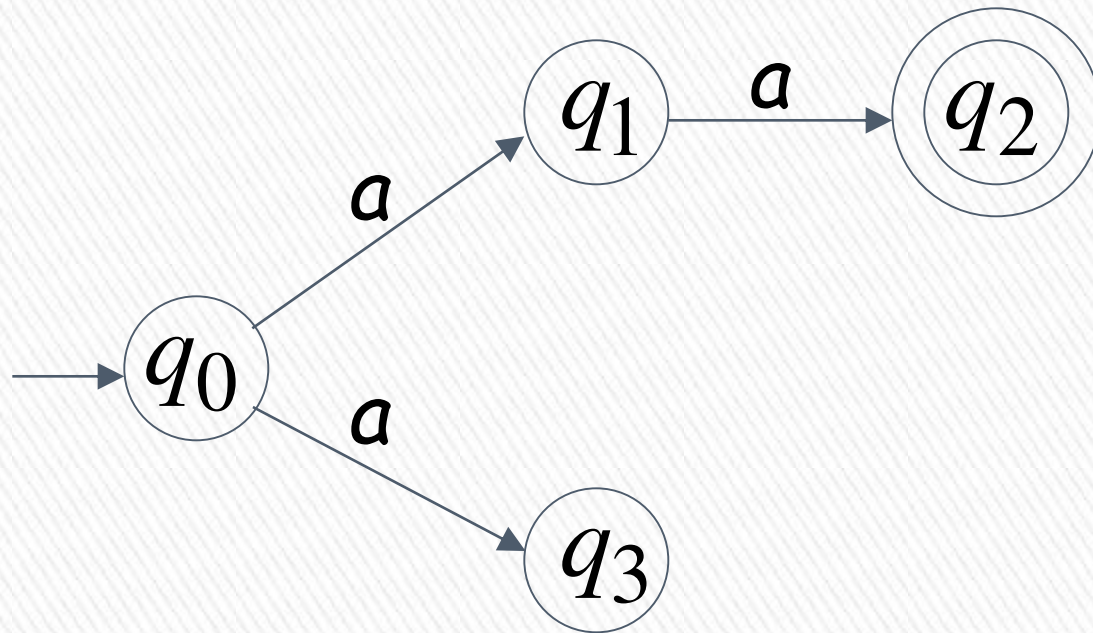
All possible computations lead to rejection

aaa is rejected by the NFA:

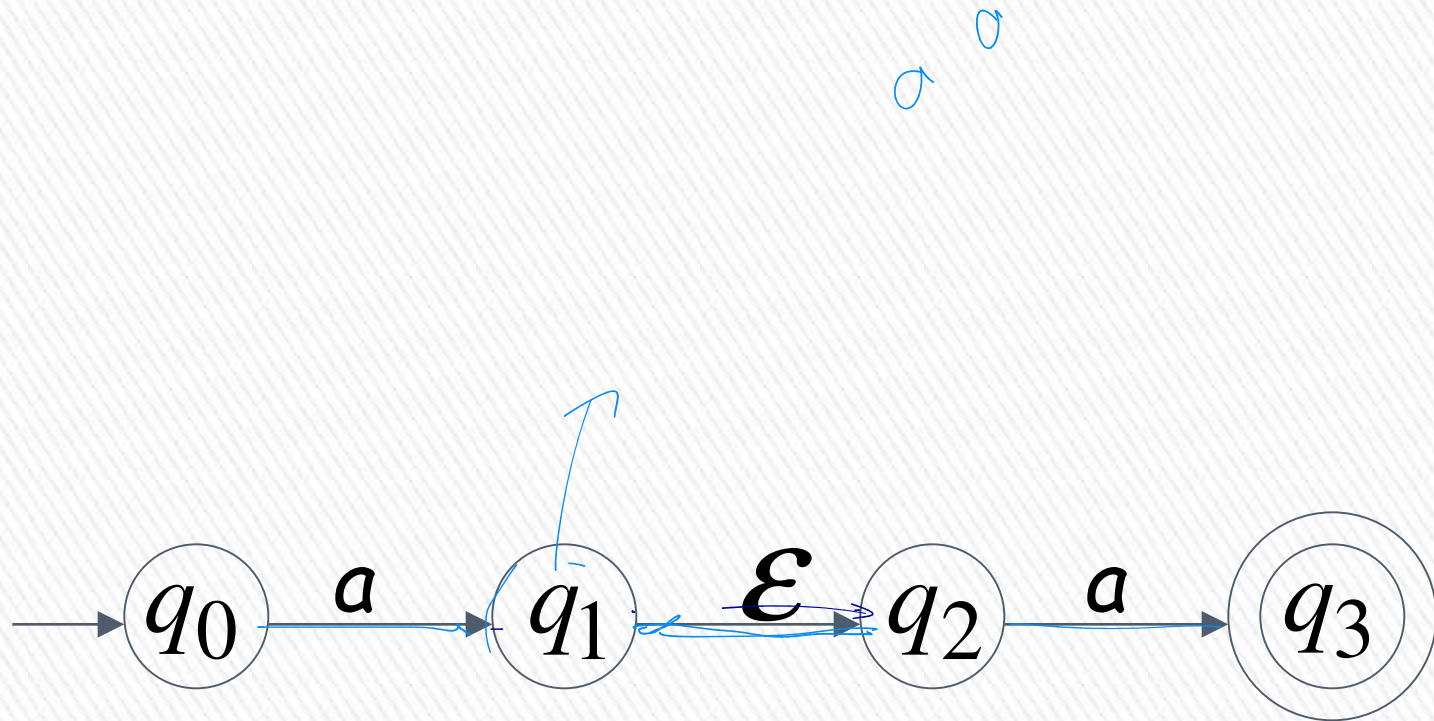


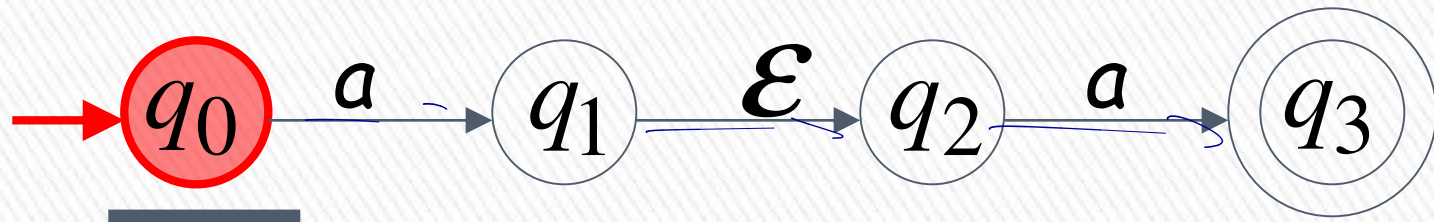
All possible computations lead to rejection

Language accepted: $L = \{aa\}$



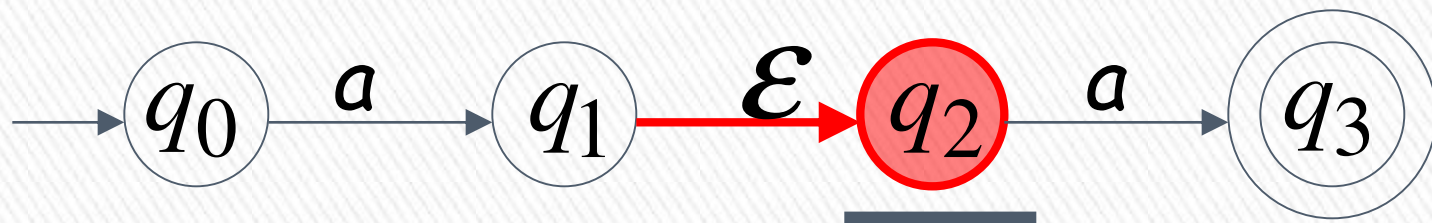
Epsilon Transition



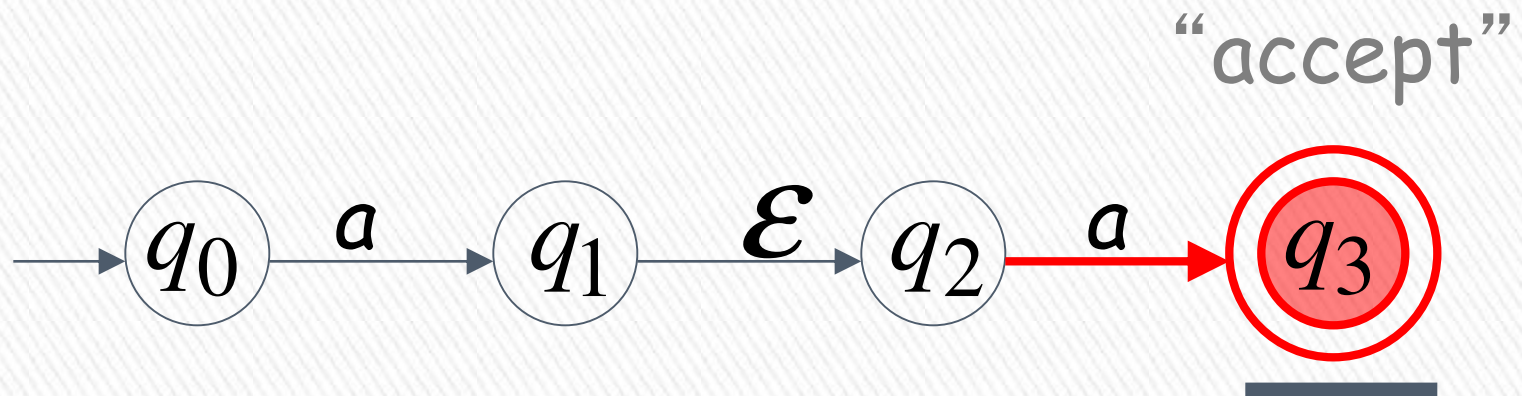




input tape head does not move



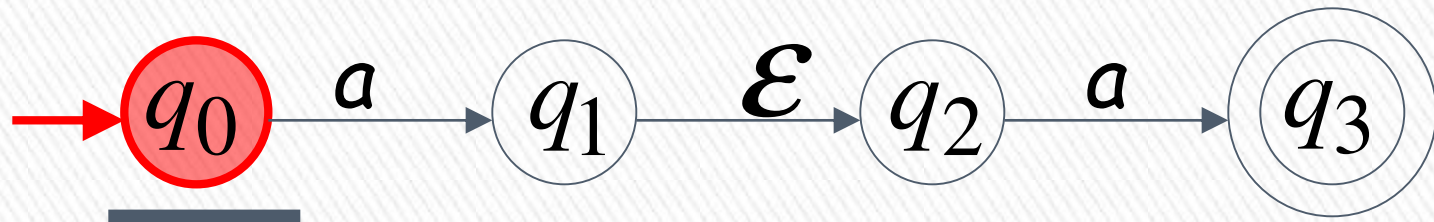
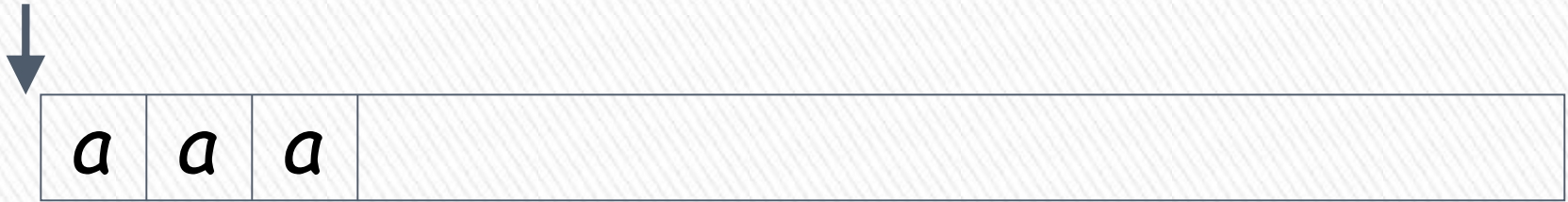
all input is consumed



String aa is accepted

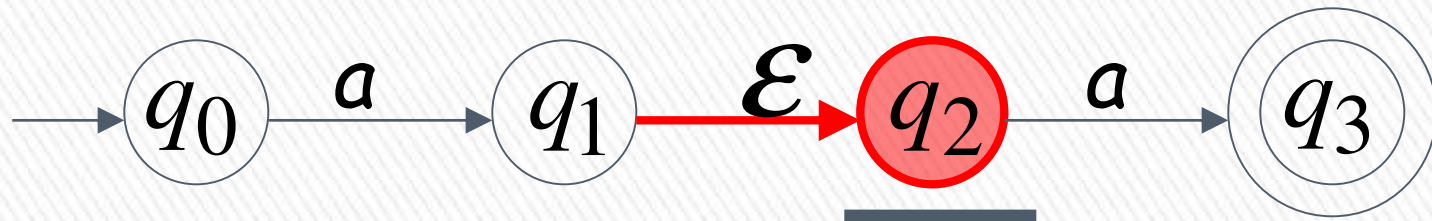


Rejection Example





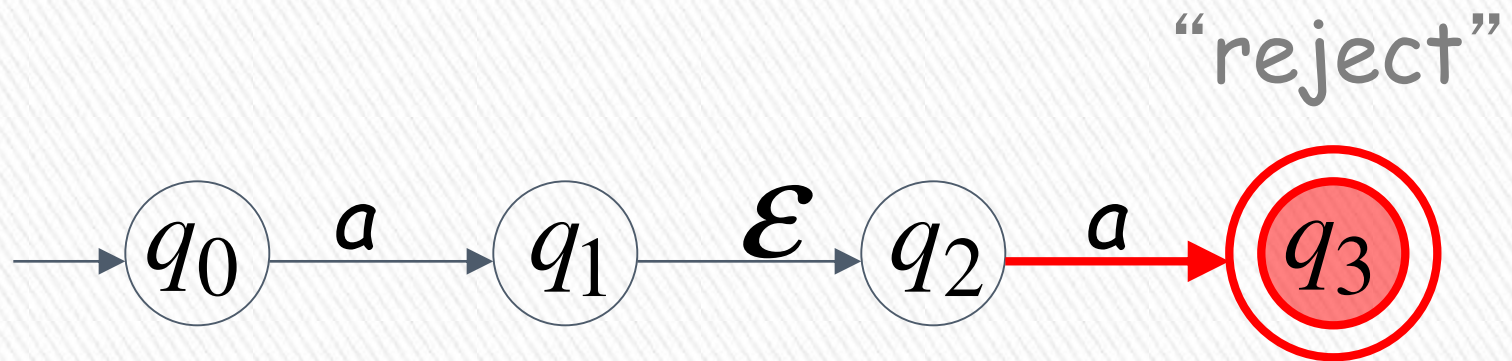
(read head doesn't move)



Input cannot be consumed



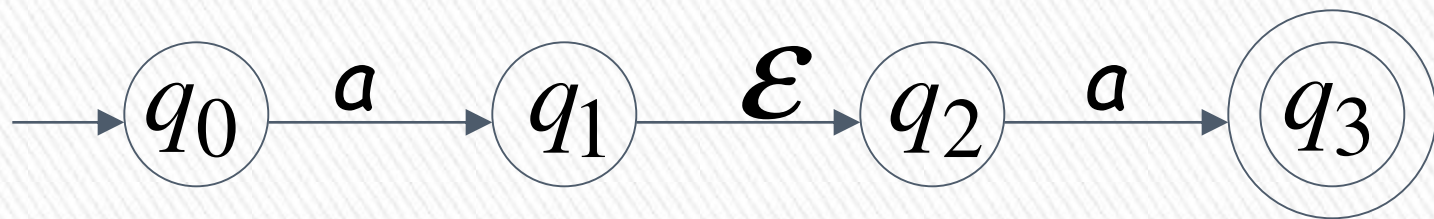
Automaton halts



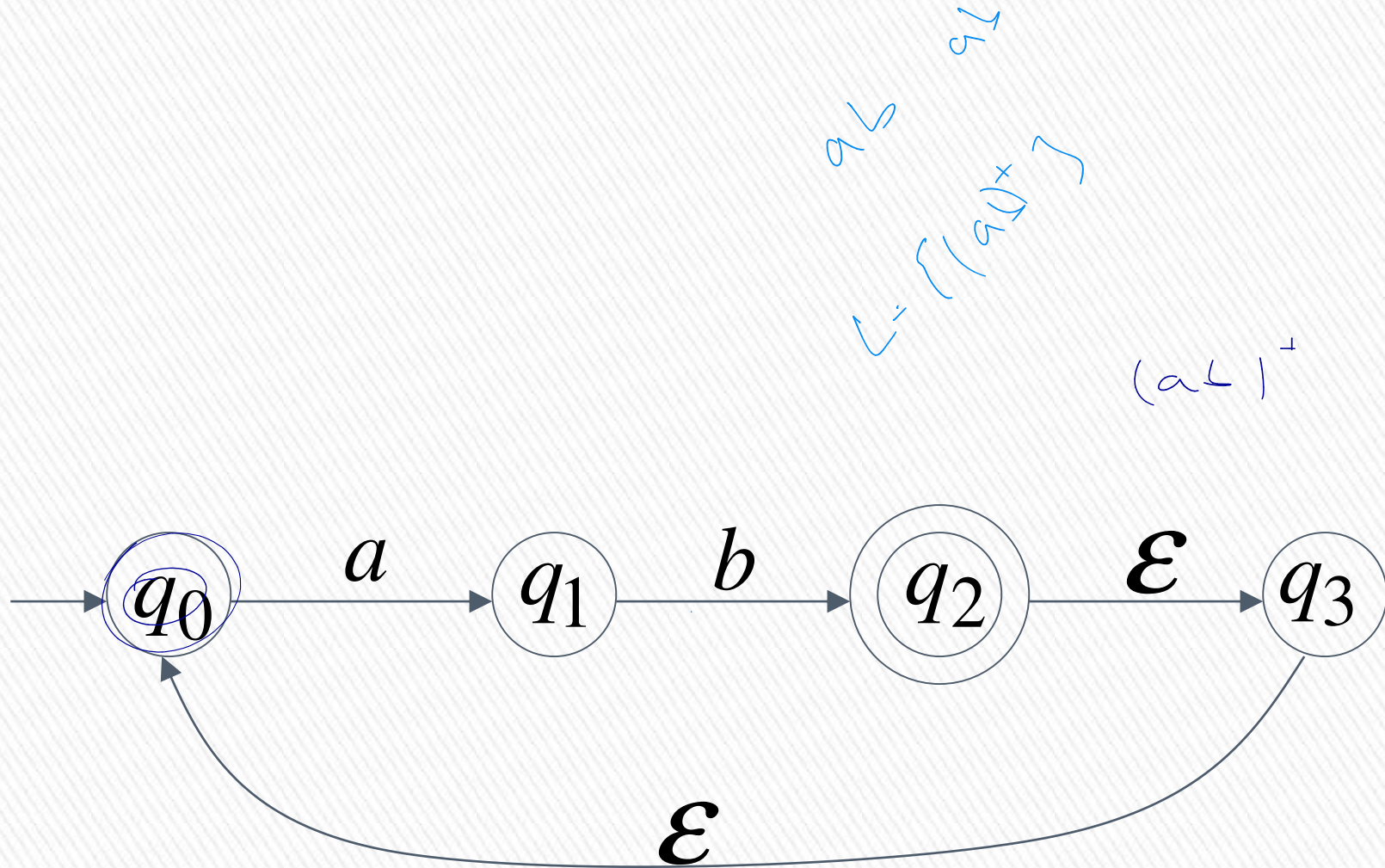
String **aaa** is rejected

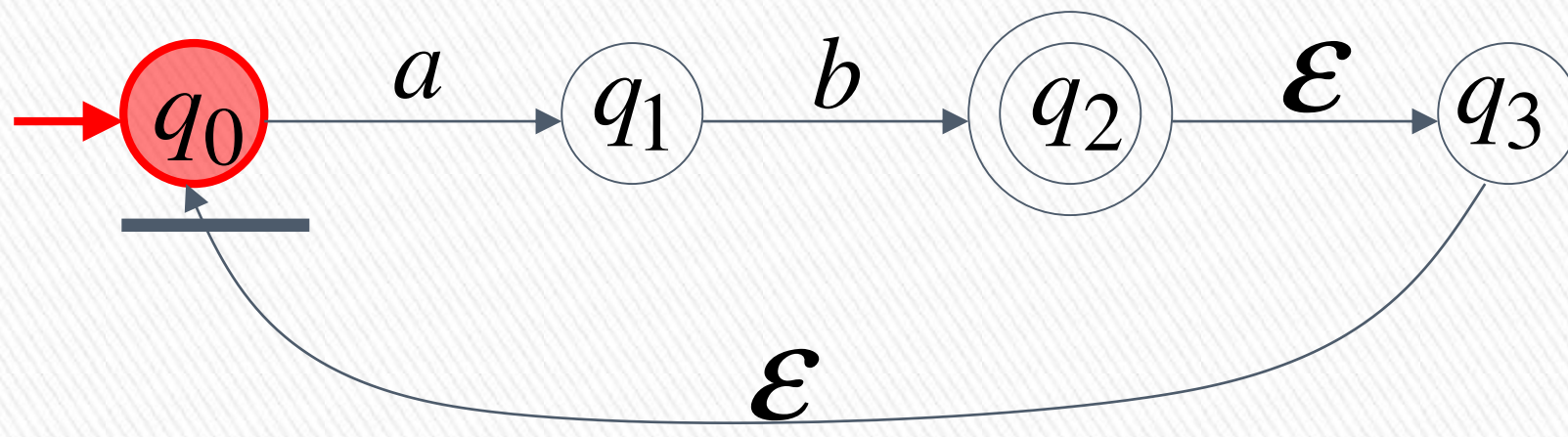


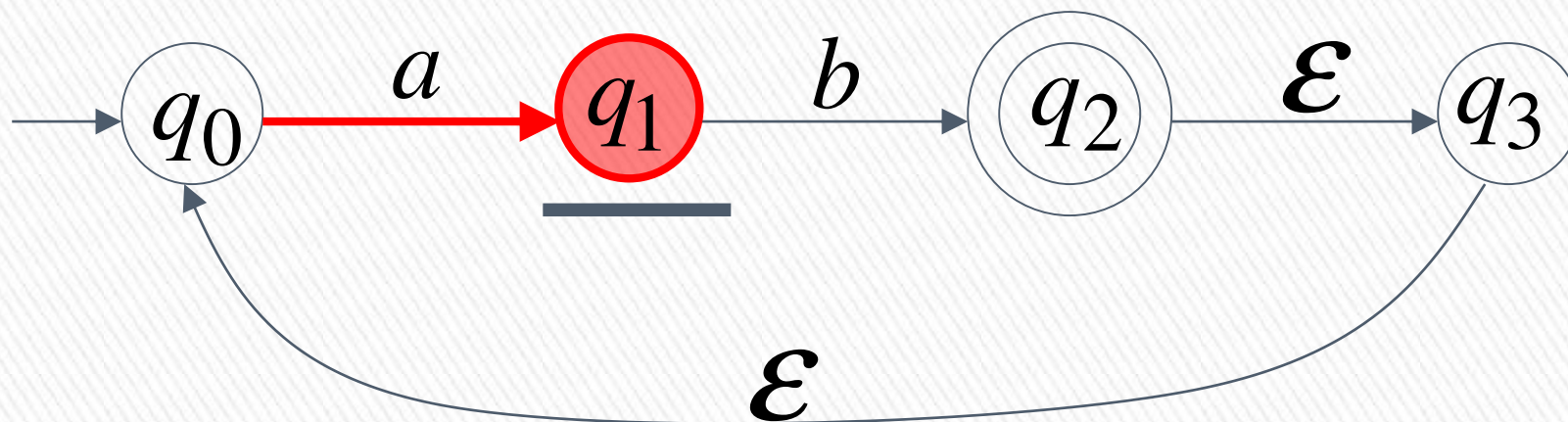
Language accepted: $L = \{aa\}$

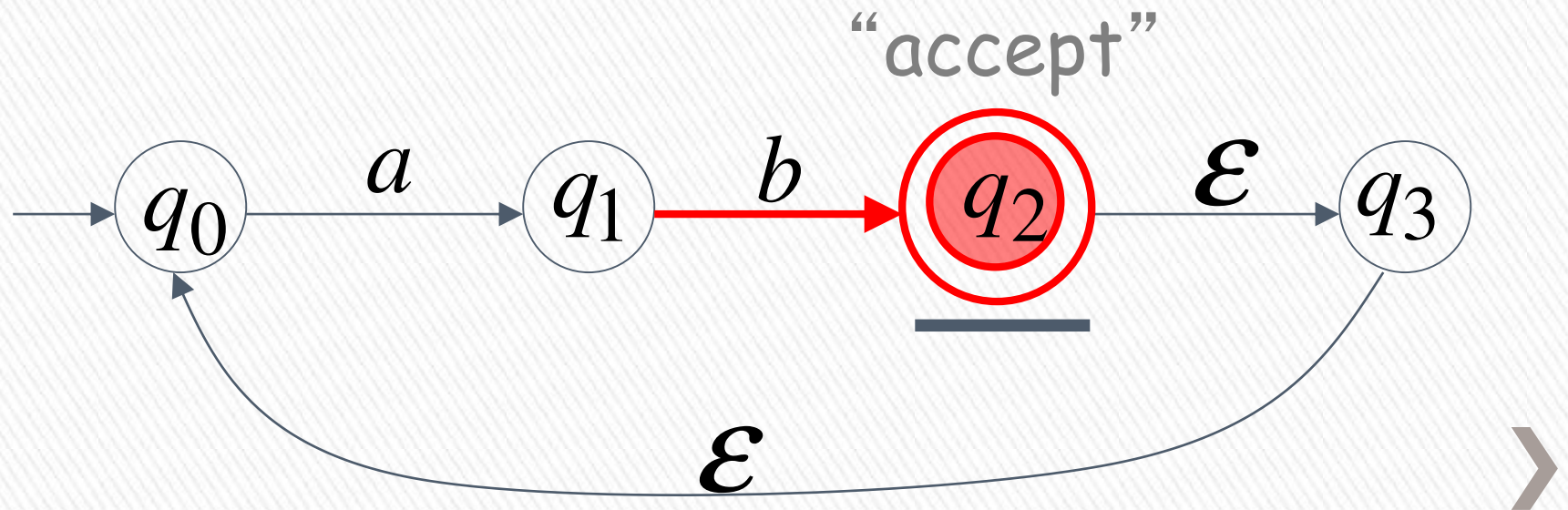


Another NFA Example

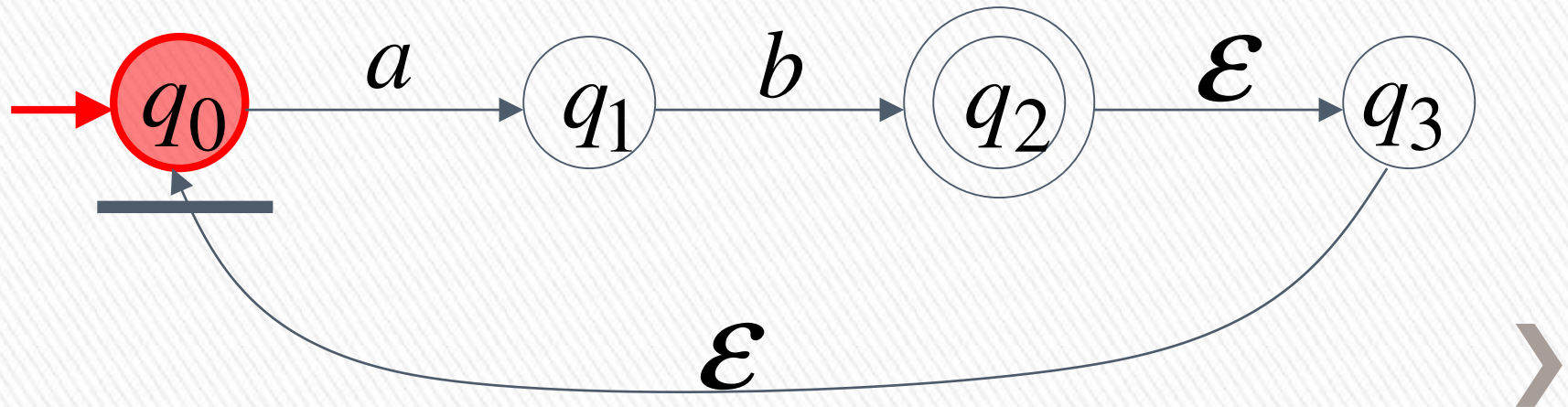
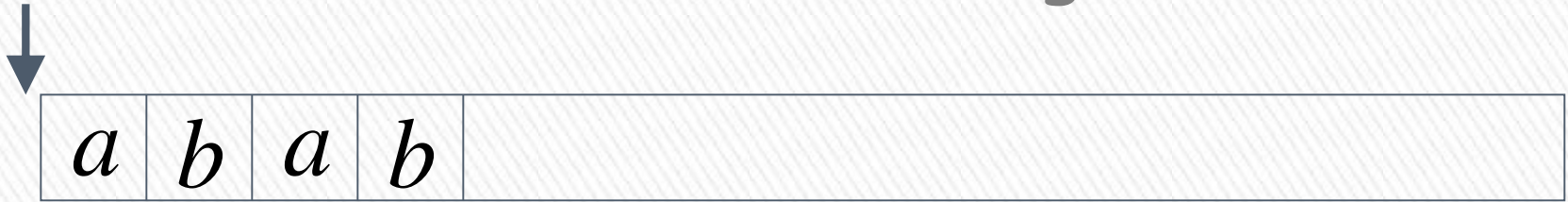


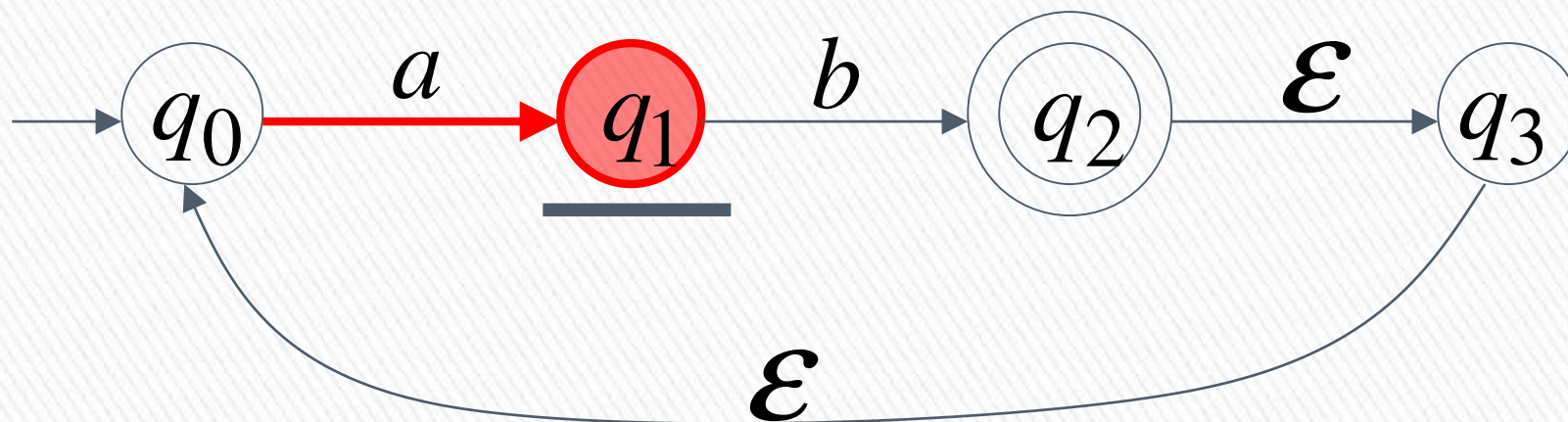


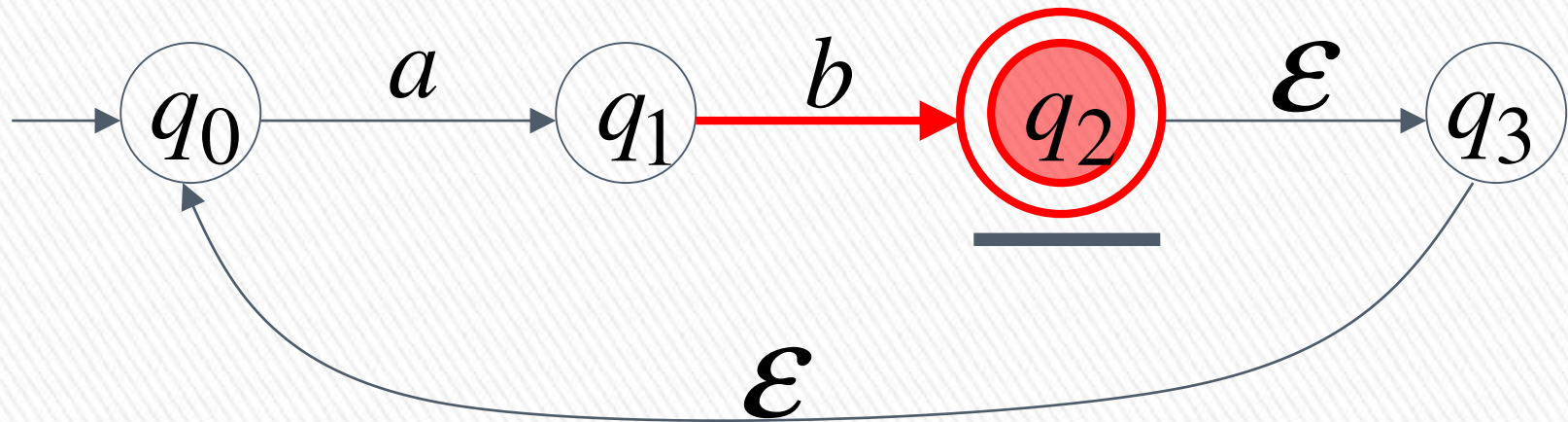


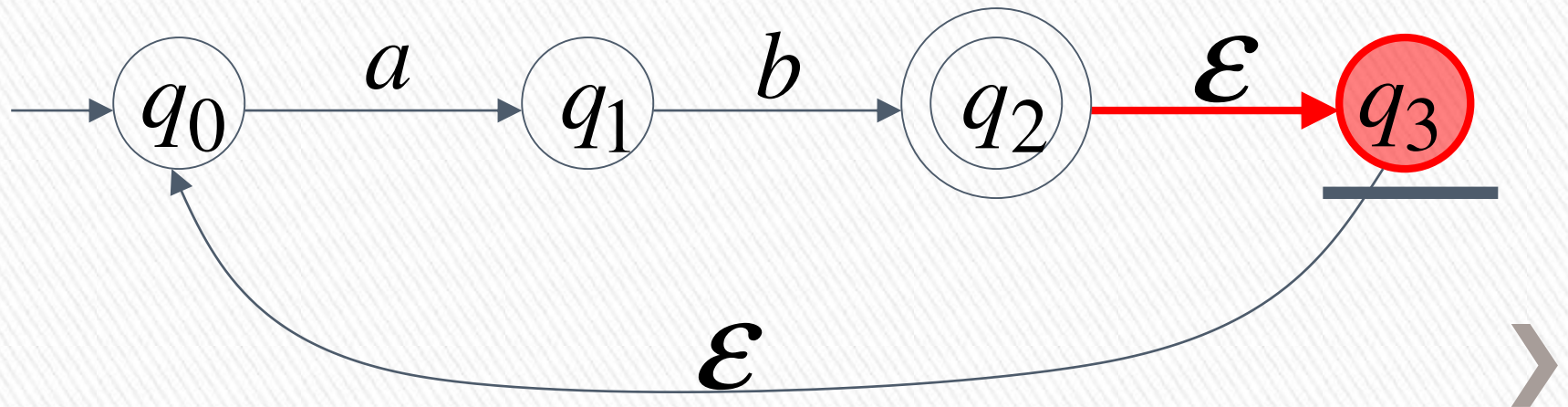


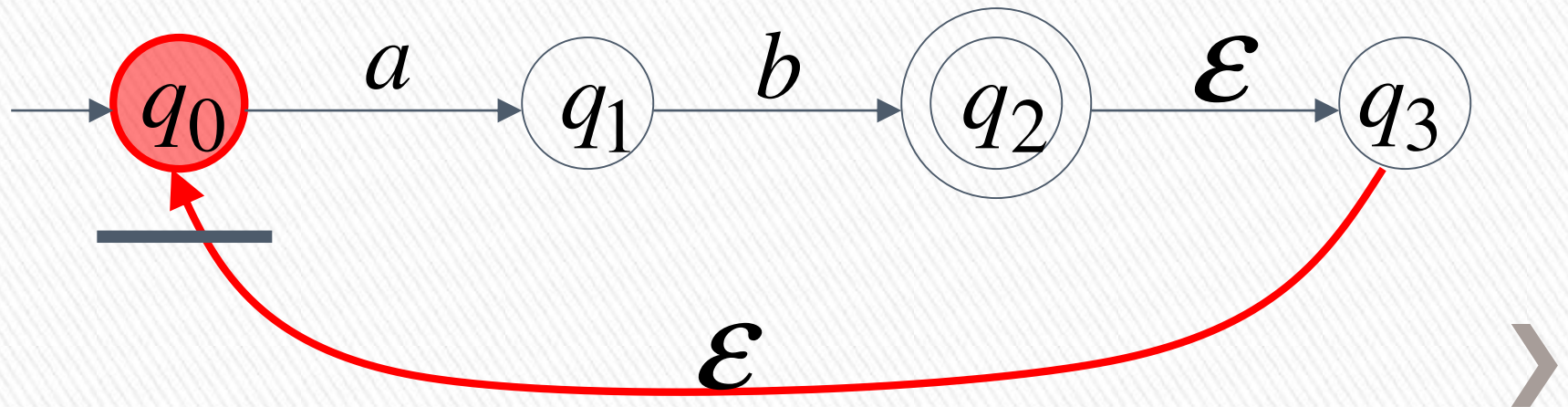
Another String

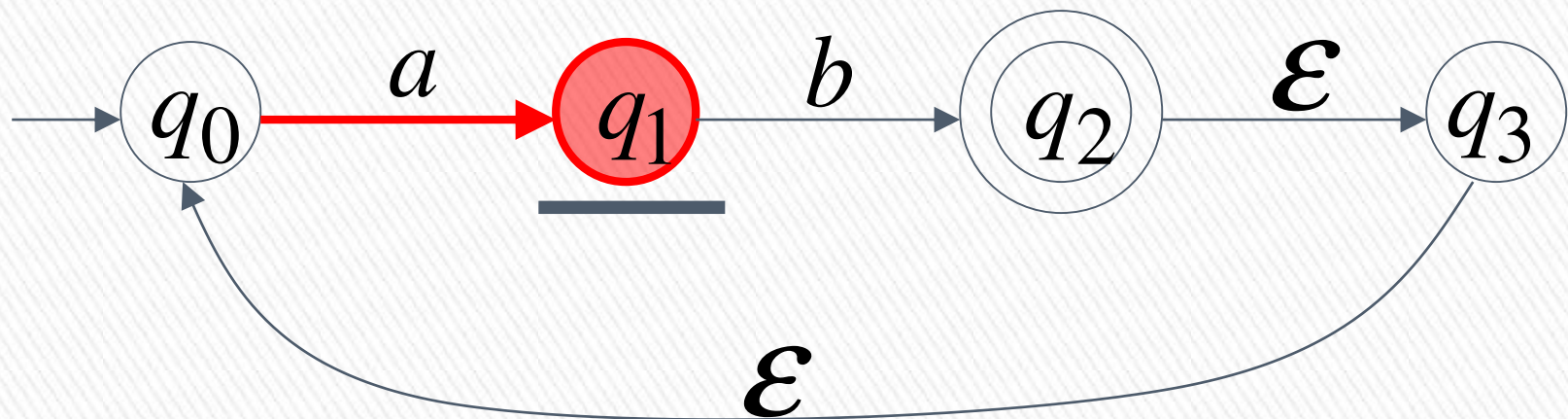


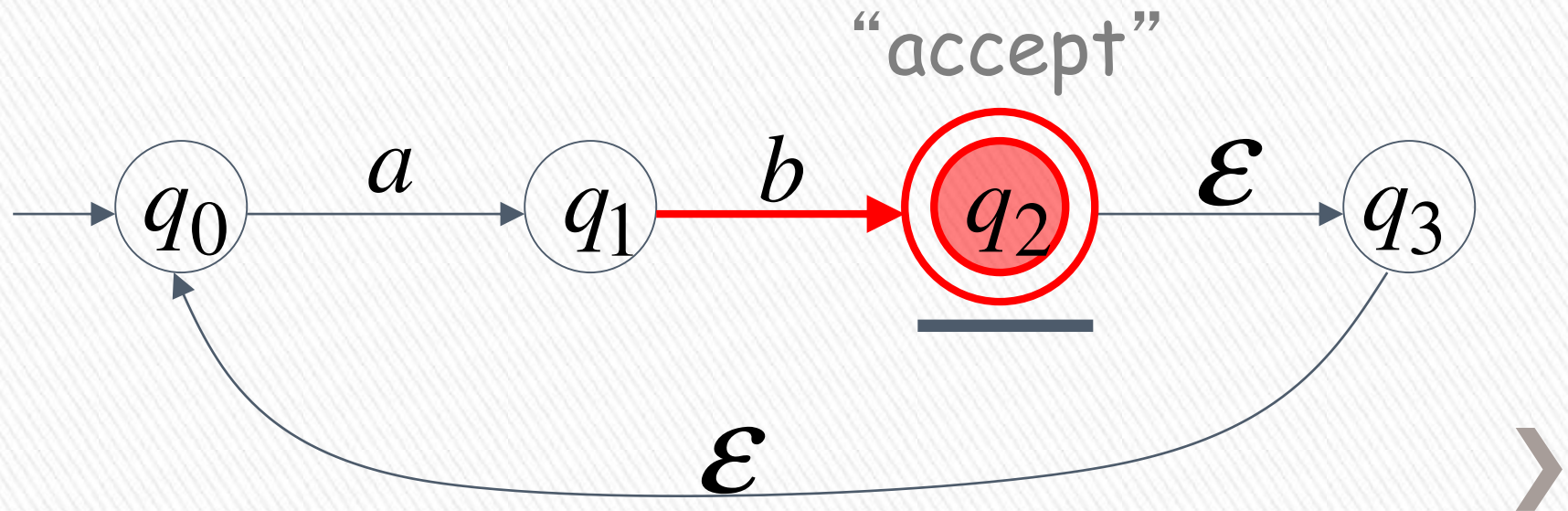






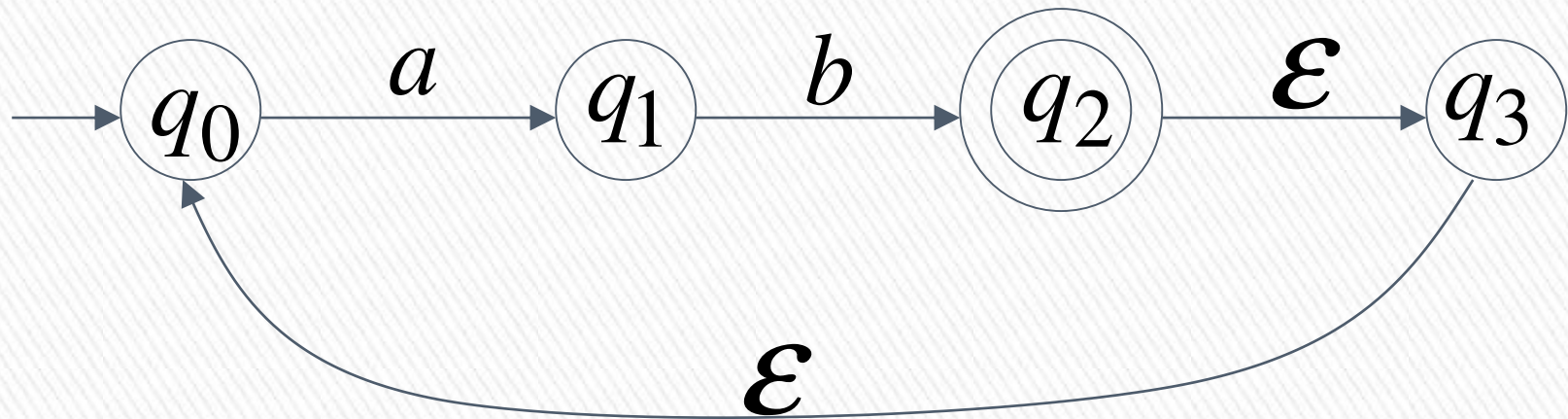






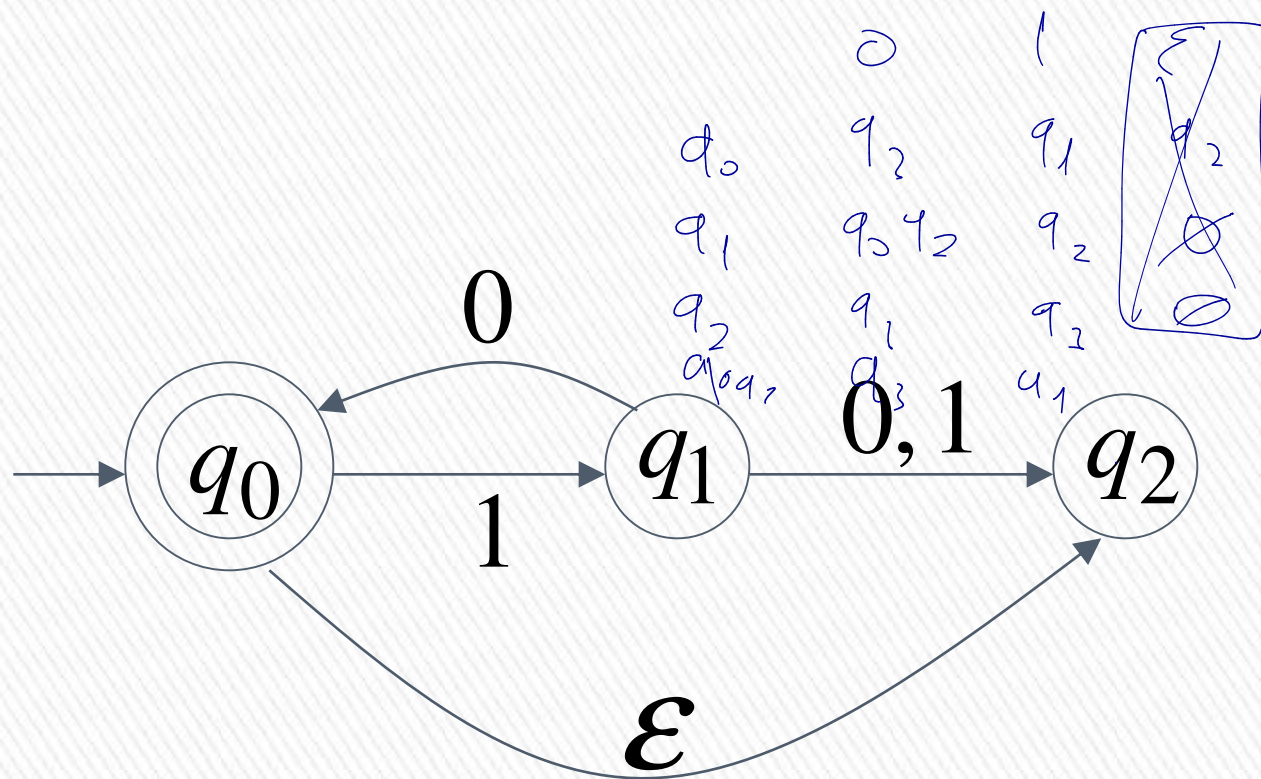
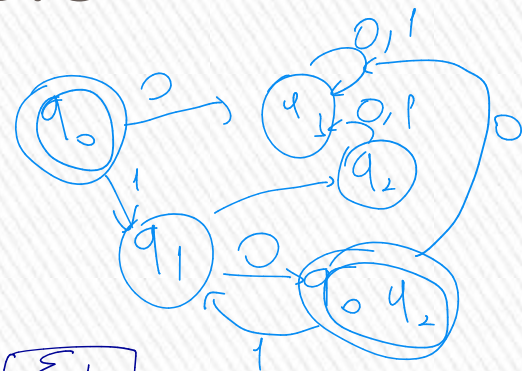
Language accepted

$$L = \{ab, abab, ababab, \dots\}$$
$$= \{ab\}^+$$



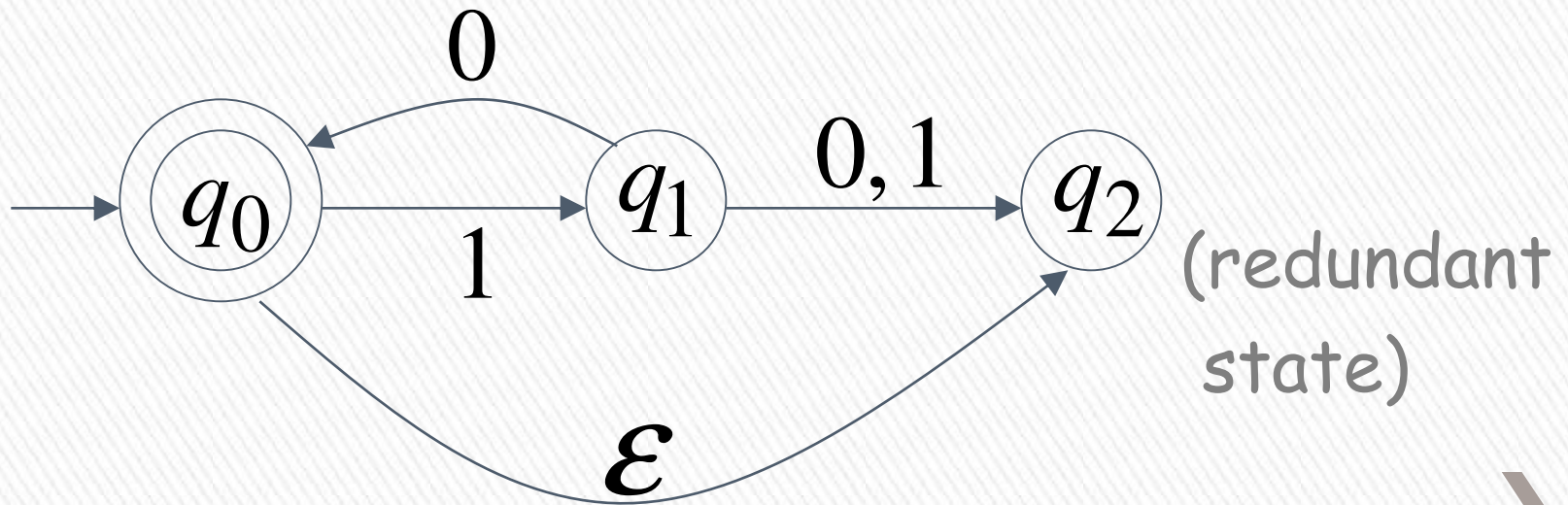
Another NFA Example

	0	1	ϵ
q_0	q_3	q_1	q_2
q_1	q_0, q_2	q_2	q_1
q_0, q_2	q_3	q_1	q_2



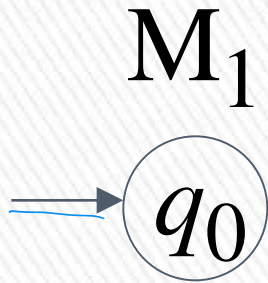
Language accepted

$$\begin{aligned} L(M) &= \{\varepsilon, 10, 1010, 101010, \dots\} \\ &= \{10\}^* \end{aligned}$$

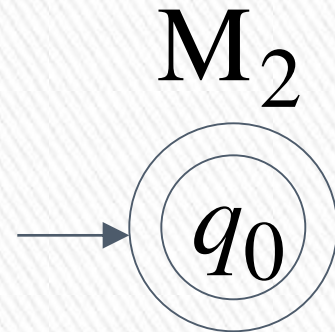


Remarks:

- The ε symbol never appears on the input tape
- Simple automata:



$$L(M_1) = \{ \}$$



$$L(M_2) = \{ \varepsilon \}$$



- NFAs are interesting because we can express languages easier than DFAs

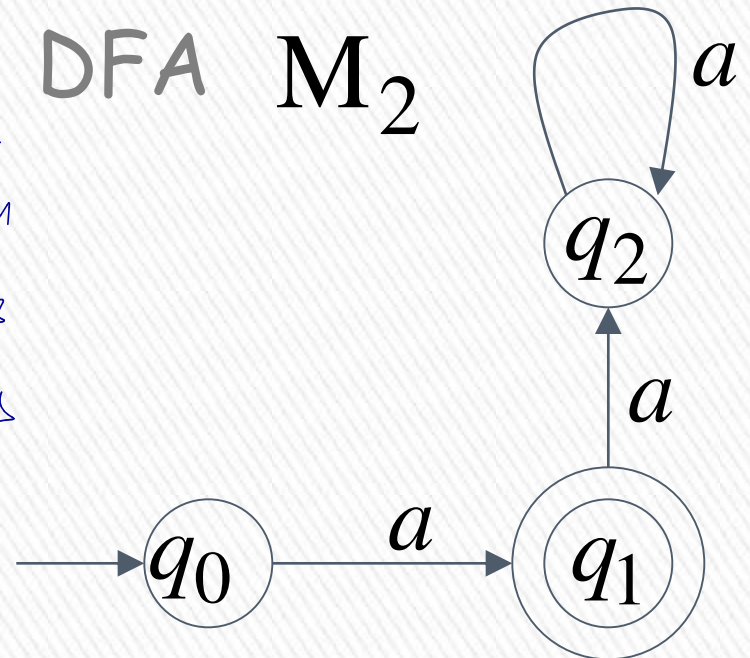
NFA M_1



$$L(M_1) = \{a\}$$

DFA M_2

q_0 q_1
 q_1 q_3
 q_2 q_4

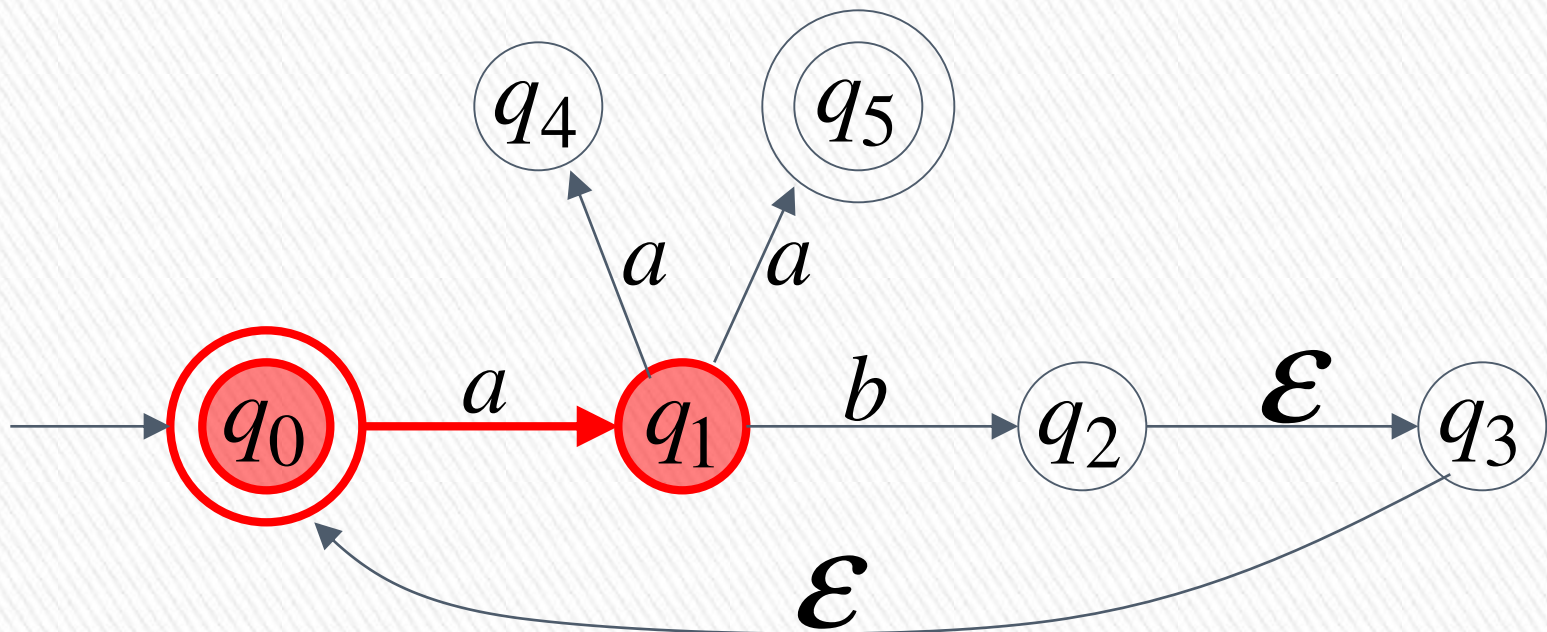


$$L(M_2) = \{a\}$$

Extended Transition Function δ^*

Same with δ but applied on strings

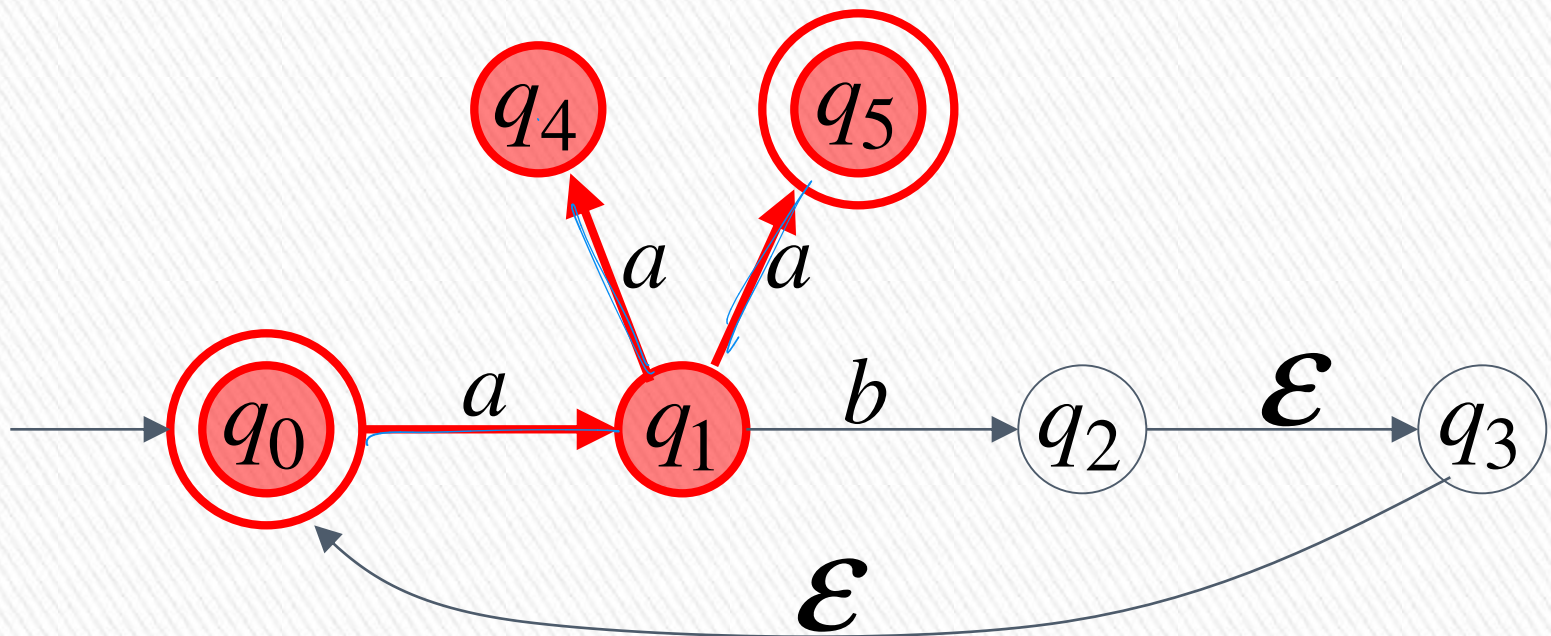
$$\delta^*(q_0, a) = \{q_1\}$$



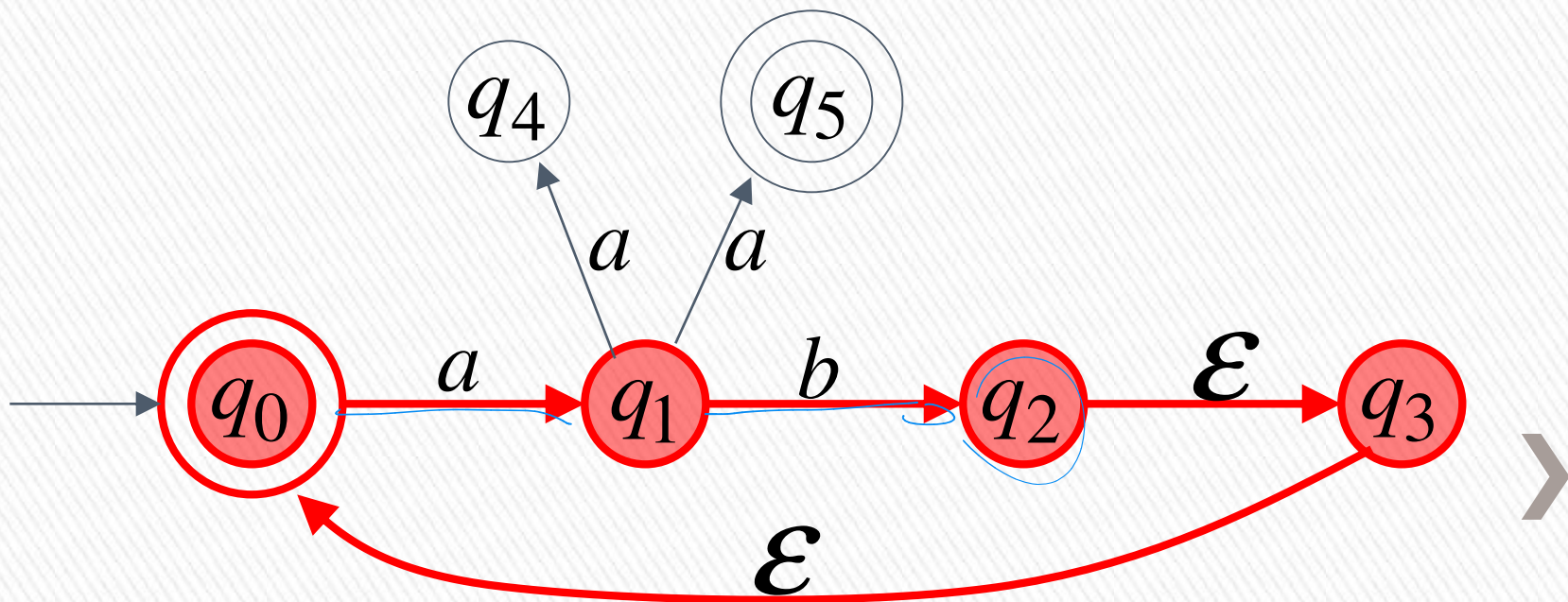
$$\delta^*(q_0, aa) = \{q_4, q_5\}$$

$$\delta^*(q_0, a) = \{q_1\}$$

$$\delta^*(q_1, a) = \{q_4, q_5\}$$



$$\delta^*(q_0, ab) = \{q_2, q_3, q_0\}$$



In general

$q_j \in \delta^*(q_i, w)$: there is a walk from q_i to q_j
with label w



$$w = \sigma_1 \sigma_2 \cdots \sigma_k$$



The Language of an NFA

» The language accepted by M is:

$$L(M) = \{w_1, w_2, \dots, w_n\}$$

» where

$$\delta^*(q_0, w_m) = \{q_i, \dots, q_k, \dots, q_j\}$$

» and there is some $q_k \in F$

(accepting state)

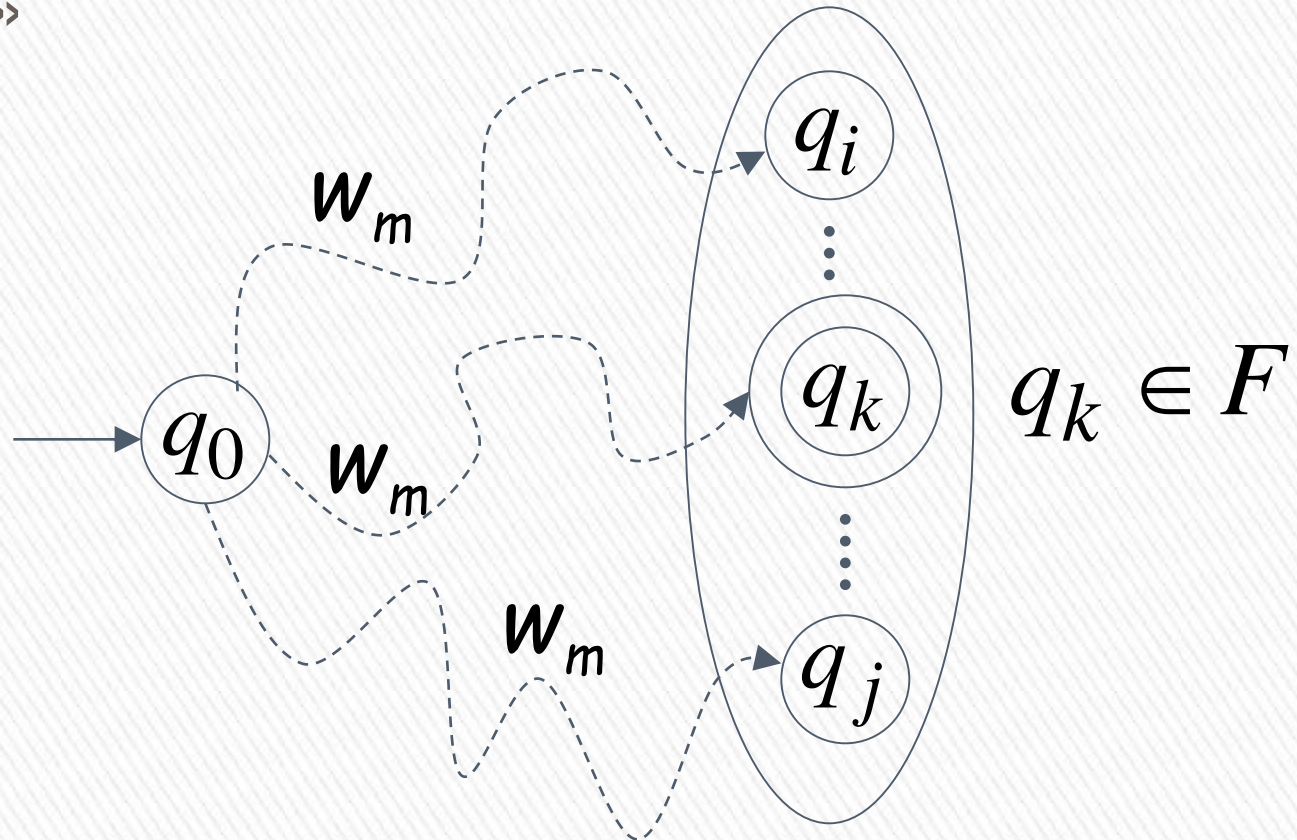


$$w_m \in L(M)$$

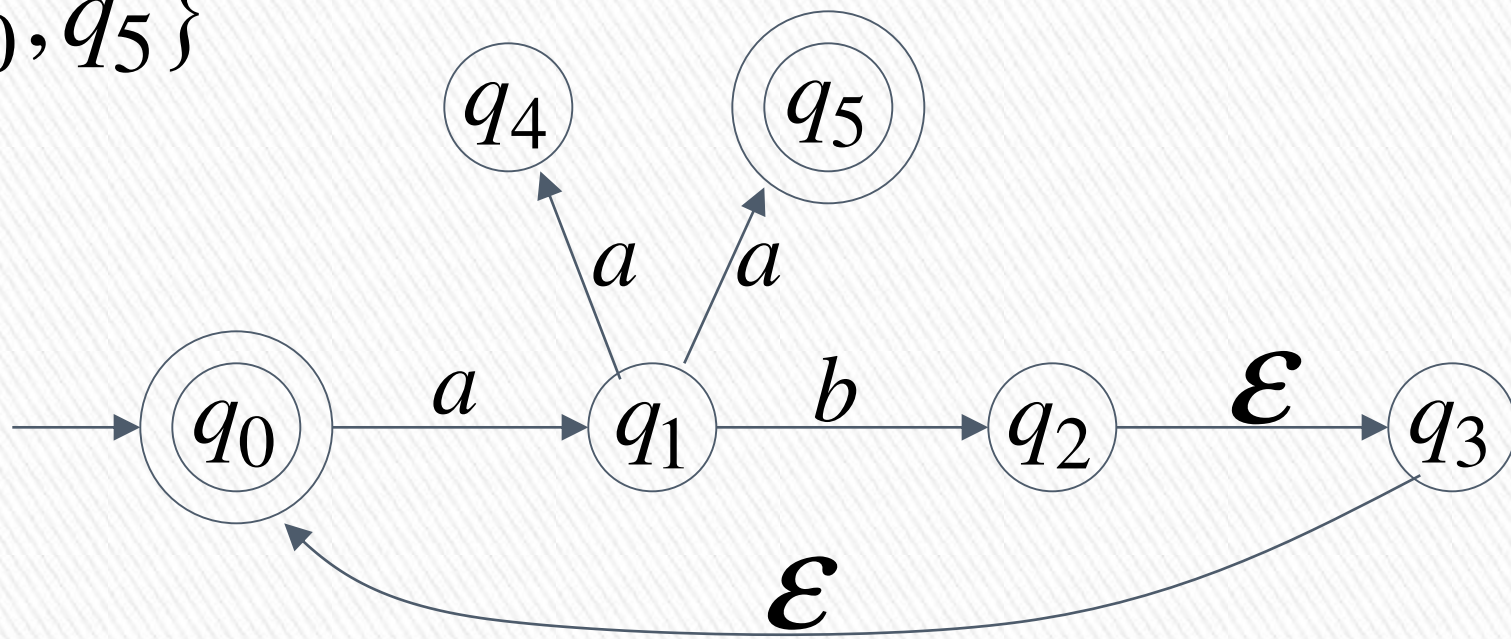
>>

>>

$$\delta^*(q_0, w_m)$$



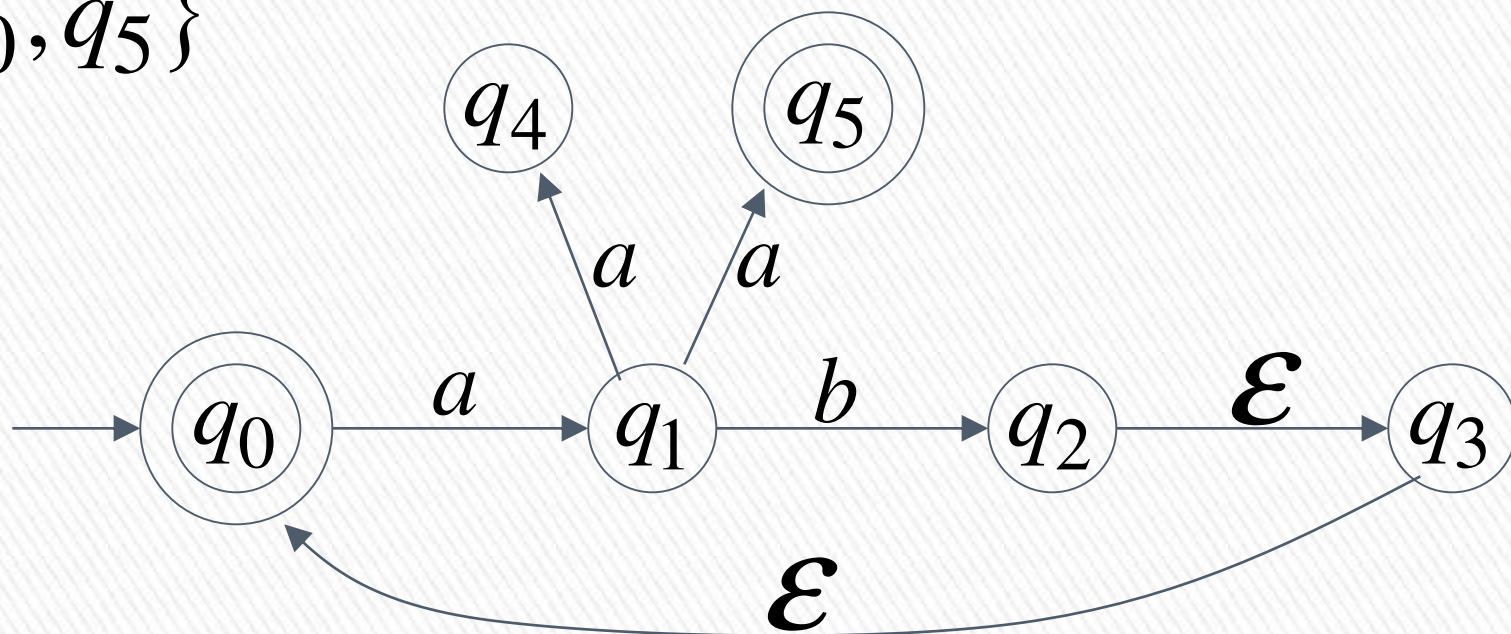
$$F = \{q_0, q_5\}$$



$$\delta^*(q_0, aa) = \{q_4, \underline{q_5}\} \xrightarrow{\quad} aa \in L(M)$$

$\swarrow \in F$

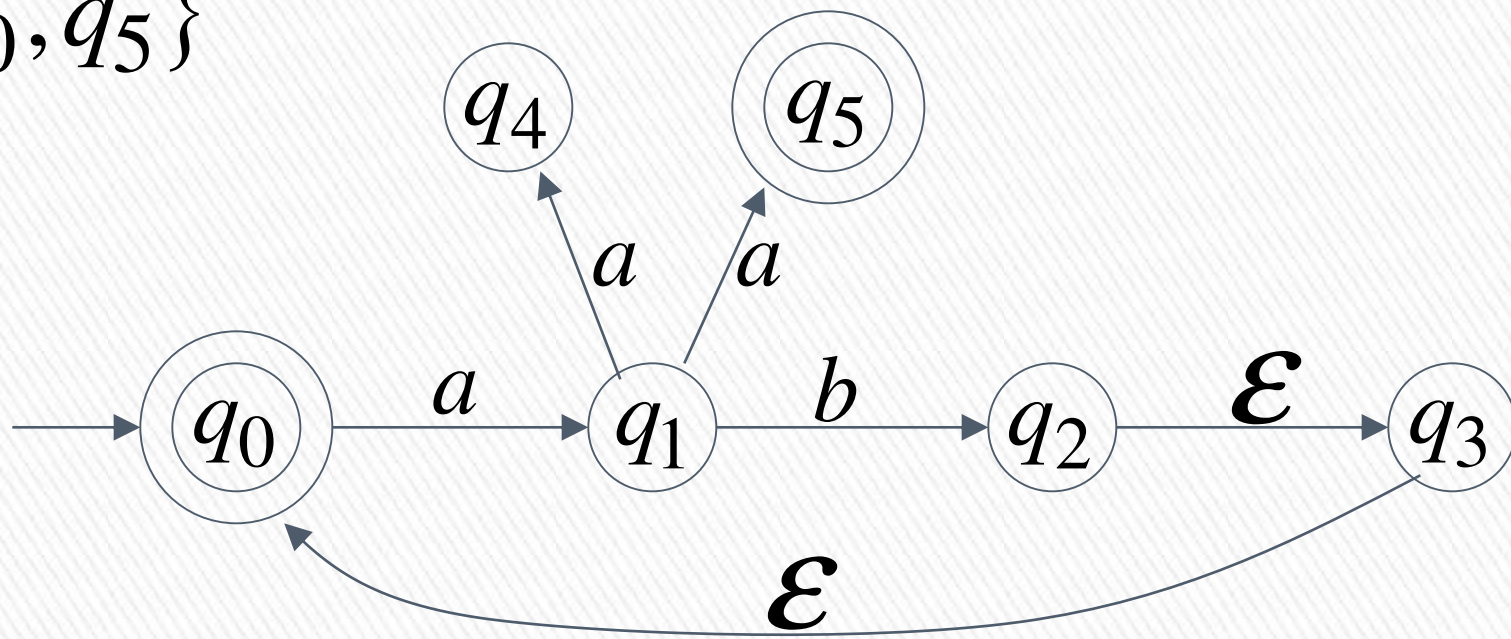
$$F = \{q_0, q_5\}$$



$$\delta^*(q_0, ab) = \{q_2, q_3, \underline{q_0}\} \xrightarrow{\quad} ab \in L(M) \gg$$

$\swarrow$
 $\in F$

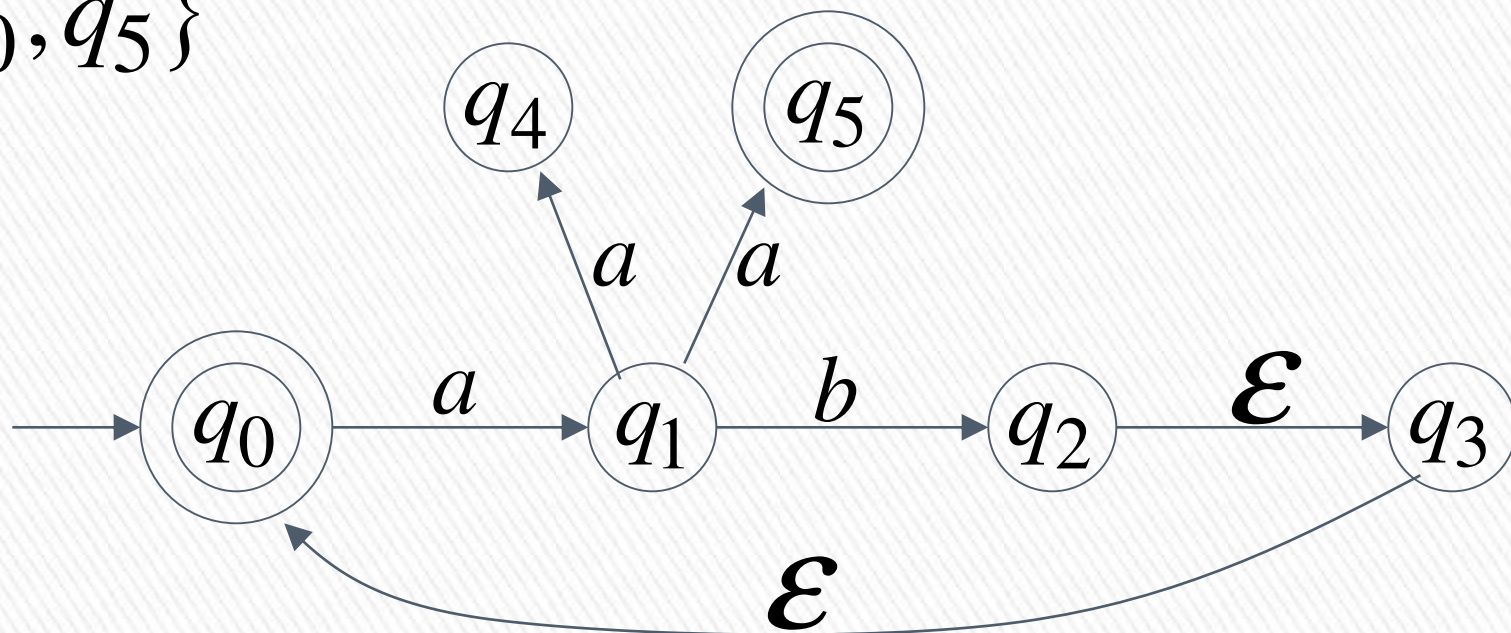
$$F = \{q_0, q_5\}$$



$$\delta^*(q_0, abaa) = \{q_4, \underline{q_5}\} \quad \xrightarrow{\text{yellow arrow}} \quad abaa \in L(M)$$

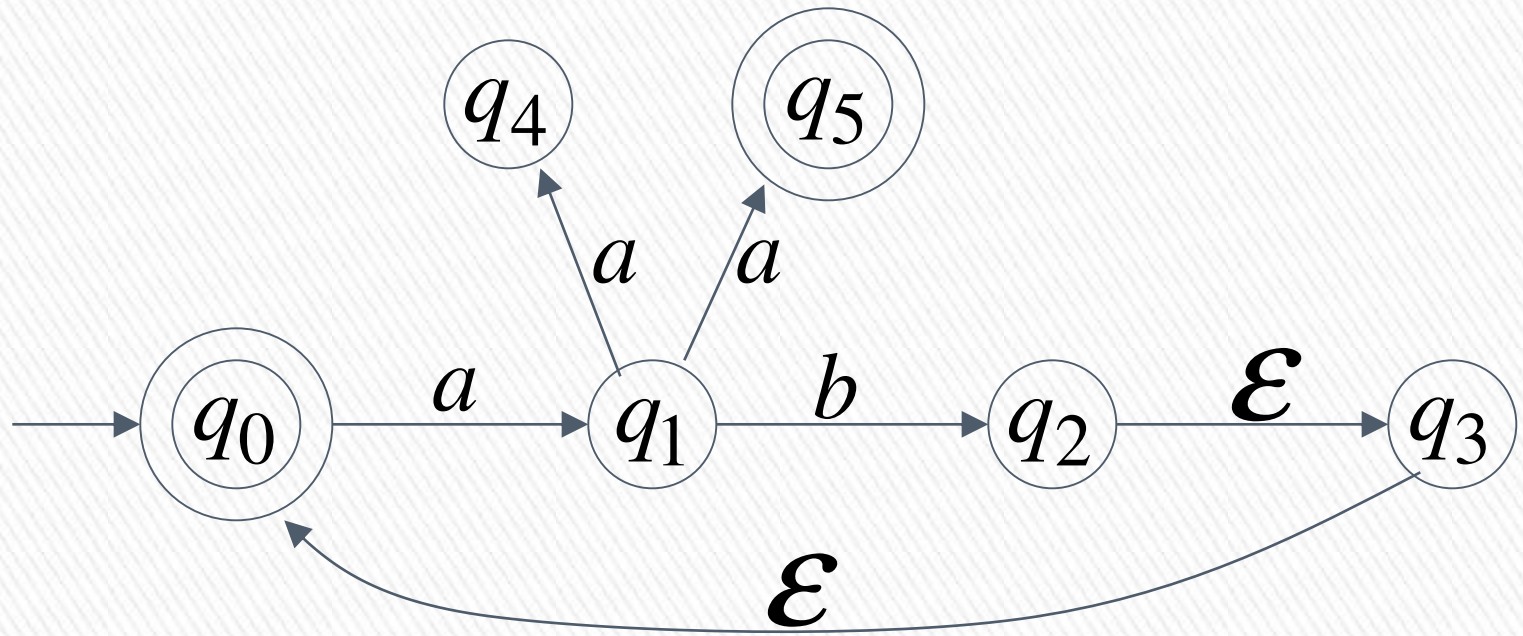
$\nwarrow \in F$
➤

$$F = \{q_0, q_5\}$$



$$\delta^*(q_0, aba) = \{q_1\} \xrightarrow{\text{yellow arrow}} aba \notin L(M) \succ$$

$\nwarrow \notin F$



$$L(M) = \{ab\}^* \bigcup \{ab\}^* \{aa\}$$

$\swarrow$
 or



Equivalence of Machines

» Definition:

if: machine M_1 and machine M_2 differ:
accept condition exists

» Machine M_1 is equivalent to machine M_2

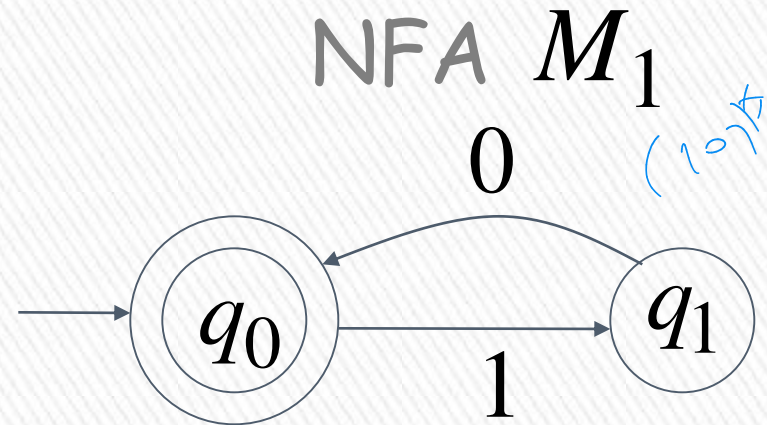
if $L(M_1) = L(M_2)$



Example of equivalent machines

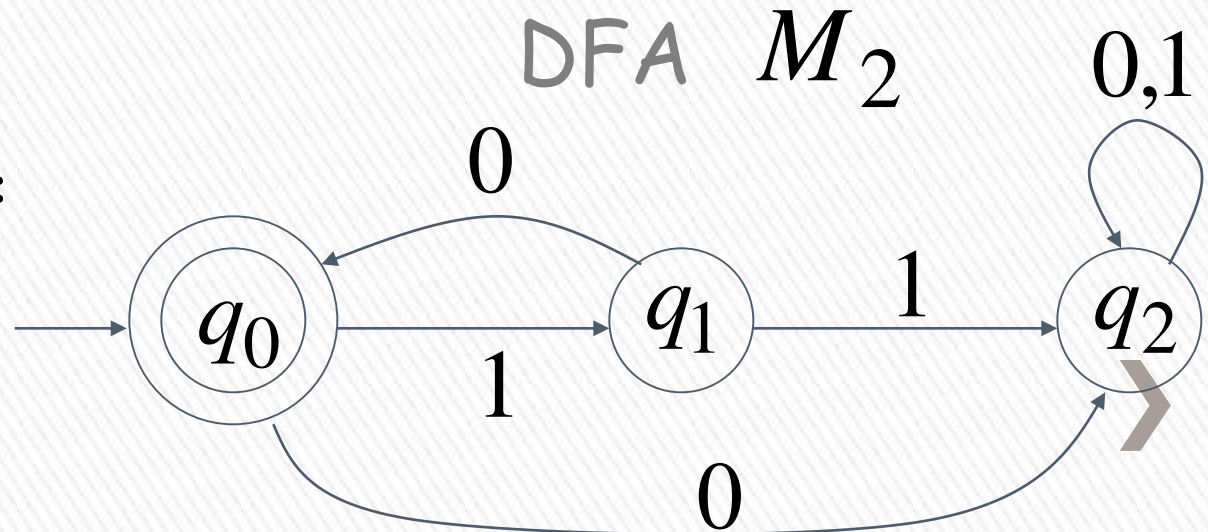
>>

$$L(M_1) = \{10\}^*$$



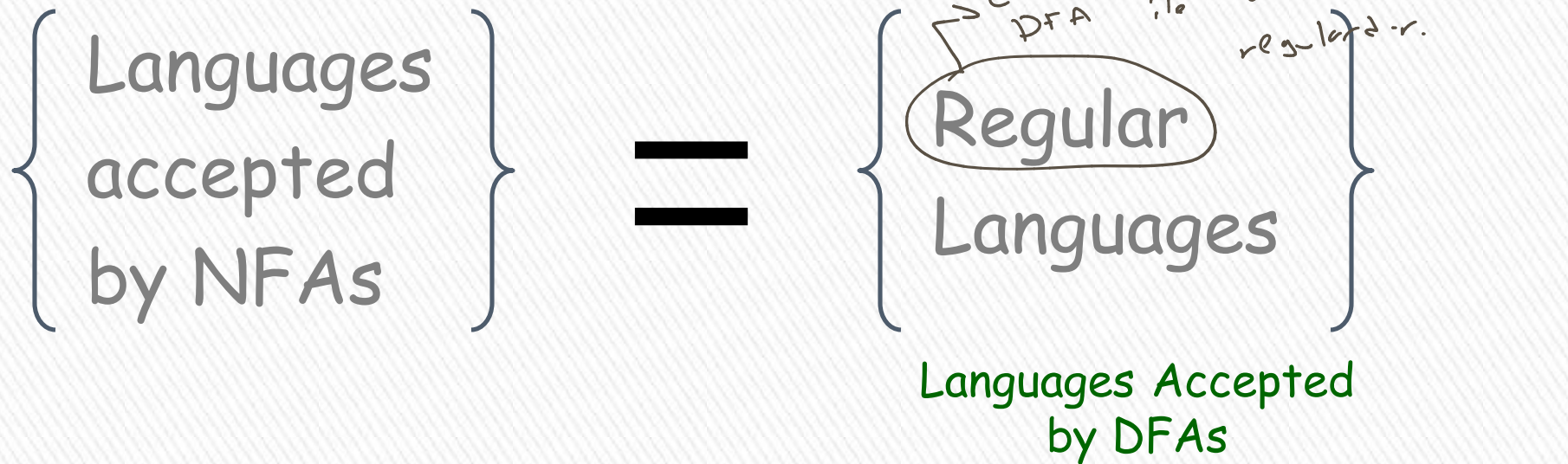
2. kann weniger oder äquivalent mit? versuchen
hier die $L(M_1) = \{10\}^*$ krip. maschine: DFA oder NFA
zeichnen.

$$L(M_2) = \{10\}^*$$



NFAs accept Regular Languages

Theorem:



NFAs and DFAs have the same computation power, accept the same set of languages

Proof: we only need to show

NFA $\rightarrow$ DFA's
for all L's

DFA $\rightarrow$ NFA's

{ Languages
accepted
by NFAs }

$\supseteq$

{ Regular
Languages }

AND

{ Languages
accepted
by NFAs }

$\subseteq$

{ Regular
Languages }



Proof-Step 1

$$\left\{ \begin{array}{l} \text{Languages} \\ \text{accepted} \\ \text{by NFAs} \end{array} \right\} \supseteq \left\{ \begin{array}{l} \text{Regular} \\ \text{Languages} \end{array} \right\}$$

Every DFA is trivially an NFA



Any language L accepted by a DFA
is also accepted by an NFA



Proof-Step 2

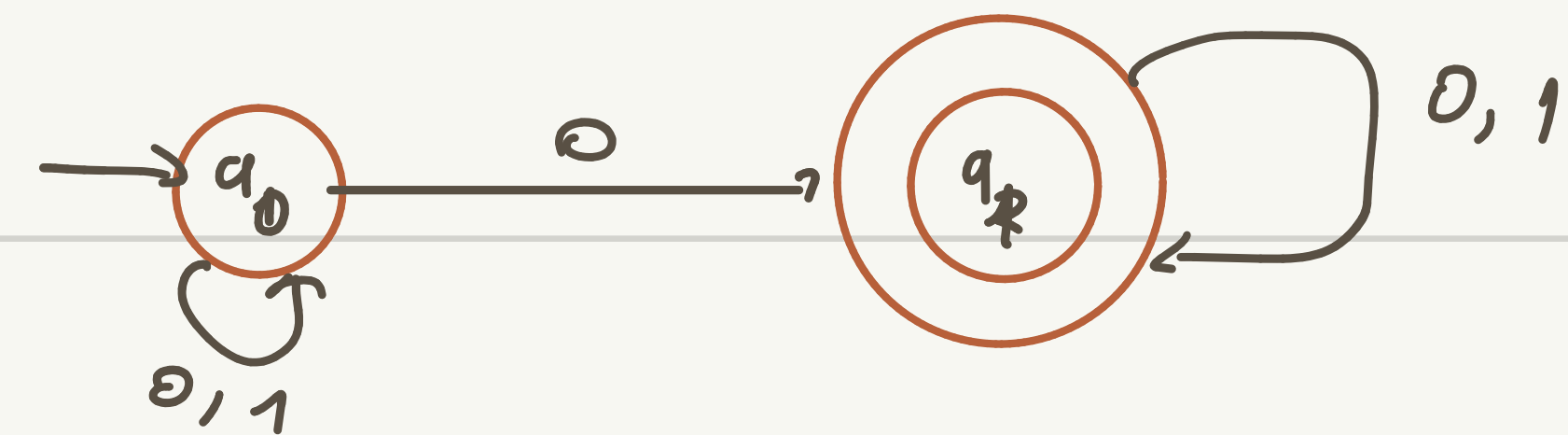
$$\left\{ \begin{array}{l} \text{Languages} \\ \text{accepted} \\ \text{by NFAs} \end{array} \right\} \subseteq \left\{ \begin{array}{l} \text{Regular} \\ \text{Languages} \end{array} \right\}$$

Any NFA can be converted to an equivalent DFA



Any language L accepted by an NFA is also accepted by a DFA 

$L_1 = \{ \text{set of all strings that contain 0} \}$



$\Sigma = \{0, 1\}$

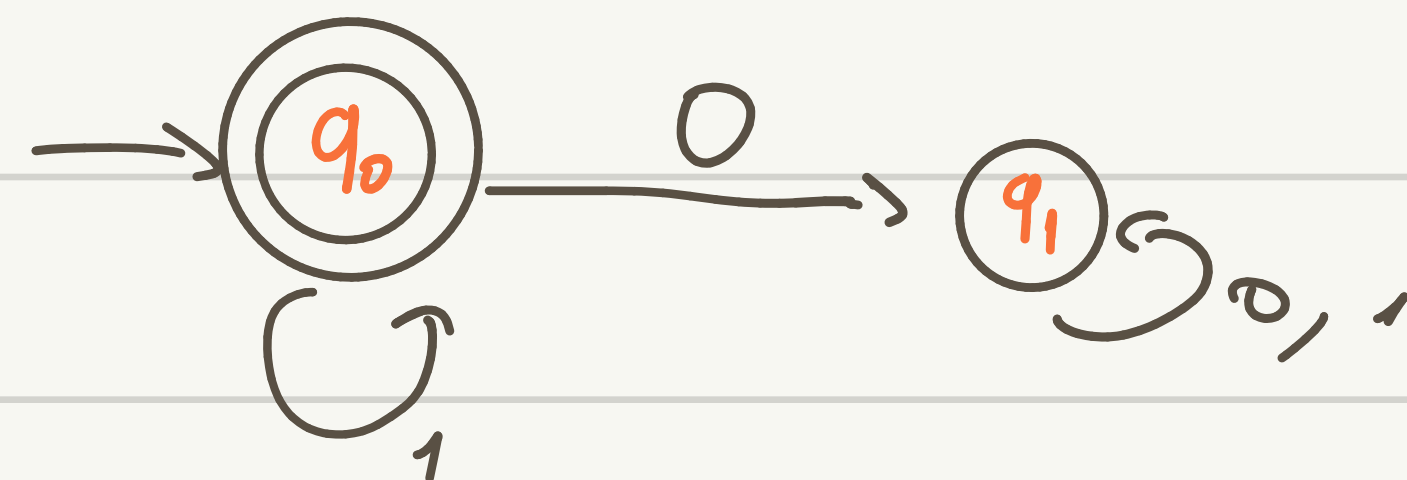
$Q = \{q_0, q_1\}$

q_0 initial state

$F = \{q_1\}$

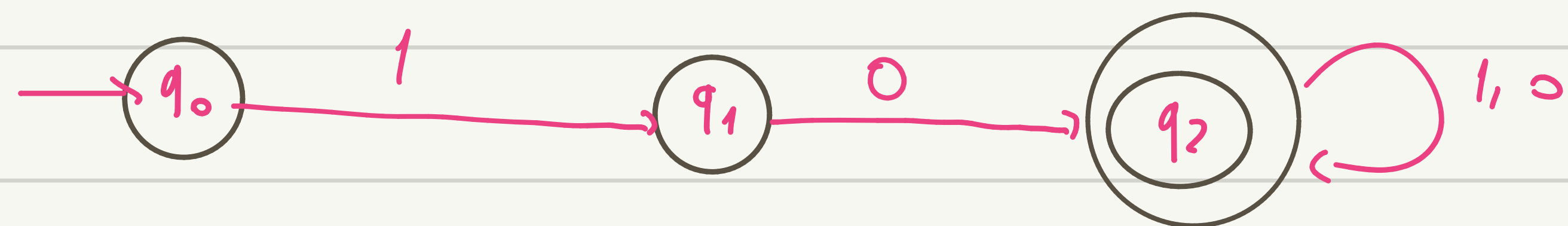
δ	0	1	ϵ
q_0	q_0, q_1	q_0	$\emptyset$
q_1	q_1	q_1	$\emptyset$

$L_2 = \{ \text{string not containing 0} \}$

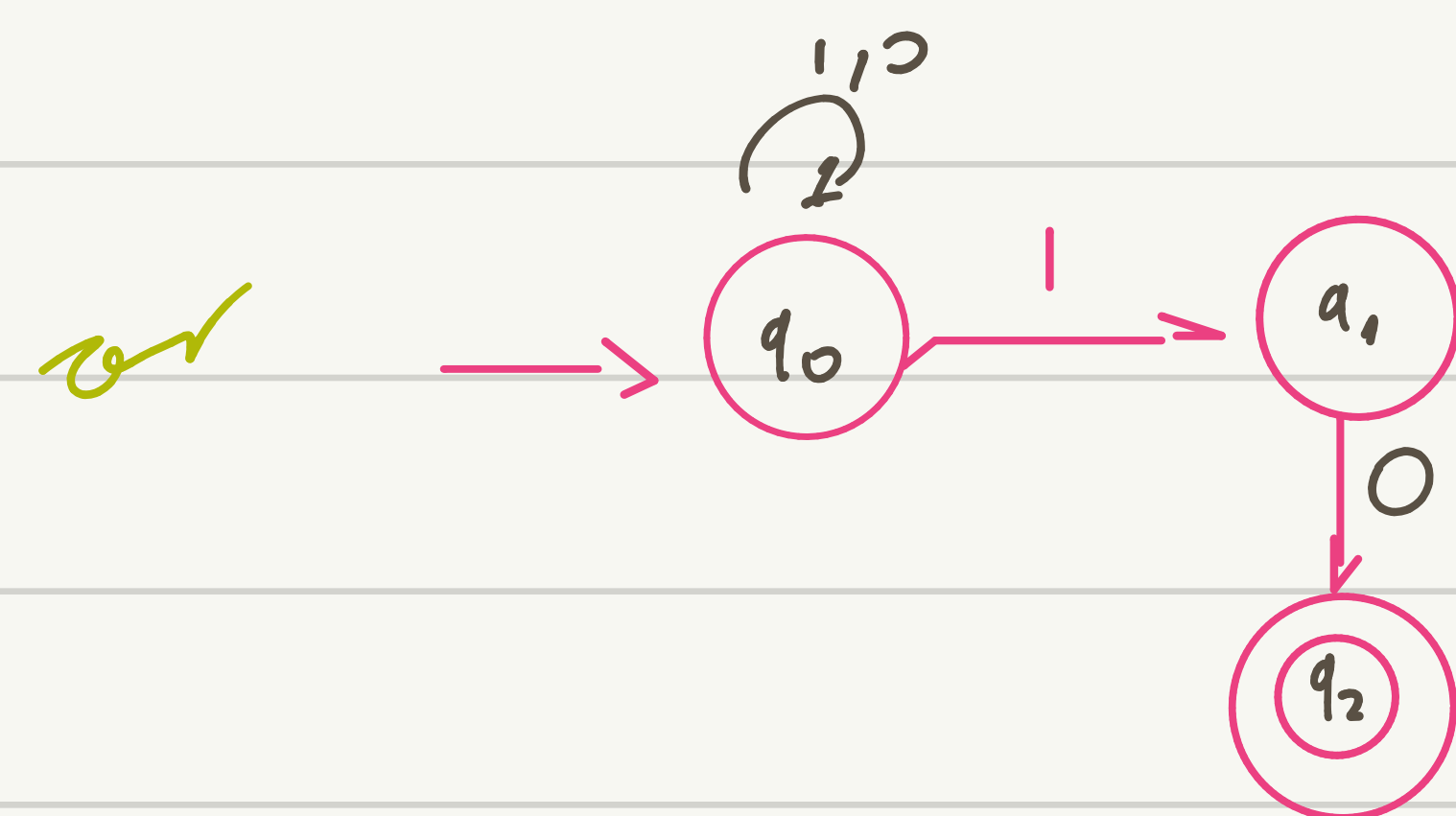
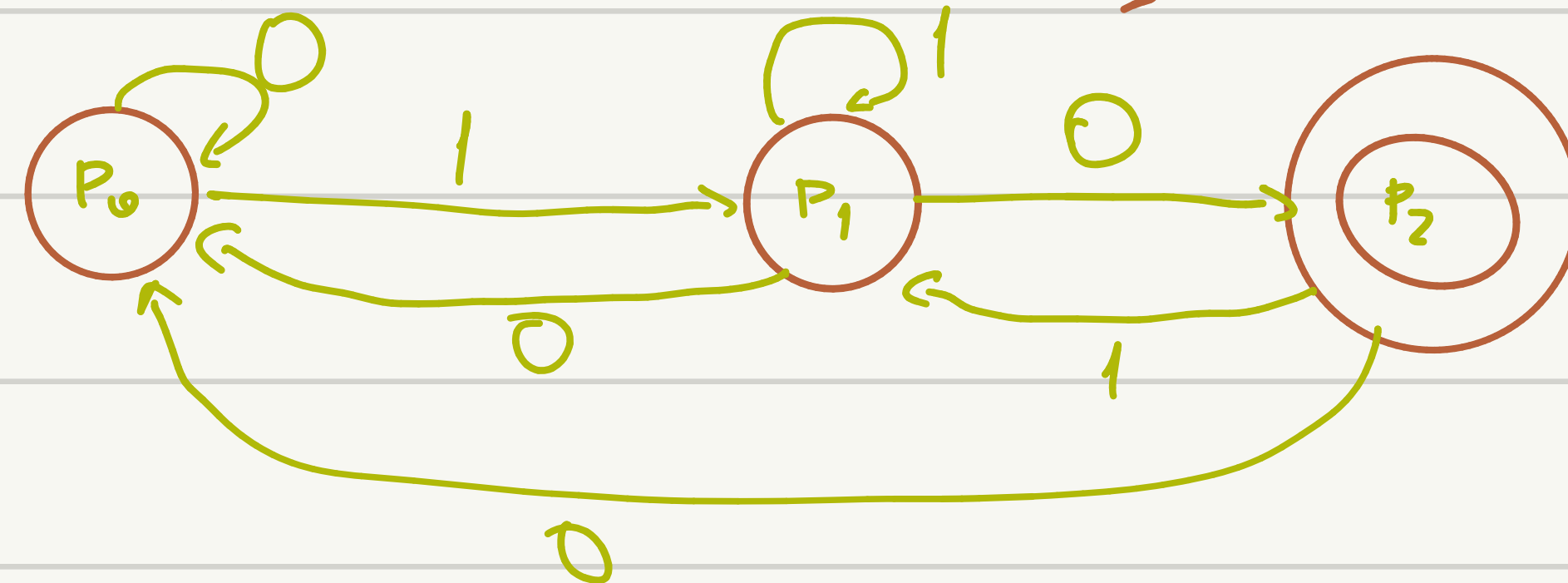


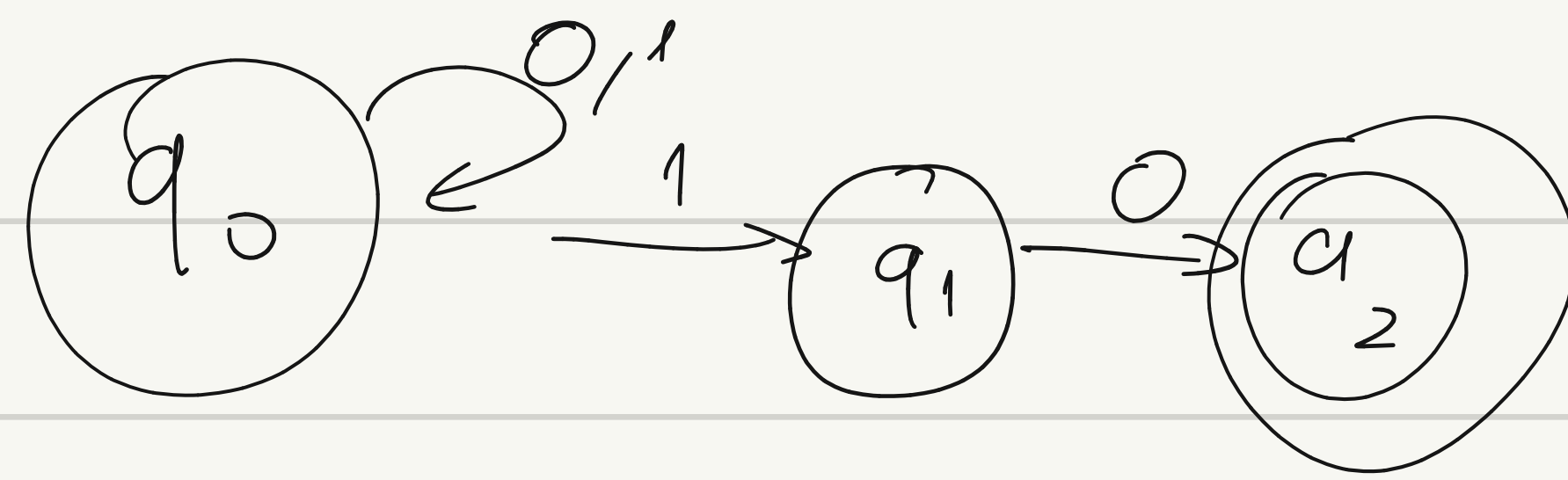
δ	0	1	ϵ
q_0	q_1	q_0	q_0
q_1	q_1	q_1	$\emptyset$

$L_3 = \{ \text{set of strings starts with 10} \}$

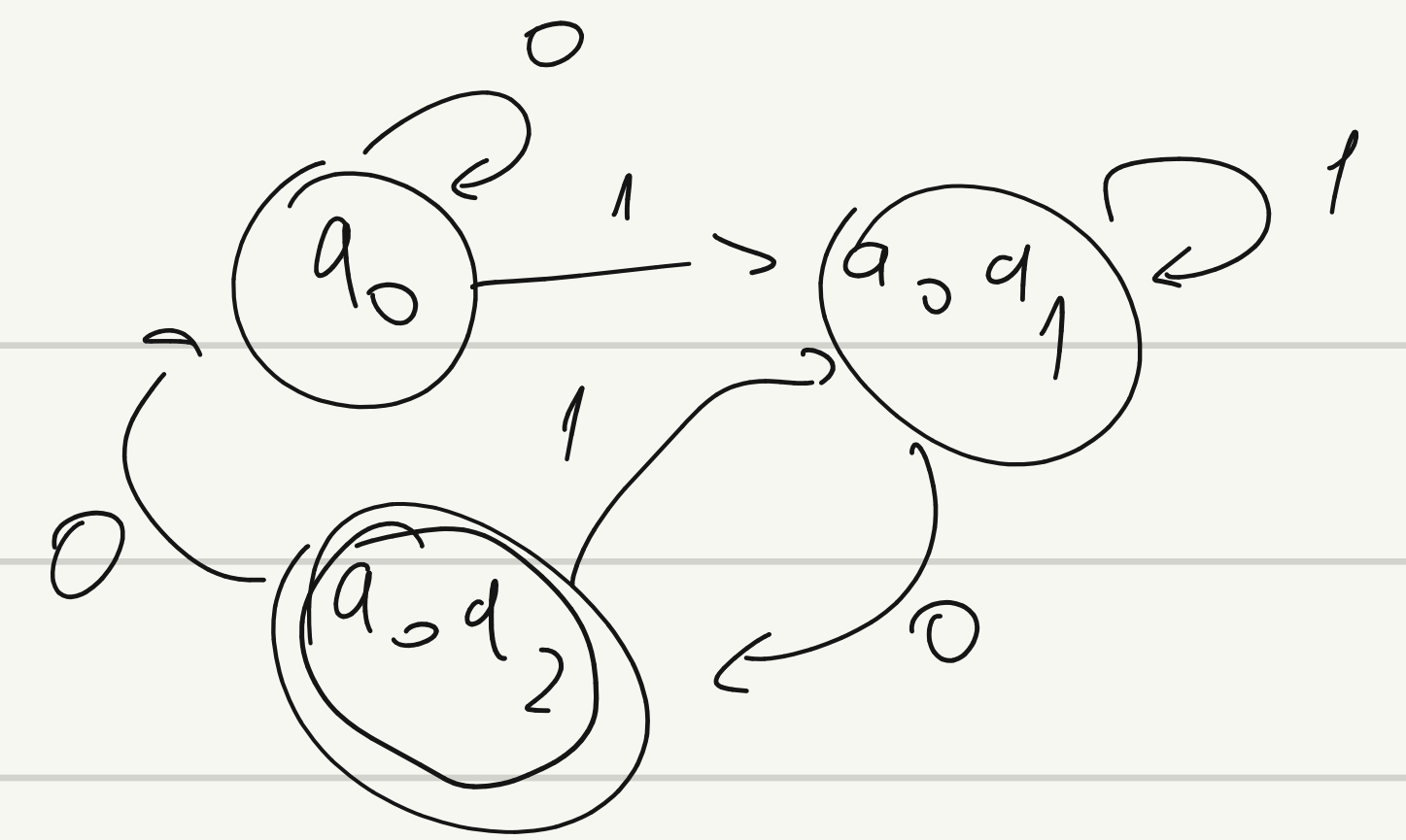


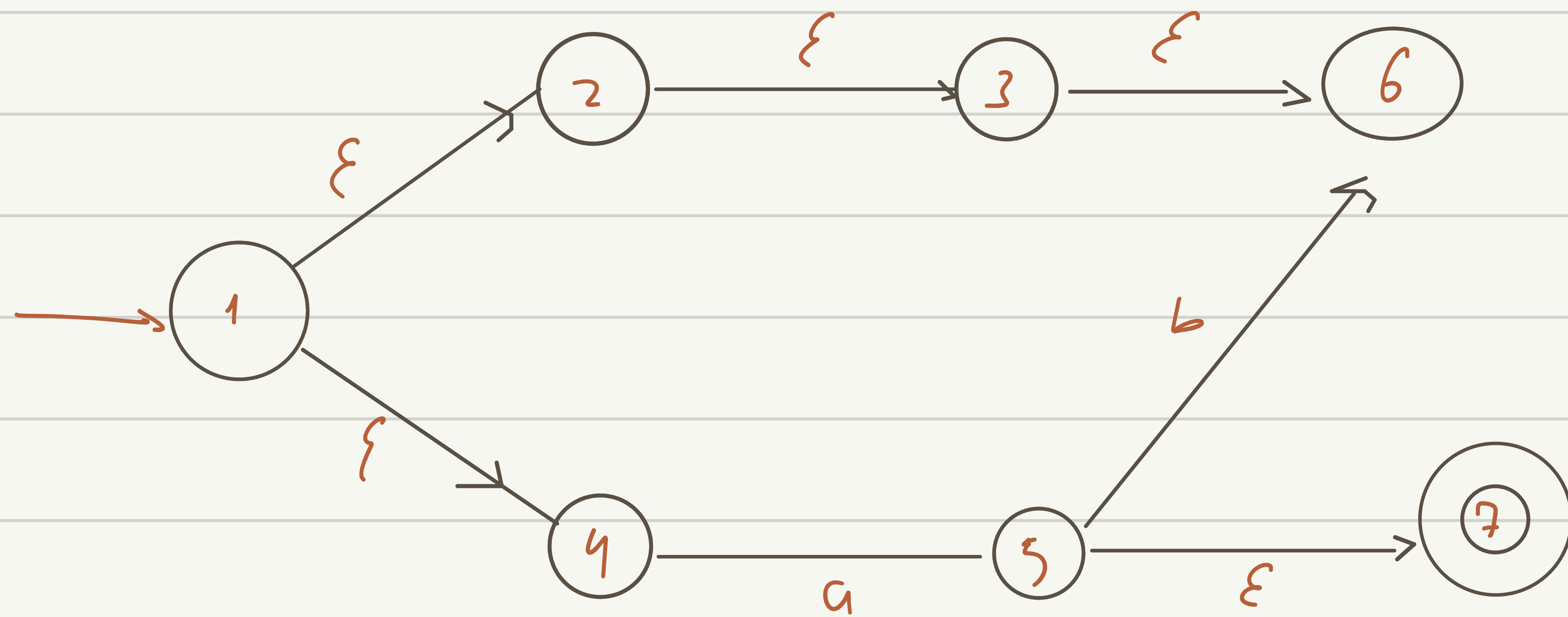
$L_4 = \{ \text{ends with 10} \}$





	0	1
q_0	q_0	$q_0 q_1$
$q_0 q_1$	$q_0 q_2$	$q_0 q_1$
$q_0 q_2$	q_0	$q_0 q_1$





$L = \{a, aa\}$

Lemma:

If we convert NFA M to DFA M'
then the two automata are equivalent:

$$L(M) = L(M')$$

Proof:

We only need to show: $L(M) \subseteq L(M')$

AND

$$L(M) \supseteq L(M') \quad \rangle$$

First we show: $L(M) \subseteq L(M')$

We only need to prove:

$$w \in L(M) \quad \longrightarrow \quad w \in L(M')$$



NFA

Consider $w \in L(M)$



symbols

$$w = \sigma_1 \sigma_2 \cdots \sigma_k$$



symbol



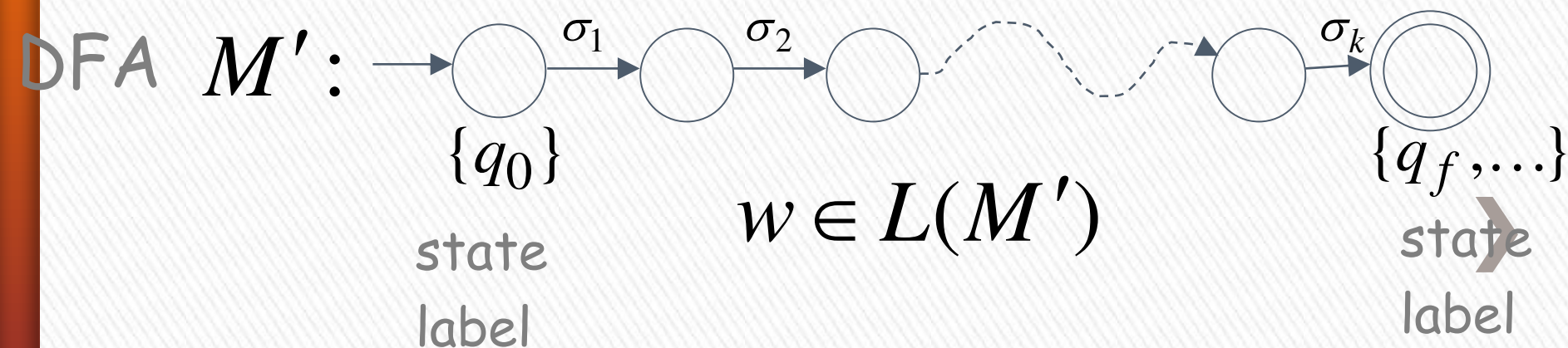
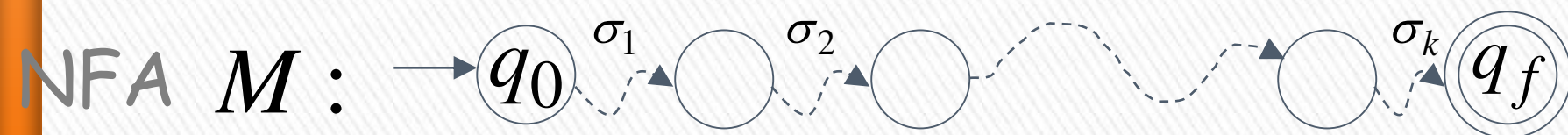
denotes a possible sub-path like

symbol



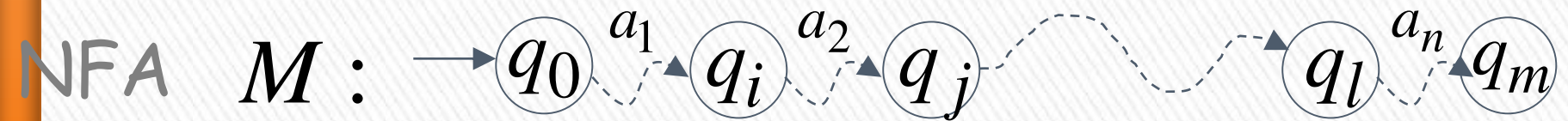
We will show that if $w \in L(M)$

$$w = \sigma_1 \sigma_2 \cdots \sigma_k$$

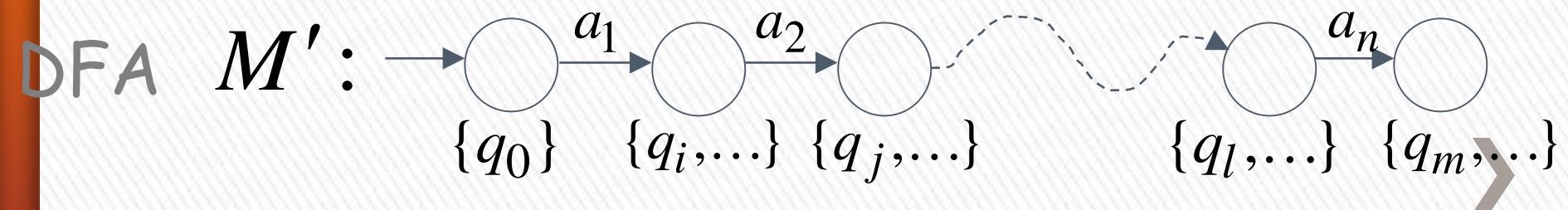


More generally, we will show that if in M :

(arbitrary string) $v = a_1 a_2 \cdots a_n$

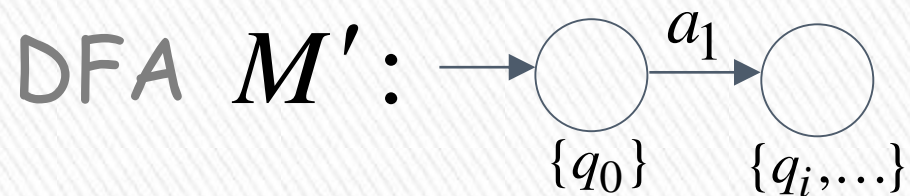


then



Proof by induction on $|v|$

Induction Basis: $|v| = 1$ $v = a_1$



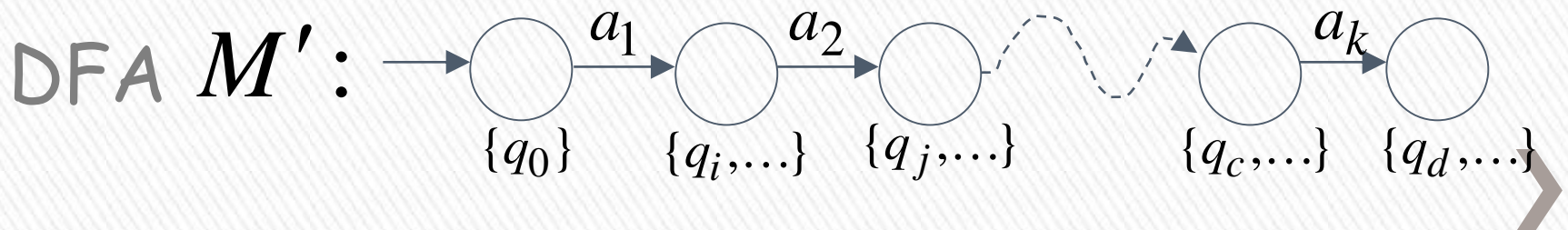
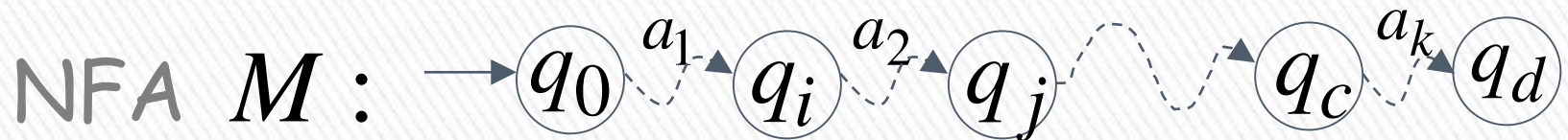
is true by construction of M'



Induction hypothesis: $1 \leq |v| \leq k$

$$v = a_1 a_2 \cdots a_k$$

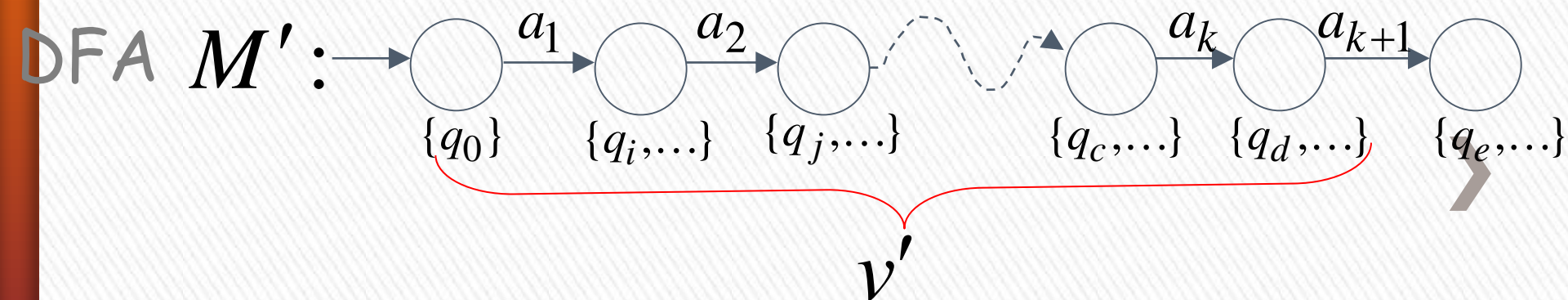
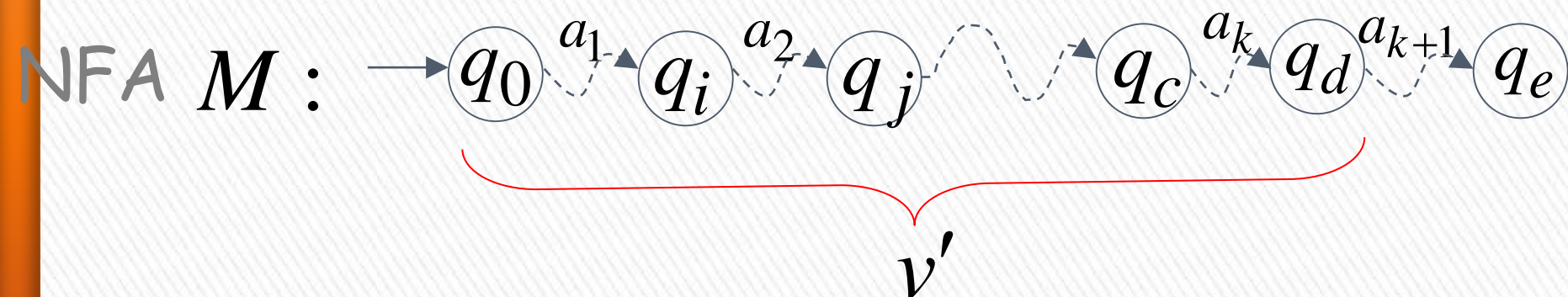
Suppose that the following hold



Induction Step: $|v| = k + 1$

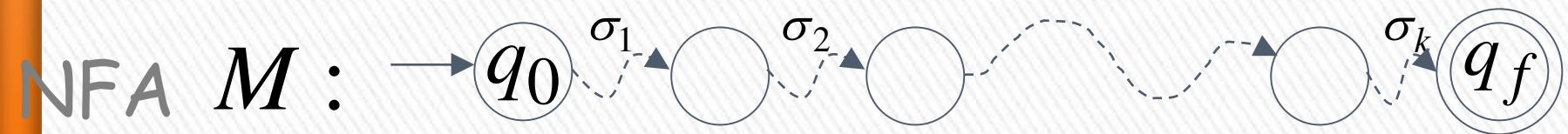
$$v = \underbrace{a_1 a_2 \cdots a_k}_{v'} a_{k+1} = v' a_{k+1}$$

Then this is true by construction of M'

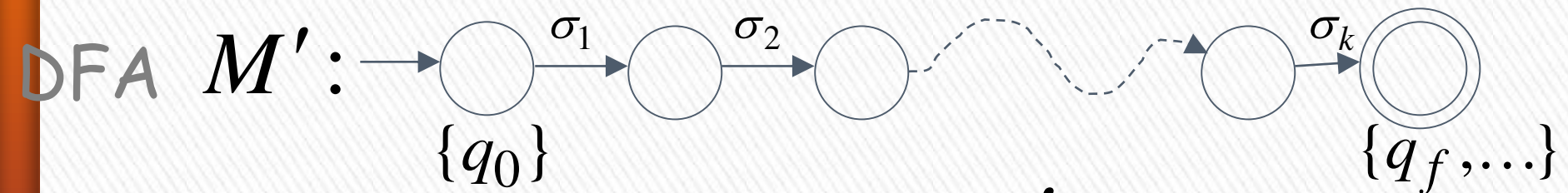


Therefore if $w \in L(M)$

$$w = \sigma_1 \sigma_2 \cdots \sigma_k$$



then



$$w \in L(M')$$



We have shown: $L(M) \subseteq L(M')$

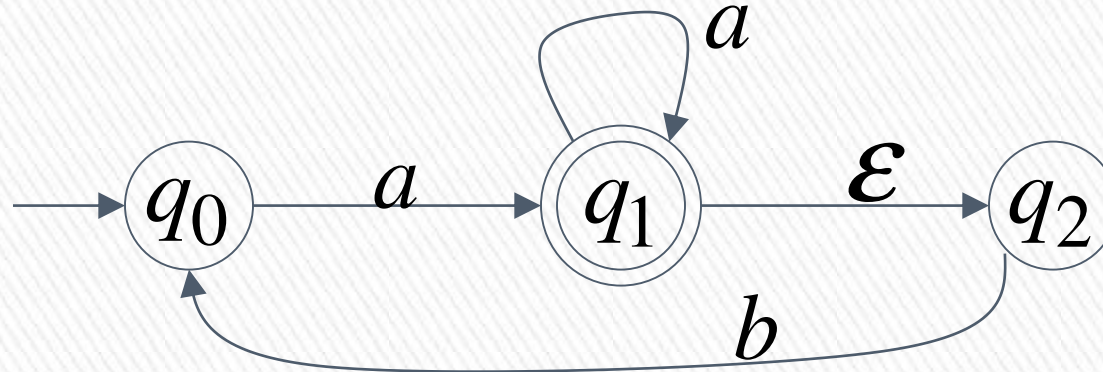
With a similar proof
we can show: $L(M) \supseteq L(M')$

Therefore: $L(M) = L(M')$

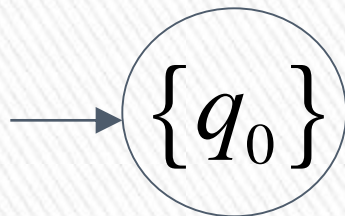


END OF LEMMA PROOF

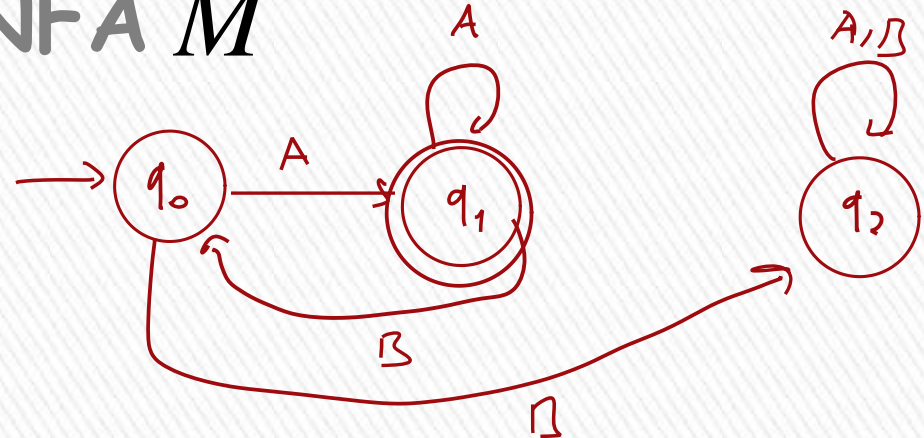
Conversion NFA to DFA



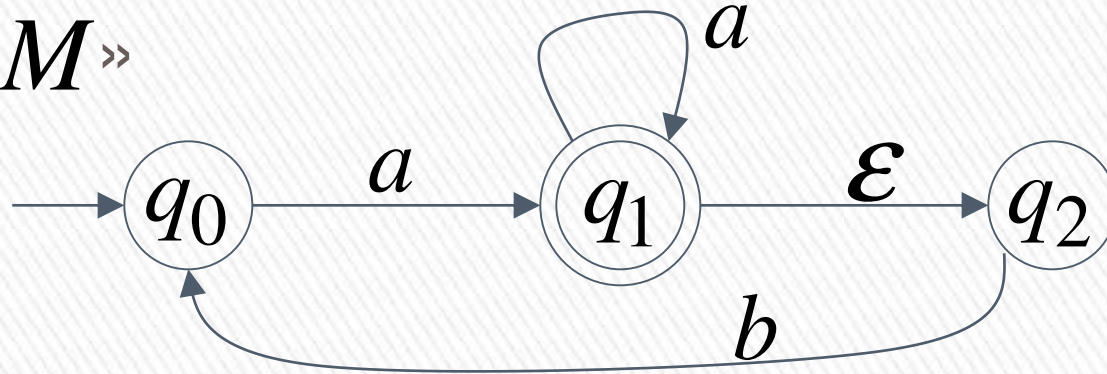
NFA M



DFA M'

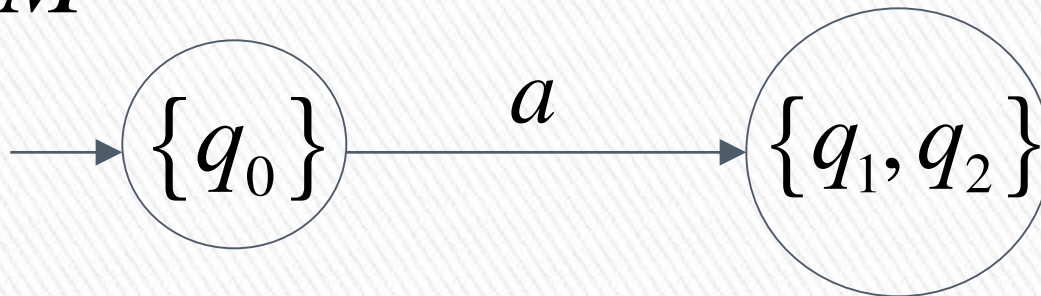


NFA $M \gg$



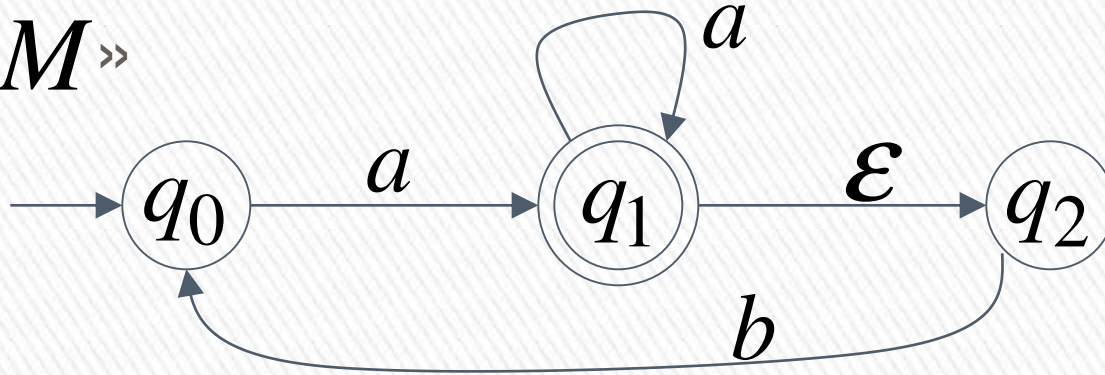
$$\delta^*(q_0, a) = \{q_1, q_2\}$$

DFA M'

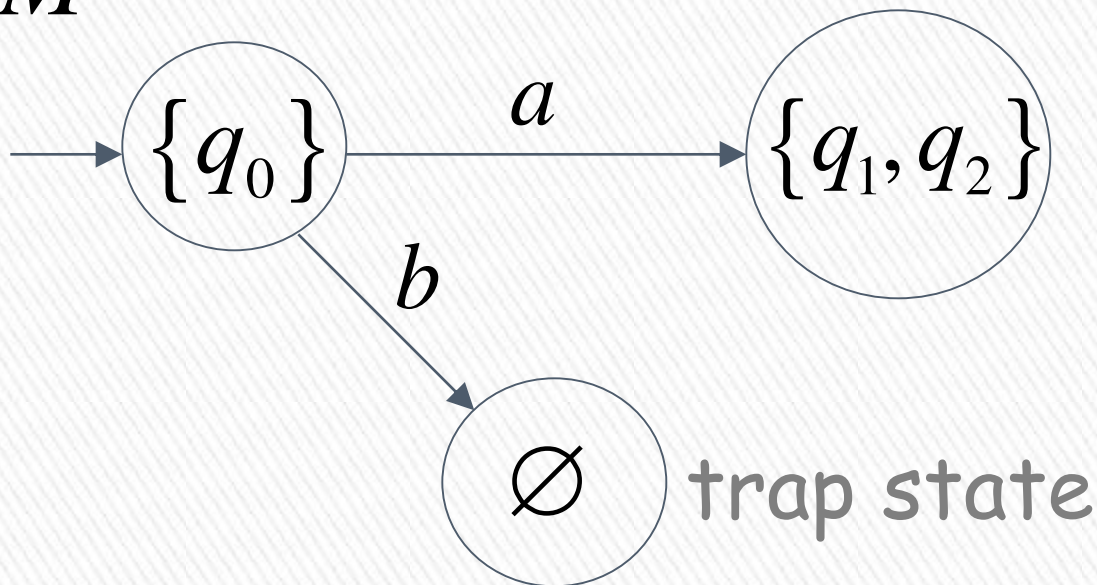


$$\delta^*(q_0, b) = \emptyset \quad \text{empty set}$$

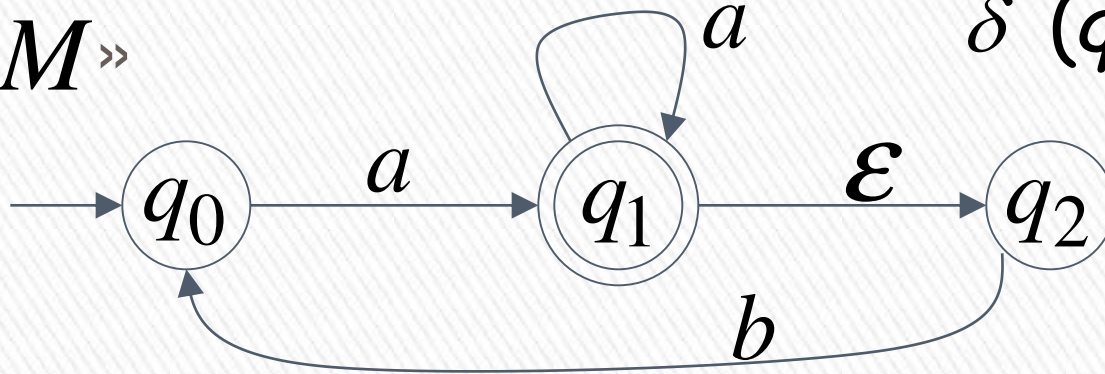
NFA M



DFA M'



NFA $M \gg$



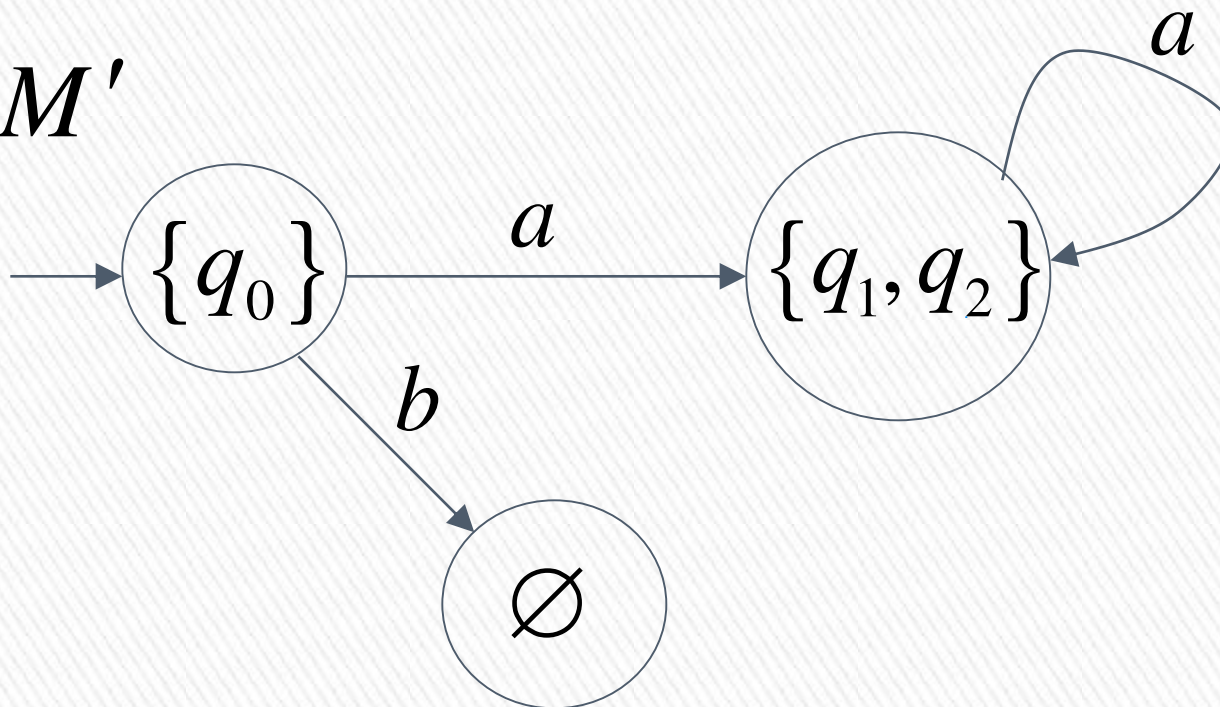
$$\delta^*(q_1, a) = \{q_1, q_2\}$$

$$\delta^*(q_2, a) = \emptyset$$

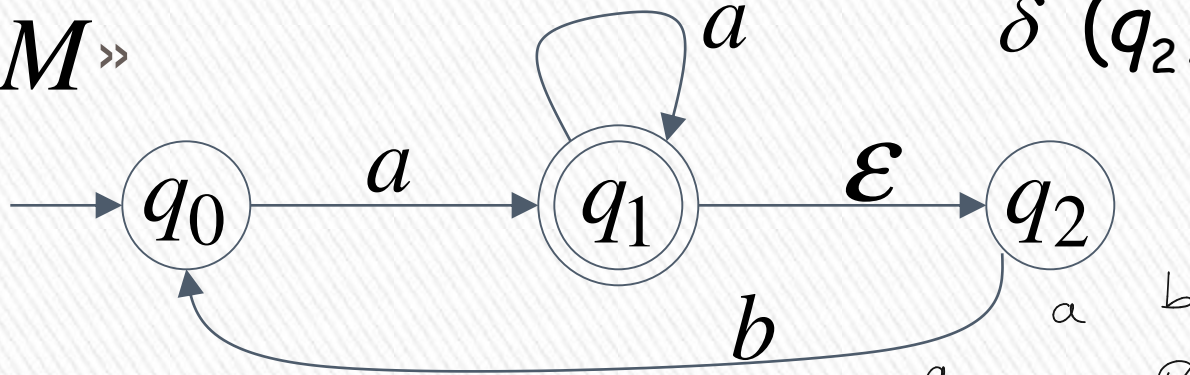
union

$$\{q_1, q_2\}$$

DFA M'



NFA $M \gg$



$$\delta^*(q_1, b) = \{q_0\}$$

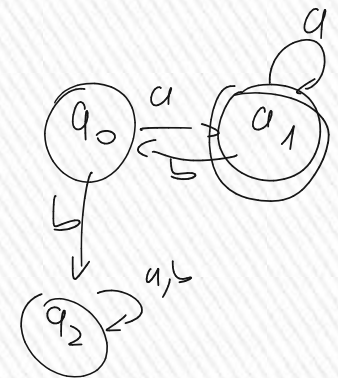
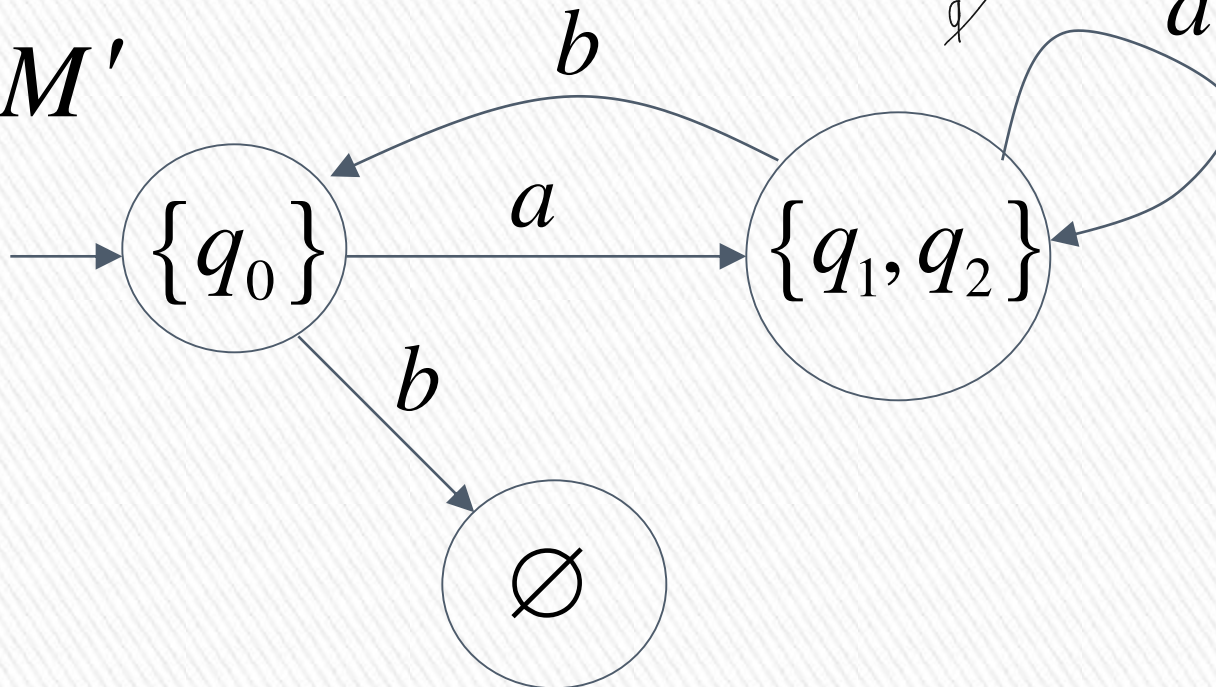
$$\delta^*(q_2, b) = \{q_0\}$$

union

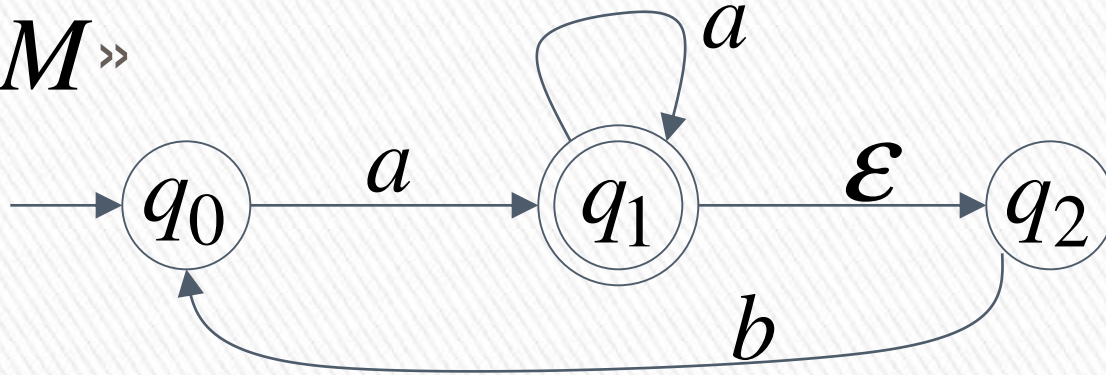
$\{q_0\}$

q_0	q_1	$\emptyset$
q_1	q_1	q_0
q_2		a

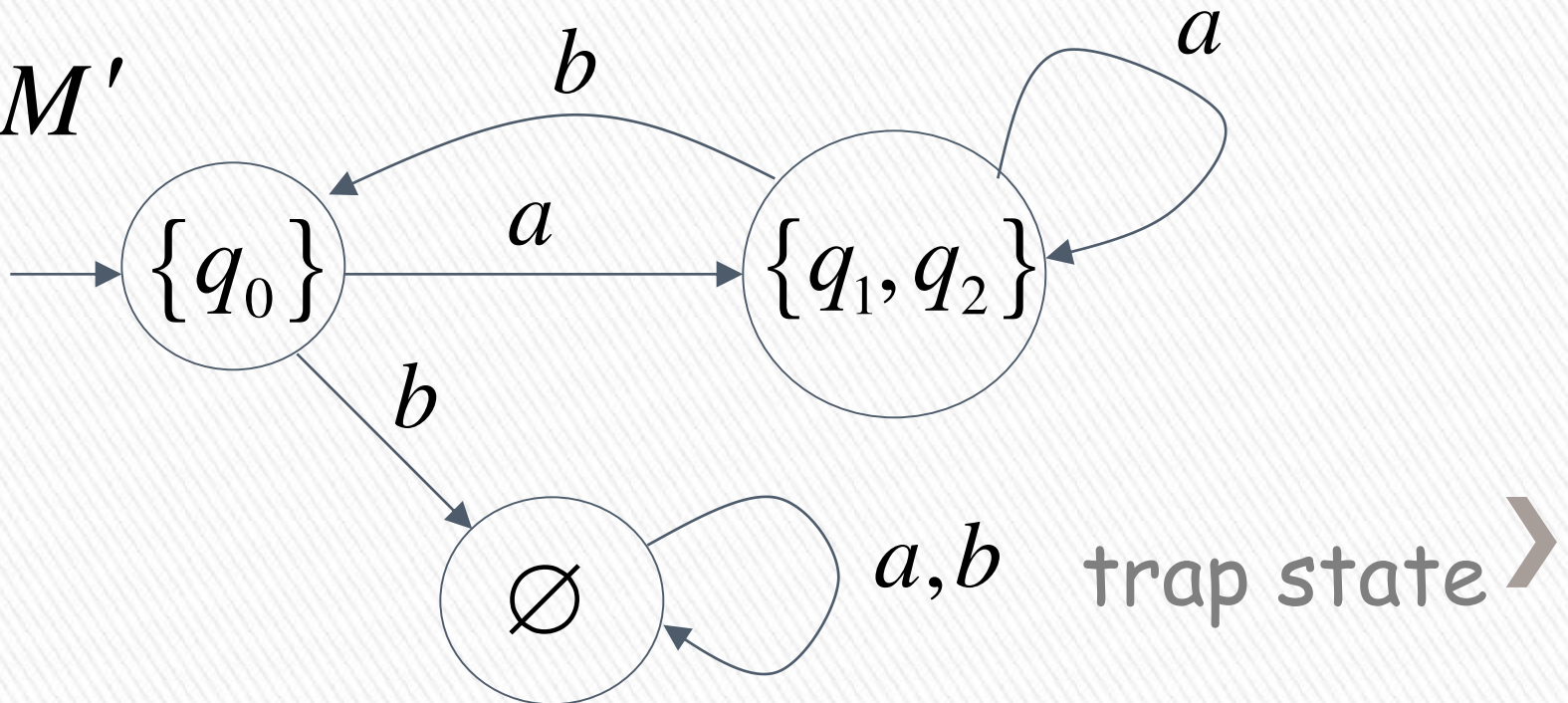
DFA M'



NFA $M_{\gg}$

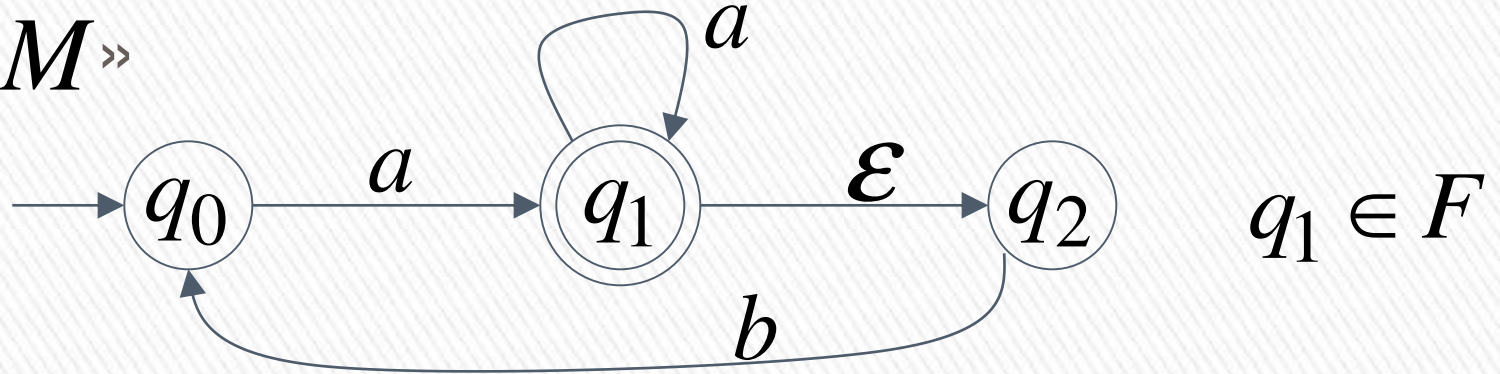


DFA M'

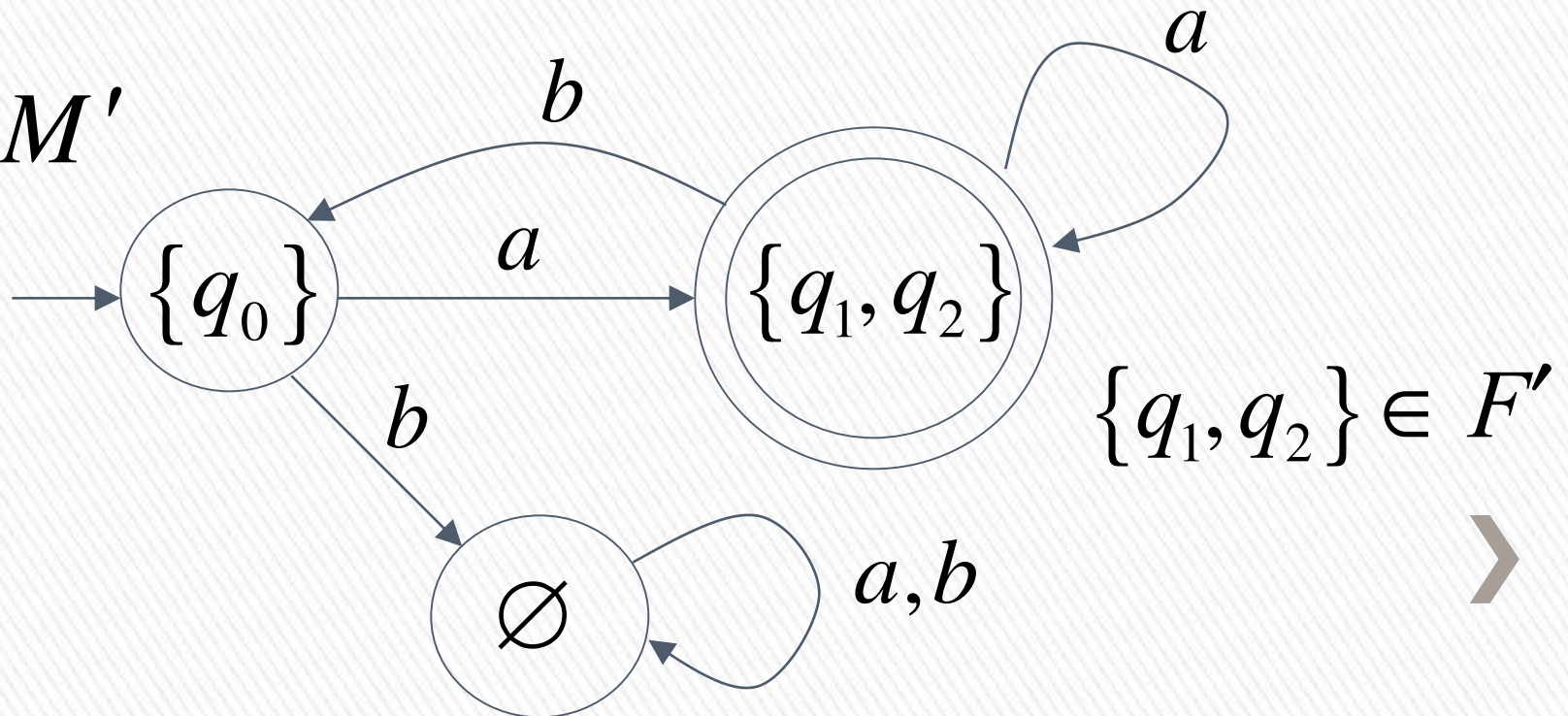


END OF CONSTRUCTION

NFA $M_{\gg}$



DFA M'



General Conversion Procedure

» Input: an NFA M

» Output: an equivalent DFA M'
with $L(M) = L(M')$



» The NFA has states

$$q_0, q_1, q_2, \dots$$

» The DFA has states from the power set

$$\emptyset, \{q_0\}, \{q_1\}, \{q_0, q_1\}, \{q_1, q_2, q_3\}, \dots$$



Conversion Procedure Steps

»

Step 1

» Initial state of NFA: q_0

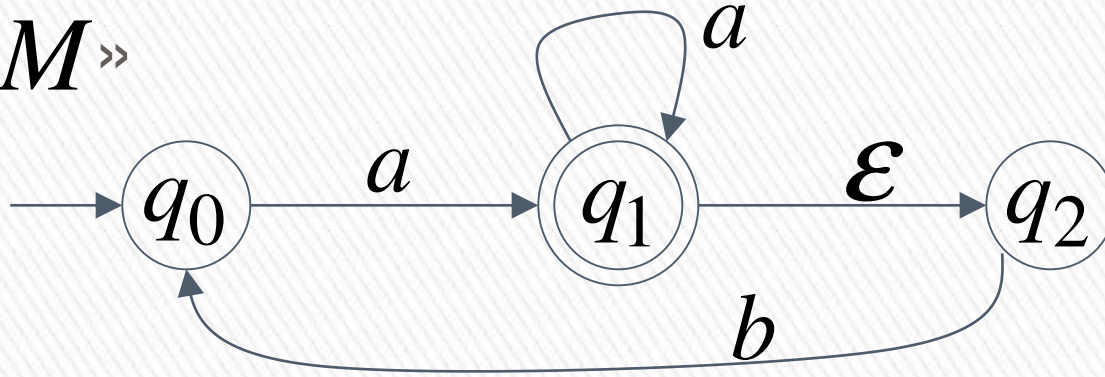


» Initial state of DFA: $\{q_0\}$

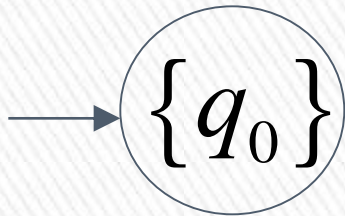


Example

NFA M



DFA M'



Step 2

For every DFA's state $\{q_i, q_j, \dots, q_m\}$

compute in the NFA

$$\left. \begin{array}{l} \delta^*(q_i, a) \\ \cup \delta^*(q_j, a) \\ \dots \\ \cup \delta^*(q_m, a) \end{array} \right\} = \text{Union } \{q'_k, q'_l, \dots, q'_n\}$$

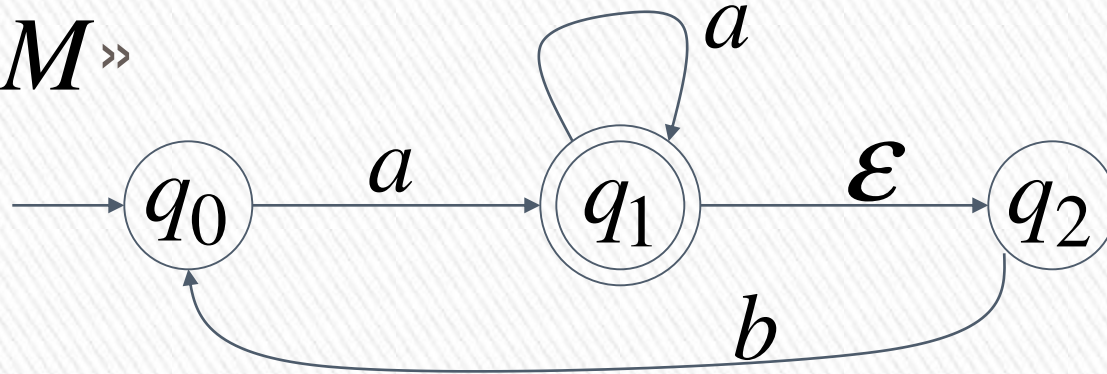
add transition to DFA

$$\delta(\{q_i, q_j, \dots, q_m\}, a) = \{q'_k, q'_l, \dots, q'_n\}$$

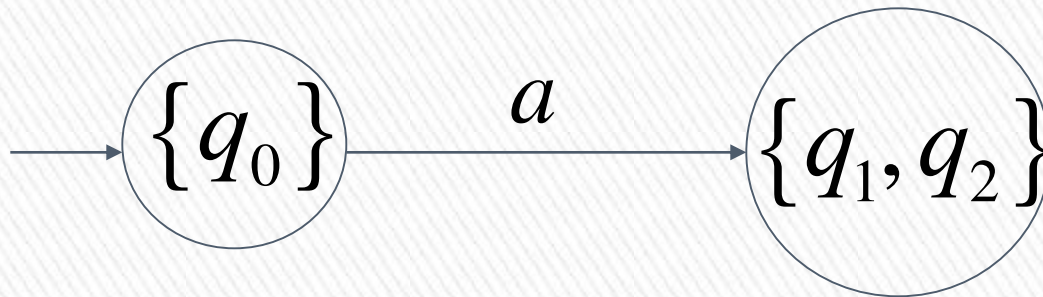


Example $\delta^*(q_0, a) = \{q_1, q_2\}$

NFA $M \gg$



DFA M' $\delta(\{q_0\}, a) = \{q_1, q_2\}$



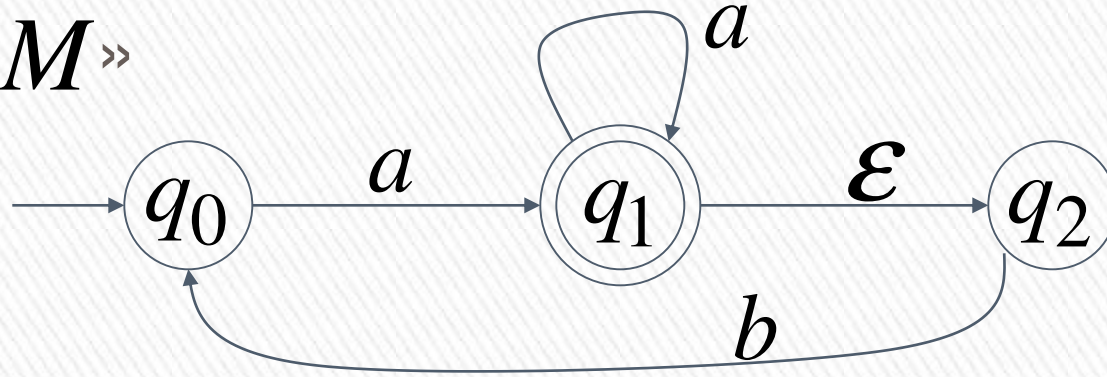
Step 3

- » Repeat Step 2 for every state in DFA and symbols in alphabet until no more states can be added in the DFA

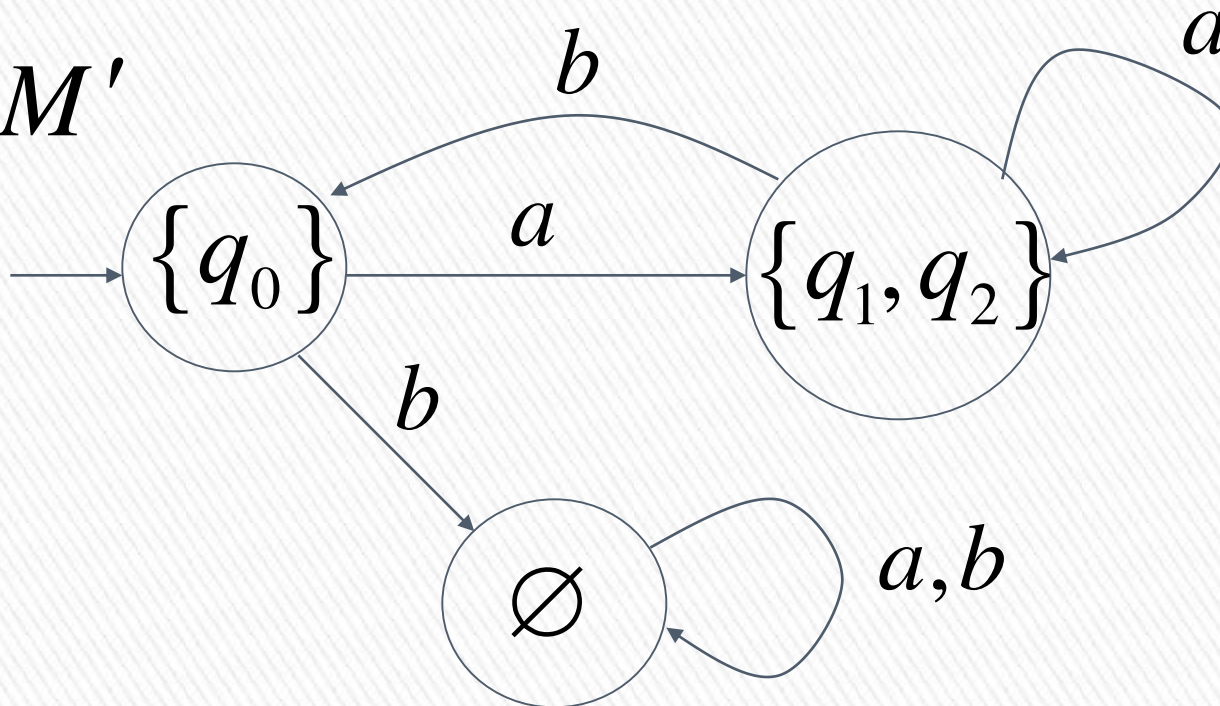


Example

NFA M



DFA M'



Step 4

» For any DFA state $\{q_i, q_j, \dots, q_m\}$

» if some q_j is accepting state in NFA

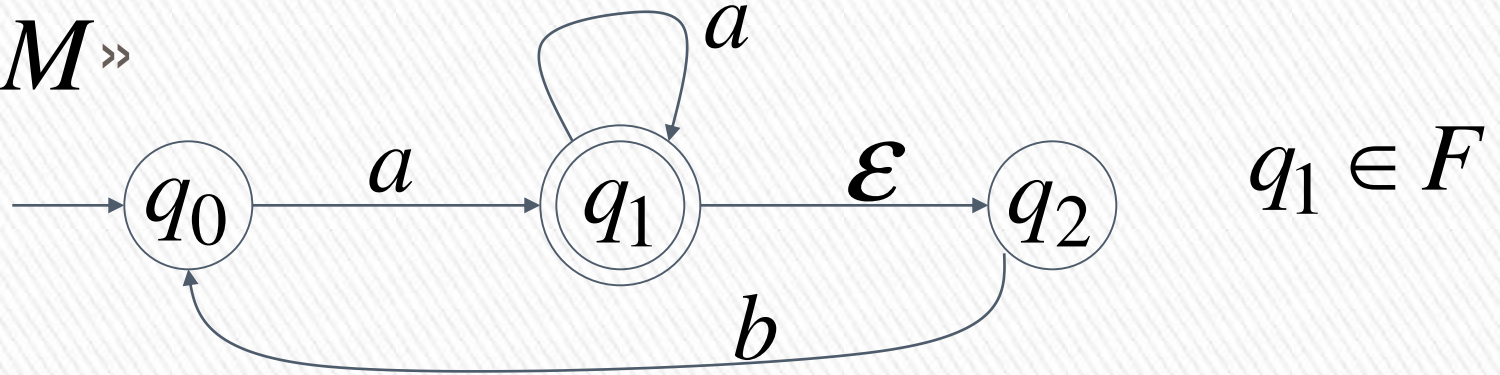
» Then, $\{q_i, q_j, \dots, q_m\}$
is accepting state in DFA

↳: qinda accept state olonlon
hepsini final state yur ~~***~~

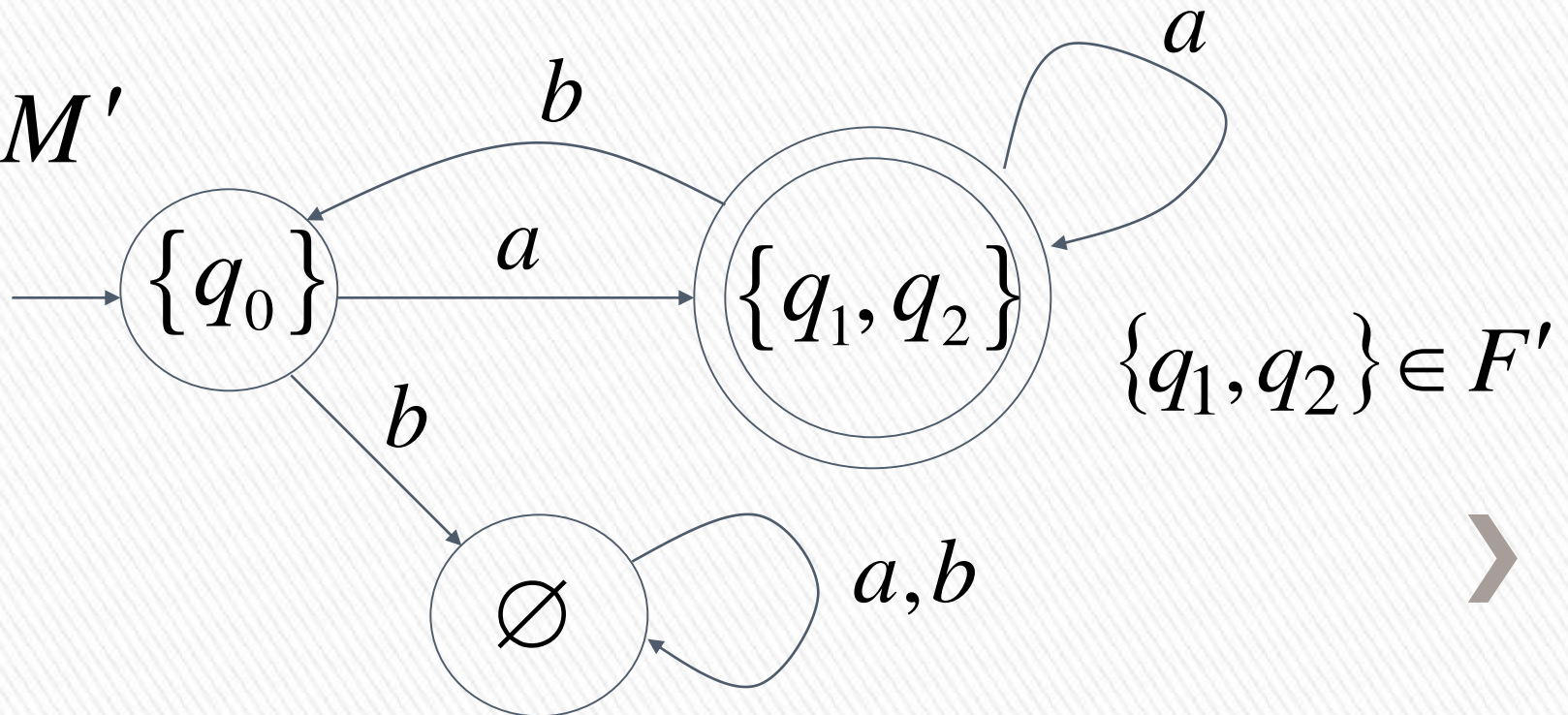


Example

NFA $M \gg$

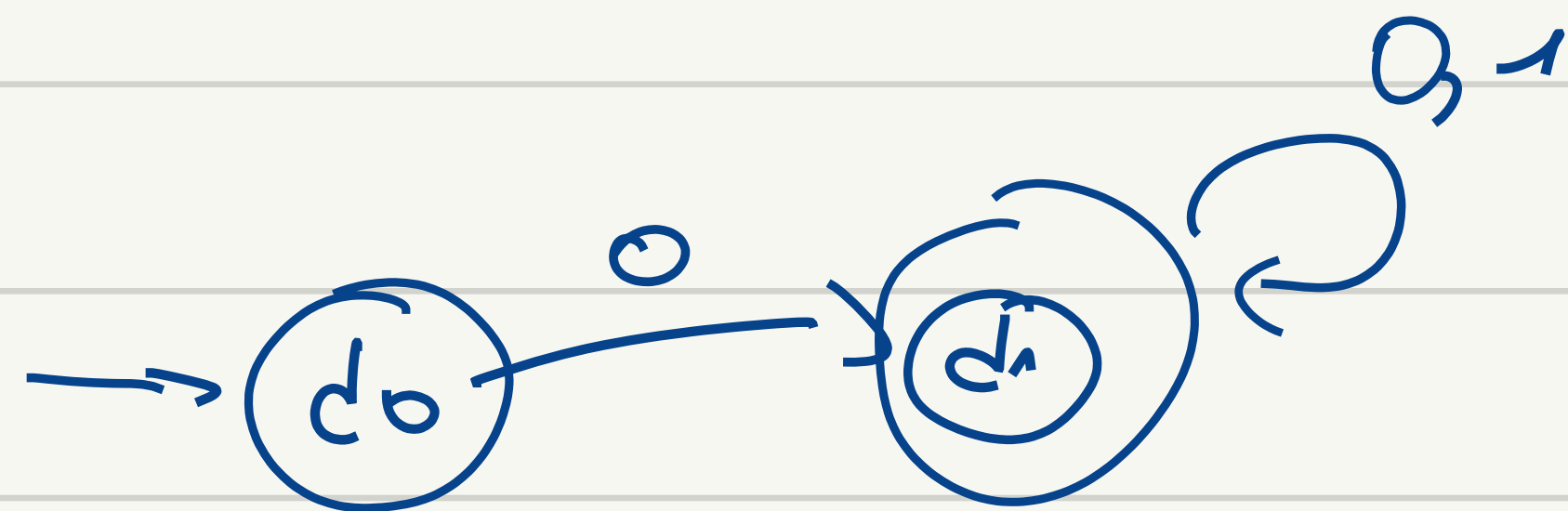


DFA M'



- An NFA accepts all strings over the alphabet $\Sigma = \{0, 1\}$ that begin with 0. Draw nfa, dfa

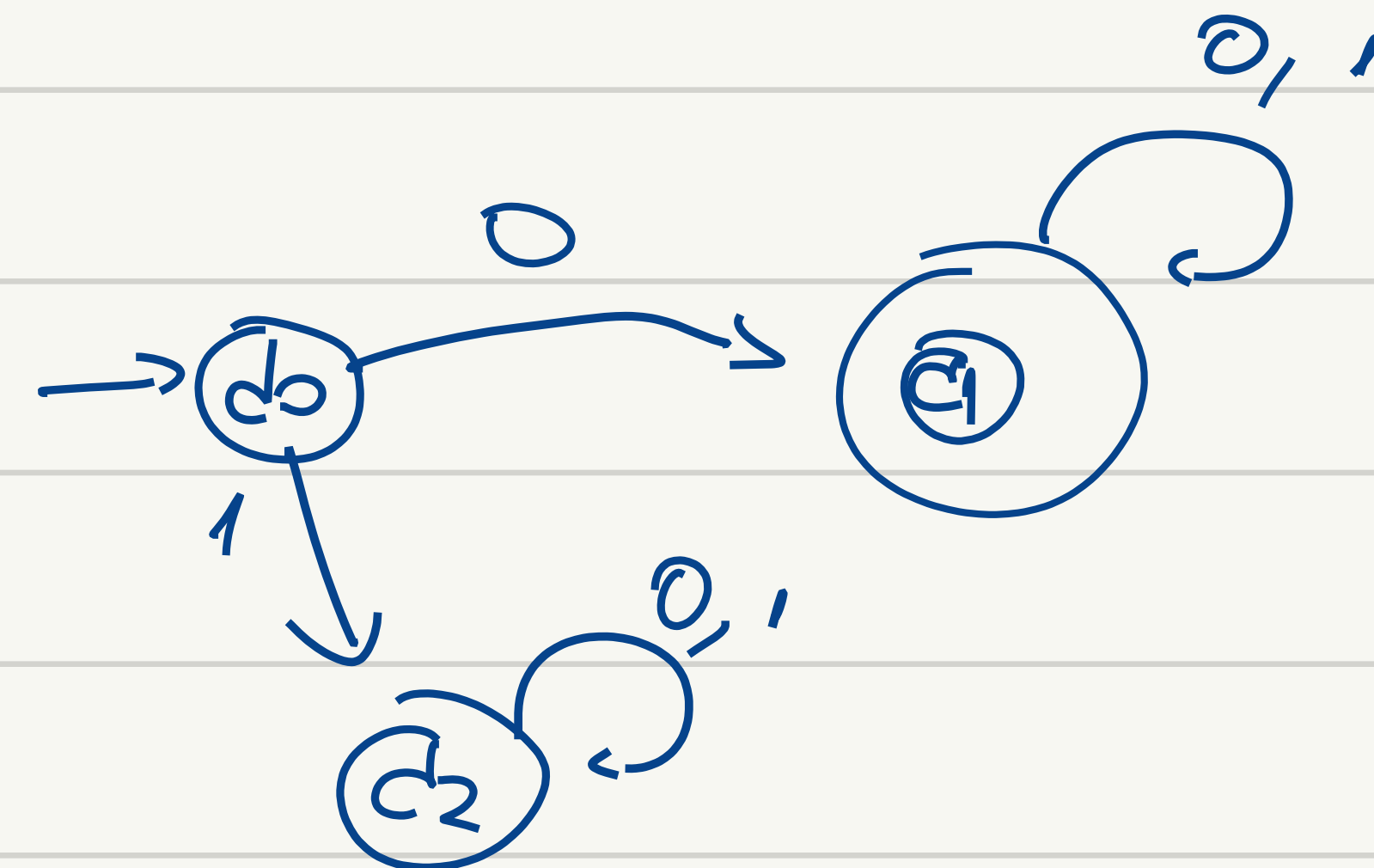
NFA



	0	1
q ₀	q ₁	∅
q ₁	q ₁	q ₁

	0	1
q ₀	q ₁	q ₂
q ₂	q ₂	q ₂
q ₁	q ₁	q ₁

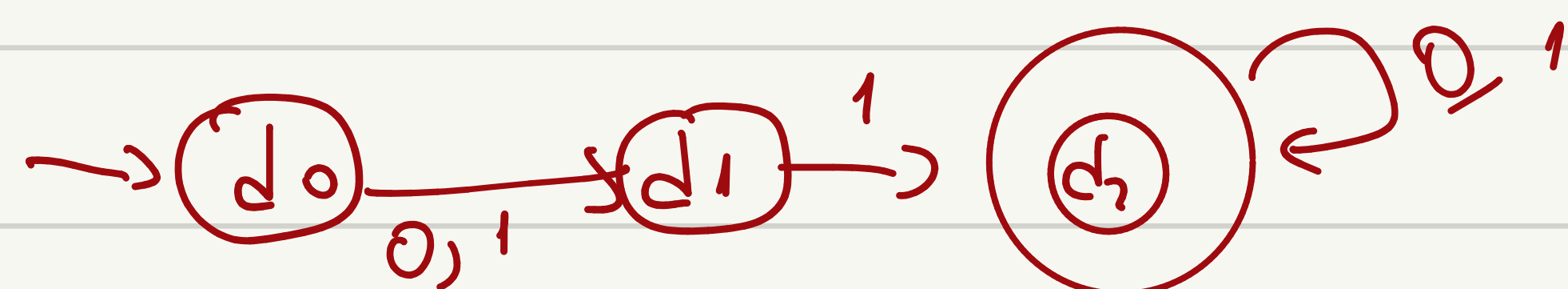
DFA



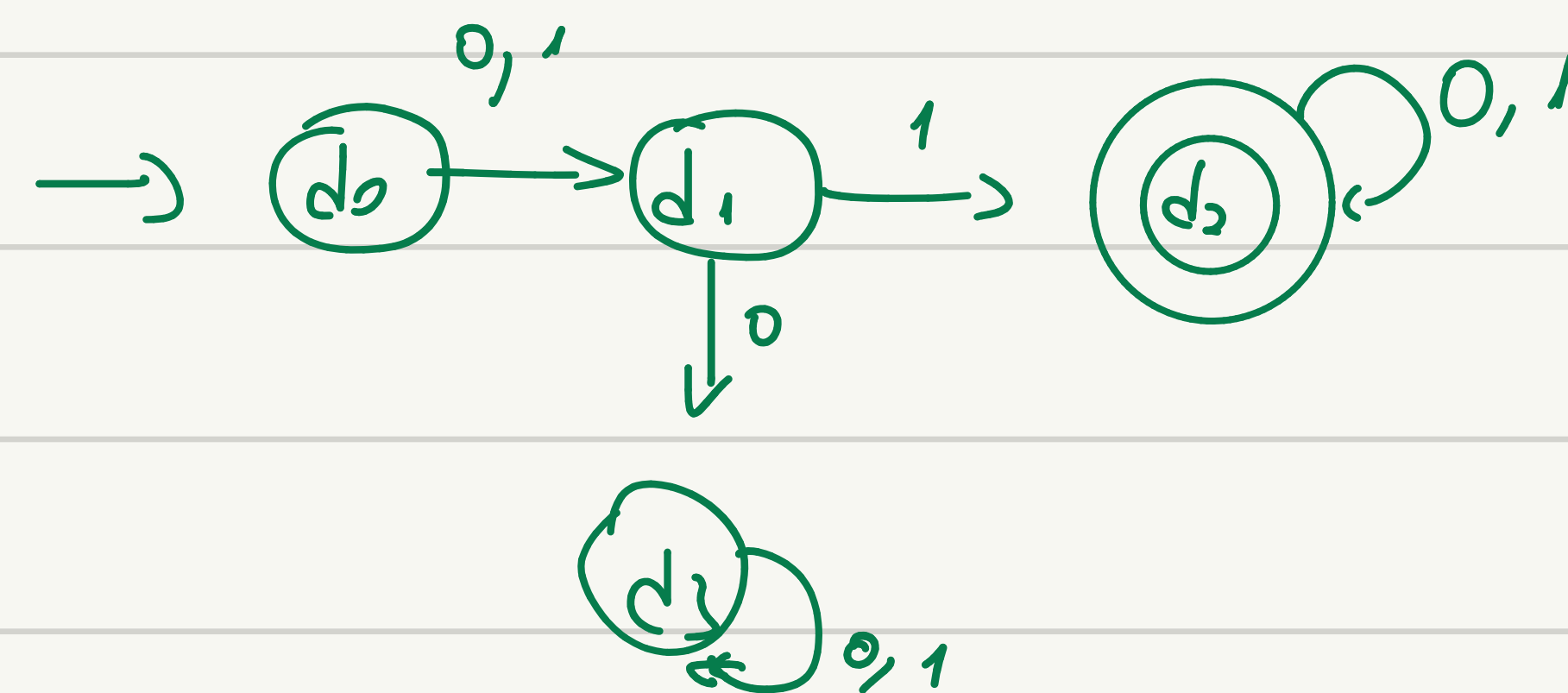
	0	1
q ₀	q ₁	q ₂
q ₁	q ₁	q ₁
q ₂	q ₂	q ₂

- Design an NFA for the language that accept strings over $\{0,1\}$ second symbol 2.

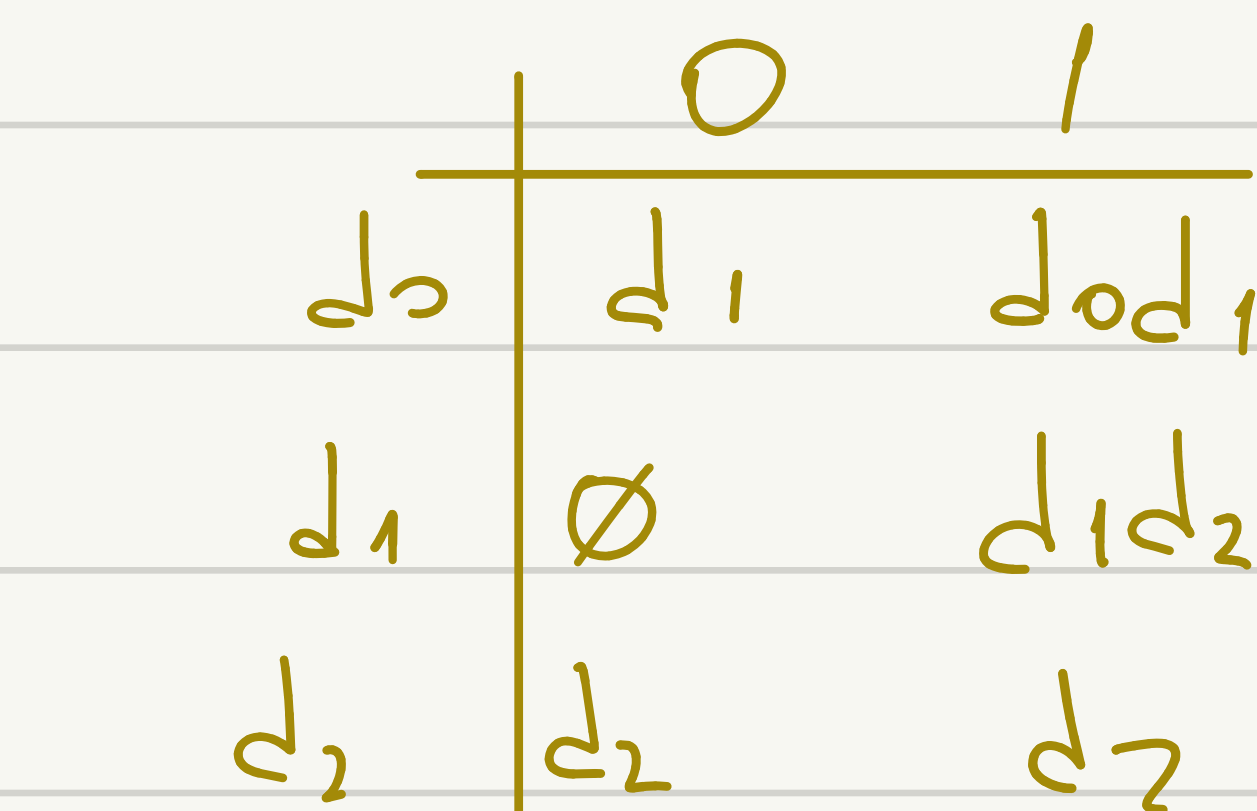
NFA



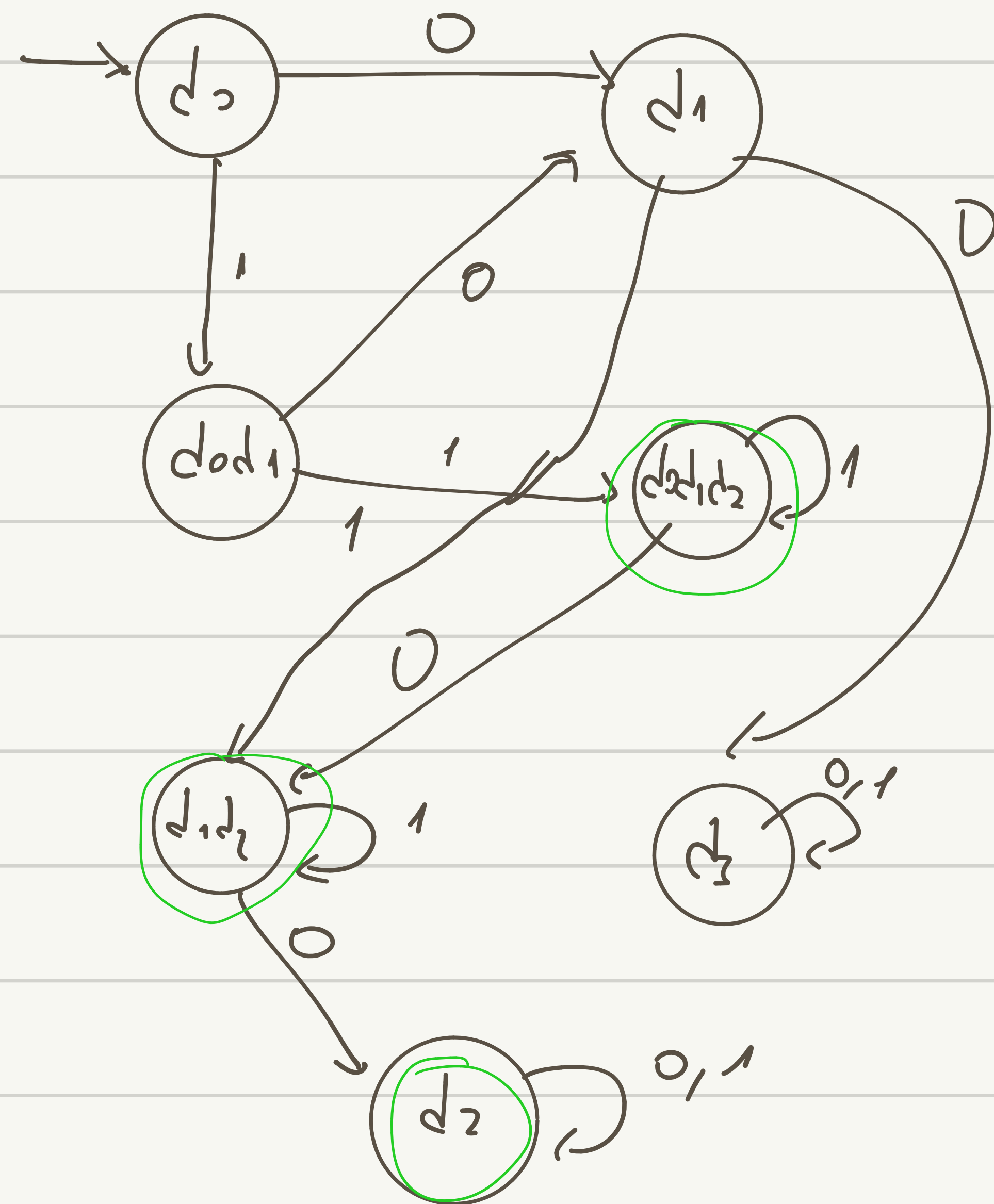
	0	1
d ₀	d ₁	d ₁
d ₁	∅	d ₂
d ₂	d ₂	d ₂



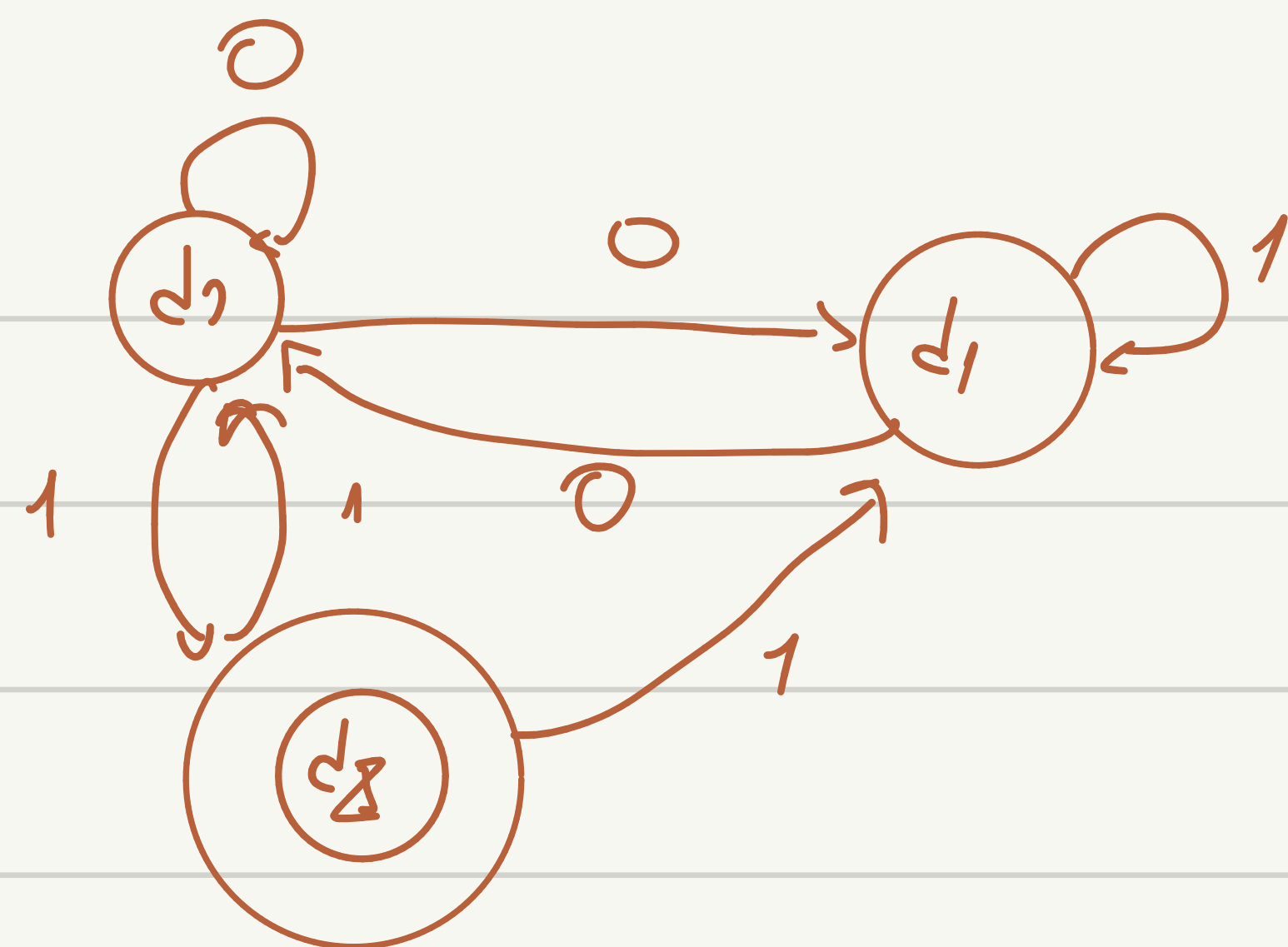
	0	1
d ₀	d ₁	d ₁
d ₁	d ₃	d ₂
d ₂	d ₂	d ₂
d ₃	d ₃	d ₂



	0	1
d ₀	d ₁	d' ₀ d ₁
d ₀ d ₁	d ₁	d ₀ d ₁ d ₂
d ₁	d ₃	d ₁ d ₂
d ₁ d ₂	d ₂	d' ₁ d ₂
d ₀ d ₁ d ₂	d ₁ d ₂	d ₀ d ₁ d ₂
d ₂	d ₂	d ₂
d ₃	d ₃	d ₃

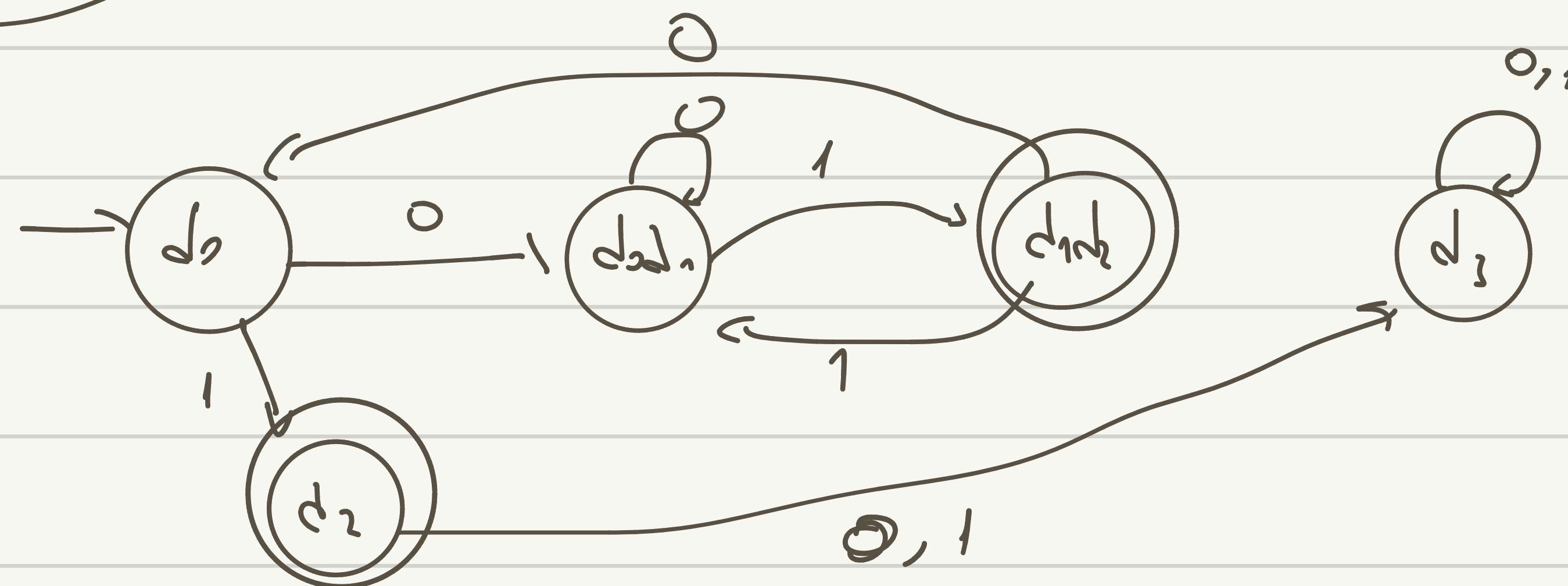


	0	1
d_0	$d_0 d_1$	d_2
d_1	d_0	d_1
d_2	$\emptyset$	$d_0 d_1$

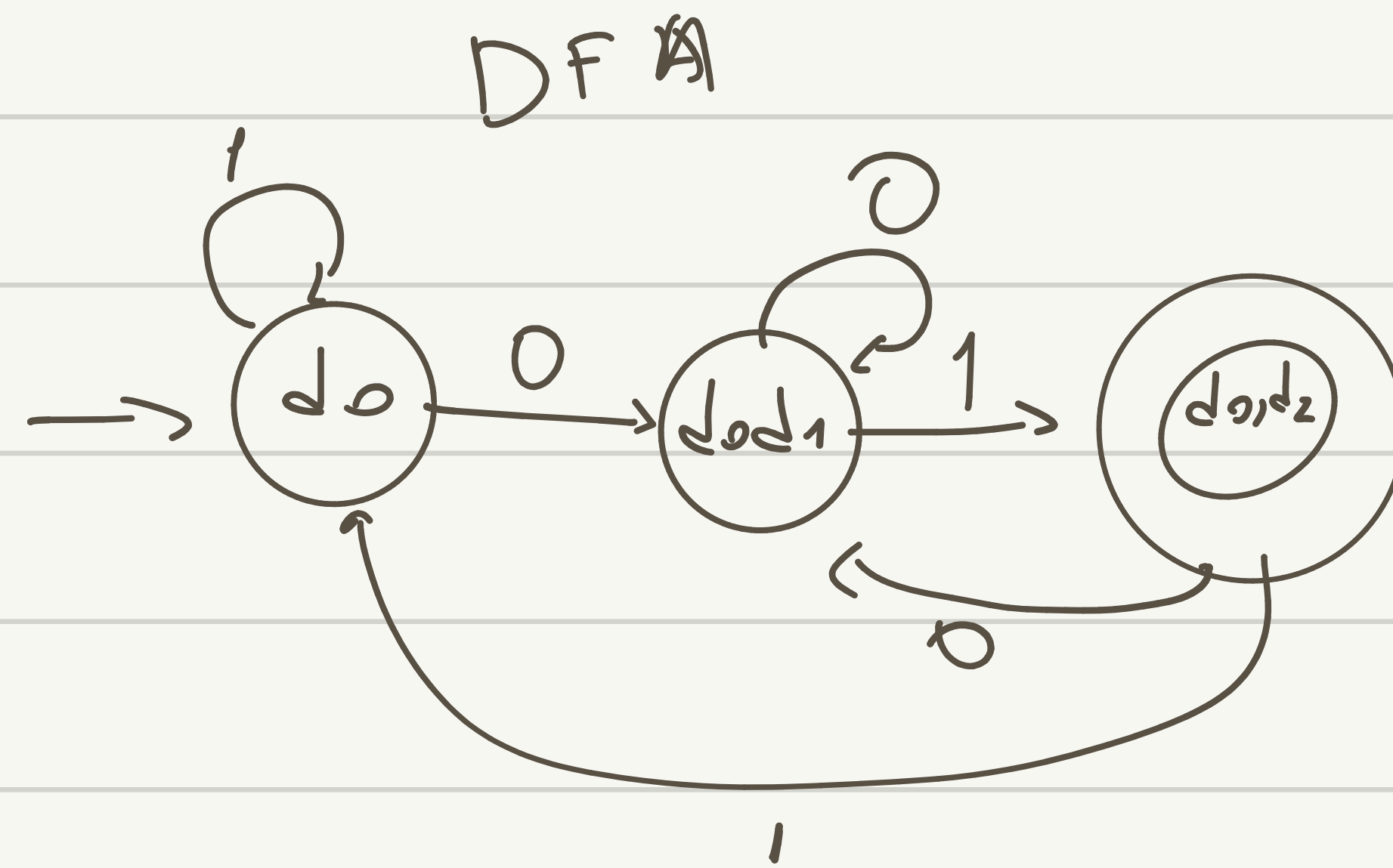
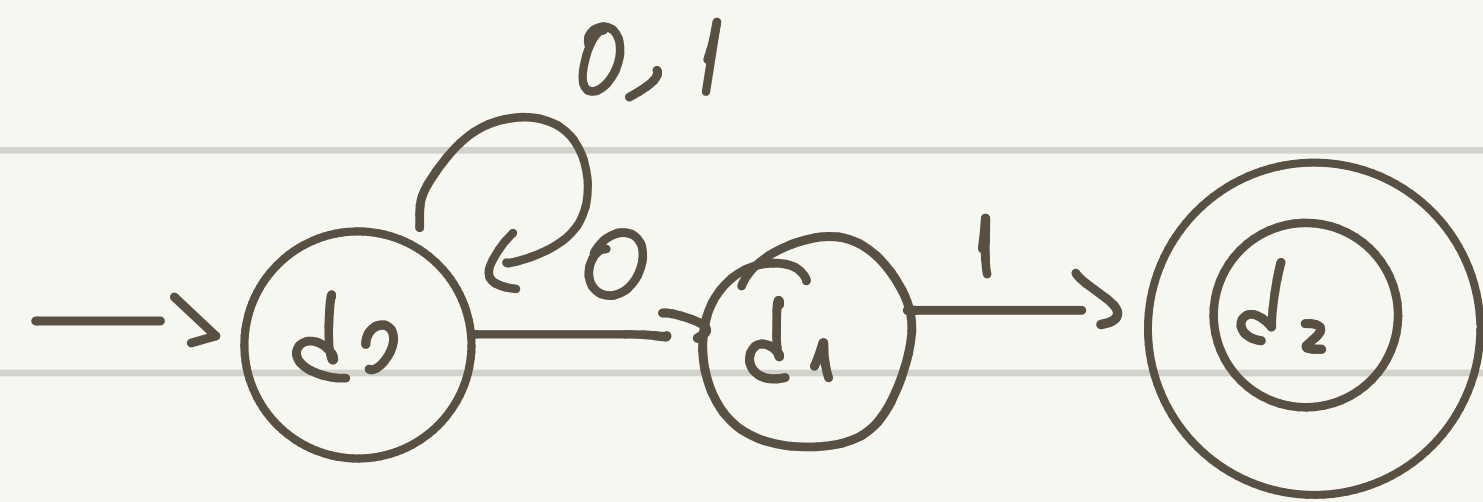


	0	1
d_0	$d_0 d_1$	$\overline{d_2}$
$d_0 d_1$	$d_0 d_1$	$d_1 d_2 \leftarrow \{d_0, 1\} \cup \{1, 1\}$
$d_1 d_2$	d_0	$d_0 d_1$
d_2	d_3	d_3
d_3	d_3	d_3

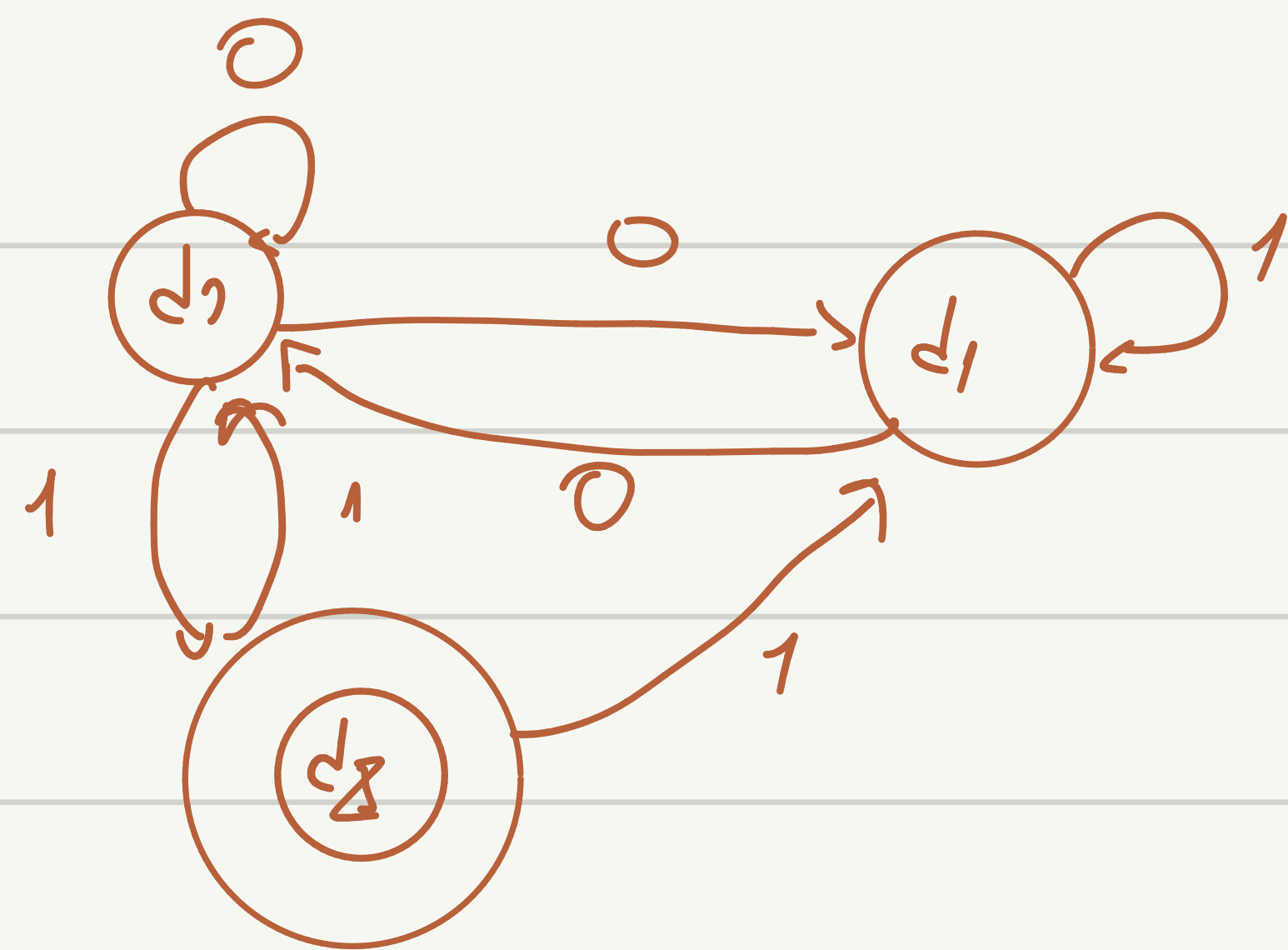
$\{d_0, 0\} \cup \{d_1, 0\}$



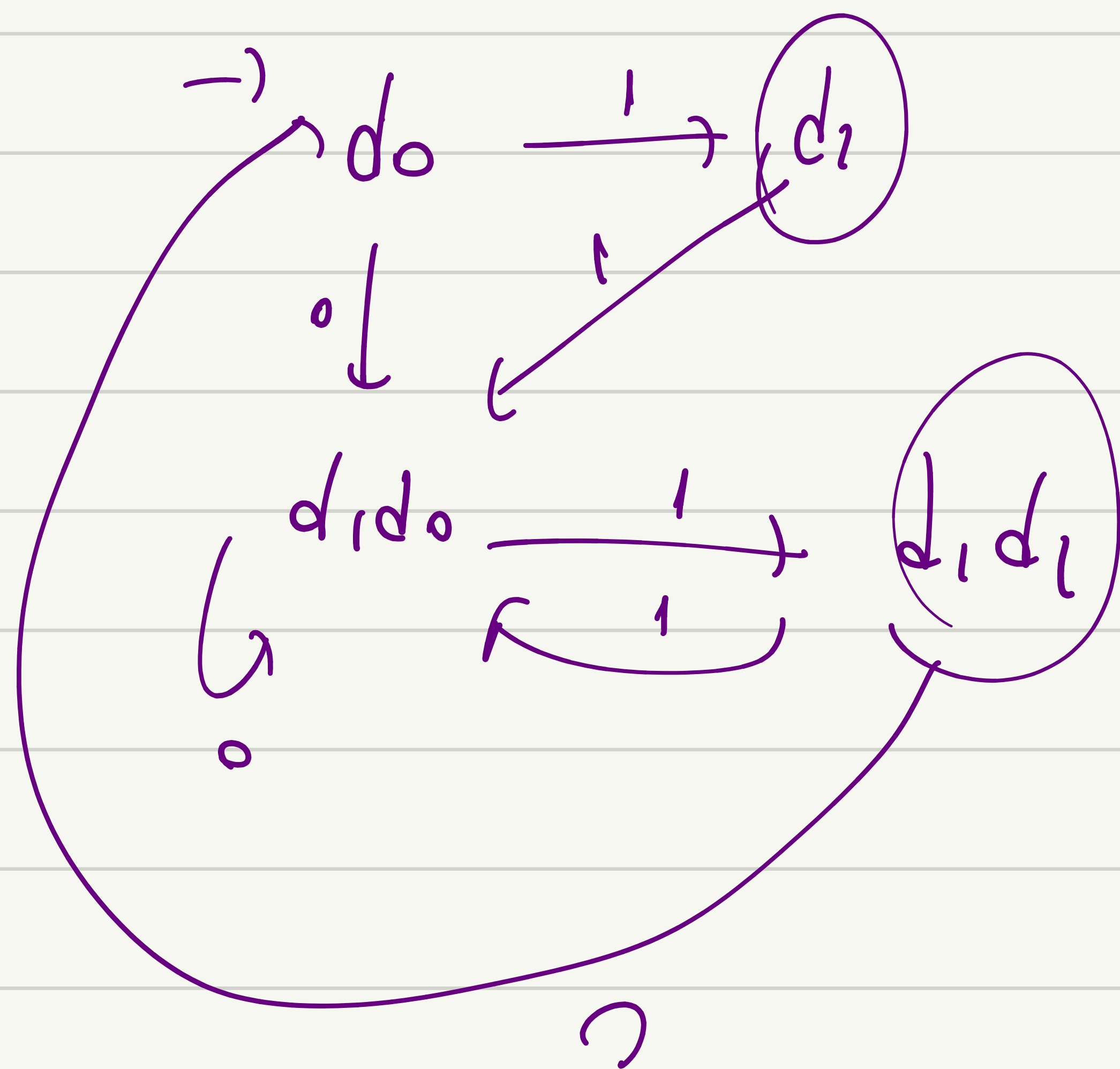
NFA ends with '01'



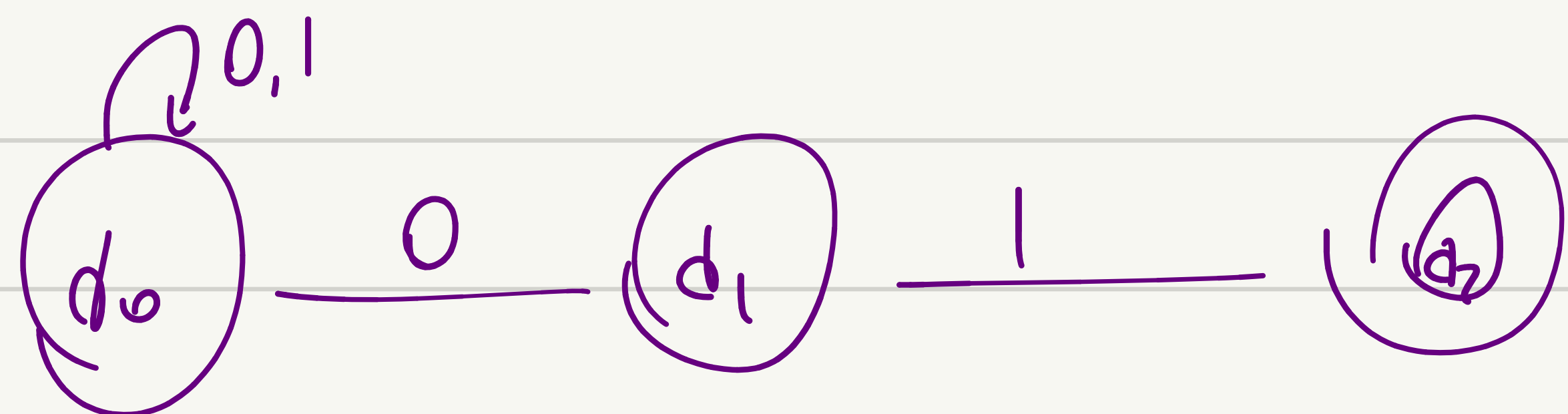
	0	1
d ₀	d ₀ d ₁	d ₀
d ₀ d ₁	d ₀ d ₁	d ₀ d ₁ d ₂
d ₀ d ₁ d ₂	d ₀	d ₀ d ₁



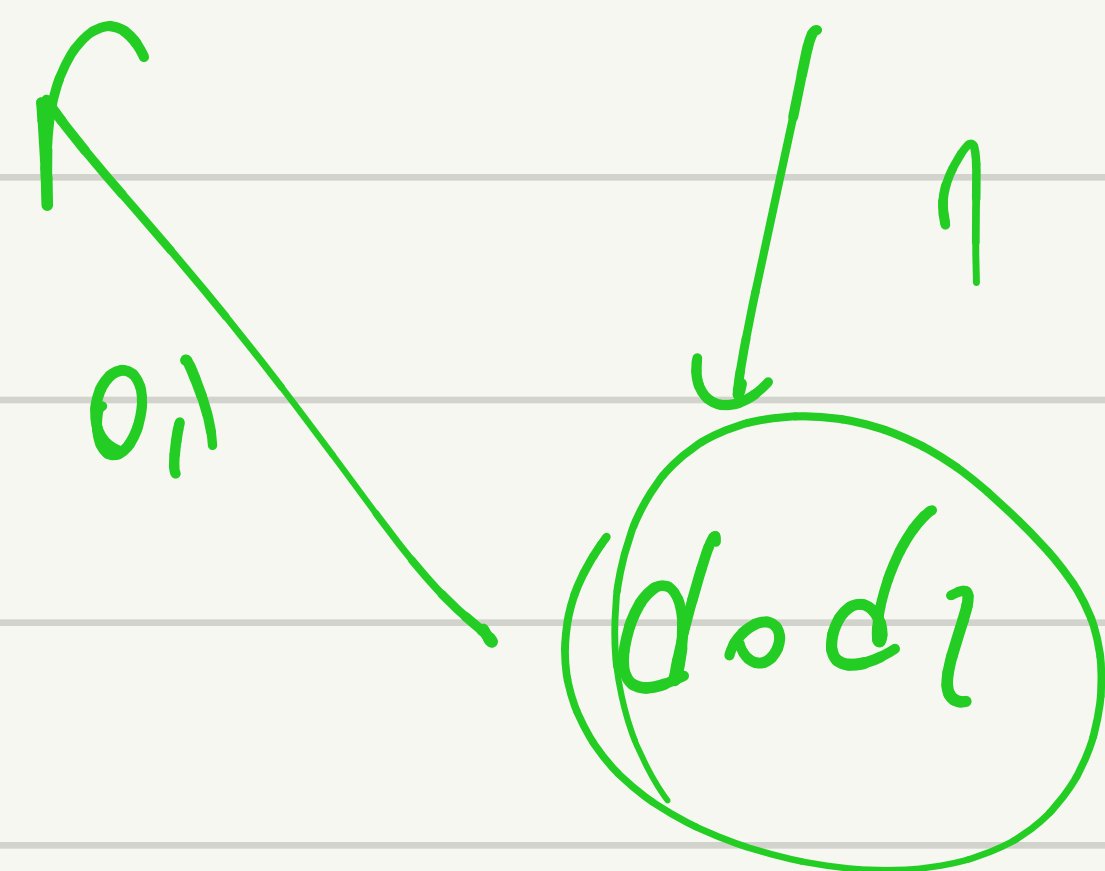
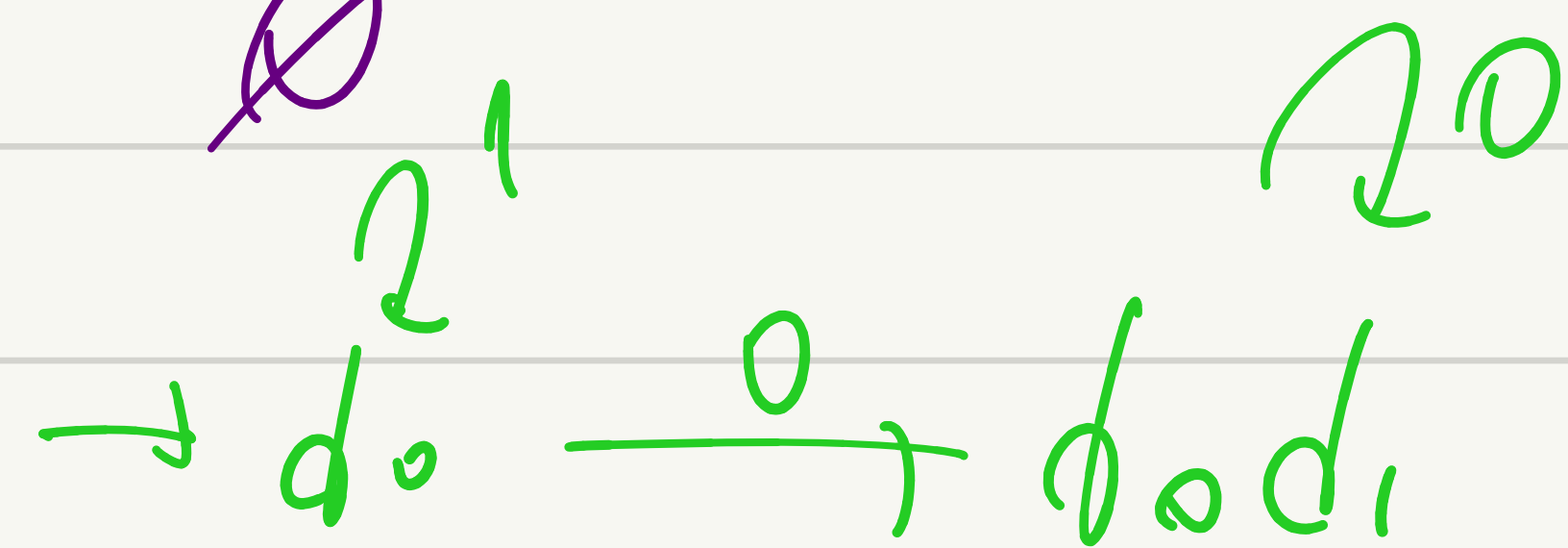
	0	1
d_0	d_0	d_1
d_1	$\emptyset$	d_0
$d_0 d_1$	$d_0 d_1$	$d_1 d_2$
$d_1 d_2$	d_0	$d_0 d_1$
d_3	d_3	d_3



NFA ends with '01'



	0	1
q ₀	q ₀ q ₁	q ₀
q ₁	∅	q ₂
q ₂	∅	∅



	0	1
q ₀	q ₀ q ₁	q ₀
q ₀ q ₁	q ₀ q ₁	q ₀ q ₂
q ₀ q ₂	q ₀	q ₀
q ₁		

Regular Expressions

Regular Expressions

Regular expressions
describe regular languages

recognizes $\rightarrow$ similar in
 $\downarrow$
accept?

Example:

$(a + b \cdot c)^*$

$a \leq b \leq a \leq b$

describes the language

$$\{a, bc\}^* = \{\varepsilon, a, bc, aa, abc, bca, \dots\}$$

Recursive Definition

Primitive regular expressions: $\emptyset$, ε , a

Handwritten notes:
 $\emptyset$ → empty set
 ε → empty string

Given regular expressions r_1 and r_2

Operator precedence

$\wedge$ → highest

*$\cdot$
+*

$r_1 + r_2$

$r_1 \cdot r_2$

r_1^*

(r_1)

Are regular expressions

Examples

A regular expression: $(a + b \cdot c)^* \cdot (c + \overset{\varepsilon}{\uparrow} \emptyset)$
c

Not a regular expression: ~~$(a + b +)$~~

Languages of Regular Expressions

$L(r)$: language of regular expression r

Example

$$L((a + b \cdot c)^*) = \{\varepsilon, a, bc, aa, abc, bca, \dots\}$$

Definition

For primitive regular expressions:

$$L(\emptyset) = \emptyset$$

$$L(\varepsilon) = \{\varepsilon\}$$

$$L(a) = \{a\}$$

Definition (continued)

For regular expressions r_1 and r_2

$$L(r_1 + r_2) = L(r_1) \cup L(r_2)$$

$$L(r_1 \cdot r_2) = L(r_1) L(r_2)$$

$$L(r_1^*) = (L(r_1))^*$$

$$L((r_1)) = L(r_1)$$

Example

ε

Regular expression: $(a + b) \cdot a^*$

$$\begin{aligned} L((a + b) \cdot a^*) &= L((a + b)) L(a^*) \\ &= L(a + b) L(a^*) \\ &= (L(a) \cup L(b)) (L(a))^* \\ &= (\{a\} \cup \{b\}) (\{a\})^* \\ &= \{a, b\} \{\varepsilon, a, aa, aaa, \dots\} \\ &= \{a, aa, aaa, \dots, b, ba, baa, \dots\} \end{aligned}$$

Example

Regular expression

$$r = (a + b)^* (a + bb)$$

$a \mid a \mid a \mid \dots \mid a \mid a \mid \dots$

$$L(r) = \{a, bb, aa, abb, ba, bbb, \dots\}$$

Example

Regular expression

$$r = (\cancel{aa})^* (bb)^* b$$

$$L(r) = \{a^{2n}b^{2m}b : n, m \geq 0\}$$

Example

Regular expression

$$r = (0 + 1)^* 00 (0 + 1)^*$$

$$L(r) = \{ \text{all strings containing substring } 00 \}$$

Equivalent Regular Expressions

Definition:

Regular expressions r_1 and r_2

are **equivalent** if $L(r_1) = L(r_2)$

Example

$L = \{ \text{all strings without substring } 00 \}$

$$r_1 = (1 + 01)^* (0 + \varepsilon)$$

$$r_2 = (1^* 0 1^*)^* (0 + \varepsilon) + 1^* (0 + \varepsilon)$$

$L(r_1) = L(r_2) = L \longrightarrow r_1 \text{ and } r_2$
are equivalent
regular expressions

Regular Expressions and Regular Languages

Theorem

$$\left\{ \begin{array}{l} \text{Languages} \\ \text{Generated by} \\ \text{Regular Expressions} \end{array} \right\} = \left\{ \begin{array}{l} \text{Regular} \\ \text{Languages} \end{array} \right\}$$

Proof:

$$\left\{ \begin{array}{l} \text{Languages} \\ \text{Generated by} \\ \text{Regular Expressions} \end{array} \right\} \subseteq \left\{ \begin{array}{l} \text{Regular} \\ \text{Languages} \end{array} \right\}$$

$$\left\{ \begin{array}{l} \text{Languages} \\ \text{Generated by} \\ \text{Regular Expressions} \end{array} \right\} \supseteq \left\{ \begin{array}{l} \text{Regular} \\ \text{Languages} \end{array} \right\}$$

Proof - Part 1

$$\left\{ \begin{array}{l} \text{Languages} \\ \text{Generated by} \\ \text{Regular Expressions} \end{array} \right\} \subseteq \left\{ \begin{array}{l} \text{Regular} \\ \text{Languages} \end{array} \right\}$$

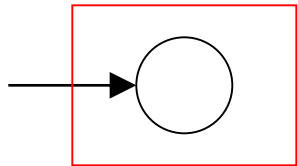
For any regular expression r
the language $L(r)$ is regular

Proof by induction on the size of r

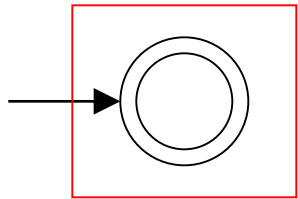
Induction Basis

Primitive Regular Expressions: $\emptyset$, ε , a

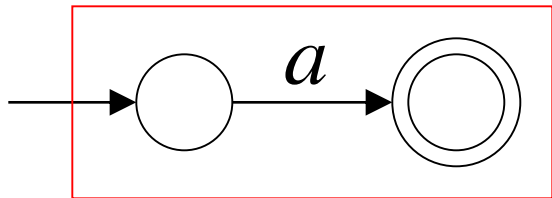
Corresponding
NFAs



$$L(M_1) = \emptyset = L(\emptyset)$$



$$L(M_2) = \{\varepsilon\} = L(\varepsilon)$$



$$L(M_3) = \{a\} = L(a)$$

regular
languages

Inductive Hypothesis

Suppose

that for regular expressions r_1 and r_2 ,
 $L(r_1)$ and $L(r_2)$ are regular languages

Inductive Step

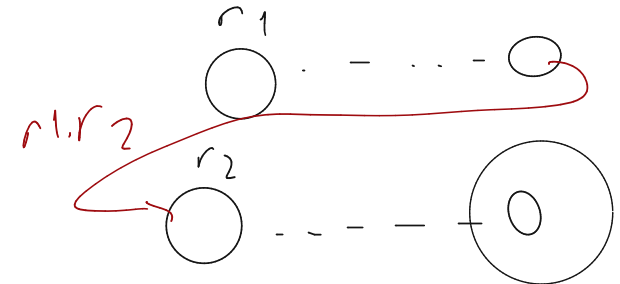
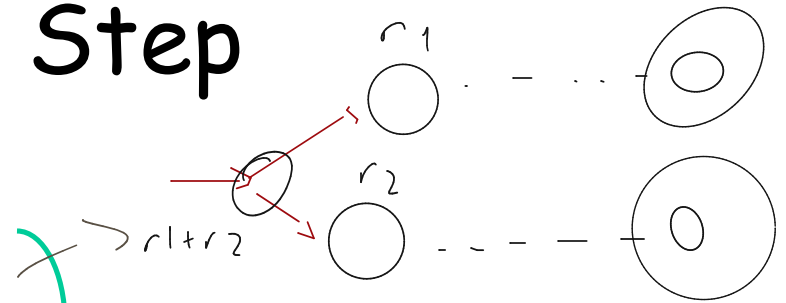
We will prove:

$$L(r_1 + r_2)$$

$$L(r_1 \cdot r_2)$$

$$L(r_1^*)$$

$$L((r_1))$$



Are regular
Languages



By definition of regular expressions:

$$L(r_1 + r_2) = L(r_1) \cup L(r_2)$$

$$L(r_1 \cdot r_2) = L(r_1) L(r_2)$$

$$L(r_1^*) = (L(r_1))^*$$

$$L((r_1)) = L(r_1)$$

By inductive hypothesis we know:

$L(r_1)$ and $L(r_2)$ are regular languages

We also know:

Regular languages are closed under:

Union

$$L(r_1) \cup L(r_2)$$

Concatenation

$$L(r_1) L(r_2)$$

Star

$$(L(r_1))^*$$

Therefore:

$$L(r_1 + r_2) = L(r_1) \cup L(r_2)$$

$$L(r_1 \cdot r_2) = L(r_1) L(r_2)$$

$$L(r_1^*) = (L(r_1))^*$$

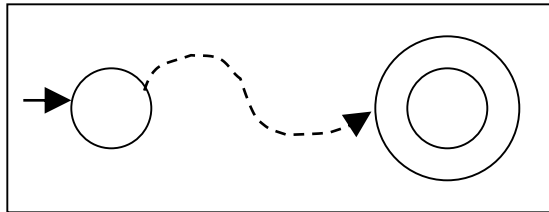
Are regular
languages

$L((r_1)) = L(r_1)$ is trivially a regular language
(by induction hypothesis)

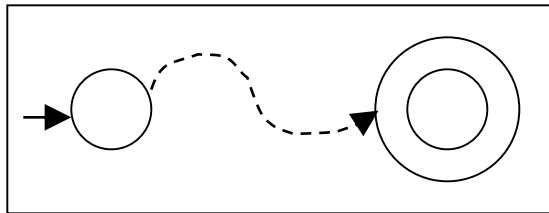
Using the regular closure of operations,
we can construct recursively the NFA M
that accepts $L(M) = L(r)$

Example: $r = r_1 + r_2$

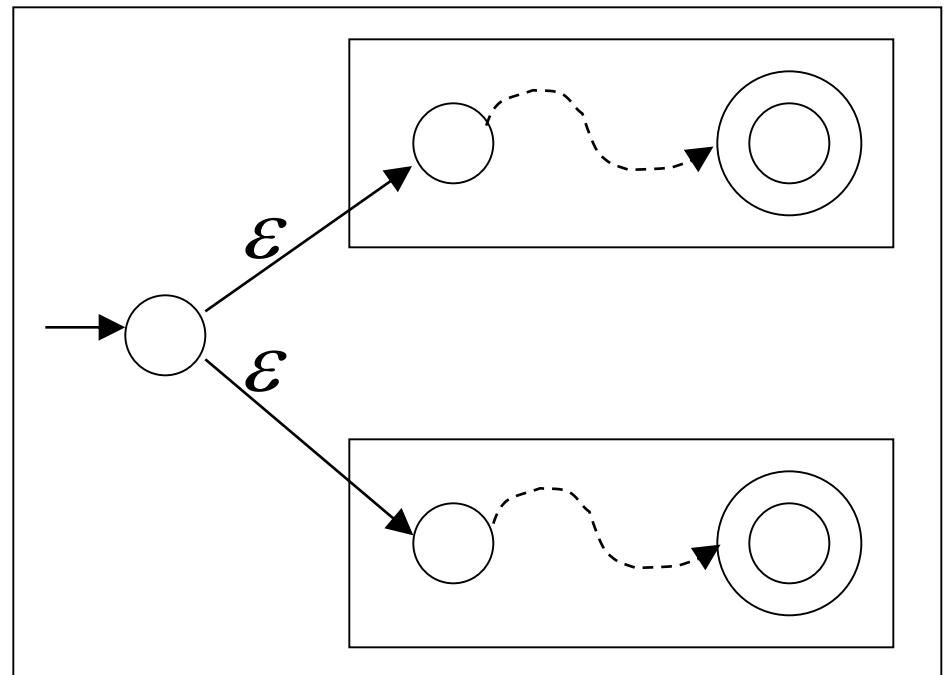
$L(M_1) = L(r_1)$



$L(M_2) = L(r_2)$



$L(M) = L(r)$



Proof - Part 2

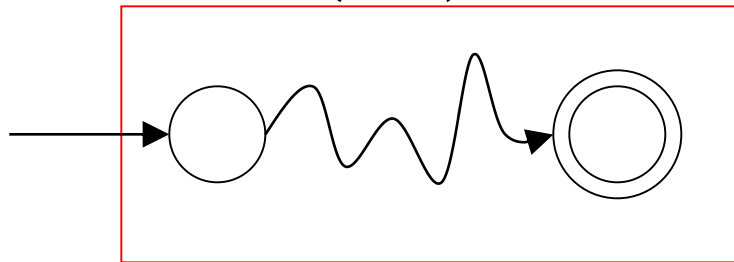
$$\left\{ \begin{array}{l} \text{Languages} \\ \text{Generated by} \\ \text{Regular Expressions} \end{array} \right\} \supseteq \left\{ \begin{array}{l} \text{Regular} \\ \text{Languages} \end{array} \right\}$$

For any regular language L there is
a regular expression r with $L(r) = L$

We will convert an NFA that accepts L
to a regular expression

Since L is regular, there is a NFA M that accepts it

$$L(M) = L$$



Take it with a single accept state

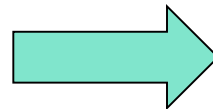
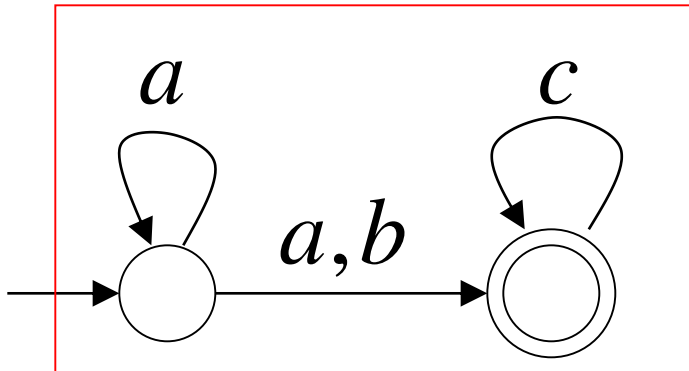
From M construct the equivalent

Generalized Transition Graph

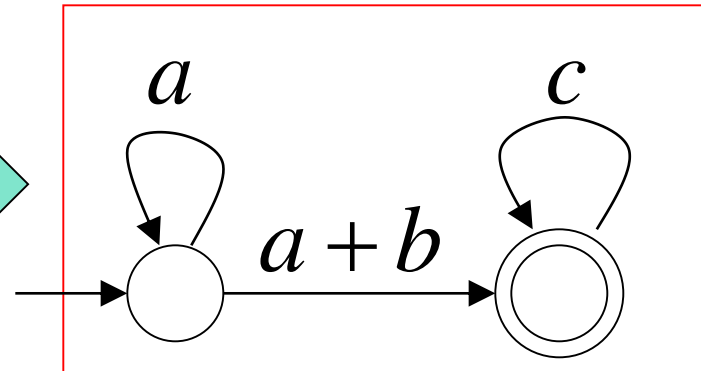
in which transition labels are regular expressions

Example:

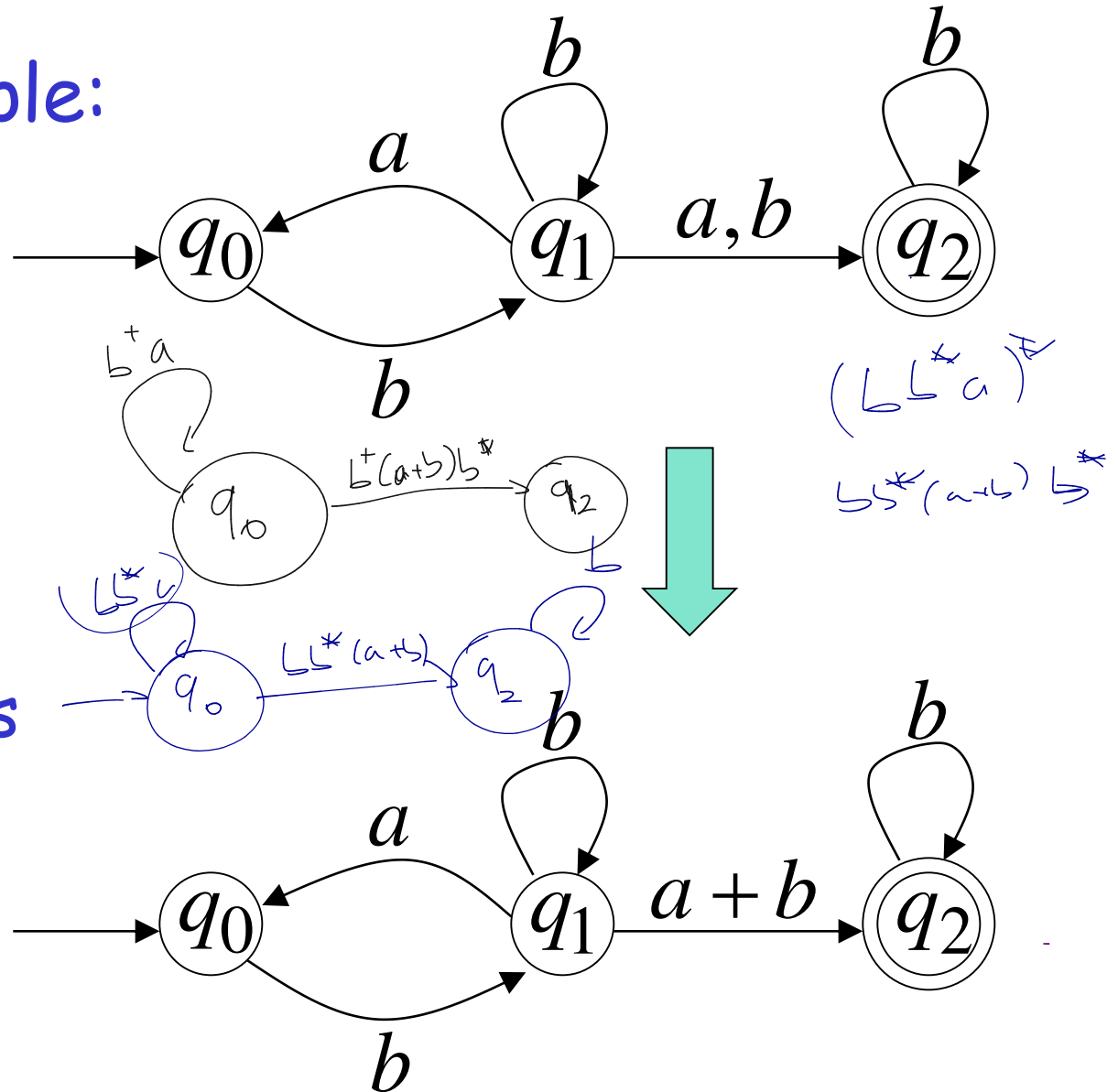
M



Corresponding
Generalized transition graph

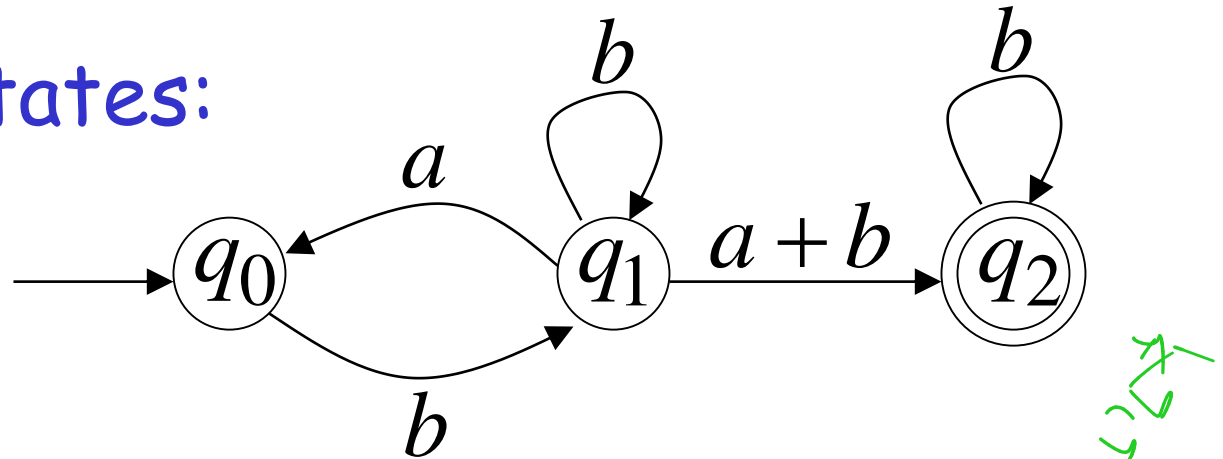


Another Example:

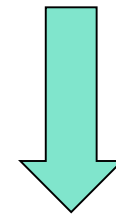


Transition labels
are regular
expressions

Reducing the states:

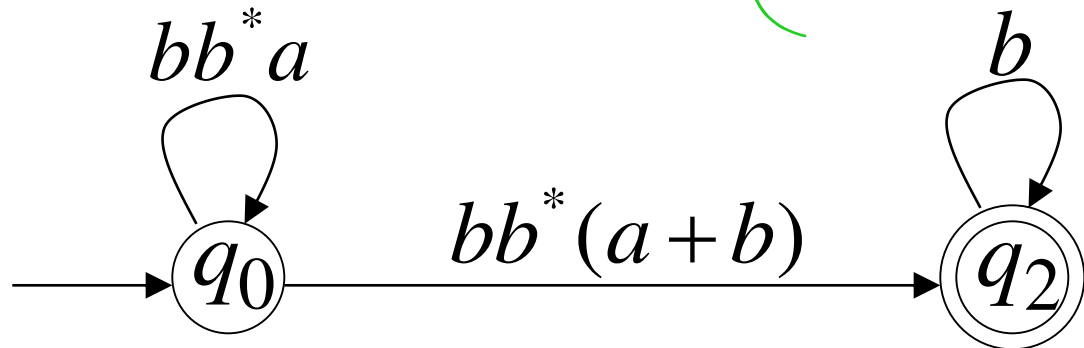


$(b^* a)^* b^* (a + b) b^*$

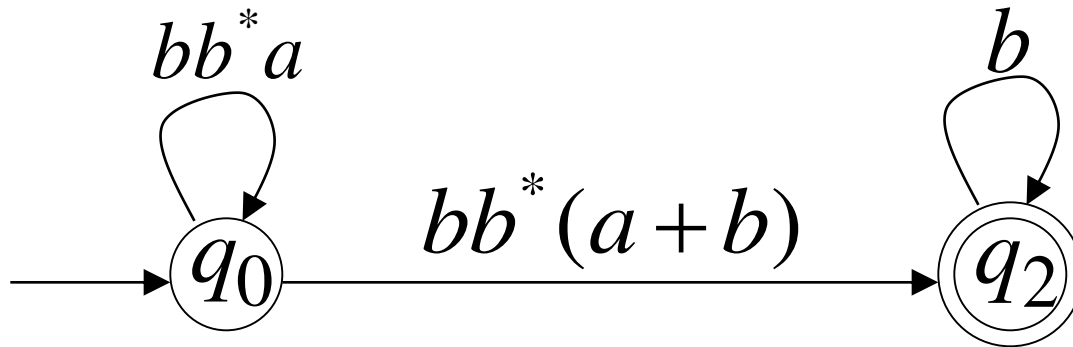


$(b^* a)^* b^* (a + b) b^*$

Transition labels
are regular
expressions



Resulting Regular Expression:

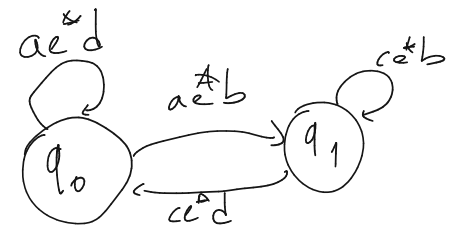
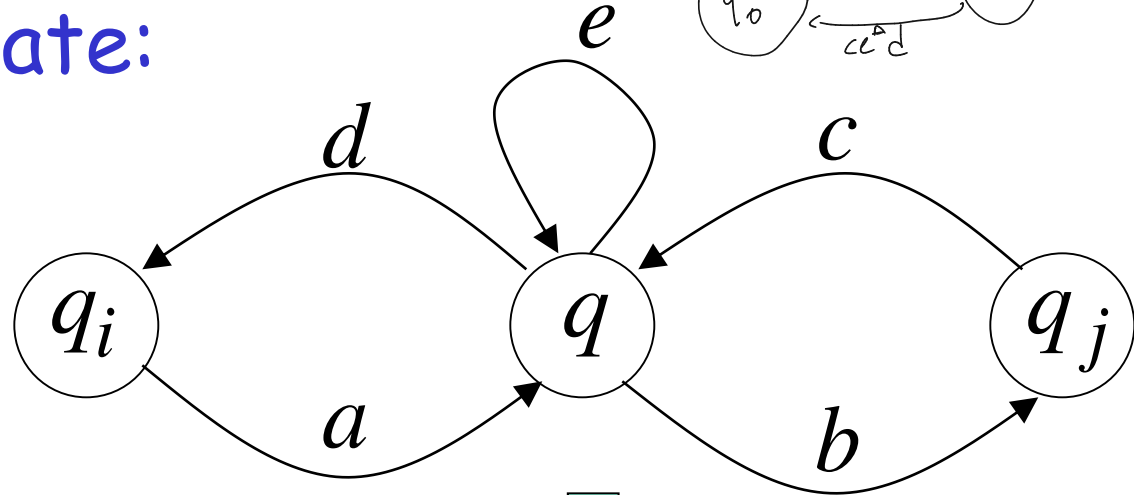


$$r = (bb^*a)^*bb^*(a+b)b^*$$

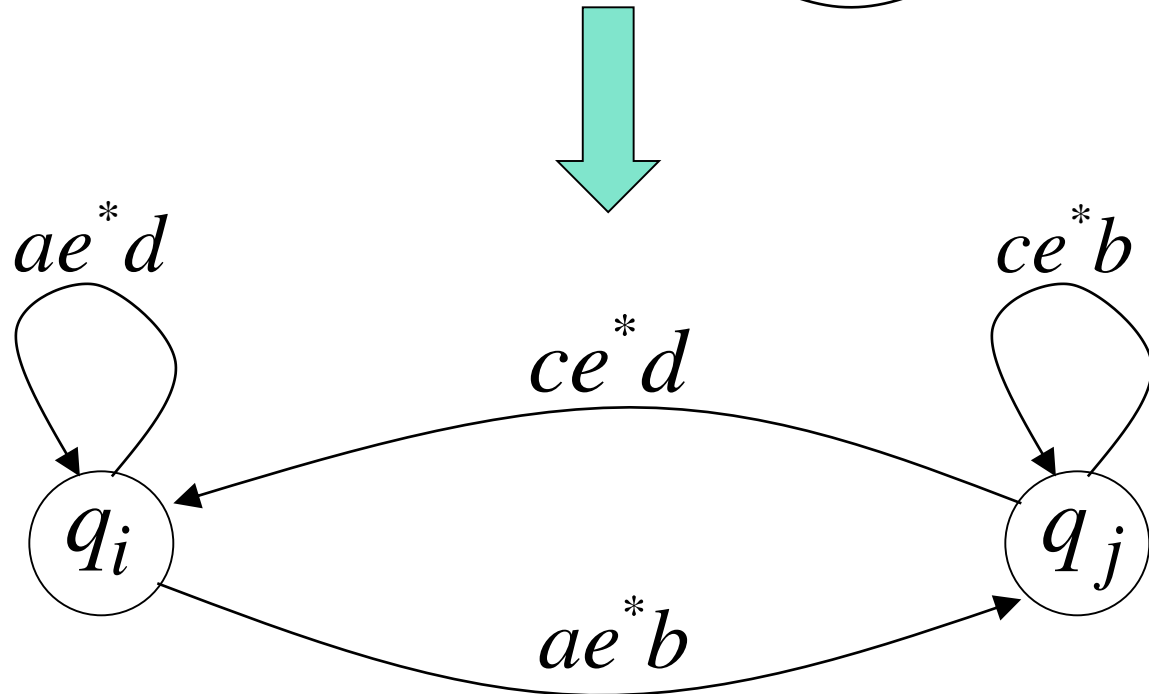
$$L(r) = L(M) = L$$

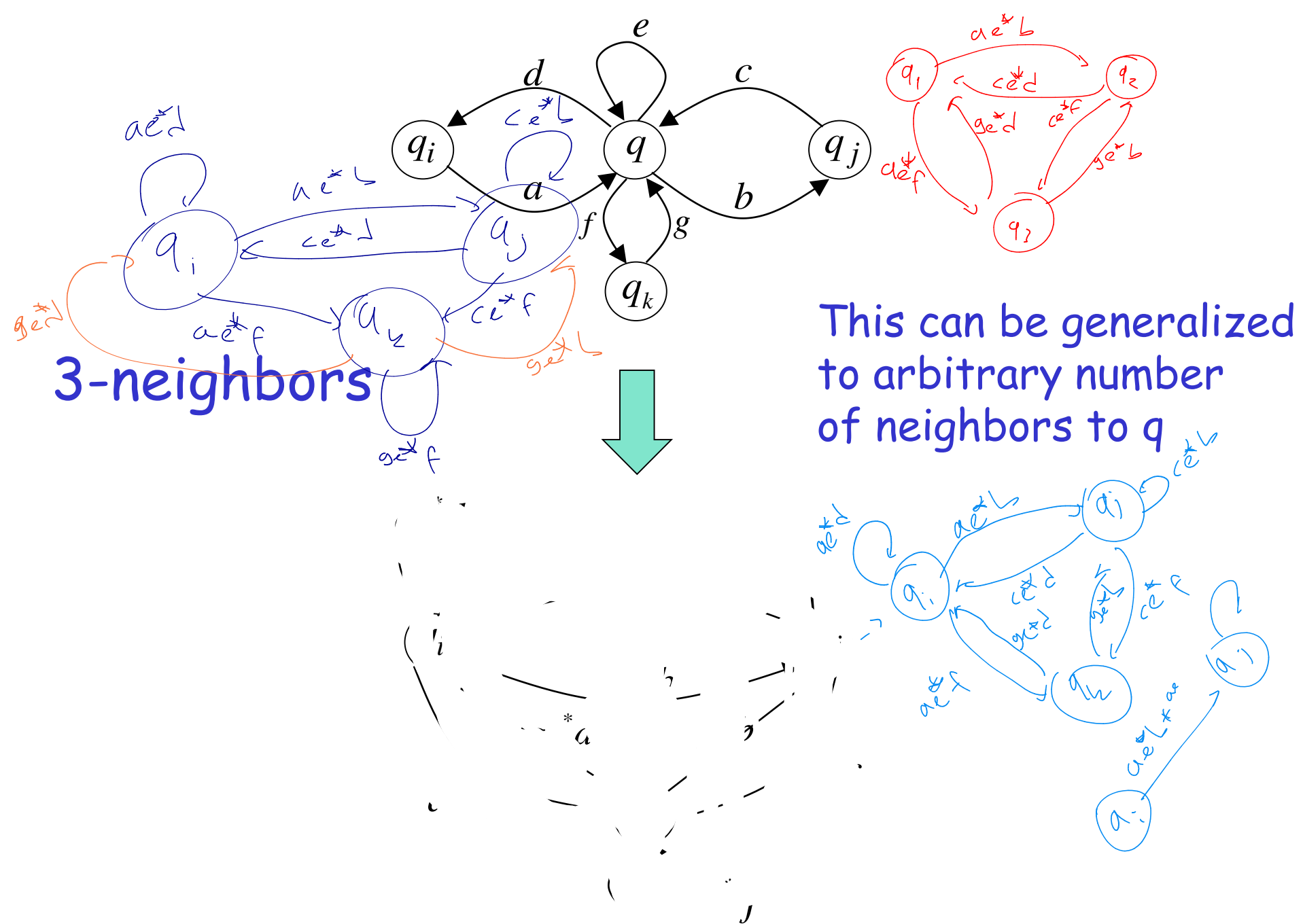
In General

Removing a state:

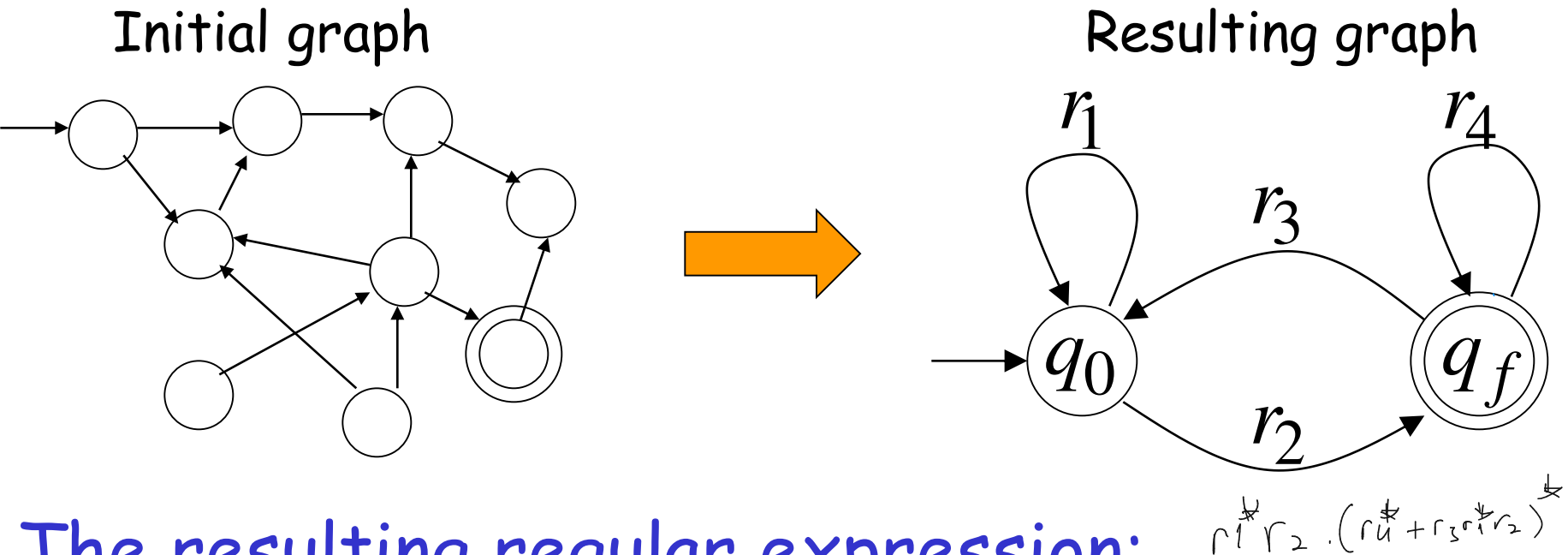


2-neighbors





By repeating the process until two states are left, the resulting graph is

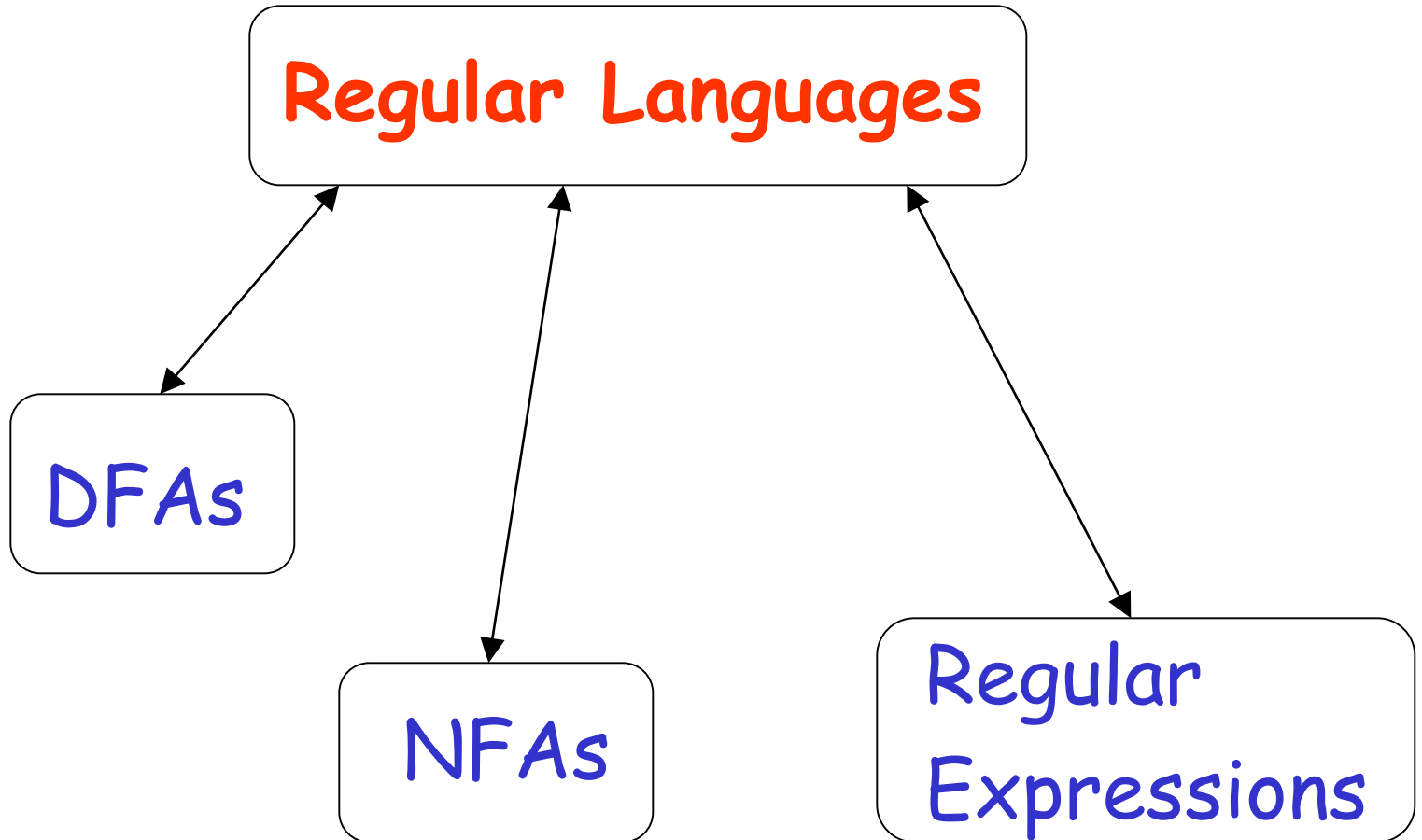


The resulting regular expression:

$$r = r_1^* r_2 (r_4 + r_3 r_1^* r_2)^*$$

$$L(r) = L(M) = L$$

Standard Representations of Regular Languages



When we say: We are given
a Regular Language L



We mean: Language L is in a standard
representation

(DFA, NFA, or Regular Expression)

BLM2502

Theory of Computation

BLM2502 Theory of Computation

Course Outline

Week	Content
1	Introduction to Course
2	Computability Theory, Complexity Theory, Automata Theory, Set Theory, Relations, Proofs, Pigeonhole Principle
3	Regular Expressions
4	Finite Automata
5	Deterministic and Nondeterministic Finite Automata
6	Epsilon Transition, Equivalence of Automata
7	Pumping Theorem
8	April 10 - 14 week is the first midterm week
9	Context Free Grammars
10	Parse Tree, Ambiguity,
11	Pumping Theorem
12	Turing Machines, Recognition and Computation, Church-Turing Hypothesis
13	Turing Machines, Recognition and Computation, Church-Turing Hypothesis
14	May 22 - 27 week is the second midterm week
15	Review
16	Final Exam date will be announced

Week I - Introduction

Non-regular languages

(Pumping Lemma)

Non-regular languages

$$\{a^n b^n : n \geq 0\}$$

$$\{vv^R : v \in \{a,b\}^*\}$$

Regular languages

$$a^*b$$

$$b^*c + a$$

$$b + c(a + b)^*$$

etc...

How can we prove that a language L is not regular?

Prove that there is no DFA or NFA or RE that accepts L

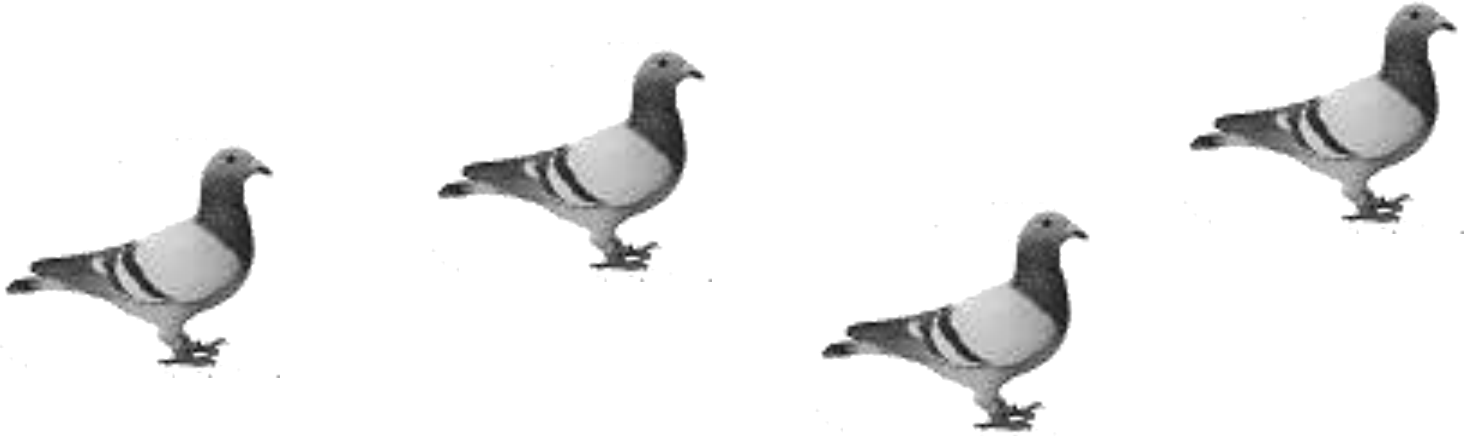
Difficulty: this is not easy to prove
(since there is an infinite number of them)

Solution: use the Pumping Lemma !!!

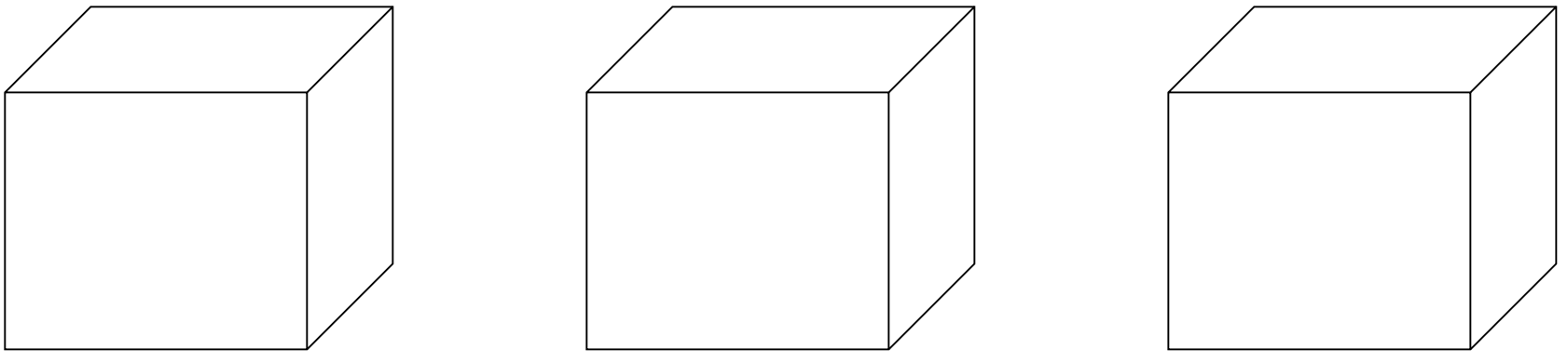


The Pigeonhole Principle

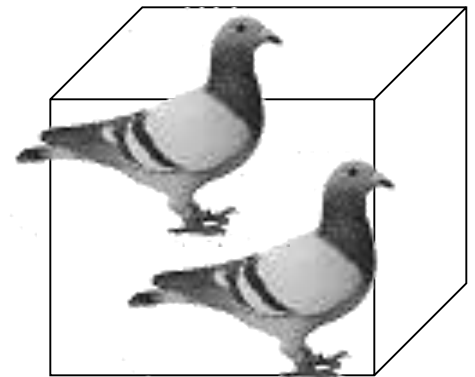
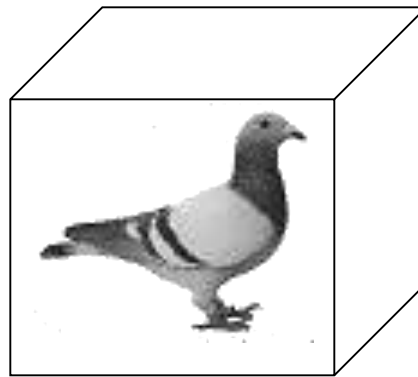
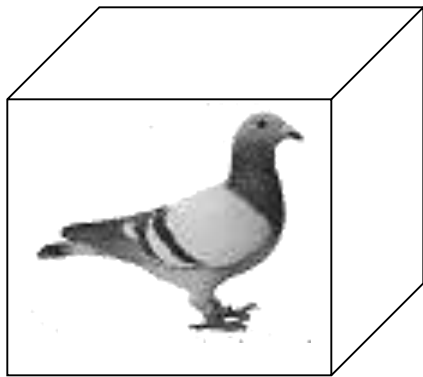
4 pigeons



3 pigeonholes



A pigeonhole must
contain at least two pigeons

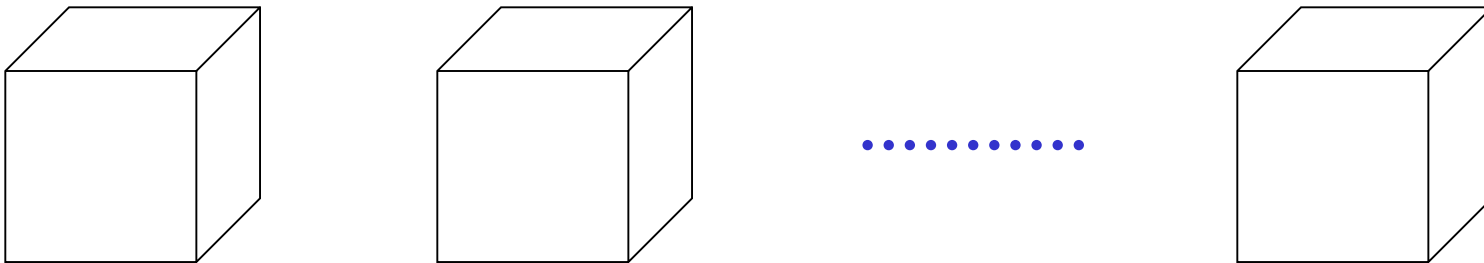


n pigeons



m pigeonholes

$n > m$



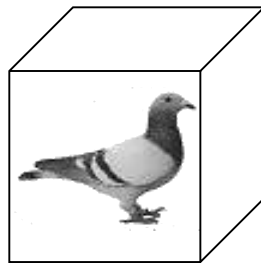
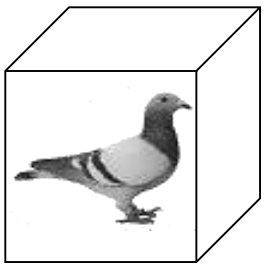
The Pigeonhole Principle

n pigeons

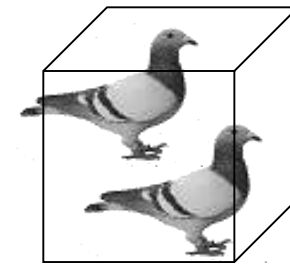
m pigeonholes

$n > m$

There is a pigeonhole
with at least 2 pigeons



.....

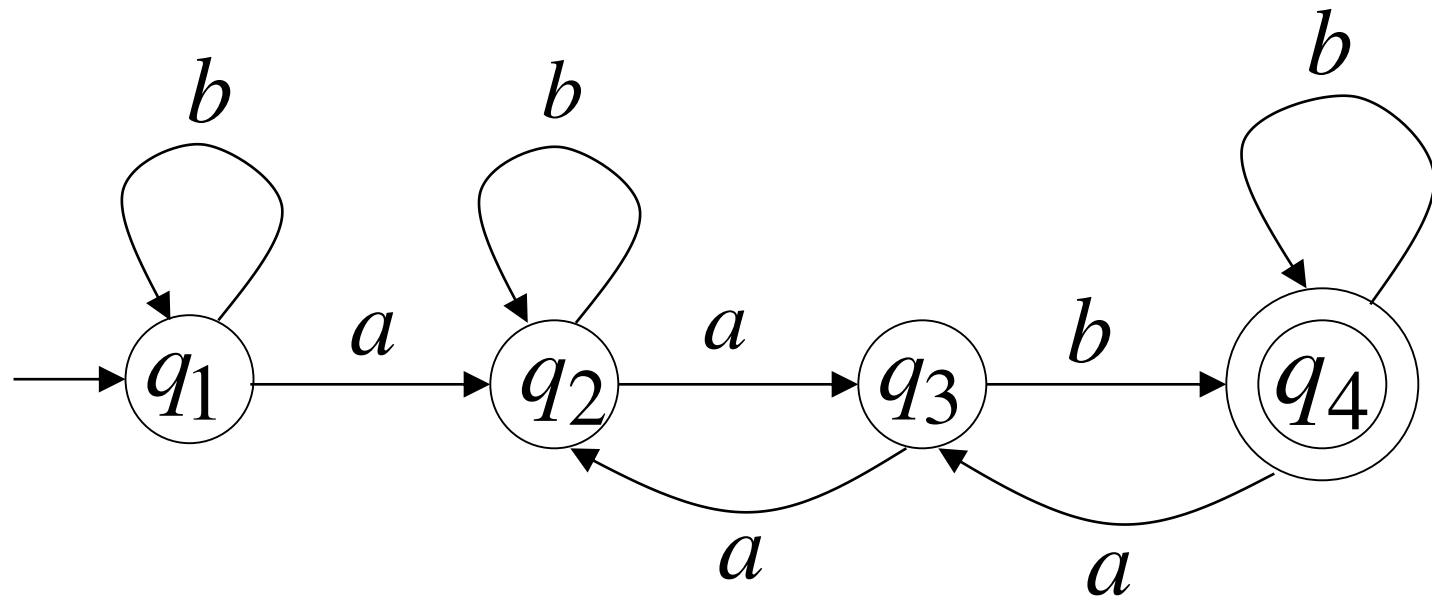


The Pigeonhole Principle

and

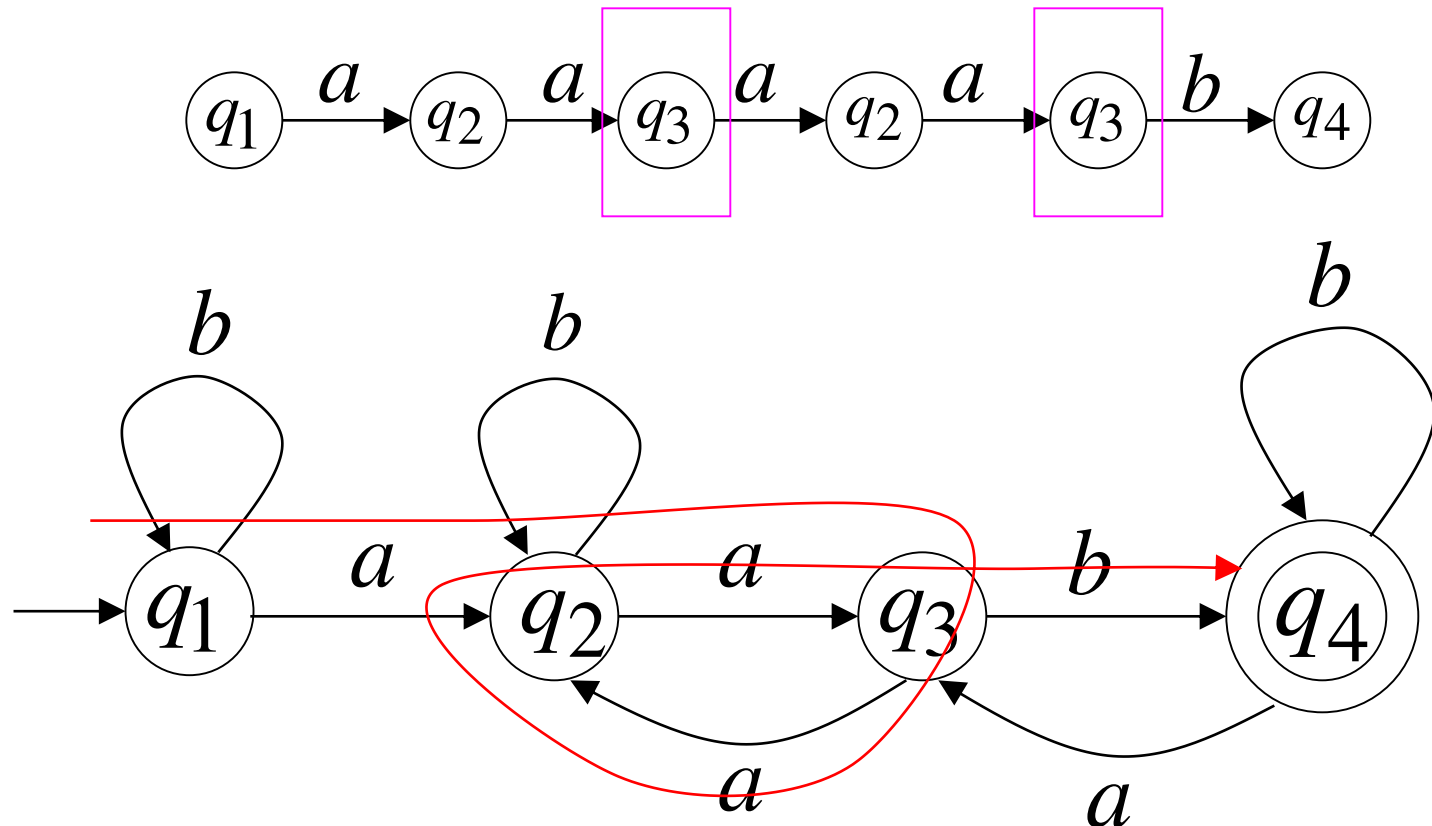
DFAs

Consider a DFA with 4 states



Consider the walk of a “long” string: $aaaaab$
(length at least 4)

A state is repeated in the walk of $aaaaab$

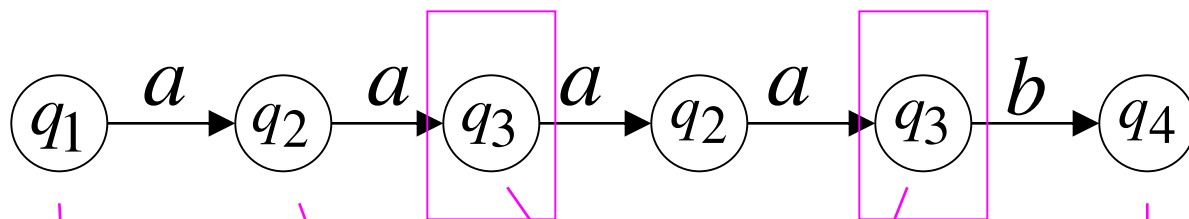


The state is repeated as a result of the pigeonhole principle

*N state
N+1 input
1 state can
be repeated*

Walk of *adaab*

Pigeons:
(walk states)



Are more than

Nests:
(Automaton states)

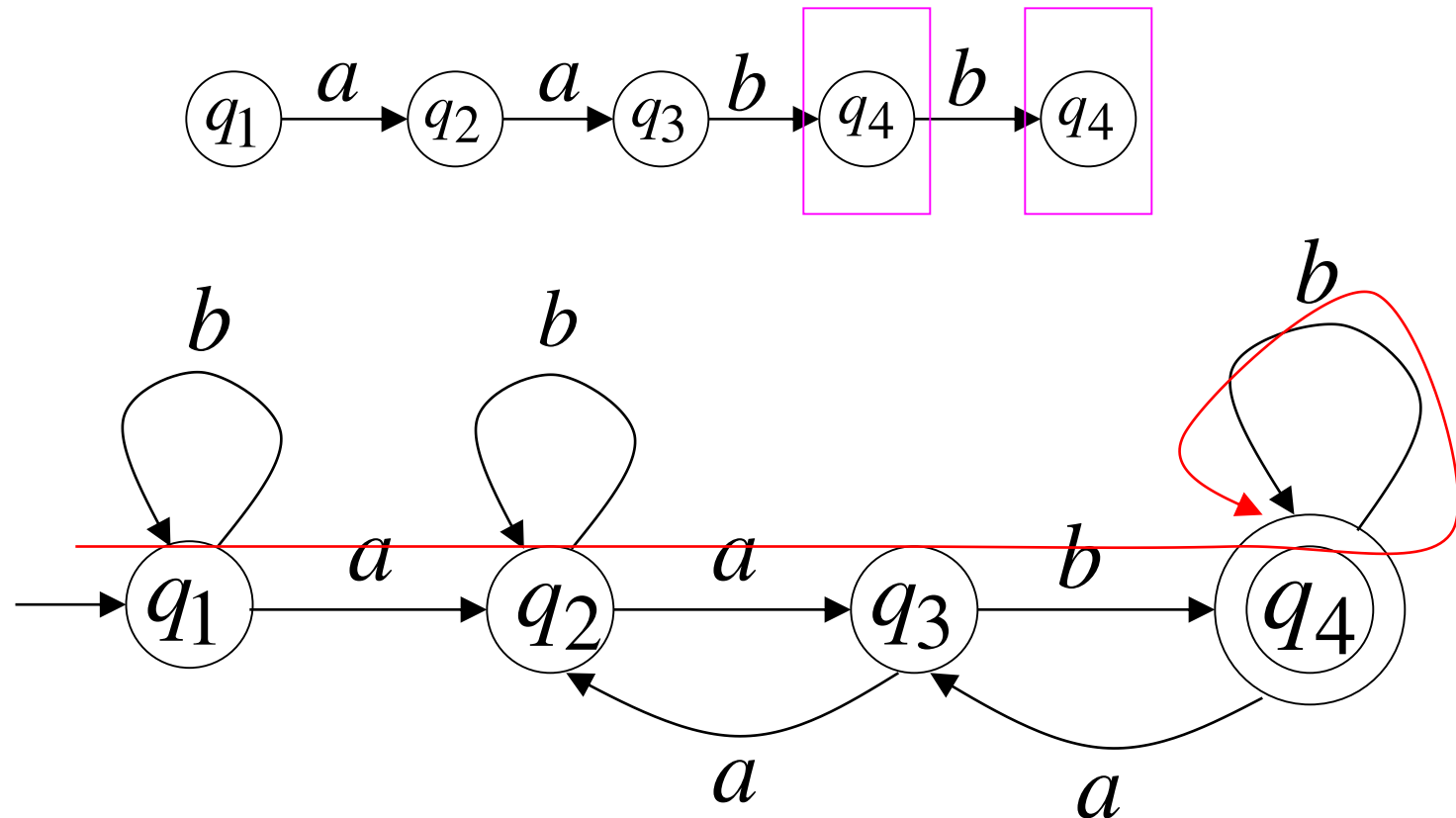


Repeated
state

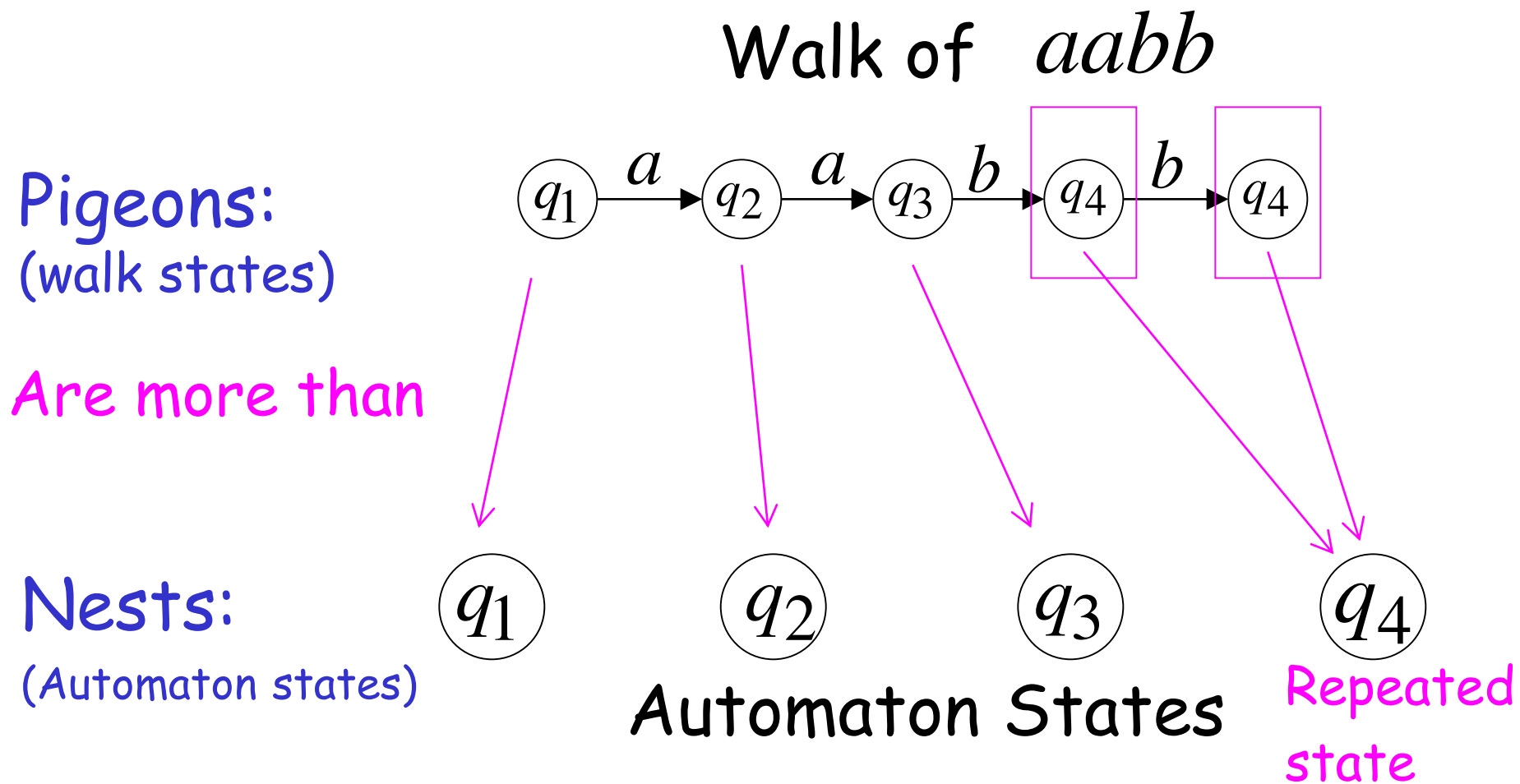
Consider the walk of a “long” string: $aabb$
(length at least 4)

Due to the pigeonhole principle:

A state is repeated in the walk of $aabb$

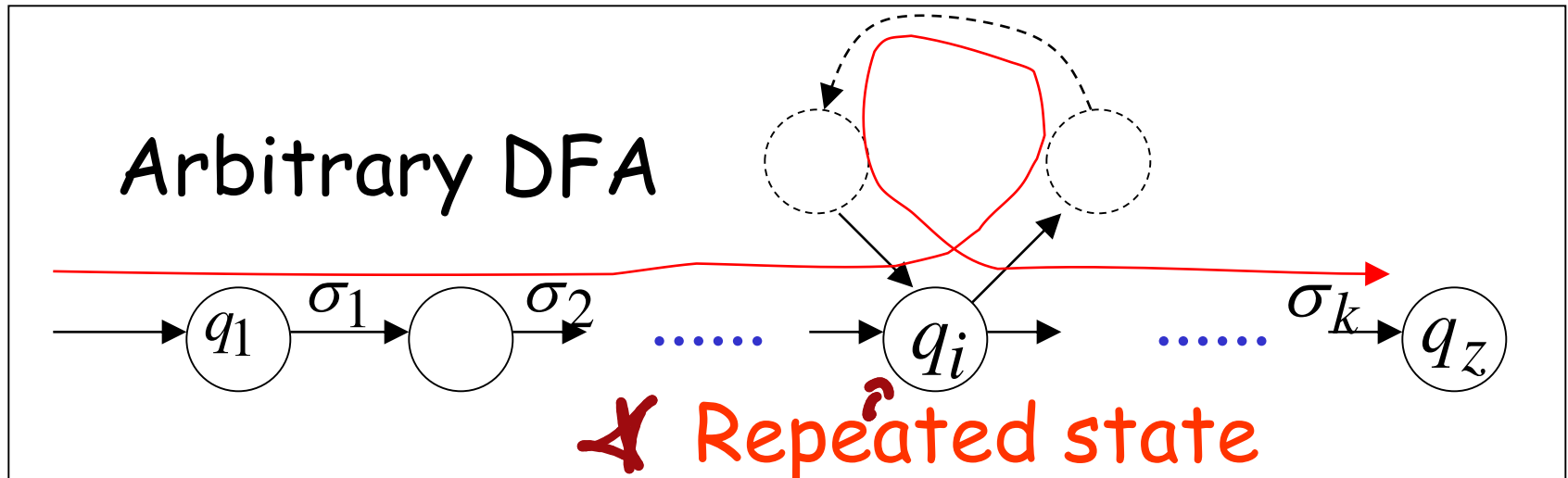
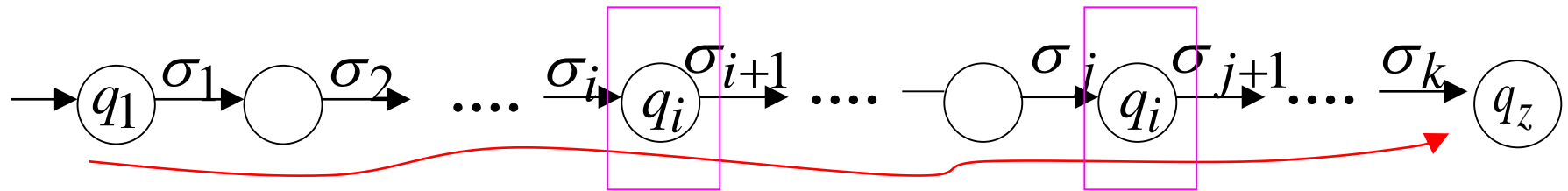


The state is repeated as a result of the pigeonhole principle

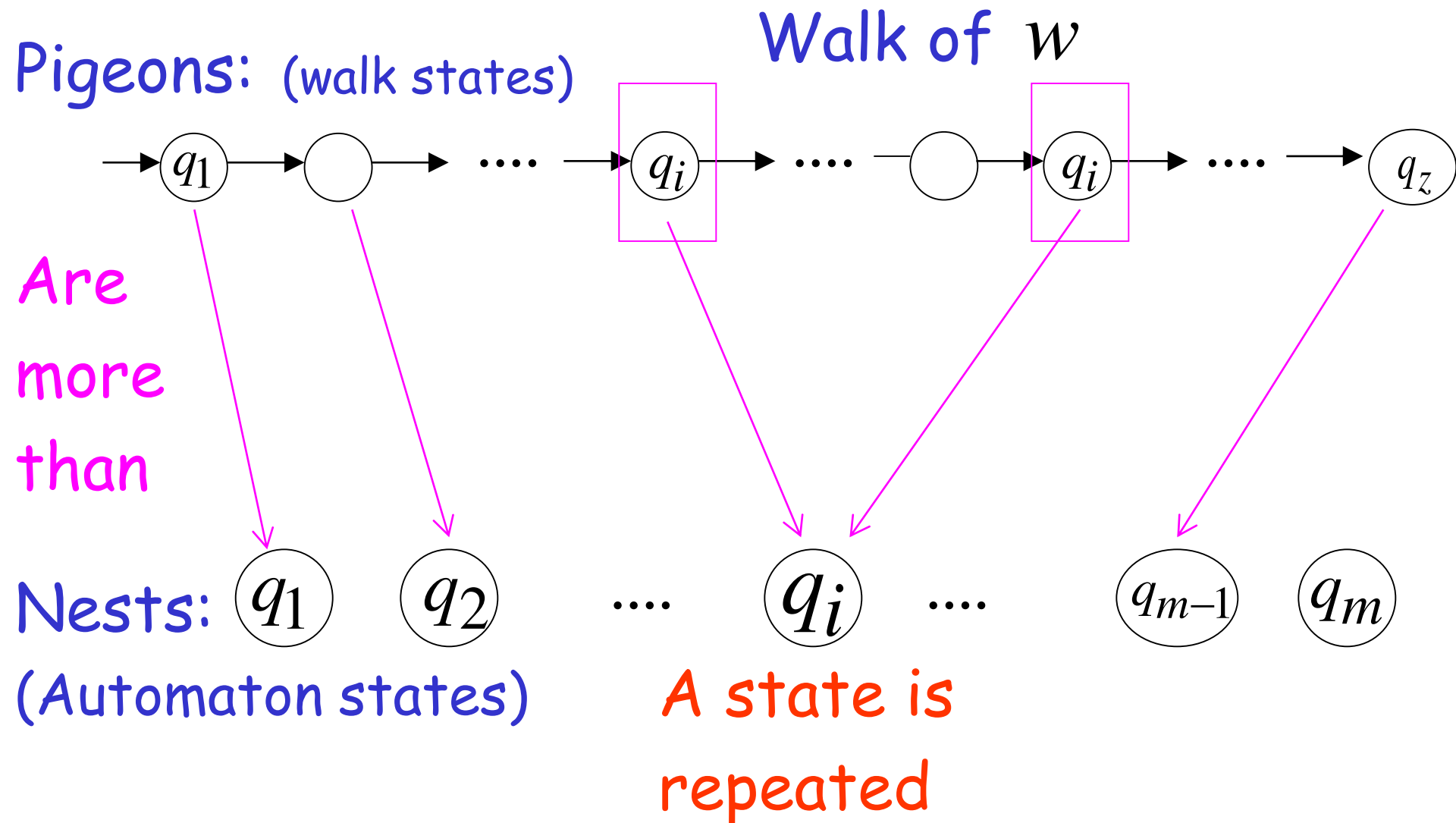


In General: If $|w| \geq \# \text{states of DFA}$,
by the pigeonhole principle,
a state is repeated in the walk w

Walk of $w = \sigma_1 \sigma_2 \cdots \sigma_k$



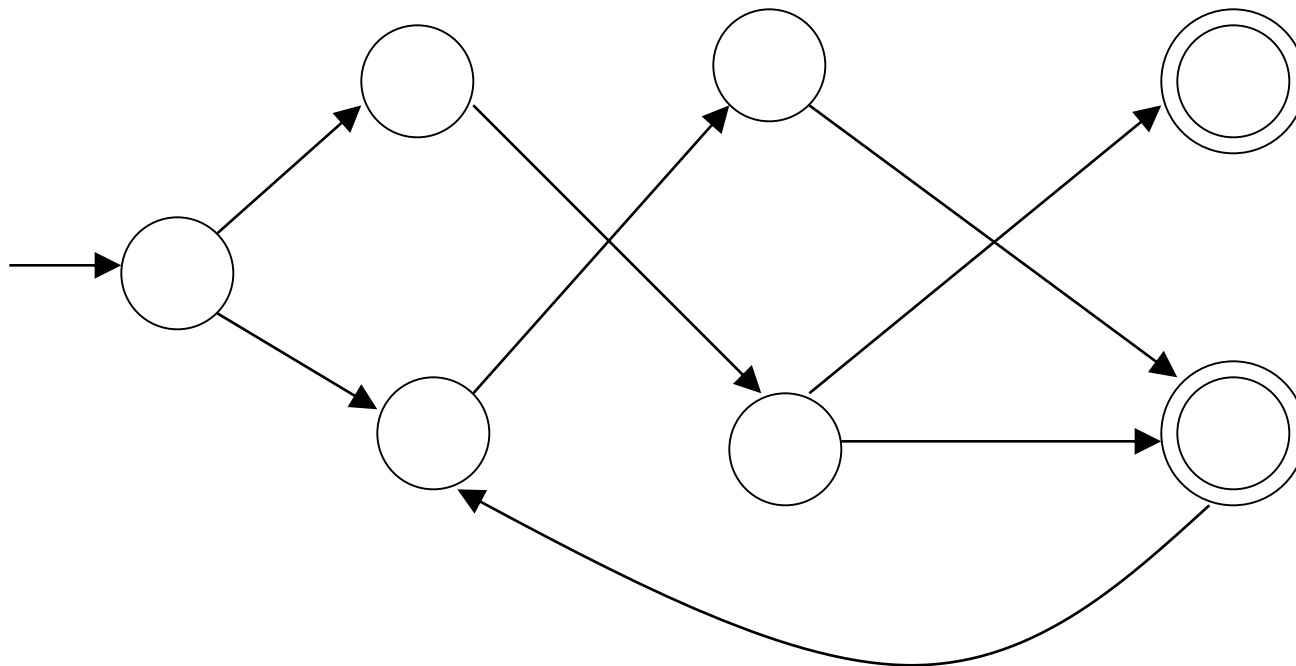
$$|w| \geq \# \text{states of DFA} = m$$



The Pumping Lemma

Take an **infinite** regular language L
(contains an infinite number of strings)

There exists a DFA that accepts L

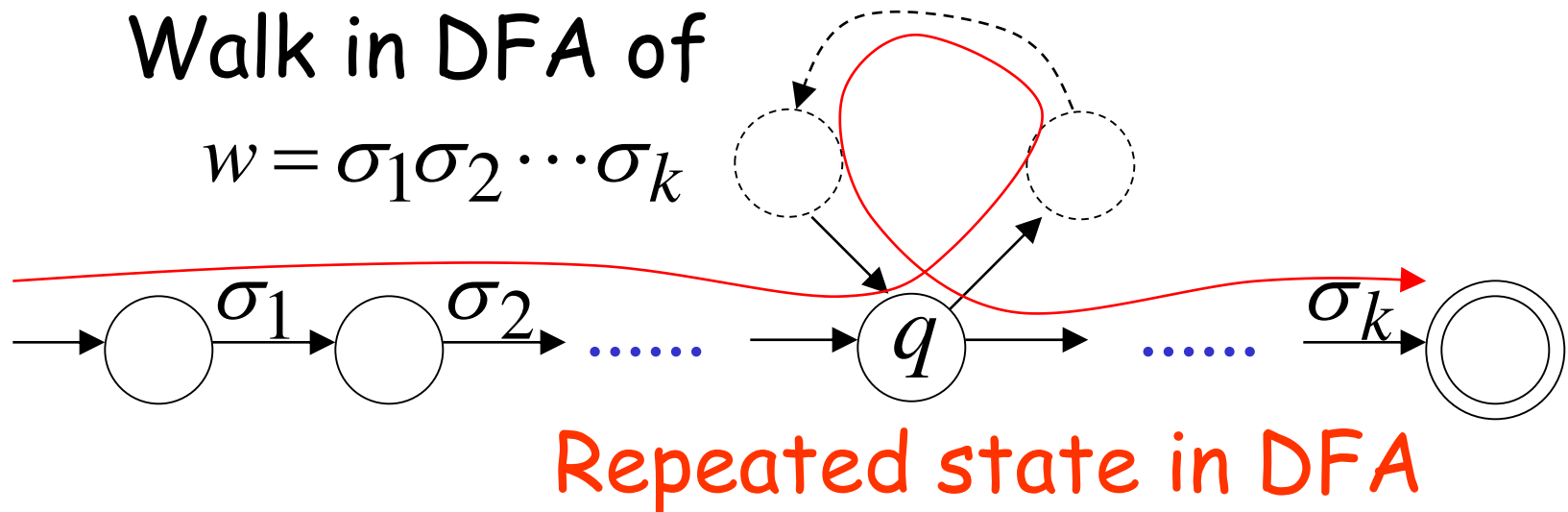


m
states

Take string $w \in L$ with $|w| \geq m$

(number of
states of DFA)

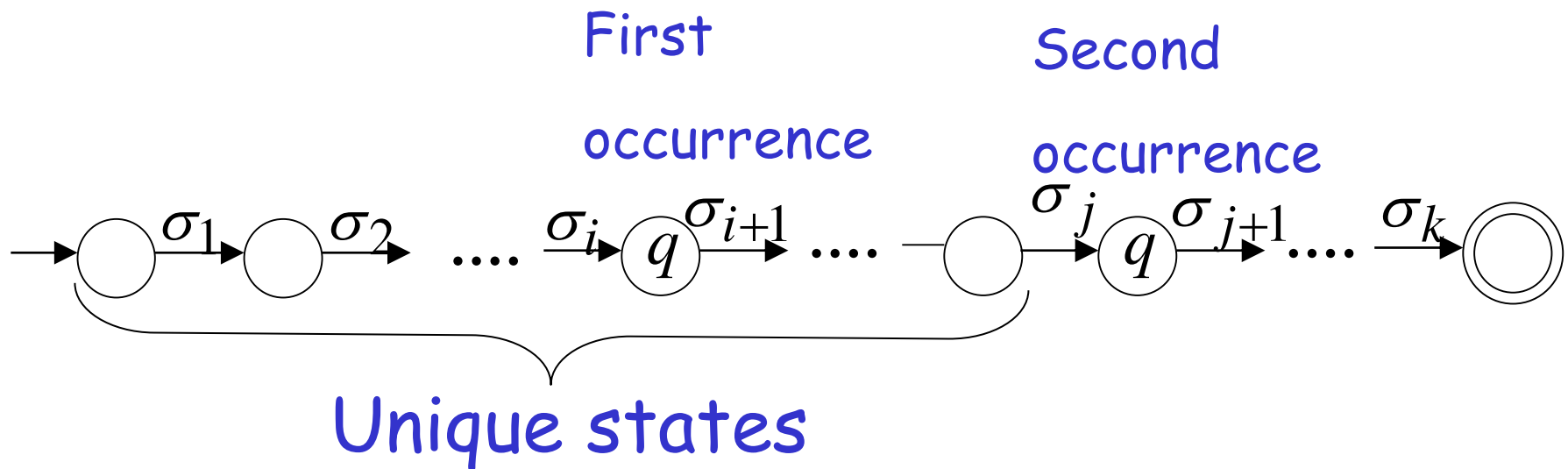
then, at least one state is repeated
in the walk of w



There could be many states repeated

Take q to be the first state repeated

One dimensional projection of walk w :



We can write

$$w = xyz$$

*xin de yin
de uzunluğ
olabilir*

*q idar
q ile
q ya
geçilirse
+ y a*

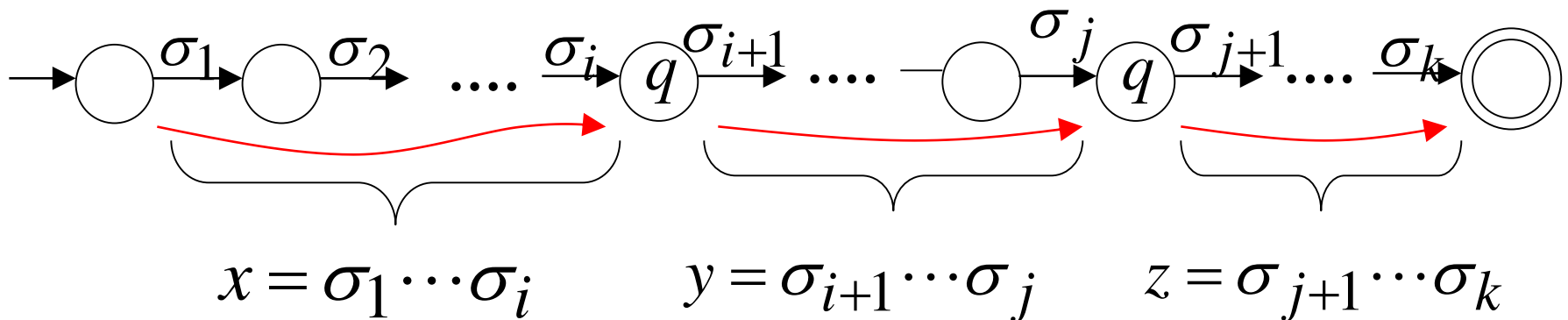
One dimensional projection of walk w :

First

Second

occurrence

occurrence

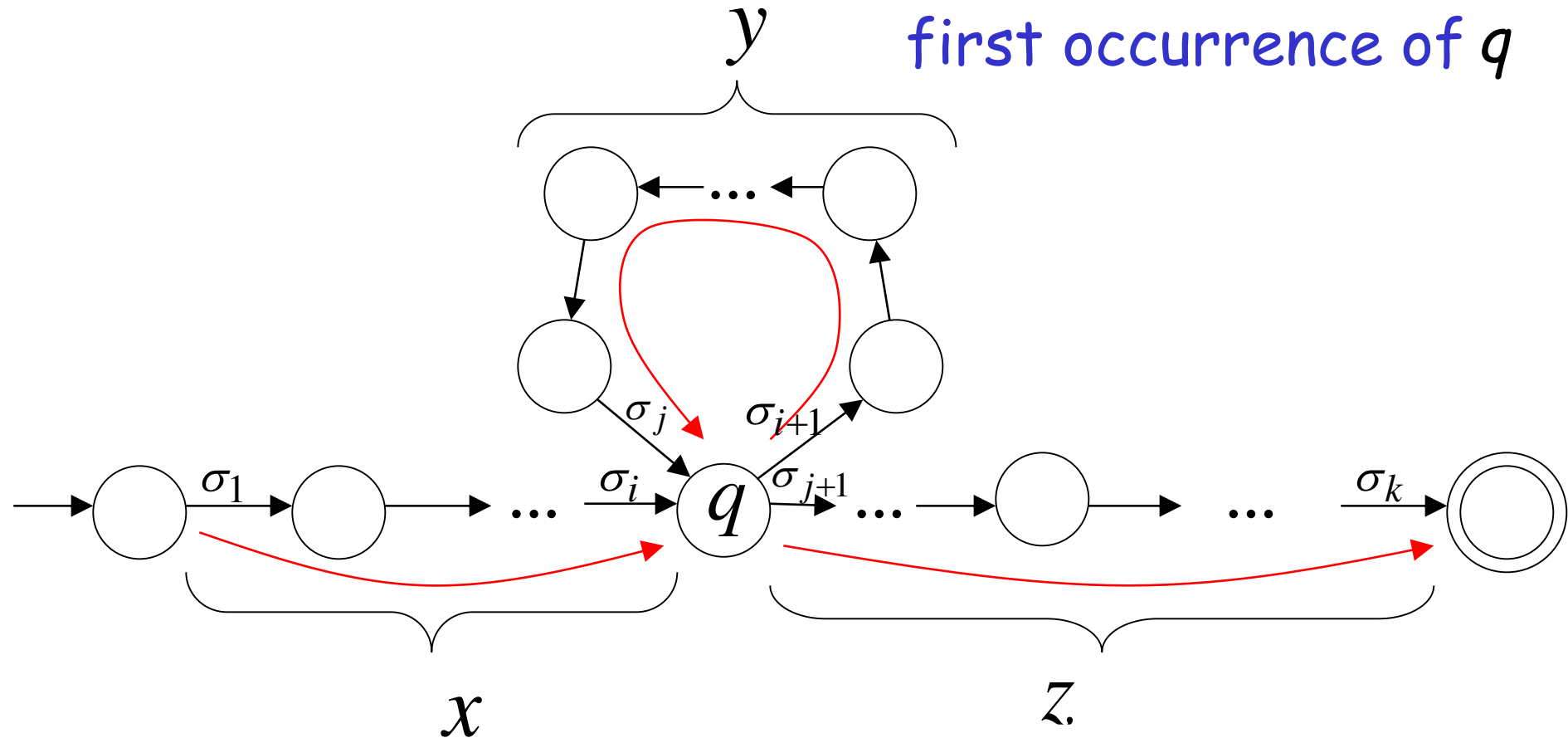


In DFA:

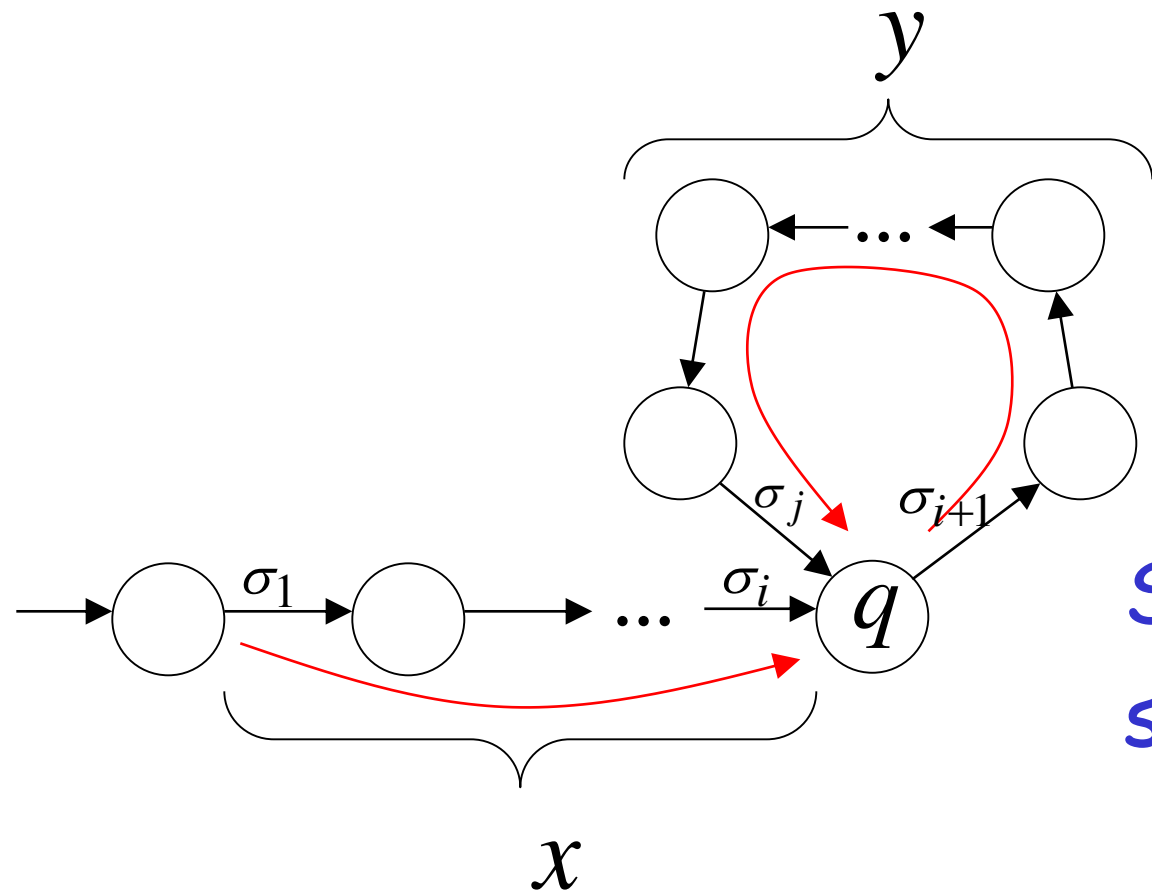
$$w = x y z$$

$x y z^m$

contains only
first occurrence of q



Observation: $\text{length } |x y| \leq m$ number of states of DFA

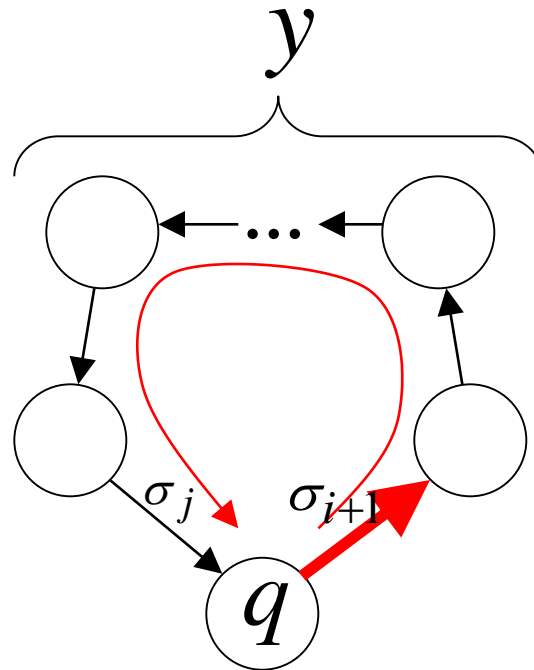


Unique States

Since, in xy no state is repeated (except q)

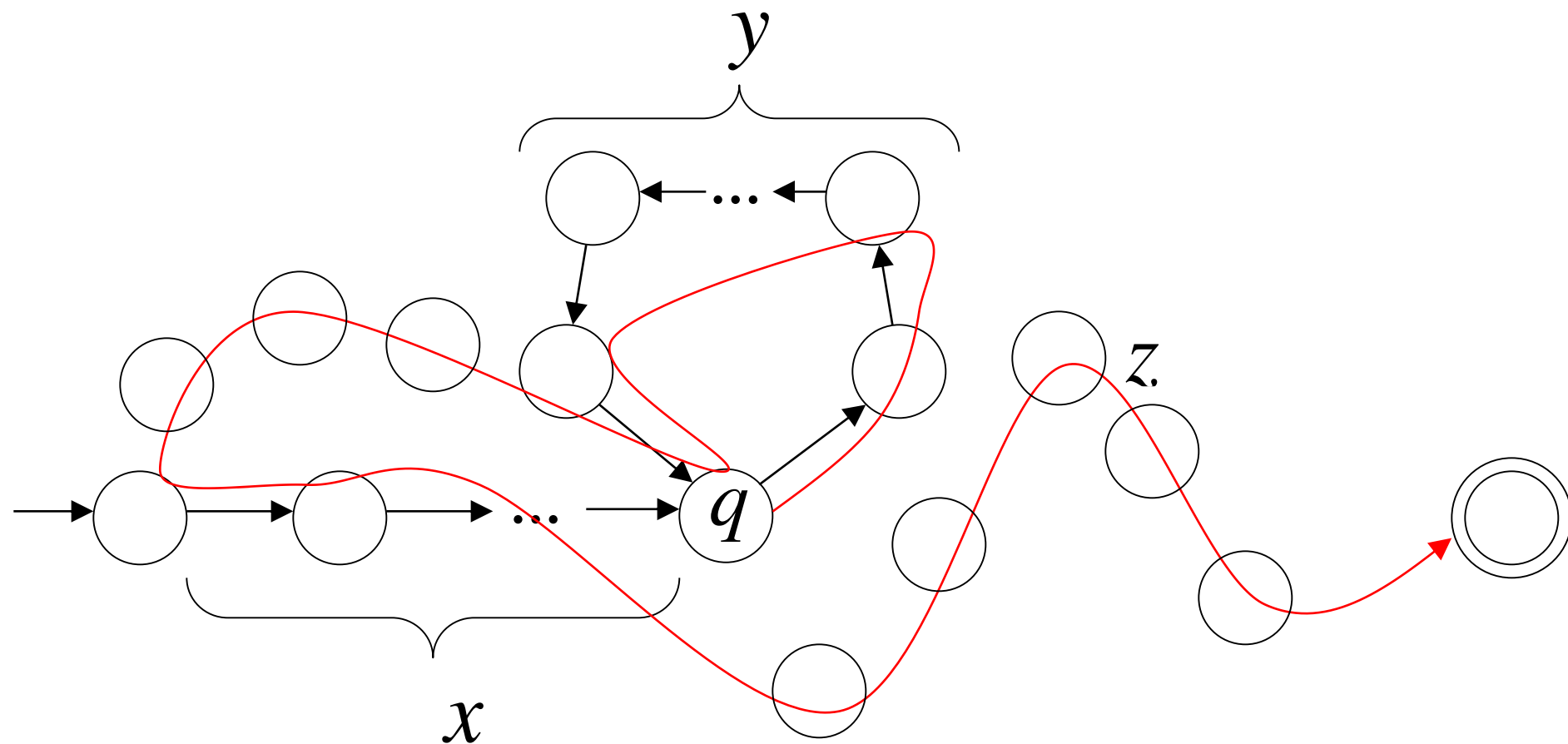
Observation: $\text{length } |y| \geq 1$

Since there is at least one transition in loop



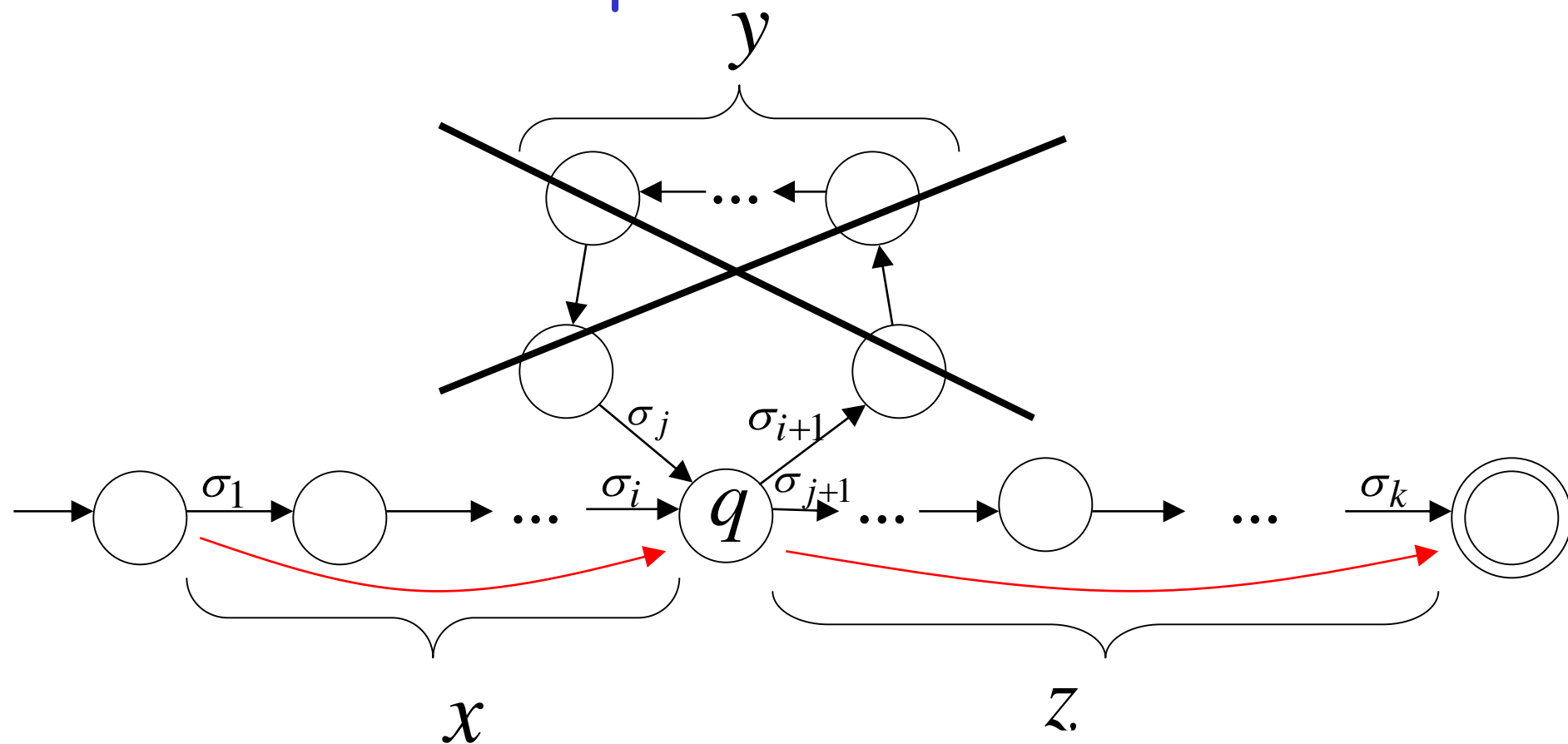
We do not care about the form of string z .

z may actually overlap with the paths of x and y



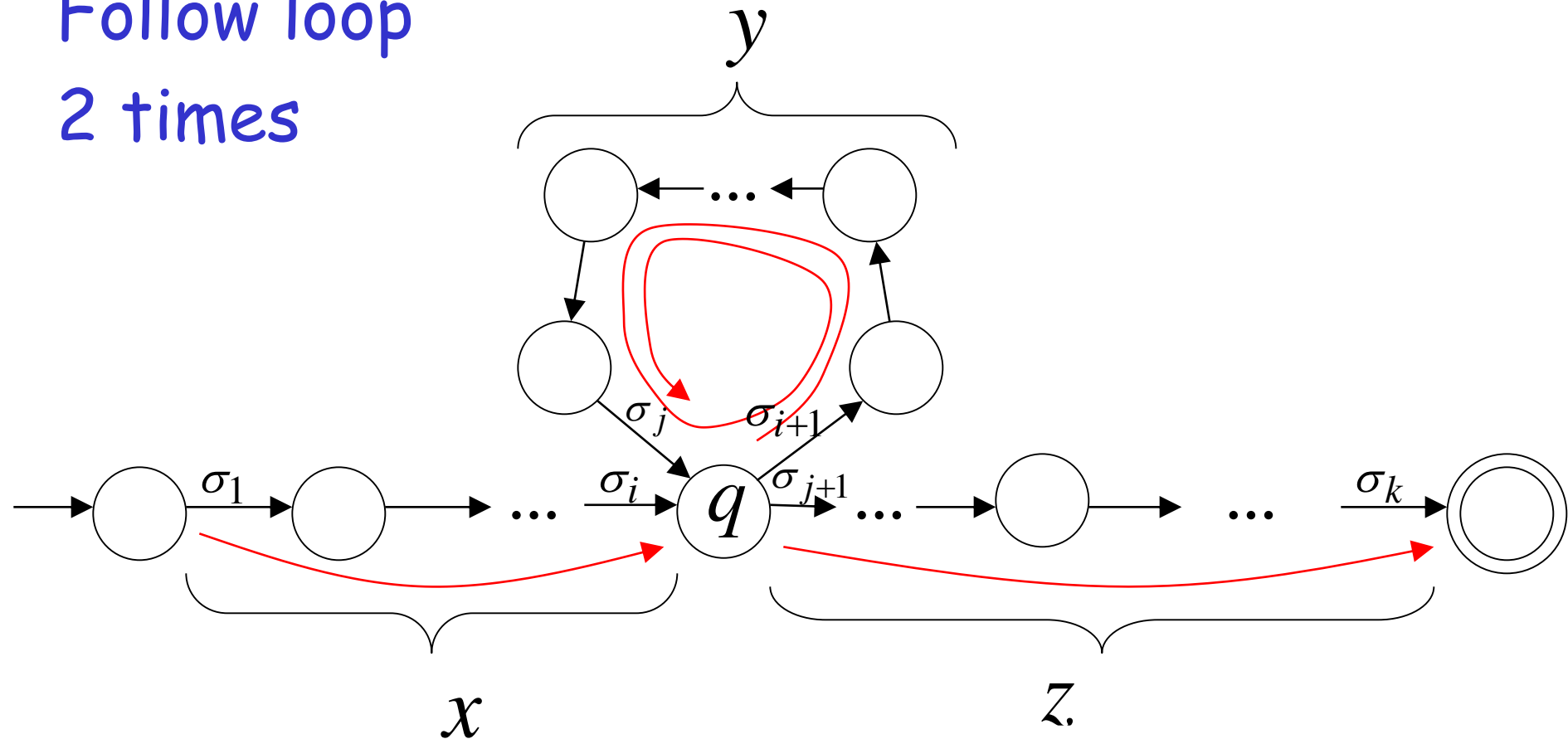
Additional string: The string xz is accepted

Do not follow loop



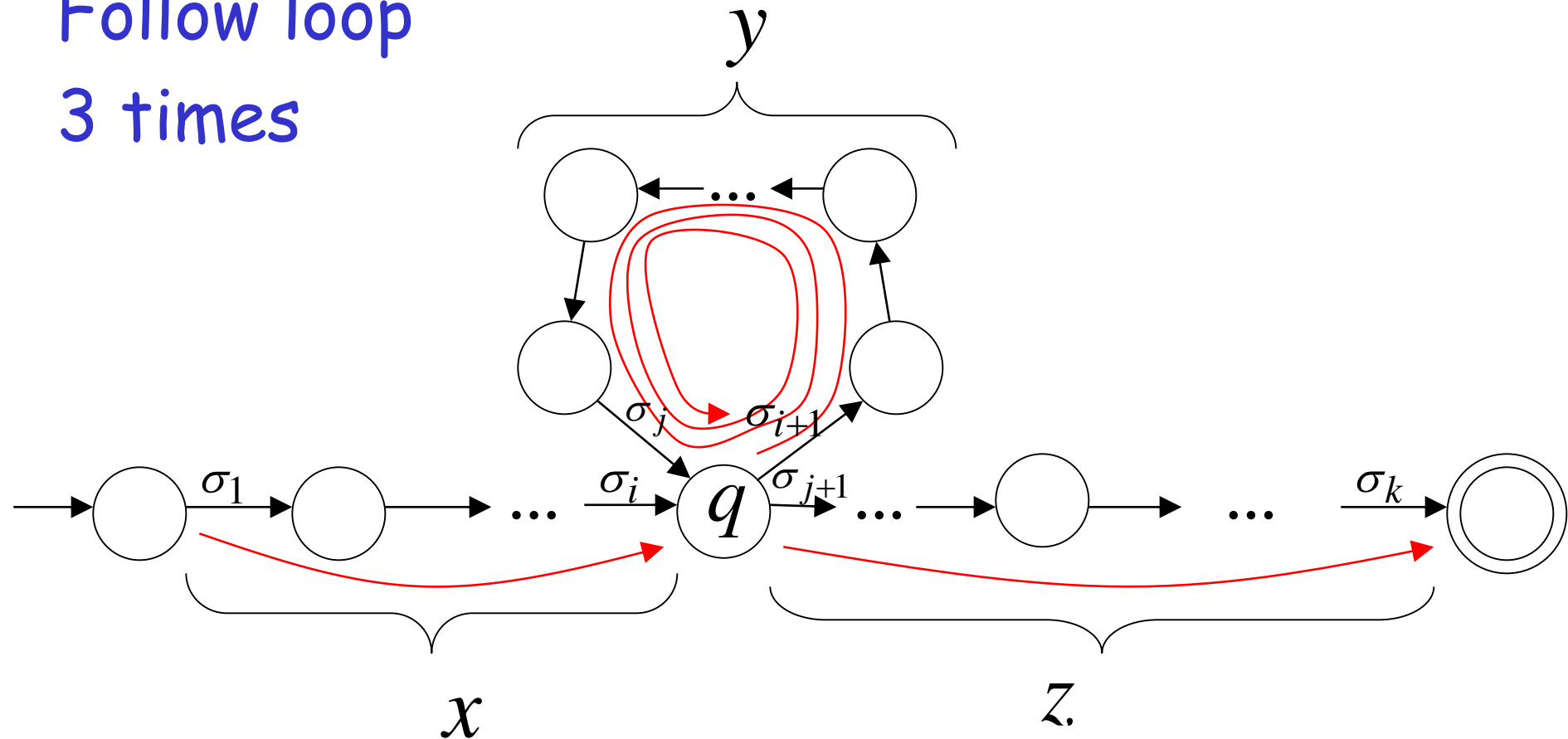
Additional string: The string $x y y z$ is accepted

Follow loop
2 times



Additional string: The string $x y y y z$ is accepted

Follow loop
3 times



In General:

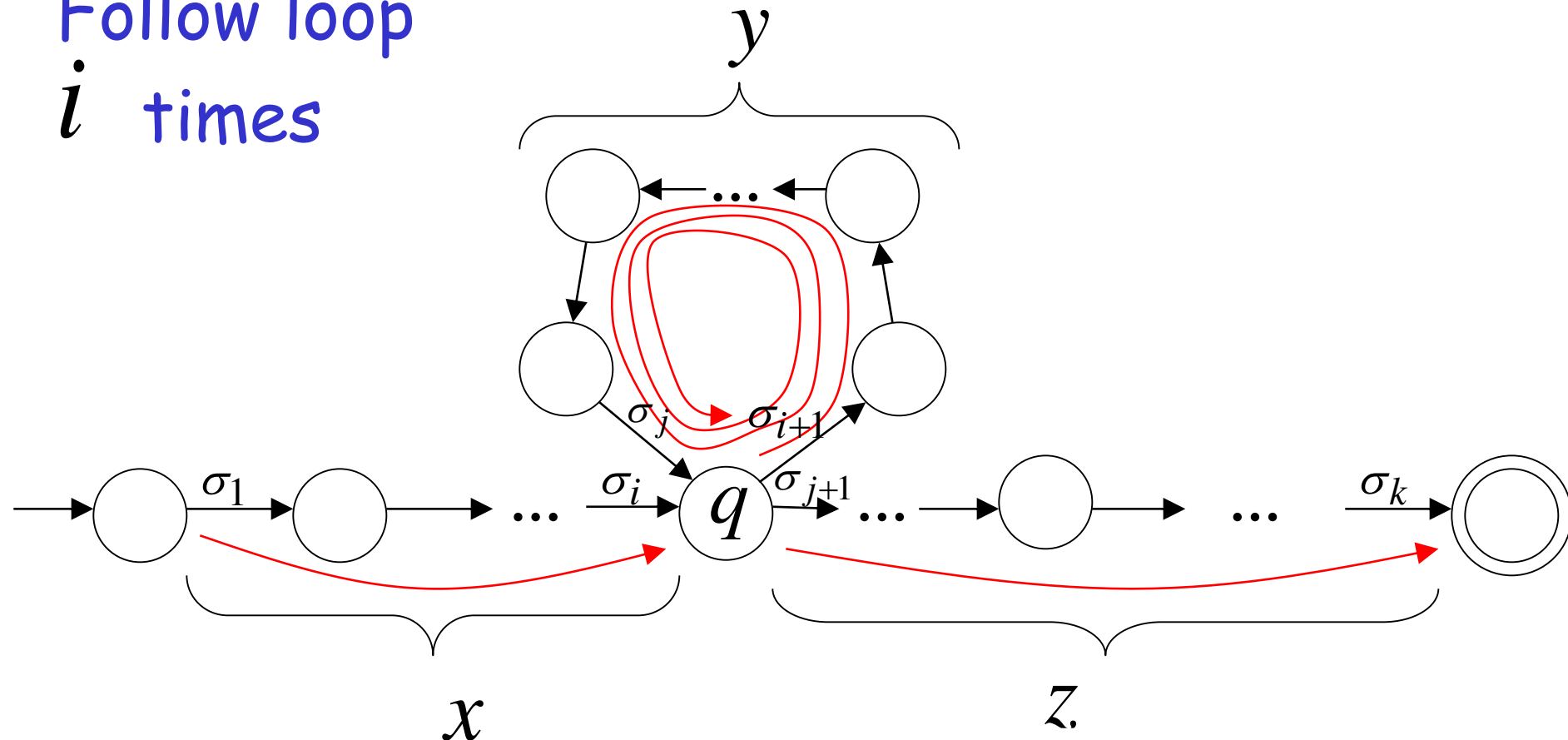
The string

$x y^i z$

is accepted

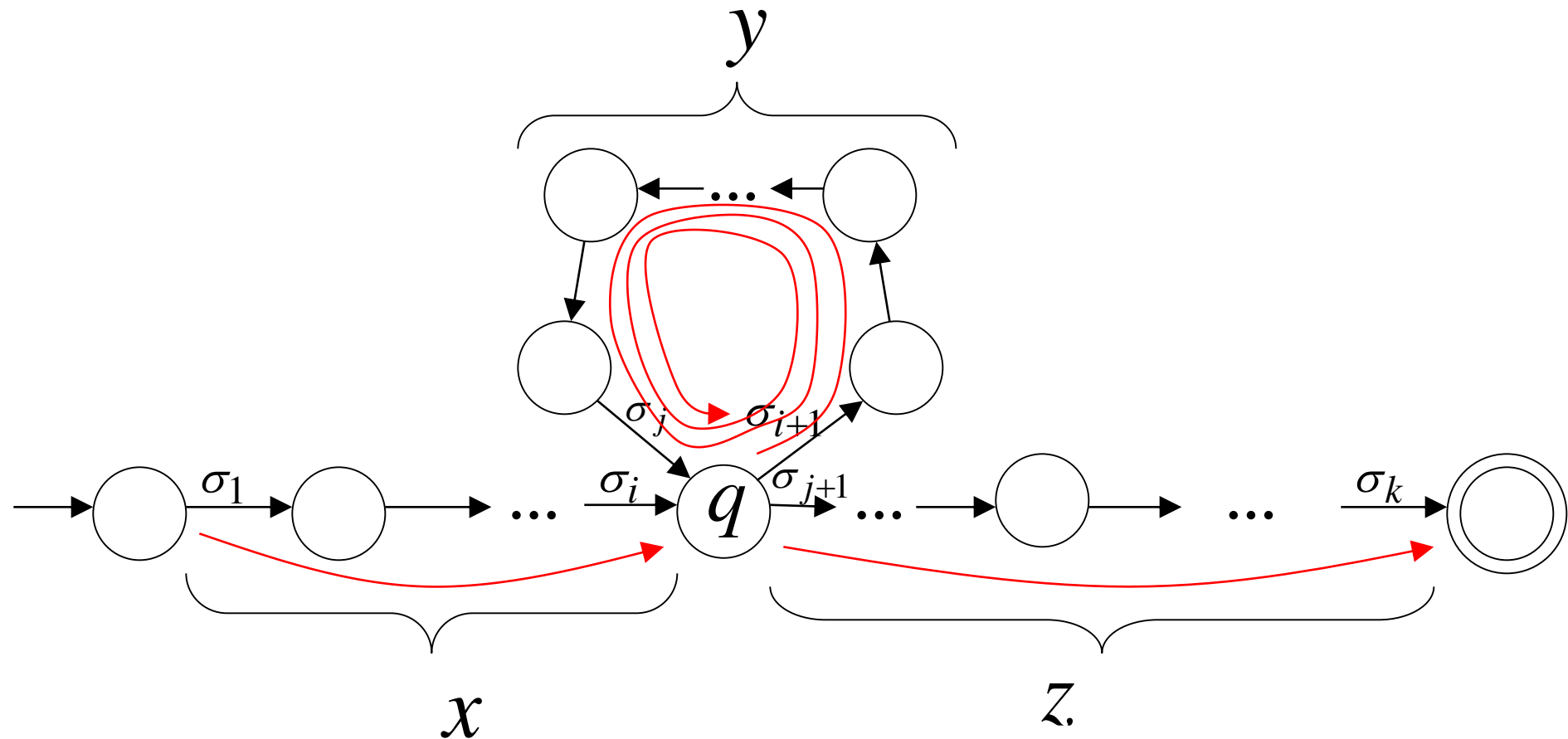
$i = 0, 1, 2, \dots$

Follow loop
 i times

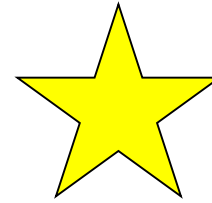
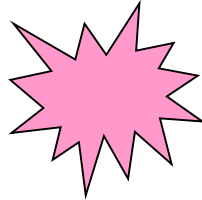


Therefore: $x y^i z \in L \quad i = 0, 1, 2, \dots$

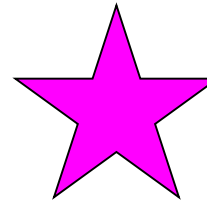
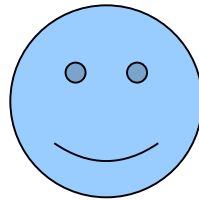
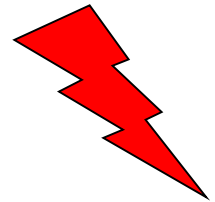
Language accepted by the DFA



In other words, we described:



The Pumping Lemma !!!



The Pumping Lemma:

- Given a infinite regular language L
- there exists an integer m (critical length)
- for any string $w \in L$ with length $|w| \geq m$
- we can write $w = x y z$
- with $|x y| \leq m$ and $|y| \geq 1$
- such that: $x y^i z \in L \quad i = 0, 1, 2, \dots$

In the book:

Critical length m = Pumping length p

Applications of the Pumping Lemma

Observation:

Every language of finite size has to be regular

(we can easily construct an NFA
that accepts every string in the language)

Therefore, every non-regular language
has to be of infinite size

(contains an infinite number of strings)

→ finite
state language

Suppose you want to prove that
An infinite language L is not regular

1. Assume the opposite: L is regular
2. The pumping lemma should hold for L
3. Use the pumping lemma to obtain a contradiction
 $\hookrightarrow$ L is not regular
4. Therefore, L is not regular

Explanation of Step 3: How to get a contradiction

1. Let m be the critical length for L
2. Choose a particular string $w \in L$ which satisfies the length condition $|w| \geq m$
3. Write $w = xyz$
not disjoint, desirable
4. Show that $w' = xy^i z \notin L$ for some $i \neq 1$
5. This gives a contradiction, since from pumping lemma $w' = xy^i z \in L$

Note: It suffices to show that
only one string $w \in L$
gives a contradiction

You don't need to obtain
contradiction for every $w \in L$

Example of Pumping Lemma application

Theorem: The language $L = \{a^n b^n : n \geq 0\}$
is not regular

$$a^m b^m$$

$$xyz = aabbb$$

$$y = a$$

$$xyz^4 = a^{m+3} b^m \neq a^n b^n$$

Proof: Use the Pumping Lemma

$$L = \{a^n b^n : n \geq 0\}$$

xyz

$a^n b^n \neq a^i b^i$

$aa \underbrace{bb}_2$

$x \rightarrow a$

$y \rightarrow a$

Assume for **contradiction**
 that L is a regular language

a
 $xyz \notin a^n b^n$

$aaabb \notin a^n b^n$

Since L is **infinite**
 we can apply the **Pumping Lemma**

$$L = \{a^n b^n : n \geq 0\}$$

Let m be the critical length for L

Pick a string w such that: $w \in L$

and length $|w| \geq m$

We pick $w = a^m b^m$

From the Pumping Lemma:

we can write $w = a^m b^m = x y z$

with lengths $|x y| \leq 2m$, $|y| \geq 1$

$$w = xyz = a^m b^m = \underbrace{a \dots a}_{m \text{ times}} \underbrace{a \dots a}_{m \text{ times}} \underbrace{b \dots b}_{m \text{ times}}$$

$x \quad y \quad z$

Thus: $y = a^k$, $1 \leq k \leq m$

$$x y z = a^m b^m$$

$$y = a^k, \quad 1 \leq k \leq m$$

From the Pumping Lemma: $x y^i z \in L$
 $i = 0, 1, 2, \dots$

Thus: $x y^2 z \in L$

$$x y z = a^m b^m \quad y = a^k, \quad 1 \leq k \leq m$$

From the Pumping Lemma: $x y^2 z \in L$

$$xy^2z = \overbrace{a \dots a a \dots a a \dots a a \dots a}^{m+k} \overbrace{b \dots b}^m \in L$$

$\underbrace{\hspace{1.5cm}}_x \quad \underbrace{\hspace{1.5cm}}_y \quad \underbrace{\hspace{1.5cm}}_y \quad \underbrace{\hspace{3.5cm}}_z$

Thus: $a^{m+k} b^m \in L$

$$a^{m+k}b^m \in L \quad k \geq 1$$

BUT: $L = \{a^n b^n : n \geq 0\}$



$$a^{m+k}b^m \notin L$$

CONTRADICTION!!!

Therefore: Our assumption that L
is a regular language is not true

Conclusion: L is not a regular language

END OF PROOF

Non-regular language $\{a^n b^n : n \geq 0\}$

Regular languages

$$L(a^* b^*)$$

More Applications of the Pumping Lemma

The Pumping Lemma:

- Given a infinite regular language L
- there exists an integer m (critical length)
- for any string $w \in L$ with length $|w| \geq m$
- we can write $w = x y z$
- with $|x y| \leq m$ and $|y| \geq 1$
- such that: $x y^i z \in L \quad i = 0, 1, 2, \dots$

Non-regular languages

$$L = \{vv^R : v \in \Sigma^*\}$$

$a^m b^m b^m a^m$

Regular languages

$xyz = a^m b^m a^m$

$y = a$

$xyz = a^m b^m a^m$

Theorem: The language

$$L = \{vv^R : v \in \Sigma^*\} \quad \Sigma = \{a, b\}$$

is not regular

Proof: Use the Pumping Lemma

$$L = \{vv^R : v \in \Sigma^*\}$$

Assume for contradiction
that L is a regular language

Since L is infinite
we can apply the Pumping Lemma

$$L = \{vv^R : v \in \Sigma^*\}$$

Let m be the critical length for L

Pick a string w such that: $w \in L$

$$a^n a^m$$

$$\underbrace{a}_x \underbrace{a}_y \underbrace{a^2}_z$$

$$a^{m+k} a^m \neq a^{2m} \text{ while } k \text{ odd}$$

and length $|w| \geq m$

We pick $w = a^m b^m b^m a^m$

From the Pumping Lemma:

we can write: $w = a^m b^m b^m a^m = x y z$

with lengths: $|x y| \leq m, |y| \geq 1$

$$w = xyz = \underbrace{a \dots a}_{x} \underbrace{a \dots a}_{y} \underbrace{a \dots a \dots b \dots b \dots b \dots a \dots a}_{z}$$

$\begin{matrix} m & m & m & m \\ \text{---} & \text{---} & \text{---} & \text{---} \\ & & & \end{matrix}$

Thus: $y = a^k, 1 \leq k \leq m$

$$x y z = a^m b^m b^m a^m \quad y = a^k, \quad 1 \leq k \leq m$$

From the Pumping Lemma: $x y^i z \in L$
 $i = 0, 1, 2, \dots$

Thus: $x y^2 z \in L$

$$x y z = a^m b^m b^m a^m \quad y = a^k, \quad 1 \leq k \leq m$$

From the Pumping Lemma: $x y^2 z \in L$

$$xy^2z = \overbrace{a \dots a}^{m+k} \overbrace{a \dots a}^m \overbrace{a \dots a}^m \overbrace{a \dots a}^m \in L$$

$\underbrace{\hspace{1.5cm}}_x \quad \underbrace{\hspace{1.5cm}}_y \quad \underbrace{\hspace{1.5cm}}_y \quad \underbrace{\hspace{4cm}}_z$

Thus: $a^{m+k} b^m b^m a^m \in L$

$$a^{m+k}b^mb^ma^m \in L \quad k \geq 1$$

BUT: $L = \{vv^R : v \in \Sigma^*\}$



$$a^{m+k}b^mb^ma^m \notin L$$

CONTRADICTION!!!

Therefore: Our assumption that L
is a regular language is not true

Conclusion: L is not a regular language

END OF PROOF

Non-regular languages

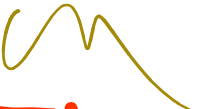
$$L = \{a^n b^l c^{n+l} : n, l \geq 0\}$$

$a^m b^m c^{2m} \rightarrow \text{not in } L$

$y = a$

Regular languages

$a^{m+k} b^m c^{2m} \neq a^m b^m c^{2m}$

 **Theorem:** The language

$$L = \{a^n b^l c^{n+l} : n, l \geq 0\}$$

is not regular

Handwritten notes:
has a
pumping
...
once
...
...
...
...
...
...

Proof: Use the Pumping Lemma

$$L = \{a^n b^l c^{n+l} : n, l \geq 0\}$$

Assume for contradiction
that L is a regular language

Since L is infinite
we can apply the Pumping Lemma

$$L = \{a^n b^l c^{n+l} : n, l \geq 0\}$$

Let m be the critical length of L

Pick a string w such that: $w \in L$ and

$$\text{length } |w| \geq m$$

We pick $w = a^m b^m c^{2m}$

From the Pumping Lemma:

We can write $w = a^m b^m c^{2m} = x y z$

With lengths $|x y| \leq m, |y| \geq 1$

$$w = xyz = \overbrace{a \dots a}^m \overbrace{a \dots a}^m \overbrace{a \dots a}^{2m} \overbrace{b \dots b}^m \overbrace{c \dots c}^{2m}$$

$\underbrace{\hspace{1.5cm}}_x \underbrace{\hspace{1.5cm}}_y \underbrace{\hspace{4.5cm}}_z$

Thus: $y = a^k, 1 \leq k \leq m$

$$x y z = a^m b^m c^{2m}$$

$$y = a^k, \quad 1 \leq k \leq m$$

From the Pumping Lemma: $x y^i z \in L$
 $i = 0, 1, 2, \dots$

Thus: $x y^0 z = xz \in L$

$$x y z = a^m b^m c^{2m} \quad y = a^k, \quad 1 \leq k \leq m$$

From the Pumping Lemma: $xz \in L$

$$xz = \overbrace{a \dots a}^{m+k} \overbrace{b \dots b}^m \overbrace{c \dots c}^{2m} \in L$$

$$\underbrace{a \dots a}_x \underbrace{a \dots a b \dots b c \dots c}_z$$

Thus: $a^{m+k} b^m c^{2m} \in L$

$$a^{m+k}b^mc^{2m} \in L \quad k \geq 1$$

BUT: $L = \{a^n b^l c^{n+l} : n, l \geq 0\}$



$$a^{m+k}b^mc^{2m} \notin L$$

CONTRADICTION!!!

Therefore: Our assumption that L
is a regular language is not true

Conclusion: L is not a regular language

END OF PROOF

Non-regular languages

$$L = \{a^{n!} : n \geq 0\}$$

Regular languages

[illegible]

Theorem: The language $L = \{a^{n!} : n \geq 0\}$
is not regular

$$n! = 1 \cdot 2 \cdots (n-1) \cdot n$$

Proof: Use the Pumping Lemma

$$L = \{a^{n!} : n \geq 0\}$$

Assume for contradiction
that L is a regular language

Since L is infinite
we can apply the Pumping Lemma

$$L = \{a^{n!} : n \geq 0\}$$

Let m be the critical length of L

Pick a string w such that: $w \in L$

length $|w| \geq m$

We pick $w = a^{m!}$

From the Pumping Lemma:

We can write $w = a^{m!} = x y z$

With lengths $|x y| \leq m, |y| \geq 1$

$$w = xyz = a^{m!} = \overbrace{a \dots a}^m \overbrace{a \dots a}^{m!-m}$$

The diagram illustrates the decomposition of the string $w = a^{m!}$ into xyz . The string is represented as a sequence of $m!$ 'a' characters. A bracket above the first m characters is labeled m , and a bracket above the remaining $m! - m$ characters is labeled $m! - m$. Below the string, three brackets indicate the decomposition: x covers the first part, y covers the second part, and z covers the third part. The length of xy is at most m , and the length of y is at least 1.

Thus: $y = a^k, 1 \leq k \leq m$

$$x y z = a^{m!}$$

$$y = a^k, \quad 1 \leq k \leq m$$

From the Pumping Lemma: $x y^i z \in L$
 $i = 0, 1, 2, \dots$

Thus: $x y^2 z \in L$

$$x y z = a^{m!}$$

$$y = a^k, \quad 1 \leq k \leq m$$

From the Pumping Lemma: $x y^2 z \in L$

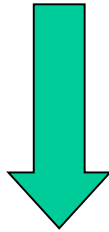
$$xy^2z = \overbrace{a \dots a a \dots a a \dots a a \dots a a \dots a a \dots a}^{m+k} \overbrace{a \dots a}^{m!-m} \in L$$

$\underbrace{\hspace{1.5cm}}_x \quad \underbrace{\hspace{1.5cm}}_y \quad \underbrace{\hspace{1.5cm}}_y \quad \underbrace{\hspace{4cm}}_z$

Thus: $a^{m!+k} \in L$

$$a^{m!+k} \in L \quad 1 \leq k \leq m$$

Since: $L = \{a^{n!} : n \geq 0\}$



There must exist p such that:

$$m!+k = p!$$

However: $m!+k \leq m!+m$ for $m > 1$

$$\leq m!+m!$$

$$< m!m + m!$$

$$= m!(m+1)$$

$$= (m+1)!$$



$$m!+k < (m+1)!$$



$$m!+k \neq p! \quad \text{for any } p$$

$$a^{m!+k} \in L \quad 1 \leq k \leq m$$

BUT: $L = \{a^{n!} : n \geq 0\}$



$$a^{m!+k} \notin L$$

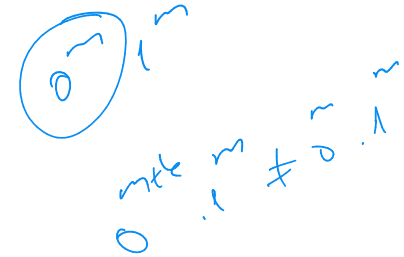
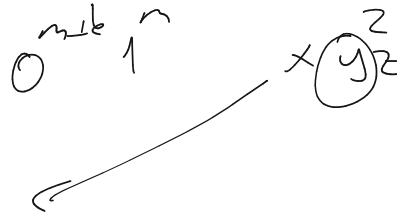
CONTRADICTION!!!

Therefore: Our assumption that L
is a regular language is not true

Conclusion: L is not a regular language

END OF PROOF

Example 2



Theorem: $B = \{0^n 1^n \mid n \geq 0\}$ is not regular.

Proof. Assume the contrary that B is regular.

Let p be the pumping length. Then, $s = 0^p 1^p$ can be decomposed as $s = xyz$ with $|y| \geq 1$, and $xy^n z \in B$ for any $n \geq 0$.

There can be three cases $y = 0^k$, $y = 0^k 1^l$, and $y = 1^l$, for some nonzero k, l . In each case, we can easily see that $xy^n z \notin B$, which leads to contradiction. ($n \geq 2$)

Example 3

0 1



Theorem: $C = \{w \mid w \text{ has an equal number of 0s and 1s}\}$
is not regular.

Proof. Assume the contrary that C is regular.

Let p be the pumping length. Then, $s = 0^p 1^p$ can be decomposed as $s = xyz$ with $|y| \geq 1$ and $|xy| \leq p$, and $xy^n z \in C$ for any $n \geq 0$.

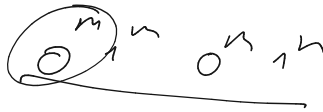
Then we must have $y = 0^k$ for some nonzero k .

We can easily see that $xy^n z \notin C$, which contradicts the assumption.

Alternative proof of Example 3

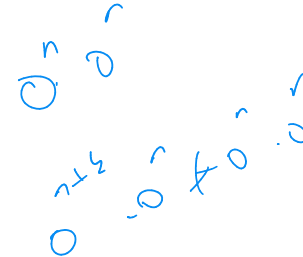
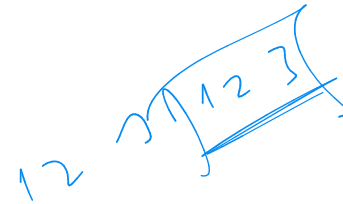
- The class of regular languages is closed under the intersection operation. This is easy to prove if we run two DFAs parallelly and accept only string which are accepted by both of the DFAs.
- Now, assume C is regular.
- Then, $C \cap 0^*1^* = B$ is also regular.
- This contradicts what we proved in Example 1.38.???

Example 4



$$a^m a^m$$

$$a^{m+k} a^m \neq a^{k+m} \text{ for } k \text{ odd}$$



Theorem: $F = \{ww \mid w \in \{0,1\}^*\}$ is not regular.

Proof. Assume the contrary that F is regular.

Let p be the pumping length and let $s = \underline{0^p 1} \underline{0^p 1} \in F$. This s can be split into pieces like $s = xyz$ with $|y| \geq 1$ and $|xy| \leq p$, and $xy^n z \in F$ for any $n \geq 0$.

Then we must have $y = 0^k$ for some nonzero k .

We can easily see that $xy^n z \notin F$, which contradicts the assumption.

Example 5

$$1^{n^2+k}$$

$$s = 1^{n^2+k} \neq 1^{n^2}$$

Theorem : $D = \{1^{n^2} \mid n \geq 0\}$ is not regular.

Proof. Assume the contrary that D is regular.

Let p be the pumping length and let $s = 1^{p^2} \in D$. This s can be split into pieces like $s = xyz$ with $|y| \geq 1$ and $|xy| \leq p$, and $xy^n z \in D$ for any $n \geq 0$.

The length of $xy^n z$ grows linearly with n , while the lengths of strings in D grows as $0, 1, 4, 9, 16, 25, 36, 49, \dots$

These two facts are incompatible as can be easily seen.

Example 6 (Pumping Down)

Theorem: $E = \{0^i 1^j \mid i > j\}$ is not regular.

Proof. Assume the contrary that E is regular.

Let p be the pumping length and let $s = \underline{0^{p+1}} 1^p$. This s can be split into $s = xyz$ with $|y| \geq 1$ and $|xy| \leq p$, and $xy^n z \in E$ for any $n \geq 0$.

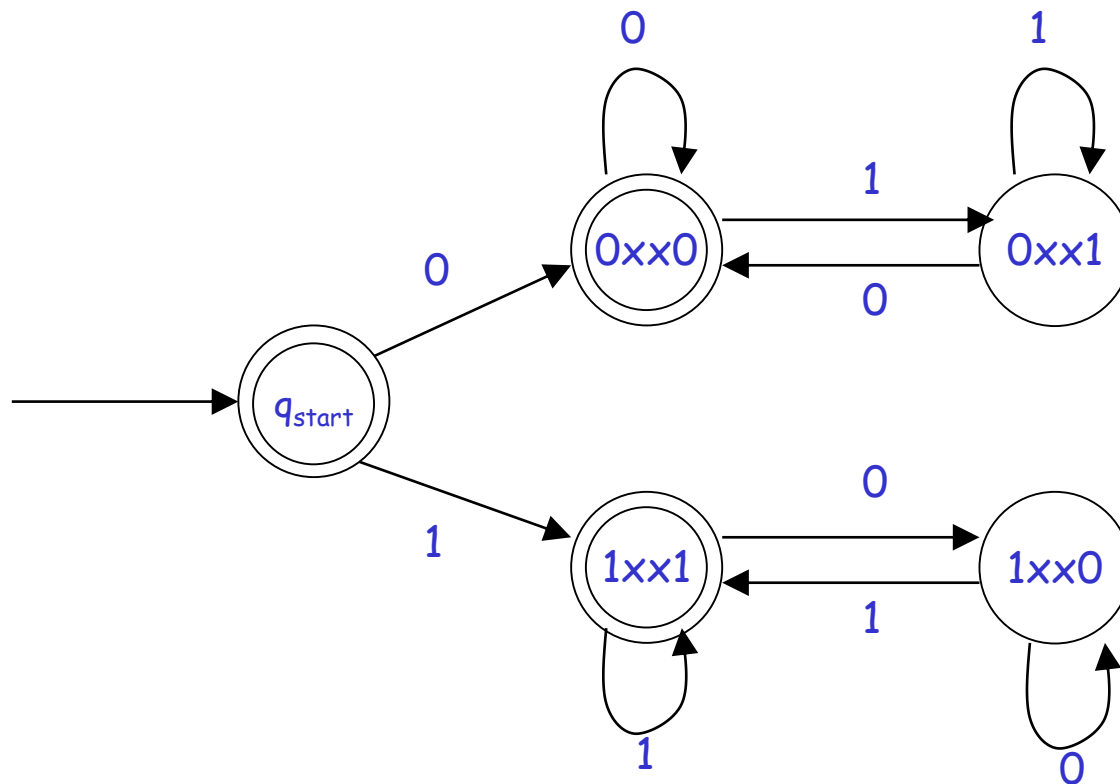
Then we must have $y = 0^k$ for some nonzero k .

We can easily see that $xy^0 z = xz \notin E$, which contradicts the assumption.



Example 6 (Differential Encoding)

Claim : $D = \{w \mid w \text{ contains equal number of occurrences of the substrings } 01 \text{ and } 10\}$ is regular.





BLM2502

Theory of

Computation

Spring 2015

BLM2502 Theory of Computation

» Course Outline

» Week	Content
» 1	Introduction to Course
» 2	Computability Theory, Complexity Theory, Automata Theory, Set Theory, Relations, Proofs, Pigeonhole Principle
» 3	Regular Expressions
» 4	Finite Automata
» 5	Deterministic and Nondeterministic Finite Automata
» 6	Epsilon Transition, Equivalence of Automata
» 7	Pumping Theorem
» 8	April 10 - 14 week is the first midterm week
» 9	Context Free Grammars
» 10	Parse Tree, Ambiguity,
» 11	Pumping Theorem
» 12	Turing Machines, Recognition and Computation, Church-Turing Hypothesis
» 13	Turing Machines, Recognition and Computation, Church-Turing Hypothesis
» 14	May 22 – 27 week is the second midterm week
» 15	Review
» 16	Final Exam date will be announced



Context-Free Languages

»

Context-Free Languages

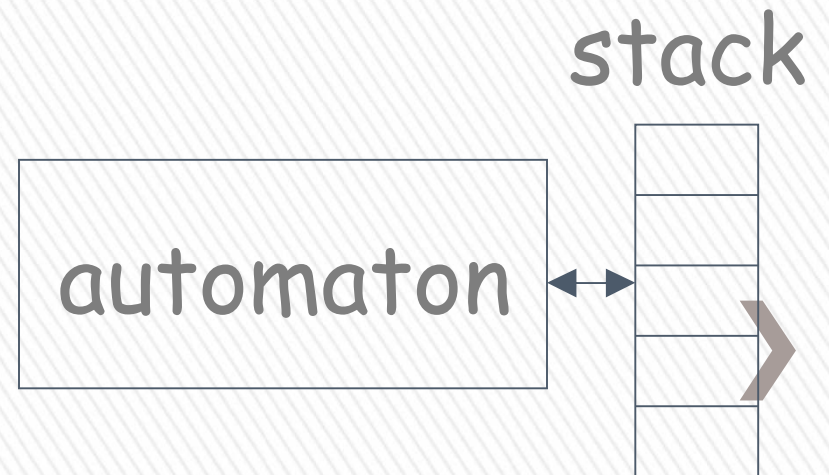
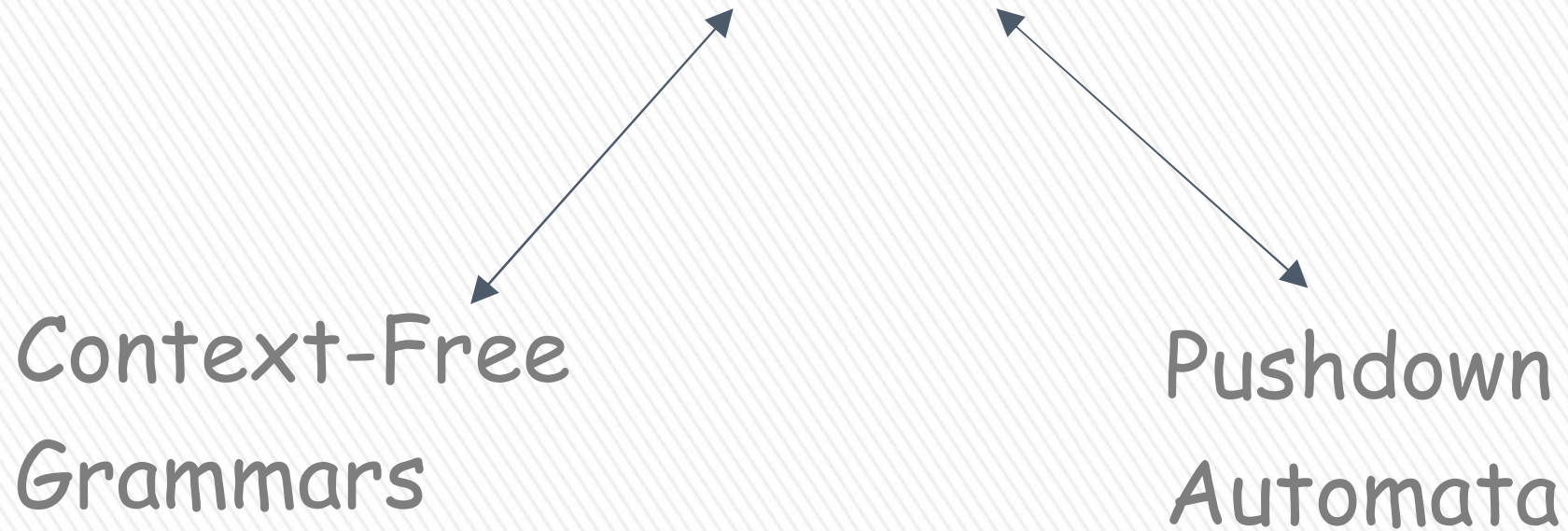
$$\{\underline{a}^n b^n : n \geq 0\} \quad \{\underline{w} w^R\}$$

Regular Languages

$$a^* b^* \quad (a + b)^*$$



Context-Free Languages





Context-Free Grammars

Grammars

- » Grammars express languages
- » Example: the English language grammar

$\langle \textit{sentence} \rangle \rightarrow \langle \textit{noun_phrase} \rangle \langle \textit{predicate} \rangle$

$\langle \textit{noun_phrase} \rangle \rightarrow \langle \textit{article} \rangle \langle \textit{noun} \rangle$

$\langle \textit{predicate} \rangle \rightarrow \langle \textit{verb} \rangle$

inglese grammi



$\langle \textit{article} \rangle \rightarrow a$

$\langle \textit{article} \rangle \rightarrow \textit{the}$

$\langle \textit{noun} \rangle \rightarrow \textit{cat}$

$\langle \textit{noun} \rangle \rightarrow \textit{dog}$

$\langle \textit{verb} \rangle \rightarrow \textit{runs}$

$\langle \textit{verb} \rangle \rightarrow \textit{sleeps}$



» Derivation of string “the dog walks” :

$\langle sentence \rangle \Rightarrow \langle noun_phrase \rangle \langle predicate \rangle$
 $\Rightarrow \langle noun_phrase \rangle \langle verb \rangle$
 $\Rightarrow \langle article \rangle \langle noun \rangle \langle verb \rangle$
 $\Rightarrow the \langle noun \rangle \langle verb \rangle$
 $\Rightarrow the \ dog \langle verb \rangle$
 $\Rightarrow the \ dog \ sleeps$



» Derivation of string “a cat runs” :

$\langle sentence \rangle \Rightarrow \langle noun_phrase \rangle \langle predicate \rangle$
 $\Rightarrow \langle noun_phrase \rangle \langle verb \rangle$
 $\Rightarrow \langle article \rangle \langle noun \rangle \langle verb \rangle$
 $\Rightarrow a \langle noun \rangle \langle verb \rangle$
 $\Rightarrow a \ cat \langle verb \rangle$
 $\Rightarrow a \ cat \ runs$



» Language of the grammar:

$$L = \{ \text{"a cat runs"}, \\ \text{"a cat sleeps"}, \\ \text{"the cat runs"}, \\ \text{"the cat sleeps"}, \\ \text{"a dog runs"}, \\ \text{"a dog sleeps"}, \\ \text{"the dog runs"}, \\ \text{"the dog sleeps"} \}$$


Productions

Sequence of
Terminals (symbols)

$\langle \text{noun} \rangle \rightarrow \text{cat}$

$\langle \text{sentence} \rangle \rightarrow \langle \text{noun_phrase} \rangle \langle \text{predicate} \rangle$

Variables
└─> non-terminal

Sequence of Variables



Another Example

Sequence of
terminals and variables

Grammar:

$$S \rightarrow \overbrace{aSb} \mid \varepsilon$$

Handwritten notes: aSb and ε are circled in red.

$$S \rightarrow \varepsilon$$

Handwritten notes: aSb and $a a \cancel{S} b b$ are written in red.

Variable

The right side
may be ε



» Grammar:

$$S \rightarrow aSb$$

$$S \rightarrow \varepsilon$$

Handwritten notes:
 aS
 aSb
 ab

» Derivation of string : ab

$$S \Rightarrow aSb \Rightarrow ab$$

$$S \rightarrow aSb$$

$$S \rightarrow \varepsilon$$



» Grammar:

$$S \rightarrow aSb$$

$$S \rightarrow \varepsilon$$

» Derivation of string :

aabb

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$$



$$S \rightarrow aSb$$



$$S \rightarrow \varepsilon$$



Grammar: $S \rightarrow aSb$

$S \rightarrow \varepsilon$

Other derivations:

$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaaSbbb \Rightarrow aaabbbb$

$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaaSbbb$
 $\Rightarrow aaaaSbbbb \Rightarrow aaabbbbb$



Grammar:

$$S \rightarrow aSb$$

$$S \rightarrow \varepsilon$$

Language of the grammar:

$$L = \{\underline{a}^n b^n : n \geq 0\}$$



A Convenient Notation

» We write: $S \xRightarrow{*} aaabbb$

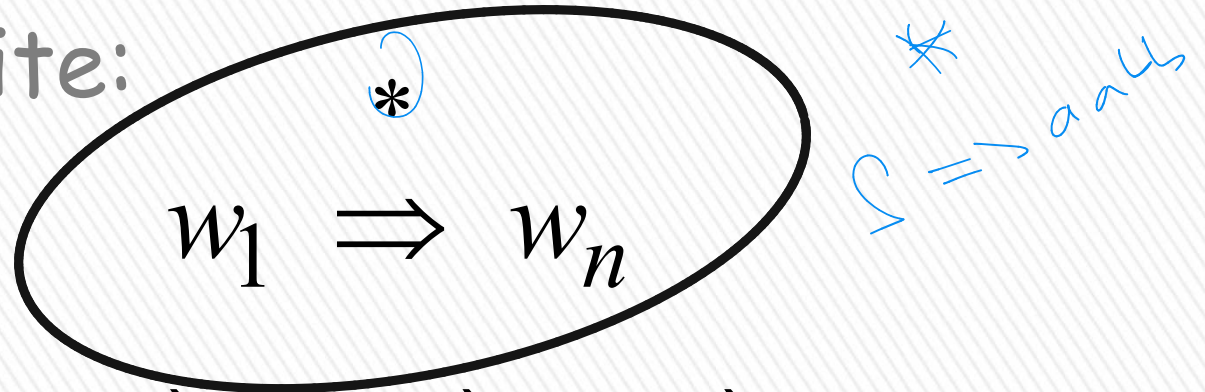
for one or more derivation steps

» Instead of:

$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaasbbb \Rightarrow aaabbb$



In general we write:

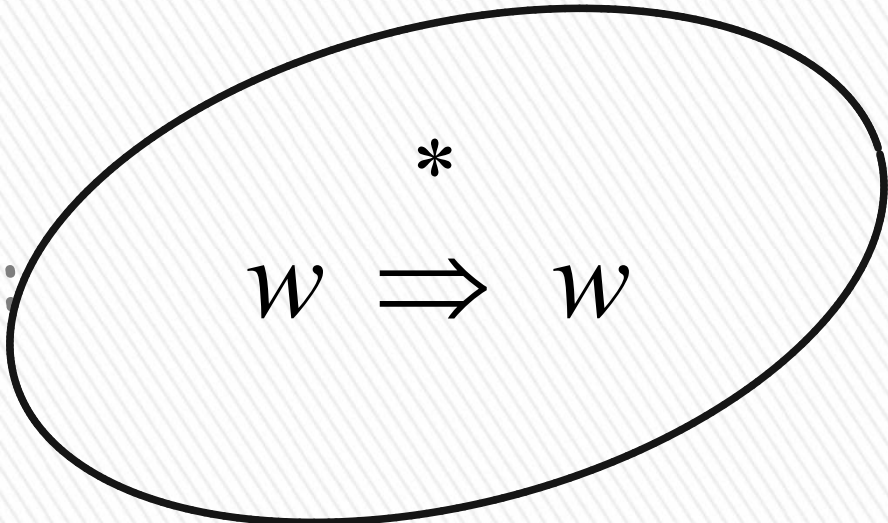

$$w_1 \Rightarrow w_n$$

If:

$$w_1 \Rightarrow w_2 \Rightarrow w_3 \Rightarrow \cdots \Rightarrow w_n$$

in zero or more derivation steps

Trivially:


$$w \Rightarrow w$$



Example Grammar

Possible Derivations

$$S \rightarrow aSb$$

$$S \rightarrow \varepsilon$$

$$S \xRightarrow{*} \varepsilon$$

$$S \xRightarrow{*} ab$$

$$S \xRightarrow{*} aaabbbb$$

$$S \xRightarrow{*} aaSbb \xRightarrow{*} aaaaaaSbbbbbb$$



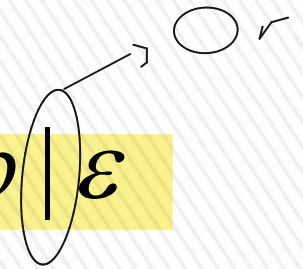
Another convenient notation:

$$S \rightarrow aSb \mid \varepsilon$$

$$S \rightarrow \varepsilon$$



$$S \rightarrow aSb \mid \varepsilon$$



$$\langle \text{article} \rangle \rightarrow a$$

$$\langle \text{article} \rangle \rightarrow the$$



$$\langle \text{article} \rangle \rightarrow a \mid the$$



Formal Definition

Grammar:

$$G = (V, T, S, P)$$

Gramerde
oldugu
bircisi
igin
hang
kural
kuralda
serketigini
baslamiz
bilmeniz
luzin

Set of
variables

Set of
terminal
symbols

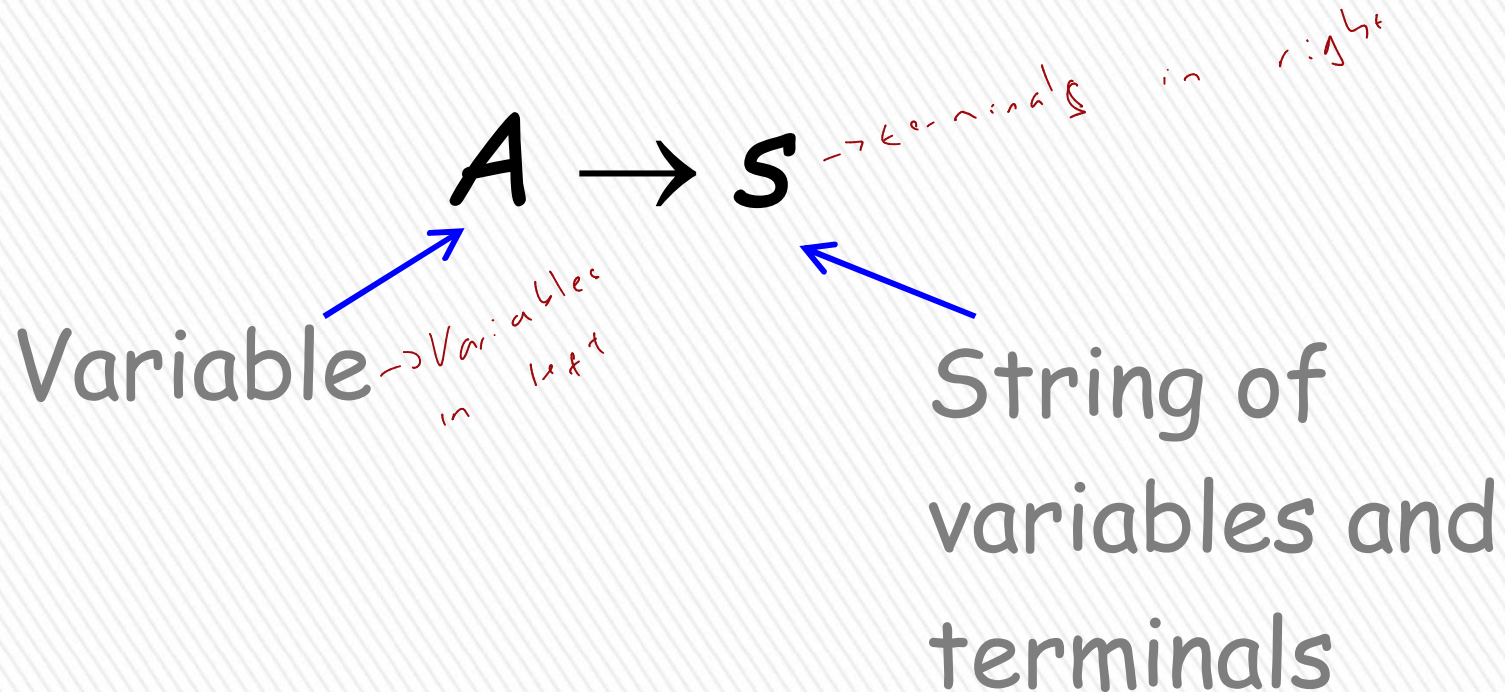
Start
variable

Set of
productions



Context-Free Grammar: $G = (V, T, S, P)$

All productions in P are of the form



Example for Context-Free Grammar

$$S \rightarrow aSb \mid \varepsilon$$

productions

$$P = \{S \rightarrow aSb, S \rightarrow \varepsilon\}$$

$$V = \{S, L, P\}$$

$$T = \{a, b\}$$

$$S = L$$

$$P = \{S \rightarrow aSL, S \rightarrow aSaL, S \rightarrow \varepsilon\}$$

$$G = (V, T, S, P)$$

$$V = \{S\}$$

variables

begin
symbol
half

$$T = \{a, b\}$$

terminals

start variable



Language of a Grammar:

» For a grammar G with start variable S

$$L(G) = \{w : S \Rightarrow w, w \in T^*\}$$

**
terminaller*

String of terminals or ε



Example:

context-free grammar $G : \boxed{S \rightarrow aSb \mid \varepsilon}$

$$L(G) = \{a^n b^n : n \geq 0\}$$

Since, there is derivation

$$S \xRightarrow{*} a^n b^n \quad \text{for any } n \geq 0 \rangle$$

$$L = \{0^*1^*\}$$

Context-Free Language:

regular
Language
Context
Language
all
business
S → ε | 0S | 1S

- » A language L is context-free
- » if there is a context-free grammar G
- » with $L = L(G)$



Example:

$$L = \{a^n b^n : n \geq 0\}$$

is a context-free language

since context-free grammar G :

$$S \rightarrow aSb \mid \varepsilon$$

generates $L(G) = L$



Another Example

Context-free grammar G : → Palindrome

$$S \rightarrow \underline{aSa} \mid \underline{bSb} \mid \underline{\varepsilon} \mid a \mid b$$

Example derivations:

$$S \Rightarrow aSa \Rightarrow abSba \Rightarrow abba$$

$$S \Rightarrow aSa \Rightarrow abSba \Rightarrow abaSaba \Rightarrow abaaba$$

$$L(G) = \{ww^R : w \in \{a,b\}^*\}$$

Palindromes of even length



Another Example

Context-free grammar G :

$$S \rightarrow aSb \mid SS \mid \varepsilon$$

$\rightarrow \varepsilon$ -eulerian



aSb
 $aSSb$
 $aSSSSb$

Example derivations:

$$S \Rightarrow SS \Rightarrow aSbS \Rightarrow abS \Rightarrow ab$$

$$S \Rightarrow SS \Rightarrow aSbS \Rightarrow abS \Rightarrow abaSb \Rightarrow abab$$

$$L(G) = \{w : n_a(w) = n_b(w),$$

$$\text{and } n_a(v) \geq n_b(v)$$

in any prefix v

Describes
matched

parentheses:

$() ((())) (())$ $a = ($, $b =)$



Derivation Trees

Derivation Order

Consider the following example grammar with 5 productions:

- | | | |
|-----------------------|--------------------------------|--------------------------------|
| 1. $S \rightarrow AB$ | 2. $A \rightarrow aaA$ | 4. $B \rightarrow Bb$ |
| | 3. $A \rightarrow \varepsilon$ | 5. $B \rightarrow \varepsilon$ |

$aaA \xrightarrow{S}$



1. $S \rightarrow AB$	2. $A \rightarrow aaA$	4. $B \rightarrow Bb$
	3. $A \rightarrow \varepsilon$	5. $B \rightarrow \varepsilon$

soldan başlıyoruz

Leftmost derivation order of string aab:

$$\begin{array}{ccccccccc}
 1 & & 2 & & 3 & & 4 & & 5 \\
 S & \Rightarrow & AB & \Rightarrow & aaAB & \Rightarrow & aaB & \Rightarrow & aaBb \Rightarrow aab
 \end{array}$$

At each step, we substitute the leftmost variable



① Derivation trees
-ORDER
Left Right

```
graph TD; A["① Derivation trees"] --> B["-ORDER"]; B --> C["Left"]; B --> D["Right"];
```


$$\begin{array}{lll}
 1. S \rightarrow AB & 2. A \rightarrow aaA & 4. B \rightarrow Bb \\
 & 3. A \rightarrow \varepsilon & 5. B \rightarrow \varepsilon
 \end{array}$$

Rightmost derivation order of string *aab*:

$$\begin{array}{cccccc}
 1 & & 4 & & 5 & & 2 & & 3 \\
 S \Rightarrow AB \Rightarrow ABb \Rightarrow Ab \Rightarrow aaAb \Rightarrow aab
 \end{array}$$

At each step, we substitute the
rightmost variable



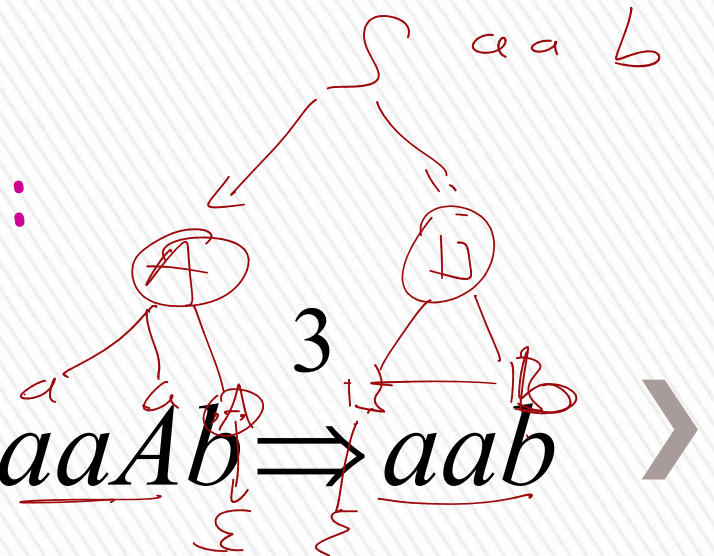
- | | | |
|-----------------------|--|--|
| 1. $S \rightarrow AB$ | 2. $A \rightarrow aaA$ | 4. $B \rightarrow \underline{Bb}$ |
| | 3. $A \rightarrow \underline{\varepsilon}$ | 5. $B \rightarrow \underline{\varepsilon}$ |

Leftmost derivation of aab :

$$\begin{array}{ccccccccc}
 1 & & 2 & & 3 & & 4 & & 5 \\
 S & \Rightarrow & \underline{A}B & \Rightarrow & aa\underline{A}B & \Rightarrow & aa\underline{B} & \Rightarrow & aa\underline{Bb} & \Rightarrow & aab
 \end{array}$$

Rightmost derivation of aab :

$$\begin{array}{ccccccccc}
 1 & & 4 & & 5 & & 2 & & 3 \\
 S & \Rightarrow & A\underline{B} & \Rightarrow & A\underline{Bb} & \Rightarrow & \underline{A}b & \Rightarrow & aa\underline{A}b & \Rightarrow & aab
 \end{array}$$



Consider the same example grammar:

$$S \rightarrow AB \qquad A \rightarrow aaA \mid \varepsilon \qquad B \rightarrow Bb \mid \varepsilon$$

And a derivation of *aab*:

$$S \Rightarrow AB \Rightarrow aaAB \Rightarrow aaABb \Rightarrow aaBb \Rightarrow aab$$

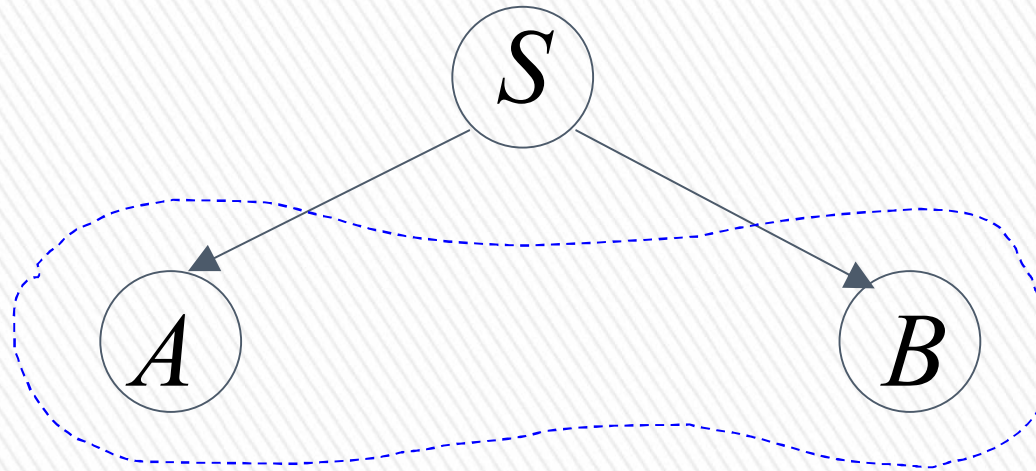


Derivation Tree

$$S \rightarrow AB$$

$$A \rightarrow aaA \mid \varepsilon \quad B \rightarrow Bb \mid \varepsilon$$

$$S \Rightarrow AB$$



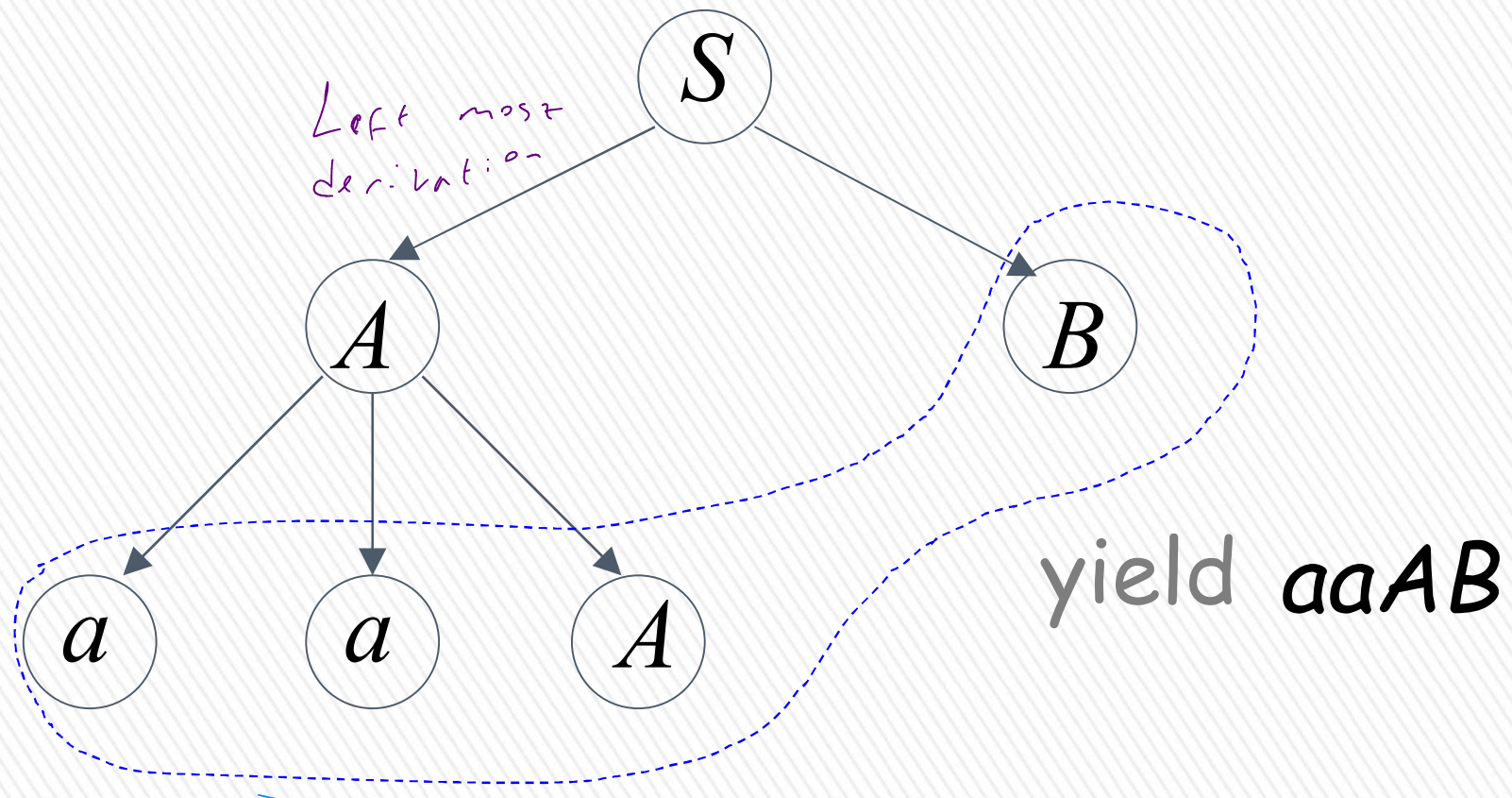
yield AB



$$S \rightarrow AB$$

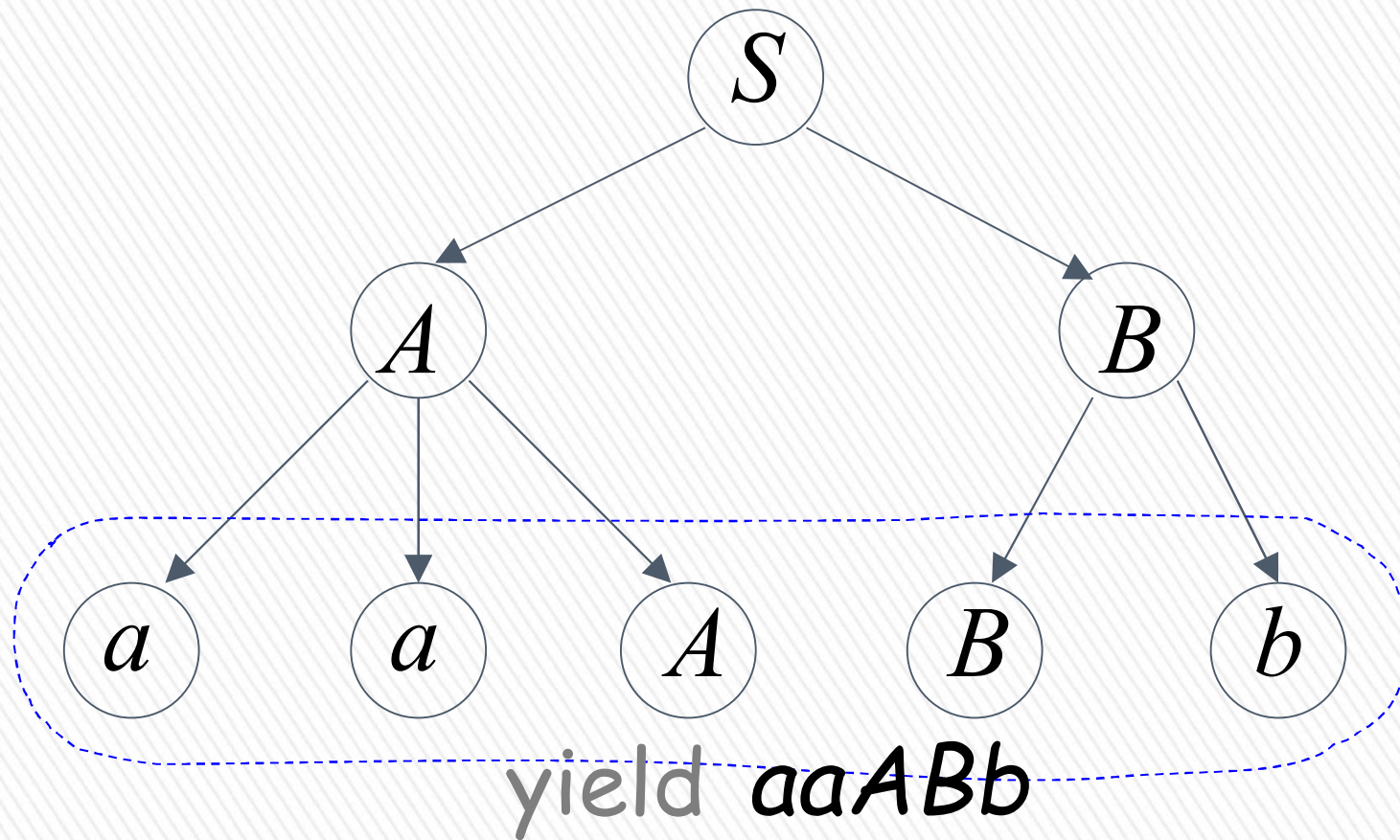
$$A \rightarrow aaA \mid \varepsilon \quad B \rightarrow Bb \mid \varepsilon$$

$$S \Rightarrow AB \Rightarrow aaAB$$



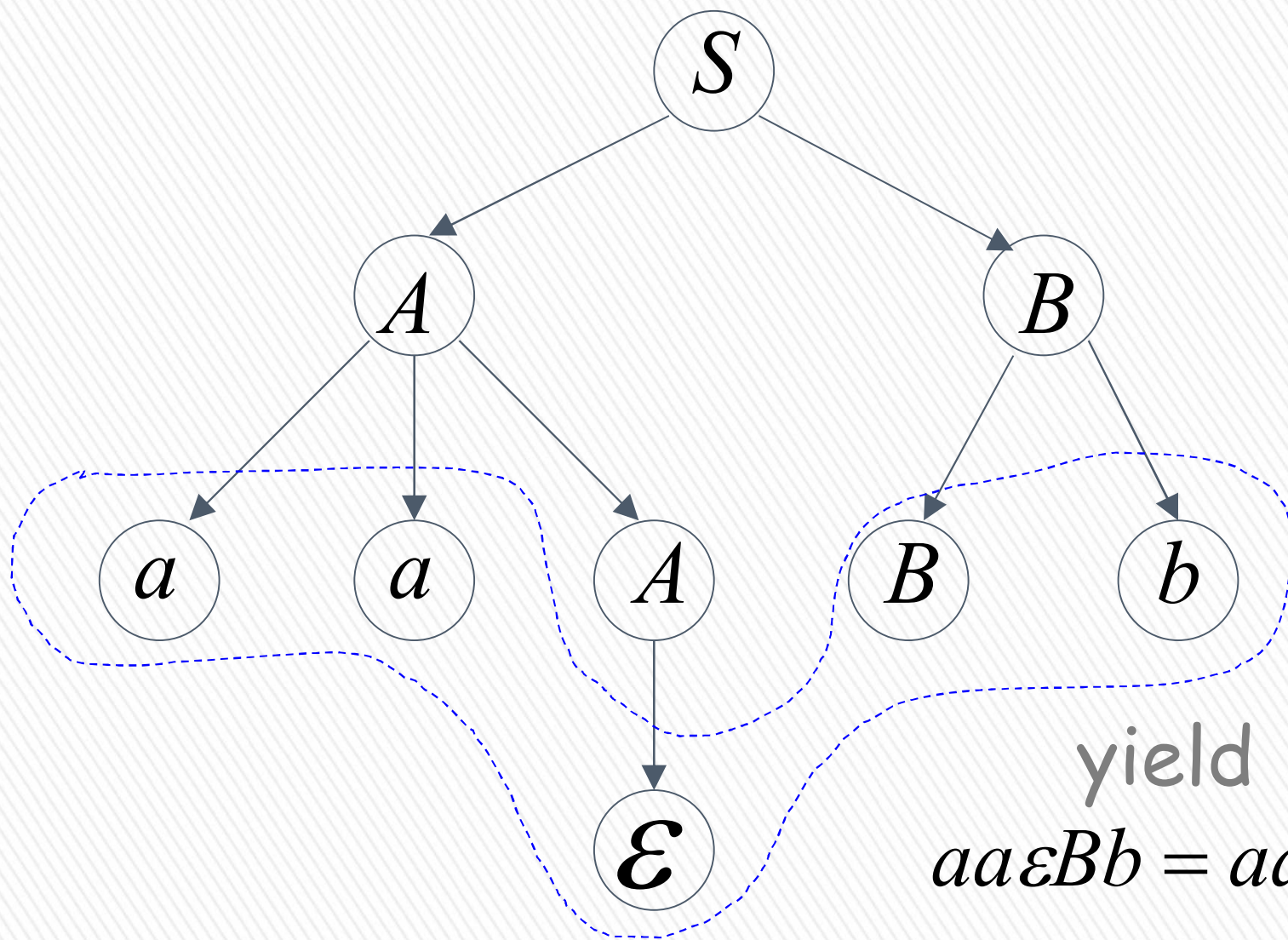
$$S \rightarrow AB \qquad A \rightarrow aaA \mid \varepsilon \quad B \rightarrow Bb \mid \varepsilon$$

$$S \Rightarrow AB \Rightarrow aaAB \Rightarrow aaABb$$



$$S \rightarrow AB \qquad A \rightarrow aaA \mid \varepsilon \qquad B \rightarrow Bb \mid \varepsilon$$

$$S \Rightarrow AB \Rightarrow aaAB \Rightarrow aaABb \Rightarrow aaBb$$

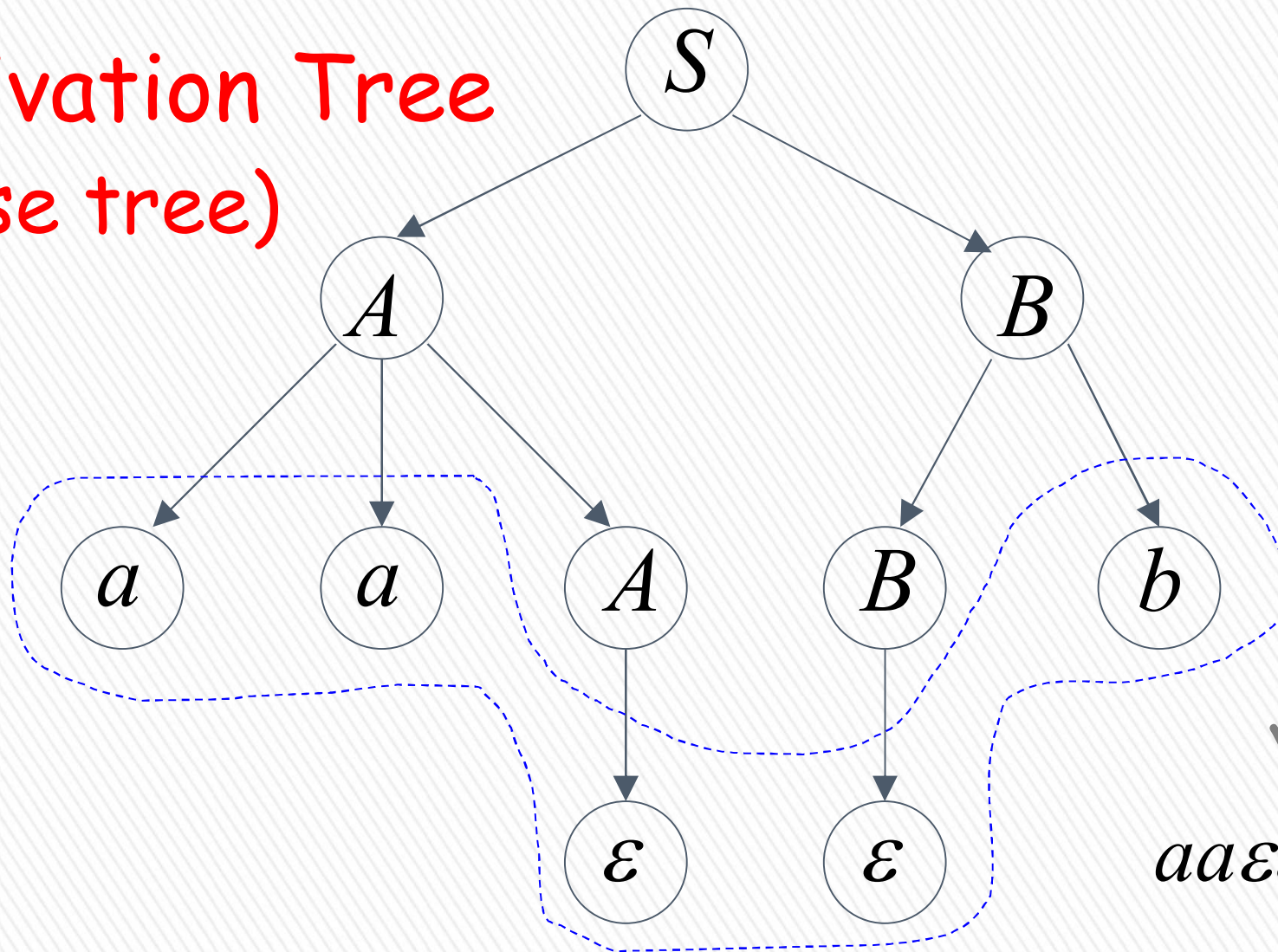


$$S \rightarrow AB$$

$$A \rightarrow aaA \mid \varepsilon \quad B \rightarrow Bb \mid \varepsilon$$

$$S \Rightarrow AB \Rightarrow aaAB \Rightarrow aaABb \Rightarrow aaBb \Rightarrow aab$$

Derivation Tree
(parse tree)



yield $\triangleright$
 $aa\varepsilon\varepsilon b = aab$

Sometimes, derivation order doesn't matter

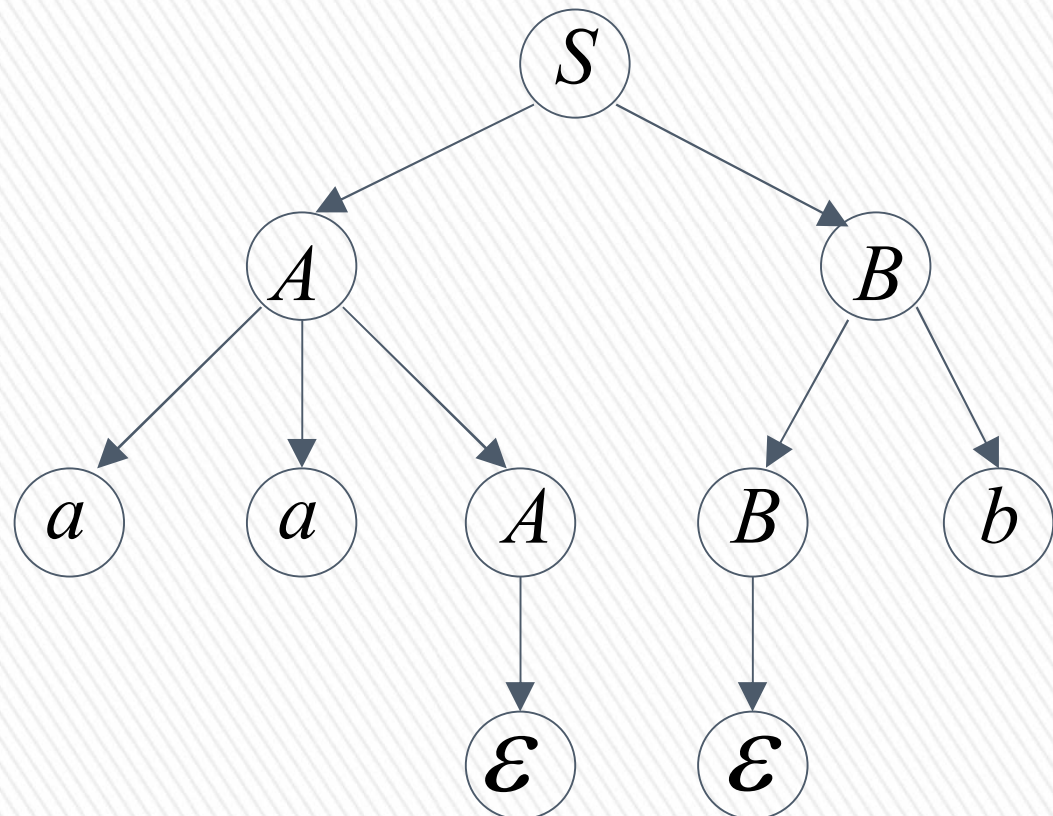
Leftmost derivation:

$$S \Rightarrow AB \Rightarrow aaAB \Rightarrow aaB \Rightarrow aaBb \Rightarrow aab$$

Rightmost derivation:

$$S \Rightarrow AB \Rightarrow ABb \Rightarrow Ab \Rightarrow aaAb \Rightarrow aab$$

Give same
derivation tree





→ Ağaçlar benzer legilsin

Ambiguity

Grammar for mathematical expressions

$$E \rightarrow E + E \mid E * E \mid (E) \mid a$$

Example strings:

$$(a + a) * a + (a + a * (a + a))$$



Denotes any number

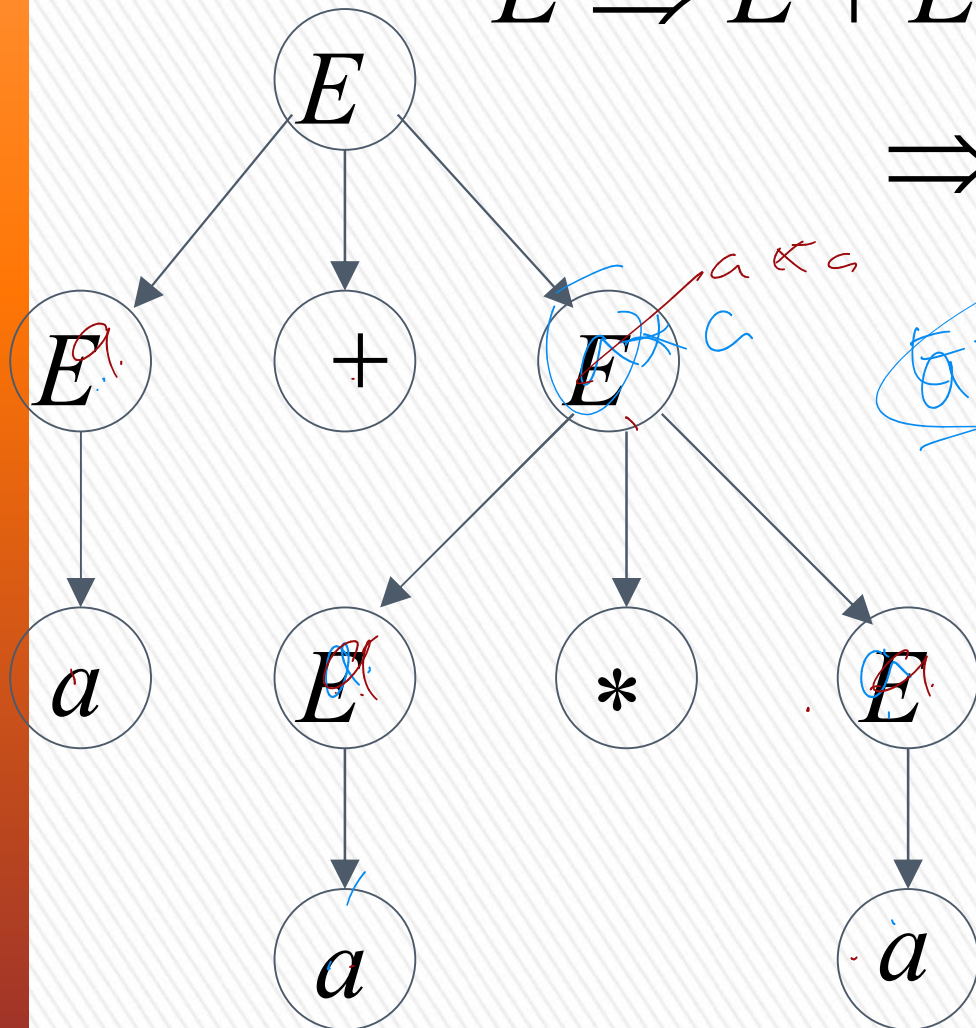


$$E \rightarrow E + E \mid E * E \mid (E) \mid a$$

$$a + a * a$$

$$E \Rightarrow E + E \Rightarrow a + E \Rightarrow a + E * E$$

$$\Rightarrow a + a * E \Rightarrow a + a * a$$



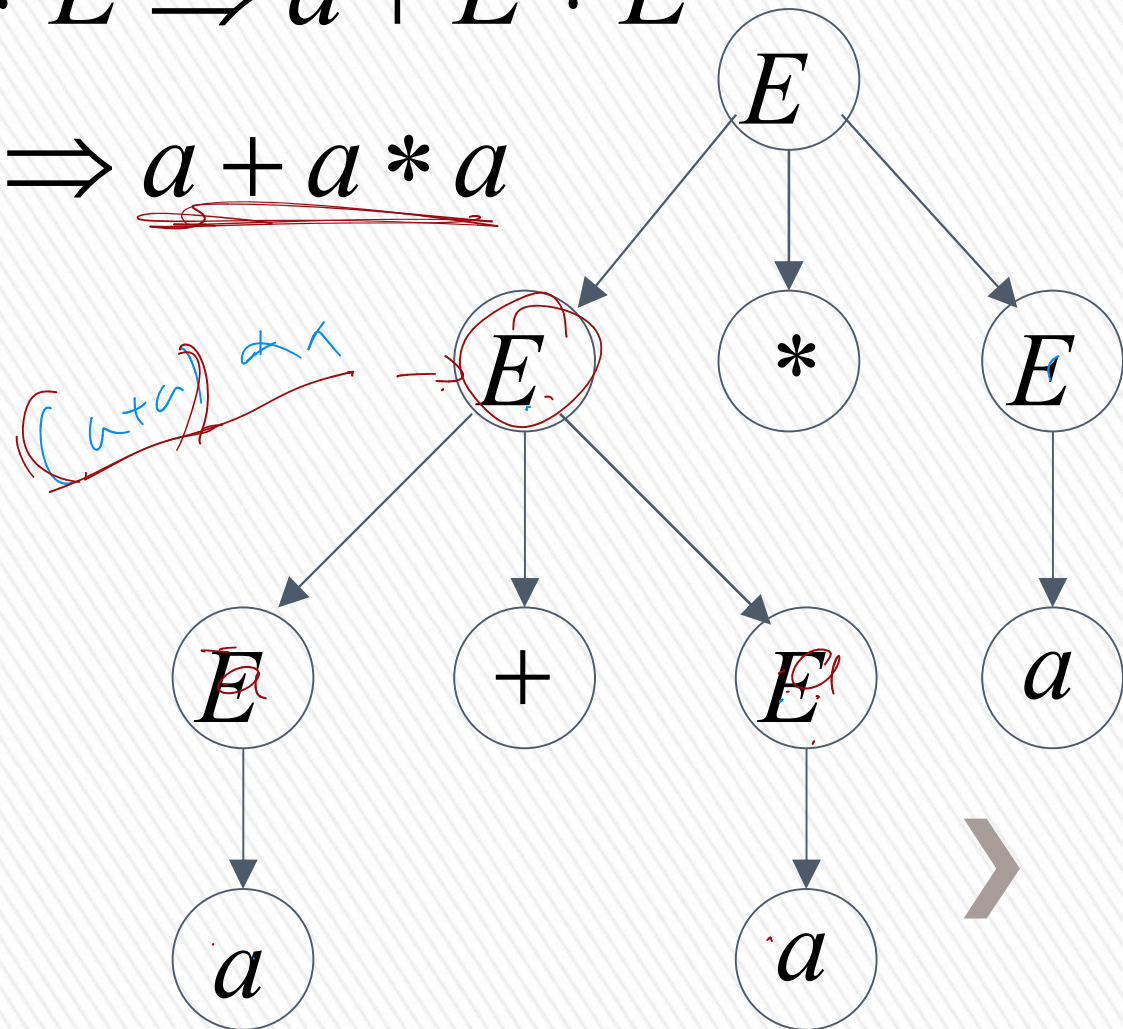
A leftmost derivation
for $a + a * a$



$$E \rightarrow E + E \mid E * E \mid (E) \mid a$$

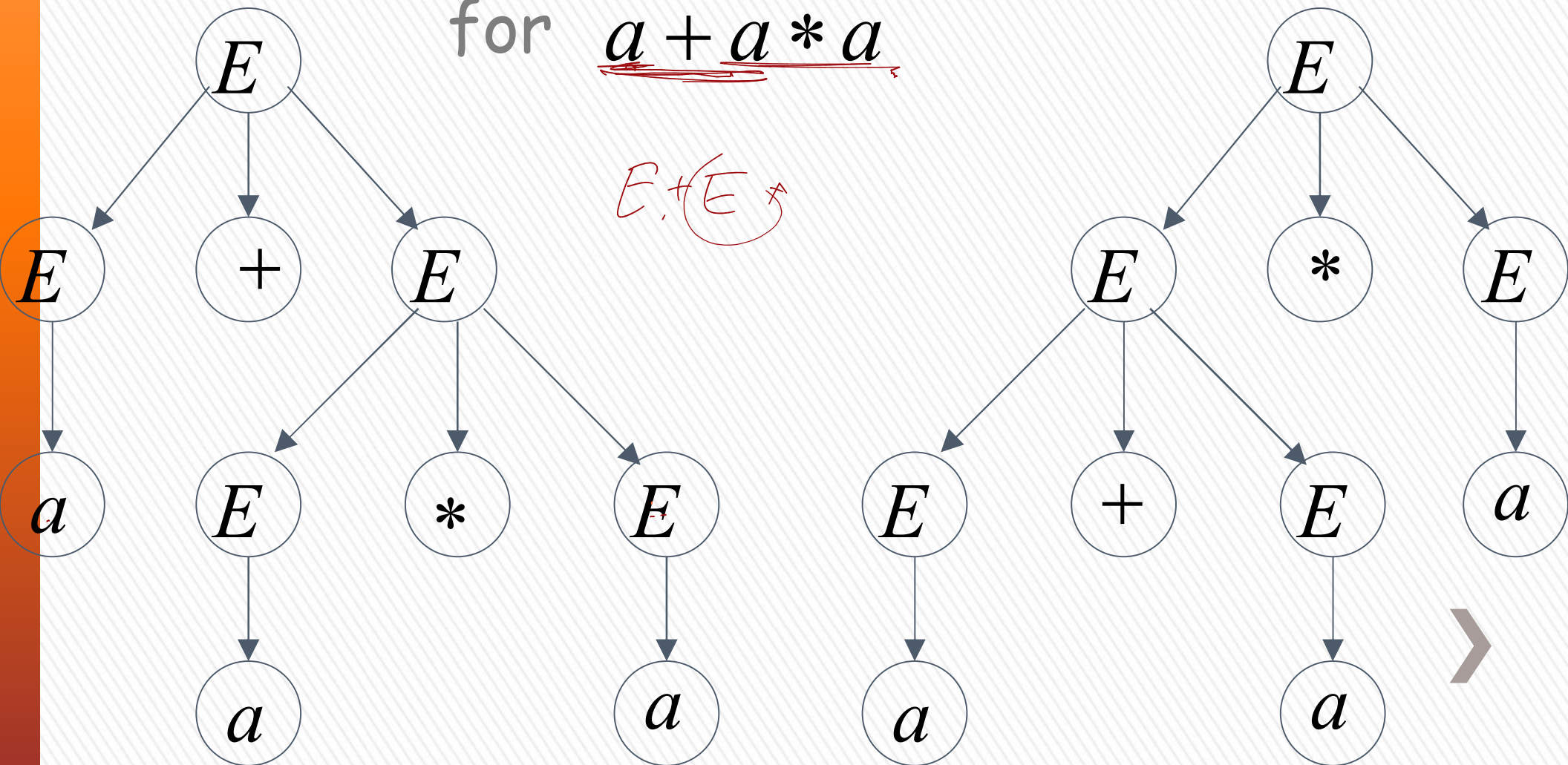
$$E \Rightarrow E * E \Rightarrow E + E * E \Rightarrow a + E * E \\ \Rightarrow a + a * E \Rightarrow \underline{a + a * a}$$

Another
leftmost derivation
for $a + a * a$



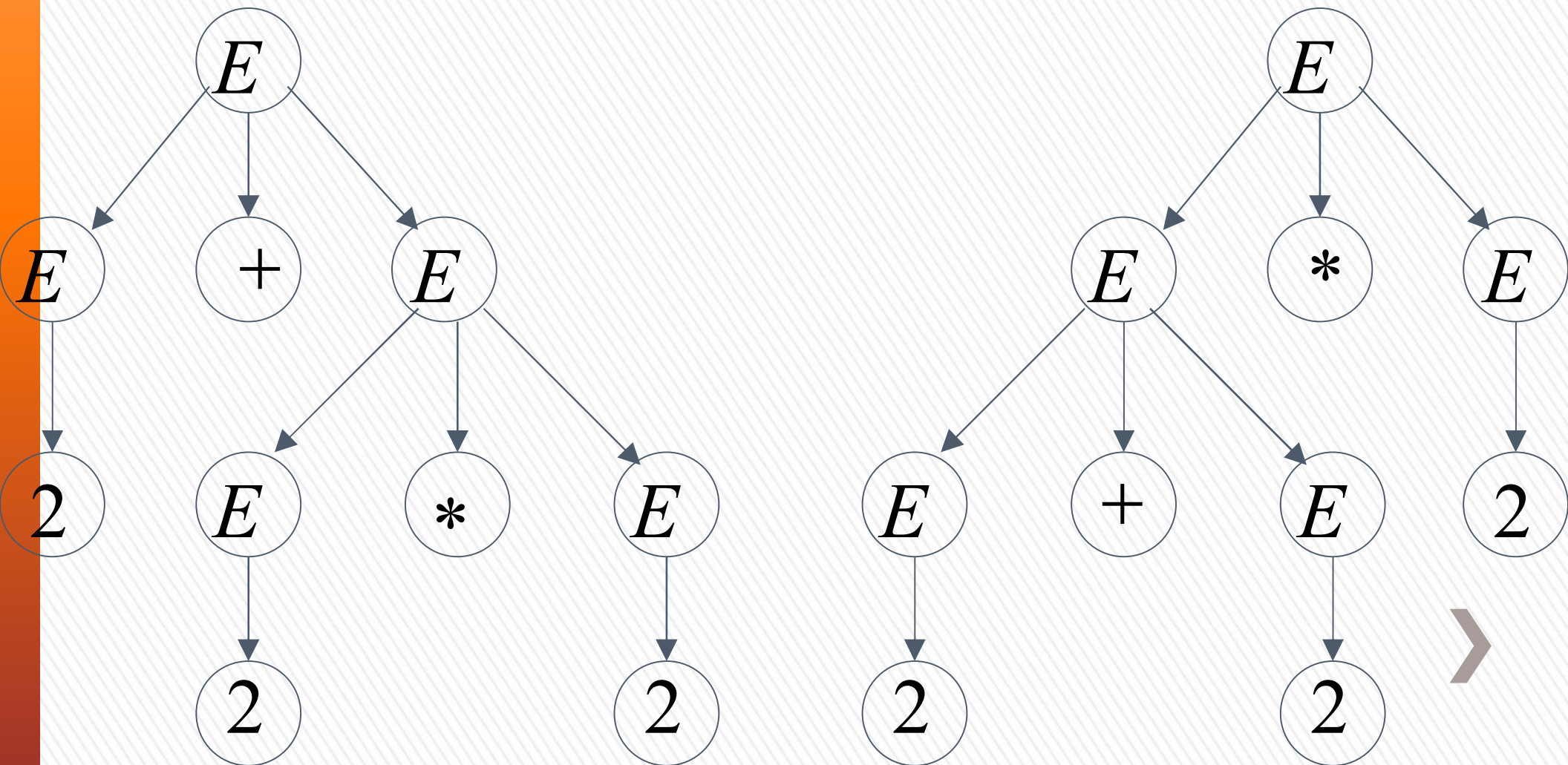
$$E \rightarrow \underline{E + E} \mid E * E \mid (E) \mid a$$

Two derivation trees
for $a + a * a$



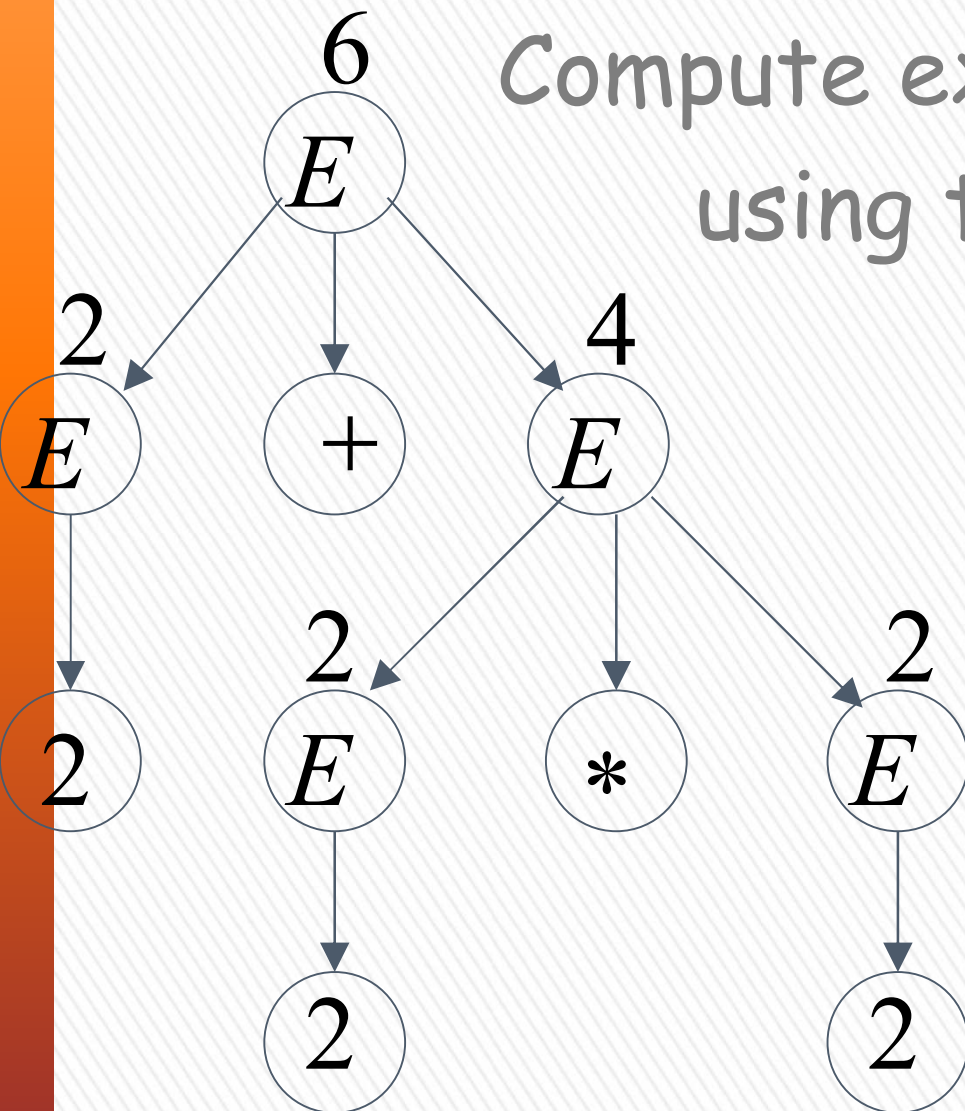
take $a = 2$

$$a + a * a = 2 + 2 * 2$$



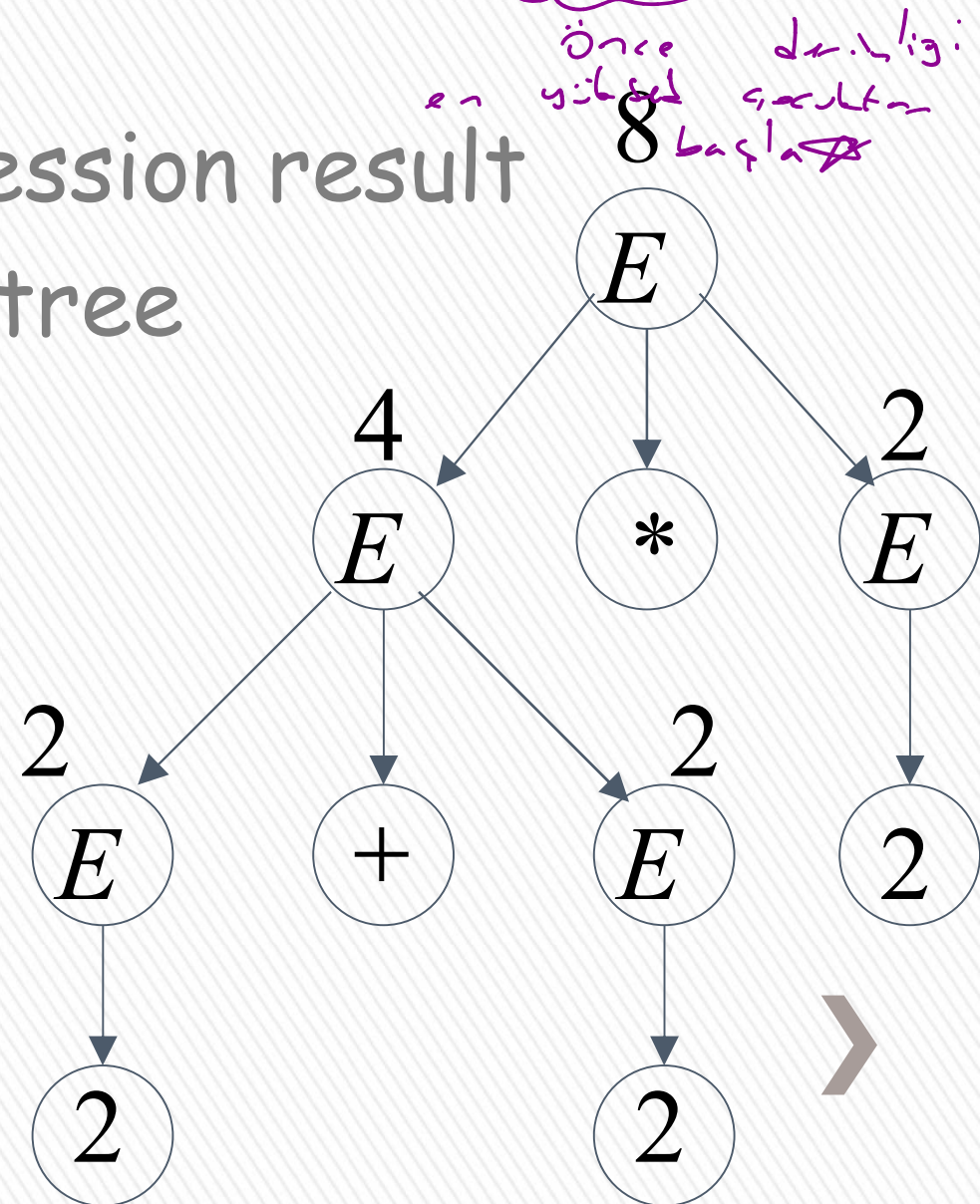
Good Tree

$$2 + 2 * 2 = 6$$



Bad Tree

$$2 + 2 * 2 = 8$$



Two different derivation trees
may cause problems in applications which
use the derivation trees:

- Evaluating expressions
- In general, in compilers
for programming languages



Ambiguous Grammar:

→ 2 farklı ağaç
Ambiguous

A context-free grammar G is ambiguous if there is a string $w \in L(G)$ which has:

two different derivation trees

or

two leftmost derivations

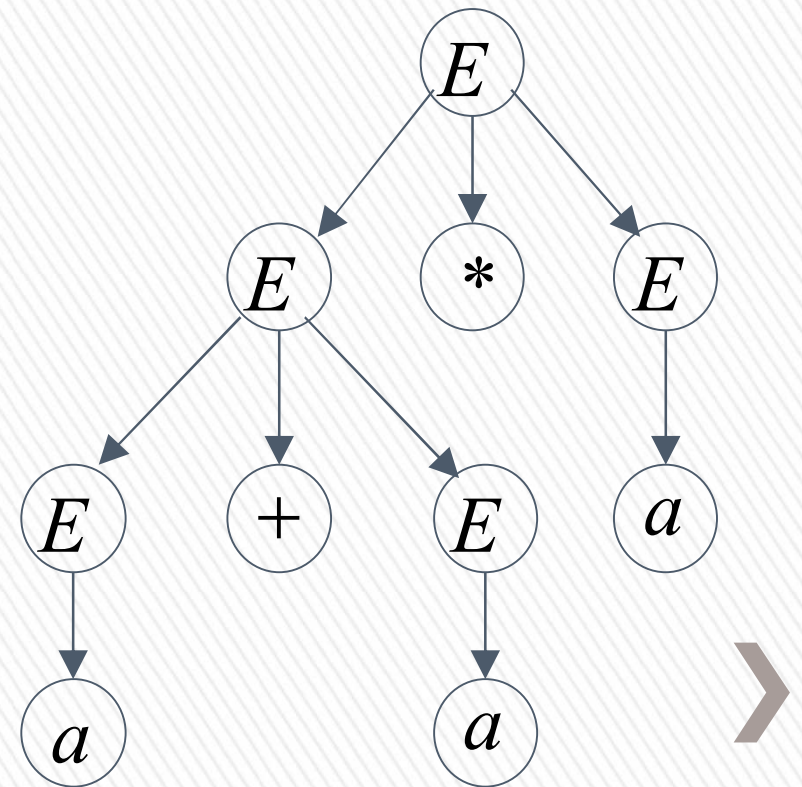
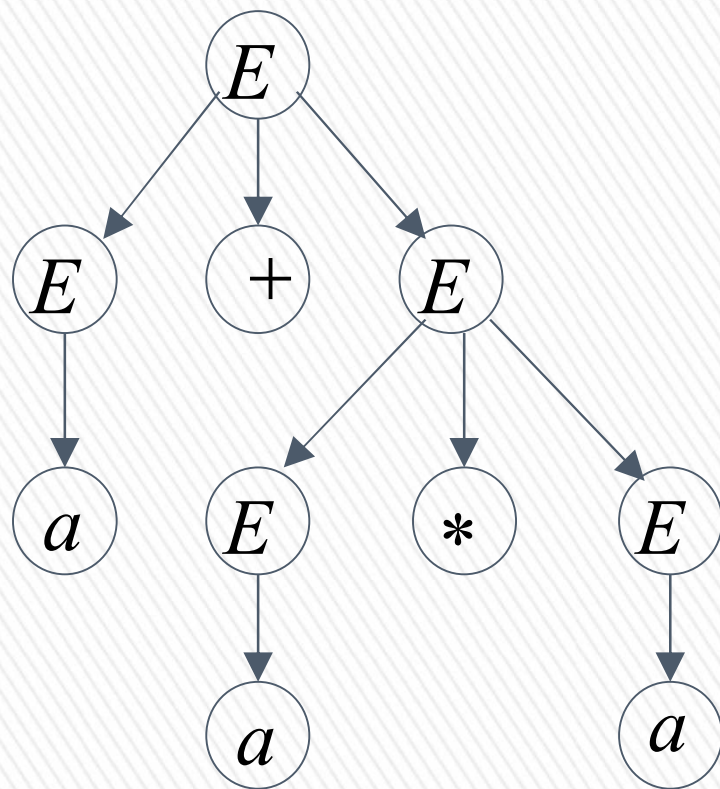
(Two different derivation trees give two different leftmost derivations and vice-versa)



Example:

$$E \rightarrow E + E \mid E * E \mid (E) \mid a$$

this grammar is ambiguous since
string $a + a * a$ has two derivation trees



$$E \rightarrow E + E \mid E * E \mid (E) \mid a$$

this grammar is ambiguous also because
string $a + a * a$ has two leftmost derivations

$$\begin{aligned} E &\Rightarrow E + E \Rightarrow a + E \Rightarrow a + E * E \\ &\Rightarrow a + a * E \Rightarrow a + a * a \end{aligned}$$

$$\begin{aligned} E &\Rightarrow E * E \Rightarrow E + E * E \Rightarrow a + E * E \\ &\Rightarrow a + a * E \Rightarrow a + a * a \end{aligned}$$



Another ambiguous grammar:

IF_STMT $\rightarrow$ if EXPR then STMT
 | if EXPR then STMT else STMT

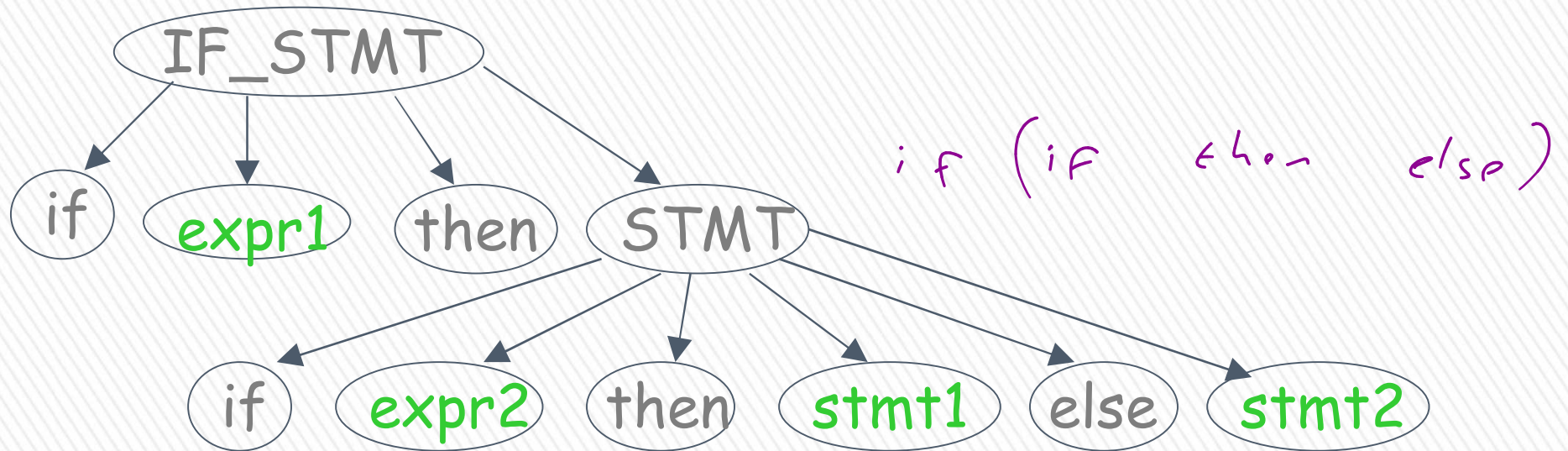


Variables Terminals

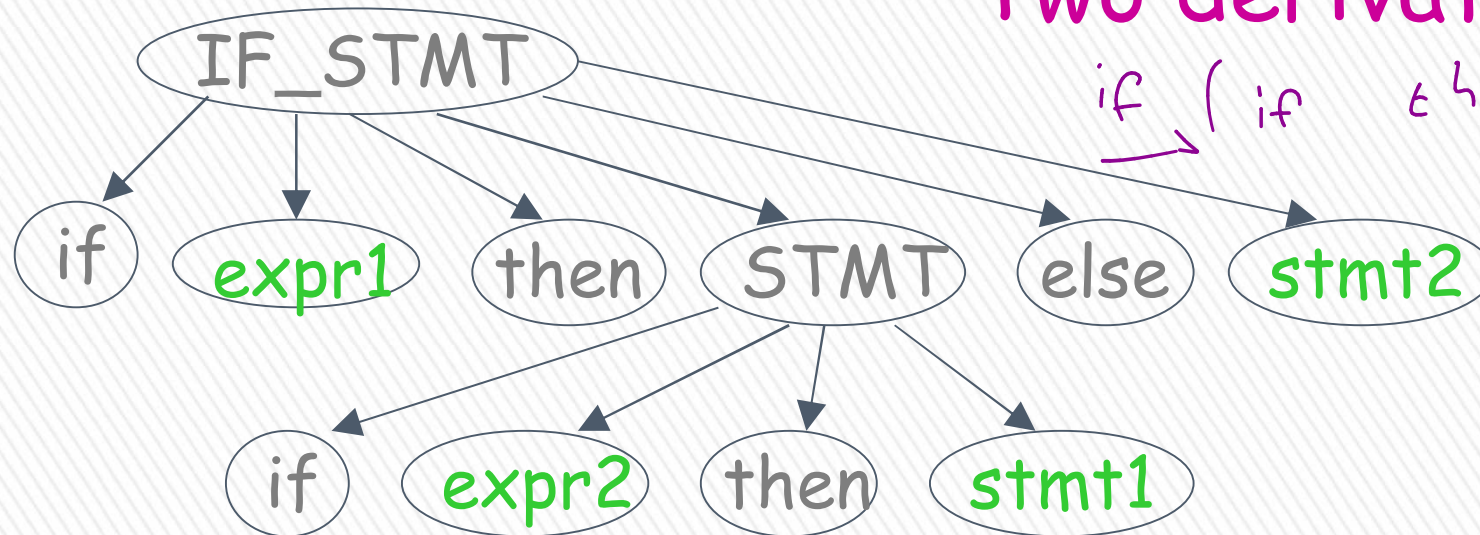
Very common piece of grammar
in programming languages



If *expr1* then if *expr2* then *stmt1* else *stmt2*



Two derivation trees



In general, ambiguity is bad
and we want to remove it

Sometimes it is possible to find
a non-ambiguous grammar for a language

But, in general we cannot do so



A successful example:

Ambiguous
Grammar

$$\begin{aligned} E &\rightarrow E + E \\ E &\rightarrow E * E \\ E &\rightarrow (E) \\ E &\rightarrow a \end{aligned}$$

Equivalent

Non-Ambiguous
Grammar

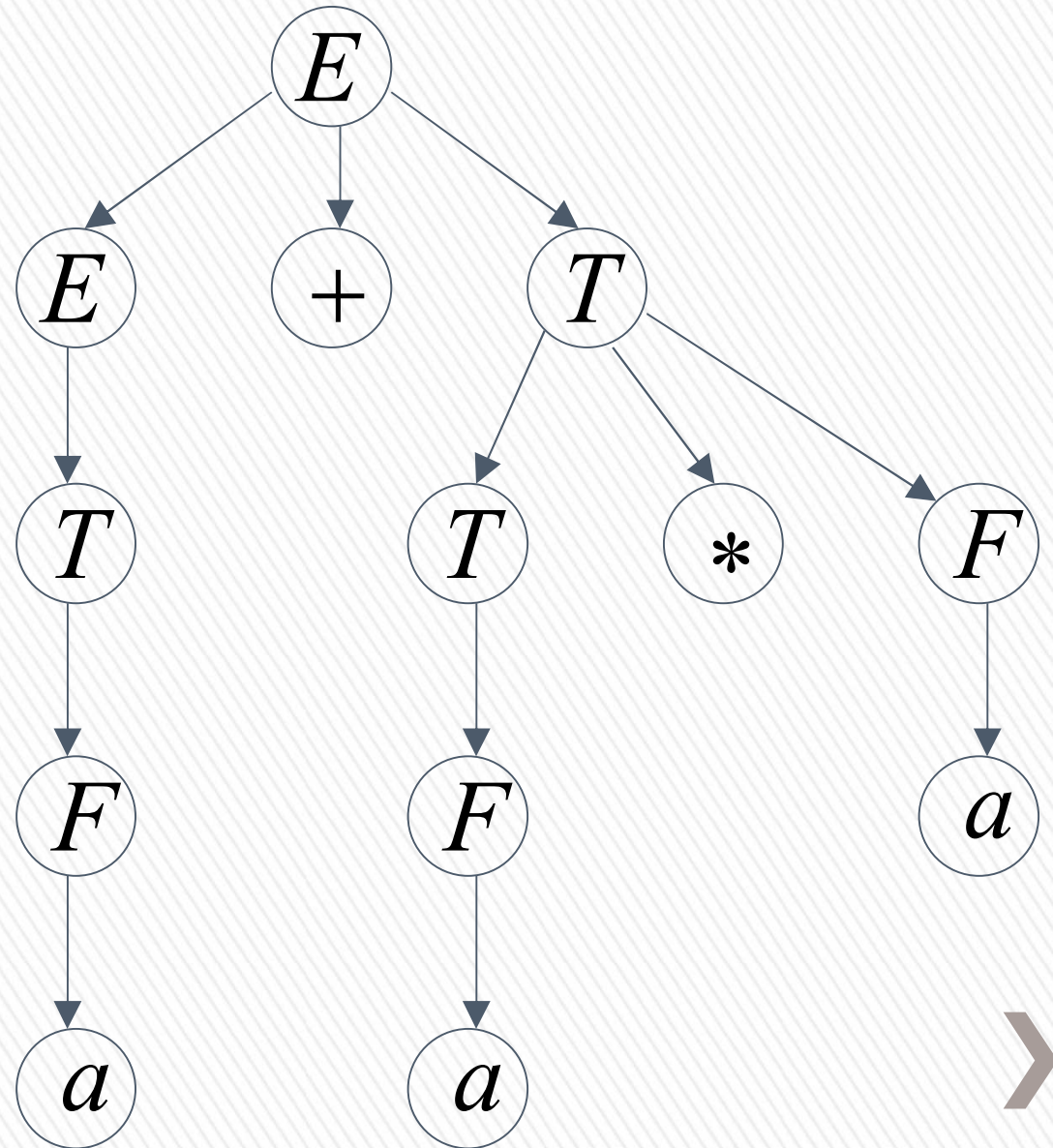
$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid a \end{aligned}$$

generates the same
language

$$\begin{aligned}
 E &\Rightarrow E + T \Rightarrow T + T \Rightarrow F + T \Rightarrow a + T \Rightarrow a + T * F \\
 &\Rightarrow a + F * F \Rightarrow a + a * F \Rightarrow a + a * a
 \end{aligned}$$

$$\begin{aligned}
 E &\rightarrow E + T \mid T \\
 T &\rightarrow T * F \mid F \\
 F &\rightarrow (E) \mid a
 \end{aligned}$$

Unique
derivation tree
for $a + a * a$



An un-successful example:

$$L = \{a^n b^n c^m\} \cup \{a^n b^m c^m\}$$

$$n, m \geq 0$$

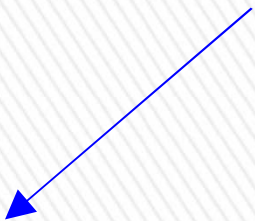
L is inherently ambiguous: [دېڭىن ئۆزى ۋە دېڭىن بىرلىشىمى دەمبۇرۇن]

every grammar that generates this language is ambiguous



Example (ambiguous) grammar for L :

$$L = \{a^n b^n c^m\} \cup \{a^n b^m c^m\}$$


$$S \rightarrow S_1 \mid S_2$$


$$S_1 \rightarrow S_1 c \mid A$$

$$A \rightarrow aAb \mid \lambda$$


$$S_2 \rightarrow aS_2 \mid B$$

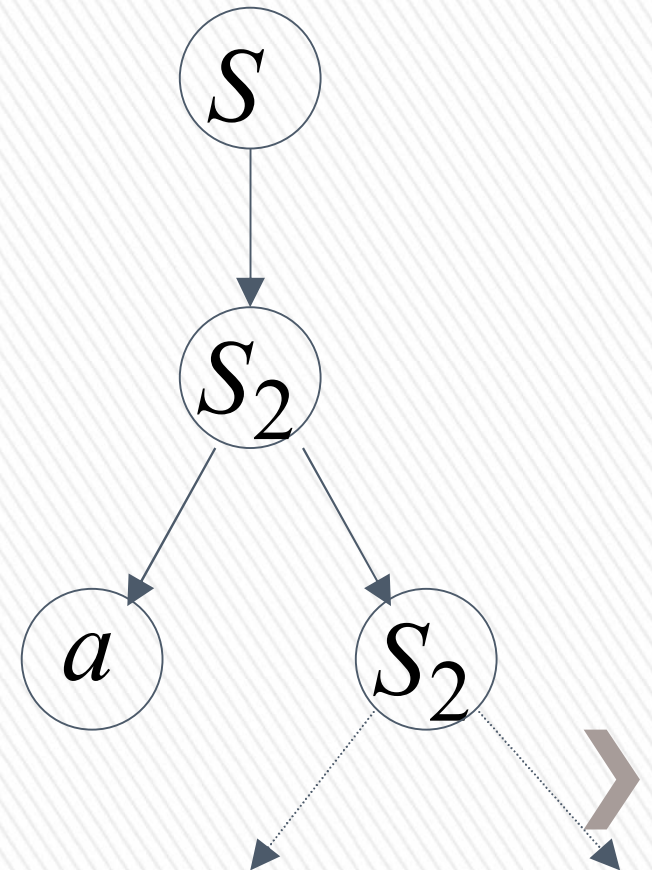
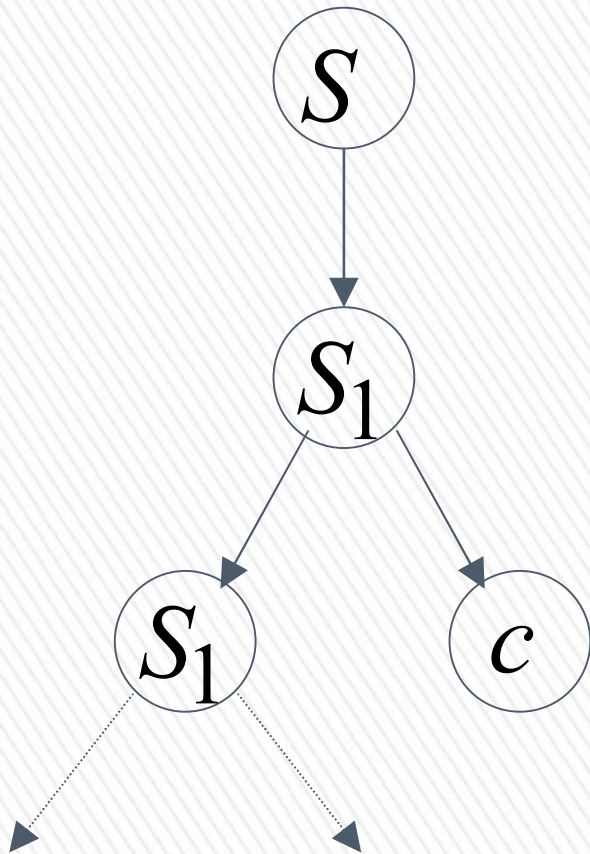
$$B \rightarrow bBc \mid \lambda$$



The string $a^n b^n c^n \in L$

has always two different derivation trees
(for any grammar)

For example



BLM2502 Theory of Computation





BLM2502

Theory of

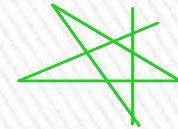
Computation

Spring 2016

BLM2502 Theory of Computation

» Course Outline

- | » Week | Content |
|-------------|---|
| » 1 | Introduction to Course |
| » 2 | Computability Theory, Complexity Theory, Automata Theory, Set Theory, Relations, Proofs, Pigeonhole Principle |
| » 3 | Regular Expressions |
| » 4 | Finite Automata |
| » 5 | Deterministic and Nondeterministic Finite Automata |
| » 6 | Epsilon Transition, Equivalence of Automata |
| » 7 | Pumping Theorem |
| » 8 | April 10 - 14 week is the first midterm week |
| » 9 | Context Free Grammars |
| » 10 | Parse Tree, Ambiguity, |
| » 11 | Pumping Theorem |
| » 12 | Turing Machines, Recognition and Computation, Church-Turing Hypothesis |
| » 13 | Turing Machines, Recognition and Computation, Church-Turing Hypothesis |
| » 14 | May 22 – 27 week is the second midterm week |
| » 15 | Review |
| » 16 | Final Exam date will be announced |



The Pumping Lemma for CFL's

Pumping Lemma

- » Recall the pumping lemma for regular languages.
- » It told us that if there was a string long enough to cause a **cycle** in the DFA for the language, then we could "pump" the cycle and discover an infinite sequence of strings that had to be in the language.
- » For CFL's the situation is a little more complicated.
- » We can always find two pieces of any sufficiently long string to "pump" in tandem. *String 2 k times instead of 1 k times pump*
accept k times 2 or more
 - > That is: if we repeat each of the two pieces the same number of times, we get another string in the language.



Statement of the CFL Pumping Lemma

$$|z| > n \\ z \in L$$

» For every context-free language L There is an integer n , such that For every string z in L of length $> n$ There exists $z = uvwxy$ such that:

1. $|vwx| < n$.
2. $|vx| > 0$.
3. For all $i > 0$, uv^iwx^iy is in L .

✓ it's stack push/pop
it's stack push/pop
it's stack push/pop



Proof of the Pumping Lemma

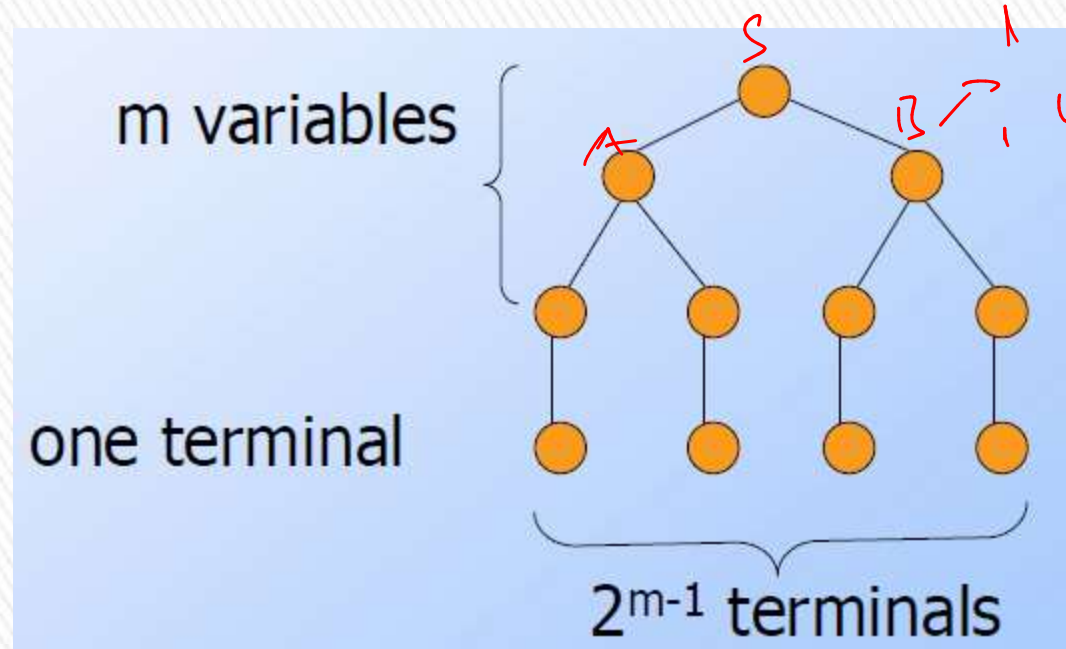
- » Start with a CNF grammar for $L - \{\epsilon\}$.
- » Let the grammar have m variables.
 - > Pick $n = 2^m$.
 - > Let $|z| > n$.
- » We claim ("Lemma 1") that a parse tree with yield z must have a path of length $m+2$ or more.

→ grammar has m variables
want to show $|z| > n$
pick $n = 2^m$
let $|z| > n$



Proof of Lemma 1

- » If all paths in the parse tree of a CNF grammar are of length $< m+1$, then the longest yield has length 2^{m-1} , as in figure:



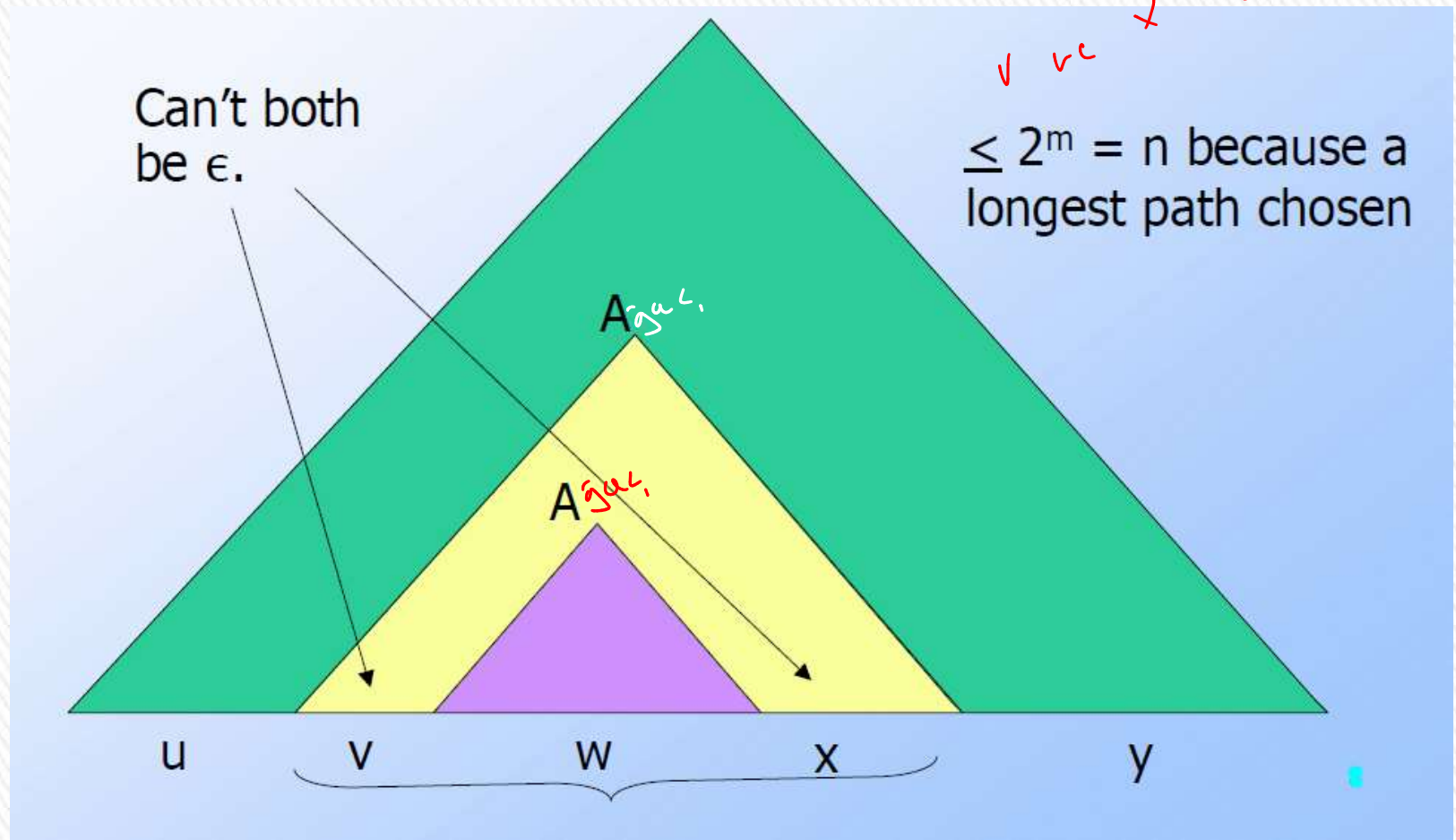
Proof of the Pumping Lemma

- » Now we know that the parse tree for z has a path with at least $m+1$ variables.
- » Consider some longest path.
- » There are only m different variables, so among the lowest $m+1$ we can find two nodes with the same label, say A .
- » The parse tree thus looks like:

$|z| > 2^m$
old $|z|$
is in
m.p. $|z|$
old $|z|$



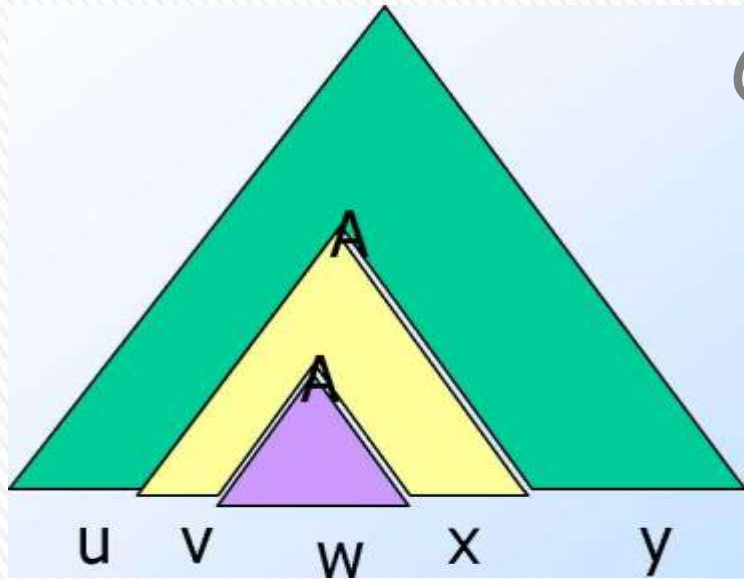
Parse Tree in the Pumping-Lemma



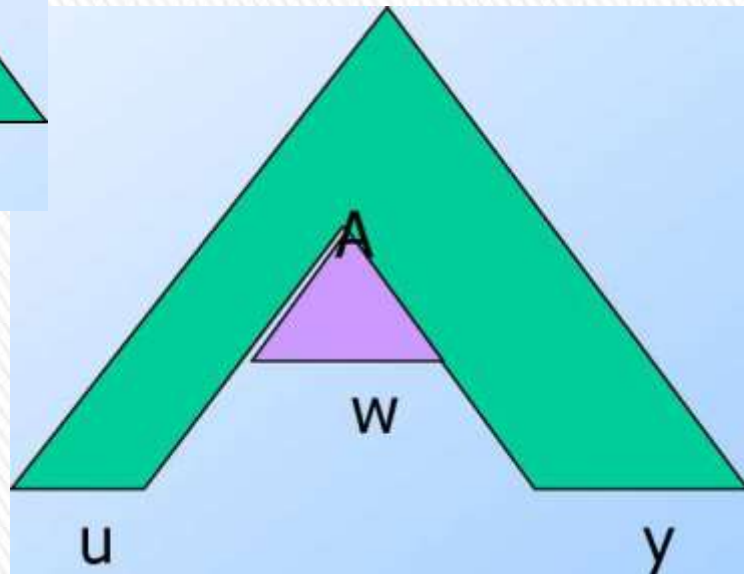
Pumping

$\rightarrow$ CFL's
 $a^k b^k c^k$

Once

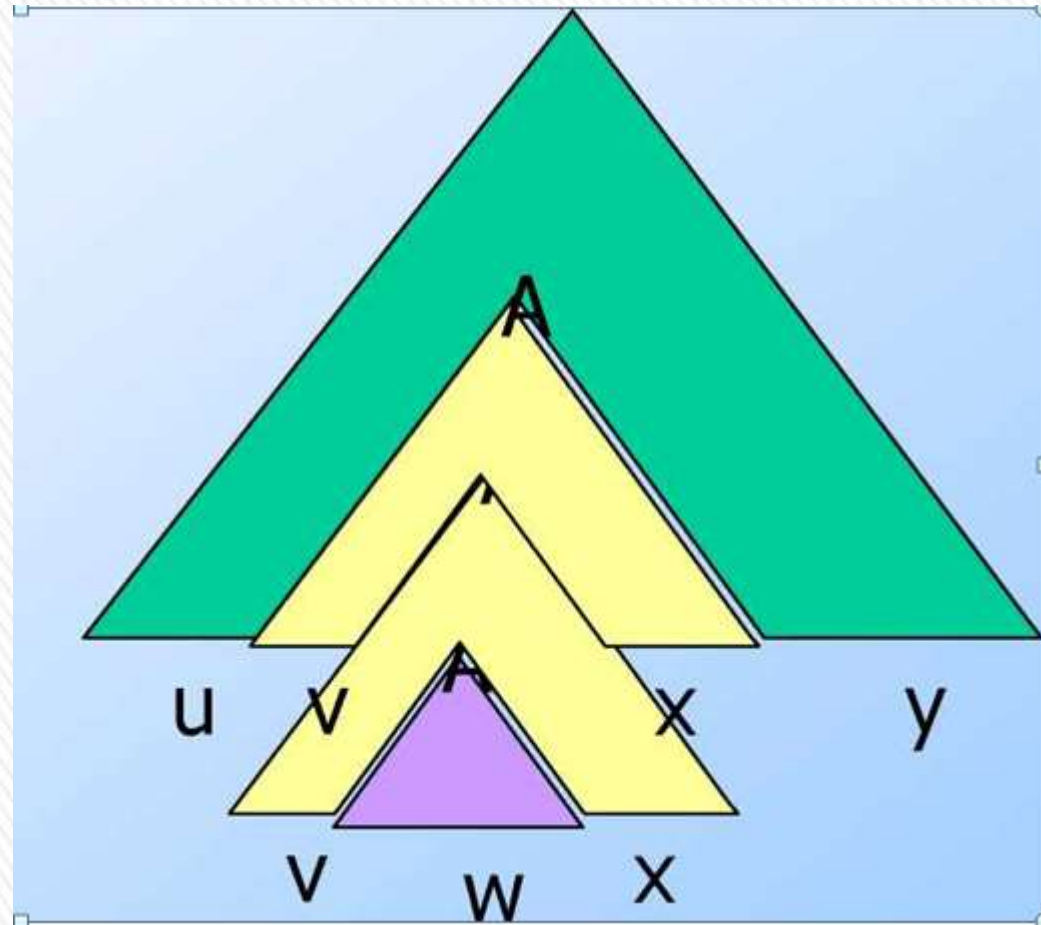


Zero times



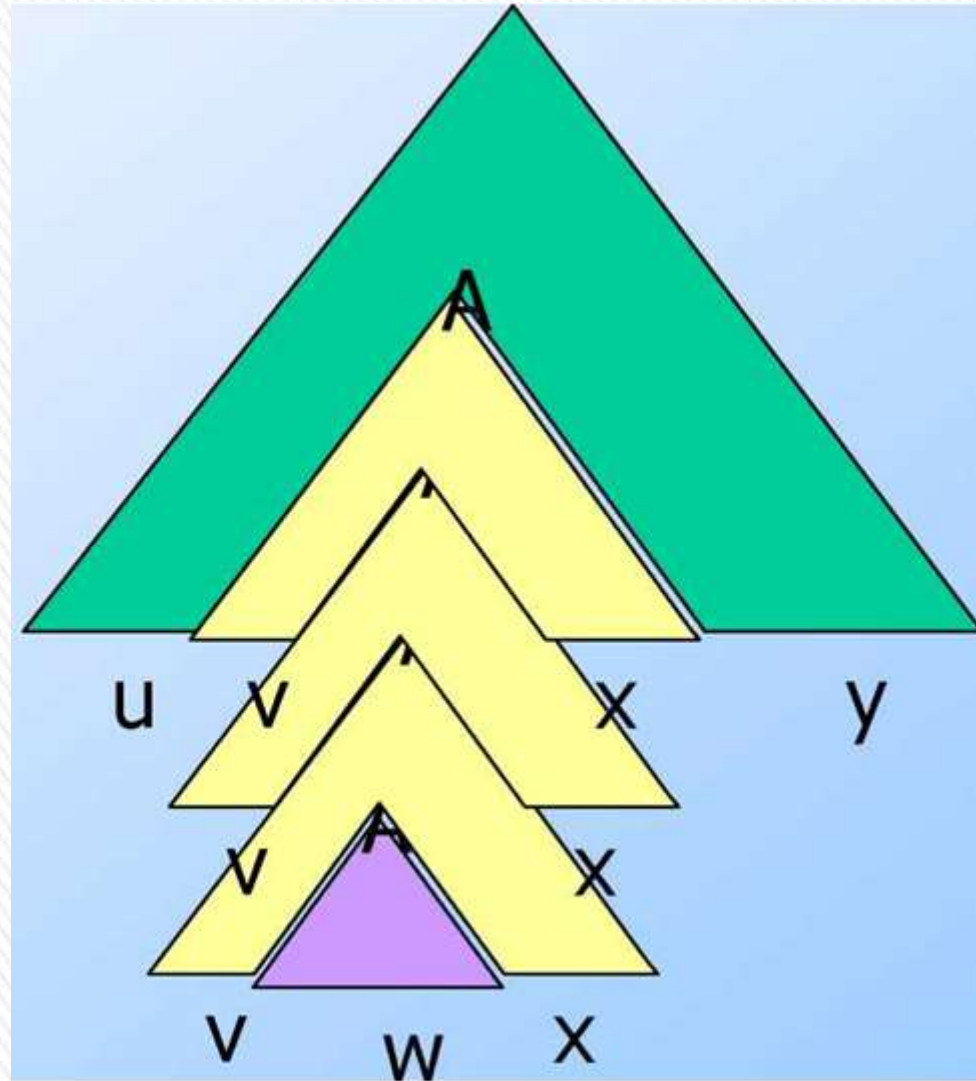
in picture
Pumping

Twice



Pumping

Thrice, ...



Using the Pumping Lemma

- » Non-CFL's typically involve trying to match two pairs of counts or match two strings.
- » Example: pumping lemma can be used to show that $L = \{ww \mid w \in (0+1)^*\}$ is not a CFL.
- » $\{0^i 10^i \mid i > 1\}$ is a CFL.
 - > We can match one pair of counts.

✓ can't get earlier or
only delay
CFL delay
push pop

$u \otimes w \otimes v$

$\forall x^n w y^n$
 $a b^n c b^n$

$a b^{n+k} c b^{n+k} \rightarrow$



Using the Pumping Lemma

- » $L = \{0^i10^i10^i \mid i > 1\}$ is not a CFL
 - > We can't match two pairs, or three counts as a group.
 - > Proof using the pumping lemma.
 - > Suppose L were a CFL.
 - > Let n be L 's pumping-lemma constant.
 - > Consider $z = 0^n10^n10^n$.
 - > We can write $z = uvwxy$, where $|vwx| < n$, and $|vx| > 1$.
 - > Case 1: vx has no 0's.
 - > Then at least one of them is a 1, and uwv has at most one 1, which no string in L does.



Using the Pumping Lemma

- » Still considering $z = 0^n 1 0^n 1 0^n$.
- » Case 2: vx has at least one 0.
- » vw is too short ($\text{length} < n$) to extend to all three blocks of 0's in $0^n 1 0^n 1 0^n$.
- » Thus, uwy has at least one block of n 0's, and at least one block with fewer than n 0's.
- » Thus, uwy is not in L .

uwy





BLM2502

Theory of

Computation

Spring 2016

BLM2502 Theory of Computation

» Course Outline

- | » Week | Content |
|-------------|---|
| » 1 | Introduction to Course |
| » 2 | Computability Theory, Complexity Theory, Automata Theory, Set Theory, Relations, Proofs, Pigeonhole Principle |
| » 3 | Regular Expressions |
| » 4 | Finite Automata |
| » 5 | Deterministic and Nondeterministic Finite Automata |
| » 6 | Epsilon Transition, Equivalence of Automata |
| » 7 | Pumping Theorem |
| » 8 | April 10 - 14 week is the first midterm week |
| » 9 | Context Free Grammars |
| » 10 | Parse Tree, Ambiguity, |
| » 11 | Pumping Theorem |
| » 12 | Turing Machines, Recognition and Computation, Church-Turing Hypothesis |
| » 13 | Turing Machines, Recognition and Computation, Church-Turing Hypothesis |
| » 14 | May 22 – 27 week is the second midterm week |
| » 15 | Review |
| » 16 | Final Exam date will be announced |



The Pumping Lemma for CFL's



Simplifications of Context-Free Grammars

A Substitution Rule

$$S \rightarrow \cancel{aB} \mid ab \mid aaA$$

$$A \rightarrow aaA \mid$$

$$A \rightarrow \cancel{abBc} \mid$$

$$\cancel{B} \rightarrow aA$$

$$\cancel{B} \rightarrow b$$

Substitute

$abbc \mid abaA$

$$B \rightarrow b$$

Equivalent
grammar

$$S \rightarrow \bar{a}B \mid ab$$

$$A \rightarrow aaA$$

$$A \rightarrow abBc \mid abbc$$

$$B \rightarrow aA$$



$$S \rightarrow aB \mid ab$$

$$A \rightarrow aaA$$

$$A \rightarrow abBc \mid abbc$$

$$B \rightarrow aA$$

Substitute

$$B \rightarrow aA$$

$$S \rightarrow \cancel{aB} \mid ab \mid aaA$$

$$A \rightarrow aaA$$

$$A \rightarrow \cancel{abBc} \mid abbc \mid abaAc$$

Equivalent
grammar ➤

In general: $A \rightarrow xBz$

$$B \rightarrow y_1$$

Substitute

$$B \rightarrow y_1$$

$$A \rightarrow xBz \mid xy_1z$$

equivalent
grammar >

Nullable Variables

ε – production :

$$X \rightarrow \varepsilon$$

Nullable Variable:

$$Y \Rightarrow \dots \Rightarrow \varepsilon$$

Example:

$$S \rightarrow aMb \mid ab$$

$$M \rightarrow aMb \mid ab$$

~~$$M \rightarrow \varepsilon$$~~

Nullable variable

ε – production



Removing ε – productions

$$S \rightarrow aMb \quad | \quad ab$$

$$M \rightarrow aMb \quad | \quad ab$$

~~$$M \rightarrow \varepsilon$$~~

Substitute

$$M \rightarrow \varepsilon$$

$$S \rightarrow aMb \mid ab$$

$$M \rightarrow aMb \mid ab$$

After we remove all the ε – productions
all the nullable variables disappear
(except for the start variable)



Unit-Productions

Unit Production: $X \rightarrow Y$

(a single variable in both sides)

Example: $S \rightarrow aA$

$A \rightarrow a$

$A \rightarrow B$

$B \rightarrow A$

$B \rightarrow bb$

Unit Productions



Removal of unit productions:

$$S \rightarrow aA \mid \cancel{aB} \mid abb \mid aa \mid aa$$

$$A \rightarrow a$$

$$\cancel{A \rightarrow B}$$

$$B \rightarrow A \mid a$$

$$B \rightarrow bb$$

Substitute

$$A \rightarrow B$$

$$S \rightarrow aA \mid aB$$

$$A \rightarrow a$$

$$B \rightarrow A \mid B$$

$$B \rightarrow bb$$



Unit productions of form $X \rightarrow X$
can be removed immediately

$$S \rightarrow aA \mid aB$$

$$A \rightarrow a$$

$$B \rightarrow A \mid \cancel{B}$$

$$B \rightarrow bb$$

Remove

$$B \rightarrow B$$

$$S \rightarrow aA \mid aB$$

$$A \rightarrow a$$

$$B \rightarrow A$$

$$B \rightarrow bb$$



$$S \rightarrow aA \mid aB$$

$$A \rightarrow a$$

~~$$B \rightarrow A$$~~

$$B \rightarrow bb$$

göndörün gere A gærn

Substitute

$$B \rightarrow A$$

$$S \rightarrow \cancel{aA} \mid aB \mid aA$$

$$A \rightarrow a$$

$$B \rightarrow bb$$



Remove repeated productions

$$S \rightarrow \textcircled{aA} \mid aB \mid \cancel{aA}$$

$$A \rightarrow a$$

$$B \rightarrow bb$$



Final grammar

$$S \rightarrow aA \mid aB$$

$$A \rightarrow a$$

$$\textcircled{B} \rightarrow bb$$



Useless Productions

$$S \rightarrow aSb$$

$$S \rightarrow \lambda$$

$$S \rightarrow A$$

~~$A \rightarrow aA$~~ *sonuçlanamaz* **Useless Production**

Some derivations never terminate...

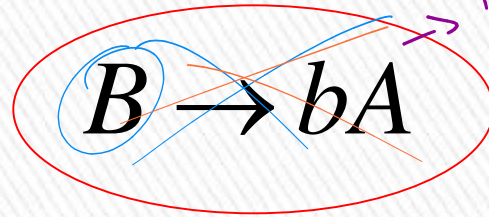
$$S \Rightarrow A \Rightarrow aA \Rightarrow aaA \Rightarrow \dots \Rightarrow aa \dots aA \Rightarrow \dots$$

Another grammar:

$$S \rightarrow A$$

$$A \rightarrow aA$$

$$A \rightarrow \lambda$$



The production $B \rightarrow bA$ is enclosed in a red oval. A blue circle highlights the non-terminal B . A blue circle highlights the non-terminal A in the right-hand side. A blue arrow points from the B in the left-hand side to the A in the right-hand side. A purple arrow points from the production to the handwritten text above it.

Bye his gel-mage(eh)?

Useless Production

Not reachable from S



In general:

If there is a derivation

$$S \Rightarrow \dots \Rightarrow xAy \Rightarrow \dots \Rightarrow w \in L(G)$$



consists of
terminals

Then variable A is useful

Otherwise, variable A is useless



A production $A \rightarrow x$ is useless
if any of its variables is useless

$$S \rightarrow aSb$$

$$S \rightarrow \varepsilon$$

Productions

Variables

$$S \rightarrow A$$

useless

useless

$$A \rightarrow aA$$

useless

useless

$$B \rightarrow C$$

useless

useless

$$C \rightarrow D$$

useless



Removing Useless Variables and Productions

Example Grammar:

$$S \rightarrow aS \mid A \mid \cancel{C}$$

$$A \rightarrow a$$

$$\cancel{B \rightarrow aa}$$

$$\cancel{C \rightarrow aCb}$$



First: find all variables that can produce strings with only terminals or ε (possible useful variables)

$$S \rightarrow aS \mid \textcircled{A} \mid C$$

$$\textcircled{A \rightarrow a}$$

$$\textcircled{B \rightarrow aa}$$

$$C \rightarrow aCb$$

Round 1: $\{A, B\}$

(the right hand side of production that has only terminals)

Round 2: $\{A, B, S\}$

(the right hand side of a production has terminals and variables of previous round)

This process can be generalized >

Then, remove productions that use variables other than $\{A, B, S\}$

$$S \rightarrow aS \mid A \mid \cancel{C}$$

$$A \rightarrow a$$

$$B \rightarrow aa$$

$$\cancel{C \rightarrow aCb}$$



$$S \rightarrow aS \mid A$$

$$A \rightarrow a$$

$$B \rightarrow aa$$



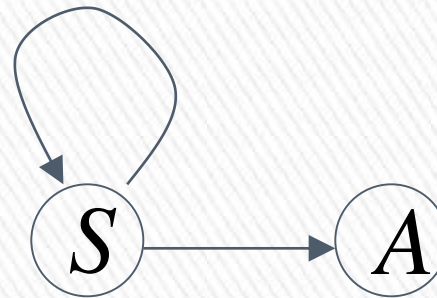
Second: Find all variables
reachable from S

Use a Dependency Graph
where nodes are variables

$$S \rightarrow aS \mid A$$

$$A \rightarrow a$$

$$B \rightarrow aa$$



unreachable



Keep only the variables
reachable from S

$$S \rightarrow aS \mid A$$

$$A \rightarrow a$$

~~$$B \rightarrow aa$$~~



Final Grammar

$$S \rightarrow aS \mid A$$

$$A \rightarrow a$$

Contains only
useful variables

Removing All

» **Step 1:** Remove Nullable Variables

$$\hookrightarrow A \rightarrow \epsilon$$

» **Step 2:** Remove Unit-Productions

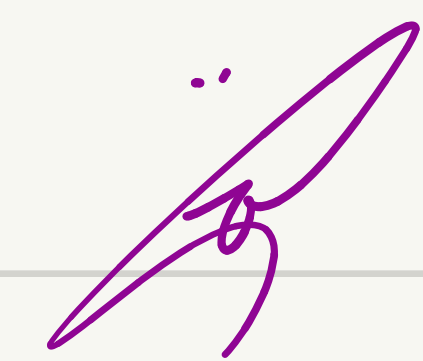
$$\hookrightarrow A \rightarrow B$$

» **Step 3:** Remove Useless Variables

$$\hookrightarrow A \rightarrow A_a$$

This sequence guarantees that unwanted variables and productions are removed





$$S \rightarrow ABA C$$

$$A \rightarrow qA \mid \epsilon$$

$$B \rightarrow bB \mid \epsilon$$

$$C \rightarrow c$$

① Remove null productions

$$A \rightarrow \epsilon, B \rightarrow \epsilon$$

$$S \rightarrow ABA C \mid ABC \mid BAC \mid BC \mid C \mid AAC \mid AC$$

$$A \rightarrow qA \mid q$$

$$B \rightarrow bB \mid b$$

$$C \rightarrow c$$

$$S \rightarrow ABA c \mid ABc \mid BA c \mid Bc \mid c \mid AA c \mid Ac$$

$$A \rightarrow qA \mid q$$

$$B \rightarrow bB \mid b$$

$$S \rightarrow ABA \epsilon \mid AB \epsilon \mid BA \epsilon \mid B \epsilon \mid AA \epsilon \mid A \epsilon \mid \epsilon$$

$$A \rightarrow qA \mid q$$

$$B \rightarrow qB \mid b$$

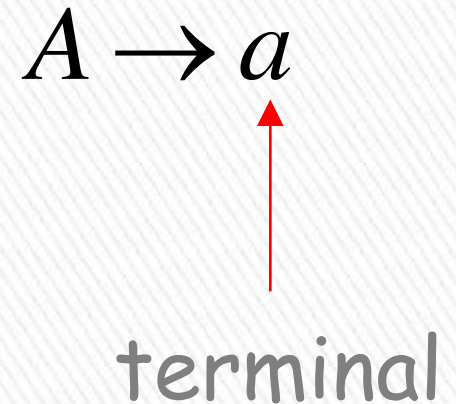
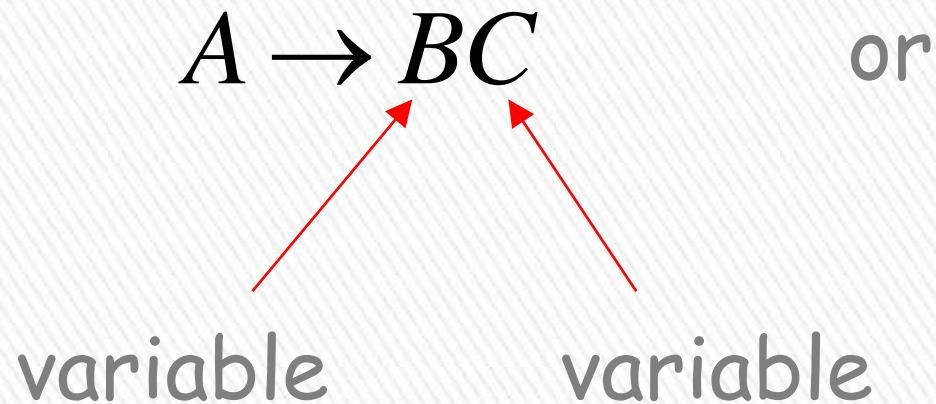
$$C \rightarrow \epsilon$$



Normal Forms for Context-free Grammars

Chomsky Normal Form

Each productions has form:



Examples:

$$\underline{S \rightarrow AS}$$

$$\underline{S \rightarrow a}$$

$$\underline{A \rightarrow SA}$$

$$\underline{A \rightarrow b}$$

$$\begin{array}{l} S \rightarrow \underline{AV_1} \\ V_1 \rightarrow AS \end{array}$$

$$T_1 \rightarrow a$$

$$S \rightarrow \underline{AS}$$

$$S \rightarrow \underline{AAS}$$

$$A \rightarrow SA$$

$$A \rightarrow \underline{aa}$$
$$A \rightarrow T_1 t_1$$

Chomsky

Normal Form

Not Chomsky

Normal Form

Conversion to Chomsky Normal Form

» Example: $S \rightarrow ABa$
 $A \rightarrow aab$
 $B \rightarrow Ac$

Not in Chomsky Normal Form

$$V_1 \rightarrow a$$

$$V_2 \rightarrow \square V_1$$

$$V_3 \rightarrow b$$

$$V_4 \rightarrow V_1 V_3$$

$$V_5 \rightarrow c$$

We will convert it to Chomsky Normal Form



Introduce new variables for the terminals:

$$T_a, T_b, T_c$$

$$S \rightarrow ABT_a$$

$$A \rightarrow T_a T_a T_b$$

$$B \rightarrow AT_c$$

$$T_a \rightarrow a$$

$$T_b \rightarrow b$$

$$T_c \rightarrow c$$



$$\begin{array}{l} S \rightarrow ABa \\ A \rightarrow aab \\ B \rightarrow Ac \end{array}$$



$$\begin{array}{l} T_1 \rightarrow a \\ T_2 \rightarrow b \\ T_3 \rightarrow c \end{array}$$

Introduce new intermediate variable V_1
to break first production:

$$S \rightarrow ABT_a$$

$$A \rightarrow T_a T_a T_b$$

$$B \rightarrow AT_c$$

$$T_a \rightarrow a$$

$$T_b \rightarrow b$$

$$T_c \rightarrow c$$



$$S \rightarrow AV_1$$

$$V_1 \rightarrow BT_a$$

$$A \rightarrow T_a T_a T_b$$

$$B \rightarrow AT_c$$

$$T_a \rightarrow a$$

$$T_b \rightarrow b$$

$$T_c \rightarrow c$$



Introduce intermediate variable: V_2

$$S \rightarrow AV_1$$

$$V_1 \rightarrow BT_a$$

$$A \rightarrow T_a T_a T_b$$

$$B \rightarrow AT_c$$

$$T_a \rightarrow a$$

$$T_b \rightarrow b$$

$$T_c \rightarrow c$$



$$S \rightarrow AV_1$$

$$V_1 \rightarrow BT_a$$

$$A \rightarrow T_a V_2$$

$$V_2 \rightarrow T_a T_b$$

$$B \rightarrow AT_c$$

$$T_a \rightarrow a$$

$$T_b \rightarrow b$$

$$T_c \rightarrow c$$



Final grammar in Chomsky Normal Form:

$$S \rightarrow AV_1$$

$$V_1 \rightarrow BT_a$$

$$A \rightarrow T_aV_2$$

$$V_2 \rightarrow T_aT_b$$

$$B \rightarrow AT_c$$

$$T_a \rightarrow a$$

$$T_b \rightarrow b$$

$$T_c \rightarrow c$$

Initial grammar

$$S \rightarrow ABa$$

$$A \rightarrow aab$$

$$B \rightarrow Ac$$



In general:

From any context-free grammar
(which doesn't produce ε)
not in Chomsky Normal Form

we can obtain:

an equivalent grammar
in Chomsky Normal Form



The Procedure

First remove:

Nullable variables

Unit productions

(Useless variables optional)



Then, for every symbol a :

New variable: T_a

Add production $T_a \rightarrow a$

In productions with length at least 2
replace a with T_a

Productions of form $A \rightarrow a$
do not need to change!



Replace any production $A \rightarrow C_1 C_2 \cdots C_n$

with $A \rightarrow C_1 V_1$

$V_1 \rightarrow C_2 V_2$

...

$V_{n-2} \rightarrow C_{n-1} C_n$

New intermediate variables: $V_1, V_2, \dots, V_{n-2}$ 

Observations

- Chomsky normal forms are good for parsing and proving theorems
- It is easy to find the Chomsky normal form for any context-free grammar



Greinbach Normal Form

All productions have form:

$$A \rightarrow a V_1 V_2 \cdots V_k \quad k \geq 0$$

symbol

variables



Examples:

$$S \rightarrow \underline{c}AB$$

$$A \rightarrow aA \mid bB \mid b$$

$$B \rightarrow b$$

Greibach
Normal Form

$\underline{b} \checkmark$

$$T_1 = b$$

$$S \rightarrow aT_1ST_1$$

$$S \rightarrow abSb$$

$$S \rightarrow \underline{aa}$$

$$T_2 = a$$

$$S \rightarrow aT_2$$

Not Greibach
Normal Form

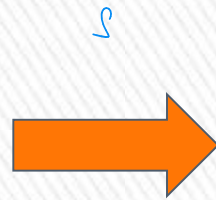


Conversion to Greinbach Normal Form:

$$\begin{aligned} S &\rightarrow abSb \\ S &\rightarrow aa \end{aligned}$$

$$V_1 \Rightarrow b$$

$$V_2 \Rightarrow a$$



$$S \rightarrow aT_bST_b$$

$$S \rightarrow aT_a$$

$$T_a \rightarrow a$$

$$T_b \rightarrow b$$

Greinbach
Normal Form >

Observations

- Greinbach normal forms are very good for parsing strings (better than Chomsky Normal Forms)
- However, it is difficult to find the Greinbach normal of a grammar



BLM2502 Theory of Computation





BLM2502

Theory of

Computation



Pushdown Automata

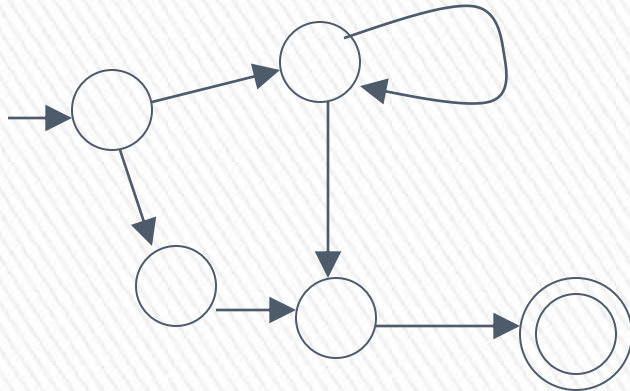
PDA

Pushdown Automaton -- PDA

Input String



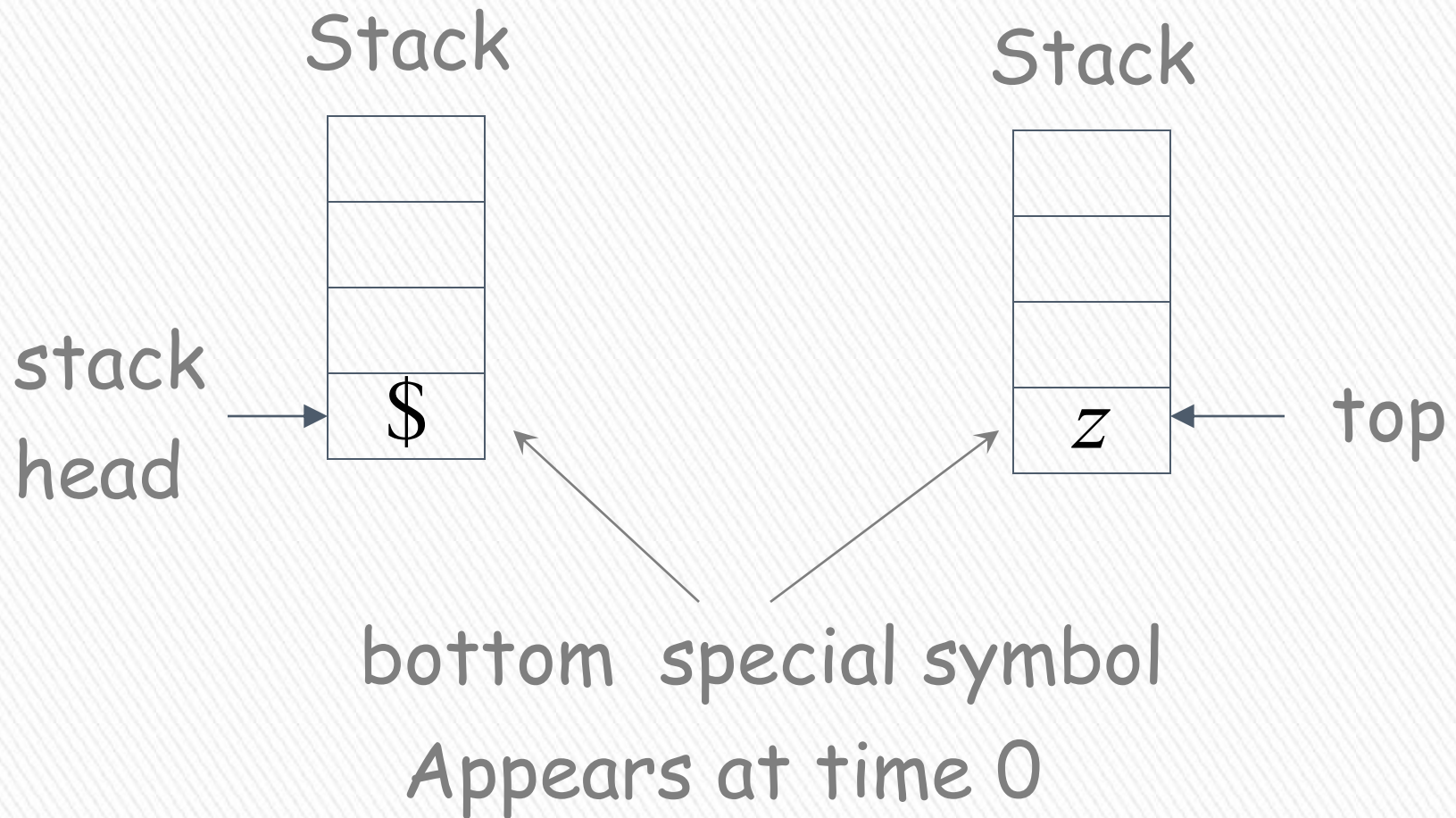
States



Stack



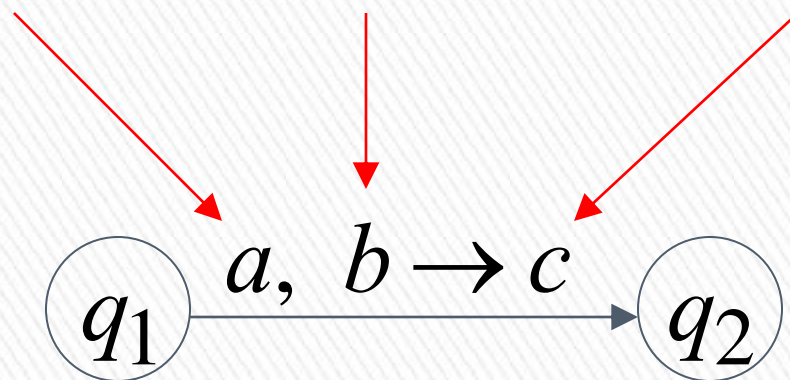
Initial Stack Symbol



Input
symbol

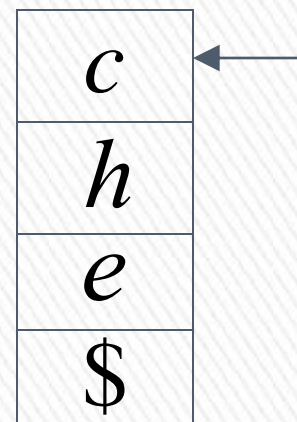
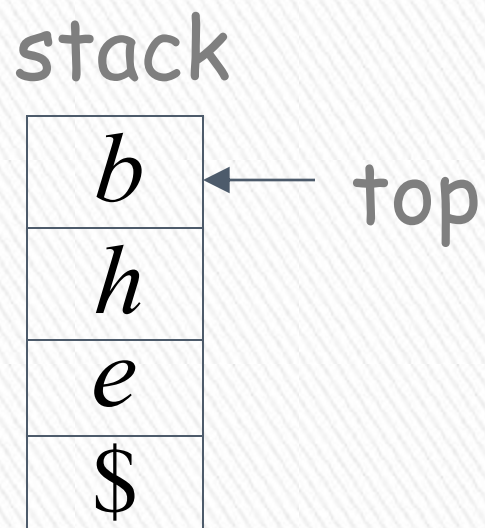
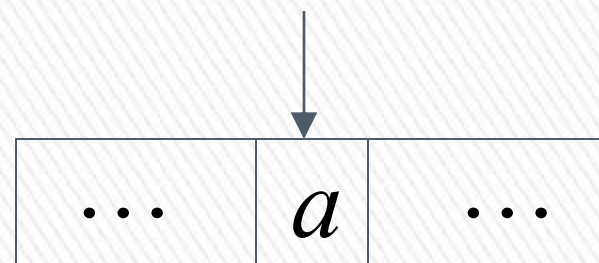
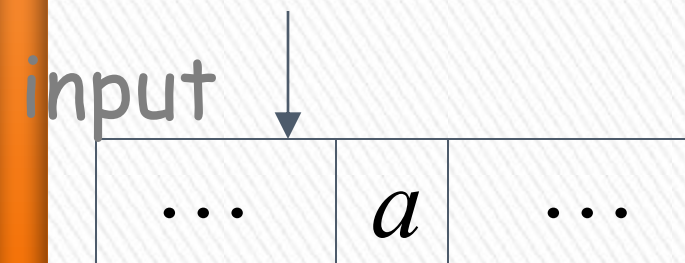
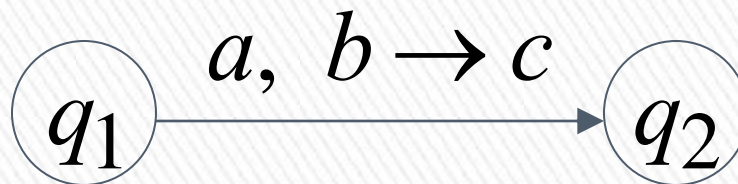
Pop
symbol

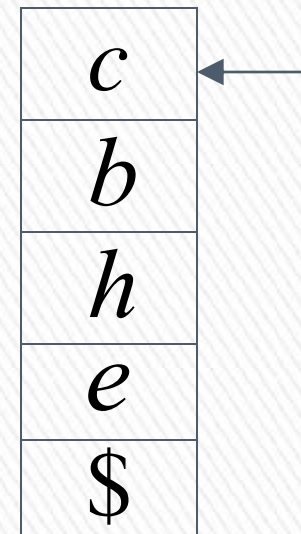
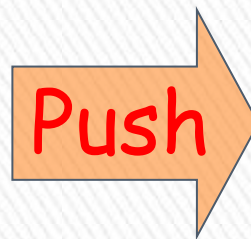
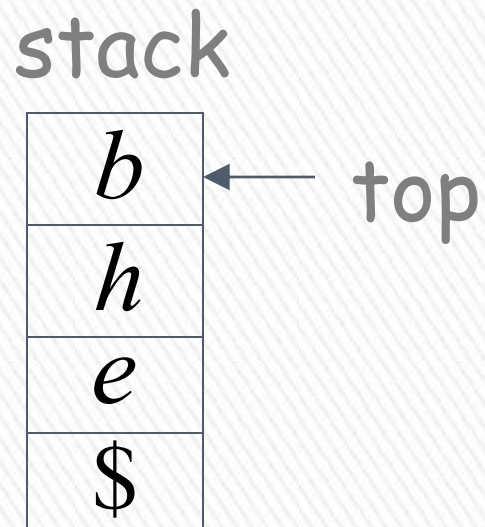
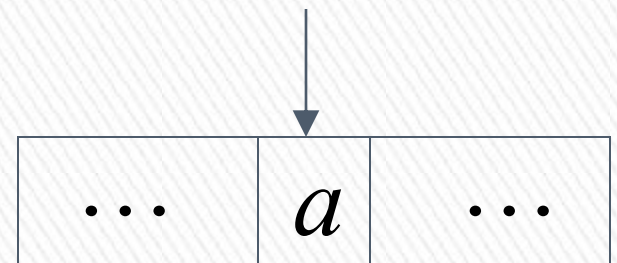
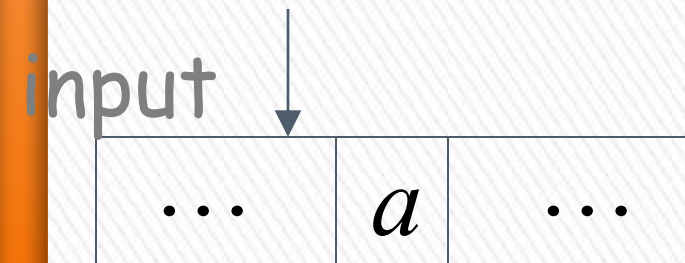
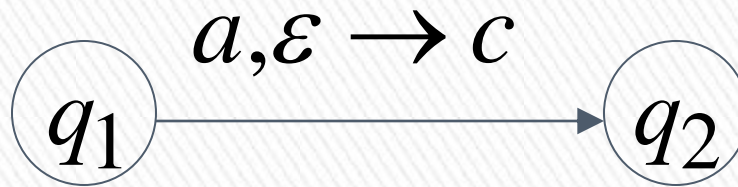
Push
symbol

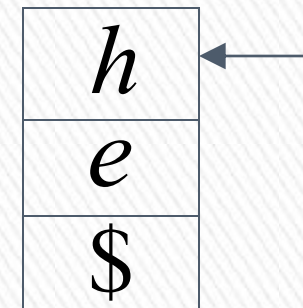
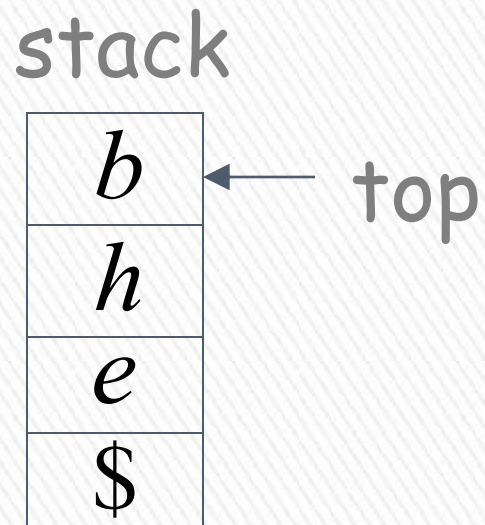
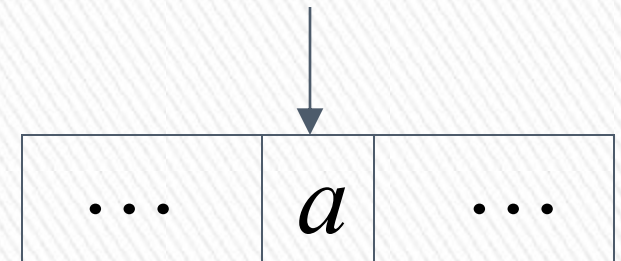
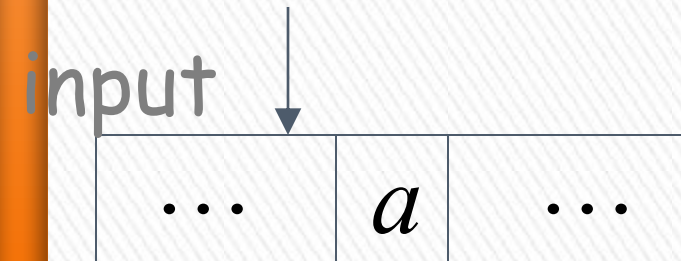
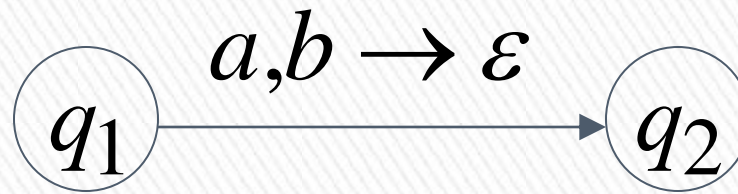


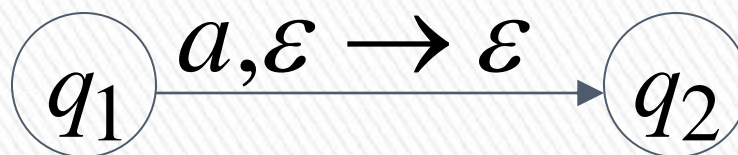
The States



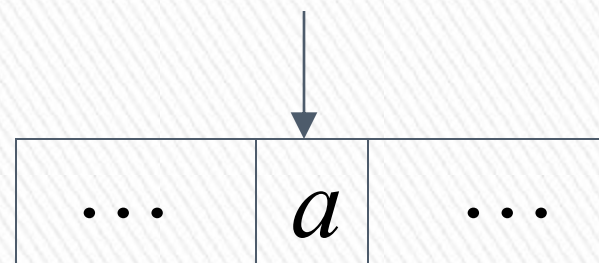
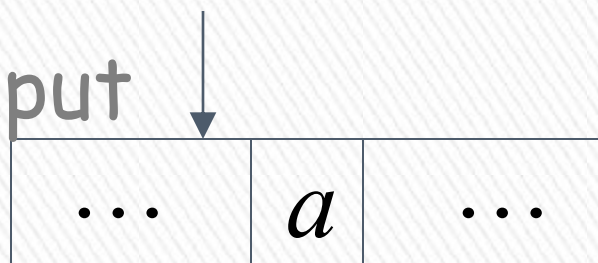




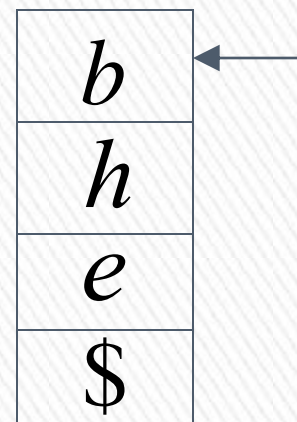
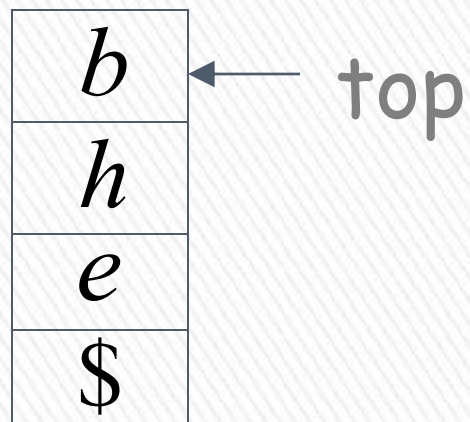




input



stack

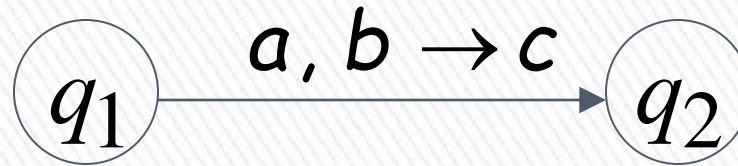


Pop from Empty Stack

input



stack



Stack
underflow

Pop

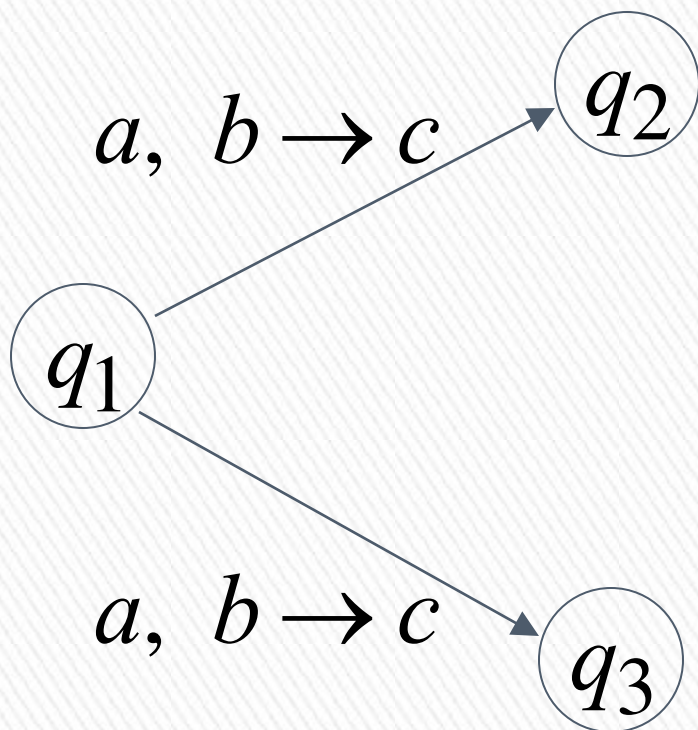
Automaton halts!

If the automaton attempts to pop from empty stack then it halts and rejects input

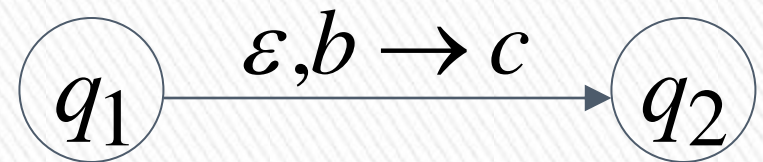
Non-Determinism

PDAs are non-deterministic

Allowed non-deterministic transitions



$\hookrightarrow$ NFA
§.6:



ϵ – transition

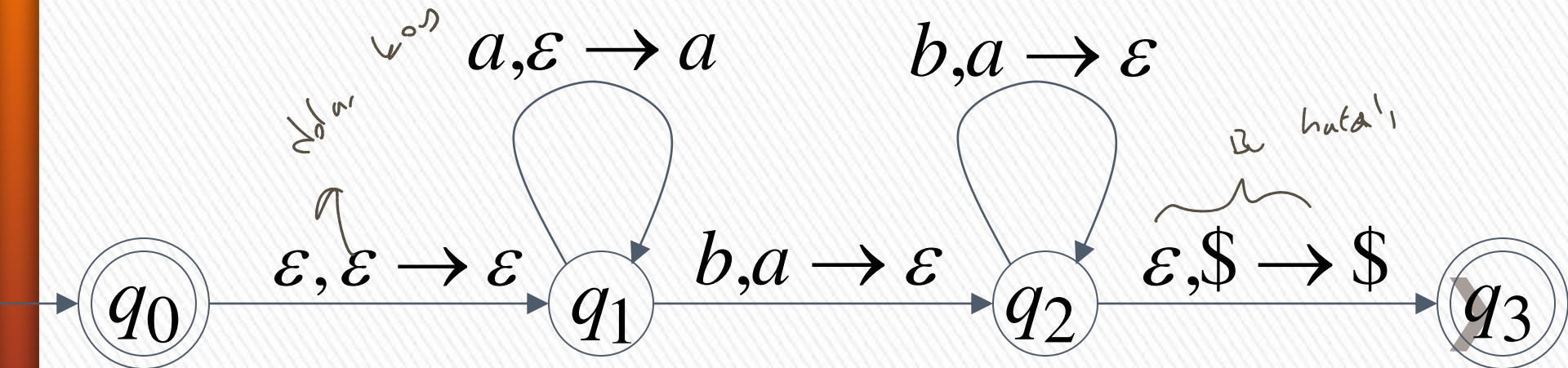


Example PDA

PDA M : $L(M) = \{a^n b^n : n \geq 0\}$

a

b



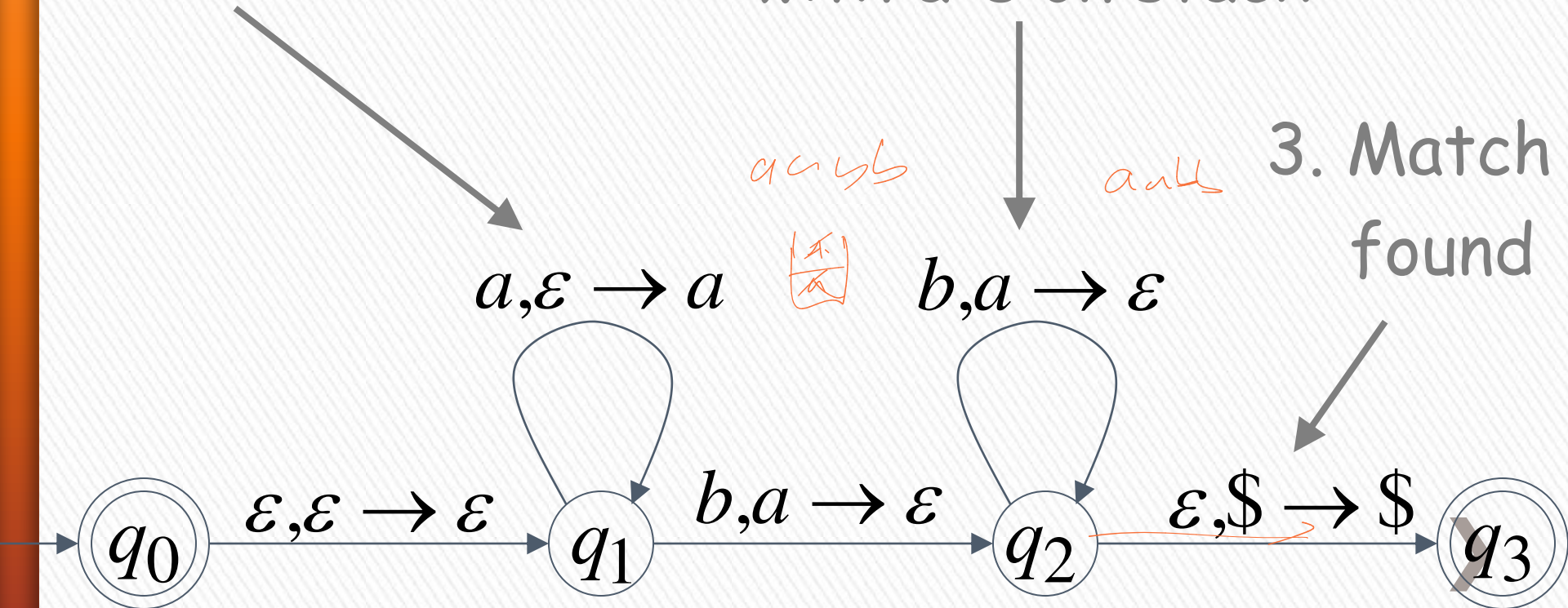
$$L(M) = \{a^n b^n : n \geq 0\}$$

Basic Idea:

1. Push the a's on the stack

2. Match the b's on input with a's on stack

3. Match found



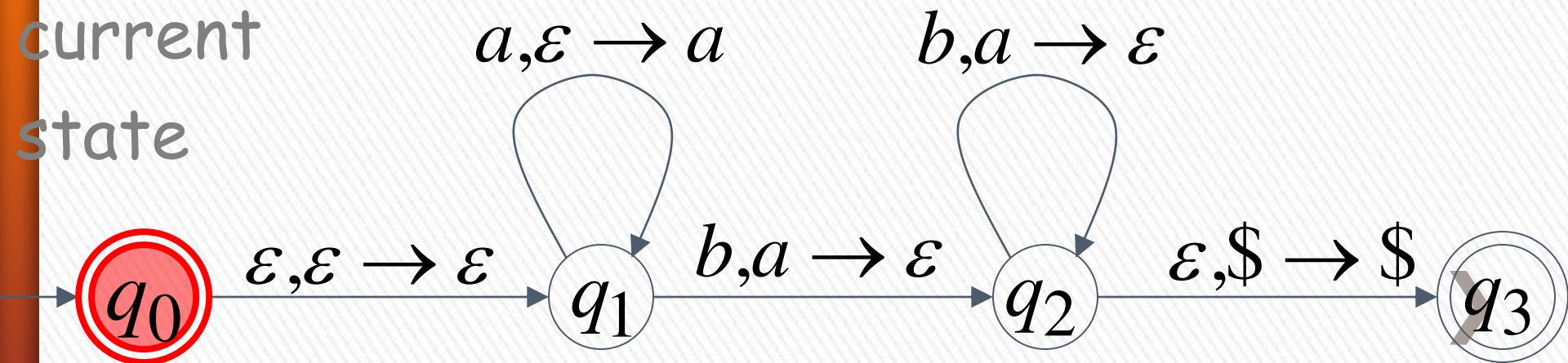
Execution Example: Time 0

Input



Stack

current
state

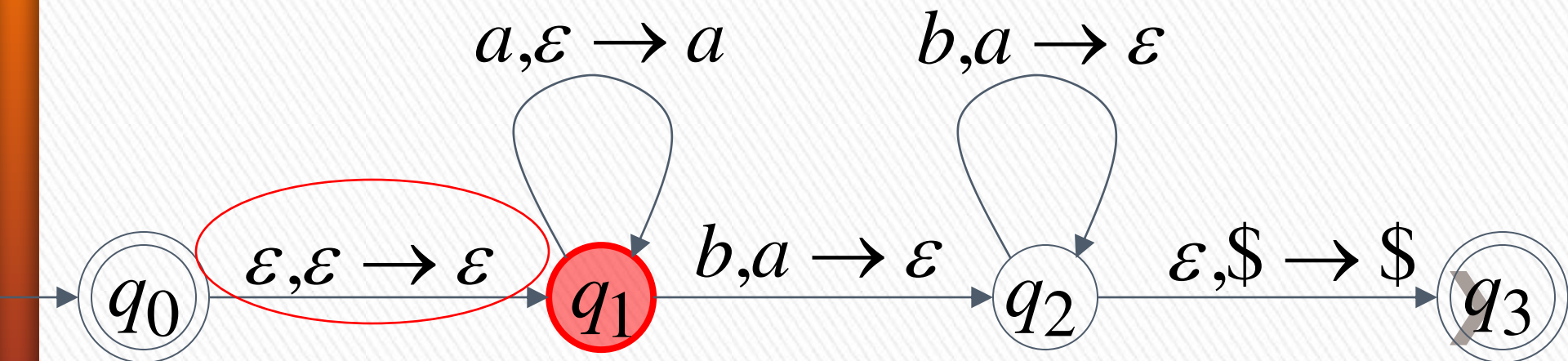


Time 1

Input

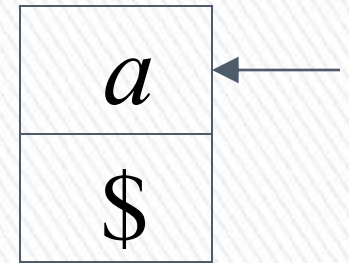
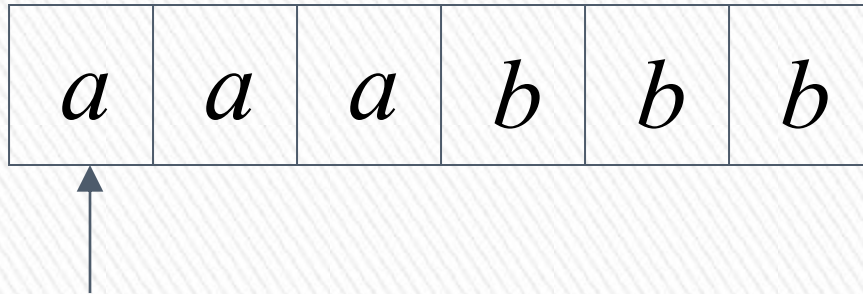


Stack

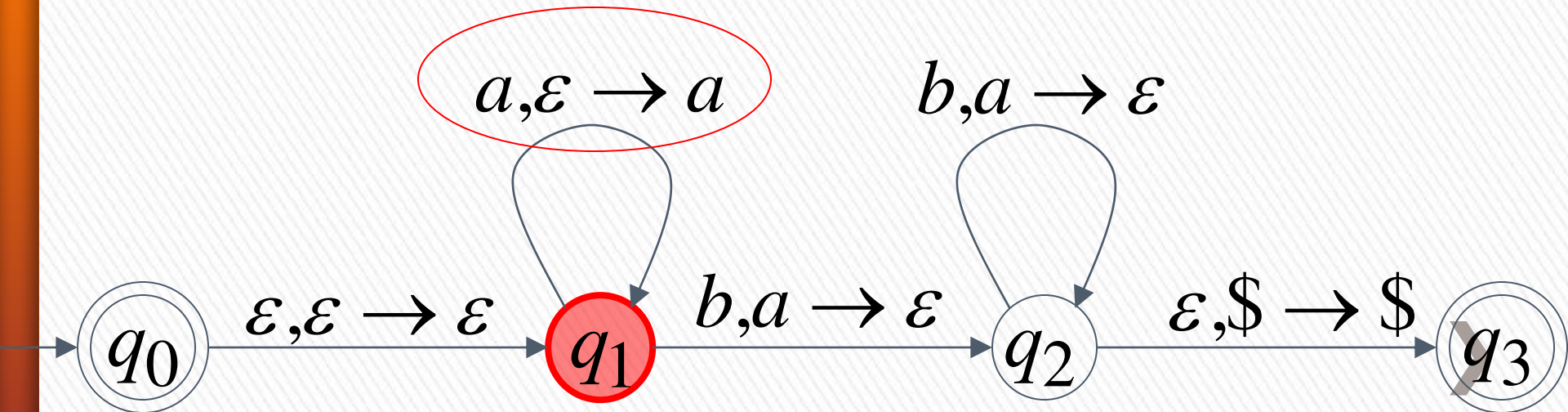


Time 2

Input

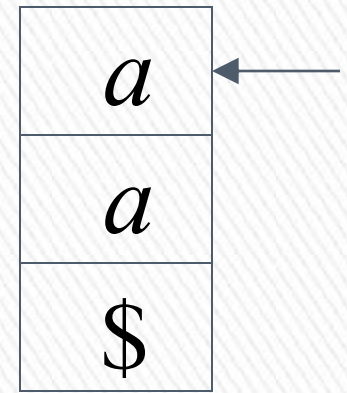
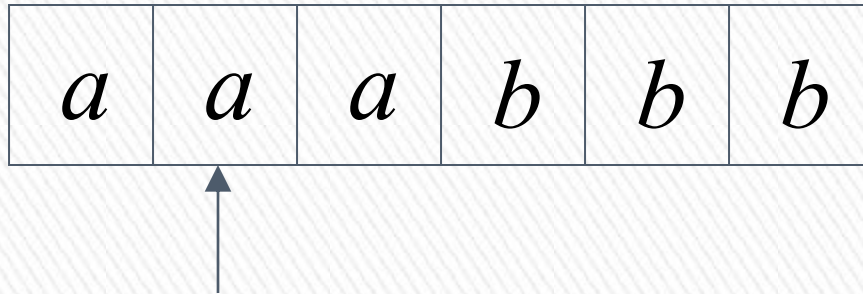


Stack

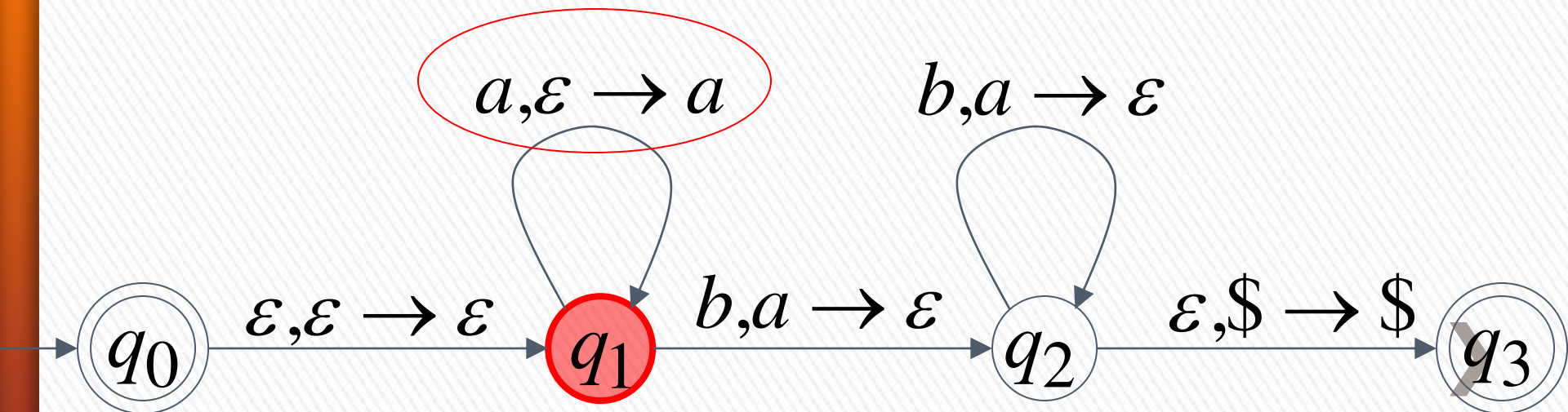


Time 3

Input

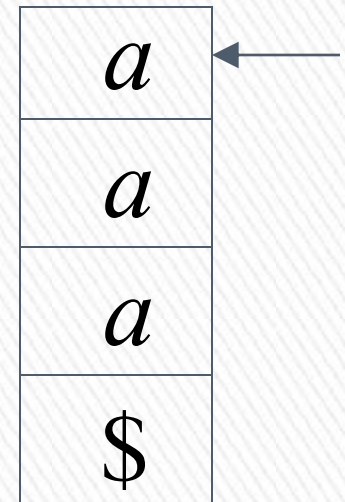


Stack

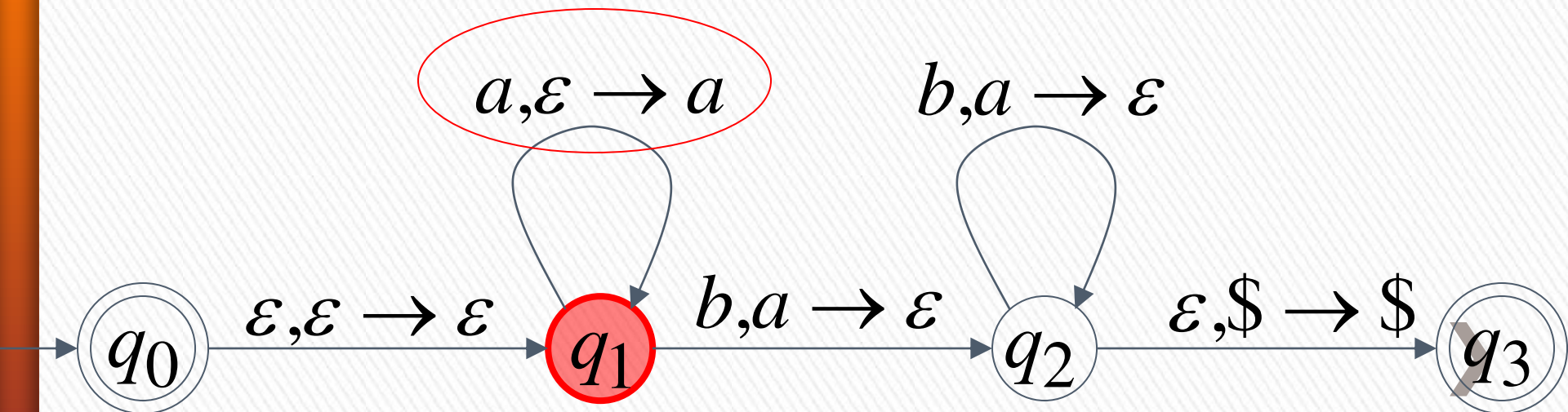


Time 4

Input

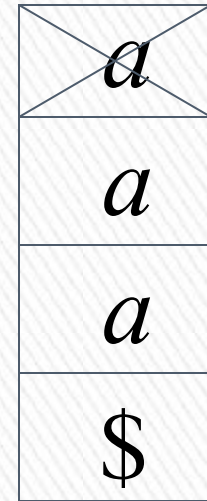


Stack

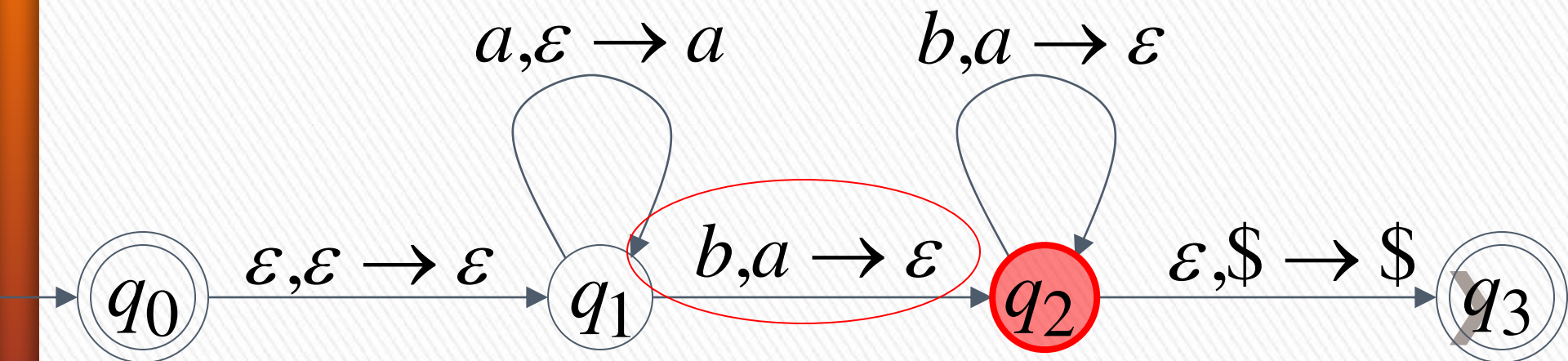


Time 5

Input

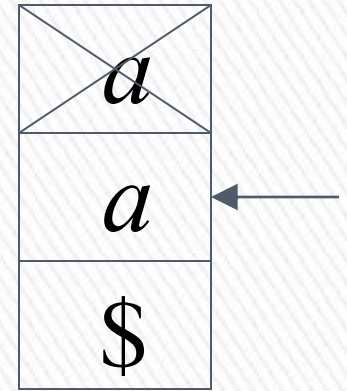


Stack

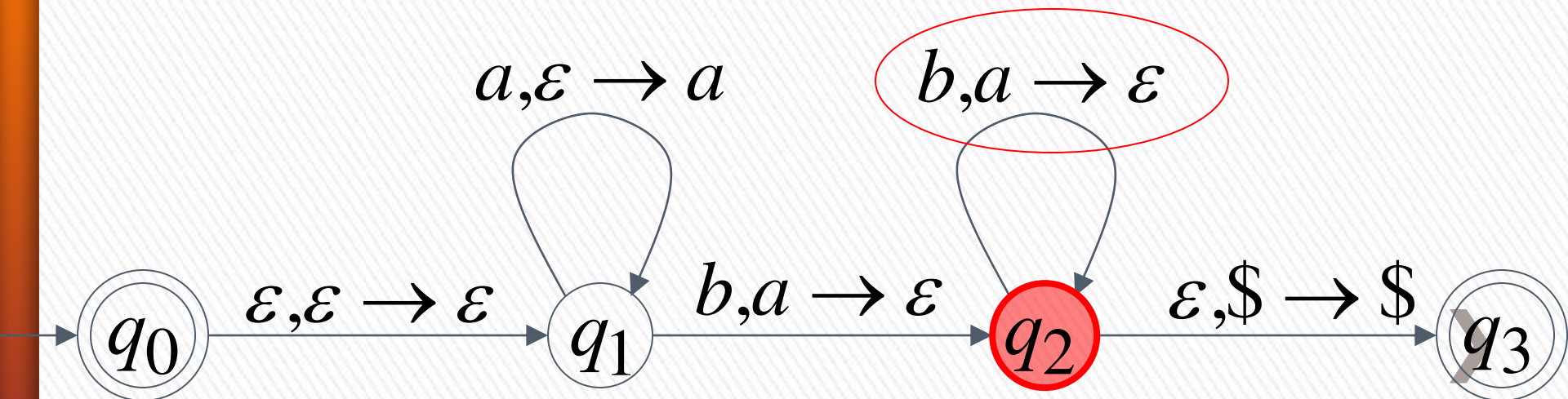


Time 6

Input

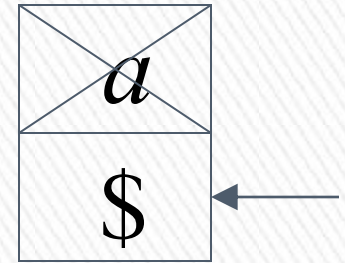
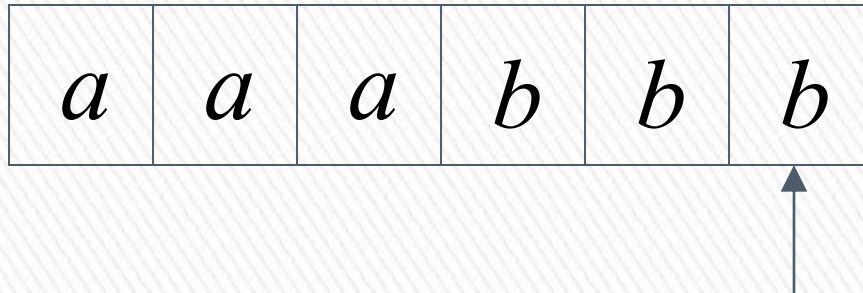


Stack

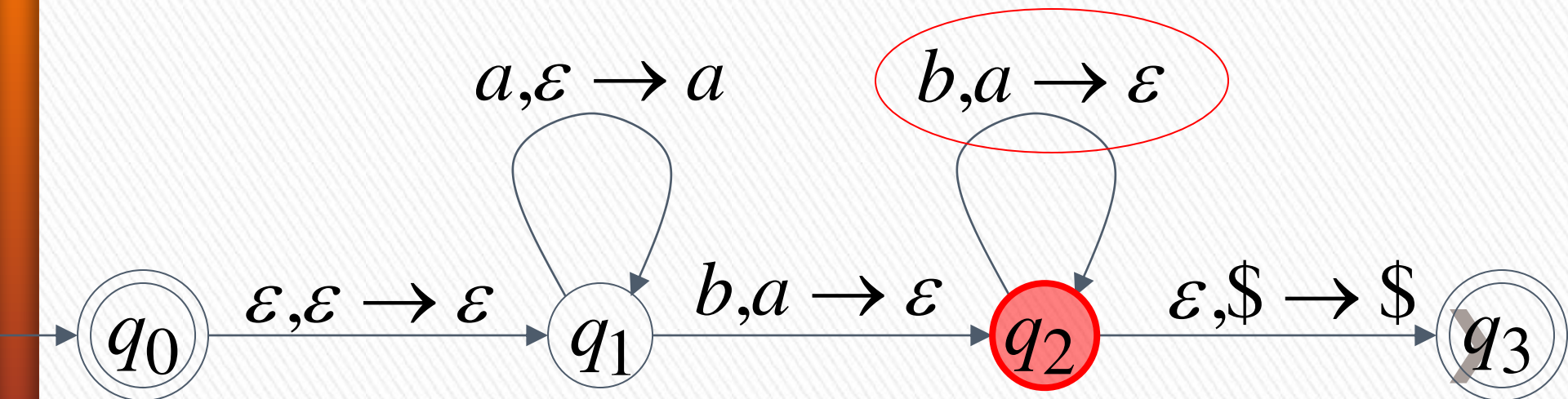


Time 7

Input



Stack

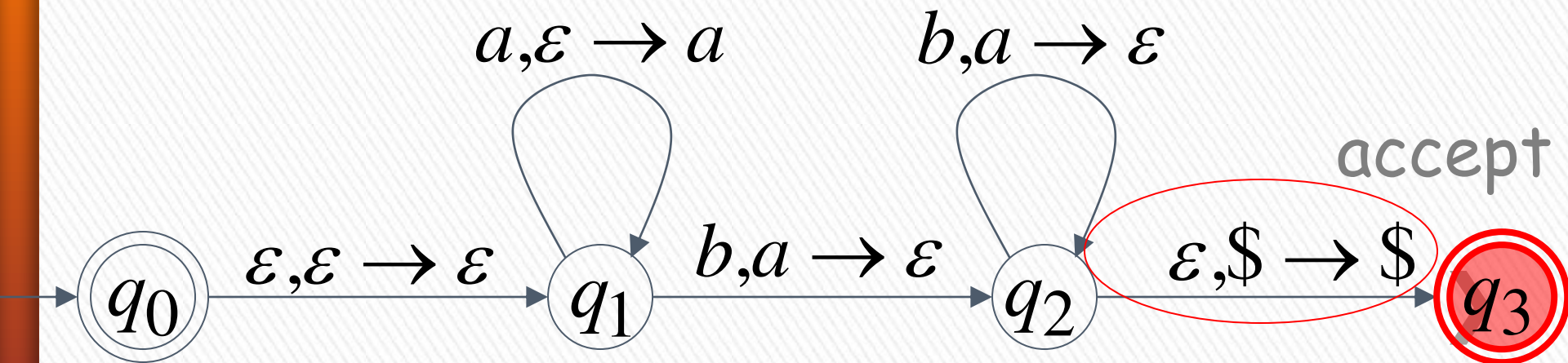


Time 8

Input



Stack



A string is accepted if there is
a computation such that:

All the input is consumed

AND

The last state is an accepting state

we do not care about the stack contents
at the end of the accepting computation ➤

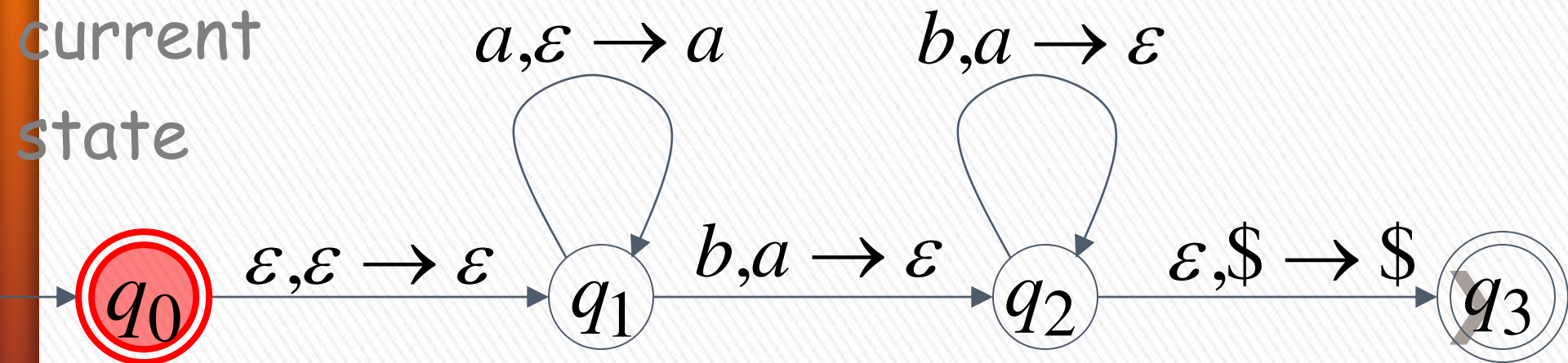
Rejection Example: Time 0

Input



Stack

current
state



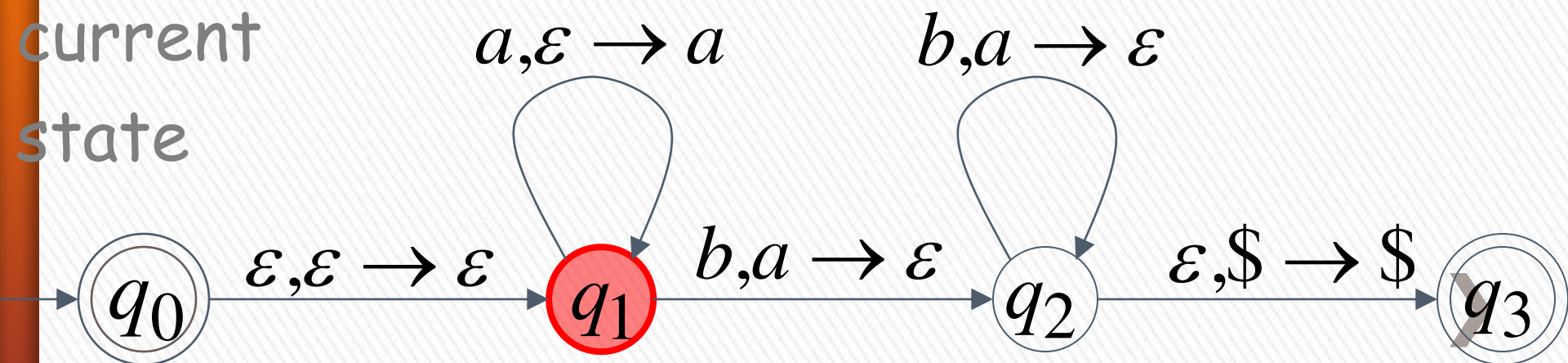
Rejection Example: Time 1

Input



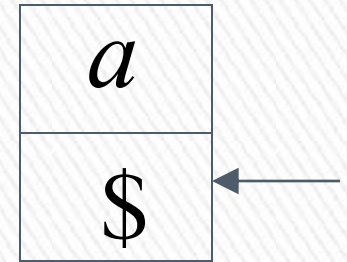
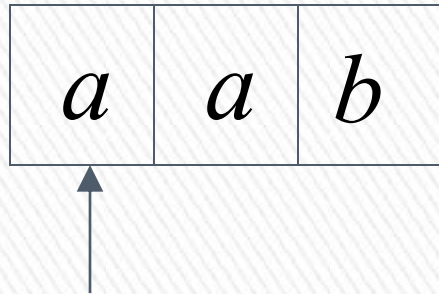
Stack

current
state



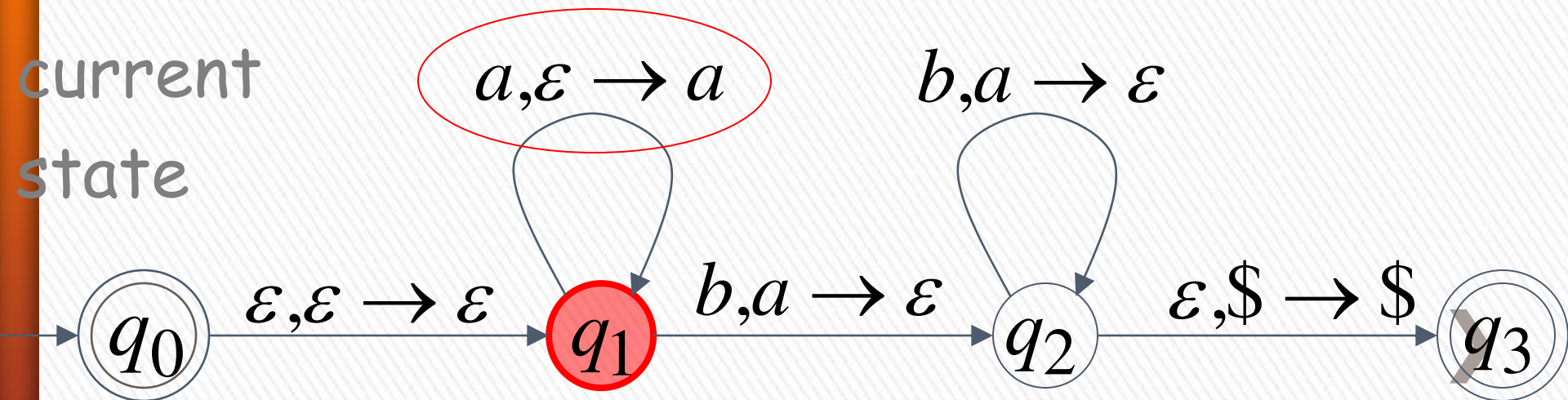
Rejection Example: Time 2

Input



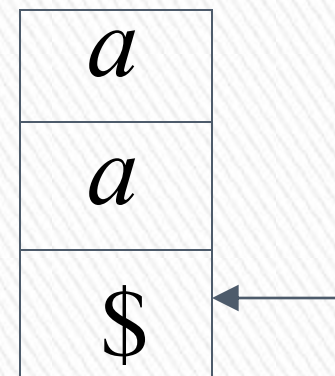
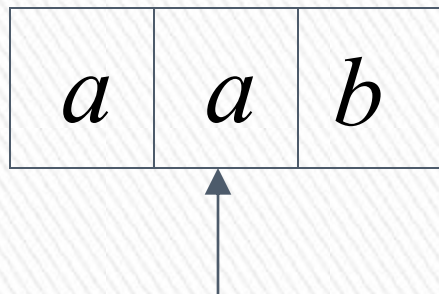
Stack

current
state



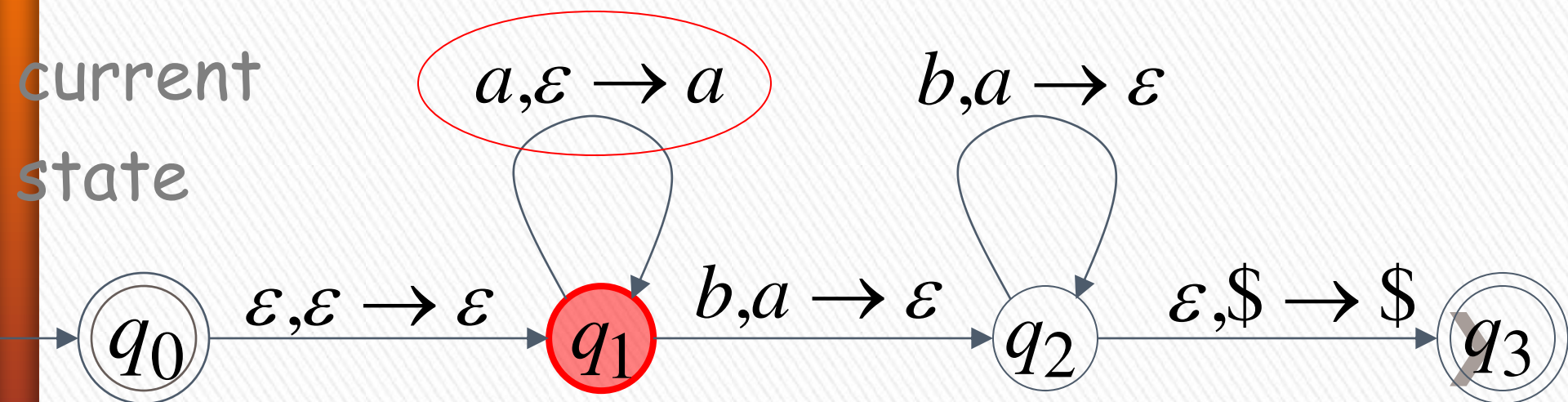
Rejection Example: Time 3

Input



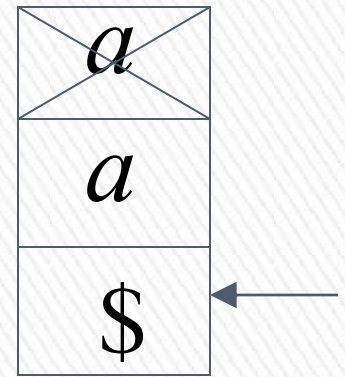
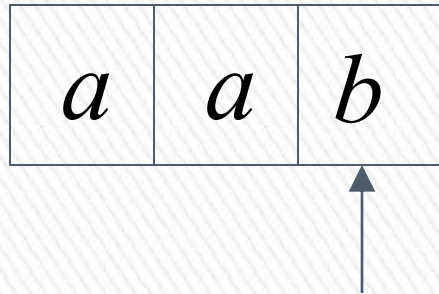
Stack

current
state



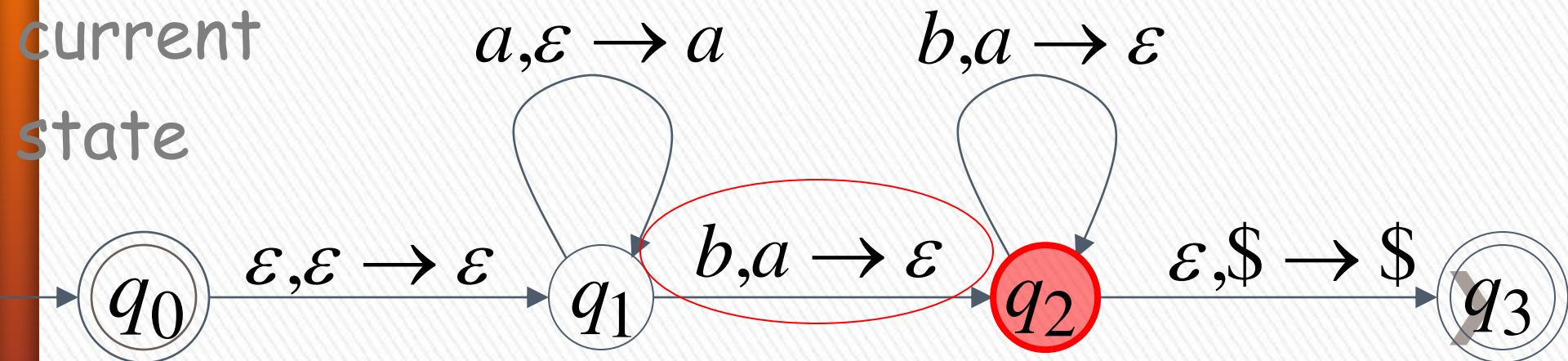
Rejection Example: Time 4

Input



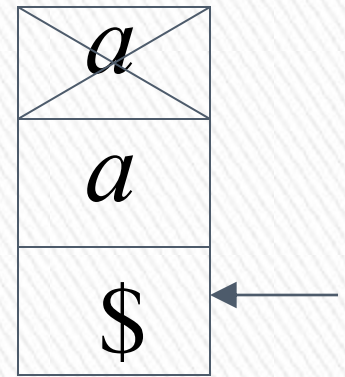
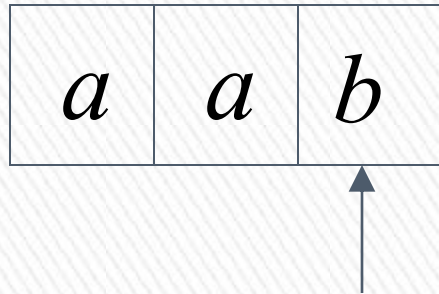
Stack

current
state



Rejection Example: Time 4

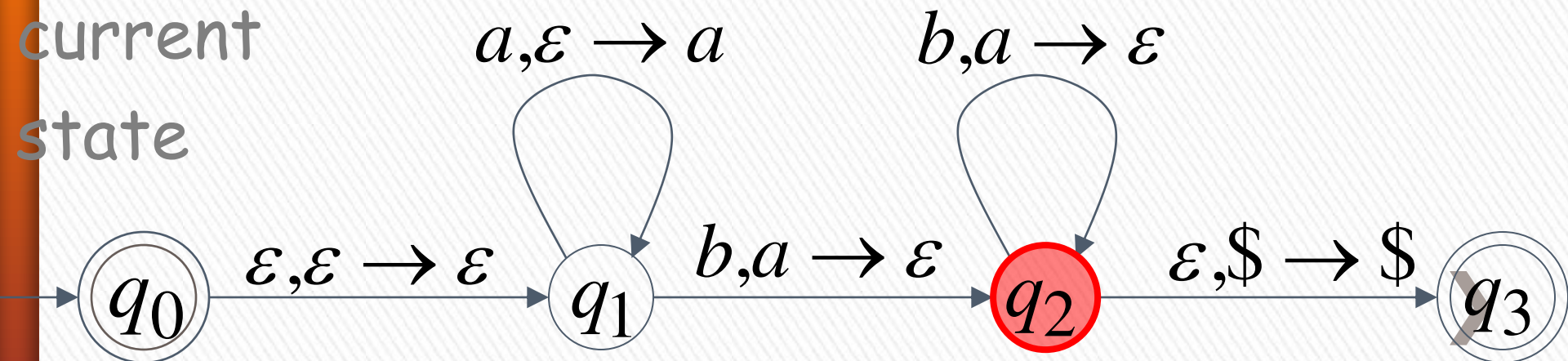
Input



Stack

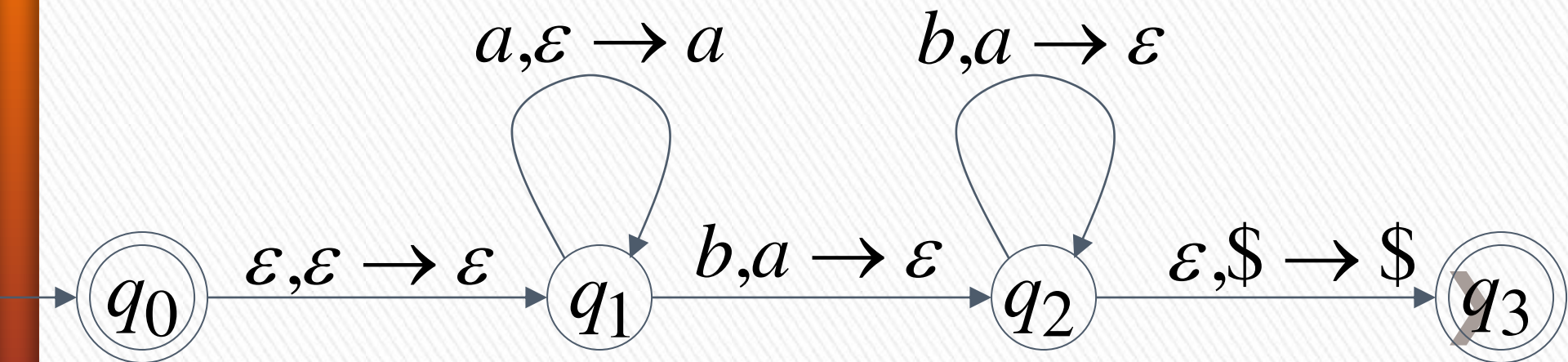
reject

current
state



There is no accepting computation for aab

The string aab is rejected by the PDA



Another PDA example

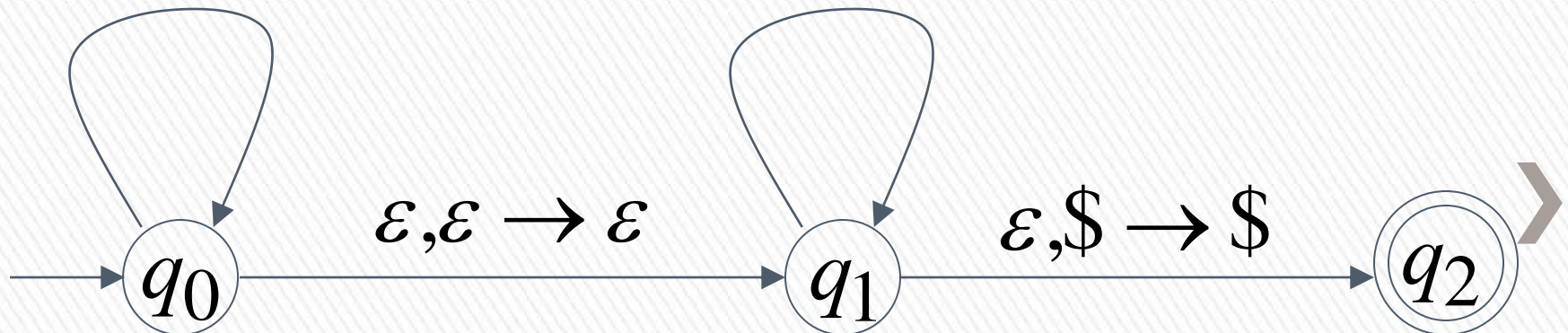
PDA M : $L(M) = \{vv^R : v \in \{a,b\}^*\}$

$a, \varepsilon \rightarrow a$

$a, a \rightarrow \varepsilon$

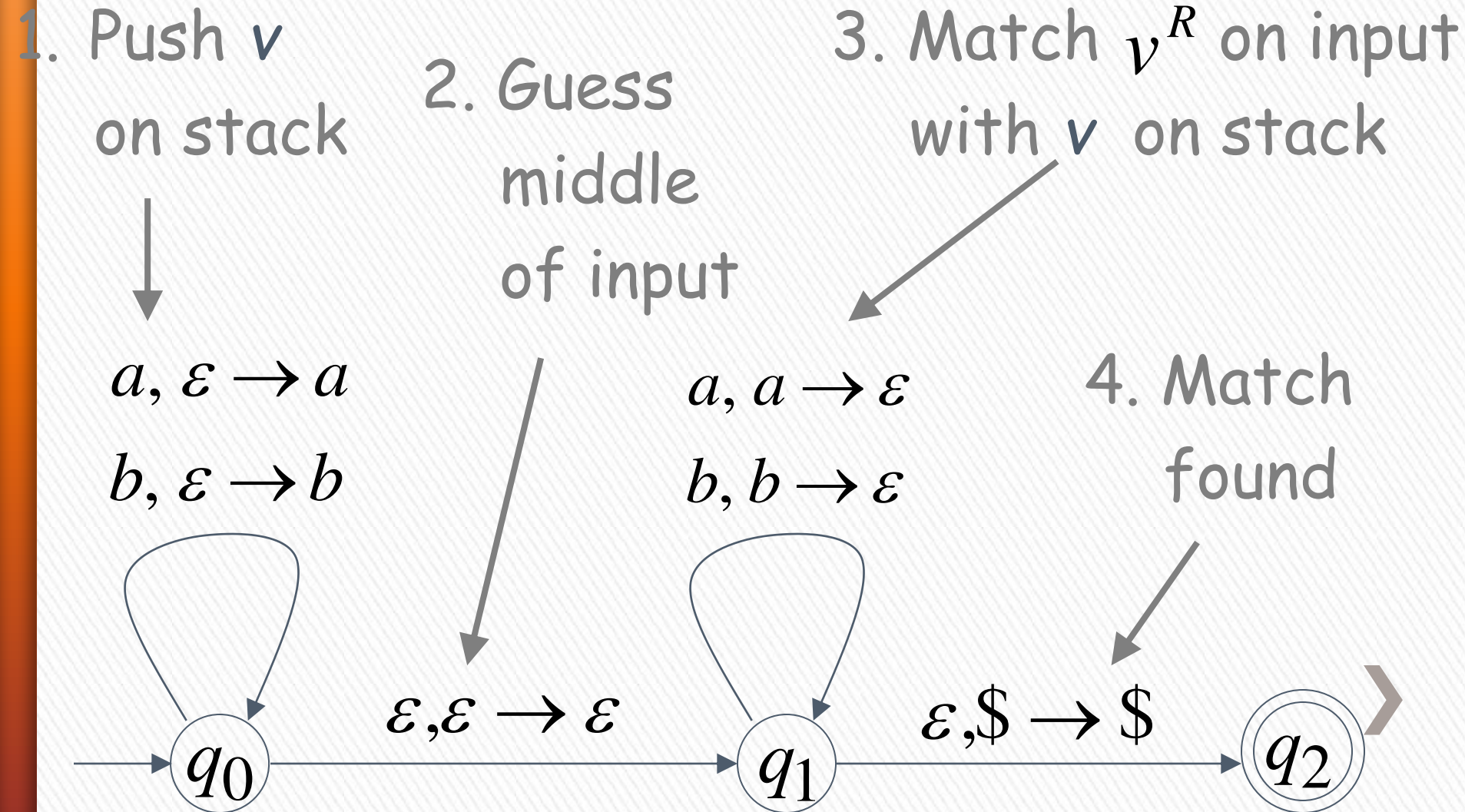
$b, \varepsilon \rightarrow b$

$b, b \rightarrow \varepsilon$



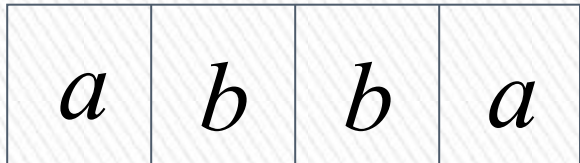
Basic Idea:

$$L(M) = \{vv^R : v \in \{a,b\}^*\}$$



Execution Example: Time 0

Input



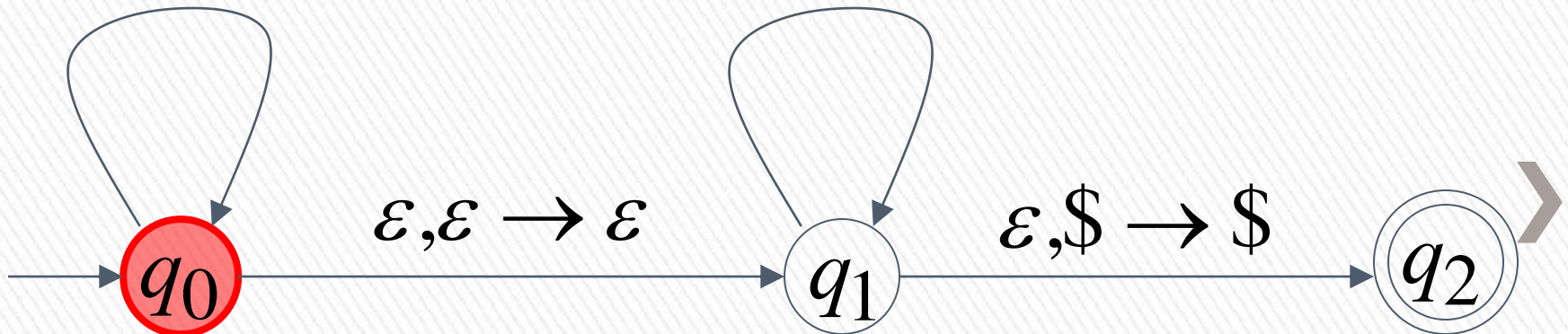
Stack

$a, \varepsilon \rightarrow a$

$a, a \rightarrow \varepsilon$

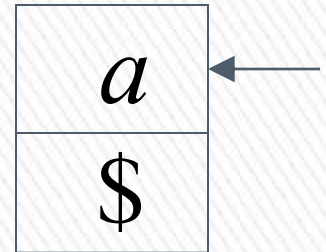
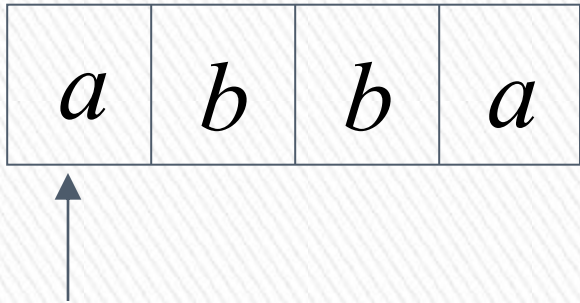
$b, \varepsilon \rightarrow b$

$b, b \rightarrow \varepsilon$



Time 1

Input



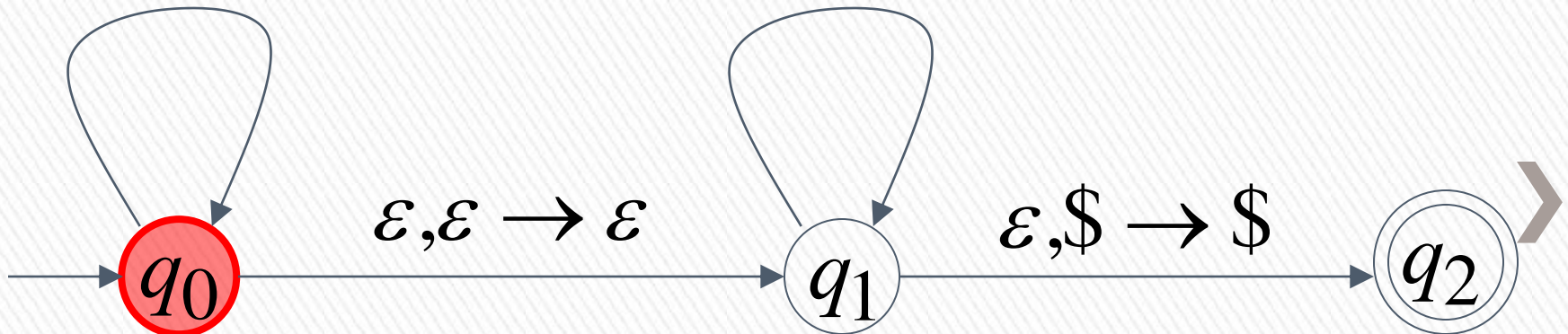
Stack

$a, \varepsilon \rightarrow a$

$b, \varepsilon \rightarrow b$

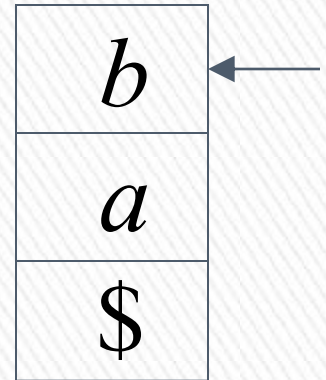
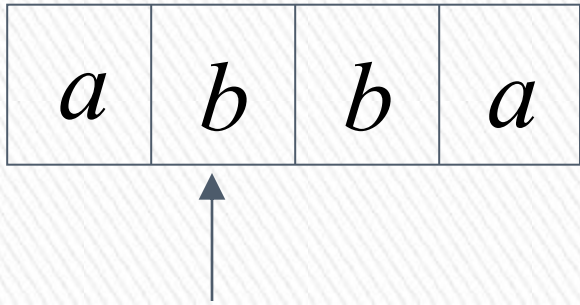
$a, a \rightarrow \varepsilon$

$b, b \rightarrow \varepsilon$



Time 2

Input



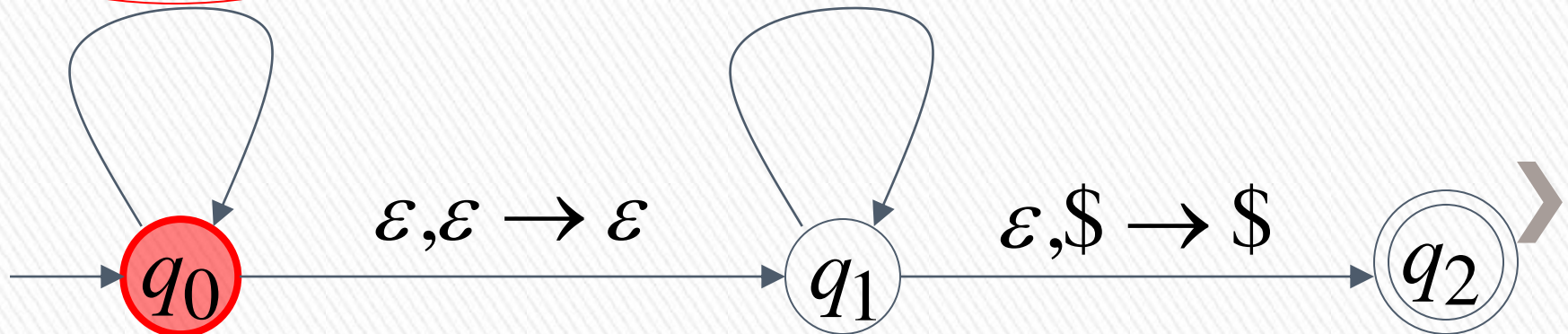
Stack

$a, \varepsilon \rightarrow a$

$b, \varepsilon \rightarrow b$

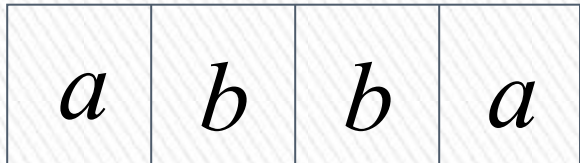
$a, a \rightarrow \varepsilon$

$b, b \rightarrow \varepsilon$

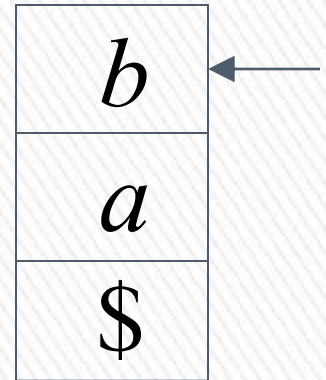


Time 3

Input



Guess the middle
of string



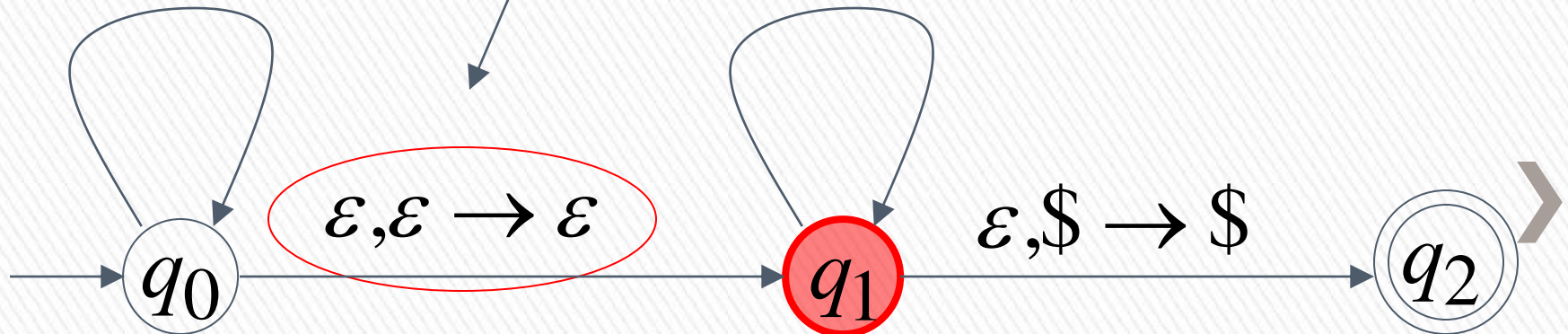
Stack

$a, \varepsilon \rightarrow a$

$b, \varepsilon \rightarrow b$

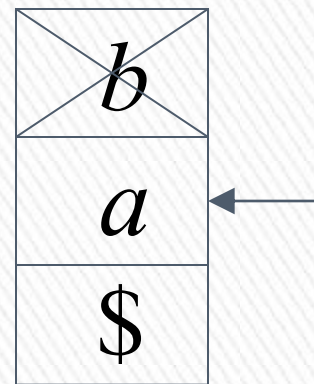
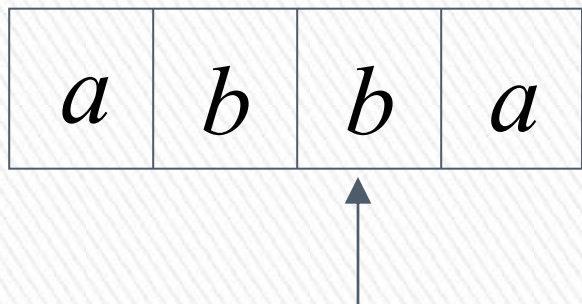
$a, a \rightarrow \varepsilon$

$b, b \rightarrow \varepsilon$



Time 4

Input



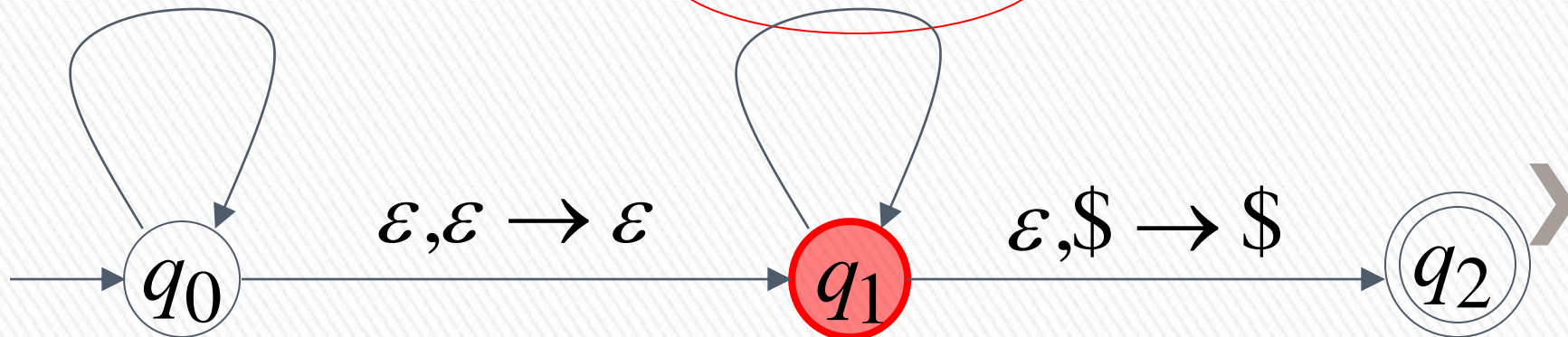
Stack

$a, \varepsilon \rightarrow a$

$b, \varepsilon \rightarrow b$

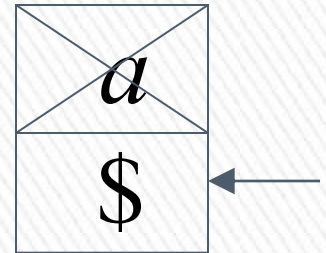
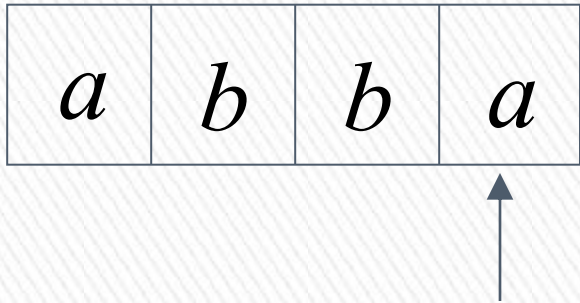
$a, a \rightarrow \varepsilon$

$b, b \rightarrow \varepsilon$



Time 5

Input



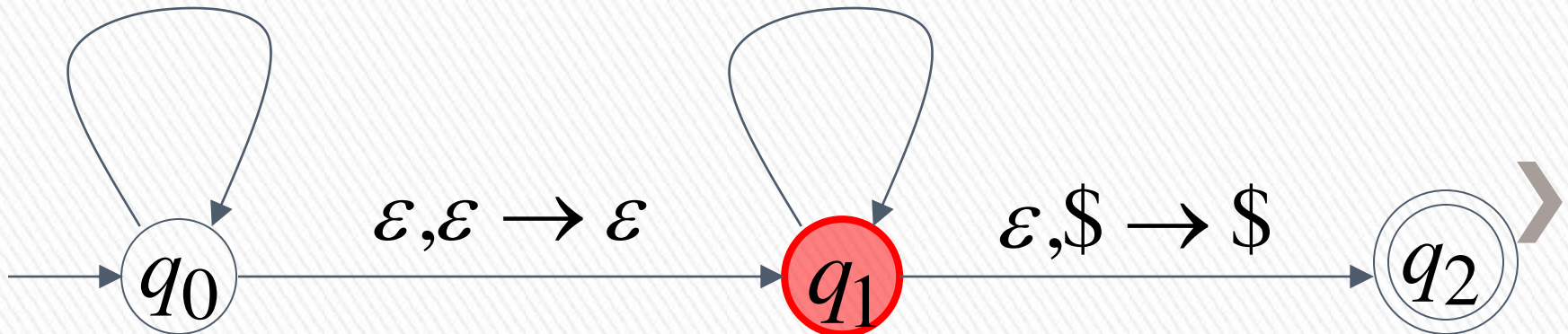
Stack

$a, \varepsilon \rightarrow a$

$b, \varepsilon \rightarrow b$

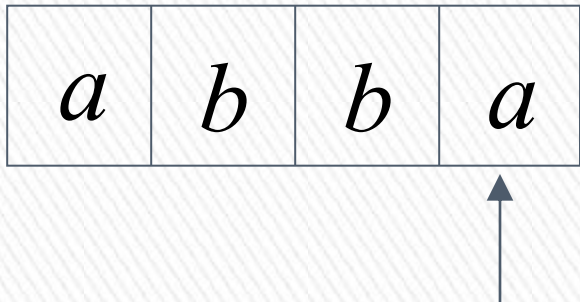
$a, a \rightarrow \varepsilon$

$b, b \rightarrow \varepsilon$



Time 6

Input



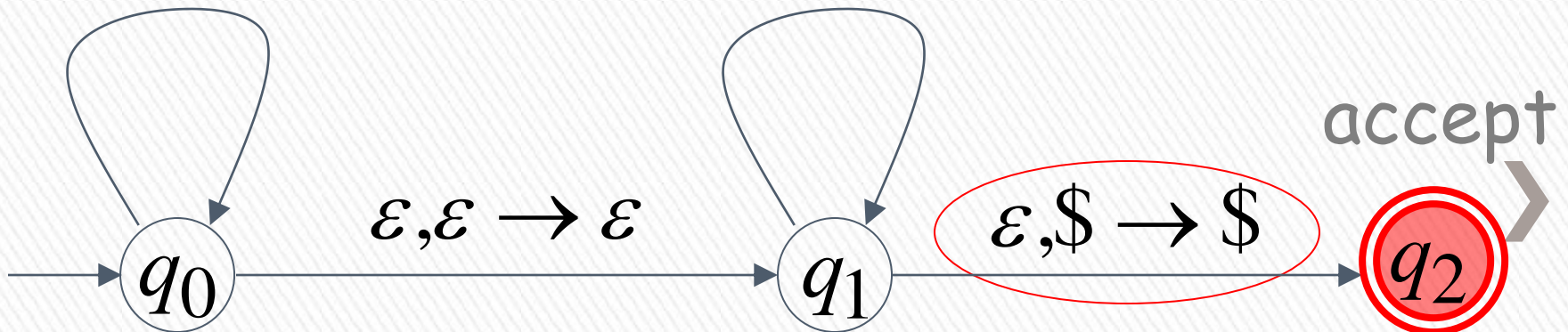
Stack

$a, \varepsilon \rightarrow a$

$a, a \rightarrow \varepsilon$

$b, \varepsilon \rightarrow b$

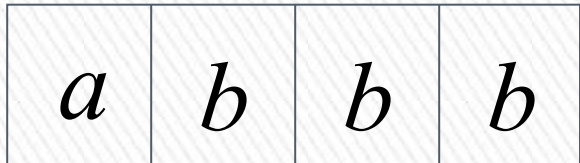
$b, b \rightarrow \varepsilon$



Rejection Example:

Time 0

Input



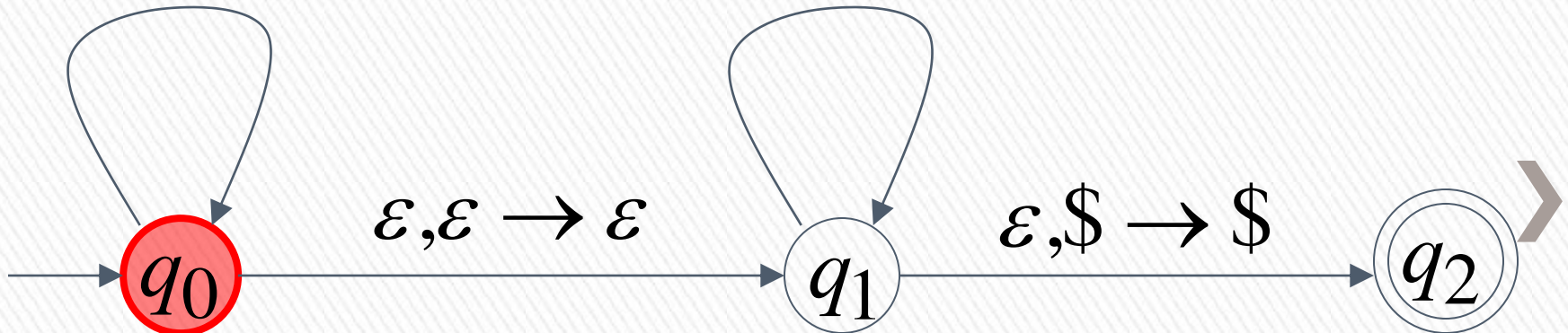
Stack

$a, \varepsilon \rightarrow a$

$a, a \rightarrow \varepsilon$

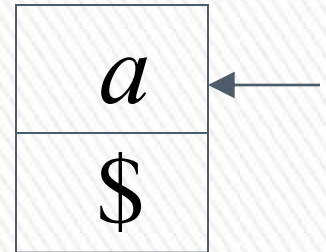
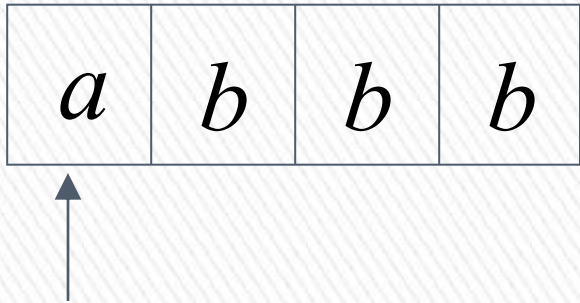
$b, \varepsilon \rightarrow b$

$b, b \rightarrow \varepsilon$



Time 1

Input



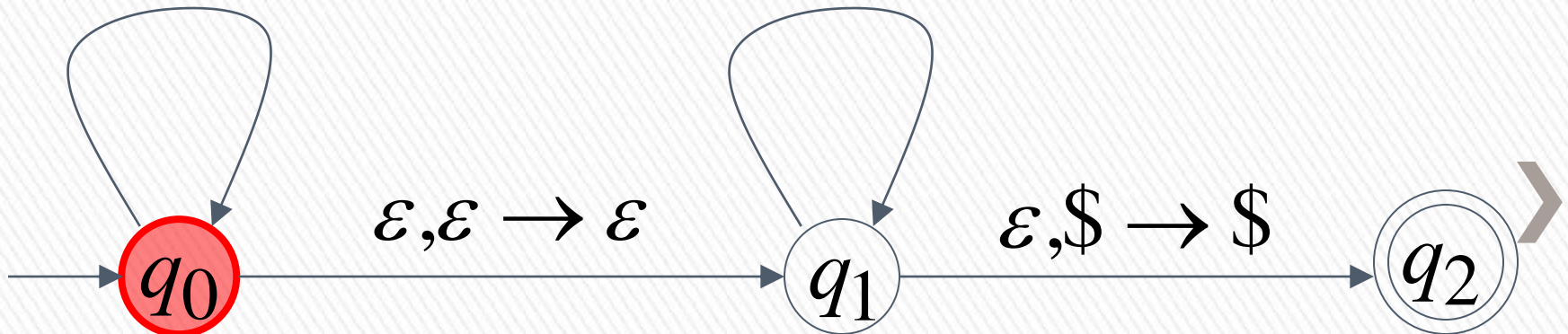
Stack

$a, \varepsilon \rightarrow a$

$b, \varepsilon \rightarrow b$

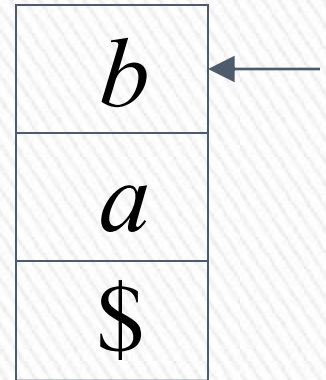
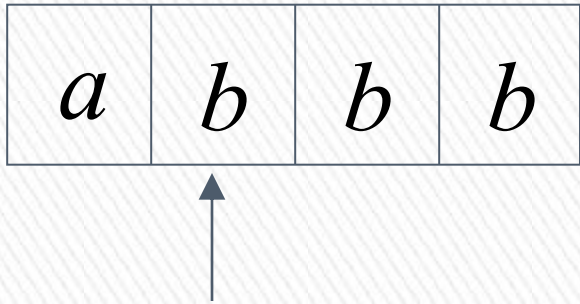
$a, a \rightarrow \varepsilon$

$b, b \rightarrow \varepsilon$



Time 2

Input



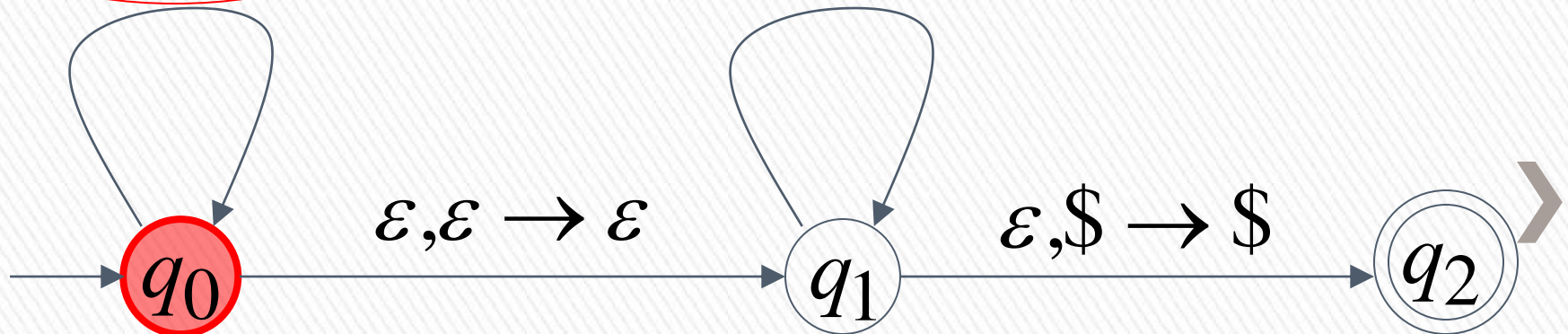
Stack

$a, \varepsilon \rightarrow a$

$b, \varepsilon \rightarrow b$

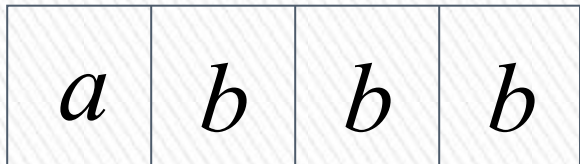
$a, a \rightarrow \varepsilon$

$b, b \rightarrow \varepsilon$

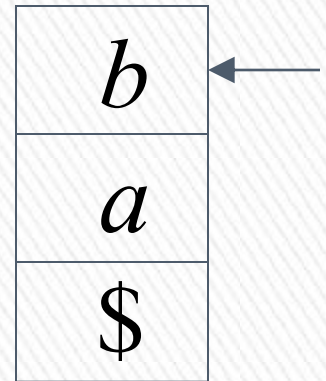


Time 3

Input



Guess the middle
of string



Stack

$a, \varepsilon \rightarrow a$

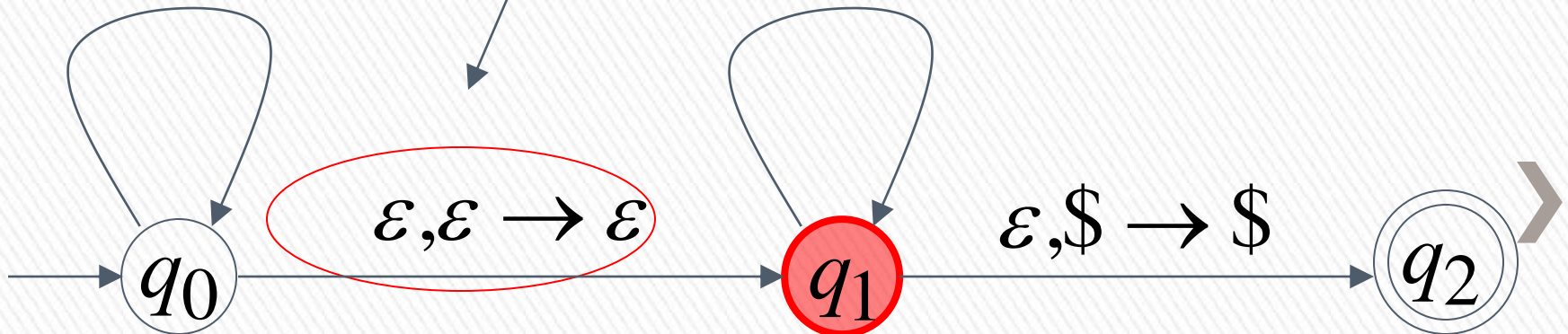
$b, \varepsilon \rightarrow b$

$a, a \rightarrow \varepsilon$

$b, b \rightarrow \varepsilon$

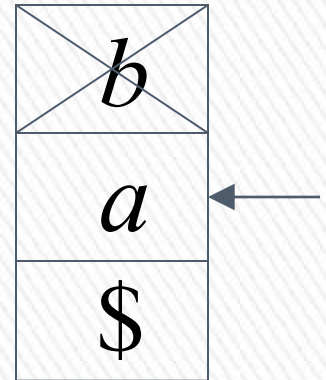
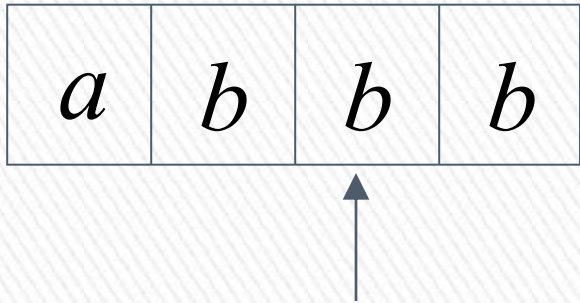
$\varepsilon, \varepsilon \rightarrow \varepsilon$

$\varepsilon, \$ \rightarrow \$$



Time 4

Input



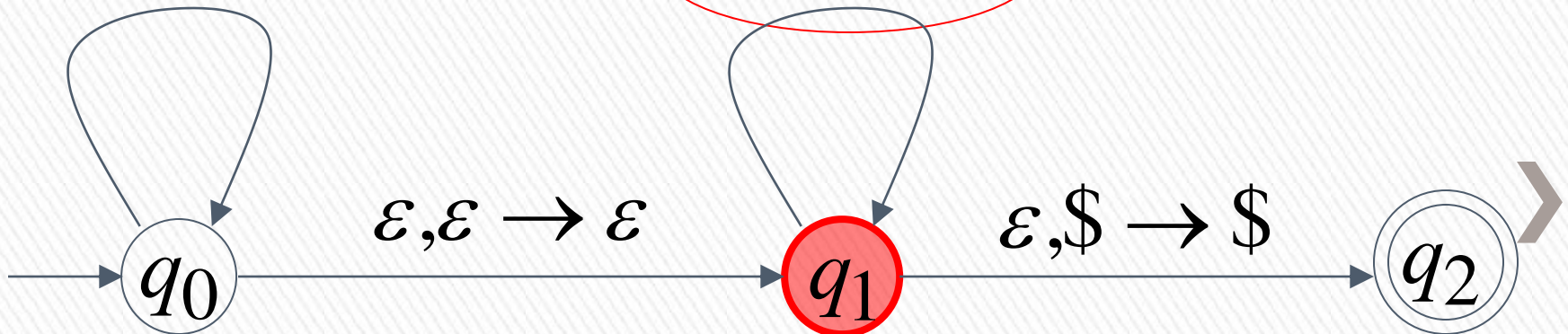
Stack

$a, \varepsilon \rightarrow a$

$b, \varepsilon \rightarrow b$

$a, a \rightarrow \varepsilon$

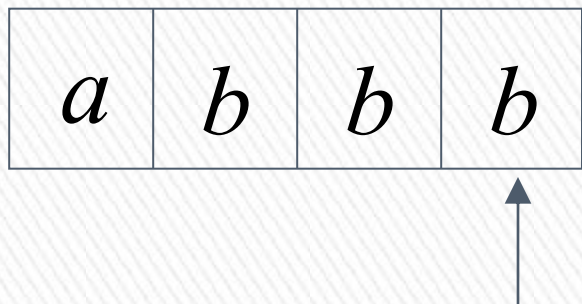
$b, b \rightarrow \varepsilon$



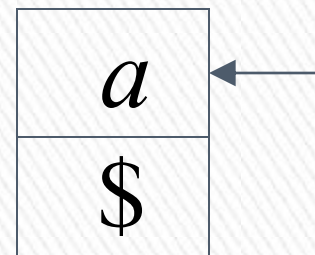
Time 5

Input

There is no possible transition.



Input is not consumed



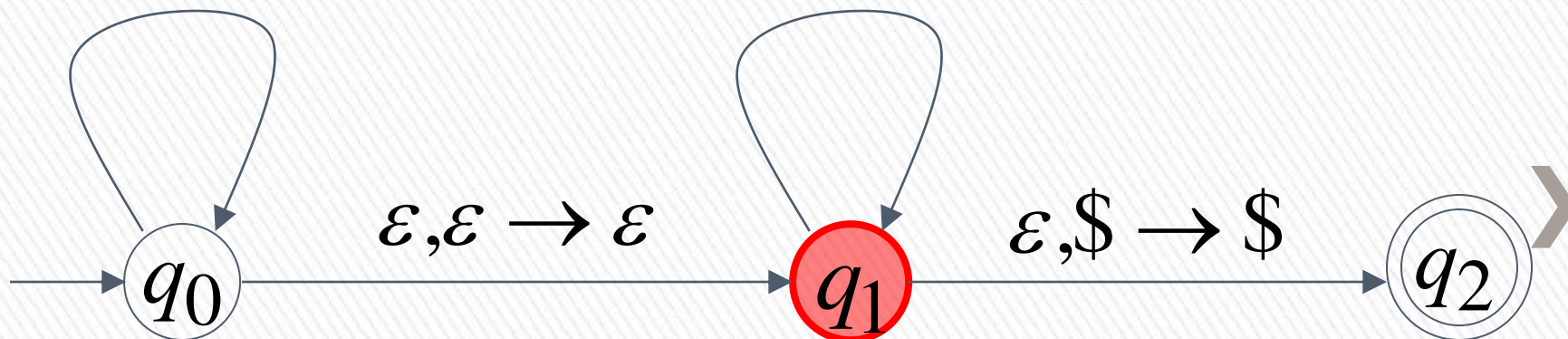
Stack

$a, \varepsilon \rightarrow a$

$a, a \rightarrow \varepsilon$

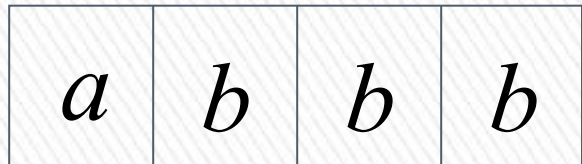
$b, \varepsilon \rightarrow b$

$b, b \rightarrow \varepsilon$



Another computation on same string:

Input



Time 0



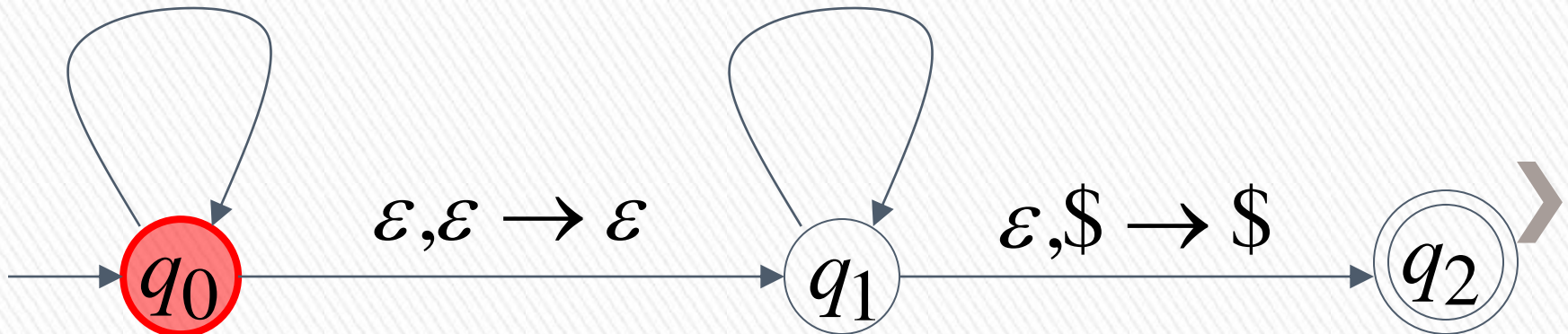
Stack

$a, \varepsilon \rightarrow a$

$a, a \rightarrow \varepsilon$

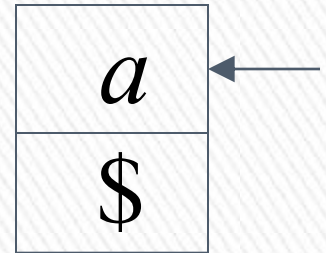
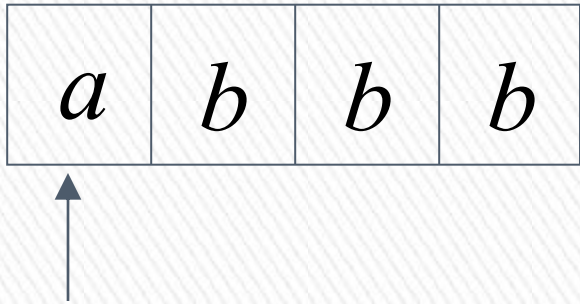
$b, \varepsilon \rightarrow b$

$b, b \rightarrow \varepsilon$



Time 1

Input



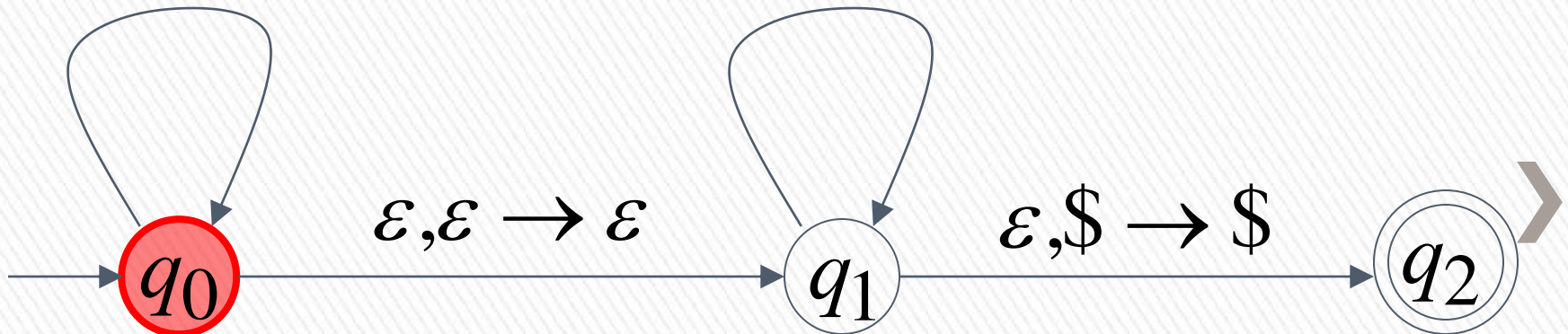
Stack

$a, \varepsilon \rightarrow a$

$b, \varepsilon \rightarrow b$

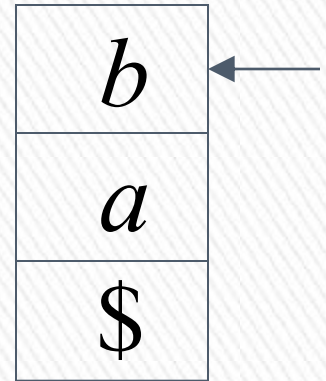
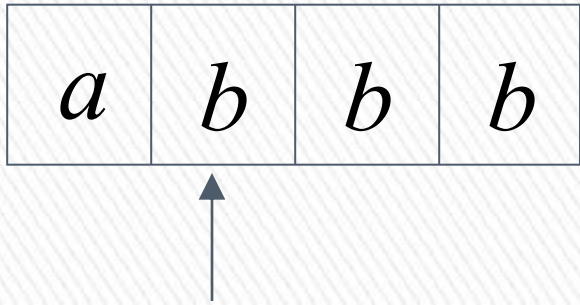
$a, a \rightarrow \varepsilon$

$b, b \rightarrow \varepsilon$



Time 2

Input



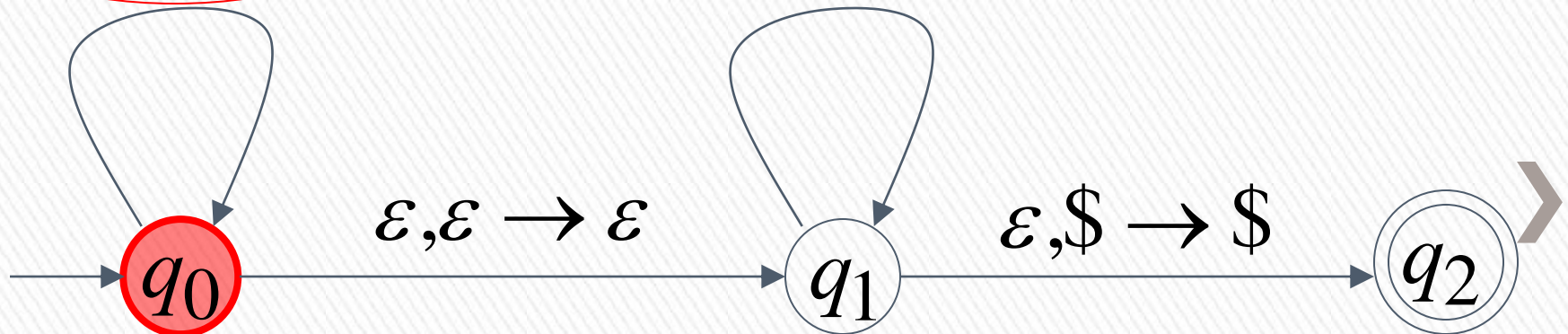
Stack

$a, \varepsilon \rightarrow a$

$b, \varepsilon \rightarrow b$

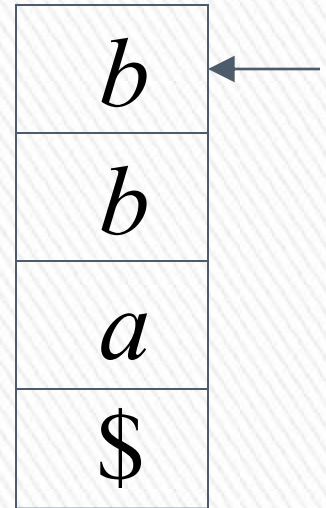
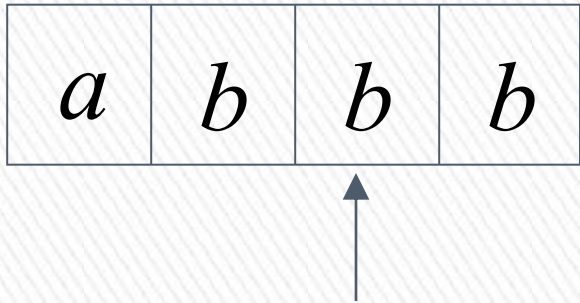
$a, a \rightarrow \varepsilon$

$b, b \rightarrow \varepsilon$



Time 3

Input



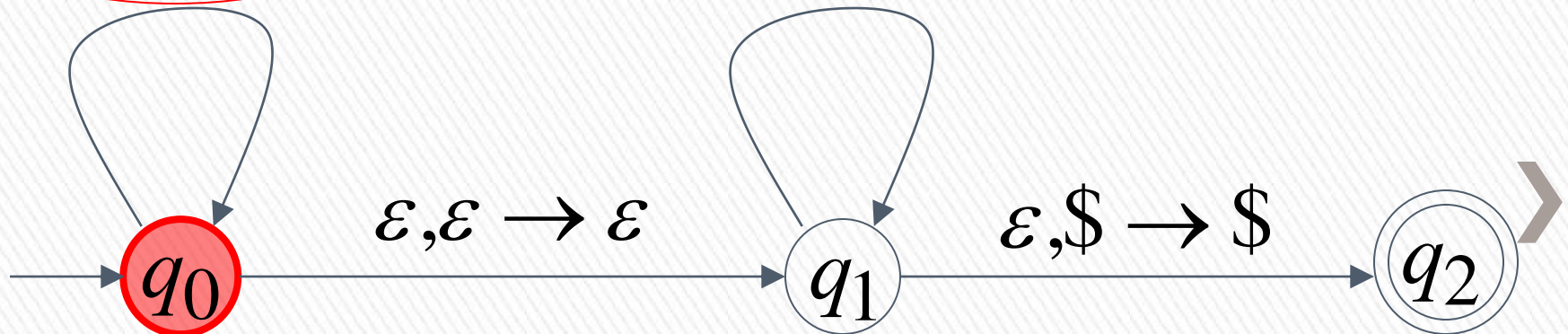
Stack

$a, \varepsilon \rightarrow a$

$b, \varepsilon \rightarrow b$

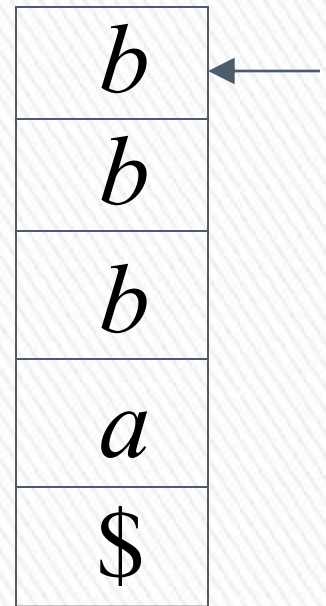
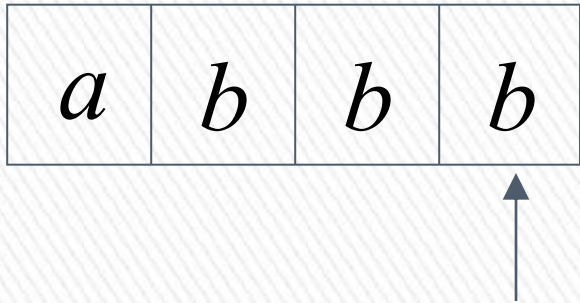
$a, a \rightarrow \varepsilon$

$b, b \rightarrow \varepsilon$



Time 4

Input



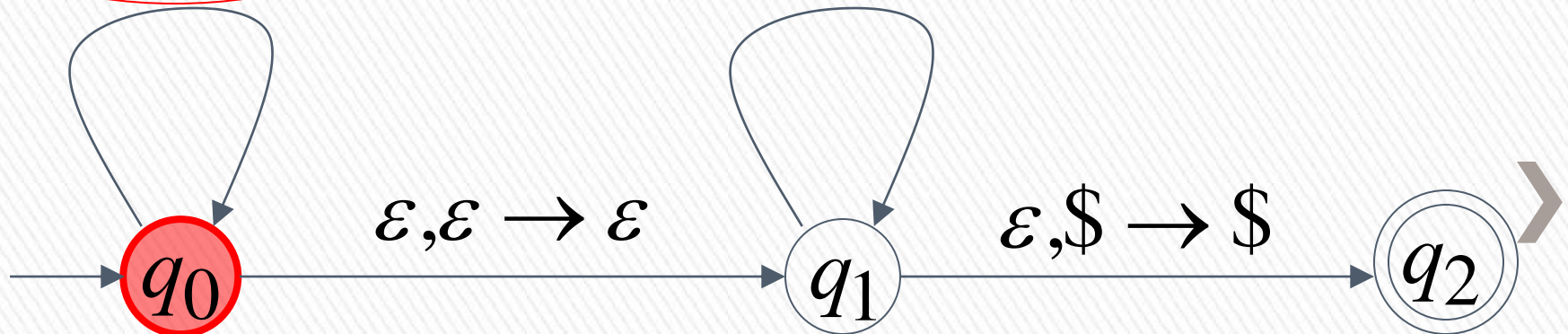
Stack

$a, \varepsilon \rightarrow a$

$b, \varepsilon \rightarrow b$

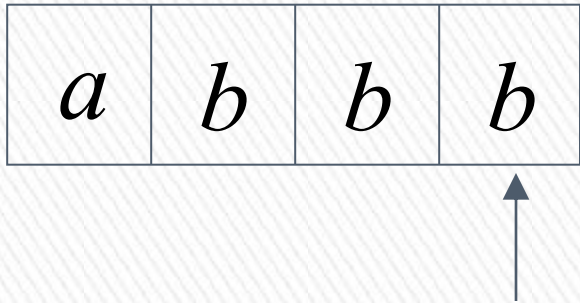
$a, a \rightarrow \varepsilon$

$b, b \rightarrow \varepsilon$

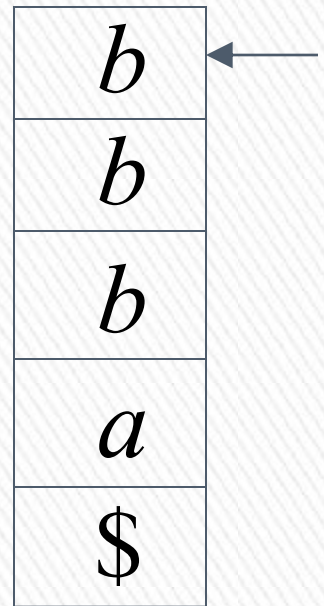


Time 5

Input



No accept state is reached



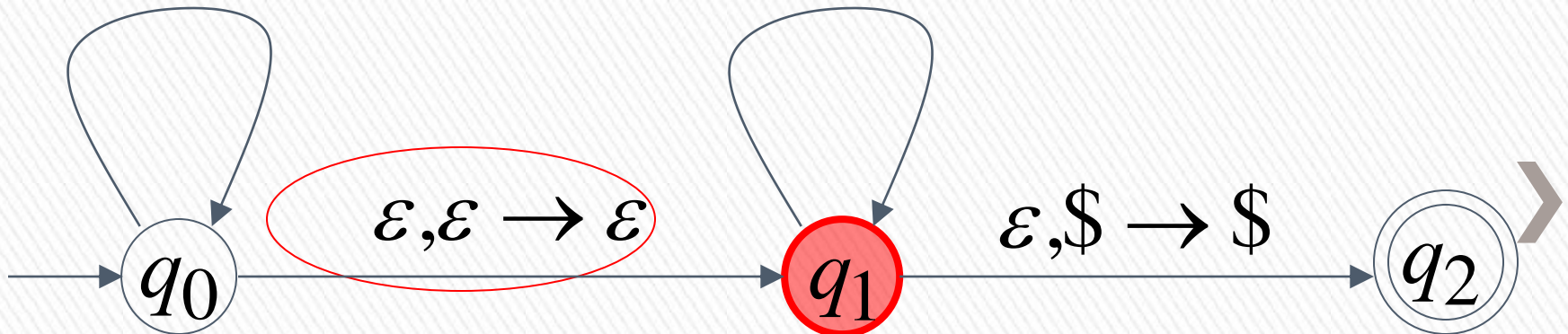
Stack

$a, \varepsilon \rightarrow a$

$b, \varepsilon \rightarrow b$

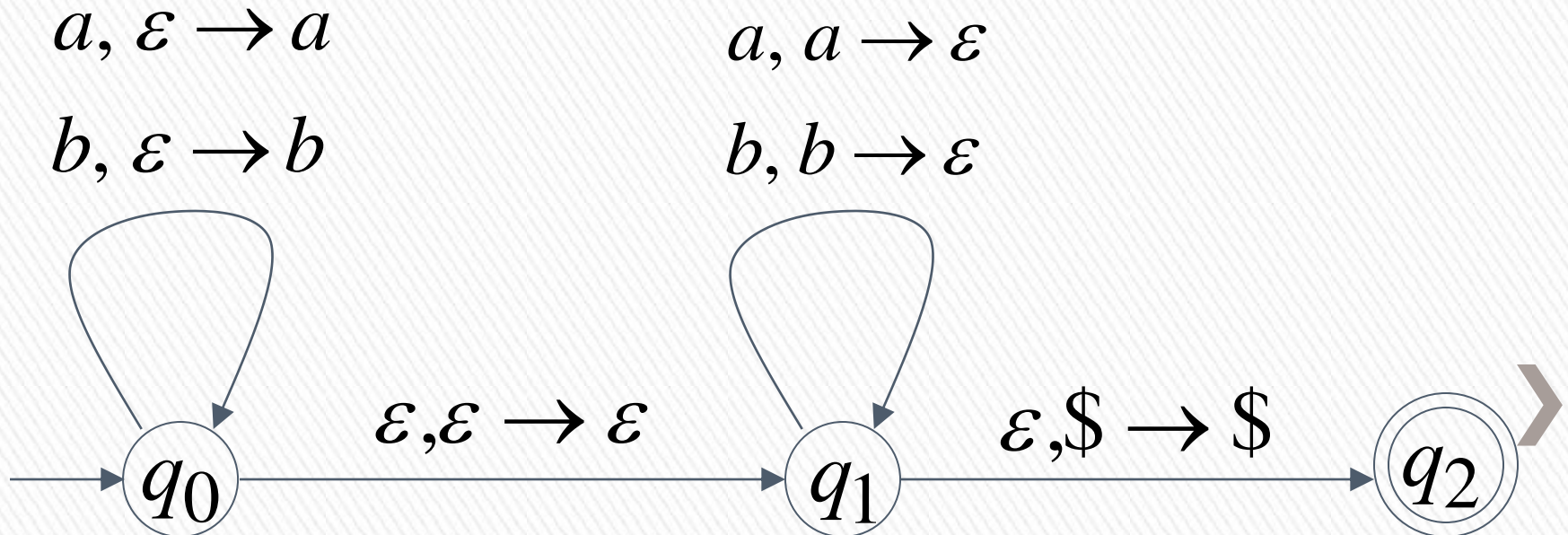
$a, a \rightarrow \varepsilon$

$b, b \rightarrow \varepsilon$

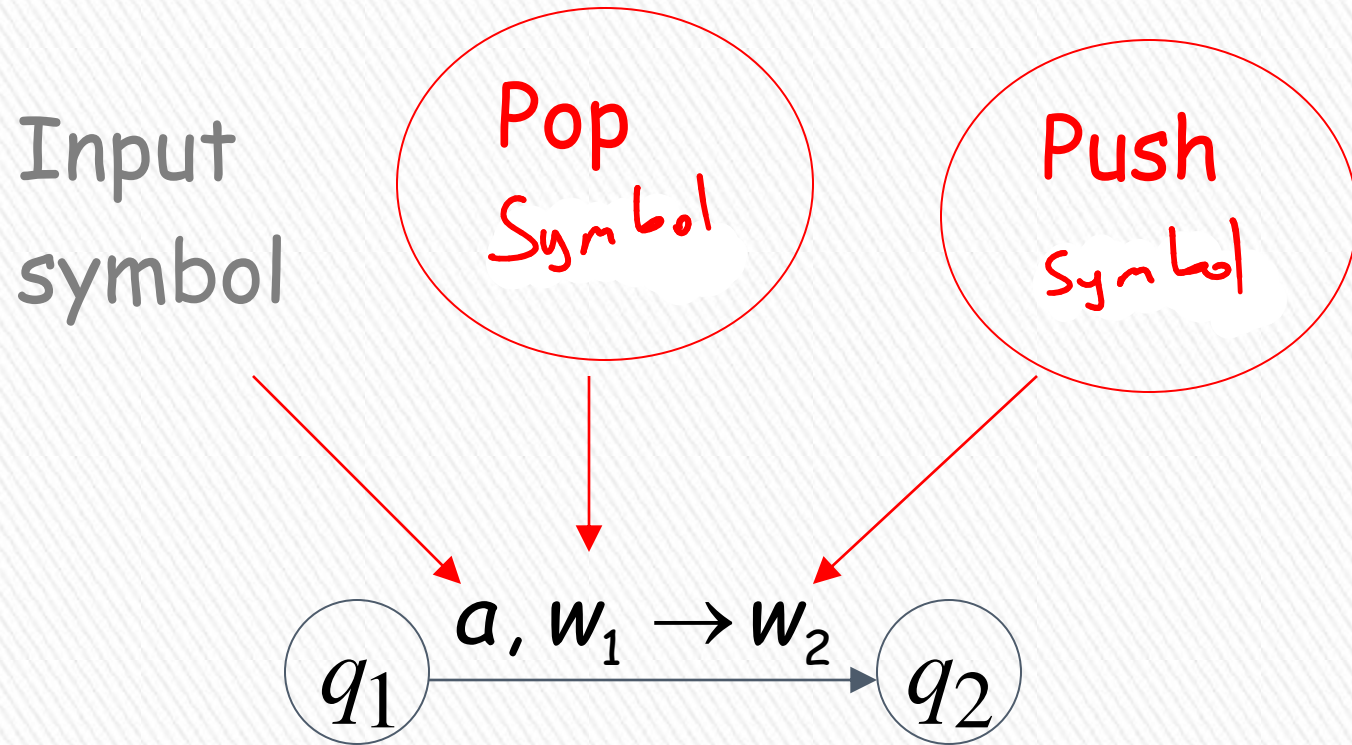


There is no computation
that accepts string *abbb*

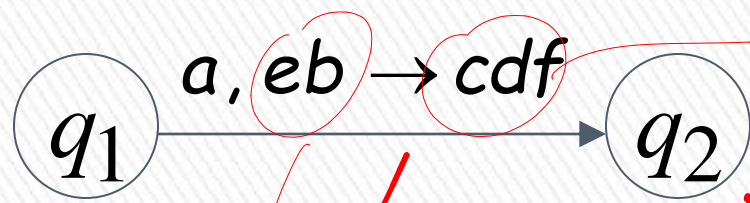
$$abbb \notin L(M)$$



Pushing & Popping Strings

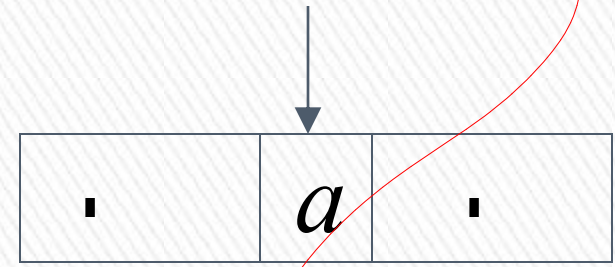
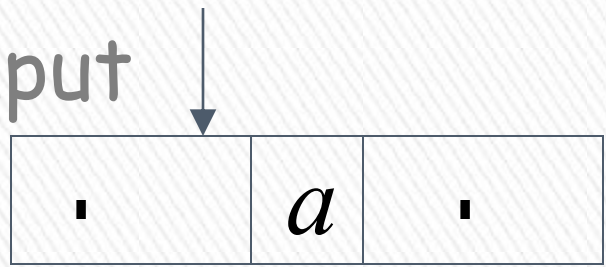


Example:

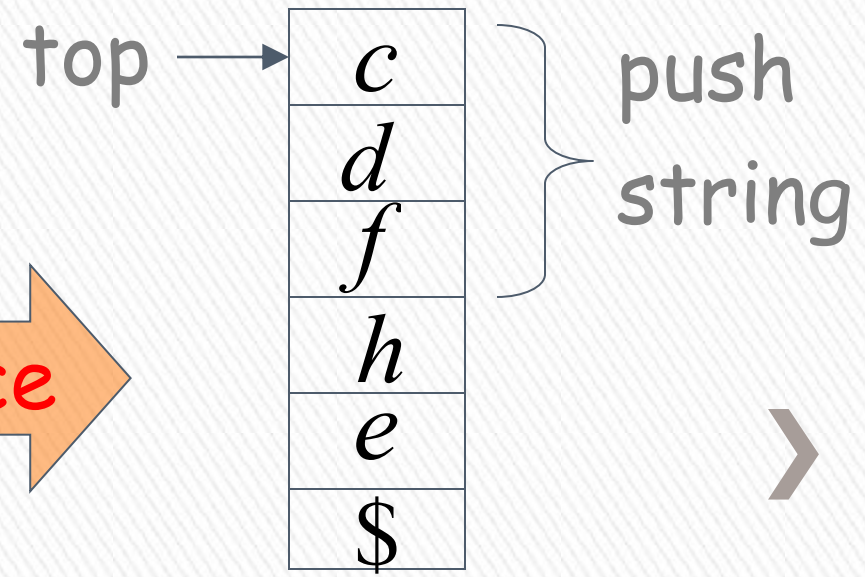
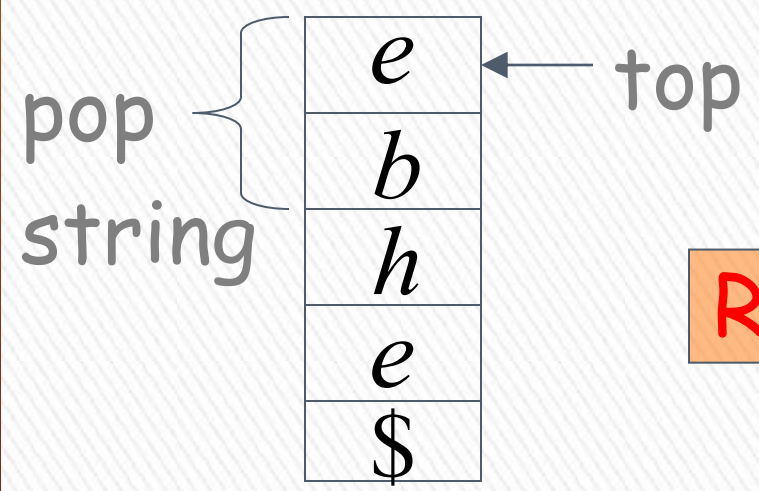


↳ *çizim kolaylığı*

input

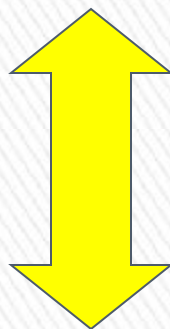
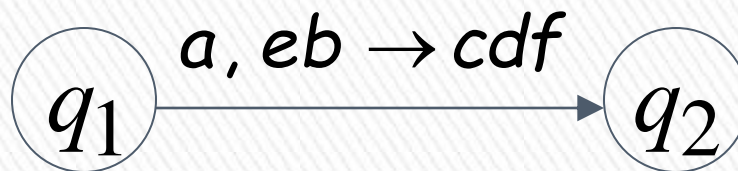


stack



Replace





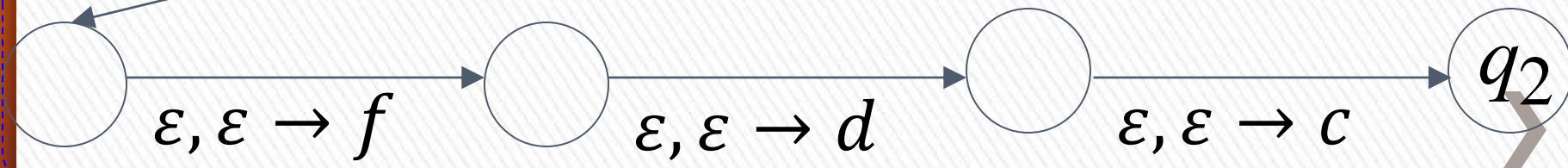
Equivalent
transitions

pop



$\varepsilon, \varepsilon \rightarrow \varepsilon$

push



Another PDA example

$$L(M) = \{w \in \{a,b\}^* : n_a(w) = n_b(w)\}$$

a says 's' - b says 's'

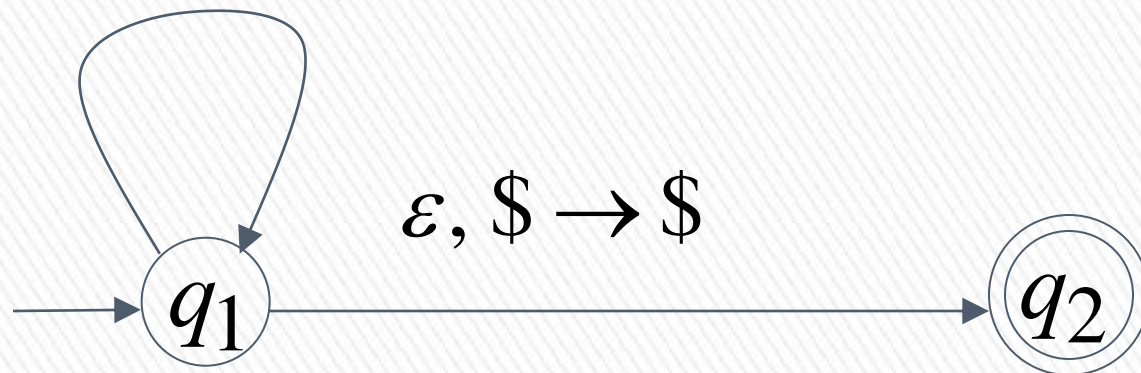
PDA M

q a u a

$a, \$ \rightarrow 0\$$ $b, \$ \rightarrow 1\$$

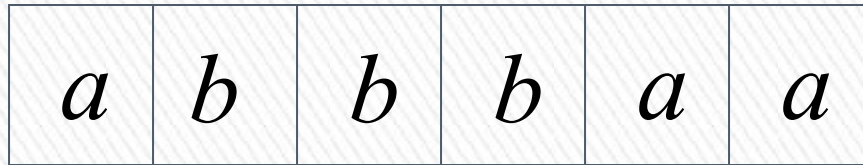
$a, 0 \rightarrow 00$ $b, 1 \rightarrow 11$

$a, 1 \rightarrow \varepsilon$ $b, 0 \rightarrow \varepsilon$

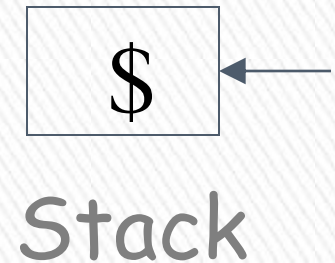


Execution Example: Time 0

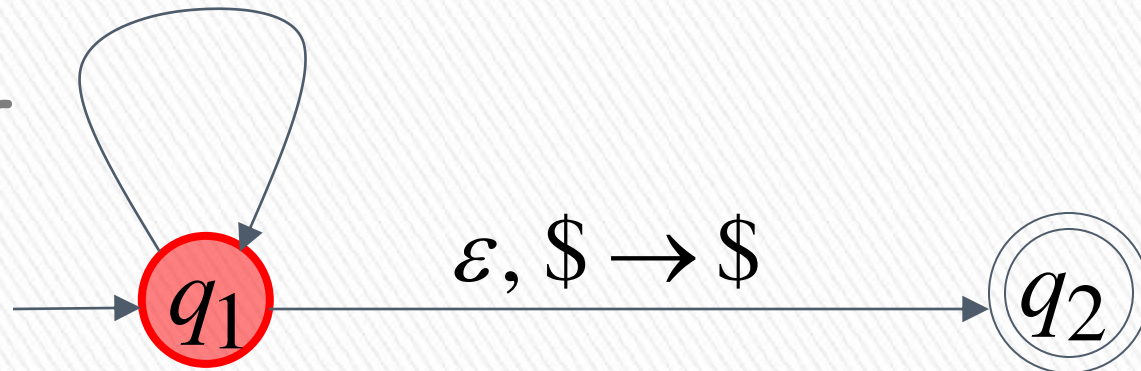
Input



$a, \$ \rightarrow a\$$ $b, \$ \rightarrow b\$$
 $a, a \rightarrow aa$ $b, b \rightarrow bb$
 $a, b \rightarrow \varepsilon$ $b, a \rightarrow \varepsilon$

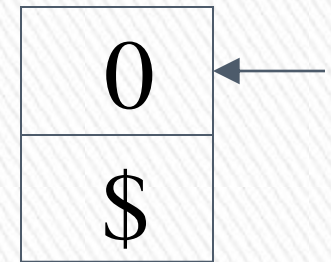
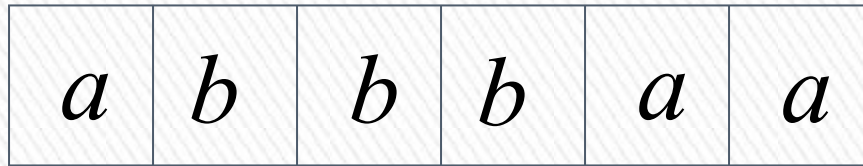


current
state



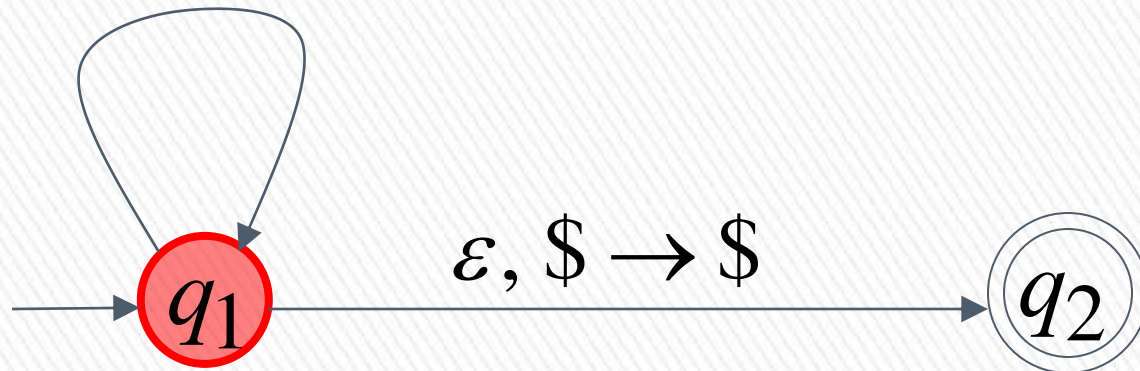
Time 1

Input



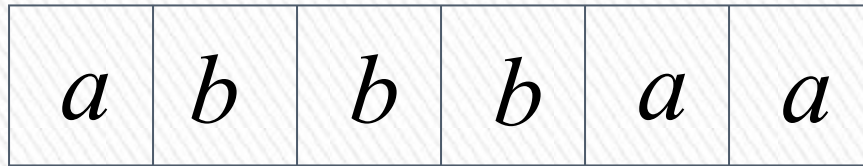
Stack

$a, \$ \rightarrow 0\$$ $b, \$ \rightarrow 1\$$
 $a, 0 \rightarrow 00$ $b, 1 \rightarrow 11$
 $a, 1 \rightarrow \varepsilon$ $b, 0 \rightarrow \varepsilon$



Time 3

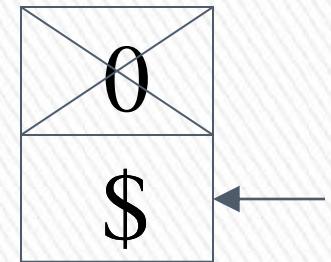
Input



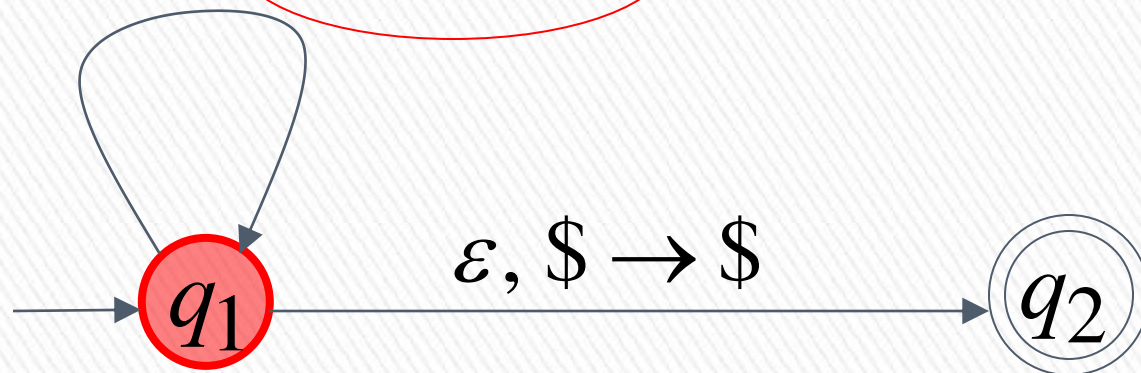
$a, \$ \rightarrow 0\$$ $b, \$ \rightarrow 1\$$

$a, 0 \rightarrow 00$ $b, 1 \rightarrow 11$

$a, 1 \rightarrow \varepsilon$ $b, 0 \rightarrow \varepsilon$

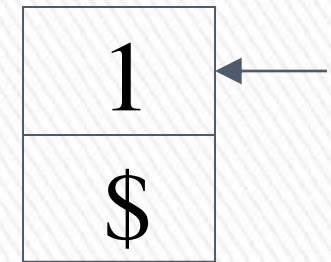
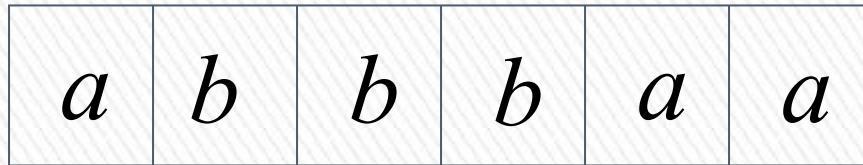


Stack



Time 4

Input

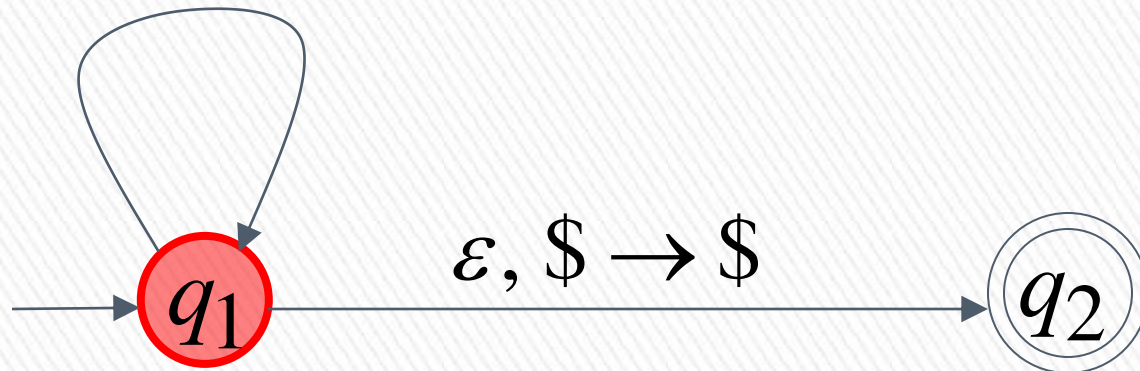


Stack

$a, \$ \rightarrow 0\$$ $b, \$ \rightarrow 1\$$

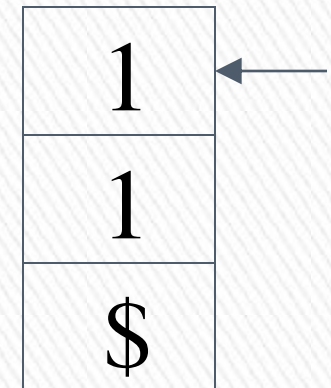
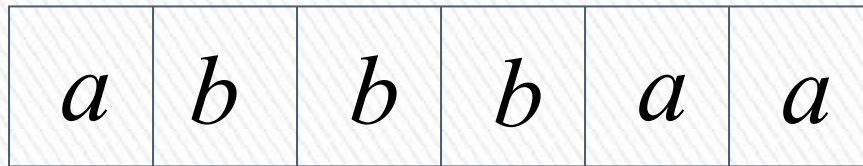
$a, 0 \rightarrow 00$ $b, 1 \rightarrow 11$

$a, 1 \rightarrow \varepsilon$ $b, 0 \rightarrow \varepsilon$



Time 5

Input

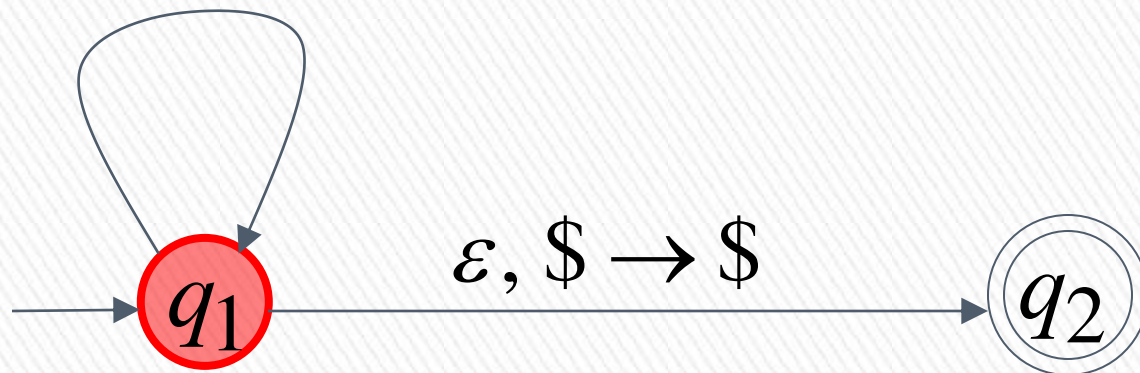


Stack

$a, \$ \rightarrow 0\$$ $b, \$ \rightarrow 1\$$

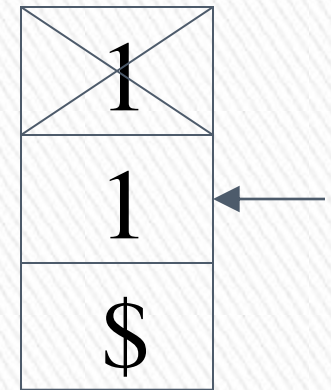
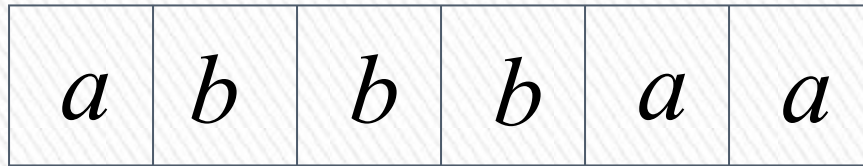
$a, 0 \rightarrow 00$ $b, 1 \rightarrow 11$

$a, 1 \rightarrow \varepsilon$ $b, 0 \rightarrow \varepsilon$



Time 6

Input

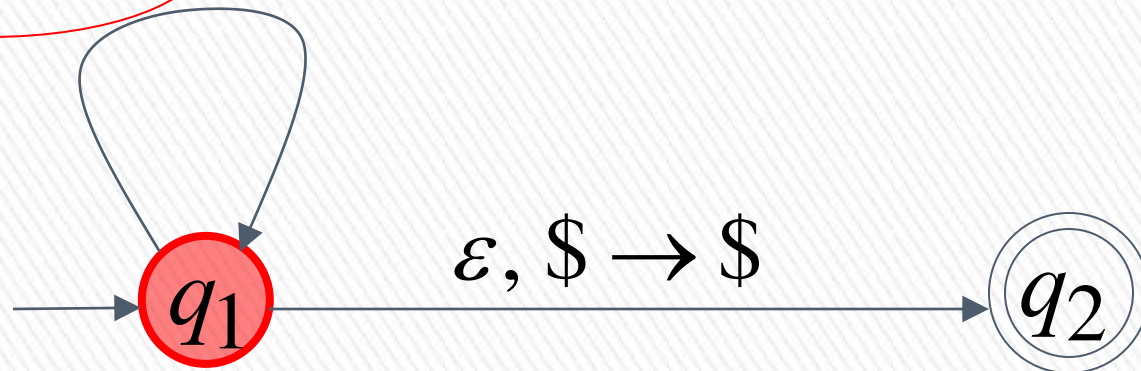


Stack

$a, \$ \rightarrow 0\$$ $b, \$ \rightarrow 1\$$

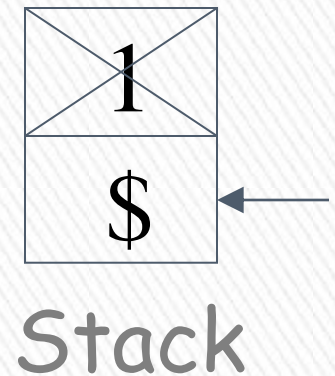
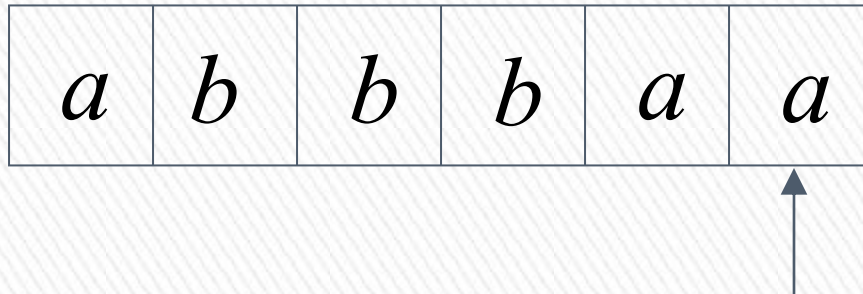
$a, 0 \rightarrow 00$ $b, 1 \rightarrow 11$

$a, 1 \rightarrow \varepsilon$ $b, 0 \rightarrow \varepsilon$



Time 7

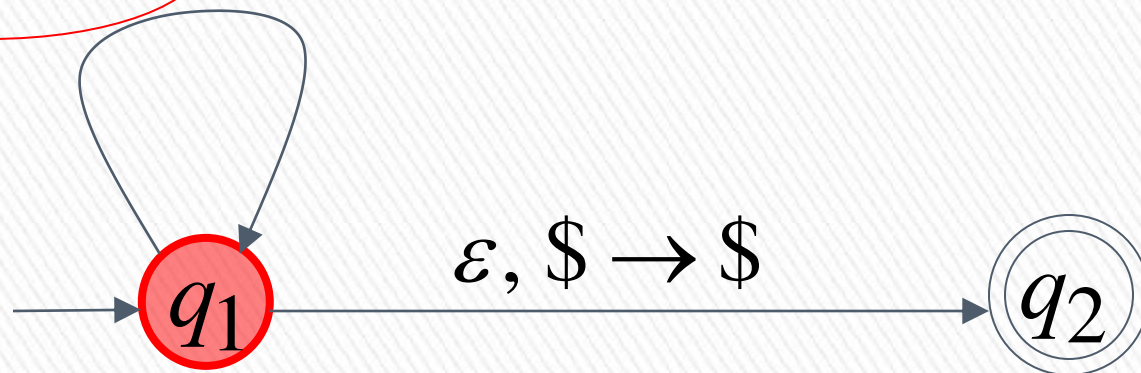
Input



$a, \$ \rightarrow 0\$$ $b, \$ \rightarrow 1\$$

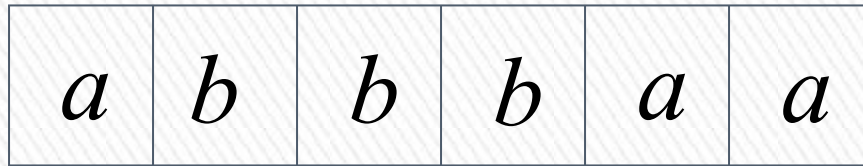
$a, 0 \rightarrow 00$ $b, 1 \rightarrow 11$

$a, 1 \rightarrow \varepsilon$ $b, 0 \rightarrow \varepsilon$



Time 8

Input



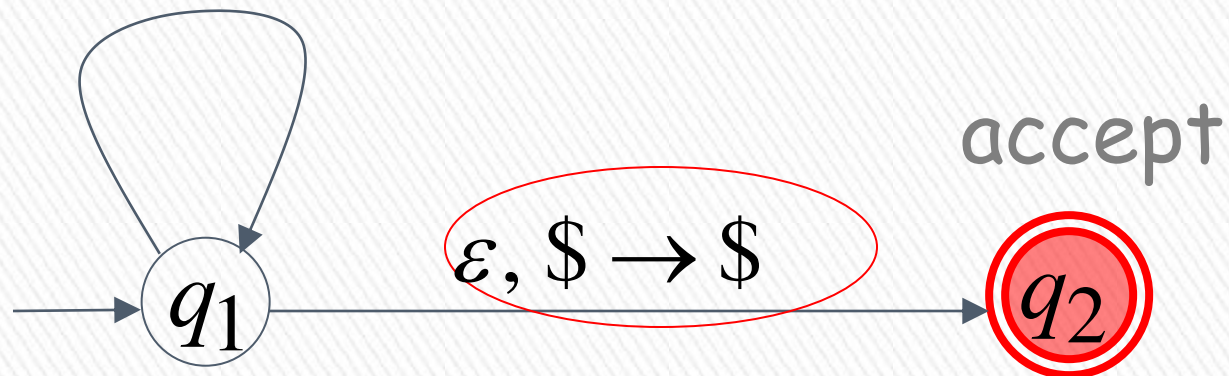
$a, \$ \rightarrow 0\$$ $b, \$ \rightarrow 1\$$

$a, 0 \rightarrow 00$ $b, 1 \rightarrow 11$

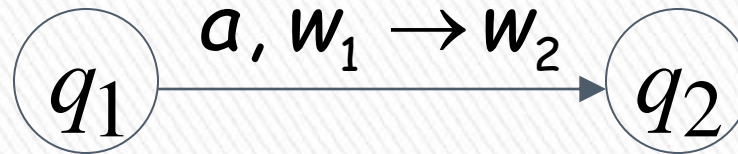
$a, 1 \rightarrow \varepsilon$ $b, 0 \rightarrow \varepsilon$



Stack



Formalities for PDAs



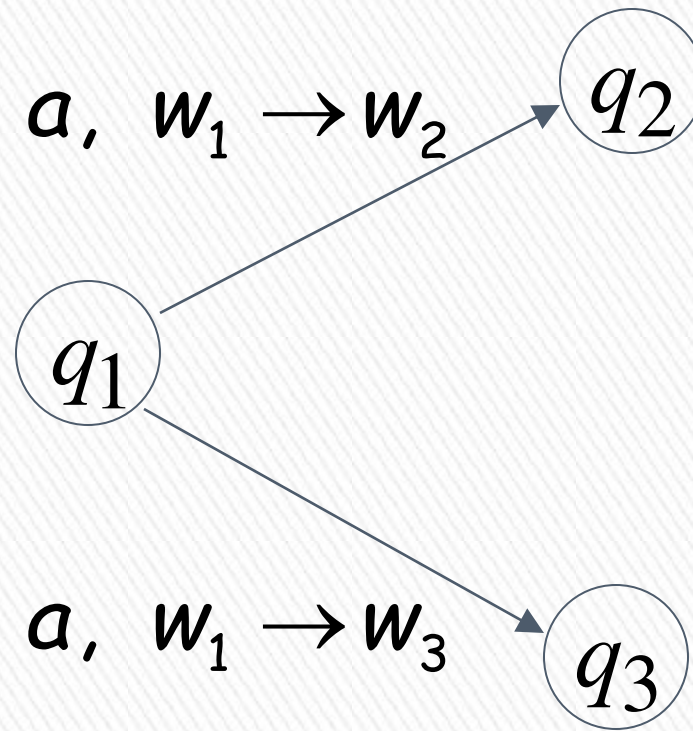
Transition function:

$$\delta(q_1, a, w_1) = \{(q_2, w_2)\}$$

Handwritten annotations in red:

- Arrows pointing to q_1 and a are labeled "state & input".
- An arrow pointing to w_1 is labeled "push-pop".
- An arrow pointing to q_2 is labeled "state".
- An arrow pointing to w_2 is labeled "push-pop".





Transition function:

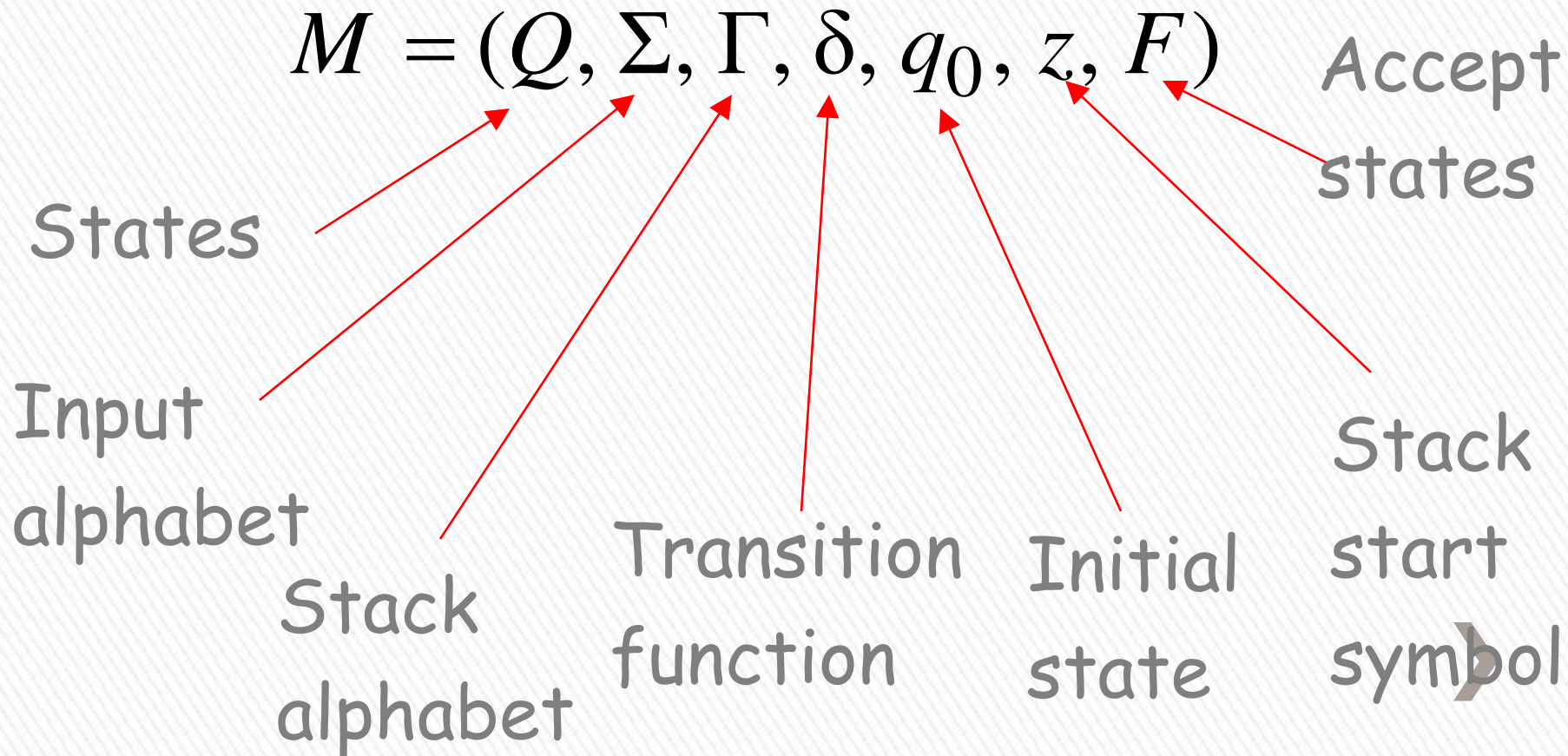
$$\delta(q_1, a, w_1) = \{(q_2, w_2), (q_3, w_3)\}$$



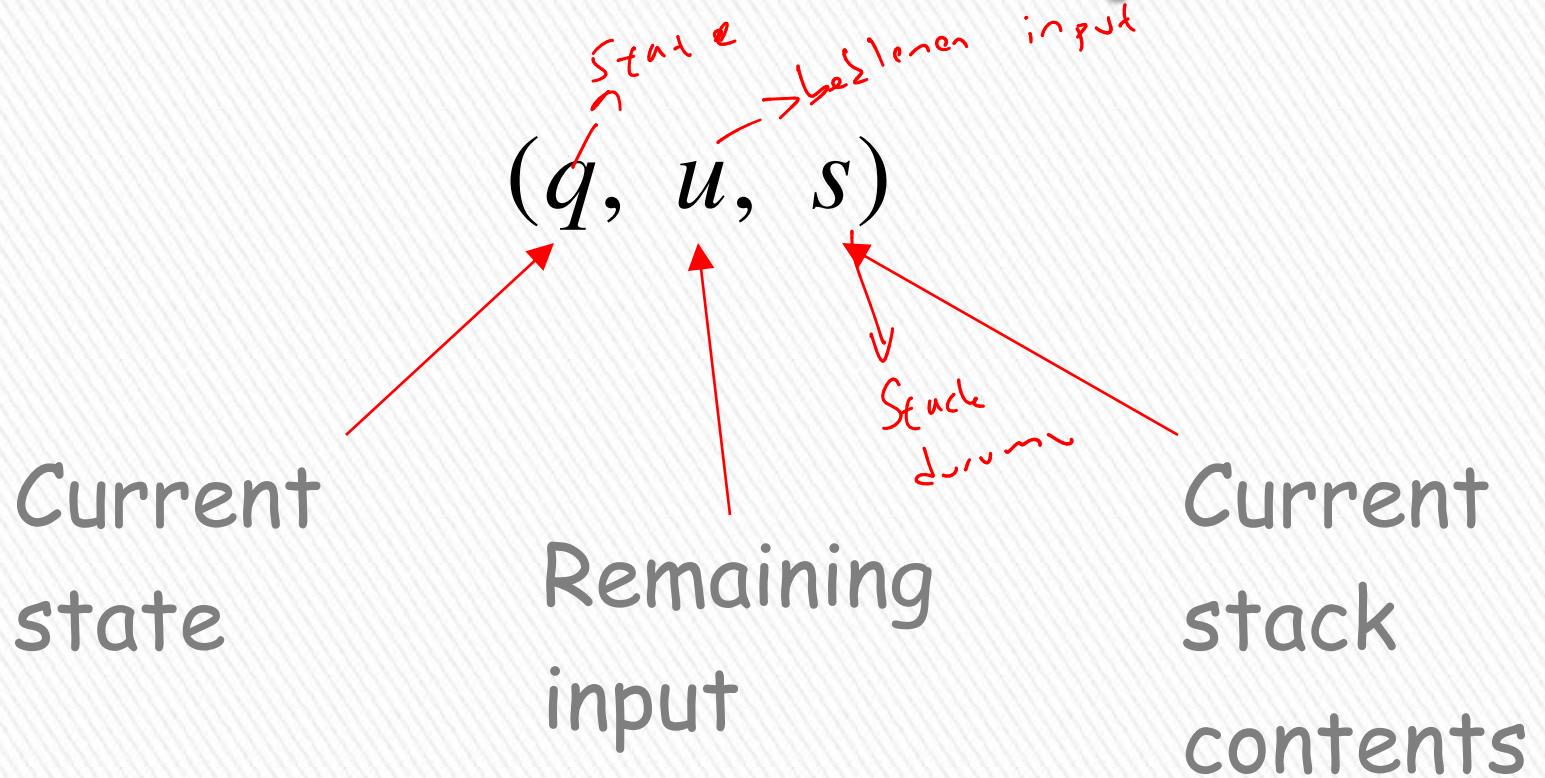
Formal Definition

Pushdown Automaton (PDA)

$$M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$$



Instantaneous Description



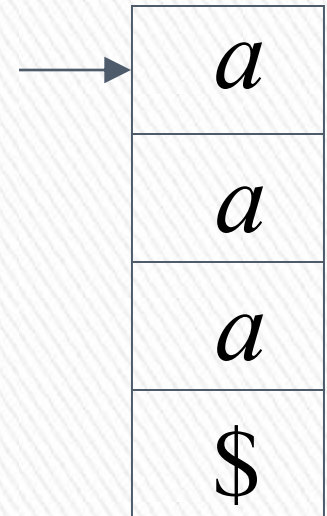
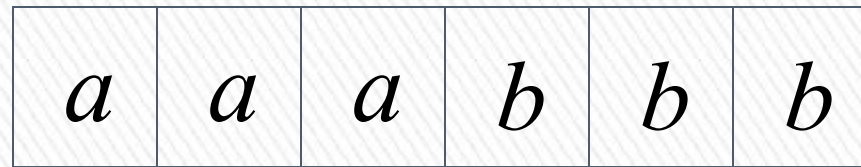
Example:

Instantaneous Description

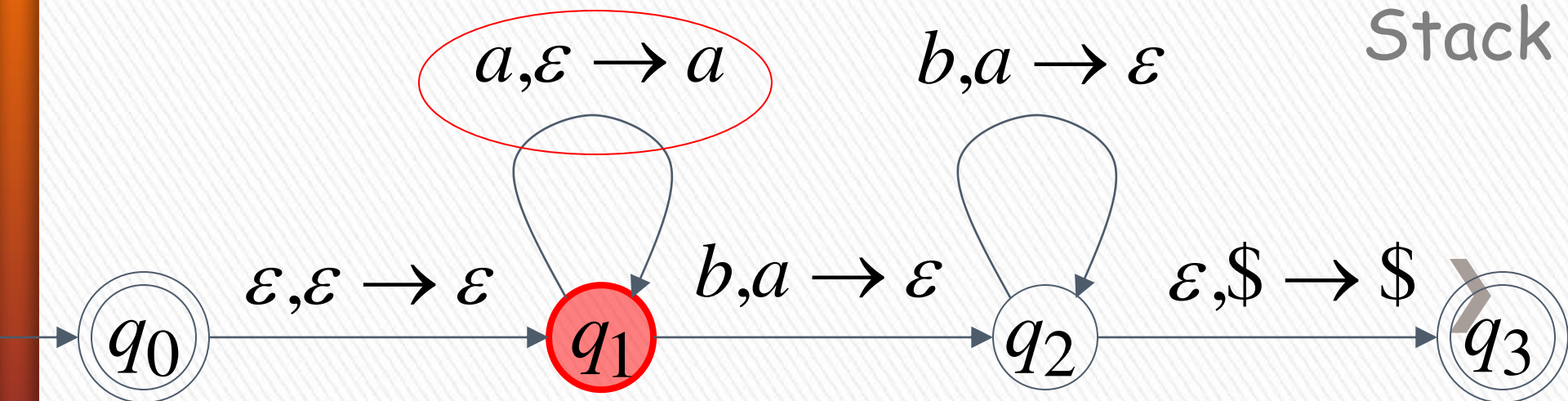
$(q_1, bbb, aaa\$)$

Time 4:

Input



Stack



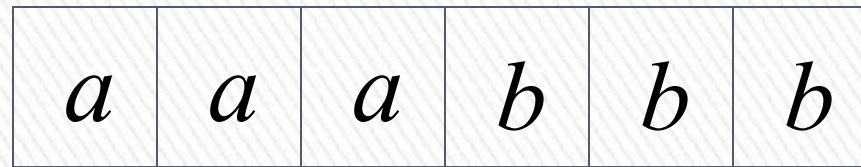
Example:

Instantaneous Description

$(q_2, bb, aa\$)$

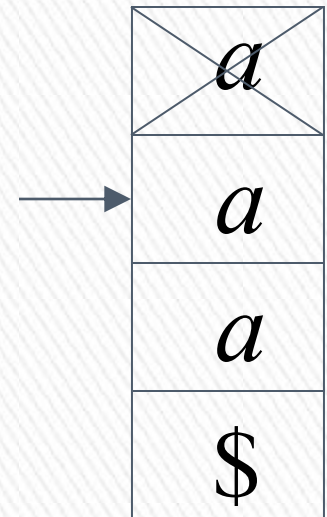
Time 5:

Input

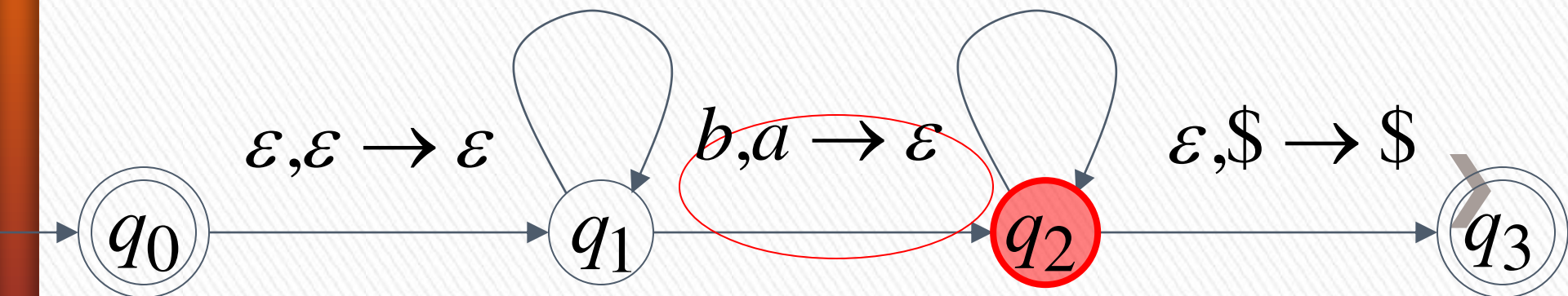


$a, \varepsilon \rightarrow a$

$b, a \rightarrow \varepsilon$



Stack



We write:

$$(q_1, bbb, aaa\$) \succ (q_2, bb, aa\$)$$

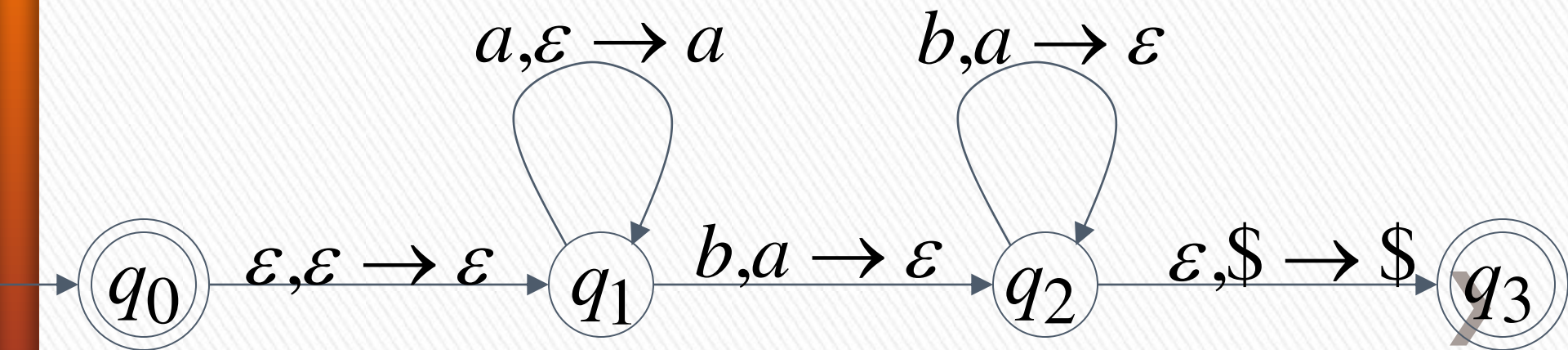
Time 4

Time 5



A computation:

$(q_0, aaabbbb, \$) \succ (q_1, aaabbbb, \$) \succ$
 $(q_1, aabbbb, a\$) \succ (q_1, abbbb, aa\$) \succ (q_1, bbbb, aaa\$) \succ$
 $(q_2, bb, aa\$) \succ (q_2, b, a\$) \succ (q_2, \varepsilon, \$) \succ (q_3, \varepsilon, \$)$



$$\begin{aligned}
 (q_0, aaabbbb, \$) &\succ (q_1, aaabbbb, \$) \succ \\
 (q_1, aabbbb, a\$) &\succ (q_1, abbbb, aa\$) \succ (q_1, bbbb, aaa\$) \succ \\
 (q_2, bb, aa\$) &\succ (q_2, b, a\$) \succ (q_2, \varepsilon, \$) \succ (q_3, \varepsilon, \$)
 \end{aligned}$$

For convenience we write:

$$(q_0, aaabbbb, \$) \overset{*}{\succ} (q_3, \varepsilon, \$)$$



Language of PDA $\rightarrow$ CFL *derinli* *ci:2:1:1:1*

Language $L(M)$ accepted by PDA M :

$$L(M) = \{w : (q_0, w, z) \xrightarrow{*} (q_f, \varepsilon, s)\}$$

Initial state

A blue arrow points from the text 'Initial state' to the symbol q_0 in the configuration notation (q_0, w, z) of the equation above.

Accept state

A blue arrow points from the text 'Accept state' to the symbol q_f in the configuration notation (q_f, ε, s) of the equation above.

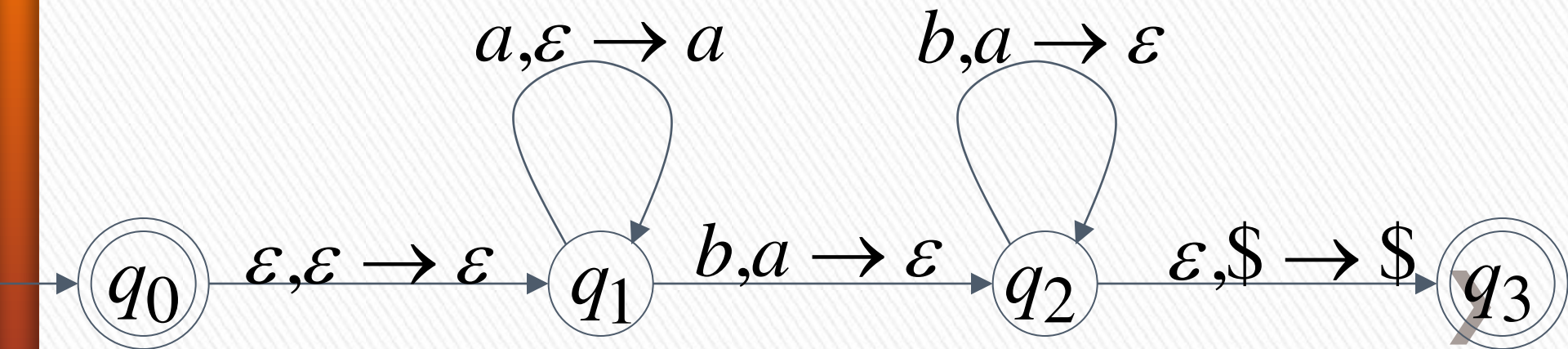
Example:

$$(q_0, aaabbbb, \$) \stackrel{*}{\succ} (q_3, \varepsilon, \$)$$



$$aaabbbb \in L(M)$$

PDA M :

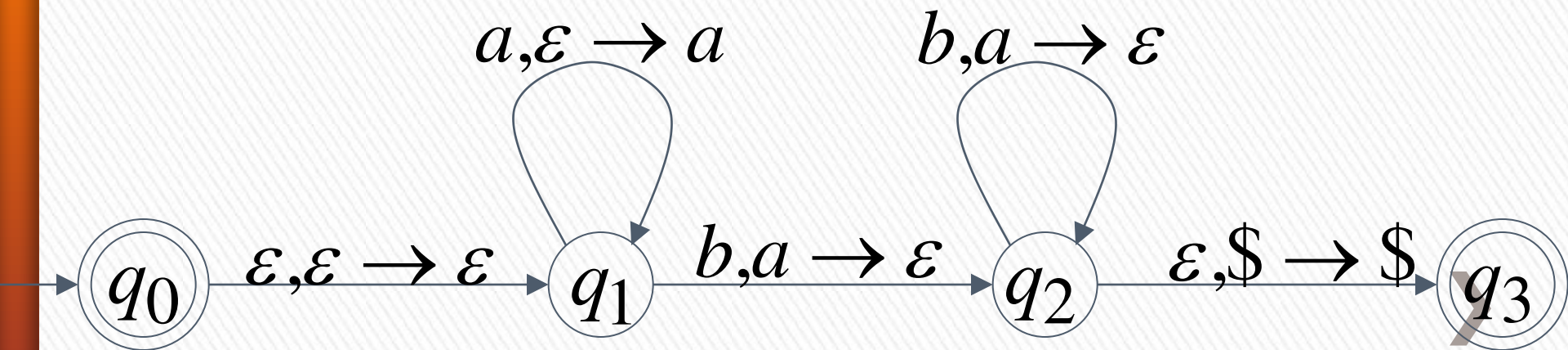


$$(q_0, a^n b^n, \$) \stackrel{*}{\succ} (q_3, \varepsilon, \$)$$



$$a^n b^n \in L(M)$$

PDA M :



Therefore:

$$L(M) = \{a^n b^n : n \geq 0\}$$

PDA M :

